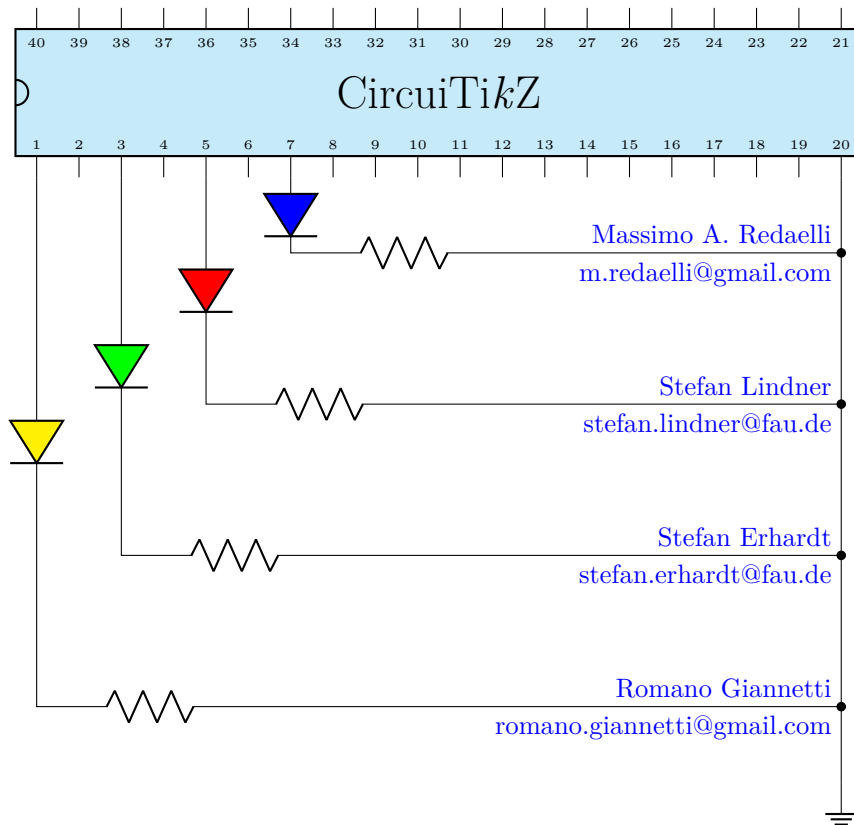




CircuiTikZ
version 1.8.7-unreleased-54e8a88 (2026/07/05)

Massimo A. Redaelli (m.redaelli@gmail.com)
 Stefan Lindner (stefan.lindner@fau.de)
 Stefan Erhardt (stefan.erhardt@fau.de)
 Romano Giannetti (romano.giannetti@gmail.com)

July 5, 2026

Contents

1	Introduction	10
1.1	About	10
1.2	License	10
1.3	Loading the package	10
1.4	Installing a new version of the package.	11
1.5	Requirements	11
1.6	Incompatible packages	11
1.7	Related and extension packages	12
1.7.1	Related packages	12
1.7.2	Extension packages	12
1.8	Known bugs and limitation	12
1.9	Scale factor inaccuracies	13
1.10	Incompatibilities between versions	14
1.11	Feedback	17
1.12	Package options	17
2	Tutorials	21
2.1	Getting started with CircuiTikZ: a current shunt	21
2.2	A non-inverting op-amp amplifier	24
2.2.1	Reusing the circuit: the easy way	26
2.3	A transistor-based amplifier	27
2.4	A logic circuit	32
3	The components: usage	35
3.1	Path-style components	35
3.1.1	Anchors	35
3.1.2	Border anchors	36
3.1.3	Relative coordinates	36
3.1.4	Customization	37
3.1.4.1	Components size	37
3.1.4.2	Mirroring and flipping path-style components	39
3.1.4.3	Thickness of the lines	39
3.1.4.4	Shape of the components	39
3.1.5	Descriptions	40
3.2	Node-style components	41
3.2.1	Mirroring and flipping	41
3.2.2	Anchors	42
3.2.2.1	Text anchors	42
3.2.3	Descriptions	42

3.3	Styling circuits and components	43
3.3.1	Relative size	44
3.3.2	Fill color	45
3.3.3	Line thickness	46
3.3.4	Style files	46
3.3.5	Style files: how to write them	47
3.4	Subcircuits	48
3.4.1	Subcircuit definition	49
3.4.2	Using the subcircuit	50
3.4.2.1	Scaling, flipping and rotating subcircuits	50
3.4.3	Parameters in subcircuits	50
3.4.4	Using <code>pics</code> (with parameters)	51
3.4.5	Using macros	52
4	The components: list	55
4.1	Grounds and supply voltages	55
4.1.1	Grounds	55
4.1.1.1	Grounds anchors	55
4.1.1.2	Grounds customization	56
4.1.2	Power supplies	56
4.1.2.1	Power supply anchors	56
4.1.2.2	Power supplies customization	56
4.2	Resistive bipoles	57
4.2.1	Potentiometers: wiper position	59
4.2.2	Generic sensors anchors	60
4.2.3	Resistive components customization	60
4.2.3.1	Geometry.	60
4.2.3.2	Thickness.	61
4.2.3.3	Arrows	61
4.2.3.4	Details of American (“zig-zag”) resistors.	62
4.3	Capacitors and inductors: dynamical bipoles	63
4.3.1	Capacitors	63
4.3.2	Capacitive sensors anchors	64
4.3.3	Capacitors customizations	65
4.3.4	Inductors	65
4.3.5	Inductors customizations	66
4.3.5.1	Chokes	67
4.3.6	Inductors anchors	67
4.3.6.1	Taps.	67
4.3.6.2	Core anchors.	68
4.3.6.3	Dot anchors.	68

4.4	Diodes and such	69
4.4.1	Tripole-like diodes	71
4.4.2	Thyristors anchors and customization	75
4.4.3	Diode customizations	75
4.4.3.1	Optical devices arrows	75
4.4.3.2	Zener and TVS diode options	77
4.5	Sources and generators	77
4.5.1	Batteries	77
4.5.2	Stationary sources	78
4.5.3	Sinusoidal sources	79
4.5.4	Controlled sources	79
4.5.5	Noise sources	80
4.5.6	Special sources	81
4.5.7	Nullator and norator	83
4.5.8	DC sources	84
4.5.9	Sources customizations	84
4.5.9.1	Size.	84
4.5.9.2	Waveform symbols.	85
4.5.9.3	Polarity symbols.	85
4.5.9.4	Orientation of the polarity symbols.	85
4.5.9.5	Three-phase symbols.	86
4.5.9.6	Rotating the three-phase symbols.	87
4.5.10	Source borders	87
4.5.10.1	Additional winding.	87
4.5.10.2	Additional winding anchors.	88
4.6	Instruments	88
4.6.1	Basic round instruments	88
4.6.2	Square instruments	89
4.6.3	Oscilloscopes and current probes	90
4.6.4	Instruments customizations	90
4.6.4.1	Current probes.	90
4.6.4.2	Square instruments index position.	91
4.6.4.3	Oscilloscope waveform.	91
4.6.5	Rotation-invariant elements	92
4.6.6	Instruments as node elements	93
4.6.7	Measuring voltage and currents, multiple ways	93
4.7	Mechanical Analogy	96
4.7.1	Mechanical elements customizations	96
4.8	Miscellaneous bipoles and symbols	96
4.8.1	Miscellaneous bipoles	97
4.8.2	Miscellaneous element customization	99

4.8.2.1	Wiggly fuses.	99
4.8.2.2	Neon lamps.	100
4.8.2.3	Spark gap.	100
4.8.3	Miscellaneous symbols	100
4.8.4	Customization of ac/dc symbols.	101
4.9	Multiple wires (buses)	101
4.10	Crossings	102
4.10.1	Crossing customization	103
4.11	Arrows (fake and real)	104
4.11.1	Arrows size	104
4.11.2	Generic Tunable Arrows	105
4.11.3	Arrow tips	105
4.12	Terminal shapes	106
4.13	Connectors	107
4.13.1	BNC connector/terminal	107
4.13.2	IEC 60617 socket-plug connectors	107
4.13.3	Jack connectors	109
4.14	Block diagram components	110
4.14.1	Blocks anchors	116
4.14.2	Blocks customization	119
4.14.2.1	Multi ports	120
4.14.2.2	Labels and custom two-port boxes	121
4.14.2.3	Box option	122
4.14.2.4	Dash optional parts	122
4.14.2.5	Dashing the DC symbol in blocks.	123
4.14.2.6	Blocks inner drawing.	123
4.14.2.7	Plot-type filter blocks customization.	124
4.15	Transistors	125
4.15.1	Standard bipolar transistors	125
4.15.2	Multi-terminal bipolar transistors	126
4.15.3	Field-effect transistors	126
4.15.4	Transistor texts (labels)	131
4.15.5	Transistors customization	133
4.15.5.1	Size.	133
4.15.5.2	Thickness.	133
4.15.5.3	Rounded caps for gate and base.	133
4.15.5.4	Arrows.	133
4.15.5.5	Circles.	135
4.15.5.6	Body diodes and similar things.	136
4.15.5.7	HEMT customization.	137
4.15.5.8	Schottky transistors.	137

4.15.5.9	Ferroelectric transistors	138
4.15.5.10	IGBT outer base.	138
4.15.5.11	UJT transistors.	139
4.15.5.12	Base/Gate terminal.	139
4.15.5.13	Bulk terminals.	139
4.15.5.14	Solder dots.	141
4.15.5.15	Simplified symbols for depletion-mode MOSFETs	141
4.15.5.16	JFET gate position.	142
4.15.5.17	Gate/Base gap coloring.	142
4.15.6	Multiple terminal transistors customization	142
4.15.7	Transistor circle customization	143
4.15.7.1	Position and size.	143
4.15.7.2	Line and color.	143
4.15.7.3	Partially drawn circle borders	144
4.15.8	Transistor bodydiode customization	145
4.15.9	Transistors anchors	146
4.15.10	Transistor paths	150
4.16	Electronic Tubes	151
4.16.1	Tubes customization	153
4.16.2	Tubes anchors	154
4.16.3	Partially drawn tube borders	154
4.16.4	Multi-anode tube	155
4.16.5	Other tubes-like components	156
4.16.5.1	Dynode customization.	157
4.17	RF components	158
4.17.1	RF elements customization	160
4.17.2	Microstrip customization	160
4.18	Electro-Mechanical Devices	161
4.18.1	Electro-Mechanical Devices anchors	161
4.19	Double bipoles (transformers)	162
4.19.1	Transformer anchors	163
4.19.2	Transformers customization	165
4.19.3	Styling transformer's coils independently	166
4.19.4	Generic double bipoles	167
4.20	Amplifiers	169
4.20.1	Amplifiers anchors	170
4.20.2	Amplifiers customization	172
4.20.2.1	Input polarity.	172
4.20.2.2	Input and output pins symbols.	173
4.20.2.3	Input and output pins length.	173
4.20.2.4	Main amplifier label.	174

4.20.2.5	European-style amplifier customization.	174
4.20.3	Designing your own amplifier	176
4.21	Switches, buttons and jumpers	176
4.21.1	Traditional switches	176
4.21.2	Cute switches	178
4.21.2.1	Cute switches anchors	179
4.21.2.2	Cute switches customization	179
4.21.3	Proximity switches	180
4.21.4	Rotary switches	181
4.21.4.1	Rotary switch anchors	182
4.21.4.2	Rotary switch customization	183
4.21.5	Switch arrows	183
4.21.5.1	Rotary switch arrows.	184
4.21.6	Jumpers	185
4.21.6.1	Simple jumpers.	185
4.21.6.2	Two-ways (three-pins) jumpers.	186
4.21.7	Solder jumpers.	186
4.22	Logic gates	187
4.22.1	American Logic gates	187
4.22.2	IEEE logic gates	188
4.22.3	European Logic gates	190
4.22.4	Path-style logic ports	191
4.22.5	American ports usage	192
4.22.5.1	American logic port customization	192
4.22.5.2	American logic port anchors	193
4.22.6	IEEE logic gates usage.	195
4.22.6.1	Stacking and aligning IEEE standard gates.	197
4.22.6.2	IEEE standard ports customization	198
4.22.6.3	IEEE standard ports anchors	199
4.22.6.4	Transmission gate symbols.	199
4.22.7	European logic port usage	200
4.22.7.1	European logic port customization	200
4.22.7.2	European logic port anchors	201
4.23	Flip-flops	201
4.23.1	Custom flip-flops	203
4.23.2	Flip-flops anchors	204
4.23.3	Flip-flops customization	204
4.24	Multiplexer and de-multiplexer	206
4.24.1	Mux-Demux: design your own shape	207
4.24.2	Mux-Demux customization	208
4.24.3	Mux-Demux anchors	209

4.24.4	Adding labels to the pins	210
4.24.4.1	Inner pin labels	210
4.24.4.2	Outer pin labels	211
4.24.4.3	Border labels	211
4.24.4.4	Clock and negation symbols	212
4.24.5	Manually adding wedges or circular inversion markers	213
4.24.6	Adding a background drawing to the muxdemux	213
4.24.7	Mux-Demux special usage	214
4.25	Chips (integrated circuits)	215
4.25.1	DIP and QFP chips customization	216
4.25.2	Chip anchors	217
4.25.3	Chip rotation	218
4.25.4	Chip special usage	218
4.26	Seven segment displays	219
4.26.1	Seven segments anchors	219
4.26.2	Seven segments customization	220
5	Labels, voltages and currents	221
5.1	Labels and Annotations	221
5.1.1	Label and annotation position	222
5.1.1.1	Adjust label and annotation position.	222
5.1.2	Special symbols in labels and annotations.	222
5.1.3	Labels and annotation orientation.	223
5.1.4	Stacked (two lines) labels.	224
5.2	Currents and voltages	225
5.2.1	Common properties of voltages and currents	228
5.2.2	Special treatment for generators	229
5.3	Currents	231
5.4	Flows	233
5.5	Voltages	234
5.5.1	European style	234
5.5.2	Straight European style	236
5.5.3	American style	236
5.5.4	Raised American style	237
5.5.5	Voltage position	238
5.5.6	American voltages customization	240
5.5.7	Combining different styles	240
5.6	Changing the style of labels, voltages, and other text ornaments	241
5.7	Accessing labels text nodes	242
5.8	Advanced voltages, currents and flows	243
5.8.1	Using standard label with custom symbols	243

5.8.2	Activating the anchors	244
5.8.3	Auxiliary information	246
5.8.4	Fixed voltage arrows: an example of advanced voltage usage	246
5.8.5	Automatic advanced voltages, current and flows (experimental)	248
5.8.6	Creating new voltage/current/flows commands	250
5.9	Integration with <code>siunitx</code>	251
6	Using bipoles in circuits	253
6.1	Nodes (also called poles)	253
6.1.1	Transparent poles	255
6.2	Mirroring and Inverting	256
6.3	Putting them together	256
6.4	Line joins between Path Components	257
7	Colors	258
7.1	Shape colors	259
7.2	Fill colors	261
7.2.1	Background colors different from white	262
8	FAQ: Frequently asked questions	264
8.1	Using named nodes in circuits	264
8.2	Using dashed (or colored) wires in circuits	265
8.3	Errors when externalizing pictures	266
8.4	Labels, voltages and currents woes	266
8.5	Global scaling and rotating	267
8.6	Tunable components	267
9	Defining new components	269
9.1	Suggested setup	269
9.2	Path-style component	270
9.3	Node-style component	273
9.3.1	The internal hook system	274
9.3.2	Finishing your work	274
10	Examples	275
10.1	A red diode	275
10.2	Using the (experimental) <code>siunitx</code> syntax	276
10.3	Photodiodes	277
10.4	A Sallen-Key cell	277
10.5	Mixing circuits and graphs	278
10.6	RF circuit	279
10.7	A styled low noise input stage	280
10.8	An example with the <code>compatibility</code> option	281
10.9	3-phases block schematic	282
10.10	Using components in <code>pics</code> (since <code>v1.8.0</code>)	283
10.11	Automatic user-defined voltages, currents and flows (since <code>v1.8.5</code>)	284

11 Changelog and Release Notes	285
Index	304

1 Introduction

*Lorenzo and Mirella, 57 years ago, started a trip that
eventually lead to a lot of things — among them,
CircuiTikZ v1.0.
In loving memory — R. G., 2020-02-04*

1.1 About

CircuiTikZ was initiated by Massimo Redaelli in 2007, who was working as a research assistant at the Polytechnic University of Milan, Italy, and needed a tool for creating exercises and exams. After he left University in 2010 the development of CircuiTikZ slowed down, since L^AT_EX is mainly established in the academic world. In 2015 Stefan Lindner and Stefan Erhardt, both working as research assistants at the University of Erlangen-Nürnberg, Germany, joined the team and now maintain the project together with the initial author. In 2018 Romano Giannetti, full professor of Electronics at Comillas Pontifical University of Madrid, joined the team.

The use of CircuiTikZ is, of course, not limited to academic teaching. The package gets widely used by engineers for typesetting electronic circuits for articles and publications all over the world.

1.2 License

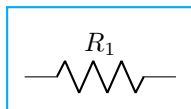
Copyright © 2007–2026 by Massimo Redaelli, 2013–2026 by Stefan Erhardt, 2015–2026 by Stefan Lindner, and 2018–2026 by Romano Giannetti. This package is author-maintained. Permission is granted to copy, distribute and/or modify this software under the terms of the L^AT_EX Project Public License, version 1.3.1, or the GNU Public License. This software is provided “as is”, without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose.

1.3 Loading the package

L ^A T _E X	ConT _E Xt ¹
<code>\usepackage{circuitikz}</code>	<code>\usemodule[circuitikz]</code>

TikZ will be automatically loaded; additionally, the TikZ libraries `calc`, `arrows.meta`, `bending`, and `fpu` are loaded (the last one is used only on demand).

CircuiTikZ commands are just TikZ commands, so a minimum usage example would be:



```
\tikz \draw (0,0) to[R=$R_1$] (2,0);
```

There is really no support for Plain TeX — the maintainers are willing to consider patches if somebody is interested.

¹ConT_EXt support was added mostly thanks to Mojca Miklavc and Aditya Mahajan. **Please notice** that since ConT_EXt switched to the new lmt_x engine (March 2023), there can be problems to compile TikZ with some version; in May 2023 the problem has been fixed. Please check [this issue](#) for details.

1.4 Installing a new version of the package.

The stable version of the package should come with your L^AT_EX distribution. Downloading the files from CTAN and installing them locally is, unfortunately, a distribution-dependent task and sometimes not so trivial. If you search for `local texmf tree` and the name of your distribution on <https://tex.stackexchange.com/> you will find a lot of hints.

Anyway, the easiest way of using whichever version of CircuiTikZ is to point to the GitHub page <https://circuitikz.github.io/circuitikz/> of the project, and download the version you want. You will download a simple (biggish) file, called `circuitikzgit.sty`.

Now you can just put this file in your local `texmf` tree, if you have one, or simply adding it into the same directory where your main file resides, and then use

```
\usepackage[...options...]{circuitikzgit}
```

instead of `circuitikz`. This is also advantageous for “future resilience”; the authors try hard not to break backward compatibility with new versions, but sometimes, things happen.

1.5 Requirements

- `latex` (if using it), version from 2019-10-01 (better from 2020-10-01);
- `tikz`, version $\geq 3.1.5b$ (it *should* work with any version from 3.1.4b and up, but better use a newer one);
- `xstring`, not older than 2009/03/13;
- `siunitx`, if using `siunitx` option (better v2 or newer).

A similar approach for ConT_EXt is available, with the file `t-circuitikzgit.tex`; the compatibility in this case is not guaranteed, but provided on a *best effort* base.

This manual has been typeset with CircuiTikZ 1.8.7-unreleased-54e8a88 (2026/07/05) on TikZ 3.1.10 (2023-01-15).

1.6 Incompatible packages

TikZ’s own `circuit` library, which was based on CircuiTikZ, (re?)defines several styles used by this library. In order to have them work together you can use the `compatibility` package option, which basically prefixes the names of all CircuiTikZ `to[]` styles with an asterisk.

So, if loaded with said option, one must write `(0,0) to[*R] (2,0)` and, for transistors on a path, `(0,0) to[*Tmos] (2,0)`, and so on (but `(0,0) node[nmos] {}`). See example at page 281.

Anyway, the compatibility code is a *best effort* task and only very lightly tested — the authors advice is to choose one or the other, without mixing them.

Another thing to take into account is that any TikZ figure (and CircuiTikZ ones qualify) **will** have problems if you use the `babel` package with a language that changes active characters (most of them). The solution is normally to add the line `\usetikzlibrary{babel}` in your preamble, after loading TikZ or CircuiTikZ. This will normally solve the problem; some languages also require using `\deactivatequoting` or the option `shorthands=off` for `babel`. Please check the documentation of TikZ or this question on [T_EX stackexchange site](https://tex.stackexchange.com/).

Finally, the TikZ library `bending` is loaded by the package, and its effects (the bending of the arrows on curved paths) will also affect the rest of your drawings.

1.7 Related and extension packages

1.7.1 Related packages

At the Friedrich-Alexander-Universität, a group of developers are implementing a graphical user interface to draw circuits with CircuiTikZ. You can find more information [in this GitHub issue](#) or, better, on their [main site](#). You can try out the GUI [here](#).

A group at the University of Colorado Colorado Springs (UCCS) are building a cross-platform desktop editor called [Heaviside](#) to draw circuit diagrams with CircuiTikZ. You draw the schematic on a grid and it generates the corresponding CircuiTikZ code, with a live preview of the compiled figure. It integrates into L^AT_EX, Overleaf, and LyX workflows. More information and downloads are on its [GitHub page](#).

1.7.2 Extension packages

CircuiTikZ is meant to be, as much as possible, format-agnostic (which means that it can be used from L^AT_EX, plain T_EX and ConT_EXt). It is growing in functionality and components, but not everything can be added to the package. In this section, there is a list of packages, available at CTAN, that extend or enhance CircuiTikZ but are distributed separately.²

[tikzquads](#) is a package for easily drawing the equivalent electrical circuits for quadrupoles and similar blocks. L^AT_EX only.

[tikzdotncross](#) offers a few alternative ways for declaring and marking coordinates and drawing a line with "jumps" over an already existent path³. L^AT_EX only.

1.8 Known bugs and limitation

CircuiTikZ will **not work** correctly with global (in the main `circuitikz` environment, or in `scope` environments) *negative* scale parameters (`scale`, `xscale` or `yscale`), unless `transform shape` is also used, and even in this cases the behavior is not guaranteed. Neither will it work with angle-changing scaling (when `xscale` is different from `yscale`) and with the global `rotate` parameter.

Correcting this will need a big rewrite of the path routines, and although the authors are thinking about solving it, don't hold your breath; it will need changing a lot of interwoven code (labels, voltages, currents and so on). Contributions and help would be highly appreciated.

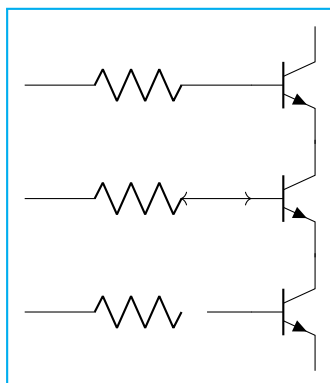
Hopefully most of the issues with compatibility between CircuiTikZ and the `pic` TikZ feature should have been solved in v1.8.0, but the feature is only lightly tested (see example [10.10](#)).

Also, notice that several components will interact in a funny way with global path options. Depending on the specific component, some parameters are inherited by the internal shape, and some others are reset. This is not easy to fix in general. We want some options to go through — fill color, dashed pattern for example — and some others to stay only in the outer path; and if the background shape needs some option for drawing the internal shape, like for example a rounded corner, it *must* reset the external option. So there is no perfect solution, although since v1.5.0 the shapes have been “robustified”, so that by default arced corners and arrows parameters will *not* be propagated into the shape. Arrows with `to[]` components don't work, anyway, so basically avoid this situation. In some cases, also the engine you are using (as `pdflatex`, `xelatex`, and so

²If you have a package or know a package that should be listed here, please contact the CircuiTikZ authors, or, better, send a pull request to the project and it will be added.

³...read the introduction: try not to overuse the jumping symbol; it's not a standard for the last 25 years. Nevertheless, sometimes is very useful, especially for didactic reasons (Romano's personal opinion in this comment).

on) can impact coloring in corner cases (or even not so in the corner, like in [american-style voltage source signs](#)).



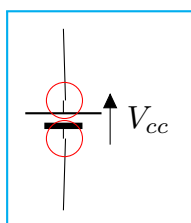
```
\begin{circuitikz}[]
  \draw (0,3) to[R] ++(3,0) node[npn, anchor=B]{};
  % arrows will not work and give strange results
  % so basically do not use them!
  \draw[<->] (0,1.5) to[R] ++(3,0) node[npn, anchor=B]{};
  \draw[shorten <=10pt] (0,0) to[R] ++(3,0)
    node[npn, anchor=B]{};
\end{circuitikz}
```

Lastly, voltage styles interacts in strange ways with general (such as [american](#), [european](#) style), in the sense that sometimes the order in which you enact them is important. That should be arguably fixed, but it will change (read break) a lot of existing code, so it'll stay; more information and workarounds in section [5.5.7](#).

As a final notice, if you want to use the `externalize` library, do not use the `circuitikz` environment: use `tikzpicture` (which is really the same thing, see the [FAQ 8.3](#)).

1.9 Scale factor inaccuracies

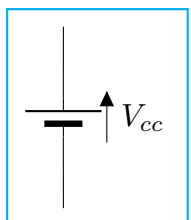
Sometimes, when using fractional scaling factors and big values for the coordinates, the basic layer inaccuracies from $\text{\TeX}$ can bite you, producing results like the following one:



```
\begin{circuitikz}[scale=1.2, transform shape,
]
  \draw (60,1) to [battery2, v_=$V_{cc}$, name=B] ++(0,2);
  \node[draw,red,inner sep=4pt] at(B.left) {};
  \node[draw,red,inner sep=4pt] at(B.right) {};
\end{circuitikz}
```

A general solution for this problem is difficult to find; probably the best approach is to use a `scalebox` command to scale the circuit instead of relying on internal scaling.

Nevertheless, [Schrödinger's cat](#) found a solution which has been ported to CircuiTiKZ: you can use the key `use fpu reciprocal` which will patch a standard low-level math routine with a more precise one.



```
\begin{circuitikz}[scale=1.2, transform shape,
  use fpu reciprocal,
]
  \draw (60,1) to [battery2, v_=$V_{cc}$] ++(0,2);
\end{circuitikz}
```

The `use fpu reciprocal` key seems to have no side effects, but given that it is patching an internal interface of $\text{\TeX}$ it can break any time, so it is advisable to use it only if and when needed.

1.10 Incompatibilities between versions

Here, we will provide a list of incompatibilities between different versions of CircuiTikZ. We will try to hold this list short, but sometimes it is easier to break with old syntax than include a lot of switches and compatibility layers. In general, changes that would invalidate a circuit (changes of polarity of components and so on) are almost always protected by a flag; the same is not true for purely aesthetic changes. If unsure, you can check the version in your local installation by using the macro `\pgfcircversion{}`.

- In version 1.8.2, all the text anchors of the components have been made stable, meaning they are meaningful after the component has been drawn.⁴ That was the case only for part of the components. It *shouldn't* affect anything for standard usage, but according to the TikZ manual, “[this is the way](#)”. Almost no text (see below) should change position (see also section 3.2.2.1).

Two minor changes are that the previously *incorrectly* described `text` anchor for `flowarrow` is not where it used to be shown (use anchor `center` instead); and that the anchors for `oscillator` and `gridnode`, which were completely bogus, have been fixed.

- In version 1.8.0, the path routines changed to allow using path-style components in TikZ's `pic`. The change is a big one, so a rollback point (see below) for v1.7.2 has been added.
- In version 1.7.2, the drawing of the “double-zero” source components `voosource`, `ioosource`, and `oosourcetrans` changed so that now the circle radius depend on the height and not on the width. If you didn't play with internal parameters, that should be invisible.
- In version 1.7.1, the default widths of `barrier` and `openbarrier` were reduced so that no wire is drawn as part of the symbols by default. This is accompanied by a change to the meanings of the configuration keys `bipoles/barrier/width`, `bipoles/barrier/height` and `bipoles/openbarrier/gap` (as well as the new key `bipoles/openbarrier/width`). The change avoids problems when using barriers on short pieces of path or as nodes, cf. the details in [the relevant pull request on GitHub](#). The new default values are chosen such that the components are identical to the default appearance before the change.
- Since version 1.6.3 the default symbol for the minus sign changed from the simple `-$` to `$(\vphantom{+})-$`. The reason is that in some (most?) font, the minus sign is enclosed in a smaller bounding box than the plus sign and that leads to poorly aligned minus symbols in `american` and `raised` voltages. This was not noticed before because the two symbols share the same bounding box in the default Computer Modern font. You can look at [this issue on GitHub](#) for more details; if you want to go back to the previous definitions you can write

```
\ctikzset{amplifiers/minus=$-$}  
\ctikzset{bipoles/cvsourcemap/inner minus=$-$}  
\ctikzset{bipoles/vsourcemap/inner minus=$-$}  
\ctikzset{voltage/american minus=$-$}
```

- Since version 1.6.3 the size of the solder dot and the connection dot of the body diodes for transistors has changed (now they are the same and are configurable). The new default scale of 0.7 makes the dots *area* more or less one half the one of the external connections. You can go back to the previous values with

```
\ctikzset{transistor bodydiode/dot scale=0.5,  
transistor solderdot scale=1.0}
```

⁴The problem has been noticed by [user @JPWiedemann on GitHub](#).

- Since version 1.6.2 `siunitx` will **not** work anymore with `ConTeXt` (it was a very poor simulation layer, anyway); it has been disabled in upstream `ConTeXt`, in favor of [its own units module](#).
- Version 1.6.0 has a big rewrite of the block's code. In principle the changes are backward-compatible, but there were several bugs (wrong anchors, errors with rotations, and so on) that have been fixed in the process.
- Since v1.5.1⁵ color management (see section 7) and the details of how the shapes are drawn and protected by the external drawing options has changed. There should be no substantial changes to the circuits, though.
- The TikZ fix for `to[...] +(x,y)` behavior (see 3.1.3) uncovered a bug in the positioning of the labels in `CircuiTikZ` that had been present since v0.8. So you **must** upgrade to v1.4.1 or better if you have TikZ newer than 3.1.8 (and you want/need to use the `+(x,y)` syntax).
- There have been changes in (internal) parameters for capacitors in v1.4.1; now to change them you should use the style interface (see 4.3.3).
- `CircuiTikZ` v1.4.0 introduced the rollback system for the package when using LaTeX; that (at least in principle) should be completely backward-compatible.
- The path construction in v1.4.0 has been changed a bit (again). The change shouldn't break any circuit and will correct a behavior that should have been fixed with the v1.2.1 change (see below).
- Version 1.3.6 fixes several problems with the stacked labels; the most important change is that now the bracing of arguments is respected as in version 1.3.0 for the other labels. The special treatment in stacked labels (and only in stacked labels!) for the (still experimental⁶) `siunitx` compact syntax `<...>` has been removed: it was completely buggy before, and silently ignored, now will throw an error.
- Version 1.3.3 fixes the direction of the arrows in tunable elements; before this version, they were more or less random, now the arrow goes from bottom left to top right. You have the option to go back to the old behavior with `\ctikzset{bipoles/fix tunable direction=false}`. As a compensation for the fuss, now the arrows are configurable. To learn more, see the FAQ: 8.6.
- Version 1.3.1 removes the warning if you do not specify a voltage direction.
- Version 1.3.0 fixes the buggy stripping of braces from labels and annotations (see 5.1.2).
- After 1.2.7 a big code reorganization (which had the collateral effect of fixing some bugs) has been made; no changes should be visible, but a fallback point at 1.2.7 has been added.
- You **must** upgrade to v1.2.7 or newer if you use a TikZ 3.1.8 or 3.1.8a (but better upgrade both packages to the current version). You can check the TikZ version installed using the macro `\pgfversion`.
- After v1.2.1: **Important:** the routine that implements the `to[...]` component positioning has been rewritten. That should enhance the line joins in paths, and it's safer, but it can potentially change some old behavior.
One of the changes is that the previous routine did the wrong thing if you used `(node)` `to[...]` (you should use an anchor or a coordinate, not a node there — like `(node.anchor)` `to[...]`).

⁵Do not use v1.5.0, it's buggy.

⁶and, really, not advised...

The other one was that in the structure `... to[...] node[pos=something] (coord)` the value of `pos` was completely wrong (even if you don't use `pos` explicitly, remember it's `pos=0.5` by default).

Additionally, the old code disrupted the TikZ path-fill mechanism, so that you could get away with using the `fill` option on paths and having just the components filled, not the path. That was incorrect, although sometime it was handy (sorry).

See the FAQ at section [8.1](#) for more information.

- After v1.2.0: voltage arrows, symbols and label positions are calculated with a rewritten routine. There should be little change, *unless* you touched internal values...
- After v1.1.3: from version 1.1.0 to version 1.1.2, the inverted Schmitt buffer in IEEE style ports was called `inv schmitt` (with an additional space). The correct name is `invschmitt port` (the same as the legacy american port).
- After v1.1.2: the position of `american` voltages for the `open` bipoles changed (you can revert to the old behavior, see section [5.5.5](#)).
- After v0.9.7: the position of the text of transistor nodes has changed; see section [4.15.4](#).
- After v0.9.4: added the concept of styling of circuits. It should be backward compatible, but it's a big change, so be ready to use the 0.9.3 snapshot (see below for details).
- After v0.9.0: the parameters `tripoles/american` or `port/aaa, ...bbb, ...ccc` and `...ddd` are no longer used and are silently ignored; the same stands for the similarly named parameters in `nor`, `xor`, and `xnor` ports.
- After v0.9.0: voltage and current directions/signs (plus and minus signs in case of `american voltages` and arrows in case of `european voltages`) have been rationalized with a couple of new options (see details in section [5.2](#)). The default case is still the same as v0.8.3, to avoid potentially wrong circuits, but you would be better off with one of the new voltage directions (`EFvoltages` or `RPvoltages`) for newer circuits.
- Since v0.8.2: voltage and current label directions (`v<=` / `i<=`) do NOT change the orientation of the drawn source shape anymore. Use the `invert` option to rotate the shape of the source. Furthermore, from this version on, the current label (`i=`) at current sources can be used independent of the regular label (`l=`).
- Since v0.7: The label behavior at mirrored bipoles has changed, this fixes the voltage drawing, but perhaps you will have to adjust your label positions.
- Since v0.5.1: The parts `pfet`, `pigfete`, `pigfetebulk`, and `pigfetd` are now mirrored by default. Please adjust your `yscale`-option to correct this.
- Since v0.5: New voltage counting direction, there exists an option to use the old behavior.

If you have older projects that show compatibility problems, you have two options:

- you can use an older version locally using the git-version and picking the correct commit from the repository (branch `gh-pages`) or the main GitHub site directly;
- if you are using L^AT_EX, the distribution has embedded several important old versions: 0.4, 0.6, 0.7, 0.8.3, 0.9.3, 0.9.6, 1.0, 1.1.2, 1.2.7, 1.4.6, and 1.7.2. To switch to use them, since v1.4.0 you simply use the [new LaTeX kernel rollback system](#), changing your `\usepackage` invocation to something like:

```
% or v0.4, v0.6, ...  
\usepackage[circuitikz]{v0.8.3}
```

You can also specify a date instead of a version number: if you write

```
\usepackage[circuitikz]{2020/02/05}
```

the rollback system will load the version that was current on February 5th, 2020 (in this case it will be v1.0 which was released the day before).

If for whatever reasons your kernel is older, you can still use the old method of loading the *package-version* package; for example:

```
% or circuitikz-0.4, 0.6...  
\usepackage[circuitikz-0.8.3]
```

which is an inferior solution because it can fool any package you use that depend on `circuitikz`.

Either way, you have to take care of the options that may have changed between versions (and sometime styles, if you use them).

- if you are using ConT_EXt, only versions 0.8.3, 0.9.3, 0.9.6, 1.0, 1.1.2, 1.2.7, 1.4.6, and 1.7.2 are packaged; you can use it with

```
\usemodule[circuitikz-0.8.3]
```

1.11 Feedback

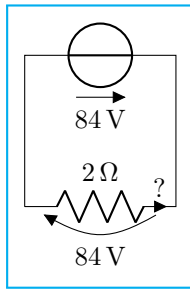
The easiest way to contact the authors is via the official GitHub repository: <https://github.com/circuitikz/circuitikz/issues>. For general help question, a lot of nice people are quite active on <https://tex.stackexchange.com/questions/tagged/circuitikz> — be sure to read the help pages for the site and ask!

1.12 Package options

Circuit people are very opinionated about their symbols. In order to satisfy the individual taste you can set a bunch of package options.

There are arguably way too many options in CircuiTikZ, as you can see in the following list. Since version 1.0, it is recommended to just use the basic ones — voltage directions (you **should** specify one of them), `siunitx` (only for L^AT_EX), the global style (`american` or `european`) and use styles (see 3.3) for the remaining options.

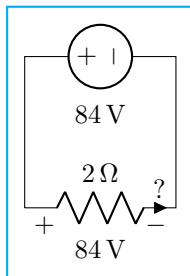
The standard options are set by historical reason, and reflect the preferences of the author that introduced them. For example you get this:



```
\begin{circuitikz}
  \draw (0,0) to[R=2<\ohm>, i=?, v=84<\volt>] (2,0) --
    (2,2) to[V<=84<\volt>] (0,2)
  -- (0,0);
\end{circuitikz}
```

Feel free to load the package with your own cultural options:

L ^A T _E X	ConT _E Xt
<code>\usepackage[american]{circuitikz}</code>	<code>\usemodule[circuitikz][american]</code>



```
\begin{circuitikz}
  [circuitikz/voltage=american, circuitikz/resistor=american] % line
  not printed
  \draw (0,0) to[R=2<\ohm>, i=?, v=84<\volt>] (2,0) --
    (2,2) to[V<=84<\volt>] (0,2)
  -- (0,0);
\end{circuitikz}
```

However, most of the global package options are not available in ConT_EXt; in that case you can always use the appropriate `\tikzset{}` or `\IndexKey{}` command after loading the package.

Here is the list of all the options:

- **europenvoltages**: uses arrows to define voltages, and uses european-style voltage sources;
- **straightvoltages**: uses arrows to define voltages, and uses straight voltage arrows;
- **americanvoltages**: uses $-$ and $+$ to define voltages, and uses american-style voltage sources;
- **europencurrents**: uses european-style current sources;
- **americancurrents**: uses american-style current sources;
- **europenresistors**: uses rectangular empty shape for resistors, as per european standards;
- **americanresistors**: uses zig-zag shape for resistors, as per american standards;
- **europeninductors**: uses rectangular filled shape for inductors, as per european standards;
- **americaninductors**: uses “4-bumps” shape for inductors, as per american standards;
- **cuteinductors**: uses my personal favorite, “pig-tailed” shape for inductors;
- **americanports**: uses triangular logic ports, as per american standards;
- **europenports**: uses rectangular logic ports, as per european standards;
- **americangfsgearrester**: uses round gas filled surge arresters, as per american standards;
- **europengfsgearrester**: uses rectangular gas filled surge arresters, as per european standards;

- `european`: equivalent to `europeancurrents`, `europeanvoltages`, `europeanresistors`, `europeaninductors`, `europeanports`, `europeangfsgeararrester`;
- `american`: equivalent to `americancurrents`, `americanvoltages`, `americanresistors`, `americaninductors`, `americanports`, `americangfsgeararrester`;
- `siunitx`: integrates with `SIunitx` package. If labels, currents or voltages are of the form `#1<#2>` then what is shown is actually `\SI{#1}{#2}` (not supported in `ConTeXt`; it has been disabled in upstream `ConTeXt`, in favor of [its own units module](#));
- `nosiuunitx`: labels are not interpreted as above;
- `fulldiode`: the various diodes are drawn *and* filled by default, i.e. when using styles such as `diode`, `D`, `sD`, ... Other diode styles can always be forced with e.g. `Do`, `D-`, ...
- `strokediode`: the various diodes are drawn *and* stroke by default, i.e. when using styles such as `diode`, `D`, `sD`, ... Other diode styles can always be forced with e.g. `Do`, `D*`, ...
- `emptydiode`: the various diodes are drawn *but not* filled by default, i.e. when using styles such as `D`, `sD`, ... Other diode styles can always be forced with e.g. `Do`, `D-`, ...
- `arrowmos`: pmos and nmos have arrows analogous to those of pnp and npn transistors;
- `noarrowmos`: pmos and nmos do not have arrows analogous to those of pnp and npn transistors;
- `fetbodydiode`: draw the body diode of a FET;
- `nofetbodydiode`: do not draw the body diode of a FET;
- `fetsolderdot`: draw solderdot at bulk-source junction of some transistors;
- `nofetsolderdot`: do not draw solderdot at bulk-source junction of some transistors;
- `emptypmoscircle`: the circle at the gate of a pmos transistor does not get filled;
- `lazymos`: draws lazy nmos and pmos transistors. Chip designers with huge circuits prefer this notation;
- `legacytransistorstext`: the text of transistor nodes is typeset near the collector;
- `nolegacytransistorstext` or `centertransistorstext`: the text of transistor nodes is typeset near the center of the component;
- `straightlabels`: labels on bipoles are always printed straight up, i.e. with horizontal baseline;
- `rotatelabels`: labels on bipoles are always printed aligned along the bipole;
- `smartlabels`: labels on bipoles are rotated along the bipoles, unless the rotation is very close to multiples of 90° ;
- `compatibility`: makes it possible to load `CircuitikZ` and `TikZ` circuit library together.
- Voltage directions: until v0.8.3, there was an error in the coherence between american and european voltages styles (see section 5.2) for the batteries. This has been fixed, but to guarantee backward compatibility and to avoid nasty surprises, the fix is available with new options:
 - `oldvoltagedirection`: Use old way of voltage direction having a difference between european and american direction, with wrong default labeling for batteries;

- **nooldvoltagedirection**: The standard from 0.5 onward, utilizes the (German?) standard of voltage arrows in the direction of electric fields (without fixing batteries);
- **RPvoltages** (meaning Rising Potential voltages): the arrow is in the direction of rising potential, like in **oldvoltagedirection**, but batteries and current sources are fixed to follow the passive/active standard;
- **EFvoltages** (meaning Electric Field voltages): the arrow is in the direction of the electric field, like in **nooldvoltagedirection**, but batteries are fixed;

If none of these option are given, the package will default to **nooldvoltagedirection**. The behavior is also selectable circuit by circuit with the **voltage dir** style.

- **betterproportions**⁷: nicer proportions of transistors in comparison to resistors; notice that this option is superseded by **styles** and it's kept just for compatibility, do not use it in new projects;

The old options in the singular (like **american voltage**) are still available for compatibility, but are discouraged.

Loading the package with no options is equivalent to the following options: **[nofetsolderdot, europeancurrents, europeanvoltages, americanports, americanresistors, cuteinductors, europeangfsurgearrester, nosiunitx, noarrowmos, smartlabels, nocompatibility, centertransistorstext]**.

In ConT_EXt the options are similarly specified: **current= european|american, voltage= european|american, resistor= american|european, inductor= cute|american|european, logic= american|european, arrowmos= false|true**.

⁷May change in the future!

2 Tutorials

Before even starting with CircuiTikZ you should be sure to have understood the basics of TikZ. It is *highly recommended* that you read and go through *at least* the following parts of the TikZ manual:

- “Tutorial: A Picture for Karl’s Students” (around page 30);
- “Specifying Coordinates” (around page 131)
- “Nodes and their shapes” (around page 220)

... but obviously a good knowledge of TikZ will help you a lot. Remember, a circuit drawn with CircuiTikZ is nothing more than a `tikzpicture` with an (albeit powerful and extended) set of shapes and commodity macros.

Said that, to draw a circuit, you have to load the CircuiTikZ package; this can be done with

```
\usepackage[siunitx, RPvoltages]{circuitikz}
```

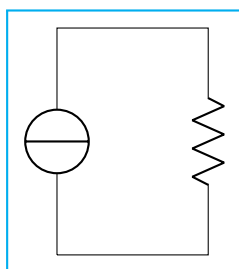
somewhere in your document preamble. It will load automatically the needed packages if not already done before.

2.1 Getting started with CircuiTikZ: a current shunt

Let’s say we want to prepare a circuit to teach how a current shunt works; the idea is to draw a current generator, a couple of resistors in parallel, and the indication of currents and voltages for the discussion.

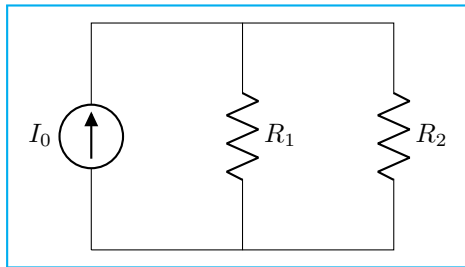
A circuit in CircuiTikZ is drawn into a `circuitikz` environment (which is really an alias for `tikzpicture`). In this first example we will use absolute coordinates. The electrical components can be divided in two main categories: the ones that are bipoles and are placed along a path (also known as `to-style` component, for their usage), and components that are nodes and can have any number of poles or connections.

Let’s start with the first type of component, and build a basic mesh:



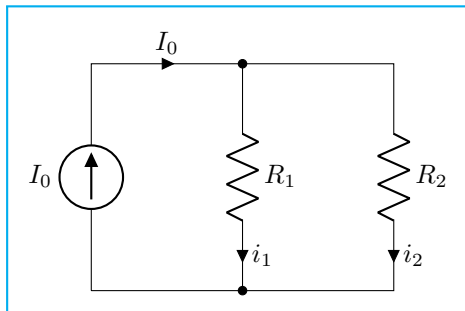
```
\begin{circuitikz}[]
  \draw (0,0) to[isource] (0,3) -- (2,3)
    to[R] (2,0) -- (0,0);
\end{circuitikz}
```

The symbol for the current source might surprise some; this is actually the european-style symbol, and the type of symbol chosen reflects the default options of the package (see section 1.12). Let’s change the style for now (the author of the tutorial, Romano, is European — but he has always used American-style circuits, so...); and while we’re at it, let’s add the other branch and some labels.



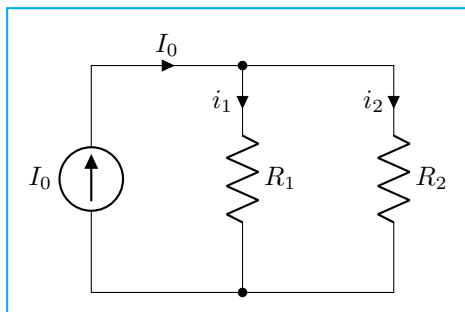
```
\begin{circuitikz}[american]
  \draw (0,0) to[isource, l=$I_0$] (0,3) --
    (2,3)
    to[R=$R_1$] (2,0) -- (0,0);
  \draw (2,3) -- (4,3) to[R=$R_2$]
    (4,0) -- (2,0);
\end{circuitikz}
```

You can use a single path or multiple paths when drawing your circuit, it's just a question of style (but be aware that closing paths perfectly could be non-trivial, see section 6.4), and you can use standard TikZ lines (`--`, `|-` or similar) for the wires. Nonetheless, sometime using the CircuitikZ specific `short` component for the wires can be useful, because then we can add labels and poles to them, as for example in the following circuit, where we add a current (with the key `i=...`, see section 5.3) and a connection dot (with the special shortcut `-*` which adds a `circ` node at the end of the connection, see sections 4.12 and 6.1).



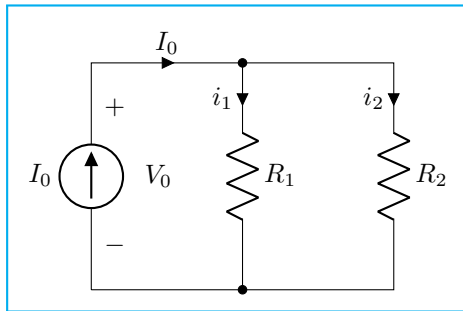
```
\begin{circuitikz}[american]
  \draw (0,0) to[isource, l=$I_0$] (0,3)
    to[short, -*, i=$I_0$] (2,3)
    to[R=$R_1$, i=$i_1$] (2,0) -- (0,0);
  \draw (2,3) -- (4,3)
    to[R=$R_2$, i=$i_2$]
    (4,0) to[short, -*] (2,0);
\end{circuitikz}
```

One of the problems with this circuit is that we would like to have the current labels in a different position, such as for example on the upper side of the resistors, so that Kirchhoff's Current Law at the node is better shown to students. No problem; as you can see in section 5.2 you can use the position specifiers `<>^_` after the key `i`:



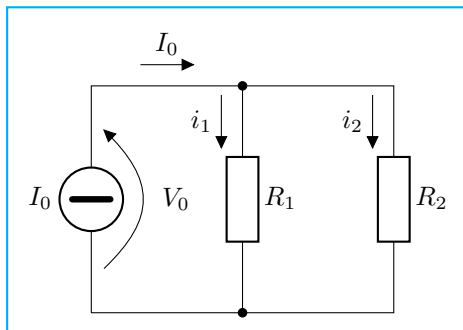
```
\begin{circuitikz}[american]
  \draw (0,0) to[isource, l=$I_0$] (0,3)
    to[short, -*, i=$I_0$] (2,3)
    to[R=$R_1$, i>_=$i_1$] (2,0) -- (0,0);
  \draw (2,3) -- (4,3)
    to[R=$R_2$, i>_=$i_2$]
    (4,0) to[short, -*] (2,0);
\end{circuitikz}
```

Finally, we would like to add voltages indication for carrying out the current formulas; as the default position of the voltage signs seems a bit cramped to me, I am adding the `voltage shift` parameter to make a bit more space for it...



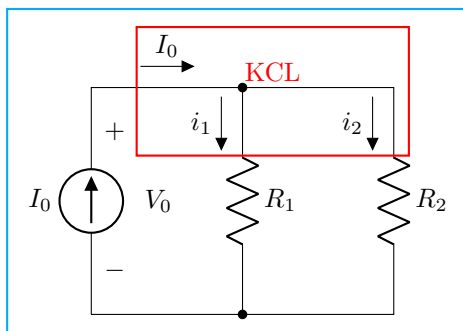
```
\begin{circuitikz}[american, voltage shift=0.5]
\draw (0,0)
to[isource, l=$I_0$, v=$V_0$] (0,3)
to[short, -, i=$I_0$] (2,3)
to[R=$R_1$, i>_=$i_1$] (2,0) -- (0,0);
\draw (2,3) -- (4,3)
to[R=$R_2$, i>_=$i_2$]
(4,0) to[short, -] (2,0);
\end{circuitikz}
```

Et voila! Remember that this is still $\text{\LaTeX}$, which means that you have done a description of your circuit, which is, in a lot of way, independent of the visualization of it. If you ever have to adapt the circuit to, say, a journal that forces European style and flows instead of currents, you just change a couple of things and you have what seems a completely different diagram:



```
\begin{circuitikz}[european, voltage shift=0.5]
\draw (0,0)
to[isourceC, l=$I_0$, v=$V_0$] (0,3)
to[short, -, f=$I_0$] (2,3)
to[R=$R_1$, f>_=$i_1$] (2,0) -- (0,0);
\draw (2,3) -- (4,3)
to[R=$R_2$, f>_=$i_2$]
(4,0) to[short, -] (2,0);
\end{circuitikz}
```

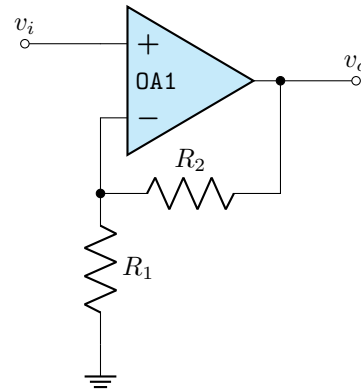
And finally, this is still $\text{\LaTeX}$, so that you can freely mix other graphics element to the circuit.



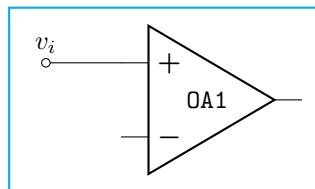
```
\begin{circuitikz}[american, voltage shift=0.5]
\draw (0,0)
to[isource, l=$I_0$, v=$V_0$] (0,3)
to[short, -, f=$I_0$] (2,3)
to[R=$R_1$, f>_=$i_1$] (2,0) -- (0,0);
\draw (2,3) -- (4,3)
to[R=$R_2$, f>_=$i_2$]
(4,0) to[short, -] (2,0);
\draw[red, thick] (0.6,2.1) rectangle
(4.2,3.8)
node[pos=0.5, above]{KCL};
\end{circuitikz}
```


2.2 A non-inverting op-amp amplifier

Let's now try to draw a non-inverting amplifier based on op-amps; the canonical implementation can be, for example, [this one from “electronics tutorials”](#). Obviously, the style and form of drawing a circuit is often a matter of personal tastes and, maybe even more important, of the details you are focusing on; drawing a non-inverting amplifier will be different if you are drawing it to *explain how it works* or if you are simply using it in a more complex circuit, assuming its operation well known by the reader. Anyway, the final objective is to have a circuit like the one on the right, drawn so that it is easy to reuse.



We have to start the drawing from a generic point. Given that the idea is to have a reusable block, instead of positioning the op-amp and build around it, we will start from the input “pole”:



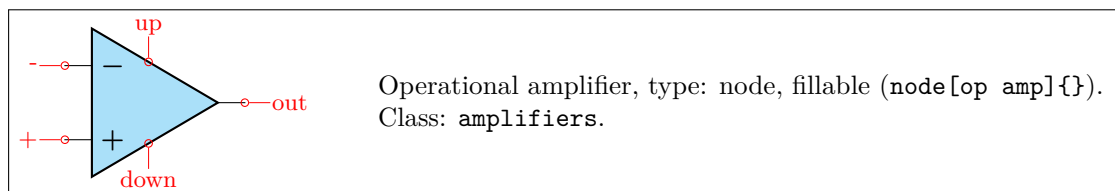
```
\begin{circuitikz}[]
  \draw (0,0) node[above]{$v_i$} to[short, o-] ++(1,0)
  node[op amp, noinv input up, anchor=+] (OA){\texttt{OA1}}
  ;
\end{circuitikz}
```

In this snippet, notice that the only absolute coordinate is the first one; that will enable us to “copy and paste” the circuit in several places, or create a macro for it. We position a text node above it, and then draw a wire with a pole to a relative $(1,0)$ coordinate: in other words, we *move* 1 unit to the right drawing a short-circuit, which is the same as a wire. The usage of `to[short...]` simplifies the position of the pole, but notice that we could have also written:

```
\draw (0,0) node[above]{$v_i$} node[ocirc]{} -- ++(1,0) ...
```

with the same result.

The second step is to position the op-amp. We can check the manual and see the component's description (section 4.20):



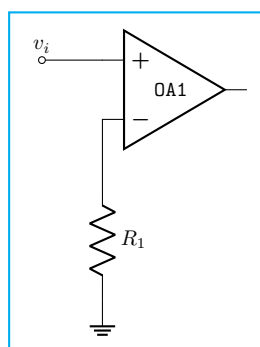
where we notice the type of the component (it is a node-type component, so we have to use `node` to position it) and the available “anchors”: points we can use to position the shape or to connect to. Not all the anchors are *explicitly* printed in the description box; you should read further in the manual and you'll see a “*component anchors*” (4.20.1) section with the relevant information.

Anyway, the op-amp must be connected with the `+` anchor to our input wire, so we say `anchor=+` in the option lists; this shifts the whole element so that the named anchor will lie at the current position of the path. Moreover, normally the shape has the inverting input on the bottom side, and we want it the other way around, so we use also `noinv input up` in the keys defining the

node. We could also have flipped the shape with `yscale=-1`, but in this case we would need to consider the effects on anchors and on the text; see section 3.2.1.

Now we can draw the resistors; let's start with R_1 . We will draw it going down vertically from the - anchor — we have named the node OA so that will be OA.-. We will need to connect the R_2 also, so we do the following:

- draw a wire going down, and mark a point where we want the feedback resistor to connect;
- then draw R_1 and finally
- draw the ground node.



```
\begin{circuitikz}[scale=0.8, transform shape]
  \draw (0,0) node[above]{$v_i$} to[short, o-] ++(1,0)
  node[op amp, noinv input up, anchor=+] (OA){\texttt{OA1}}
  (OA.-) -- ++(0,-1) coordinate(FB)
  to[R=$R_1$] ++(0,-2) node[ground]{}
;
\end{circuitikz}
```

We are only missing the feedback resistor now. We will use *orthogonal coordinates*, writing:

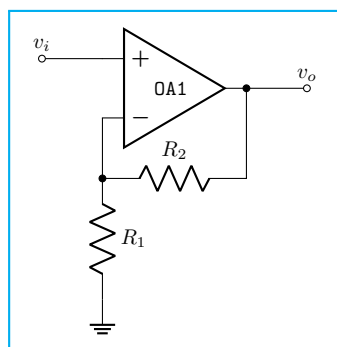
```
\draw (FB) to[R=$R_2$] (FB -| OA.out) -- (OA.out)
```

The meaning is the following:

- move the current point to the coordinate named FB;
- put a resistor, with label R_2 , from here **to**...
- the coordinates, which are at the intersection of a horizontal line through FB and a vertical line through OA.out: the -| coordinate operation is quite mnemonic;
- then continue drawing to OA.out.

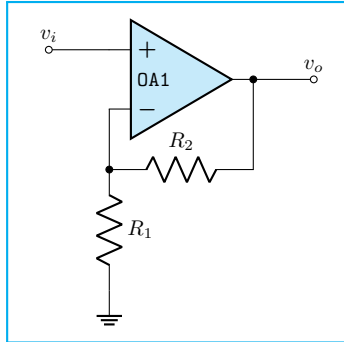
You can use a separate `\draw` command or just continue the path you were writing; the choice is just personal preference, but be warned that it can affect the drawing of poles (see section 6.1 about this if you notice strange things).

Finally, we add the output and a couple of nodes:



```
\begin{circuitikz}[scale=0.8, transform shape]
  \draw (0,0) node[above]{$v_i$} to[short, o-] ++(1,0)
  node[op amp, noinv input up, anchor=+] (OA){\texttt{OA1}}
  (OA.-) -- ++(0,-1) coordinate(FB)
  to[R=$R_1$] ++(0,-2) node[ground]{}
  (FB) to[R=$R_2$, *-] (FB -| OA.out) -- (OA.out)
  to [short, *-o] ++(1,0) node[above]{$v_o$}
;
\end{circuitikz}
```

The last step to obtain the final look is to add a bit of styling. We want the op-amp filled with a light cyan color, and we prefer to have the label aligned with the left side of the device:



```
\ctikzset{amplifiers/fill=cyan!20, component text=left}
\begin{circuitikz}[scale=0.8, transform shape]
  \draw (0,0) node[above]{$v_i$} to[short, o-] ++(1,0)
    node[op amp, noinv input up, anchor=+] (OA) {\texttt{OA1}}

  (OA.-) -- ++(0,-1) coordinate(FB)
  to[R=$R_1$] ++(0,-2) node[ground]{}
  (FB) to[R=$R_2$, *-] (FB -| OA.out) -- (OA.out)
  to [short, *-o] ++(1,0) node[above]{$v_o$}
;
\end{circuitikz}
```

The `\ctikzset` command choosing the style is better placed in the preamble, though. Style should be coherent for all the document body, so avoiding stating it for every circuit is normally the best strategy.

2.2.1 Reusing the circuit: the easy way

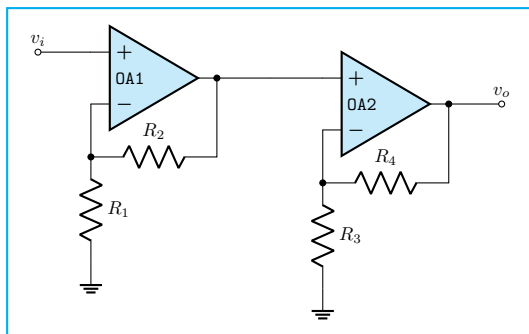
The easiest way to reuse the circuit is to put it in a macro. This is a very flexible way of doing it; the only drawback is that the only easy way to position it is using the first coordinate: you will not be able to move the component using “anchors”; that is more complex and will need the use of subcircuits (but you will lose parameters...see section 3.4).

Defining a macro for our amplifier could be as easy as this:

```
\ctikzset{amplifiers/fill=cyan!20, component text=left}
\newcommand\myNIA[4]{%1: name of this amplifier, %2 start coordinate, %3 R1, %4 R2
  \draw #2 coordinate(#1-in) to[short] ++(1,0)
  node[op amp, noinv input up, anchor=+] (#1-OA) {\texttt{#1}}
  (#1-OA.-) -- ++(0,-1) coordinate(#1-FB)
  to[R=#3] ++(0,-2) node[ground]{}
  (#1-FB) to[R=#4, *-] (#1-FB -| #1-OA.out) -- (#1-OA.out)
  to [short, *-] ++(1,0) coordinate(#1-out)
;
}
```

We remove the open poles (it's better to draw them at the end to avoid artifacts) and then we make the names of the coordinates and of the nodes unique, by prepending a parameter that we will provide at every invocation. Then we remove the labels (for simplicity here) and add a couple of coordinates that we will be able to use from the outside when building our circuit.

And we can use it like in the following:



```
\begin{circuitikz}[scale=0.7, transform shape]
]
\myNIA{OA1}{(0,0)}{$R_1$}{$R_2$}
% start drawing from the output of OA1
\myNIA{OA2}{(OA1-out)}{$R_3$}{$R_4$}
\node [ocirc] at (OA1-in) {};
\node [above] at (OA1-in) {$v_i$};
\node [ocirc] at (OA2-out) {};
\node [above] at (OA2-out) {$v_o$};
\draw (OA1-out) -| (OA2-in);
\end{circuitikz}
```

2.3 A transistor-based amplifier

The idea is to draw a two-stage amplifier for a lesson, or exercise, on the different qualities of BJT and MOSFET transistors.

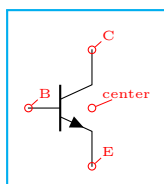
Please notice that this section uses the “new” position for transistor labels, enabled since version 0.9.7. You should refer to older manuals to see how to do the same with older versions; basically the transistor’s names were output using an additional `node{}` command.

Also notice that this is a more “personal” tutorial, showing a way to draw circuits that is, in the author’s opinion, highly reusable and easy to do. The idea is to use relative coordinates and named nodes as much as possible, so that changes in the circuit are easily done by changing just a few numbers that select relative positions and by using symmetries. Crucially, this kind of approach makes each block reusable in other diagrams by just changing one coordinate.

First of all, let’s define a handy function to show the position of nodes:

```
\def\normalcoord(#1){coordinate(#1)}
\def\showcoord(#1){coordinate(#1) node[circle, red, draw, inner sep=1pt,
    pin=[{red, overlay, inner sep=0.5pt, font=\tiny, pin distance=0.1cm,
    pin edge={red, overlay}}]45:#1]}(){}
\let\coord=\showcoord
```

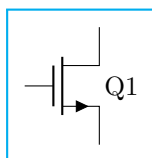
The idea is that you can use `\coord()` instead of `coordinate()` in paths, and that will draw small red *markers* showing them. For example:



```
\begin{circuitikz}[american]
    \draw (0,0) node[npn] (Q){};
    \path (Q.center) \coord(center)
    (Q.B) \coord(B) (Q.C) \coord(C)
    (Q.E) \coord(E);
    \useasboundingbox
    (Q.south west) (Q.north east) +(.75,.3);
\end{circuitikz}
```

After the circuit is drawn, simply commenting out the second `\let` command will hide all the markers.

So let’s start with the first stage transistor; given that my preferred way of drawing a MOSFET is with arrows, I’ll start with the command `\ctikzset{tripoles/mos style/arrows}`:



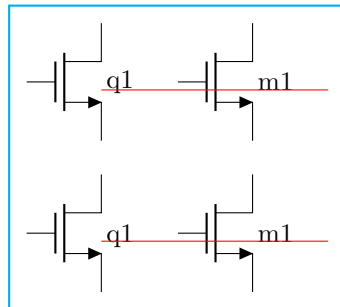
```
\begin{circuitikz}[american]
\ctikzset{tripoles/mos style/arrows}
\def\killdepth#1{\raisebox{0pt}[\height][0pt]{#1}}
\path (0,0) -- (.5,0); % bounding box
\draw (0,0) node[nmos] (Q1){\killdepth{Q1}};
\end{circuitikz}
```

I had to do draw an invisible line to take into account the text for Q1 — the text is not taken into account in calculating the bounding box. This is because the “geographical” anchors (`north`, `north west`, ...) are defined for the symbol only. In a complex circuit, this is rarely a problem.

Another thing I like to modify with respect to the standard is the position of the arrows in transistors, which are normally in the middle of the symbol. Using the following setting (see section 4.15.5) will move the arrows to the start or end of the corresponding pin.

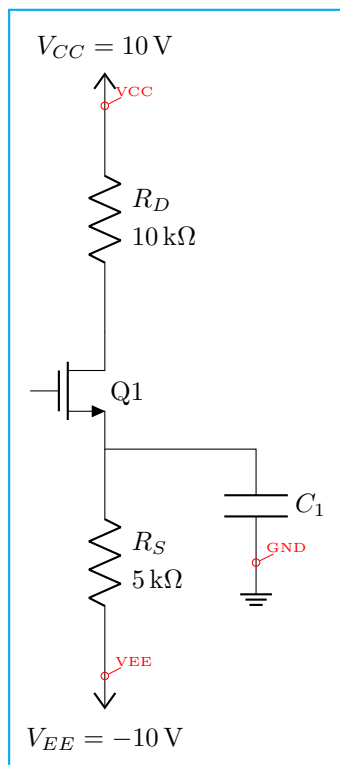
```
\ctikzset{transistors/arrow pos=end}
```

The tricky thing about `\killdepth{}` macro is the finicky details. Without the `\killdepth` macro, the labels of different transistors will be adjusted so that the vertical center of the box is at the `center` anchor, and as an effect, labels with descenders (like Q) will have a different baseline than labels without. You can see this here (it's really subtle):



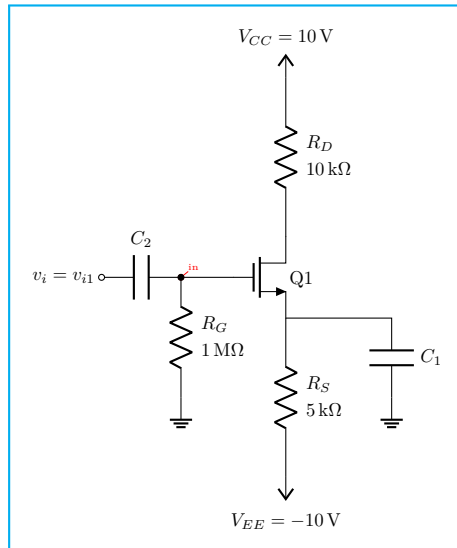
```
\begin{circuitikz}[american,]
\draw (0,0) node[nmos] (Q1){q1} ++(2,0)
      node[nmos] (M1){m1};
\draw [red] (Q1.center) ++(0,-0.7ex) -- ++(3,0);
\draw (0,-2) node[nmos] (Q1){\killdepth{q1}} ++(2,0)
      node[nmos] (M1){\killdepth{m1}};
\draw [red] (Q1.center) ++(0,-0.7ex) -- ++(3,0);
\end{circuitikz}
```

We will start connecting the first transistor with the power supply with a couple of resistors. Notice that I am naming the nodes `GND`, `VCC` and `VEE`, so that I can use the coordinates to have all the supply rails at the same vertical position (more on this later).



```
\begin{circuitikz}[american,]
\draw (0,0) node[nmos,] (Q1){\killdepth{Q1}};
\draw (Q1.S) to[R, l2^=$R_S$ and \SI{5}{k\ohm}]
      ++(0,-3) node[vee] (VEE){$V_{EE}=\SI{-10}{V}$};
\draw (Q1.D) to[R, l2^=$R_D$ and \SI{10}{k\ohm}]
      ++(0,3) node[vcc] (VCC){$V_{CC}=\SI{10}{V}$};
\draw (Q1.S) to[short] ++(2,0) to[C=$C_1$]
      ++(0,-1.5) node[ground] (GND){};
% show the named coordinates!
\path (GND) \coord(GND)
      (VCC) \coord(VCC)
      (VEE) \coord(VEE);
\end{circuitikz}
```

After that, let's add the input part. I will use a named node here, referring to it to add the input source. Notice how the ground node is positioned: the coordinate (`in | - GND`) is the point with the horizontal coordinate of (`in`) and the vertical one of (`GND`), lining it up with the ground of the capacitor C_1 (you can think it as “the point aligned vertically with `in` and horizontally with `GND`”).



```
\begin{circuitikz}[american, scale=0.7, transform
shape]
\draw (0,0) node[nmos,](Q1){\killdepth{Q1}};
\draw (Q1.S) to[R, l2^=$R_S$ and \SI{5}{k\ohm}]
++(0,-3) node[vee](VEE){$V_{EE}=\SI{-10}{V}$};
\draw (Q1.D) to[R, l2_=$R_D$ and \SI{10}{k\ohm}]
++(0,3) node[vcc](VCC){$V_{CC}=\SI{10}{V}$};
\draw (Q1.S) to[short] ++(2,0) to[C=$C_1$]
++(0,-1.5) node[ground](GND){};
\draw (Q1.G) to[short] ++(-1,0)
\coord (in) to[R, l2^=$R_G$ and \SI{1}{M\ohm}]
(in |- GND) node[ground]{};
\draw (in) to[C, l_=$C_2$,*-o]
++(-1.5,0) node[left](vi1){$v_i=v_{i1}$};
\end{circuitikz}
```

Notice that the only absolute coordinate here is the first one, (0,0); so the elements are connected with relative movements and can be moved by just changing one number (for example, changing the `to[C=$C_1$] ++(0,-1.5)` will move *all* the grounds down).

This is the final circuit, with the nodes still marked:

```
% this is for the blue brackets under the circuit
\tikzset{blockdef/.style={%
  {Straight Barb[harpoon, reversed, right, length=0.2cm]}-{Straight Barb[harpoon,
    reversed, left, length=0.2cm]},
  blue,
}}
\def\killdepth#1{{\raisebox{0pt}{\height}[0pt]{#1}}}
\def\coord(#1){coordinate(#1)}
\def\coord(#1){coordinate(#1) node[circle, red, draw, inner sep=1pt,
  pin={[red, overlay, inner sep=0.5pt, font=\tiny, pin distance=0.1cm,
    % we reset the arrow in pin edge to avoid carrying over the path one!
    pin edge={red, overlay,-}]45:#1}](#1-node){}}
\begin{circuitikz}[american, ]
  \draw (0,0) node[nmos,](Q1){\killdepth{Q1}};
  \draw (Q1.S) to[R, l2^=$R_S$ and \SI{5}{k\ohm}] ++(0,-3) node[vee](VEE){$V_{EE}=\SI{-10}{V}$}; %define VEE level
  \draw (Q1.S) to[short] ++(2,0) to[C=$C_1$] ++(0,-1.5) node[ground](GND){};
  \draw (Q1.G) to[short] ++(-1,0) \coord (in) to[R, l2^=$R_G$ and \SI{1}{M\ohm}] (in |- GND) node[ground]{};
  \draw (in) to[C, l_=$C_2$,*-o] ++(-1.5,0) node[left](vi1){$v_i=v_{i1}$};
  \draw (Q1.D) to[R, l2_=$R_D$ and \SI{10}{k\ohm}] ++(0,3) node[vcc](VCC){$V_{CC}=\SI{10}{V}$};
  \draw (Q1.D) to[short, -o] ++(1,0) node[right](vo1){$v_{o1}$};
  %
  \path (vo1) -- ++(2,0) \coord(bjt);
  %
  \draw (bjt) node[npn, anchor=B](Q2){\killdepth{Q2}};
  \draw (Q2.B) to[short, -o] ++(-0.5,0) node[left](vi2){$v_{i2}$};
  \draw (Q2.E) to[R, l2^=$R_E$ and \SI{9.3}{k\ohm}] (Q2.E |- VEE) node[vee]{};
  \draw (Q2.E) to[short, -o] ++(1,0) node[right](vo2){$v_{o2}$};
  \draw (Q2.C) to[short] (Q2.C |- VCC) node[vcc]{};
  %
  \path (vo2) ++(1.5,0) \coord(load);
```

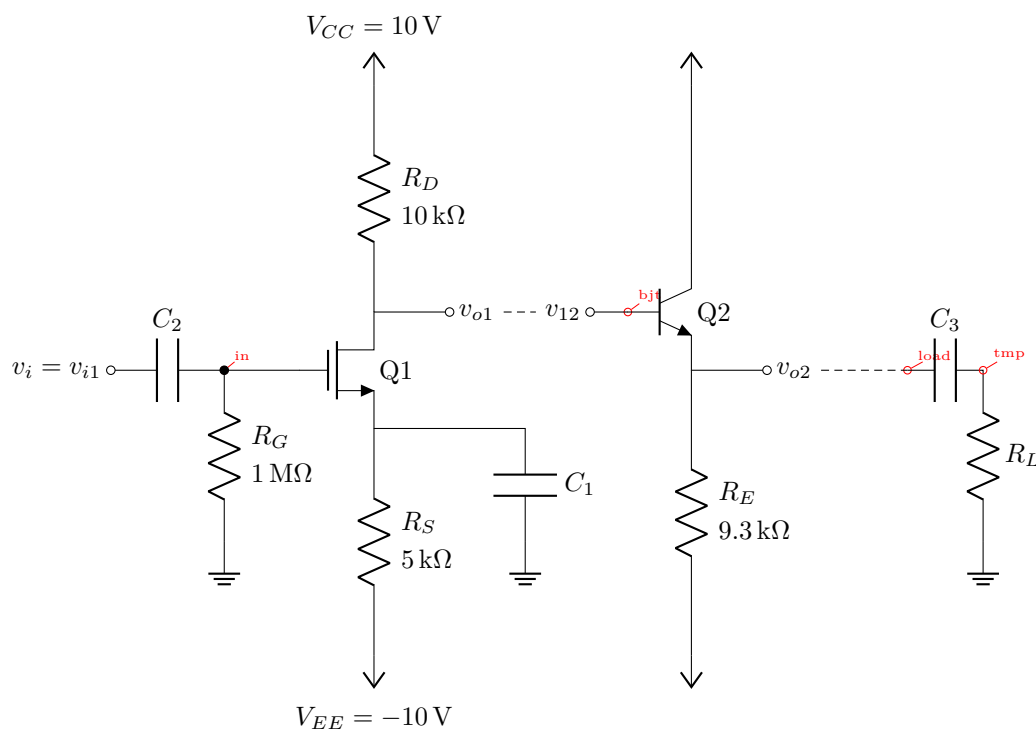
```

\draw (load) to[C=$C_3$] ++(1,0) \coord(tmp) to[R=$R_L$] (tmp |- GND) node[ground]{};

\draw [densely dashed] (vo2) -- (load);
%
\draw [densely dashed] (vo1) -- (vi2);
%
\draw [blockdef] (vi1|-VEE) ++(0,-2) \coord(tmp)
-- node[midway, fill=white]{bloque 1} (vo1|- tmp);
\draw [blockdef] (vi2|-VEE) ++(0,-2) \coord(tmp)
-- node[midway, fill=white]{bloque 2} (vo2|- tmp);

\end{circuitikz}

```



bloque 1

 bloque 2

You can see that after having found the place where we want to put the BJT transistor, we use the option `anchor=B` so that the base anchor will be put at the coordinate `bjt`.

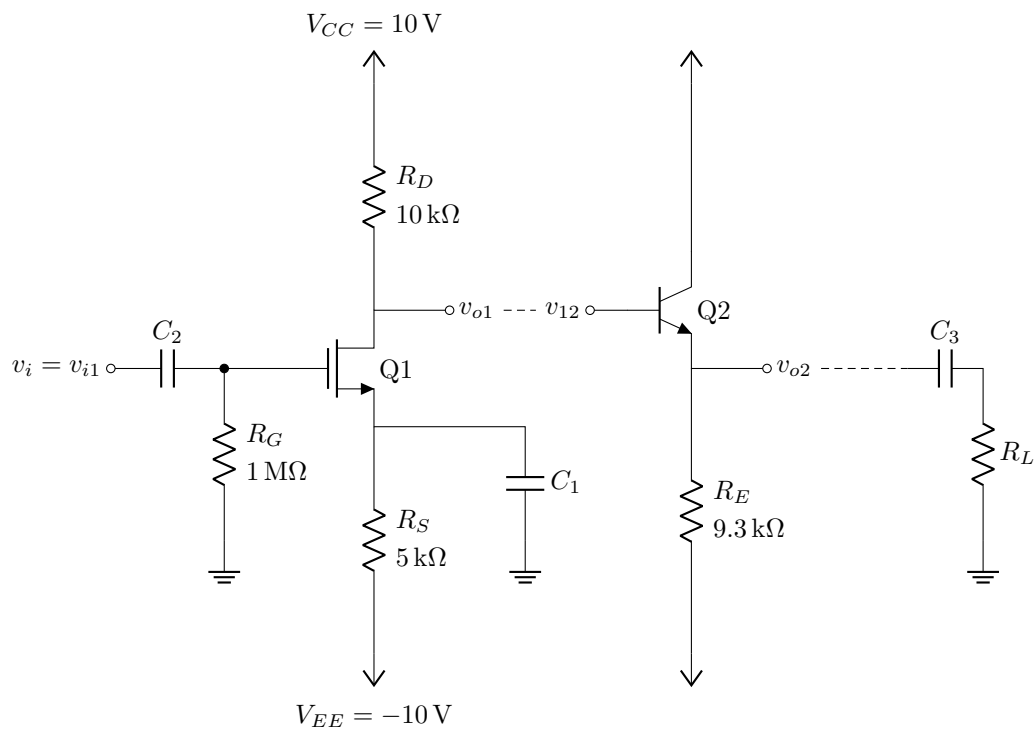
Finally, if you like a more compact drawing, you can add the options (for example):

```

\begin{circuitikz}[american, scale=0.8] % this will scale only the coordinates
\ctikzset{resistors/scale=0.7, capacitors/scale=0.6}
...
\end{circuitikz}

```

and you will obtain the following diagram with the exact same code (I just removed the second `\coord` definition to hide the coordinates markings).



⎵ bloque 1 ⎶

⎵ bloque 2 ⎶

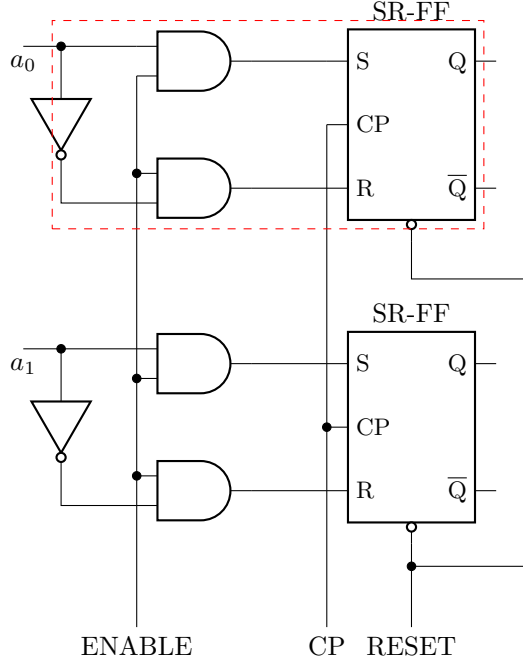
2.4 A logic circuit

Let's suppose we want to reproduce the circuit on the right⁸, maybe as part of a more complex one.

Looking at the circuit to draw, I see that there is a basic block: the flip-flop with the added three-port circuit to its left, marked with the red dashed rectangle. The main distance to respect here is that we want the two ANDs in line with the flip-flop inputs, so I'll start with the flip-flop and then add the rest of the block.

The shapes are very similar to the IEEE logic gates (see section 4.22.2); after a first check, the standard size of the port is a bit too big with respect to the flip-flop, so I scale them down a bit.

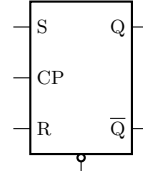
```
\ctikzset{
  logic ports=ieee,
  logic ports/scale=0.7,
}
```



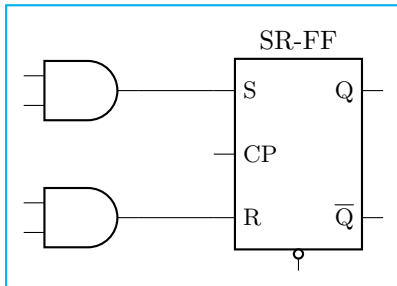
I want a reusable block, so I will start from a coordinate and then use only relative, defining coordinates along the way.

The first thing is to define a suitable flip-flop. The standard SR (see 4.23) is *almost* what we need, but not exactly the same. So let's define a new one:

```
\tikzset{sr-ff/.style={flipflop, flipflop def={
  t1=S, t2=CP, t3=R, t4={\ctikztextnot{Q}},
  t6=Q, nd=1}},
}
```



Now we can add the “and” gates. For example, we can add the gates to the right like this:



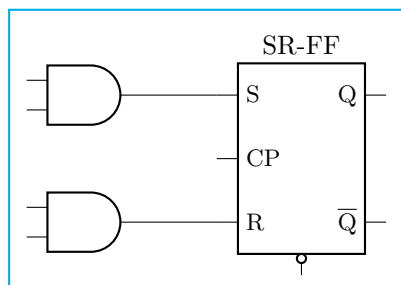
```
\begin{circuitikz}[]
\draw (0,0) node[sr-ff](FF){} (FF.bup)
node[above]{SR-FF};
\draw (FF.pin 1) -- ++(-1,0) node[and port,
anchor=out](AND1){}
(FF.pin 3) -- ++(-1,0) node[and port,
anchor=out](AND2){};
\end{circuitikz}
```

You can notice a pair of things here: first of all, the use of the `anchor=out` in the port, to tell TikZ that we want the node moved so that the out anchor is the reference one. The second one is that we have repeated the absolute shift (the `++(-1, 0)`) twice. This is a bad practice; it is much

⁸It seems a quite popular one on [tex.stackexchange](https://tex.stackexchange.com)...

better to have the “free” parameters of a schematic just stated once, so that we can change them in just one point.

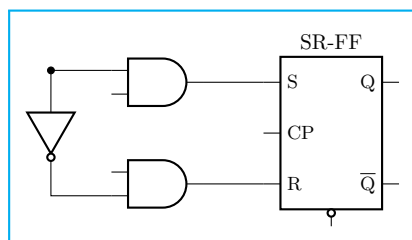
You can of course use a macro, like `\newcommand{\andshift}{(-1,0)}` but it is much more elegant to do something like this:



```
\begin{circuitikz}[]
\draw (0,0) node[sr-ff](FF){} (FF.bup)
node[above]{SR-FF};
\draw (FF.pin 1) -- ++(-1,0) node[and port,
anchor=out](AND1){}
(FF.pin 3) -- (FF.pin 3 -| AND1.out)
node[and port, anchor=out](AND2){};
\end{circuitikz}
```

In this snippet, the coordinate `(FF.pin 3 -| AND1.out)` is the TikZ way to say “the point which is horizontally straight from `FF.pin 3` and vertically from `AND1.out`”. That way one can change the number `-1` to move both AND ports nearer or farther away.

Now we can add the not port. Since version 1.1.3 you can use a path-style not port, so you can just say: this:



```
\begin{circuitikz}[scale=0.8, transform shape]
\draw (0,0) node[sr-ff](FF){} (FF.bup)
node[above]{SR-FF} (FF.pin 1) -- ++(-1,0)
node[and port, anchor=out](AND1){}
(FF.pin 3) -- (FF.pin 3 -| AND1.out)
node[and port, anchor=out](AND2){}
(AND1.in 1) to[short, -*] ++(-1,0) coordinate(in)
to[inline not] (in |- AND2.in 2) -- (AND2.in 2);
\end{circuitikz}
```

In earlier versions, you should have found the center point between the two terminal, position the “not” shape and then connect it, like for example (this code must stay into the `\draw` command):

```
% let's position the NOT in the center
% this is using the calc tikz library
($(in)!0.5!(in |- AND2.in 2)$) node[not port, rotate=-90](NOT){}
% and connect it
(in) -- (NOT.in) (NOT.out) |- (AND2.in 2)
```

Now we have the basic block; we have to use it twice, so one of the possible ways to do it is to prepare a command. We will change the names of the nodes and the coordinates to be different for any “call” of the block (another option is to use a `pic`; but this is more straightforward).

```
\newcommand*{\myblock}[1]{% Add #1- to the node and coord names
node[sr-ff](#1-FF){} (#1-FF.bup) node[above]{SR-FF}
(#1-FF.pin 1) -- ++(-1,0) node[and port, anchor=out](#1-AND1){}
(#1-FF.pin 3) -- (#1-FF.pin 3 -| #1-AND1.out)
node[and port, anchor=out](#1-AND2){}
(#1-AND1.in 1) to[short, -*] ++(-1,0) coordinate(#1-in)
to[inline not] (#1-in |- #1-AND2.in 2) -- (#1-AND2.in 2)
}
```

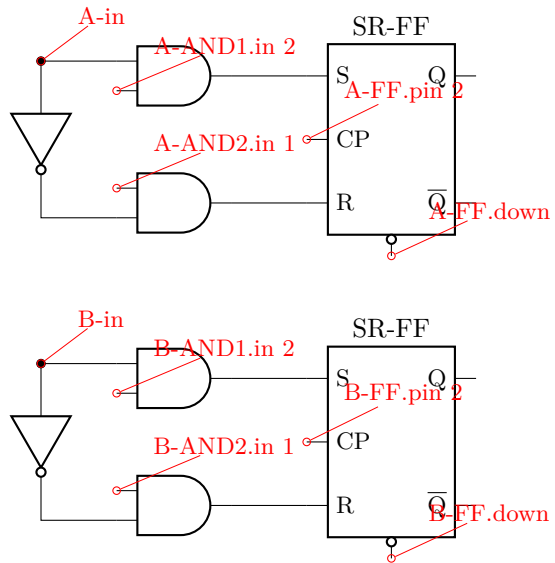
So now we can draw two of our blocks:

```
\draw (0,0) \myblock{A};
\draw (0,-4) \myblock{B};
```

Part of the anchors and coordinates that we have accessible are marked in red in the diagram at the side.

Now we have to just connect the relevant parts and add the labels. The names of the inputs are quite easy:

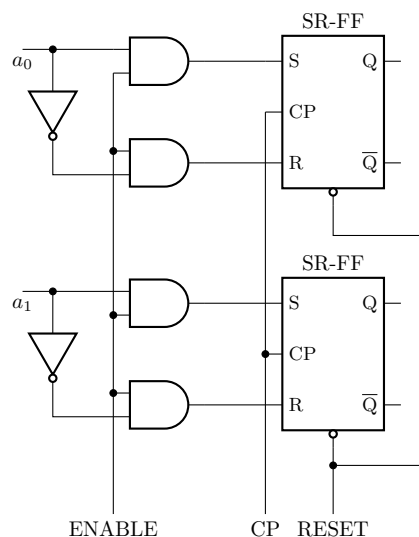
```
\draw (A-in) -- ++(-0.5, 0) node[
below]{$a_0$};
\draw (B-in) -- ++(-0.5, 0) node[
below]{$a_1$};
```



And finally:

```
\draw (A-AND1.in 2) to[short, -*] (A-AND2.in 1)
to[short, -*] (B-AND1.in 2) to[short, -*] (B-AND2.in 1)
-- ++(0, -2) coordinate(down) node[below]{ENABLE};
\draw (A-FF.pin 2) to[short, -*] (B-FF.pin 2)
-- (B-FF.pin 2 |- down) node[below]{CP};
\draw (B-FF.down) to[short, -*] ++(0,-0.3) coordinate(dd);
\draw (A-FF.down) -- ++(0,-.5) -- ++(1.5,0) |- (dd)
-- (dd |- down) node[below]{RESET};
```

Will create the final diagram:



3 The components: usage

Components in `CircuitikZ` come in two forms: path-style, to be used in a `to[component,...]` path specifications, and node-style, which will be instantiated by a `node[component,...]` specification. All the shapes defined by `CircuitikZ` are `pgf` nodes, so they are usable in both `pgf` and `TikZ`.

3.1 Path-style components

The path-style components are used as shown below:

```
\begin{circuitikz}
\draw (0,0) to[#1=#2, options] (2,0);
\end{circuitikz}
```

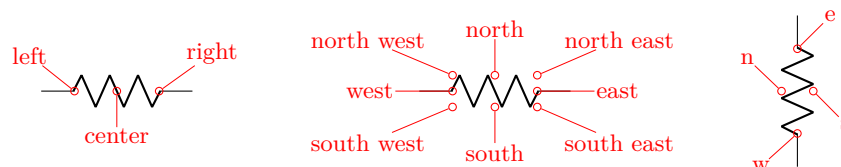
where `#1` is the name of the component, `#2` is an (optional) label, and `options` are optional labels, annotations, style specifier that will be explained in the rest of the manual.

Transistors and some other node-style components can also be placed using the syntax for bipoles. See section 4.15.10.

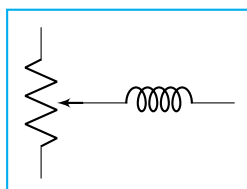
Most path-style components can be used as a node-style components; to access them, you add a `shape` to the main name of component (for example, `diodeshape`). Such a “node-shape name” is specified in the description of each component.

3.1.1 Anchors

Normally, path-style components do not need anchors, although they have them just in case you need them. You have the basic “geographical” anchors (bipoles are defined horizontally and then rotated as needed):

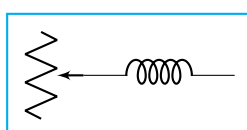


In the case of bipoles, also shortened geographical anchors exists. In the description, it will be shown when a bipole has additional anchors. To use the anchors, just give a name to the bipole element using the syntax `name=myname`.



```
\begin{circuitikz}
\draw (0,0) to[potentiometer, name=P, mirror] ++(0,2);
\draw (P.wiper) to[L] ++(2,0);
\end{circuitikz}
```

Alternatively, that you can use the shape form, and then use the `left` and `right` anchors to do your connections.



```
\begin{circuitikz}
\draw (0,0) node[potentiometershape, rotate=-90] (P){};
\draw (P.wiper) to[L] ++(2,0);
\end{circuitikz}
```

3.1.2 Border anchors

Bipoles have also installed generic border anchors — that means, anchors that start at an angle. For complexity reason, these are for most of the components simply a generic enclosing rectangle (even for most of the round ones!⁹). They interact in a non-trivial way with the `and` and `keys`, so it's best not to use them directly.



You can notice that the border anchors are a bit spaced out (this is useful because those anchors are used to position labels and annotations). You can override this if you need to reach exactly the border (whatever could that mean depends on the component) by using the key `bipoles/border margin`, which is a number that states how much the enclosing border is stretched out (default value is 1.1). For example, setting `\ctikzset{bipoles/border margin=1}` will make the border anchor coincide with the geographical shape:



The above diagram has been obtained with the code:

```
\def\showbordersfornode#1{%
\begin{circuitikz}[baseline, scale=0.8, transform shape]
  \node[#1shape, name=bip] at(0,0) {};
  \foreach \a in {0,30,...,359} \draw[red] (bip.\a) -- ++(\a:0.7)
    node[font=\tiny, fill=white, inner sep=0.5pt]{\a};
  \foreach \a in {15,45,...,359} \draw[red] (bip.\a) -- ++(\a:0.4);
  \node [font=\ttfamily\small, black, below] at (bip.-90)
    {\detokenize\expandafter{#1}};
\end{circuitikz}}
\ctikzset{bipoles/border margin=1}
\showbordersfornode{generic} \showbordersfornode{resistor}
\showbordersfornode{fulldiode} \showbordersfornode{vsource}
\showbordersfornode{capacitivesens}
```

3.1.3 Relative coordinates

As noticed by user [septatrix](#), although full relative coordinates after a component work as expected when using `++(x,y)`-style coordinates, often there are problems when using the `+(x,y)`-style coordinates (which are supposed to set a temporary relative coordinate and then going back to the starting point).

These kind of coordinate have in practice little use for the building of circuits, so have been only (very) lightly tested; avoid them if you can — the behavior will depend not only on the CircuiTikZ version, but also on the TikZ layer underneath.

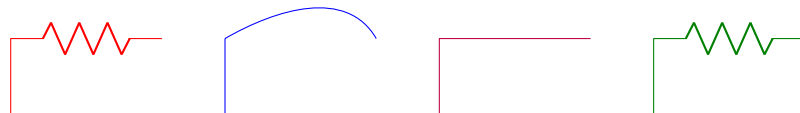
This behavior, although not optimal, was standard in to operation in plain TikZ before version 3.1.8; it was changed by Henri Menke in later versions. Notice that the change revealed a problem in CircuiTikZ that should hopefully be fixed in v1.4.1; for more details see [this issue on GitHub](#).

You can see from the example below (notice the blue curve using a spline line). If all the vertical lines are at the left, the manual has been compiled with a new CircuiTikZ and TikZ. Otherwise, the red and/or blue curve will have the vertical line at the right (which in principle is wrong).

⁹This is needed for the correct label/voltage etc. placement, and it's too much work to change it.

In the last (green) example, you can see a workaround using local path and the key `current point is local` that will work for older (and do not create problems in newer) versions.

Plotted using TikZ version 3.1.10 and CircuiTikZ version 1.8.7-unreleased-54e8a88.



Plotted using `\TikZ\ version \pgfversion{}` and `CircuiTikZ\ version \pgfcircversion{}`.

```
\begin{tikzpicture}
  \draw[color=red] (0,0) to[R] +(2,0) +(0,0) -- ++(0,-1);
\end{tikzpicture}
\quad
\begin{tikzpicture}
  \draw[color=blue] (0,0) to[out=30, in=120] +(2,0) +(0,0) -- ++(0,-1);
\end{tikzpicture}
\quad
\begin{tikzpicture}
  \draw[color=purple] (0,0) to[] +(2,0) +(0,0) -- ++(0,-1);
\end{tikzpicture}
\quad
\begin{tikzpicture}
  \draw[color=green!50!black] (0,0)
    {[current point is local] to[R] +(2,0)} +(0,0) -- ++(0,-1);
\end{tikzpicture}
```

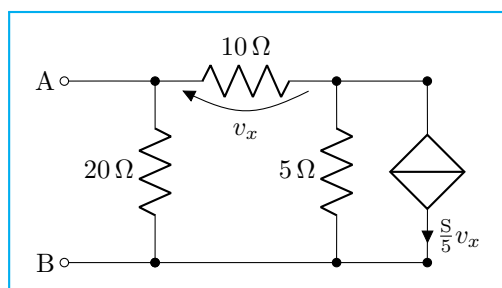
3.1.4 Customization

Pretty much all CircuiTikZ relies heavily on `pgfkeys` for value handling and configuration. Indeed, at the beginning of `circuitikz.sty` and in the file `pgfcirc.define.tex` a series of key definitions can be found that modify all the graphical characteristics of the package.

All can be varied using the `\ctikzset` command, anywhere in the code.

Note that the details of the parameters that are not described in the manual can change in the future, so be ready to use a fixed version of the package (the ones with the specific number, like `circuitikz-0.9.3`) if you dig into them.

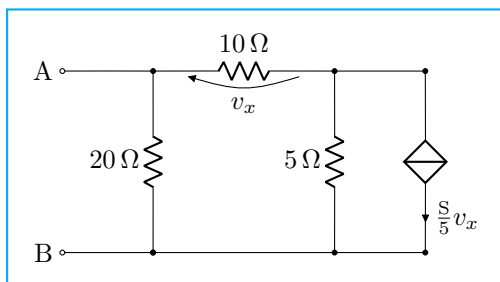
3.1.4.1 Components size Perhaps the most important parameter is `bipoles/length` (default 1.4cm, you can use the currently set value with `\ctikzvalof{bipoles/length}`), which can be interpreted as the length of a resistor (including reasonable connections): all other lengths are relative to this value. For instance:



```

\tikzset{bipoles/length=1.4cm}
\begin{circuitikz}[scale=1.2]\draw
  (0,0) node[anchor=east] {B}
    to[short, o-*] (1,0)
    to[R=20<\ohm>, *-] (1,2)
    to[R=10<\ohm>, v=$v_x$] (3,2) -- (4,2)
    to[cI=$\frac{\si{\siemens}}{5}$ v_x$, *-] (4,0) -- (3,0)
    to[R=5<\ohm>, *-] (3,2)
  (3,0) -- (1,0)
  (1,2) to[short, -o] (0,2) node[anchor=east]{A}
;\end{circuitikz}

```

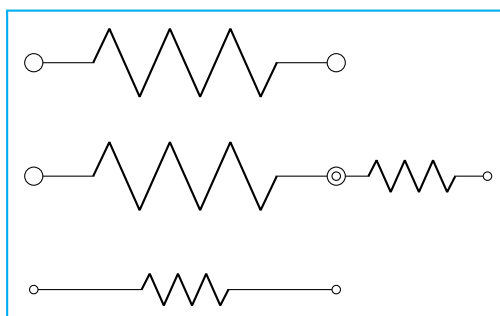


```

\tikzset{bipoles/length=.8cm}
\begin{circuitikz}[scale=1.2]\draw
  (0,0) node[anchor=east] {B}
    to[short, o-*] (1,0)
    to[R=20<\ohm>, *-] (1,2)
    to[R=10<\ohm>, v=$v_x$] (3,2) -- (4,2)
    to[cI=$\frac{\si{\siemens}}{5}$ v_x$, *-] (4,0) -- (3,0)
    to[R=5<\ohm>, *-] (3,2)
  (3,0) -- (1,0)
  (1,2) to[short, -o] (0,2) node[anchor=east]{A}
;\end{circuitikz}

```

The changes on `bipoles/length` should, however, be globally applied to every path, because they affect every element — including the poles. So you can have artifacts like the one in the second line below:

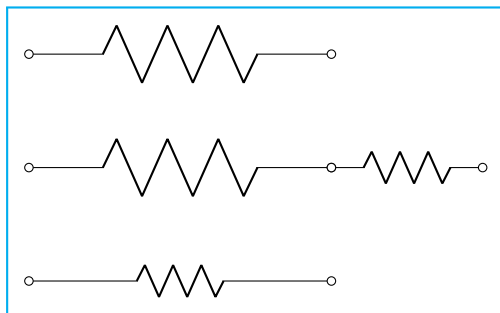


```

\begin{circuitikz}[
  bigR/.style={R, bipoles/length=3cm}
]
\draw (0,3) to [bigR, o-o] ++(4,0);
\draw (0,1.5) to [bigR, o-o] ++(4,0)
  to[R, o-o] ++(2,0); % will fail here
\draw (0,0) to [R, o-o] ++(4,0);
\end{circuitikz}

```

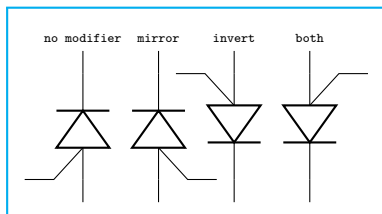
Several groups of components, on the other hand, have a special `scale` parameter that can be used safely in this case (starting with 0.9.4 — more groups of components will be added going forward); the key to use will be explained in the specific description of the components. For example, in the case of resistors you have `resistors/scale` available:



```
\begin{circuitikz}[
  bigR/.style={R, resistors/scale=1.8}
]
\draw (0,3) to [bigR, o-o] ++(4,0);
\draw (0,1.5) to [bigR, o-o] ++(4,0)
  to [R, o-o] ++(2,0); % ok now
\draw (0,0) to [R, o-o] ++(4,0);
\end{circuitikz}
```

Never use `scale`, `xscale` or `yscale` in a path-style component (i.e., inside a `to[...]` command).

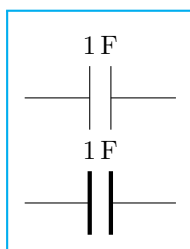
3.1.4.2 Mirroring and flipping path-style components To change the orientation of path-style components, *never* use `xscale=-1` nor `yscale=-1`. That will mess up the path completely. Use the `mirror` and `invert` options:



```
\begin{tikzpicture}[N/.style={
  font=\tiny\ttfamily, above}]
\draw (0,0) to [put] ++(0,2)
  node[N]{no modifier};
\draw (1,0) to [put, mirror] ++(0,2)
  node[N]{mirror};
\draw (2,0) to [put, invert] ++(0,2)
  node[N]{invert};
\draw (3,0) to [put, mirror, invert] ++(0,2)
  node[N]{both};
\end{tikzpicture}
```

3.1.4.3 Thickness of the lines (globally)

The best way to alter the thickness of components is using styling, see section 3.3.3. Alternatively, you can use “legacy” classes like `bipole`, `tripoles` and so on — for example changing the parameter `bipoles/thickness` (default 2). The number is relative to the thickness of the normal lines leading to the component.

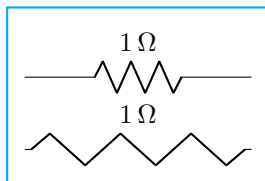


```
\ctikzset{bipoles/thickness=1}
\tikz \draw (0,0) to [C=1<\farad>] (2,0); \par
\ctikzset{bipoles/thickness=4}
\tikz \draw (0,0) to [C=1<\farad>] (2,0);
```

3.1.4.4 Shape of the components (on a per-component-class basis)

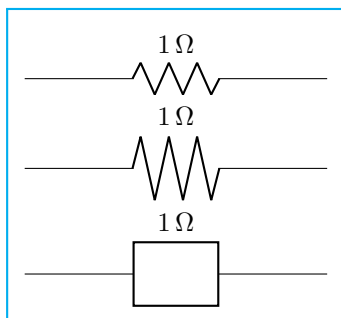
The shape of the components are adjustable with a lot of parameters; in this manual we will comment the main ones, but you can look into the source files specified above to find more. Notice however that the “internal” parameters, the ones not commented in this manual, are not part of the public interface so they can disappear or change in future versions.

It is recommended to use the styling parameters to change the shapes; they are not so fine-grained (for example, you can change the width of resistor), but they are more stable and coherent across your circuit.



```
\tikz \draw (0,0) to[R=1<\ohm>] (3,0); \par
\ctikzset{resistors/width=2}
\tikz \draw (0,0) to[R=1<\ohm>] (3,0);
```

To change the height, you can use (locally) the class scale parameter and the width; you can even define a style that will work across the resistor styles:



```
\ctikzset{american}
\tikz \draw (0,0) to[R=1<\ohm>] (4,0);

\ctikzset{tallR/.style={
    resistors/scale=2, resistors/width=0.4}}
\tikz \draw (0,0) to[R=1<\ohm>, tallR] (4,0);

\ctikzset{european}
\tikz \draw (0,0) to[R=1<\ohm>, tallR] (4,0);
```

3.1.5 Descriptions

The typical entry in the component list will be like this:

	resistor: resistor, american style, type: path-style, nodename: resistorshape. Aliases: R, american resistor. Class: resistors.
	pR: potentiometer, american style, type: path-style, nodename: potentiometershape. Aliases: pR, american potentiometer. Class: resistors.

where you have all the needed information about the bipole, with also no-standard anchors. If the component can be filled it will be specified in the description. In addition, as an example, the component shown will be filled with the option `fill=cyan!30!white`:

	ammeter: Ammeter, type: path-style, fillable, nodename: ammetershape. Class: instruments.
--	--

The *Class* of the component (see section 3.3) is printed at the end of the description.

Most path-style components can be used as a node-style components; to access them, normally you add a **shape** to the main name of component (for example, **diodeshape**). Sometimes though the “node name” is different, so it is specified in the description of each component.

3.2 Node-style components

Node-style components (monopoles, multipoles) can be drawn at a specified point with this syntax, where **#1** is the name of the component:

```
\begin{circuitikz}
  \draw (0,0) node[#1,#2] (#3) {#4};
\end{circuitikz}
```

Explanation of the parameters:

- #1**: component name¹⁰ (mandatory)
- #2**: list of comma-separated options (optional)
- #3**: name of an anchor (optional)
- #4**: text written to the text anchor of the component (optional)

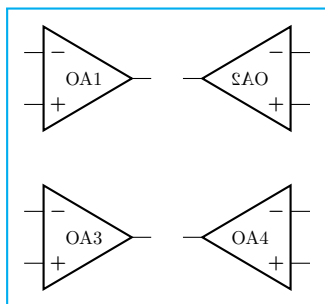
Notice: Nodes must have curly brackets at the end, even when empty. An optional anchor (**#3**) can be defined within round brackets to be addressed again later on. And please don't forget the semicolon to terminate the `\draw` command.

Also notice: If using the `\tikzexternalize` feature, as of TikZ 2.1 all pictures must end with `\end{tikzpicture}`. Thus you *cannot* use the `circuitikz` environment.

Which is OK: just use the environment `tikzpicture`: everything will work there just fine.

3.2.1 Mirroring and flipping

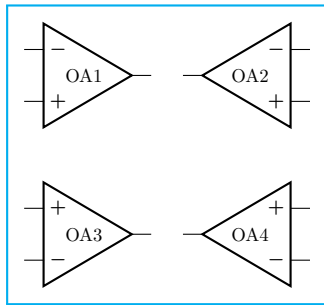
Mirroring and flipping of node components is obtained by using the TikZ keys `yscale` and `yscale`. Notice that these parameters also affect text labels, so they need to be un-scaled by hand. Notice that you **do not** use `yscale` or `yscale` in a path-style component, see section 3.1.4.2 for that case.



```
\begin{circuitikz}[scale=0.7, transform shape]
  \draw (0,3) node[op amp]{OA1};
  \draw (3,3) node[op amp, xscale=-1]{OA2};
  \draw (0,0) node[op amp]{OA3};
  \draw (3,0) node[op amp, xscale=-1]{%
    \scalebox{-1}[1]{OA4}};
\end{circuitikz}
```

To simplify this task, CircuiTikZ has three helper macros —`\ctikzflipx{}`, `\ctikzflipy{}`, and `\ctikzflipxy{}`, that can be used to “un-rotate” the text of nodes drawn with, respectively, `yscale=-1`, `yscale=-1`, and `scale=-1` (which is equivalent to `yscale=-1`, `yscale=-1`).

¹⁰For using bipoles as nodes, the name of the node is **#1shape**.



```
\begin{circuitikz}[scale=0.7, transform shape]
  \draw (0,3) node[op amp]{OA1};
  \draw (3,3) node[op amp, xscale=-1]{\ctikzflipx{OA2}};
  \draw (0,0) node[op amp, yscale=-1]{\ctikzflipy{OA3}};
  \draw (3,0) node[op amp, scale=-1]{\ctikzflipxy{OA4}};
\end{circuitikz}
```

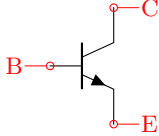
3.2.2 Anchors

Node components anchors vary a lot across the various kinds of components, so they will be described better after each category is presented in the manual. In general all components have geographic anchors (**north**, **north west**, ...), but most of the other anchors are very component-specific.

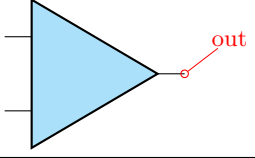
3.2.2.1 Text anchors Starting from version v1.8.2, you can always refer to the node text of a component using the anchor **.text** which will point to the baseline of the left edge of the typeset text. Using these anchors is considered an advanced topic, and should be avoided in the normal usage of the package.

3.2.3 Descriptions

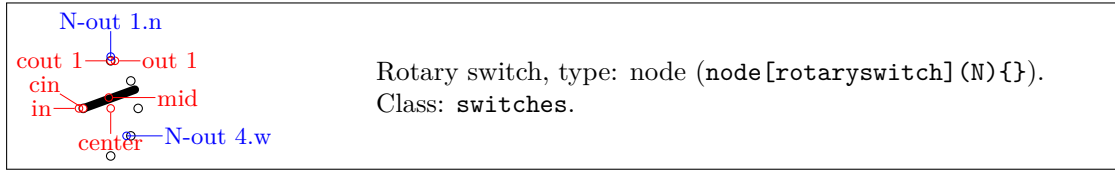
The typical entry in the component list will be like this:

	Cute spdt down with arrow, type: node (node[cute spdt down arrow]{}). Class: switches .
	NPN, TYPE: NODE (node[npn]{}). Class: transistors .

If the component can be filled it will be specified in the description. In addition, as an example, the component shown will be filled with the option **fill=cyan!30!white**:

	Plain amplifier, type: node, fillable (node[plain amp]{}). Class: amplifiers .
---	---

Sometime, components will expose internal (sub-)shapes that can be accessed with the syntax **<node name>-<internal node name>** (a dash is separating the node name and the internal node name); that will be shown in the description as a blue “anchor”:



The *Class* of the component (see section 3.3) is printed at the end of the description.

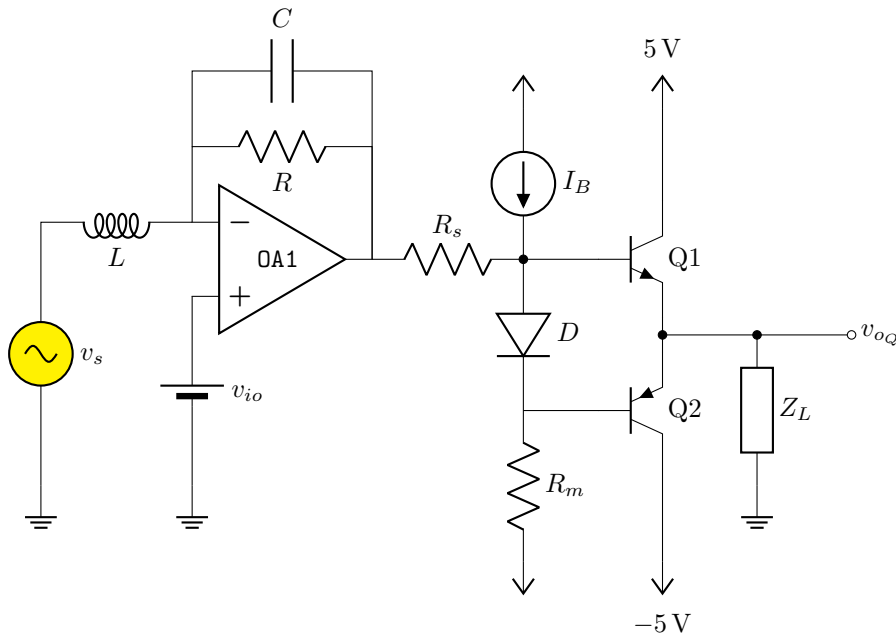
3.3 Styling circuits and components

You can change the visual appearance of a circuit by using a circuit style different from the default. For styling the circuit, the concept of *class* of a component is key: almost every component has a class, and a style change will affect all the components of that class.

Let's see the effect over a simple circuit¹¹.

```
\begin{circuitikz}[american, cute inductors]
  \node [op amp] (A1){\texttt{OA1}};
  \draw (A1.-) to[short] ++(0,1) coordinate(tmp) to[R, l_=$R$] (tmp -| A1.out) to[short] (A1.out);
  \draw (tmp) to[short] ++(0,1) coordinate(tmp) to[C=$C$] (tmp -| A1.out) to[short] (A1.out);
  \draw (A1.+) to [battery2, invert] ++(0,-2.5) node[ground](GND){};
  \draw (A1.-) to [L=$L$] ++(-2,0) coordinate(tmp) to[sV, l=$v_s$, fill=yellow] (tmp |-GND) node[ground]{};
  \draw (A1.out) to[R=$R_s$] ++(2,0) coordinate(bb) to[I, l_=$I_B$, invert] ++(0,2) node[vcc](VCC){};
  \draw (bb) to[D, l=$D$, *-] ++(0,-2) coordinate(bb1) to[R=$R_m$] ++(0,-2) node[vee](VEE){};
  \draw (bb) ---++(1,0) node[npn, anchor=B](Q1){Q1};
  \draw (bb1) ---++(1,0) node[pnp, anchor=B](Q2){Q2};
  \draw (Q1.E) -- (Q2.E) ($ (Q1.E)!0.5!(Q2.E)$) to [short, *-o, name=S] ++(2.5,0)
  node[right]{$v_{o_Q}$};
  \draw (S.s) to[european resistor, l=$Z_L$, *-] (S.s|-GND) node[ground]{};
  \draw (Q1.C) -- (Q1.C|-VCC) node[vcc]{\SI{5}{V}};
  \draw (Q2.C) -- (Q2.C|-VEE) node[vee]{\SI{-5}{V}};
\end{circuitikz}
```

This code, with the default parameters, will render like the following image.

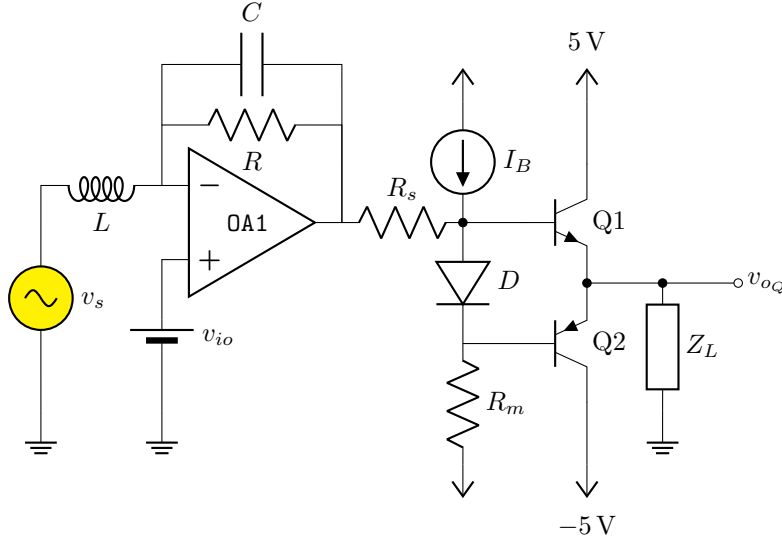


¹¹This is a just an example, the circuit is not intended to be functional.

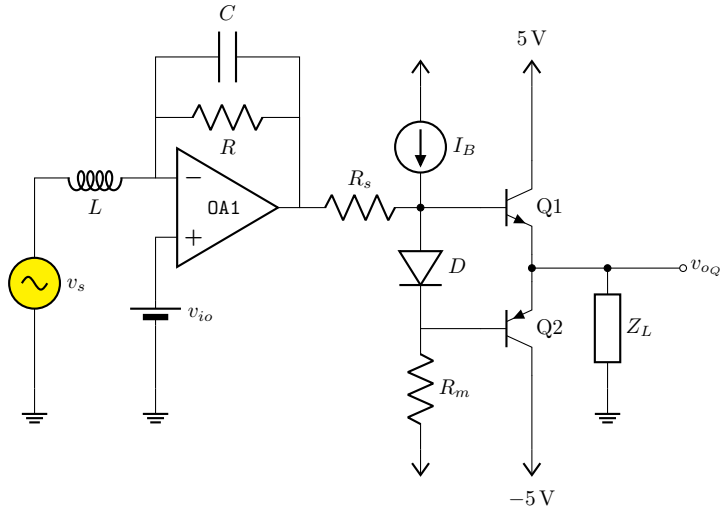
3.3.1 Relative size

Component size can be changed globally (see section 3.1.4.1), or you can change their relative size by scaling a family of components by setting the key `class/scale`; for example, you can change the size of all the diodes in your circuit by setting `diodes/scale` to something different from the default 1.0.

Remember that if you use a global scale (be sure to read section 1.8!) you change the coordinate only, so using `scale=0.8` in the environment options you have:



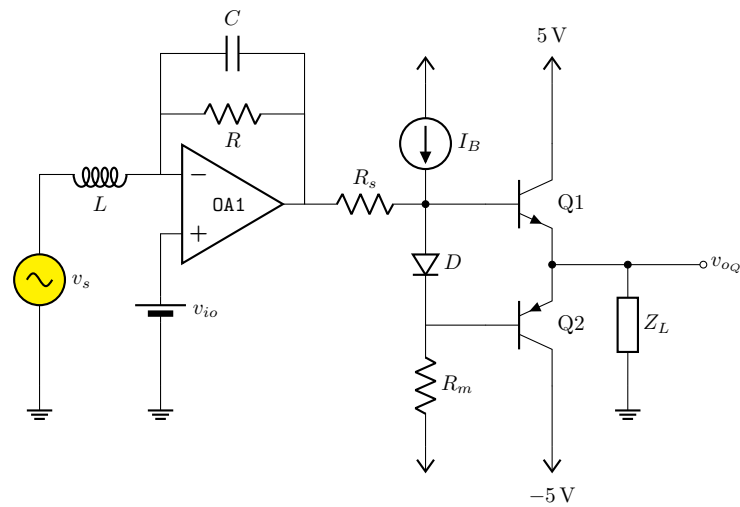
If you want to scale all the circuit, you have to use also `transform shape`:



Using relative sizes as described in section 3.1.4.1 enables your style for the circuit. For example, setting:

```
\ctikzset{resistors/scale=0.8,      % smaller R
          capacitors/scale=0.7,     % even smaller C
          diodes/scale=0.6,         % small diodes
          transistors/scale=1.3}    % bigger BJTs
```

Will result in a (much more readable in Romano's opinion) circuit:



Warning: relative scaling is meant to work for a reasonable range of stretching and shortening, so try to keep your scale parameter in the 0.5 to 2.0 range (more or less). Bigger or smaller value can result in awkward shapes.

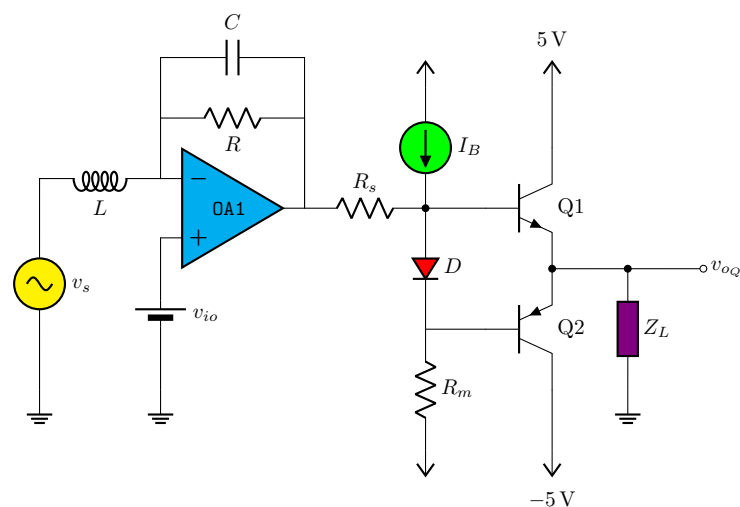
3.3.2 Fill color

You can also set a default fill color for the components. You can use the keys `class/fill` (which defaults to `none`, no fill, i.e. transparent component) for all fillable components in the library.

If you add to the previous styles the following commands:

```
\ctikzset{
  amplifiers/fill=cyan,
  sources/fill=green,
  diodes/fill=red,
  resistors/fill=violet,
}
```

you will have the following circuit (note that the first generator is *explicitly* set to be yellow, so if will not be colored green!):



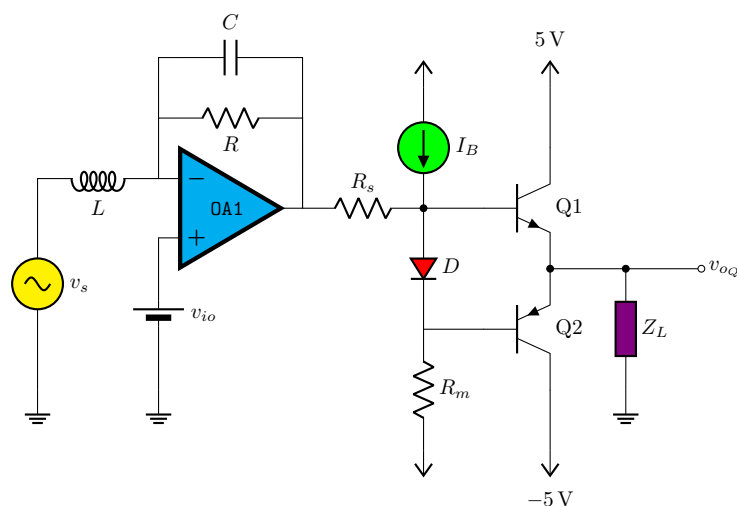
Please use this option with caution. Although two-colors circuits can be nice, using more than that can become rapidly unbearable. Old textbooks used the two-color style quite extensively, filling with a kind of light blue like `blue!30!white` “closed” components, but that was largely to hinder black-and-white photocopying...

3.3.3 Line thickness

You can change the line thickness for any class of component in an independent way. The default standard thickness of components is defined on a loose “legacy” category (like `bipoles`, `tripoles` and so on, see section 3.1.4.3); to override that you set the key `class/thickness` to any number. The default is `none`, which means that the old way of selecting thickness is used.

For example, *amplifiers* have the legacy class of `tripoles`, as well as transistors and tubes. By default they are drawn with thickness 2 (relative to the base linewidth). To change them to be thicker, you can for example add to the previous style

```
\ctikzset{amplifier/thickness=4}
```



Caveat: not every component has a “class”, so you have to play with the available ones (it’s specified in the component description) and with the absolute values to have the circuit following your taste. A bit of experimentation will create a kind of *style options* that you could use in all your documents.

3.3.4 Style files

When using styles, it is possible to use *style files* (see section 3.3.5), that then you can load with the command `\ctikzloadstyle`. For example, in the distribution you have a number of style files: `legacy`, `romano`, `example`. When you load a style name *name*, you will have available a style called *name* circuit style that you can apply to your circuits. The last style loaded is not enacted — you have to explicitly do it if you want the style used by default, by putting for example in the preamble:

```
\ctikzloadstyle{romano}
\tikzset{romano circuit style}
```

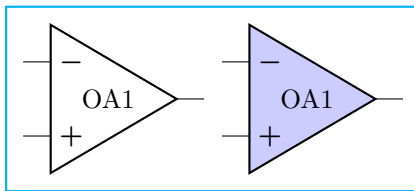
Please notice that the style is at TikZ level, not CircuiTikZ — that let’s you use it in the top option of the circuit, like:

```
\begin{circuitikz}[legacy circuit style,
..., ]
...
\end{circuitikz}
```

If you just want to use one style, you can load and activate it in one command with

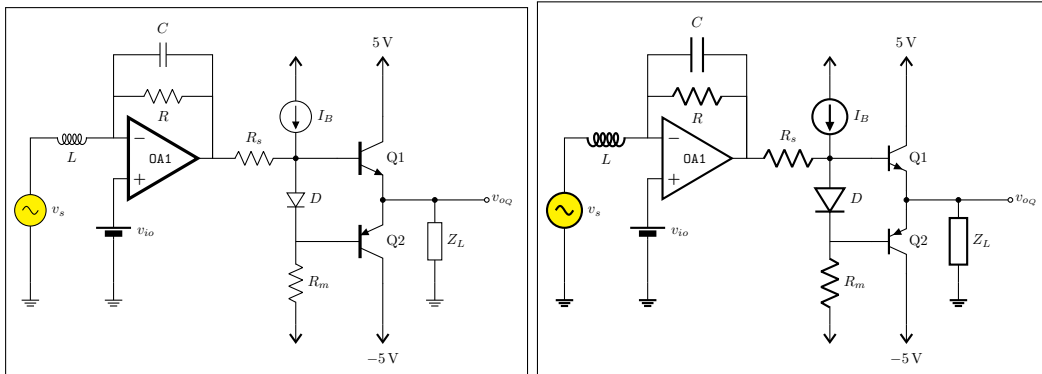
```
\ctikzsetstyle{romano}
```

The **example** style file will simply make the amplifiers filled with light blue:



```
\begin{circuitikz}
\draw (0,0) node[op amp]{OA1};
\end{circuitikz}
\ctikzloadstyle{example}
\begin{circuitikz}[example circuit style]
\draw (0,0) node[op amp]{OA1};
\end{circuitikz}
```

The style **legacy** is a style that set (most) of the style parameters to the default, and **romano** is a style used by one of the authors; you can use these styles as is or you can use them to learn to how to write new file style following the instructions in section 3.3.5. In the next diagrams, the left hand one is using the **romano** circuit style and the right hand one the legacy style.



3.3.5 Style files: how to write them

The best option is to start from `ctikzstyle-legacy.tex` and edit your style file from it. Then you just put it in your input path and that's all. If you want, you can contribute your style file to the project.

Basically, to write the style **example**, you edit a file named `ctikzstyle-romano.tex` with will define and enact TikZ style with name **example circuit style**; basically it has to be something along this:

```
% example style for circuits
% Do not use LaTeX commands if you want it to be compatible with ConTeXt
% Do not add spurious spaces
\tikzset{example circuit style/.style={%
\circuitikzbasekey/.cd,%
amplifiers/fill=blue!20!white,
},% end .style
```



```
}% end \tikzset
%
\endinput
```

This kind of style will *add* to the existing style. If you want to have a style that *substitutes* the current style, you should do like this:

```
\ctikzloadstyle{legacy}% start from a known state
\tikzset{romano circuit style/.style={%
    legacy circuit style, % load the legacy style
    \circuitikzbasekey/.cd,%
    % Resistors
    resistors/scale=0.8,
    [...]
}}
```

If you want to add a setting to your style file that has been recently added to the package (for example, the thyristor compact shapes added in 1.3.5), but you want your style file to be still compatible with older versions of CircuiTikZ, you can use the `.try` statement:

```
% Diodes
diodes/scale=0.6,
diodes/thickness=1.0,
thyristor style/.try=compact,
```

Or, in case of new values of existing “choice” keys, you can use the syntax:

```
% Logic ports
logic ports/ieee/.try,
% this way of setting the key does nothing if ieee option
% does not exist; logic ports/.try=ieee does not work
% if the key exists but the value is not defined
logic ports/scale=1.0,
```

3.4 Subcircuits

Starting from version 1.3.5, there is support for generating sub-circuits, or circuit blocks. The creation and use of subcircuits is somewhat limited, to keep them simple and easy to define and maintain.

A subcircuit is basically a path (and just one path!) of generic TikZ instructions, with a series of accessible coordinates that behave more or less like anchors in the “real” shapes. The basic limitation is that a subcircuit can be moved, replicated and placed around but it can’t be easily personalized. Even if scaling and rotation is in principle possible, it is not easily done. Nevertheless, they can be quite useful to build complex components and reusable blocks.

3.4.1 Subcircuit definition

To define a block you use the `\ctikzsubcircuitdef` macro; this macro has 3 arguments:

- the first argument is the name of the subcircuit, and it must form a valid TeX command name when prepended with a backslash: so just letters (no spaces, nor numbers, nor symbols like underscores, etc.);
- the second one is a comma-separated list of anchor names; here you can use whatever you can use for naming a coordinate or a node (so it's much more relaxed than the first one);
- finally, the commands that will draw the circuit. You must suppose you are in a `\draw` command, with the start coordinate already set-up. You can (and should) use `#1` as the name of the current node, and you *must* define the coordinates of all the anchors listed before as `coordinate(#1-anchorname)`. You should **not** finish the path here and use **only relative coordinates** or **named ones**.

Let's see that with an example:

```
\ctikzsubcircuitdef{optovishay}{in 1, out 1, in 2, out 2, center}{%
  % reference anchor is -center
  coordinate(#1-center)
  (#1-center) +(-1.2,-1) rectangle +(1.2,1)
  (#1-center) ++(-1.2,0.8) coordinate (#1-in 1)
  (#1-center) ++(-1.2,-0.8) coordinate (#1-in 2)
  (#1-center) ++(1.2,0.8) coordinate (#1-out 1)
  (#1-center) ++(1.2,-0.8) coordinate (#1-out 2)
  (#1-center) ++(0,1) coordinate (#1-up)
  (#1-in 1) -- ++(0.5,0) coordinate(#1-tmp)
    to[leD*, diodes/scale=0.6, led arrows from cathode]
    (#1-tmp|- #1-in 2) -- (#1-in 2)
  (#1-out 1) -- ++(-0.5,0) coordinate(#1-tmp)
    to[pD*, diodes/scale=0.4, mirror] ++(0,-0.5)
    edge[densely dashed] ++(0,-0.533) ++(0,-0.566)
    to[pD*, diodes/scale=0.4,mirror] (#1-tmp|- #1-out 2) -- (#1-out 2)
  % leave the position of the path at the center
  (#1-center)
}
```

Our element is a symbol for an optocoupler; in this case is the symbol used for one cell of the double [Vishay vo1263 device](#).

The name of the subcircuit is `optovishay` — notice we can use only letters here, upper or lowercase, and nothing more. Then we have a series of anchor names; here we can use letters, numbers, spaces and some symbol — but avoid the dot (.) and the hyphen (-). Additionally, the anchor named `subckt@reference` is reserved and shouldn't be used. If you use spaces, be on the safe side and *never* use two or more consecutive spaces.

After that, you have to draw your subcircuit as if you were into a `\draw` command, starting from a generic point. In this case, we decide to draw the circuit around this generic point so that it will result to be the center of the block; so as a first thing, we “mark” the position of the center anchor, with `coordinate(#1-center)`. The `#1` will be substituted with the specific name of the subcircuit's instance later — so if you then call one instance of the optocoupler `opto1`, that coordinate will be called `opto1-center`.

We continue by defining all our anchors (there is no need to do that at the start, but it's handy because then you can use them). You **must** define all the anchors!

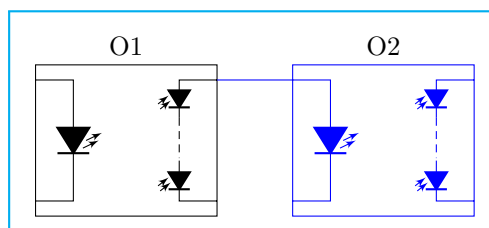
Important: all the coordinates used must be either relative, or named in the form **#1-something**; absolute coordinate will not work when instantiating the block. The block is thought to be used inside a path specification, so the idea is not to end the path — that means that changing line styles or colors is at best difficult. You can still use **edges**, though (see 8.2).

After that, we draw our circuit; in this case a LED and a couple of smaller photodiodes will do. We also define a coordinate **-up** (you can define more coordinates, in addition to the anchors, or name elements with **name=#1-something** for later access) for adding text.

3.4.2 Using the subcircuit

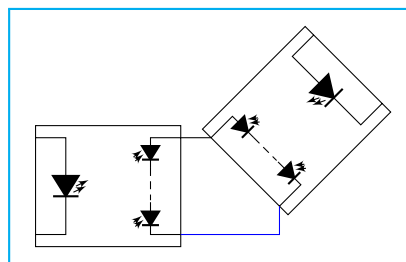
To use the subcircuit, an additional step is needed. Somewhere you have to *activate* it. This is needed to calculate the relative positions of anchors using the current set of style parameters. The normal place is to activate it just before usage; to do that you use the command `\ctikzsubcircuitactivate` with the name of the subcircuit. That will define a new command, `\nameofthesubcircuit` that you can use then in your paths.

So to check your subcircuit while defining it you can use this simple snippet:



```
\ctikzsubcircuitactivate{optovishay}
\begin{tikzpicture}
  \draw (0,0) \optovishay{one}{};
  \node [above] at (one-up) {O1};
  \draw[blue] (one-out 1) -- ++(1,0)
    \optovishay{two}{in 1};
  \node [above] at (two-up) {O2};
\end{tikzpicture}
```

3.4.2.1 Scaling, flipping and rotating subcircuits To scale and rotate a subcircuit you have to include it into a **scope** with the appropriate **scale** and rotation commands. Notice that, as in general in CircuiTiKZ, global scales that affect rotation works only if **transform shape** is issued (see 1.8); nesting **transform shape** normally works, but it has been really lightly tested.



```
\ctikzsubcircuitactivate{optovishay}
\begin{tikzpicture}[scale=0.8, transform shape]
  \draw (0,0) \optovishay{three}{};
  \draw (three-out 1) -- ++(0.5,0) coordinate(here);
  \begin{scope}[xscale=-1,rotate=-45,transform shape]
    \draw (here) \optovishay{four}{out 1};
  \end{scope}
  \draw[blue] (three-out 2) -| (four-out 2);
\end{tikzpicture}
```

3.4.3 Parameters in subcircuits

There are no additional parameters definable for subcircuit shapes; this is a bit of a pity, because sometimes they could be useful, especially for labels of objects. Given the need to use **transform shape** to translate and rotate them, though, it is better not to add invariant-direction things (like text) into the subcircuit, unless you are sure you will just translate them. One possibility is to use additional macros and anchors for positioning, like in the following example.

Suppose you have defined

```

\ctikzsubcircuitdef{divider}{in, out}{%
  coordinate (#1-in) to[R, l=~, name=#1-rh, -*] ++(2,0)
  coordinate(#1-tmp) to[R, l=~, name=#1-r1] ++(0,-2)
  node[tlground]{} (#1-tmp) --++(0.5,0) coordinate(#1-out)
}

```

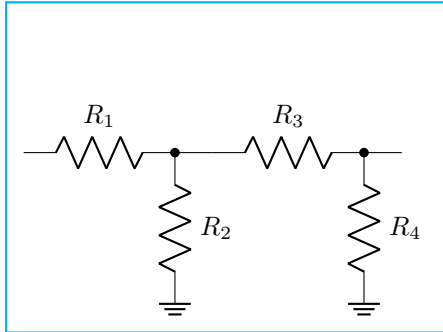
then you can additionally define:

```

\newcommand{\mydiv}[4]{
  \divider{#1}{#2} (#1-rh.n) node[above]{#3}
  (#1-r1.n) node[right]{#4} (#1-out)
}

```

And finally do:



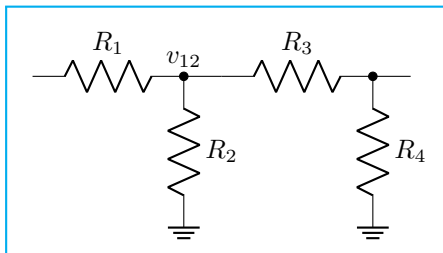
```

\begin{tikzpicture}
\ctikzsubcircuitactivate{divider}
\draw (0,0) \mydiv{a}{in}{R_1}{R_2};
\draw (a-out) -- \mydiv{b}{in}{R_3}{R_4};
\end{tikzpicture}

```

3.4.4 Using pics (with parameters)

Since v1.8.0¹² you can also use the standard TikZ pics as subcircuits, with the advantage to be able to define (up to 9) parameters.



```

\tikzset{pics/divider/.style n args={2}{
  code={
    \draw (0,0) coordinate(-in) to[R=#1, -*]
    ++(2,0)
    coordinate(-corner)
    to[R=#2] ++(0,-2) node[tlground]{}
    (-corner) -- ++(0.5,0) coordinate(-out);
  }}
\begin{tikzpicture}[]
\draw (0,0) pic(one){divider={R_1}{R_2}}
(one-out) pic(two){divider={R_3}{R_4}};
\node [above] at (one-corner) {$v_{12}$};
\end{tikzpicture}

```

The main limitation of `pics` is that natively they do not have options to place them with internal anchors, like the `anchor=...` options of nodes. Fortunately, Andrew Stacey's fine [tikzmark library](#) provides that function. You have to just add `\usetikzlibrary{tikzmark}` in your preamble, and then add the option to the pic invocation:

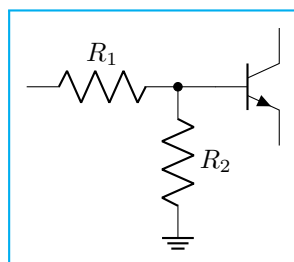
```

... pic[pic anchor=(-corner)] {divider=...}

```

¹²See [this issue](#) for more details; be warned that they will **not** work at all in earlier versions.

For example, suppose you want to attach your divider to a BJT base:



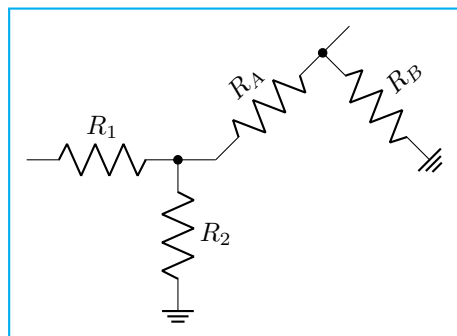
```
\begin{tikzpicture}[]
  %% \usetikzlibrary{tikzmark}
  \draw (0,0) node[npn](Q){}
    (Q.B) pic[pic anchor=(-out)]{divider={R_1}{R_2}};
\end{tikzpicture}
```

Notice that `pic anchor` needs an additional compilation pass to position the subcircuit correctly. This double pass is usually not a problem, but be prepared to have the pic positioned incorrectly after the first run. For the curious, this is needed because before drawing the pic, ‘tikzmark’ does not know where the anchors are. So, it saves their positions and adjusts them in the next run. With native subcircuits, the call to `\ctikzsubcircuitactivate` is doing the same for every possible anchor *before* using it.¹³

In some extreme conditions, when the `pic` is complex and there are many calculations (especially true in case of rotations), it could happen that the two-pass compilation can become unstable: that is, every time a slightly, invisible difference in coordinates is written out. That can confuse automatic compilations tools, like for example `latexmk`, and induce a very high number of recompilations or even errors. With a recent enough `tikzmark`¹⁴ you can avoid this by rounding the precision of the coordinate calculations to, say, 2 decimal places (which is 0.01 pt, basically invisible in normal conditions) with the option:

```
... pic[tikzmark round=2, rotate=45, pic anchor=(-corner)] {divider=...}
```

Finally, if you want to rotate the pic (warning: lightly tested!) remember that you need `transform shape` to have a correct behavior.



```
\begin{tikzpicture}[]
  \draw (0,0)
    pic(one){divider={R_1}{R_2}} (one-out)
    pic[rotate=45, transform shape](two)
      {divider={R_A}{R_B}};
\end{tikzpicture}
```

3.4.5 Using macros

Sometimes, a good old $\text{\LaTeX}$ macro can solve a problem instead of designing a new component. For example, GitHub’s user Patrick Jansky asked for a generic “chopper” block;¹⁵ Unfortunately, this kind of block is quite difficult to implement as a node: it should change size and orientation on the base of the “port” terminals you need.

¹³Native shapes do not need that because you must define the anchor positions explicitly when defining the shape.

¹⁴The version must be **greater** than `v1.15`, not yet released on May 30, 2025: see [this GitHub issue for details](#); use a development version if you need it — the instructions are on the same GitHub project.

¹⁵See [the original issue on GitHub](#).

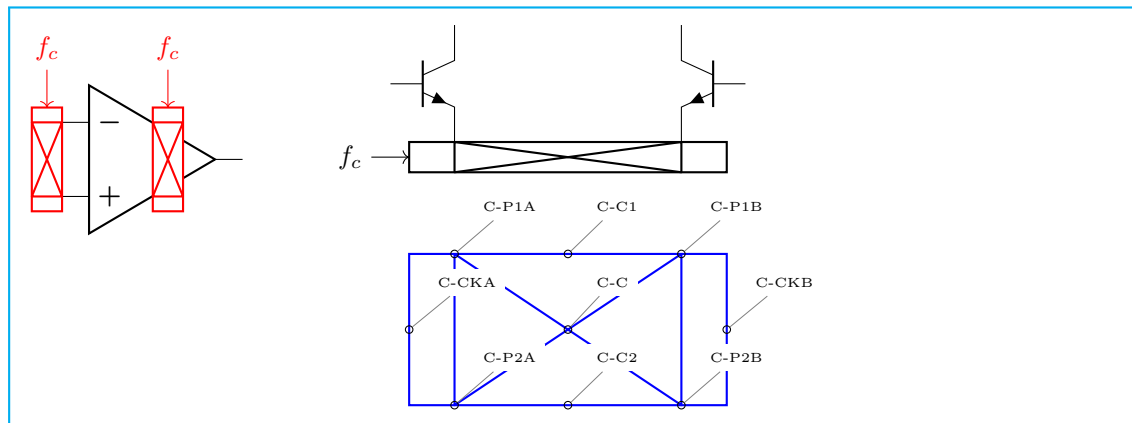
In this case, probably an old-fashioned macro, using partway coordinate specifiers,¹⁶ could be the best option for this task. Below, one possible example (not optimized at all!).

```
\NewDocumentCommand{\ChopperByTerminals}{0{} 0{0.2cm} m m m}{%
  % #1 -> options to the draw command
  % #2 -> half width
  % by default is drawn to the right (if vertical), to the left if #2 is negative
  % If horizontal, by default is above, below if #2 is negative
  % #3 -> terminal A on port 1
  % #4 -> terminal B on port 2
  % #5 -> name for the "anchors"
  \coordinate(#5-P1A) at (#3);
  \coordinate(#5-P1B) at (#4);
  % the center of the block is midway P1A to P1B, at distance #2 at 90 clockwise
  % degrees from the center of the line
  \coordinate(#5-C1) at ($(#5-P1A)!0.5!(#5-P1B)$);
  \coordinate(#5-C) at ($(#5-C1)!#2!90:(#5-P1B)$);
  \coordinate(#5-P2B) at ($(#5-P1A)!2!(#5-C)$);
  \coordinate(#5-P2A) at ($(#5-P1B)!2!(#5-C)$);
  \coordinate(#5-T1A) at ($(#5-P1B)!1.2!(#5-P1A)$);
  \coordinate(#5-T1B) at ($(#5-P1A)!1.2!(#5-P1B)$);
  \coordinate(#5-T2A) at ($(#5-P2B)!1.2!(#5-P2A)$);
  \coordinate(#5-T2B) at ($(#5-P2A)!1.2!(#5-P2B)$);
  \coordinate(#5-CKA) at ($(#5-T1A)!0.5!(#5-T2A)$);
  \coordinate(#5-CKB) at ($(#5-T1B)!0.5!(#5-T2B)$);
  \coordinate(#5-C2) at ($(#5-P2A)!0.5!(#5-P2B)$);
  \draw [draw, thick, fill=white, #1] (#5-T1A) rectangle (#5-T2B);
  \draw [draw, thick, line join=round, #1] (#5-P1A) -- (#5-P2B) --
    (#5-P1B) -- (#5-P2A) -- (#5-P1A);
}
```

You can use this definition as follows, for example:

```
\begin{tikzpicture}[]
  \node[op amp](A) at (0,0) {};
  \ChopperByTerminals[draw=red] [-0.2cm] {A.-}{A.+}{chopper in}
  \draw [red, <-] (chopper in-CKA) -- ++(0,.5) node[above]{$f_c$};
  \ChopperByTerminals[draw=red] {[xshift=1.2cm]A.-}{[xshift=1.2cm]A.+}{chopper out}
  \draw [red, <-] (chopper out-CKA) -- ++(0,.5) node[above]{$f_c$};
  %Try another one
  \draw (4,1) node[npn](Q1){} (7,1) node[npn, xscale=-1](Q2){};
  \coordinate (Qcenter) at ($ (Q1.E)!0.5!(Q2.E)$ );
  \ChopperByTerminals[] [-0.2cm] {Q1.E}{Q2.E}{cE}
  \draw [<-] (cE-CKA) -- ++(-0.5,0) node[left]{$f_c$};
  % pseudo-anchors
  \ChopperByTerminals[draw=blue] [-1cm] {4,-1.25}{7,-1.25}{C}
  \foreach \a in {P1A, P1B, P2A, P2B, CKA, CKB, C, C1, C2}
    \node[circle, draw, inner sep=1pt, pin={[font=\tiny, fill=white]60:C-\a}]
      at (C-\a) {};
\end{tikzpicture}
```

¹⁶See [TikZ manual section 13.5](#)



4 The components: list

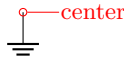
This section is dedicated to the full list of available components.

4.1 Grounds and supply voltages

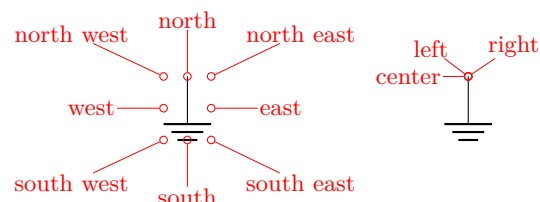
Ground symbols and power supplies — they have two different classes for styling.

4.1.1 Grounds

For the grounds, the **center** anchor is put on the connecting point of the symbol, so that you can use them directly in a **path** specification.

	Ground, type: node (<code>node[ground]{}).</code> Class: grounds .
	Tailless ground, type: node (<code>node[tlground]{}).</code> Class: grounds .
	Reference ground, type: node (<code>node[rground]{}).</code> Class: grounds .
	Signal ground, type: node, fillable (<code>node[sground]{}).</code> Class: grounds .
	Thicker tailless reference ground, type: node (<code>node[tground]{}).</code> Class: grounds .
	Noiseless ground, type: node (<code>node[nground]{}).</code> Class: grounds .
	Protective ground, type: node, fillable (<code>node[pground]{}).</code> Class: grounds .
	Chassis ground ¹⁷ , type: node (<code>node[cground]{}).</code> Class: grounds .
	European style ground, type: node (<code>node[eground]{}).</code> Class: grounds .
	European style ground, version 2 ¹⁸ , type: node (<code>node[eground2]{}).</code> Class: grounds .

4.1.1.1 Grounds anchors Anchors for grounds are a bit strange, given that they have the **center** spot at the same location than **north** and all the ground will develop “going down”:



¹⁷These last three were contributed by Luigi “Liverpool”

¹⁸These last two were contributed by @fotesan

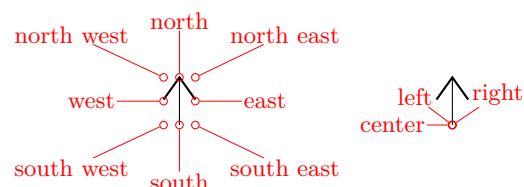
4.1.1.2 Grounds customization You can change the scale of these components (all the ground symbols together) by setting the key `grounds/scale` (default 1.0).

4.1.2 Power supplies

↑	VCC/VDD, type: node (<code>node[vcc]{}</code>). Class: power supplies .
↓	VEE/VSS, type: node (<code>node[vee]{}</code>). Class: power supplies .

The power supplies are normally drawn with the arrows shown in the list above.

4.1.2.1 Power supply anchors They are similar to ground anchors, and the geographical anchors are correct only for the default arrow.



4.1.2.2 Power supplies customization You can change the scale of the power supplies by setting the key `power supplies/scale` (default 1.0).

Given that the power supply symbols are basically arrows, you can change them using all the options of the `arrows.meta` package (see the [TikZ manual](#) for details) by changing the keys `monopoles/vcc/arrow` and `monopoles/vee/arrow` (the default for both is `legacy`, which will use the old code for drawing them). Note that the anchors are at the start of the connecting lines, and that geographical anchors are just approximation if you change the arrow symbol!

```

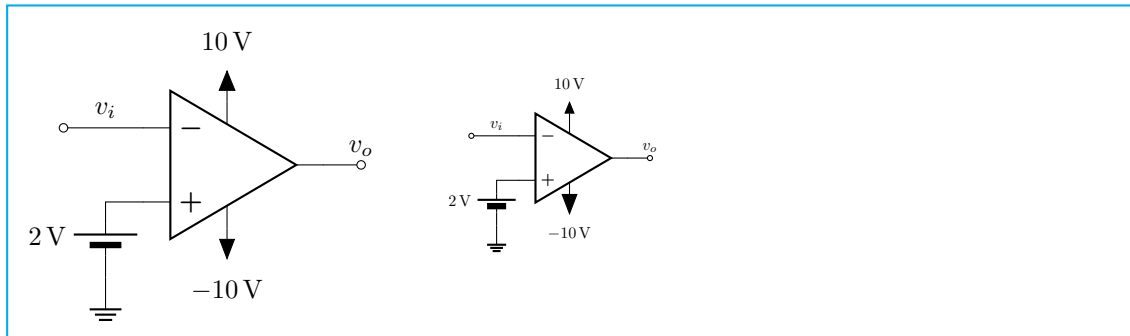
\begin{circuitikz}
  % next macro is available in ctikzmanutils.sty
  \def\coord(#1){\showcoord(#1)<0:0.3>}
  \draw (0,0)
  node[vcc](vcc){VCC} \coord(vcc) ++(2,0)
  node[vee](vee){VEE} \coord(vee);
  \ctikzset{monopoles/vcc/arrow={Stealth[red, width=6pt,
length=9pt]}}
  \ctikzset{monopoles/vee/arrow={Latex[blue]}}
  \draw (0,-2)
  node[vcc](vcc){VCC} \coord(vcc) ++(2,0)
  node[vee](vee){VEE} \coord(vee);
\end{circuitikz}

```

However, arrows in TikZ are in the same class with the line thickness, so they do not scale with neither the class `power supplies` scale nor the global scale parameter (you should use `transform canvas={scale...}` for this).

If you want the arrows to behave like the legacy symbols (which are shapes), *only in the arrow definitions*, you can use the special length parameter `\scaledwidth`¹⁹ in the arrow definition, which correspond to the width of the legacy `vcc` or `vee`. Compare the effects on the following circuit.

¹⁹Thanks to @Schrödinger's cat on [TikZ stackexchange site](#)



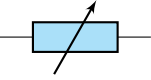
```

\ctikzset{%
  monopoles/vcc/arrow={Triangle[width=0.8*\scaledwidth, length=\scaledwidth]},
  monopoles/vee/arrow={Triangle[width=6pt, length=8pt]},
}
\begin{circuitikz}[baseline=(vo.center)]
  \node [ocirc](TW) at (0,0) {};
  \draw (TW.east) -- ++(1,0) node[midway, above]{$v_i$} node[op amp, anchor=-](A1){};
  \draw (A1.up) -- ++(0, 0.3) node[vcc]{\SI{+10}{V}};
  \draw (A1.down) -- ++(0,-0.3) node[vee]{\SI{-10}{V}};
  \draw (A1.+) -- ++(-0.5,0) to[battery2, invert, l_=\SI{2}{V}] ++(0,-1) node[ground]{};
  \draw (A1.out) to[short, -o] ++(0.5,0) node[above](vo){$v_o$};
\end{circuitikz} \quad \quad \quad
\begin{circuitikz}[baseline=(vo.center), scale=0.6, transform shape]
  \node [ocirc](TW) at (0,0) {};
  \draw (TW.east) -- ++(1,0) node[midway, above]{$v_i$} node[op amp, anchor=-](A1){};
  \draw (A1.up) -- ++(0, 0.3) node[vcc]{\SI{+10}{V}};
  \draw (A1.down) -- ++(0,-0.3) node[vee]{\SI{-10}{V}};
  \draw (A1.+) -- ++(-0.5,0) to[battery2, invert, l_=\SI{2}{V}] ++(0,-1) node[ground]{};
  \draw (A1.out) to[short, -o] ++(0.5,0) node[above](vo){$v_o$};
\end{circuitikz}

```

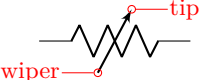
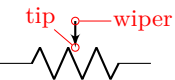
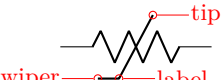
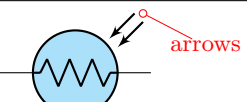
4.2 Resistive bipoles

	short: Short circuit, type: path-style, nodename: shortshape. Class: default.
	open: Open circuit, type: path-style, nodename: openshape. Class: default.
	generic: Generic (symmetric) bipole, type: path-style, fillable, nodename: genericshape. Class: resistors.
	xgeneric: Crossed generic (symmetric) bipole, type: path-style, fillable, nodename: xgenericshape. Class: resistors.
	sgeneric: Slashed generic bipole, type: path-style, fillable, nodename: sgenericshape. Class: resistors.

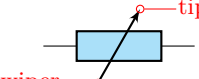
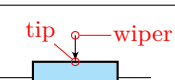
	singeneric: Sine generic bipole ²⁰ , type: path-style, fillable, nodename: singenericshape. Class: resistors.
	tgeneric: Tunable generic bipole, type: path-style, fillable, nodename: tgenericshape. Class: resistors.
	ageneric: Generic asymmetric bipole, type: path-style, fillable, nodename: agenericshape. Class: resistors.
	memristor: Memristor, type: path-style, fillable, nodename: memristorshape. Aliases: Mr. Class: resistors.

Both **shortshape** and **openshape** are not really supposed to be used; they are dummy shapes used as placeholders for the path-drawing routines.

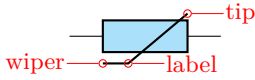
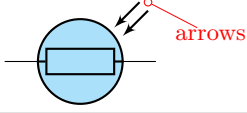
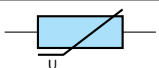
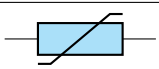
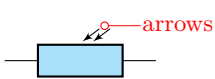
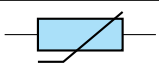
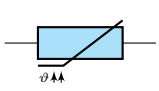
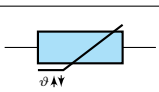
If **americanresistors** option is active (or the style **[american resistors]** is used — this is the default for the package), the resistors are displayed as follows:

	R: Resistor, type: path-style, nodename: resistorshape. Aliases: american resistor. Class: resistors.
	vR: Variable resistor, type: path-style, nodename: vresistorshape. Aliases: variable american resistor. Class: resistors.
	pR: Potentiometer, type: path-style, nodename: potentiometershape. Aliases: american potentiometer. Class: resistors.
	sR: Resistive sensor, type: path-style, nodename: resistivesensshape. Aliases: american resistive sensor. Class: resistors.
	ldR: Ligth-Dependent resistor, type: path-style, fillable, nodename: ldresistorshape. Aliases: american light dependent resistor. Class: resistors.

If instead **europeanresistors** option is active (or the style **[european resistors]** is used), the resistors, variable resistors and potentiometers are displayed as follows:

	R: Resistor, type: path-style, fillable, nodename: genericshape. Aliases: european resistor. Class: resistors.
	vR: Variable resistor, type: path-style, fillable, nodename: tgenericshape. Aliases: variable european resistor. Class: resistors.
	pR: Potentiometer, type: path-style, fillable, nodename: genericpotentiometershape. Aliases: european potentiometer. Class: resistors.

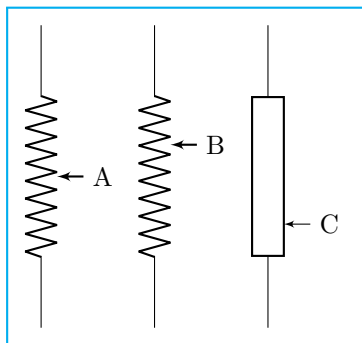
²⁰Contributed by [Jakob Leide on GitHub](#)

	sR : Resistive sensor, type: path-style, fillable, nodename: thermistorshape. Aliases: european resistive sensor. Class: resistors.
	ldR : Ligth-Dependent resistor, type: path-style, fillable, nodename: ldgenericshape. Aliases: european light dependent resistor. Class: resistors.
	varistor : Varistor, type: path-style, fillable, nodename: varistorshape. Class: resistors.
	mov : Metal-Oxide varistor, type: path-style, fillable, nodename: movshape. Class: resistors.
	phR : Photoresistor, type: path-style, fillable, nodename: photoresistorshape. Aliases: photoresistor. Class: resistors.
	thR : Thermistor, type: path-style, fillable, nodename: thermistorshape. Aliases: thermistor. Class: resistors.
	thRp : PTC thermistor, type: path-style, fillable, nodename: thermistorptcshape. Aliases: thermistor ptc. Class: resistors.
	thRn : NTC thermistor, type: path-style, fillable, nodename: thermistorntcshape. Aliases: thermistor ntc. Class: resistors.

Other miscellaneous resistor-like devices:

4.2.1 Potentiometers: wiper position

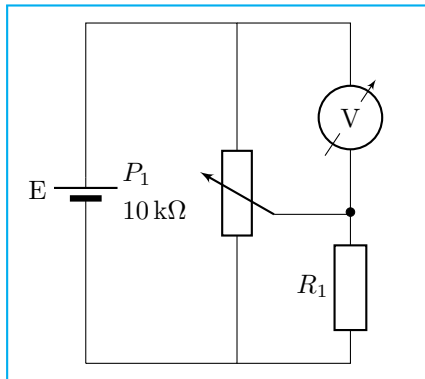
Since version 0.9.5, you can control the position of the wiper in potentiometers using the key `wiper pos`, which is a number in the range `[0,1]`. The default middle position is `wiper pos=0.5`.



```
\begin{circuitikz}[american]
  \ctikzset{resistors/width=1.5, resistors/zigs=9}
  \draw (0,0) to[pR, name=A] ++(0,-4);
  \draw (1.5,0) to[pR, wiper pos=0.3, name=B] ++(0,-4);
  \ctikzset{european resistors}
  \draw (3,0) to[pR, wiper pos=0.8, name=C] ++(0,-4);
  \foreach \i in {A, B, C}
    \node[right] at (\i.wiper) {\i};
\end{circuitikz}
```

Since version 1.6.0, potentiometers and variable resistors have extra anchors²¹, to allow this kind of circuit (that seems to be common in some region):

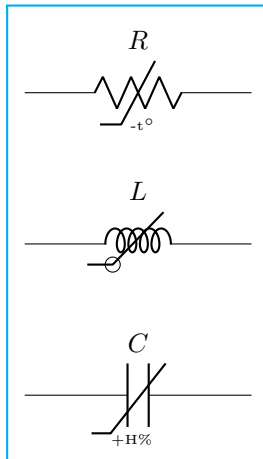
²¹Thanks to a suggestion by [Dr. Matthias Jung on GitHub](#)



```
\begin{circuitikz}[european]
\draw (0,0) to[battery2, l=E] ++(0,4.5)
-- ++(2,0) coordinate(tmp)
to[vR, l2_=$P_1$ and \SI{10}{\kohm}, mirror,
invert, name=P]
(0,0-|tmp) -- (0,0);
\draw (0,0-|tmp) -- ++(1.5,0)
to[R=$R_1$, -*] ++(0,2) coordinate(p)
|- (P.wiper);
\draw (p) to[rmeterwa, t=V] (tmp-|p) -- (tmp);
\end{circuitikz}
```

4.2.2 Generic sensors anchors

Generic sensors have an extra anchor named `label` to help position the type of dependence, if needed:



```
\begin{circuitikz}
\draw (0,2) to[sR, l=$R$, name=mySR] ++(3,0);
\node [font=\tiny, right] at(mySR.label) {-t\si{\degree}};
\draw (0,0) to[sL, l=$L$, name=mySL] ++(3,0);
\node [draw, circle, inner sep=2pt] at(mySL.label) {};
\draw (0,-2) to[sC, l=$C$, name=mySC] ++(3,0);
\node [font=\tiny, below right, inner sep=0pt] at(mySC.label)
{+H\si{\%}};
\end{circuitikz}
```

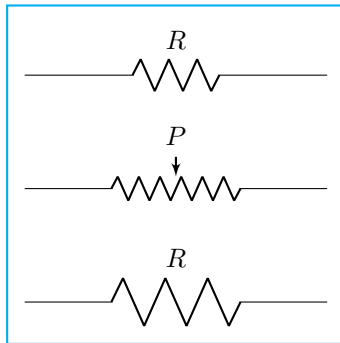
The anchor is positioned just on the corner of the segmented line crossing the component.

4.2.3 Resistive components customization

4.2.3.1 Geometry. You can change the scale of these components (all the resistive bipoles together) by setting the key `resistors/scale` (default 1.0). Similarly, you can change the widths by setting `resistors/width` (default 0.8).

You can change the width of these components (all the resistive bipoles together) by setting the key `resistors/width` to something different from the default 0.8.

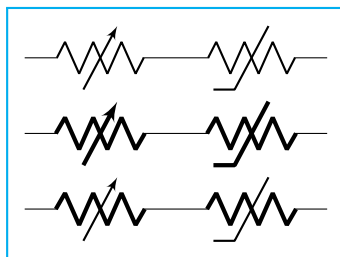
For the american style resistors, you can change the number of “zig-zags” by setting the key `resistors/zigs` (default value 3).



```
\begin{circuitikz}[
    longpot/.style = {pR, resistors/scale=0.75,
        resistors/width=1.6, resistors/zigs=6}]
\draw (0,1.5) to[R, l=$R$] ++(4,0);
\draw (0,0) to[longpot, l=$P$] ++(4,0);
\ctikzset{resistors/scale=1.5}
\draw (0,-1.5) to[R, l=$R$] ++(4,0);
\end{circuitikz}
```

4.2.3.2 Thickness. The line thickness of the resistive components is governed by the class thickness; you can change it assigning a value to the key `resistors/thickness` (default `none`, that means `bipoles/thickness` is used, and that defaults to 2.0; the value is relative to the base line thickness).

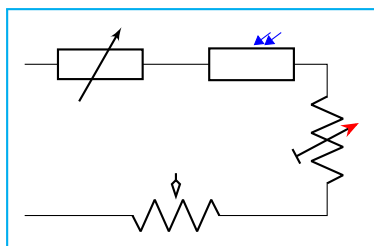
We can call *modifiers* the elements that are added to the basic shape to express some characteristics of the component; for example the arrows for the variable resistors or the bar for the sensors. Normally the thickness of these elements are the same as the one chosen for the component²². You can change their thickness with the class key modifier `thickness` which is relative to the main component thickness.



```
\begin{circuitikz}[american]
\draw (0,2) to[vR] ++(2,0) to[sR] ++(2,0);
\ctikzset{resistors/thickness=4}
\draw (0,1) to[vR] ++(2,0) to[sR] ++(2,0);
\ctikzset{resistors/modifier thickness=0.5}
\draw (0,0) to[vR] ++(2,0) to[sR] ++(2,0);
\end{circuitikz}
```

4.2.3.3 Arrows You can change the arrow tips used in tunable resistors (`vR`, `tgeneric`) with the key `tunable end arrow` and in potentiometers with the key `wiper end arrow` (by default the key is the word “default” to obtain the default arrow, which is `latexslim` for both). Also you can change the start arrow with the corresponding `tunable start arrow` or `wiper start arrow` (the default value “default” is equivalent to `{}` for both, which means no arrow).

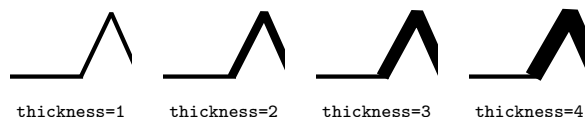
You can change that globally or locally, as ever. The tip specification is the one you can find in the TikZ manual (“[Arrow Tip Specifications](#)”). For the photoresistor and the two “flavors” of the light-dependent resistor (`ldR`, `american` or `european`), the style of the arrows follow the `opto` commands as in the photodiodes and phototransistor: see 4.4.3.1.



```
\begin{tikzpicture}[american]
% globally all the potentiometers
\ctikzset{wiper end arrow={Kite[open]},
    opto arrows/color=blue, opto end arrow={Triangle
}}
\draw (0,0) to[tgeneric] ++(2,0) to[phR] ++(2,0)
% set locally on this variable resistor
to[vR, tunable end arrow={Stealth[red]},
    tunable start arrow={Bar}, invert] ++(0,-2)
to[pR, mirror] ++(-4,0);
\end{tikzpicture}
```

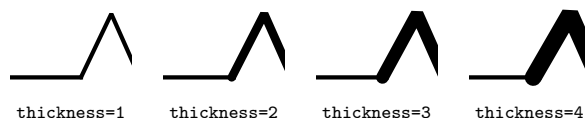
²²Due to a bug in versions before 1.3.4, that didn’t happen for thermistors

4.2.3.4 Details of American (“zig-zag”) resistors. American (zig-zag) resistors have a little joining problem²³ with the leading wires if the thickness is greater than two. In the following drawing you can see the problem when the thickness grows from 1 to 4.



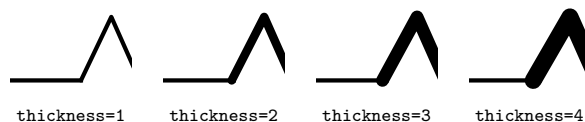
Since v1.7.0, one possibility to correct the problem is to change the type of joining of the zig-zag line, using the key `resistors/zigzag join`, which is a command that by default is void. For example, the following effect is obtained by using

```
\ctikzset{resistors/zigzag hook/.code={\pgfsetroundcap}}
```



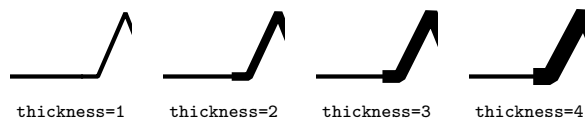
or you can even go full rounded

```
\ctikzset{resistors/zigzag hook/.code={\pgfsetroundcap\pgfsetroundjoin}}
```



Another possibility is to add a little horizontal “stub” to the shape, with the key `resistors/zigzag stub` (default 0), which will add a first part which is a continuation of the wire:

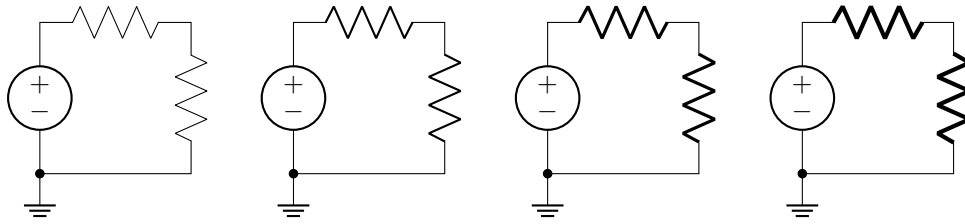
```
\ctikzset{resistors/zigzag stub=0.05}% this is relative to the resistor's length
```



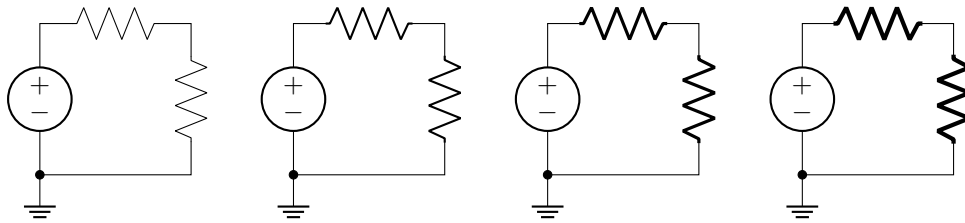
...or you can just combine all of them as you prefer. With the standard join/cap options, the look of the resistors for thickness from 1 to 4 is shown here:

Standard drawing of American resistors

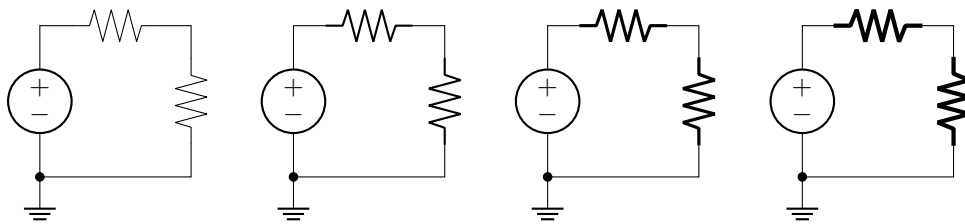
²³Noticed by [Jakob Leide on GitHub](#) and later [discussed here](#).



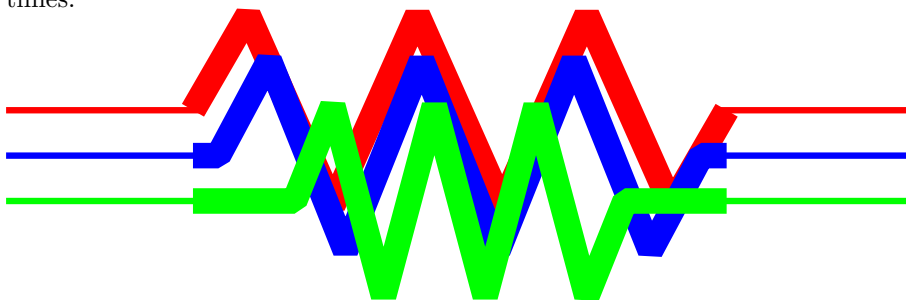
With a 5% stub:



With a 20% stub:



Finally, here is the detailed shape with thickness 2 (red=0, blue=0.05, green=0.2), magnified six times:



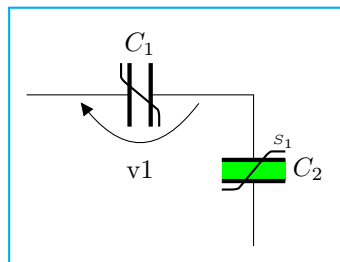
4.3 Capacitors and inductors: dynamical bipoles

4.3.1 Capacitors

	capacitor: Capacitor, type: path-style, fillable, nodename: capacitorshape. Aliases: C. Class: capacitors.
--	--

	curved capacitor: Curved (polarized) capacitor, type: path-style, fillable, nodename: ccapacitorshape. Aliases: cC. Class: capacitors.
	ecapacitor: Electrolytic capacitor, type: path-style, fillable, nodename: ecapacitorshape. Aliases: eC, elko. Class: capacitors.
	variable capacitor: Variable capacitor, type: path-style, fillable, nodename: vcapacitorshape. Aliases: vC. Class: capacitors.
	capacitive sensor: Capacitive sensor, type: path-style, fillable, nodename: capacitivesensshape. Aliases: sC. Class: capacitors.
	piezoelectric: Piezoelectric Element, type: path-style, fillable, nodename: piezoelectricshape. Aliases: PZ. Class: capacitors.
	cpe: Constant Phase Element, type: path-style, fillable, nodename: cpeshape. Aliases: cpe. Class: capacitors.
	feC: Ferroelectric capacitor ²⁴ , type: path-style, fillable, nodename: ferrocapshape. Aliases: ferrocap. Class: capacitors.

Capacitors are fillable since v1.4.1; this is normally just a stylistic option but in the case of ferroelectric capacitors that could be used to show the state of the hysteresis of the component.



```
\begin{tikzpicture}[]
  \ctikzset{capacitors/.cd,
    thickness=4, modifier thickness=0.5}
  \draw (0,0) to[feC, l=$C_1$, v=v1] ++(3,0)
    to[feC, l=$C_2$, fill=green, name=C2] ++(0,-2);
  \node [font=\tiny, above right, inner sep=1pt]
    at(C2.kink left) {$S_1$};
\end{tikzpicture}
```

There is also the (deprecated²⁵ — its polarity is not coherent with the rest of the components) `polar capacitor`; please do not use it.

4.3.2 Capacitive sensors anchors

For capacitive sensors, you have the same anchors than in the case of resistive sensors, see section 4.2.2.

²⁴suggested by [Mayeul Cantan](#)

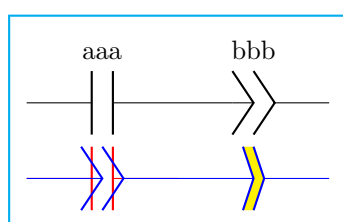
²⁵Thanks to [Anshul Singh](#) for noticing.

4.3.3 Capacitors customizations

You can change the scale of the capacitors by setting the key `capacitors/scale` to something different from the default 1.0. For thickness, you can use the same keys (applied to the `capacitors` class) as for resistors in 4.2.3.2.

Variable capacitors arrow tips follow the settings of resistors, see section 4.2.3.3.

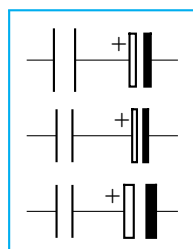
The relative size of the capacitors is a bit of a mixed bag, because each one has historically different internal parameters that makes maintaining coherence quite difficult. In v1.4.1 this has changed and now you can use styling options to change the way the capacitors look. The main parameter you can set is `capacitors/width` (default 0.2), which controls the standard distance between plates. That will change all the components (notice that `piezoelectric` and `cpe` default width is twice the size of a standard capacitor — although this is not evident for the `cpe` given its shape.)



```
\begin{circuitikz}[european]
\draw (0,1) to[C=aaa] ++(2,0) to[cpe=bbb] ++(2,0);
\draw[color=red] (0,0) to [C] ++(2,0);
\draw[color=blue] (0,0) to [cpe] ++(2,0)
to[cpe, fill=yellow, capacitors/width=0.1] ++(2,0);
\end{circuitikz}
```

The `capacitors/height` key is available also to set the height of the capacitor; the default is 0.6 for most of the capacitors, but 0.5 for electrolytic ones and 0.7 for piezoelectric. When used, it will set all of them at the same value, which is a good thing.

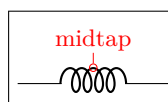
If you want that only a specific kind of capacitor has a different value for a key, you can always use a style which will have a local scope, as in the following example.



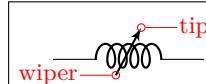
```
\begin{tikzpicture}
\draw (0,1) to [C] ++(1,0) to [elko] ++(1,0);
\ctikzset{capacitors/width=0.15, capacitors/height=0.5}
\draw (0,0) to [C] ++(1,0) to [elko] ++(1,0);
\tikzset{big elko/.style={elko=#1, capacitors/width=0.3}}
\draw (0,-1) to [C] ++(1,0) to[big elko] ++(1,0);
\end{tikzpicture}
```

4.3.4 Inductors

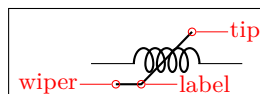
If the `cuteinductors` option is active (default behaviour), or the style `[cute inductors]` is used, the inductors are displayed as follows:



L: Inductor, type: `path-style`, nodename: `cuteinductorshape`. Aliases: `cute inductor`. Class: `inductors`.

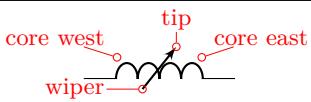
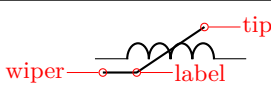


vL: Variable inductor, type: `path-style`, nodename: `vcuteinductorshape`. Aliases: `variable cute inductor`. Class: `inductors`.

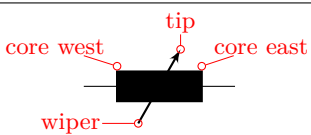
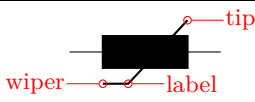


sL: Inductive sensor, type: `path-style`, nodename: `scuteinductorshape`. Aliases: `cute inductive sensor`. Class: `inductors`.

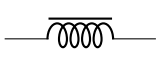
If the `americaninductors` option is active (or the style `[american inductors]` is used), the inductors are displayed as follows:

	L: Inductor, type: <code>path-style</code> , nodename: <code>americaninductorshape</code> . Aliases: <code>american inductor</code> . Class: <code>inductors</code> .
	vL: Variable inductor, type: <code>path-style</code> , nodename: <code>vamericaninductorshape</code> . Aliases: <code>variable american inductor</code> . Class: <code>inductors</code> .
	sL: Inductive sensor, type: <code>path-style</code> , nodename: <code>samericaninductorshape</code> . Aliases: <code>american inductive sensor</code> . Class: <code>inductors</code> .

Finally, if the `europeaninductors` option is active (or the style `[european inductors]` is used), the inductors are displayed as follows:

	L: Inductor, type: <code>path-style</code> , nodename: <code>fullgenericshape</code> . Aliases: <code>european inductor</code> . Class: <code>inductors</code> .
	vL: Variable inductor, type: <code>path-style</code> , nodename: <code>tfullgenericshape</code> . Aliases: <code>variable european inductor</code> . Class: <code>inductors</code> .
	sL: Inductive sensor, type: <code>path-style</code> , nodename: <code>sfullgenericshape</code> . Aliases: <code>european inductive sensor</code> . Class: <code>inductors</code> .

For historical reasons, *chokes* come only in the `cute`. You can use the `core west` and `core east` anchors (see 4.3.6.2) to build your own core lines for the other inductors.

	cute choke: Choke, type: <code>path-style</code> , nodename: <code>cutechokesshape</code> . Class: <code>inductors</code> .
---	--

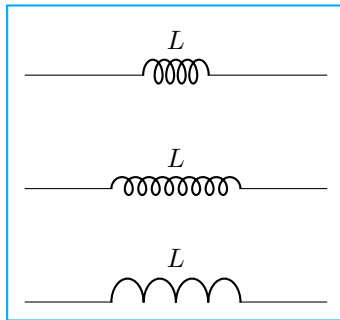
4.3.5 Inductors customizations

You can change the scale of the inductors by setting the key `inductors/scale` to something different from the default 1.0. For thickness, you can use the same keys (applied to the `inductors` class) as for resistors in 4.2.3.2.

Variable inductors arrow tips follow the settings of resistors, see section 4.2.3.3.

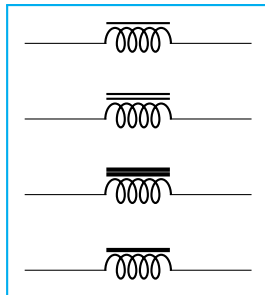
You can change the width of these components (all the inductors together, unless you use style or scoping) by setting the key `inductors/width` to something different from the default, which is 0.8 for american and european inductors, and 0.6 for cute inductors.

Moreover, you can change the number of “coils” drawn by setting the key `inductors/coils` (default value 5 for cute inductors and 4 for american ones). **Notice** that the minimum number of coils is 1 for american inductors, and 2 for cute ones.



```
\begin{circuitikz}[
    longL/.style = {cute inductor, inductors/scale=0.75,
        inductors/width=1.6, inductors/coils=9}]
\draw (0,1.5) to[L, l=$L$] ++(4,0);
\draw (0,0) to[longL, l=$L$] ++(4,0);
\ctikzset{inductors/scale=1.5, inductor=american}
\draw (0,-1.5) to[L, l=$L$] ++(4,0);
\end{circuitikz}
```

4.3.5.1 Chokes can have single and double lines, and can have the line thickness adjusted (the value is relative to the thickness of the inductor). In general, you should use the anchors (see 4.3.6.2) to add core lines to inductors.



```
\begin{circuitikz}[american]
\draw (0,0) to[cute choke] ++(3,0);
\draw (0,-1) to[cute choke, twolineschoke] ++(3,0);

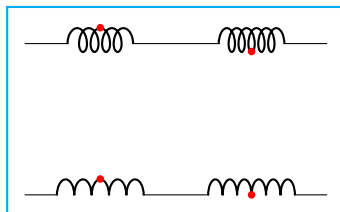
\ctikzset{bipoles/cutechoke/ctick=2, twolineschoke}

\draw (0,-2) to[cute choke] ++(3,0);
\draw (0,-3) to[cute choke, onelinechoke] ++(3,0);
\end{circuitikz}
```

4.3.6 Inductors anchors

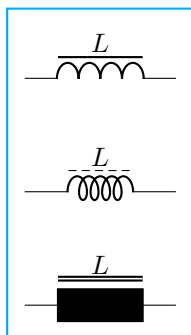
For inductive sensors, you have the same anchors than in the case of resistive sensors, see section 4.2.2.

4.3.6.1 Taps. Inductors have an additional anchor, called `midtap`, that connects to the center of the coil “wire”. Notice that this anchor could be on one side or the other of the component, depending on the number of loops of the element; if you need a fixed position, you can use the geographical anchors.



```
\begin{circuitikz}[
    loops/.style={circuitikz/inductors/coils=#1}]
\ctikzset{cute inductors}
\draw (0,2) to[L, loops=5, name=A] ++(2,0)
to[L, loops=6, name=B] ++(2,0);
\ctikzset{american inductors}
\draw (0,0) to[L, loops=5, name=C] ++(2,0)
to[L, loops=6, name=D] ++(2,0);
\foreach \i in {A, B, C, D}
\node[circle, fill=red, inner sep=1pt] at (\i.midtap){};
\end{circuitikz}
```

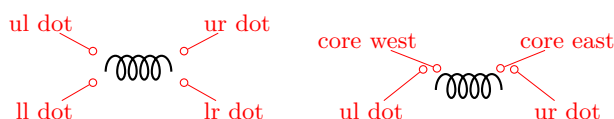
4.3.6.2 Core anchors. Inductors have additional anchors to add core lines (for historical reasons, there is a `cute choke` component also, but to use inductors in the chosen style you'd better use these anchors). The anchors are called `core west` and `core east` and they are positioned at a distance that you can tweak with the `\ctikzset` key `bipoles/inductors/core distance` (default 2pt).



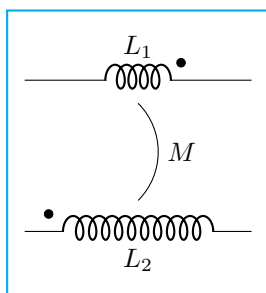
```
\begin{circuitikz}[]
\ctikzset{american}
\draw (0,3) to[L=$L$, name=myL] ++(2,0);
\draw[thick] (myL.core west) -- (myL.core east);
\ctikzset{cute inductors}
\draw (0,1.5) to[L=$L$, name=myL] ++(2,0);
\draw[densely dashed] (myL.core west) -- (myL.core east);
\ctikzset{european, bipoles/inductors/core distance=4pt}
\draw (0,0) to[L=$L$, name=myL, label distance=2pt] ++(2,0);
\draw[thick, double] (myL.core west) -- (myL.core east);
\end{circuitikz}
```

Notice that the core lines will **not** change the position of labels. You have to move them by hand if needed (or position them on the other side); see 5.1.1.1.

4.3.6.3 Dot anchors. Inductances also have “dot” anchors²⁶, to help positioning dots when specifying mutual inductance signs. The anchors are name `lr dot` (for lower right dot), `ur dot` (upper right) and so on:



and as you can see, you have to be careful if you use dot anchors and core anchors together. You can change the position of the dots by changing the keys `bipoles/inductors/dot x distance` (default 4pt), which represent how much the dot extends outside the component width, and the corresponding `dot y distance` (default 1pt), which serves the same scope in the height direction.



```
\begin{circuitikz}[]
\ctikzset{bipoles/inductors/.cd, dot x distance=3pt,
dot y distance=0pt}
\draw (0,2) to[L=$L_1$, name=l1] ++(3,0);
\draw (0,0) to[L, l_=$L_2$, name=l2, inductors/width=1.4,
inductors/coils=11] ++(3,0);
\path (l1.ur dot) node[circ]{} (l2.ul dot) node[circ]{};
\draw ([yshift=-0.2cm]l1.south)
to[out=-45, in=45] node[right]{$M$}
([yshift=0.2cm]l2.north);
\end{circuitikz}
```

Notice that the position of the dot anchors does not coincide with the position of the dots in the transformers (see section 4.19), because there they depend on the size of the complete double-bipole more than on the size of the inductances.

²⁶proposed by Romano in a discussion by [GitHub user AndreaDiPietro92](#)

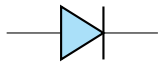
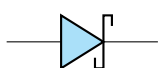
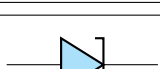
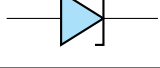
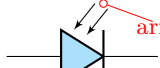
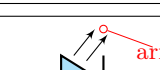
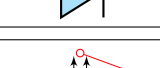
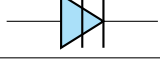
4.4 Diodes and such

There are three basic styles for diodes: **empty** (fillable in color), **full** (completely filled with the draw color) and **stroke** (empty, but with a line across them).

You can switch between the styles setting the key **diode** (for example `\ctikzset{diode=full}` or **empty** or **stroke**, or with the styles **full diodes**, **empty diodes** and **stroke diodes**.

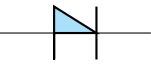
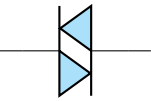
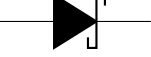
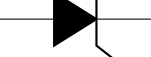
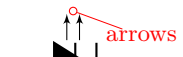
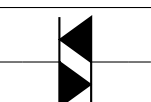
To use the default element, simply use the name shown for the empty diodes without the final “o” — that is **D**, **sD**, and so on. The names shown in the following tables will draw the specified diode independently on the style chosen (that is, **leD*** is always a full LED diode).

The package options **fulldiode**, **strokediode**, and **emptydiode** (and the styles **full diodes**, **stroke diodes**, and **empty diodes**) define which shape will be used by abbreviated commands such that **D**, **sD**, **zD**, **zzD**, **tD**, **pD**, **leD**, **VC**, **Ty**, **Tr** (no stroke symbol available!).

	empty diode : Empty diode, type: <code>path-style</code> , fillable, nodename: <code>emptydiodeshape</code> . Aliases: Do . Class: diodes .
	empty Schottky diode : Empty Schottky diode, type: <code>path-style</code> , fillable, nodename: <code>emptysdiodeshape</code> . Aliases: sDo . Class: diodes .
	empty Zener diode : Empty Zener diode, type: <code>path-style</code> , fillable, nodename: <code>emptyzdiodeshape</code> . Aliases: zDo . Class: diodes .
	empty ZZener diode : Empty ZZener diode, type: <code>path-style</code> , fillable, nodename: <code>emptyzzdiodeshape</code> . Aliases: zzDo . Class: diodes .
	empty tunnel diode : Empty tunnel diode, type: <code>path-style</code> , fillable, nodename: <code>emptytdiodeshape</code> . Aliases: tDo . Class: diodes .
	empty photodiode : Empty photodiode, type: <code>path-style</code> , fillable, nodename: <code>emptypdiodeshape</code> . Aliases: pDo . Class: diodes .
	empty led : Empty led, type: <code>path-style</code> , fillable, nodename: <code>emptylediodeshape</code> . Aliases: leDo . Class: diodes .
	empty laser diode : Empty laser diode ²⁷ , type: <code>path-style</code> , fillable, nodename: <code>emptylaserdiodeshape</code> . Aliases: lasD . Class: diodes .
	empty varcap : Empty varcap, type: <code>path-style</code> , fillable, nodename: <code>emptyvarcapshape</code> . Aliases: VCo . Class: diodes .
	empty TVS diode : Empty TVS diode, transorb ²⁸ , type: <code>path-style</code> , fillable, nodename: <code>emptytvsvdiodeshape</code> . Aliases: tvsvDo . Class: diodes .

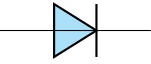
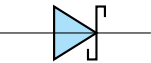
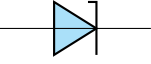
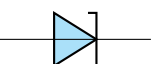
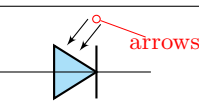
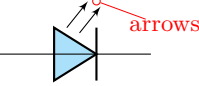
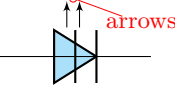
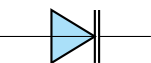
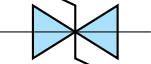
²⁷Added by André Alves in v1.4.4

²⁸Transorbs were suggested by [Anisio Braga](#)

	empty Shockley diode: Empty Shockley diode ²⁹ , type: path-style, fillable, nodename: emptyshdiodeshape. Aliases: shDo. Class: diodes.
	empty bidirectionaldiode: Empty bidirectionaldiode, type: path-style, fillable, nodename: emptybidirectionaldiodeshape. Aliases: biDo. Class: diodes.
	full diode: Full diode, type: path-style, nodename: fulldiodeshape. Aliases: D*. Class: diodes.
	full Schottky diode: Full Schottky diode, type: path-style, nodename: fullsdiodeshape. Aliases: sD*. Class: diodes.
	full Zener diode: Full Zener diode, type: path-style, nodename: fullzdiodeshape. Aliases: zD*. Class: diodes.
	full ZZener diode: Full ZZener diode, type: path-style, nodename: fullzzdiodeshape. Aliases: zzD*. Class: diodes.
	full tunnel diode: Full tunnel diode, type: path-style, nodename: fulltdiodeshape. Aliases: tD*. Class: diodes.
	full photodiode: Full photodiode, type: path-style, nodename: fullpdiodeshape. Aliases: pD*. Class: diodes.
	full led: Full led, type: path-style, nodename: fulllediodeshape. Aliases: leD*. Class: diodes.
	full laser diode: Full laser diode, type: path-style, nodename: fulllaserdiodeshape. Aliases: lasD*. Class: diodes.
	full varcap: Full varcap, type: path-style, nodename: fullvarcapshape. Aliases: VC*. Class: diodes.
	full TVS diode: Full TVS diode, transorb, type: path-style, nodename: fulltvsvdiodeshape. Aliases: tvsD*. Class: diodes.
	full Shockley diode: Full Shockley diode, type: path-style, nodename: fullshdiodeshape. Aliases: shD*. Class: diodes.
	full bidirectionaldiode: Full bidirectionaldiode, type: path-style, nodename: fullbidirectionaldiodeshape. Aliases: biD*. Class: diodes.

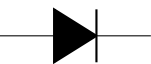
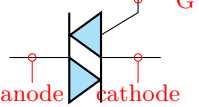
These shapes have no exact node-style counterpart, because the stroke line is built upon the empty variants:

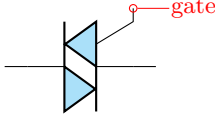
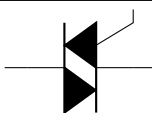
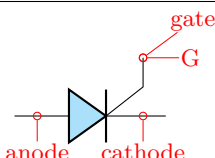
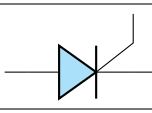
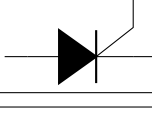
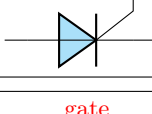
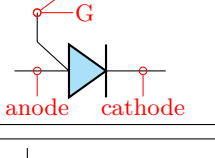
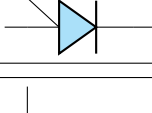
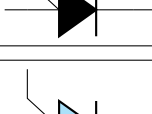
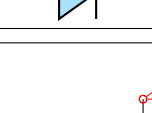
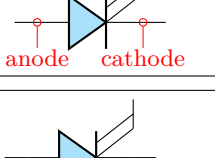
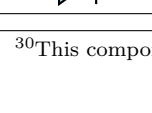
²⁹Shockley diodes were suggested by @Dauph

	stroke diode: Stroke diode, type: path-style, fillable, nodename: emptydiodeshape. Aliases: D-. Class: diodes.
	stroke Schottky diode: Stroke Schottky diode, type: path-style, fillable, nodename: emptysdiodeshape. Aliases: sD-. Class: diodes.
	stroke Zener diode: Stroke Zener diode, type: path-style, fillable, nodename: emptyzdiodeshape. Aliases: zD-. Class: diodes.
	stroke ZZener diode: Stroke ZZener diode, type: path-style, fillable, nodename: emptyzzdiodeshape. Aliases: zzD-. Class: diodes.
	stroke tunnel diode: Stroke tunnel diode, type: path-style, fillable, nodename: emptytdiodeshape. Aliases: tD-. Class: diodes.
	stroke photodiode: Stroke photodiode, type: path-style, fillable, nodename: emptypdiodeshape. Aliases: pD-. Class: diodes.
	stroke led: Stroke led, type: path-style, fillable, nodename: emptylediodeshape. Aliases: leD-. Class: diodes.
	stroke laser diode: Stroke laser diode, type: path-style, fillable, nodename: emptylaserdiodeshape. Aliases: lasD-. Class: diodes.
	stroke varcap: Stroke varcap, type: path-style, fillable, nodename: emptyvarcapshape. Aliases: VC-. Class: diodes.
	stroke TVS diode: Stroke TVS diode, transorb, type: path-style, fillable, nodename: emptytvdsdiodeshape. Aliases: tvsD-. Class: diodes.

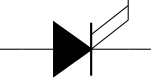
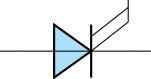
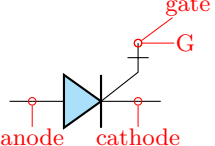
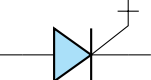
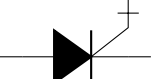
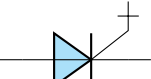
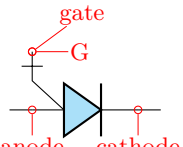
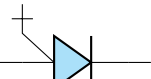
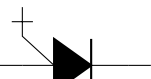
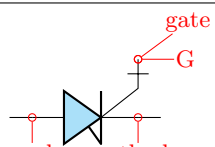
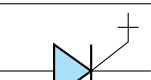
4.4.1 Tripole-like diodes

The following tripoles are entered with the usual command, of the form `to[Tr, ...]`. In the following list you can see the traditional, or *legacy*, shape of the Thyristors-type devices.

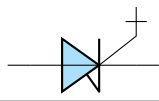
	full diode: Full diode, type: path-style, nodename: fulldiodeshape. Aliases: D*. Class: diodes.
	stroke diode: Stroke diode, type: path-style, fillable, nodename: emptydiodeshape. Aliases: D-. Class: diodes.
	triac: Standard triac (shape depends on package option), type: path-style, fillable, nodename: emptytriacshape. Aliases: Tr. Class: diodes.

	empty triac: Empty triac, type: path-style, fillable, nodename: emptytriacshape. Aliases: Tro. Class: diodes.
	full triac: Full triac, type: path-style, nodename: fulltriacshape. Aliases: Tr*. Class: diodes.
	thyristor: Standard thyristor (shape depends on package option), type: path-style, fillable, nodename: emptythyristorshape. Aliases: Ty. Class: diodes.
	empty thyristor: Empty thyristor, type: path-style, fillable, nodename: emptythyristorshape. Aliases: Tyo. Class: diodes.
	full thyristor: Full thyristor, type: path-style, nodename: fullthyristorshape. Aliases: Ty*. Class: diodes.
	stroke thyristor: Stroke thyristor, type: path-style, fillable, nodename: emptythyristorshape. Aliases: Ty-. Class: diodes.
	put: Standard Programmable Unipolar Transistor ³⁰ (shape depends on package option), type: path-style, fillable, nodename: emptyputshape. Aliases: PUT. Class: diodes.
	empty put: Empty PUT, type: path-style, fillable, nodename: emptyputshape. Aliases: PUTo. Class: diodes.
	full put: Full PUT, type: path-style, nodename: fullputshape. Aliases: PUT*. Class: diodes.
	stroke put: Stroke PUT, type: path-style, fillable, nodename: emptyputshape. Aliases: PUT-. Class: diodes.
	gto: Standard GTO (shape depends on package option), type: path-style, fillable, nodename: emptygtoshape. Aliases: GTO. Class: diodes.
	empty gto: Empty GTO, type: path-style, fillable, nodename: emptygtoshape. Aliases: GTOo. Class: diodes.

³⁰This components, and the GTO family, has been suggested by [GitHub user JetherReis](#).

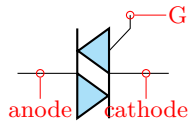
	full gto : Full GTO, type: <code>path-style</code> , nodename: <code>fullgtoshape</code> . Aliases: <code>GT0*</code> . Class: <code>diodes</code> .
	stroke gto : Stroke GTO, type: <code>path-style</code> , fillable, nodename: <code>emptygtoshape</code> . Aliases: <code>GT0-</code> . Class: <code>diodes</code> .
	gtobar : Standard GTO with bar-type gate (shape depends on package option), type: <code>path-style</code> , fillable, nodename: <code>emptygtobarshape</code> . Aliases: <code>GT0b</code> . Class: <code>diodes</code> .
	empty gtobar : Empty GTO, bar-type, type: <code>path-style</code> , fillable, nodename: <code>emptygtobarshape</code> . Aliases: <code>GT0bo</code> . Class: <code>diodes</code> .
	full gtobar : Full GTO, bar-type, type: <code>path-style</code> , nodename: <code>fullgtobarshape</code> . Aliases: <code>GT0b*</code> . Class: <code>diodes</code> .
	stroke gtobar : Stroke GTO, bar type, type: <code>path-style</code> , fillable, nodename: <code>emptygtobarshape</code> . Aliases: <code>GT0b-</code> . Class: <code>diodes</code> .
	agtobar : Standard GTO with bar-type gate on anode (shape depends on package option), type: <code>path-style</code> , fillable, nodename: <code>emptyagtobarshape</code> . Aliases: <code>aGT0b</code> . Class: <code>diodes</code> .
	empty agtobar : Empty GTO, bar-type on anode, type: <code>path-style</code> , fillable, nodename: <code>emptyagtobarshape</code> . Aliases: <code>aGT0bo</code> . Class: <code>diodes</code> .
	full agtobar : Full GTO, bar-type on anode, type: <code>path-style</code> , nodename: <code>fullagtobarshape</code> . Aliases: <code>aGT0b*</code> . Class: <code>diodes</code> .
	stroke agtobar : Stroke GTO, bar-type on anode, type: <code>path-style</code> , fillable, nodename: <code>emptyagtobarshape</code> . Aliases: <code>aGT0b-</code> . Class: <code>diodes</code> .
	igct : Standard IGCT ³¹ , type: <code>path-style</code> , fillable, nodename: <code>emptyigctshape</code> . Aliases: <code>IGCT</code> . Class: <code>diodes</code> .
	empty igct : Empty IGCT, type: <code>path-style</code> , fillable, nodename: <code>emptyigctshape</code> . Aliases: <code>IGCTo</code> . Class: <code>diodes</code> .
	full igct : Full IGCT, type: <code>path-style</code> , nodename: <code>fulligctshape</code> . Aliases: <code>IGCT*</code> . Class: <code>diodes</code> .

³¹The IGCTs family has been contributed by Paul Sacco

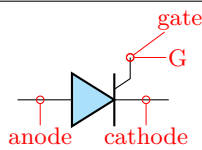


stroke igct: Stroke IGCT, type: `path-style`, fillable, nodename: `emptyigctshape`. Aliases: IGCT-. Class: diodes.

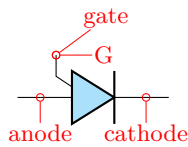
For basically stylistic reasons, there is a different, more compact, shape available for them, activated with the key **thyristor style=compact** (the default is **legacy**). All the devices above are present, we will show here just the automatic version for shortness.



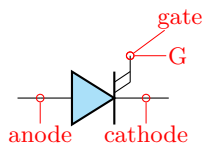
triac: Standard triac (shape depends on package option), type: `path-style`, fillable, nodename: `emptytriacshape`. Aliases: Tr. Class: diodes.



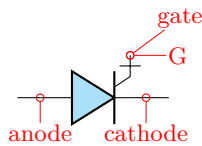
thyristor: Standard thyristor (shape depends on package option), type: `path-style`, fillable, nodename: `emptythyristorshape`. Aliases: Ty. Class: diodes.



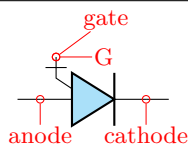
put: Standard Programmable Unipolar Transistor (shape depends on package option), type: `path-style`, fillable, nodename: `emptyputshape`. Aliases: PUT. Class: diodes.



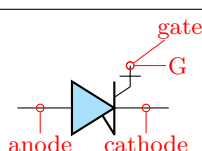
gto: Standard gto (shape depends on package option), type: `path-style`, fillable, nodename: `emptygtoshape`. Aliases: GTO. Class: diodes.



gtobar: Standard GTO with a bar symbol on the gate (shape depends on package option), type: `path-style`, fillable, nodename: `emptygtobarshape`. Aliases: GTOb. Class: diodes.



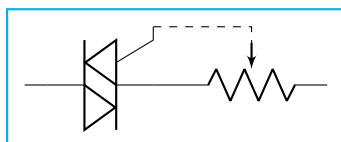
agto: Standard GTO with bar-type gate on anode (shape depends on package option), type: `path-style`, fillable, nodename: `emptyagto`. Aliases: aGTOb. Class: diodes.



igct: Standard igct (shape depends on package option), type: `path-style`, fillable, nodename: `emptyigctshape`. Aliases: IGCT. Class: diodes.

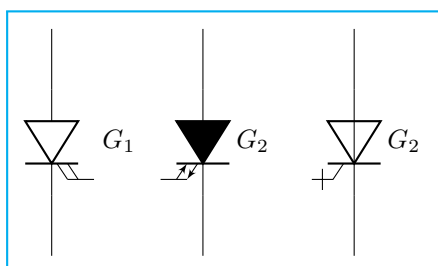
4.4.2 Thyristors anchors and customization

When inserting a thyristor, a triac or a potentiometer, one needs to refer to the third node-gate (gate or G) for the former two; wiper (wiper or W) for the latter one. This is done by giving a name to the bipole:



```
\begin{circuitikz} \draw
(0,0) to[Tr, n=TRI] (2,0)
to[pR, n=POT] (4,0);
\draw[dashed] (TRI.G) -| (POT.wiper)
;\end{circuitikz}
```

As commented above, you can change the shape of these devices (globally or locally) setting the key `thyristor style=compact` (the default is `legacy`). Additionally, normally the plain GTO symbols come without the arrows, but you can add them using a syntax similar to the one explained in section 4.2.3.3 using the arrow group `gto gate`.

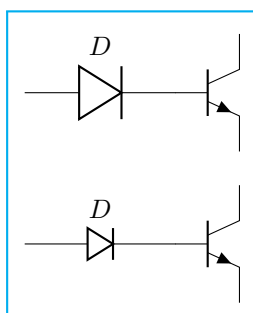


```
\begin{circuitikz}[]
\ctikzset{thyristor style=compact}
\draw (0,0) to[GTO=$G_1$] ++(0,-3);
\ctikzset{gto gate end arrow=latexslim}
\draw (2,0) to[GTO*=$G_2$, mirror] ++(0,-3);
\draw (4,0) to[GTOb-=$G_2$, mirror] ++(0,-3)
;
\end{circuitikz}
```

Notice that you can set both `gto gate end arrow` and `gto gate start arrow` — choosing just one of the two you can decide the “rotation” direction of the symbol. There is little space though, so don’t overdo it.

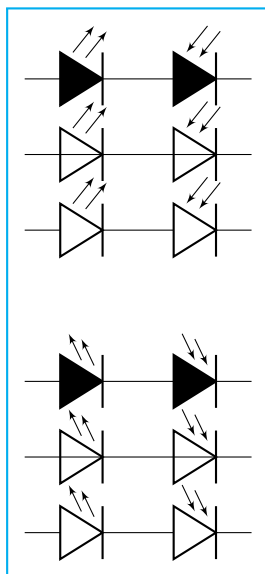
4.4.3 Diode customizations

You can change the scale of the diodes by setting the key `diodes/scale` to something different from the default 1.0. In Romano’s opinion, diodes are somewhat big with the default style of the package, so a setting like `\ctikzset{diodes/scale=0.6}` is recommended.



```
\begin{circuitikz}
\draw (0,1) to[D, l=$D$] ++(2,0)
node[npn, anchor=B]{};
\ctikzset{diodes/scale=0.6}
\draw (0,-1) to[D, l=$D$] ++(2,0)
node[npn, anchor=B]{};
\end{circuitikz}
```

4.4.3.1 Optical devices arrows You can change the direction of the LEDs and photodiodes’ arrows by using the binary keys `led arrows from cathode` and `pd arrows to cathode` (the default are `led arrows from anode` and `pd arrows to anode`), as you can see in the following example.



```
\begin{circuitikz}
\ctikzset{led arrows from anode} % default
\ctikzset{pd arrows to anode}    % default
\ctikzset{full diodes}
\draw (0,0) to[leD] ++(1.5,0) to[pD] ++(1.5,0);
\ctikzset{stroke diodes}
\draw (0,-1) to[leD] ++(1.5,0) to[pD] ++(1.5,0);
\ctikzset{empty diodes}
\draw (0,-2) to[leD] ++(1.5,0) to[pD] ++(1.5,0);

\ctikzset{led arrows from cathode}
\ctikzset{pd arrows to cathode}
\ctikzset{full diodes}
\draw (0,-4) to[leD] ++(1.5,0) to[pD] ++(1.5,0);
\ctikzset{stroke diodes}
\draw (0,-5) to[leD] ++(1.5,0) to[pD] ++(1.5,0);
\ctikzset{empty diodes}
\draw (0,-6) to[leD] ++(1.5,0) to[pD] ++(1.5,0);
\end{circuitikz}
```

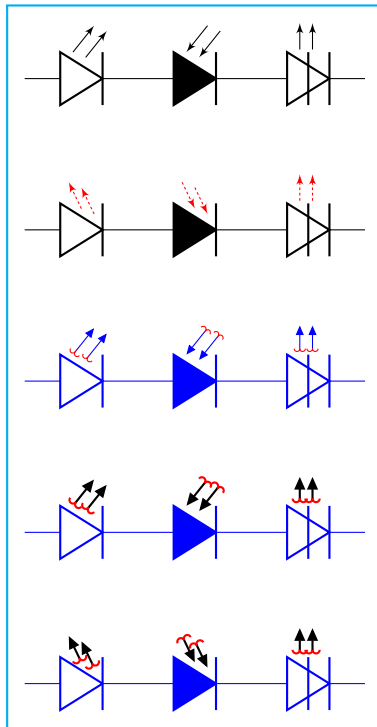
Since version 1.5.5³², you can change the arrows used for LEDs, photodiodes and laser diodes with the generic arrows options shown in 4.2.3.3, using the name `opto`, like in the following (overdone) example. Normally you want just to change the `end arrow`...

As you can see, you can also have the option to globally change the color, relative thickness, and dash pattern by setting keys with the `\ctikzset` command (or, like in the following example, directly in the node instantiation) under the `opto arrows` hierarchy. The available keys are:

parameter	default	description
<code>relative thickness</code>	<code>1.0</code>	multiply the class thickness
<code>color</code>	<code>default</code>	stroke color: <code>default</code> is the same as the component
<code>dash</code>	<code>default</code>	dash pattern: <code>default</code> means not to change the setting for the component; <code>none</code> means unbroken line; every other input is a dash pattern. ³³

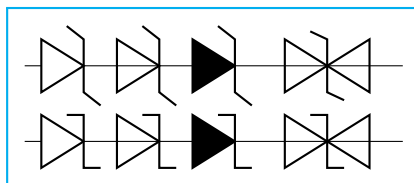
³²Thanks to the idea by [Dr. Matthias Jung on GitHub](#).

³³Follows the syntax of the pattern sequence `\pgfsetdash` — see TikZ manual [for details](#); phase is always zero. Basically you pass pairs of dash-length – blank-length dimensions, see the examples.



```
\begin{tikzpicture}
\newcommand{\optos}{%
  to[leD] ++(1.5,0) to[pD*] ++(1.5,0)
  to[lasD] ++(1.5,0)}
\begin{scope}
  \draw (0,2) \optos;
  \ctikzset{led arrows from cathode}
  \ctikzset{pd arrows to cathode}
  \ctikzset{opto arrows/.cd, color=red,
    dash={{1pt}{1pt}}}}
  \draw (0,0) \optos;
\end{scope}
\begin{scope}[color=blue, yshift=-6cm]
  \ctikzset{opto end arrow={Triangle[angle'=45]}}
  \ctikzset{opto start arrow={Hooks[red]}}
  \draw (0,4) \optos;
  \ctikzset{opto arrows/color=black}
  \ctikzset{opto arrows/relative thickness=2}
  \draw (0,2) \optos;
  \ctikzset{led arrows from cathode}
  \ctikzset{pd arrows to cathode}
  \draw (0,0) \optos;
\end{scope}
\end{tikzpicture}
```

4.4.3.2 Zener and TVS diode options You can change the shape of the Zener ZZener and TVS diodes using the flag `diode straight whiskers` (and reset it with the option `diode sloped whiskers`); this will change the “whiskers” of the diodes to be at a right angle³⁴ instead the sloped normal way.



```
\begin{tikzpicture}
\draw (0,1) to [zzDo] ++(1,0) to[zzD-] ++(1,0)
  to[zzD*] ++(1,0) to[tvsD-] ++(2,0);
\ctikzset{diode straight whiskers}
\draw (0,0) to [zzDo] ++(1,0) to[zzD-] ++(1,0)
  to[zzD*] ++(1,0) to[tvsD-] ++(2,0);
\end{tikzpicture}
```

4.5 Sources and generators

Notice that source and generators are divided in three classes that can be styled independently: traditional battery symbols (class `batteries`), independent generators (class `sources`) and dependent generators (class `csources`). This is because they are often treated differently, and so you can choose to, for example, fill the dependent sources but not the independent ones.

4.5.1 Batteries



battery: Battery, type: `path-style`, nodename: `batteryshape`. Class: `batteries`.

³⁴Suggested by [Dr. Matthias Jung](#)

	battery1: Single battery cell, type: <code>path-style</code> , nodename: <code>battery1shape</code> . Class: <code>batteries</code> .
	battery2: Single battery cell, type: <code>path-style</code> , nodename: <code>battery2shape</code> . Class: <code>batteries</code> .
	solar: Solar-driven battery (arrows follow the same style options as photodiodes, see 4.4.3.1) ³⁵ , type: <code>path-style</code> , nodename: <code>solarshape</code> . Class: <code>batteries</code> .
	baertty: Randall Munroe's baertty ³⁶ , type: <code>path-style</code> , nodename: <code>baerttyshape</code> . Class: <code>batteries</code> .

4.5.2 Stationary sources

	european voltage source: Voltage source (european style), type: <code>path-style</code> , fillable, nodename: <code>vsourceshape</code> . Aliases: <code>vsources</code> , <code>vsourcesEU</code> . Class: <code>sources</code> .
	cute european voltage source: Voltage source (cute european style), type: <code>path-style</code> , fillable, nodename: <code>vsourceshapeC</code> . Aliases: <code>vsourcesC</code> , <code>ceV</code> . Class: <code>sources</code> .
	american voltage source: Voltage source (american style), type: <code>path-style</code> , fillable, nodename: <code>vsourceshapeAM</code> . Aliases: <code>vsourcesAM</code> . Class: <code>sources</code> .
	european current source: Current source (european style), type: <code>path-style</code> , fillable, nodename: <code>isourceshape</code> . Aliases: <code>isources</code> , <code>isourcesEU</code> . Class: <code>sources</code> .
	cute european current source: Current source (cute european style), type: <code>path-style</code> , fillable, nodename: <code>isourceshapeC</code> . Aliases: <code>isourcesC</code> , <code>ceI</code> . Class: <code>sources</code> .
	american current source: Current source (american style), type: <code>path-style</code> , fillable, nodename: <code>isourceshapeAM</code> . Aliases: <code>isourcesAM</code> . Class: <code>sources</code> .

If (default behavior) `européancurrents` option is active (or the style `européan currents` is used), the shorthands `current source`, `isource`, and `I` are equivalent to `european current source`. Otherwise, if `americancurrents` option is active (or the style `[american currents]` is used) they are equivalent to `american current source`.

Similarly, if (default behavior) `européanvoltages` option is active (or the style `européan voltages` is used), the shorthands `voltage source`, `vsources`, and `V` are equivalent to `european voltage source`. Otherwise, if `americanvoltages` option is active (or the style `american voltages` is used) they are equivalent to `american voltage source`.

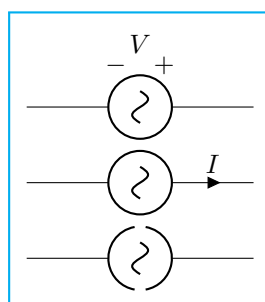
³⁵Contributed by [Dr. Matthias Jung](#).

³⁶[Mandatory xkcd](#)

4.5.3 Sinusoidal sources

These two are basically the same symbol; to distinguish among them, you have to add a label, which will be a voltage or a current. Another option would be to configure the `sinusoidal current source` as an open shape using `\ctikzset{bipoles/isourcesin/angle=80}` similar to the `dcisource` in section 4.5.8.

	sinusoidal voltage source: Sinusoidal voltage source, type: <code>path-style</code> , fillable, nodename: <code>vsourcesinshape</code> . Aliases: <code>vsourcesin</code> , <code>sV</code> . Class: <code>sources</code> .
	sinusoidal current source: Sinusoidal current source ³⁷ , type: <code>path-style</code> , fillable, nodename: <code>isourcesinshape</code> . Aliases: <code>isourcesin</code> , <code>sI</code> . Class: <code>sources</code> .

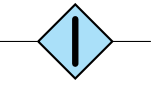
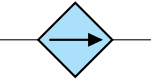
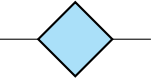


```
\begin{circuitikz}[american]
\draw (0,2) to[sV=$V$] ++(3,0);
\draw (0,1) to[sI=$I$] ++(3,0);
\ctikzset{bipoles/isourcesin/angle=80}
\draw (0,0) to[sI] ++(3,0);
\end{circuitikz}
```

4.5.4 Controlled sources

	European controlled voltage source: Controlled voltage source (European style), type: <code>path-style</code> , fillable, nodename: <code>cvsourcecshape</code> . Aliases: <code>cvsource</code> , <code>cvsourceEU</code> . Class: <code>csources</code> .
	cute European controlled voltage source: Voltage source (cute European style), type: <code>path-style</code> , fillable, nodename: <code>cvsourceCshape</code> . Aliases: <code>cvsourceC</code> , <code>cceV</code> . Class: <code>csources</code> .
	American controlled voltage source: Controlled voltage source (American style), type: <code>path-style</code> , fillable, nodename: <code>cvsourceAMshape</code> . Aliases: <code>cvsourceAM</code> . Class: <code>csources</code> .
	European controlled current source: Controlled current source (European style), type: <code>path-style</code> , fillable, nodename: <code>cisourcecshape</code> . Aliases: <code>cisource</code> , <code>cisourceEU</code> . Class: <code>csources</code> .

³⁷The configurable open shape of the `sinusoidal current source` has been added by [Maximilian Martin](#)

	cute european controlled current source: Current source (cute european style), type: <code>path-style</code> , fillable, nodename: <code>cisourceCshape</code> . Aliases: <code>cisourceC</code> , <code>cceI</code> . Class: <code>csources</code> .
	american controlled current source: Controlled current source (american style), type: <code>path-style</code> , fillable, nodename: <code>cisourceAMshape</code> . Aliases: <code>cisourceAM</code> . Class: <code>csources</code> .
	empty controlled source: Empty controlled source, type: <code>path-style</code> , fillable, nodename: <code>ecsourceshape</code> . Aliases: <code>ecsource</code> . Class: <code>csources</code> .

If (default behavior) `européancurrents` option is active (or the style `européan currents` is used), the shorthands `controlled current source`, `cisource`, and `cI` are equivalent to `européan controlled current source`. Otherwise, if `americancurrents` option is active (or the style `american currents` is used) they are equivalent to `american controlled current source`.

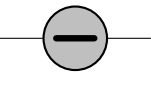
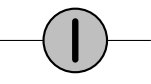
Similarly, if (default behavior) `européanvoltages` option is active (or the style `européan voltages` is used), the shorthands `controlled voltage source`, `cvsource`, and `cV` are equivalent to `européan controlled voltage source`. Otherwise, if `americanvoltages` option is active (or the style `american voltages` is used) they are equivalent to `american controlled voltage source`.

The following two behave like the corresponding independent sources, see section 4.5.3.

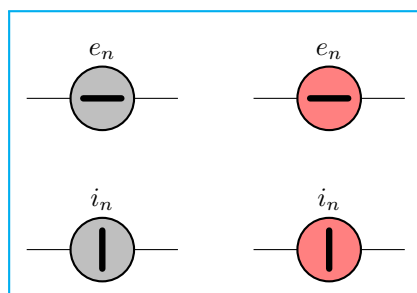
	controlled sinusoidal voltage source: Controlled sinusoidal voltage source, type: <code>path-style</code> , fillable, nodename: <code>cvsourcesinshape</code> . Aliases: <code>controlled vsourcesin</code> , <code>cvsourcesin</code> , <code>csV</code> . Class: <code>csources</code> .
	controlled sinusoidal current source: Controlled sinusoidal current source, type: <code>path-style</code> , fillable, nodename: <code>cisourcesinshape</code> . Aliases: <code>controlled isourcesin</code> , <code>cisourcesin</code> , <code>csI</code> . Class: <code>csources</code> .

4.5.5 Noise sources

In this case, the “direction” of the source is undefined. Noise sources are filled in gray by default, but if you choose the dashed style, they become fillable.

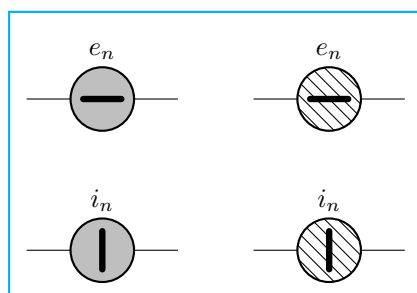
	noise voltage source: Sinusoidal voltage source, type: <code>path-style</code> , nodename: <code>vsourcenshape</code> . Aliases: <code>vsourcen</code> , <code>nV</code> . Class: <code>sources</code> .
	noise current source: Sinusoidal current source, type: <code>path-style</code> , nodename: <code>isourcenshape</code> . Aliases: <code>isourcen</code> , <code>nI</code> . Class: <code>sources</code> .

You can change the fill color with the key `circuitikz/bipoles/noise sources/fillcolor`:



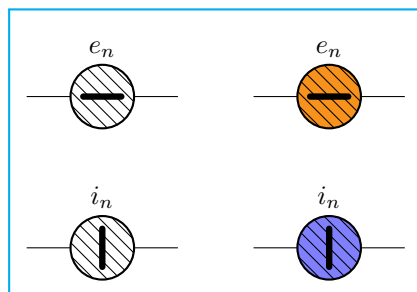
```
\begin{circuitikz}
\draw(0,0) to [nV, l=$e_n$] ++(2,0);
\draw(0,-2) to [nI, l=$i_n$] ++(2,0);
\begin{scope}[circuitikz/bipoles/noise sources/
fillcolor=red!50]
\draw(3,0) to [nV, l=$e_n$] ++(2,0);
\draw(3,-2) to [nI, l=$i_n$] ++(2,0);
\end{scope}
\end{circuitikz}
```

If you prefer a patterned noise generator (similar to the one you draw by hand) you can use the fake color `dashed`:



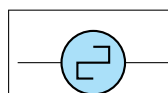
```
\begin{circuitikz}
\draw(0,0) to [nV, l=$e_n$] ++(2,0);
\draw(0,-2) to [nI, l=$i_n$] ++(2,0);
\begin{scope}[circuitikz/bipoles/noise sources/
fillcolor=dashed]
\draw(3,0) to [nV, l=$e_n$] ++(2,0);
\draw(3,-2) to [nI, l=$i_n$] ++(2,0);
\end{scope}
\end{circuitikz}
```

Notice that if you choose the dashed style, the noise sources are fillable:

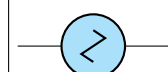


```
\begin{circuitikz}
\ctikzset{bipoles/noise sources/fillcolor=dashed}
\draw(0,0) to [nV, l=$e_n$] ++(2,0);
\draw(0,-2) to [nI, l=$i_n$] ++(2,0);
\begin{scope}
\draw(3,0) to [nV, l=$e_n$, fill=yellow!50!red] ++(2,0);
\draw(3,-2) to [nI, l=$i_n$, fill=blue!50!white] ++(2,0);
\end{scope}
\end{circuitikz}
```

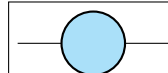
4.5.6 Special sources



square voltage source: Square voltage source, type: path-style, fillable, nodename: `vsourcesquashape`. Aliases: `vsourcesquare`, `sqV`. Class: `sources`.



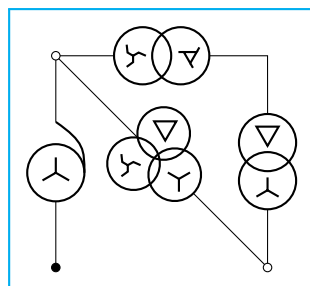
vsourcetri: Triangle voltage source, type: path-style, fillable, nodename: `vsourcetrishape`. Aliases: `tV`. Class: `sources`.



esource: Empty voltage source, type: path-style, fillable, nodename: `esourceshape`. Class: `sources`.

	pvsources: Photovoltaic-voltage source, type: path-style, fillable, nodename: pvsourceshape. Class: sources.
	pvmodule: Photovoltaic module source ³⁸ , type: path-style, fillable, nodename: pvmoduleshape. Class: sources.
	ioosources: Double Zero style current source, type: path-style, fillable, nodename: ioosourceshape. Class: sources.
	voosources: Double Zero style voltage source, type: path-style, fillable, nodename: voosourceshape. Class: sources.
	oosourceatrans: auto-transformer source ³⁹ , type: path-style, fillable, nodename: oosourceatransshape. Class: sources.
	oosourcetrans: transformer source ⁴⁰ , type: path-style, fillable, nodename: oosourcetransshape. Class: sources.
	ooosources: transformer with three windings ⁴¹ , type: path-style, fillable, nodename: ooosourceshape. Class: sources.

The transformer shapes vector group options can be specified for the primary (**prim=value**), the secondary (**sec=value**) and tertiary (**tert=value**) three-phase vector groups: the value can be one of delta, wye, eyw⁴², xdelta⁴³, and zig. The relative size of the inner triangle of the xdelta symbol can be changed with the key **sources/symbol/xdelta split/** (default value 0.58).



```
\begin{circuitikz}[scale=1.4]
\ctikzset{sources/scale=1.5}
\draw (0,0) to[oosourcetrans,
prim=zig, sec=xdelta, o-] ++(2,0)
to[oosourcetrans, prim=delta, sec=wye,-o] ++(0,-2)
to[ooosource, prim=eyw, sec=zig, tert=delta] (0,0)
to[oosourceatrans, prim=wye, -*] ++(0,-2);
\end{circuitikz}
```

³⁸Added by André Alves in v1.3.5

³⁹The **oosourceatrans** component has been suggested by [Max Börjesson on GitHub](#)

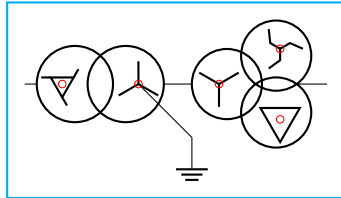
⁴⁰The **oosourcetrans** and **ooosources** components have been added by [user @olffline on GitHub](#)

⁴¹The component here is scaled up 1.5 times to better show the anchors.

⁴²The **eyw** symbol was suggested by [Jakob Leide on GitHub](#)

⁴³The **xdelta** symbol was suggested by [user @sputeanus on GitHub](#)

These two “sources” have additional anchors that reach the center of the symbol;⁴⁴ they are used sometimes to add a (symbolic) connection there, like for example a ground connection.



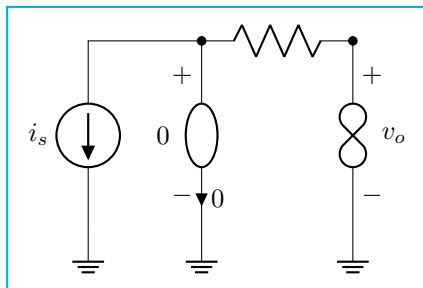
```
\begin{tikzpicture}[circuitikz/sources/scale=2,
  smalldot/.style={draw, circle,red, inner sep=1pt}]
\draw (0,0) to[oosourcetrans, name=A,
  prim=xdelta, sec=wye] ++(2,0)
  to[oosource, name=B, prim=eyw, sec=zig,
  tert=delta] ++(2,0)
  (A.symbolsec) -- ++(-45:1) node[ground]{};
\node [smalldot] at (A.symbolprim) {};
\node [smalldot] at (A.symbolsec) {};
\node [smalldot] at (B.symbolprim) {};
\node [smalldot] at (B.symbolsec) {};
\node [smalldot] at (B.symboltert) {};
\end{tikzpicture}
```

4.5.7 Nullator and norator

These are special elements used in some approaches to model ideal amplifiers⁴⁵.

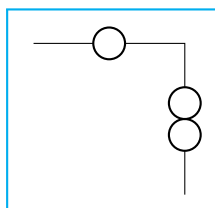
	nullator: Nullator element (virtual short circuit; forces V and I to zero), type: path-style, fillable, nodename: nullatorshape. Class: sources.
	norator: Norator element (admits any combination of V and I), type: path-style, fillable, nodename: noratorshape. Class: sources.

They are in the `sources` class, but they are not treated like sources in the labeling sense (they have both `bipoles/is voltage=false` and `bipoles/is current=false`, see 5.2.2).



```
\begin{circuitikz}[american]
\draw (0,0) node[ground] (GND){}
  to [I,l=$i_s$,invert]
  ++(0,2.5) -- ++(1.5,0) coordinate(up)
  to [nullator, v=0, i=0, name=A]
  (up|-GND) node[ground]{};
\draw (up) to [R, *-] ++(2,0) coordinate(out)
  to [norator, v^=$v_o$]
  (out|-GND) node[ground]{};
\end{circuitikz}
```

The symbol shapes used here seems to be the most common in publications; if you prefer the shapes from the Wikipedia article, you can use the following definitions:



```
\tikzset{noratorW/.style={voosource,
  bipoles/oosource/width=0.6,
  bipoles/oosource/height=0.3},
nullatorW/.style={esource, sources/scale=0.5}}
\begin{tikzpicture}
\draw (0,0) to [nullatorW] ++(2,0) to [noratorW] ++(0, -2);
\end{tikzpicture}
```

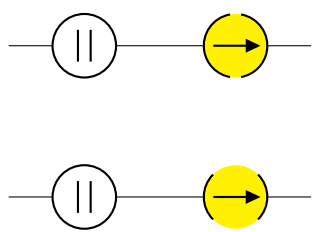
⁴⁴Suggested by [user @lapreindl on GitHub](#).

⁴⁵See [the Wikipedia article](#); suggested by [user atticus-sullivan on GitHub](#).

4.5.8 DC sources

	dcvsource: DC voltage source, type: <code>path-style</code> , fillable, nodename: <code>dcvsource</code> shape. Class: <code>sources</code> .
	dcisource: DC current source, type: <code>path-style</code> , fillable, nodename: <code>dcisource</code> shape. Class: <code>sources</code> .

The size of the broken part of the DC current source is configurable by changing the value of `bipoles/dcisource/angle` (default 80); values must be between 0 (no circle at all, probably not useful) and 90 (full circle, again not useful).

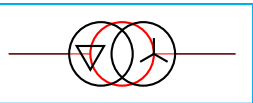


```
\begin{circuitikz}
  \draw (0,0) to[dcvsource] ++(2,0)
    to [dcisource, fill=yellow] ++(2,0) ;
  \ctikzset{bipoles/dcisource/angle=45}
  \draw (0,-2) to[dcvsource] ++(2,0)
    to [dcisource, fill=yellow] ++(2,0) ;
\end{circuitikz}
```

4.5.9 Sources customizations

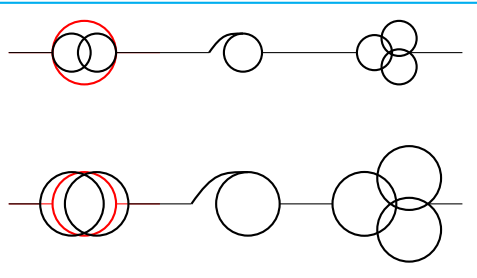
4.5.9.1 Size. You can change the scale of the batteries by setting the key `batteries/scale`, for the controlled (dependent) sources with `cources/scale`, and for all the other independent sources and generators with `sources/scale`, to something different from the default 1.0.

Notice that the size of the double-circle sources (and of the triple-circle one) are tuned so that the full source occupy more or less the same horizontal space than one of the single-circle one; as a consequence, the circles are much smaller. If you want to have the same circle radius, you have to scale (locally!) those sources by one factor that is 1.5384 (1/0.65) for `oosource`, 1.6667 (1/0.6) for `oosourceatrans`, and 1.8182 (1/0.55) for `oosource`.



```
\begin{circuitikz}
  \draw[color=red] (0,0) to[esource] ++(3,0);
  \draw (0,0) to[oosourceatrans, prim=delta, sec=wye,
    sources/scale=1.667] ++(3,0);
\end{circuitikz}
```

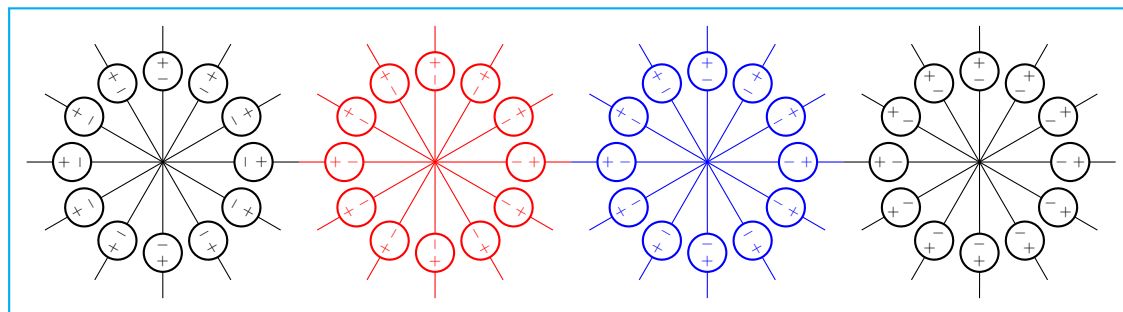
Alternatively, you can use the key `ootrans size` (which has possible values `small` and `large`, default `small`) to change the sizes of `oosourceatrans`, `oosourceatrans`, and `oosource` so that they match the size of a normal source.



```
\begin{circuitikz}
  \draw[color=red] (0,2) to[esource] ++(2,0);
  \draw (0,2) to[oosourceatrans] ++(2,0)
    to [oosourceatrans] ++(2,0)
    to [oosource] ++(2,0);
  \ctikzset{ootrans size=large}
  \draw[color=red] (0,0) to[esource] ++(2,0);
  \draw (0,0) to[oosourceatrans] ++(2,0)
    to [oosourceatrans] ++(2,0)
    to [oosource] ++(2,0);
\end{circuitikz}
```

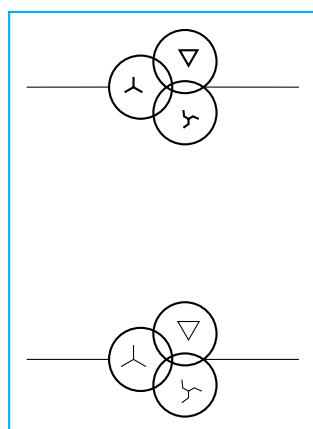

You can obtain several different rotations or rotation modes by changing the value of the key `sources/symbol/sign rotation` (with `\ctikzset`). The value `default` (also the default value!) uses the legacy, “rigid” position.

If you provide a number, the symbols are rotated by that value (0 means that the minus sign is aligned with the wire, 90 is very similar to `default`⁴⁸). If you use `straight`, the symbols are rotated to be always horizontal. With `auto`, the symbols are drawn in the same way as 0 for inclination less the 45°, and as 90 otherwise. The following drawing shows the results for several different parameter values.



```
\begin{tikzpicture}[american, scale=0.6, transform shape]
  %\ctikzset{sources/symbol/sign rotation=default}
  \foreach \a in {0,30,...,359} \draw (0,0) -- ++(\a:1) to[V] ++(\a:2);
  \ctikzset{sources/symbol/sign rotation=0}
  \foreach \a in {0,30,...,359} \draw[red] (6,0) -- ++(\a:1) to[V] ++(\a:2);
  \ctikzset{sources/symbol/sign rotation=auto}
  \foreach \a in {0,30,...,359} \draw[blue] (12,0) -- ++(\a:1) to[V] ++(\a:2);
  \ctikzset{sources/symbol/sign rotation=straight}
  \foreach \a in {0,30,...,359} \draw (18,0) -- ++(\a:1) to[V] ++(\a:2);
\end{tikzpicture}
```

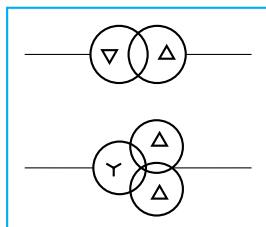
4.5.9.5 Three-phase symbols. The three-phase symbols `delta`, `wye`, `eyw`, and `zig` follows the line thickness exactly as the waveform ones (see above). Additionally, you can scale them up and down by changing the value of the keys `sources/symbol/delta scale`, `.../wye scale`, `.../eyw scale`, and `.../zig scale` (default 1).



```
\begin{circuitikz}[scale=1.8, transform shape]
  \tikzset{myoosource/.style={oosource,
    prim=wye, sec=delta, tert=zig,
  }}
  \draw (0,2) to[myoosource] ++(2,0);
  \ctikzset{%
    sources/symbol/thickness=0.5,
    sources/symbol/delta scale=1.2,
    sources/symbol/wye scale=1.4,
    sources/symbol/zig scale=1.3,
  }
  \draw (0,0) to[myoosource] ++(2,0);
\end{circuitikz}
```

⁴⁸Not exactly equal; the position of the symbols could be slightly different depending on the font.

4.5.9.6 Rotating the three-phase symbols. You have several keys to rotate, locally or globally, the three-phase symbols⁴⁹. The symbols can be rotated (with effect on every symbol of that type) with the key `sources/symbol/delta rot` and similar ones for `eyw`, `wye` and `zig`. Additionally, you can rotate the symbol on a specific winding by using the key `sources/symbol/prim rot` and similar ones for `sec`, `tert` and `upper`.

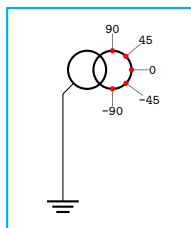


```
\begin{circuitikz}[scale=1.5, transform shape]
  \draw (0,0) to[oosourcetrans, prim=delta, sec=delta,
sources/symbol/sec rot=180] ++(2,0);
  \draw (0,-1) to[oosource, prim=eyw, sec=delta,
tert=delta, sources/symbol/delta rot=180] ++(2,0);
\end{circuitikz}
```

4.5.10 Source borders

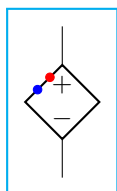
Unfortunately, the border of the sources is only easily accessed if some anchor is provided. The border anchors of the shapes are not tight on them (see section 3.1.2), which is not easily changeable, given that the algorithm that positions the labels depends on it.

On the other hand, TikZ powerful [partway coordinate calculation](#) (around section 13.5.3 of the manual) makes it possible to easily identify points on a circle if the center and one point of the circle are known, as you can see in the following example.



```
\begin{tikzpicture}[]
  \path (0,0) to[oosourcetrans, name=T, ]++(2,0);
  % Use partway modifiers to reach a point on the left circle
  \draw ($(T.centerprim)!1!45:(T.left)$) -- ++(-135:0.2)
    -- ++(0,-1) node[ground]{};
  \begin{scope}[font=\tiny\ttfamily, pin distance=2mm, inner sep=0pt]
    \foreach \a in {-90,-45,...,90}
      \node [circ, scale=0.5, pin=\a:\a, color=red] at
        ($(T.centersec)!1!\a:(T.right)$){};
  \end{scope}
\end{tikzpicture}
```

A similar approach can be used for dependent sources. Just remember that the anchors move (rotate) together with the component.



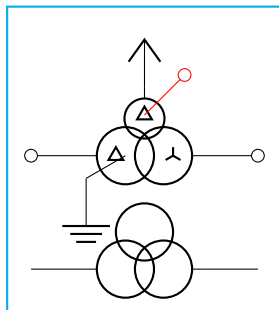
```
\begin{tikzpicture}[american]
  \draw (0,0) to [cvsource, name=S] ++(0,2);
  \node [circ,red] at ($(S.e)!0.3333!(S.n)$) {};
  \node [circ,blue] at ($(S.e)!0.6666!(S.n)$) {};
\end{tikzpicture}
```

4.5.10.1 Additional winding. The `oosourcetrans` and `oosourceatrans` components admit an additional winding, called *upper winding*⁵⁰. It can be added by specifying the key `oo upper winding`; you can pass an additional parameter specifying the size (relative to the symbol circle size) of the added circle. The default is 0.7, but even the default can be changed by setting the key `bipoles/oosourcetrans/upper circle size default`. The height of the additional winding

⁴⁹Suggested by Jakob Leide

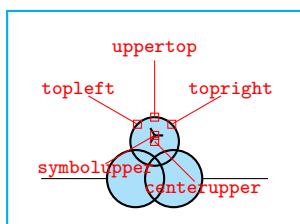
⁵⁰Suggested by Jakob Leide

can be tweaked with the key `bipoles/oosourcetrans/upper circle offset` (default 1.3, which tries to mimic the shape of `oosource`).

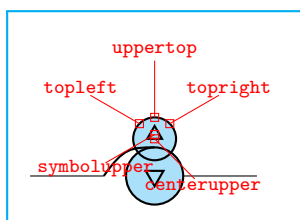


```
\begin{circuitikz}[scale=1.5, transform shape]
\draw (0,0) to[oosourcetrans, prim=delta, o-o, name=a,
sources/symbol/delta rot=180,
oo upper winding, upper=delta, sec=wye,
] ++(2,0);
\draw (a.centerprim) -- ++(210:0.4) node[ground]{};
\draw (a.uppertop) -- ++(90:0.1) node[vcc]{};
\draw [red] (a.symbolupper) to[short,-o] ++(45:0.5);
\draw (0,-1) to[oosourcetrans, oo upper winding=1
] ++(2,0);
\end{circuitikz}
```

4.5.10.2 Additional winding anchors. When using the upper winding, you have several additional anchors available.



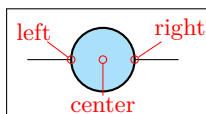
```
\begin{circuitikz}[scale=1.5, transform shape,
smallsquare/.style={red, draw, inner sep=1pt}]
\draw (0,0) to[oosourcetrans, oo upper winding=0.85,
name=a, fill=cyan!30,upper=wye,
sources/symbol/upper rot=-90] ++(2,0);
\foreach\anc/\ang in {centerupper/-45, uppertop/90,
topright/30, topleft/150, symbolupper/210}
\draw[red] (a.\anc) node[smallsquare]{} -- ++(\ang:0.5)
++(\ang:0.1) node[font=\tiny\ttfamily,]{\anc};
\end{circuitikz}
```



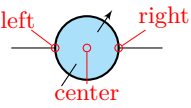
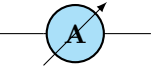
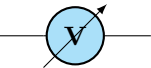
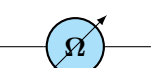
```
\begin{circuitikz}[scale=1.5, transform shape,
smallsquare/.style={red, draw, inner sep=1pt}]
\draw (0,0) to[oosourcetrans, oo upper winding=0.75,
name=a, fill=cyan!30,prim=delta,upper=delta,
sources/symbol/upper rot=180] ++(2,0);
\foreach\anc/\ang in {centerupper/-45, uppertop/90,
topright/30, topleft/150, symbolupper/210}
\draw[red] (a.\anc) node[smallsquare]{} -- ++(\ang:0.5)
++(\ang:0.1) node[font=\tiny\ttfamily,]{\anc};
\end{circuitikz}
```

4.6 Instruments

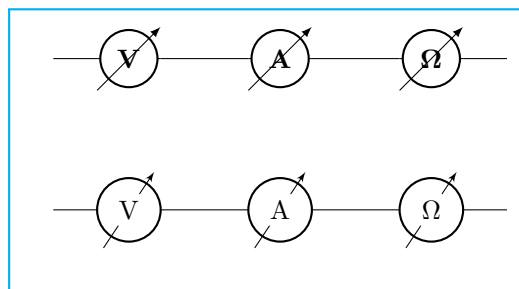
4.6.1 Basic round instruments



rmeter: Round meter (use `t=...` for the symbol), type: path-style, fillable, nodename: `rmetershape`. Class: `instruments`.

	rmeterwa : Round meter with arrow (use <code>t=...</code> for the symbol), type: <code>path-style</code> , fillable, nodename: <code>rmeterwashape</code> . Class: <code>instruments</code> .
	ammeter : Legacy ammeter, type: <code>path-style</code> , fillable, nodename: <code>ammetershape</code> . Class: <code>instruments</code> .
	voltmeter : Legacy voltmeter, type: <code>path-style</code> , fillable, nodename: <code>voltmetershape</code> . Class: <code>instruments</code> .
	ohmmeter : Legacy ohmmeter, type: <code>path-style</code> , fillable, nodename: <code>ohmmetershape</code> . Class: <code>instruments</code> .

You can define styles if you want to use the new shapes for round instrument similarly to the legacy ones:⁵¹

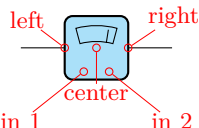
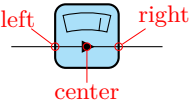
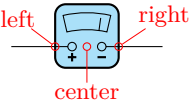


```
\tikzset{vmeter/.style={rmeterwa, t=V}}
\tikzset{ameter/.style={rmeterwa, t=A}}
\tikzset{ometer/.style={rmeterwa, t=$\Omega$}}

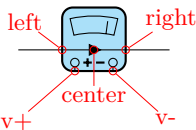
\begin{tikzpicture}
  % Old meter style
  \draw (0,2) to[voltmeter] ++(2,0)
    to[ammeter] ++(2,0)
    to[ohmmeter] ++(2,0);
  % New meter style
  \draw (0,0) to[vmeter] ++(2,0)
    to[ameter] ++(2,0)
    to[ometer] ++(2,0);
\end{tikzpicture}
```

4.6.2 Square instruments

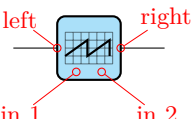
Sometimes it is better to use a shape for instruments which is very different from the round symbols used for sources.

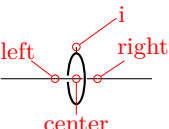
	smeter : Square meter (use <code>t=...</code> for the symbol), type: <code>path-style</code> , fillable, nodename: <code>smetershape</code> . Class: <code>instruments</code> .
	qiprobe : QUCS-style current probe, type: <code>path-style</code> , fillable, nodename: <code>qiprobeshape</code> . Class: <code>instruments</code> .
	qvprobe : QUCS-style voltage probe, type: <code>path-style</code> , fillable, nodename: <code>qvprobeshape</code> . Class: <code>instruments</code> .

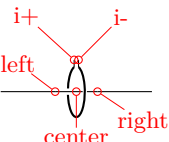
⁵¹Suggested by [user mxxmxn on GitHub](#).

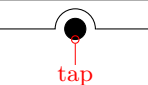
	qpprobe: QUCS-style power probe, type: <code>path-style</code> , fillable, nodename: <code>qpprobeshape</code> . Class: <code>instruments</code> .
---	---

4.6.3 Oscilloscopes and current probes

	oscope: Oscilloscope ⁵² , type: <code>path-style</code> , fillable, nodename: <code>oscopeshape</code> . Class: <code>instruments</code> .
---	--

	iloop: Current loop (symbolic), type: <code>path-style</code> , nodename: <code>iloopshape</code> . Class: <code>instruments</code> .
---	--

	iloop2: Current loop (real), type: <code>path-style</code> , nodename: <code>iloop2shape</code> . Class: <code>instruments</code> .
--	--

	currtap: Current tap (probe) ⁵³ , type: <code>path-style</code> , nodename: <code>currtapshape</code> . Class: <code>instruments</code> .
---	---

4.6.4 Instruments customizations

You can change the scale of all the instruments (including the current loops) by setting the key `instruments/scale` to something different from the default 1.0.

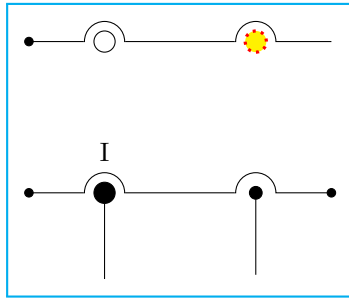
4.6.4.1 Current probes. You can change the inner dot in several way, by changing the following keys under the `\ctikzset` key `bipoles/currtap`:

parameter	default	description
thickness	default	Set the thickness of the line (default : do not change the class thickness)
color	default	stroke color: default is the same as the component
dash	default	dash pattern: none means solid line, default means keep the global pattern ⁵⁴
fill	default	fill the inner dot; default means use the wire color, none do not fill, other keys must be a valid color
dot size	0.5	relative size of the inner dot

⁵²Suggested by @nobrl on GitHub

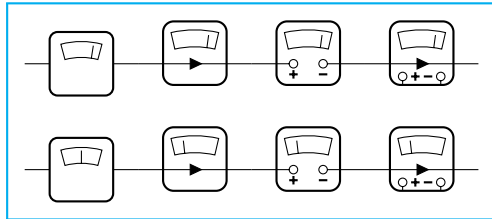
⁵³Suggested by user EEpchi on GitHub.

⁵⁴Follows the syntax of the pattern sequence `\pgfsetdash` — see TikZ manual for details; phase is always zero. Basically you pass pairs of dash-length – blank-length dimensions, see the examples.



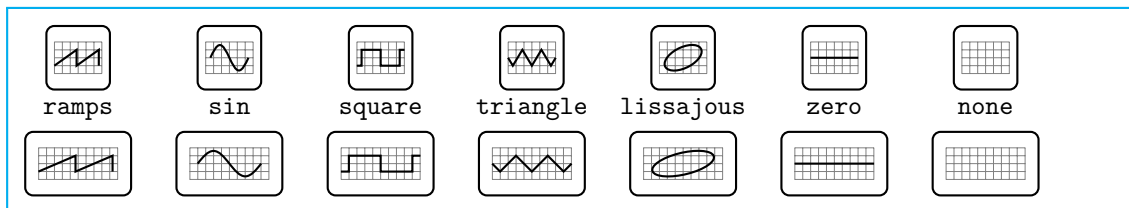
```
\begin{circuitikz}
  \draw (0,2) to[currtrap, bipoles/currtrap/!fill=none, *-]
    ++(2,0) to[currtrap, bipoles/currtrap/.cd,
      fill=yellow, color=red, thickness=3,
      dash={1.14pt}{2pt}] ++(2,0);
  \draw (0,0) to[currtrap=I, *-, name=ct] ++(2,0)
    to[currtrap, *-, name=ct2,
      bipoles/currtrap/dot size=0.3] ++(2,0);
  \draw (ct.tap) -- ++(0,-1) (ct2.tap) -- ++(0,-1);
\end{circuitikz}
```

4.6.4.2 Square instruments index position. You can change the index position of square instruments with the key `bipoles/smeter/index position` (for `smeter`) or `bipoles/qmeter/index position` (for `qprobe`, `qvprobe` and `qprobe`). Use a value between 0 and 100 (the default is 80).



```
\begin{circuitikz}
  % Default
  \foreach [count=\i] \inst in
    {smeter, qprobe, qvprobe, qprobe} {
    \draw ({1.5*\i},1.4) to[\inst]
      ({1.5*(\i+1)},1.4);
  }
  % New indexes
  \ctikzset{bipoles/smeter/index position=50}
  \ctikzset{bipoles/qmeter/index position=25}
  \foreach [count=\i] \inst in
    {smeter, qprobe, qvprobe, qprobe} {
    \draw ({1.5*\i},0) to[\inst]
      ({1.5*(\i+1)},0);
  }
\end{circuitikz}
```

4.6.4.3 Oscilloscope waveform. You can change the waveform shown in the oscilloscope “screen”⁵⁵. To change it, you just set the key `bipoles/oscope/waveform` to one of the available shape. You have available the shapes in the following list (the default is `ramps`):



```
\begin{circuitikz}
  \foreach [count=\i] \wvf in {ramps, sin, square, triangle, lissajous, zero, none} {
    \ctikzset{bipoles/oscope/waveform=\wvf}
    \draw ({2*\i},1.4) node[oscopecshape](0){}
      ({2*\i},0.65) node[anchor=base]{\texttt{\wvf}};
  }
  \ctikzset{bipoles/oscope/width=1.0}
\end{circuitikz}
```

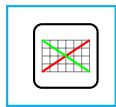
⁵⁵Suggested by [Mario Tafur on TeX.SX](#)

```

\foreach [count=\i] \wvf in {ramps, sin, square, triangle, lissajous, zero, none} {
  \ctikzset{bipoles/oscope/waveform=\wvf}
  \draw ({2*\i},0) node[oscopeshape]{};
}
\end{circuitikz}

```

If you want more or different shapes, you can define your owns, but you have to use low-level **pgf** commands (see part IX, “[The Basic Layer](#)”, in the PGF/TikZ manual). The code is executed into a `\pgfscope ... \endpgfscope` environment, and it must use the path built with a `\pgfusepath`. The coordinates have been scaled so that the external box of the scope is a rectangle between $(-1\text{cm}, -1\text{cm})$ and $(1\text{cm}, 1\text{cm})$; the oscilloscope grid is fixed and painted between $(-0.75\text{cm}, -0.5\text{cm})$ and $(0.75\text{cm}, 0.5\text{cm})$. If you stretch the scope with the `...width` or `...height` keys, the drawing will be stretched too.



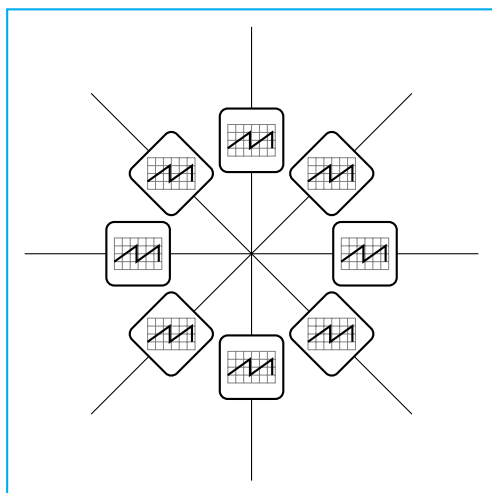
```

\ctikzset{%
  bipoles/oscope/waveform/mywave/.code={%
    \pgfsetcolor{red}
    \pgfpathmoveto{\pgfpoint{-0.75cm}{-0.5cm}}
    \pgfpathlineto{\pgfpoint{0.75cm}{0.5cm}}
    \pgfusepath{draw}
    \pgfsetcolor{green}
    \pgfpathmoveto{\pgfpoint{-0.75cm}{0.5cm}}
    \pgfpathlineto{\pgfpoint{0.75cm}{-0.5cm}}
    \pgfusepath{draw}
  }}
\begin{circuitikz}
  \ctikzset{bipoles/oscope/waveform=mywave}
  \draw (0,0) node[oscopeshape]{};
\end{circuitikz}

```

4.6.5 Rotation-invariant elements

The **oscope** element will not rotate the “graph” shown with the component:



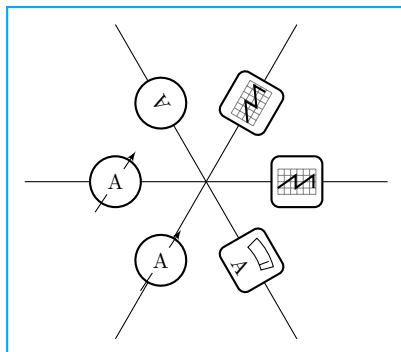
```

\begin{circuitikz}
  \foreach \a in {0,45,...,350} {
    \draw (0,0) to[oscope] (\a:3);
  }
\end{circuitikz}

```

The **rmeter**, **rmaterwa**, and **smeter** have the same behavior.

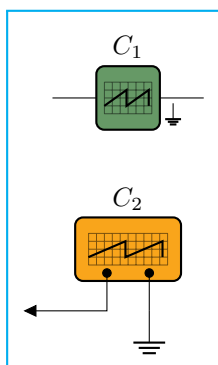
However, if you prefer that the **oscope**, **rmeter**, **smeter** and **rmeterwa** instruments rotate the text or the diagram, you can use the key or style **rotated instruments** (the default style is **straight instruments**).



```
\begin{circuitikz}[scale=0.8, transform shape]
\ctikzset{rotated instruments} % new default
\draw (0,0) to[oscope] ++(0:3);
\draw (0,0) to[oscope] ++(60:3);
\draw (0,0) to[rmeter, t=A] ++(120:3);
% local override
\draw (0,0) to[rmeterwa, t=A, straight instruments]
++(180:3);
\ctikzset{straight instruments} % back to default
\draw (0,0) to[rmeterwa, t=A] ++(240:3);
% local override
\draw (0,0) to[smeter, t=A, rotated instruments]
++(300:3);
\end{circuitikz}
```

4.6.6 Instruments as node elements

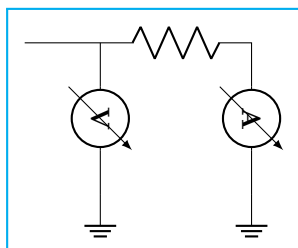
The node-style usage of the `oscope` is also interesting, using the additional `in 1` and `in 2` anchors; notice that in this case you can use the text content of the node to put labels above it. Moreover, you can change the size of the oscilloscope by changing `bipoles/oscope/width` and `bipoles/oscope/height` (which both default to 0.6).



```
\begin{circuitikz}
\draw (0,1)
to[oscope=$C_1$, fill=green!20!gray, name=01] ++(2,0);
\path (01.right)
node[ground, scale=0.5, below right=4pt]{};
\ctikzset{bipoles/oscope/width=1.0}
\draw (1,-1)
node[oscoshape, fill=yellow!20!orange] (02){$C_2$};
\draw (02.in 2) to[short, *-] ++(0,-0.5) node[ground]{};
\draw (02.in 1) to[short, *-] ++(0,-0.5)
-- ++(-1,0) node[currarrow, xscale=-1]{};
\end{circuitikz}
```

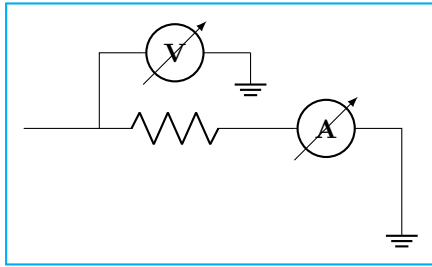
4.6.7 Measuring voltage and currents, multiple ways

This is the classical (legacy) option, with the `voltmeter` and `ammeter`. The problem is that elements are intrinsically horizontal, so they look funny if put in vertically.



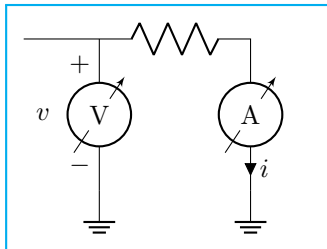
```
\begin{circuitikz}
\draw (0,0) -- ++(1,0) to[R] ++(2,0)
to [ammeter] ++(0,-2) node[ground]{};
\draw (1,0) to[voltmeter] ++(0,-2)
node[ground]{};
\end{circuitikz}
```

So the solution is often changing the structure to keep the meters in horizontal position.



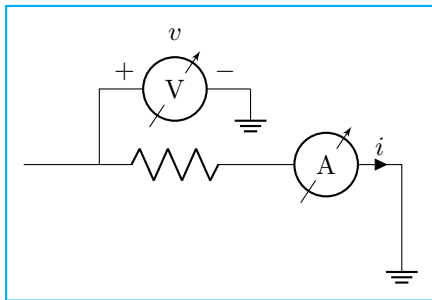
```
\begin{circuitikz}
  \draw (0,0) -- ++(1,0) to[R] ++(2,0)
  to [ammeter] ++(2,0) --
  ++(0,-1) node[ground]{};
  \draw (1,0) -- (1,1) to[voltmeter]
  ++(2,0) node[ground]{};
\end{circuitikz}
```

Since version 0.9.0 you have more options for the measuring instruments. You can use the generic **rmeterwa** (round meter with arrow), to which you can specify the internal symbol with the option **t=...** (and is fillable).



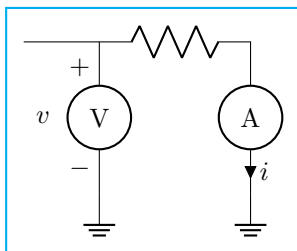
```
\begin{circuitikz}[american]
  \draw (0,0) -- ++(1,0) to[R] ++(2,0)
  to [rmeterwa, t=A, i=$i$] ++(0,-2) node[ground]{};
  \draw (1,0) to[rmeterwa, t=V, v=$v$] ++(0,-2)
  node[ground]{};
\end{circuitikz}
```

This kind of component will keep the symbol horizontal, whatever the orientation:



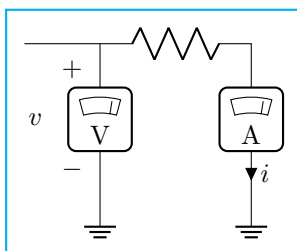
```
\begin{circuitikz}[american]
  \draw (0,0) -- ++(1,0) to[R] ++(2,0)
  to [rmeterwa, t=A, i=$i$] ++(2,0) --
  ++(0,-1) node[ground]{};
  \draw (1,0) -- (1,1) to[rmeterwa, t=V, v^=$v$]
  ++(2,0) node[ground]{};
\end{circuitikz}
```

The plain **rmeter** is the same, without the measuring arrow:



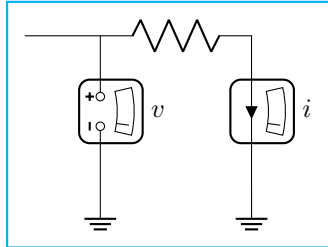
```
\begin{circuitikz}[american]
  \draw (0,0) -- ++(1,0) to[R] ++(2,0)
  to [rmeter, t=A, i=$i$] ++(0,-2) node[ground]{};
  \draw (1,0) to[rmeter, t=V, v=$v$] ++(0,-2)
  node[ground]{};
\end{circuitikz}
```

If you prefer it, you have the option to use square meters, in order to have more visual difference from generators:



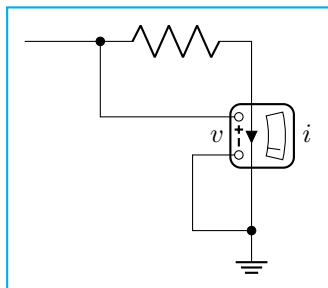
```
\begin{circuitikz}[american]
  \draw (0,0) -- ++(1,0) to[R] ++(2,0)
  to [smeter, t=A, i=$i$] ++(0,-2) node[ground]{};
  \draw (1,0) to[smeter, t=V, v=$v$] ++(0,-2)
  node[ground]{};
\end{circuitikz}
```

Another possibility is to use QUCS⁵⁶-style probes, which have the nice property of explicitly showing the type of connection (in series or parallel) of the meter:



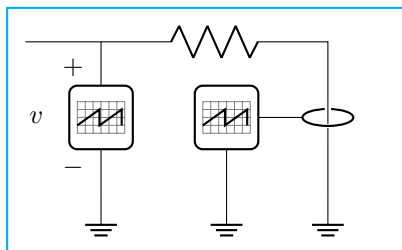
```
\begin{circuitikz}[american]
\draw (0,0) -- ++(1,0) to[R] ++(2,0)
to [qiprobe, l=$i$] ++(0,-2) node[ground]{};
\draw (1,0) to[qvprobe, l=$v$] ++(0,-2)
node[ground]{};
\end{circuitikz}
```

If you want to explicitly show a power measurement, you can use the power probe `qpprobe` and using the additional anchors `v+` and `v-` :



```
\begin{circuitikz}[american]
\draw (0,0) to[short,-*] ++(1,0) coordinate(b)
to[R] ++(2,0) to [qpprobe, l=$i$, a=$v$, name=P]
++(0,-2.5) node[ground] (GND){};
\draw (P.v-) -| ++(-0.5,-1) coordinate(a)
to [short, -*] (a|GND);
\draw (P.v+) -| (b);
\end{circuitikz}
```

The final possibility is to use oscilloscopes. For example:

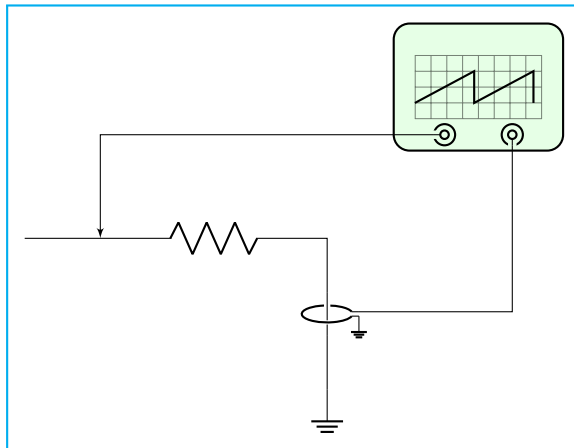


```
\begin{circuitikz}[american]
\draw (0,0) -- ++(1,0) to[R] ++(3,0)
to [iloop, mirror, name=I] ++(0,-2)
node[ground] (GND){};
\draw (1,0) to[oscope, v=$v$] ++(0,-2)
node[ground]{};
\draw (I.i) -- ++(-0.5,0) node[oscopeshape, anchor=
right, name=O]{};
\draw (O.south) -- (O.south |- GND) node[ground]{};
\end{circuitikz}
```

Or, if you want a more physical structure for the measurement setup:

```
\begin{circuitikz}[american]
\draw (0,0) -- ++(1,0) to[R] ++(3,0) to [iloop2, name=I] ++(0,-2)
node[ground] (GND){};
\ctikzset{bipoles/oscope/width=1.6}\ctikzset{bipoles/oscope/height=1.2}
\node [oscopeshape, fill=green!10] (O) at (6,2){};
\node [bnc, xscale=-1, anchor=zero] (bnc1) at (0.in 1){};
\node [bnc, , anchor=zero, rotate=-90] (bnc2) at (0.in 2){};
\draw [-latexslim] (bnc1.hot) -| (1,0);
\draw (bnc2.hot) |- (I.i+);
\draw (I.i-) node[ground, scale=0.5]{};
\end{circuitikz}
```

⁵⁶QUCS is an open source circuit simulator: <http://qucs.sourceforge.net/>



4.7 Mechanical Analogy

	damper: Mechanical Damping, type: path-style, fillable, nodename: dampershape. Class: mechanicals.
	inerter: Mechanical Inerter, type: path-style, fillable, nodename: inetershape. Class: mechanicals.
	spring: Mechanical Stiffness, type: path-style, nodename: springshape. Class: mechanicals.
	viscoe: Mechanical viscoelastic element ⁵⁷ , type: path-style, fillable, nodename: viscoeshape. Class: mechanicals.
	mass: Mechanical Mass, type: path-style, fillable, nodename: massshape. Class: mechanicals.

4.7.1 Mechanical elements customizations

You can change the scale of all the mechanical elements by setting the key `mechanicals/scale` to something different from the default 1.0.

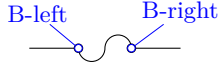
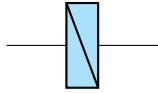
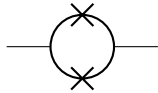
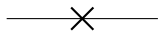
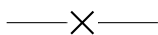
4.8 Miscellaneous bipoles and symbols

Here you'll find components that do not fit well in any category, or span several ones.

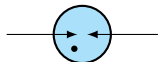
⁵⁷Suggested by @Alex in <https://tex.stackexchange.com/q/484268/38080>

4.8.1 Miscellaneous bipoles

Here you'll find bipoles that are not easily grouped in the categories above.

	thermocouple: Thermocouple, type: <code>path-style</code> , nodename: <code>thermocoupleshape</code> . Class: <code>misc</code> .
	fuse: Fuse, type: <code>path-style</code> , fillable, nodename: <code>fuseshape</code> . Class: <code>misc</code> .
	afuse: Asymmetric fuse, type: <code>path-style</code> , fillable, nodename: <code>afuseshape</code> . Aliases: <code>asymmetric fuse</code> . Class: <code>misc</code> .
	wfuse: “wiggly” fuse, type: <code>path-style</code> , name= <code>B</code> , nodename: <code>wfuseshape</code> . Aliases: <code>wiggly fuse</code> . Class: <code>misc</code> .
	relais: Relais ⁵⁸ , type: <code>path-style</code> , fillable, nodename: <code>relaishape</code> . Class: <code>misc</code> .
	squid: Squid, type: <code>path-style</code> , nodename: <code>squidshape</code> . Class: <code>misc</code> .
	barrier: Barrier, type: <code>path-style</code> , nodename: <code>barriershape</code> . Class: <code>misc</code> .
	openbarrier: Open barrier, type: <code>path-style</code> , nodename: <code>openbarriershape</code> . Class: <code>misc</code> .

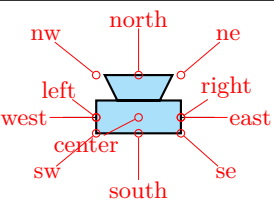
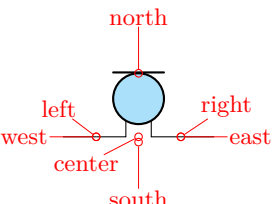
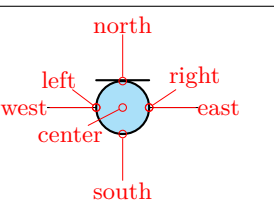
You can tune how big the gap in the `openbarrier` is by setting `bipole/openbarrier/gap` (default value 1 meaning full width). The default width `bipole/openbarrier/width` is 0.21, while `bipole/barrier/width` is 0. These settings ensure minimal width without drawing any wire when the components' node shapes are used.

	european gas filled surge arrester: European gas filled surge arrester, type: <code>path-style</code> , fillable, nodename: <code>european gas filled surge arrestershape</code> . Class: <code>misc</code> .
	american gas filled surge arrester: American gas filled surge arrester, type: <code>path-style</code> , fillable, nodename: <code>american gas filled surge arrestershape</code> . Class: <code>misc</code> .

If (default behaviour) `europeangfsurgearrester` option is active (or the style `european gas filled surge arrester` is used), the shorthands `gas filled surge arrester` and `gf surge arrester` are equivalent to the european version of the component.

If otherwise `americangfsurgearrester` option is active (or the style `american gas filled surge arrester` is used), the shorthands the shorthands `gas filled surge arrester` and `gf surge arrester` are equivalent to the american version of the component.

⁵⁸Contributed by [Jakob Leide](#)

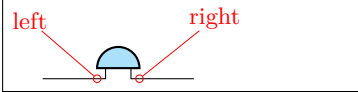
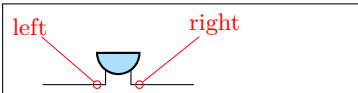
	lamp: Lamp, type: path-style, fillable, nodename: lampshape. Class: misc.
	bulb: Bulb, type: path-style, fillable, nodename: bulbshape. Class: misc.
	wbulb: Wiggly bulb shape ⁵⁹ , type: path-style, fillable, nodename: wbulbshape. Class: misc.
	neonlampcc: Neon lamp ⁶⁰ (double cathode style), type: path-style, fillable, nodename: neonlampccshape. Class: misc.
	neonlampac: Neon lamp (anode and cathode style), type: path-style, fillable, nodename: neonlampacshape. Class: misc.
	sparkgap: Spark gap ⁶¹ (unenclosed), type: path-style, fillable, nodename: sparkgapshape. Class: misc.
	sparkgap, sparkgap/circle: Spark gap, type: path-style, fillable, nodename: sparkgapshape. Class: misc.
	sparkgap, sparkgap/dot, sparkgap/circle: Spark gap (gas filled), type: path-style, fillable, nodename: sparkgapshape. Class: misc.
	loudspeaker: loudspeaker, type: path-style, fillable, nodename: loudspeakershape. Class: misc.
	mic: mic, type: path-style, fillable, nodename: micshape. Class: misc.
	tlmic: tail-less mic ⁶² , type: path-style, fillable, nodename: tlmicshape. Class: misc.

⁵⁹This lamp is drawn in the style of the textbook Physics for Scientists and Engineers: A Strategic Approach by Randall D. Knight, suggested by [Sebastiano on TeX.SX](#)

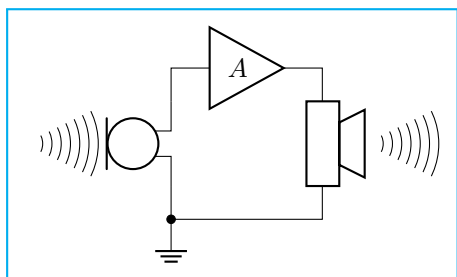
⁶⁰The neon lamps have been added in v1.6.9 thanks to a suggestion by [user bogger33 on GitHub](#).

⁶¹The spark gap has been added in v1.6.9 thanks to a suggestion by [user bogger33 on GitHub](#).

⁶²Suggested by [Dr. Matthias Jung](#).

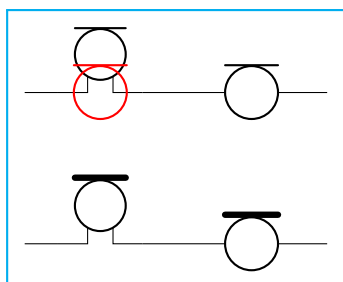
	buzzer: Buzzer ⁶³ , type: path-style, fillable, nodename: buzzershape. Class: misc.
	rbuzzer: Reversed buzzer, type: path-style, fillable, nodename: rbuzzershape. Class: misc.

You can use microphones and loudspeakers with **waves** (see section 4.17) too:



```
\begin{circuitikz}
  \draw (0,0) to[mic, name=M] ++(0,2)
    to[amp, t=$A$] ++(2,0)
    to[loudspeaker, name=L] ++(0,-2)
    to[short, -*] (0,0) node[ground]{};
  \node [waves, scale=0.7, left=5pt]
    at(M.north) {};
  \node [waves, scale=0.7, right]
    at(L.north) {};
\end{circuitikz}
```

You have two types of microphones; **mic** has protruding connection and **tlmic** (for tail-less microphone) is inline. This last one is handy for use as a separate shape (which is named **tlmicshape**). You can change the (relative) thickness of the straight bar using the key **bipoles/mic/bar thickness** (default 1).



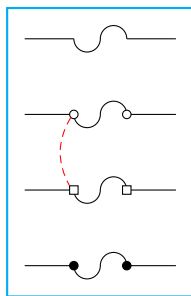
```
\begin{circuitikz}[]
  \draw (0,2) to[mic, name=M] ++(2,0) to[tlmic] ++(2,0);
  \node [color=red, tlmicshape](T) at (M.center) {};
  \ctikzset{bipoles/mic/bar thickness=3}
  \draw (0,0) to[mic] ++(2,0) to[tlmic] ++(2,0);
\end{circuitikz}
```

4.8.2 Miscellaneous element customization

You can change the scale of all the miscellaneous elements by setting the key **misc/scale** to something different from the default 1.0; relative thickness can be controlled with **misc/thickness**.

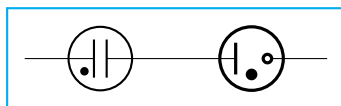
4.8.2.1 Wiggly fuses. Wiggly fuses can have (or not have) poles; you can switch between the two forms by setting to **true** or **false** (default **true**) the key **bipoles/wfuse/dots**; if they have poles, you can choose any of the pole shapes with the key **bipoles/wfuse/shape**. The pole nodes are named **-left** and **-right** so that you can access their borders.

⁶³Buzzers were suggested by [user Michael.H on TeX.SX](#)



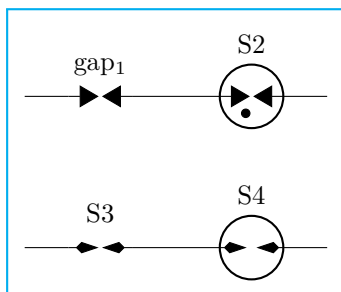
```
\begin{circuitikz}
\draw (0,3) to[wfuse, bipoles/wfuse/dots=false] ++(2,0);
\draw (0,2) to[wfuse, name=A] ++(2,0);
\ctikzset{bipoles/wfuse/shape=osquarepole}
\draw (0,1) to[wfuse, name=B] ++(2,0);
\draw [red, densely dashed]
(A-left.-135) to[bend right] (B-left.135);
\ctikzset{bipoles/wfuse/shape=circ}
\draw (0,0) to[wfuse, name=B] ++(2,0);
\end{circuitikz}
```

4.8.2.2 Neon lamps. Neon lamp “dot” size is the same as the size of poles (`circ` and `ocirc`), and they can be changed locally:



```
\begin{tikzpicture}
\draw (0,0) to[neonlampcc, nodes width=0.03] ++(2,0)
to[neonlampac, misc/thickness=3] ++(2,0);
\end{tikzpicture}
```

4.8.2.3 Spark gap. The `sparkgap` component is similar to the (American) surge arrester, but it’s more configurable; it will render bare (unenclosed) by default, but you can add a (fillable) enclosure with the key `sparkgap/circle` and a dot with `sparkgap/dot` (they are boolean keys, false by default). Moreover, the arrows are configurable like other arrows in the package (see 4.2.3.3) using the `sparkgap end arrow` key (default `Triangle[scale=2]`). The gap is tunable with `sparkgap/gap` (default 0.05).



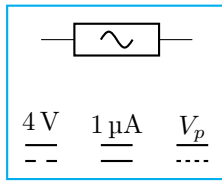
```
\begin{tikzpicture}
\draw (0,2) to[sparkgap, l=gap\textsubscript{1}] ++(2,0)
to[sparkgap, sparkgap/circle,
sparkgap/dot, l=S2] ++(2,0);
\ctikzset{sparkgap end arrow={Kite[scale=1.5]}}
\draw (0,0) to[sparkgap, l=S3] ++(2,0)
to[sparkgap, l=S4, sparkgap/circle,
sparkgap/gap=0.15] ++(2,0);
\end{tikzpicture}
```

As in neon lamps, the dot (if activated by the key `sparkgap/dot`) follows the size of poles and can be changed locally.

4.8.3 Miscellaneous symbols

	dc symbol, type: node (<code>node[dc symbol]{text}</code>). Class: <code>misc</code> .
	ac symbol, type: node (<code>node[ac symbol]{text}</code>). Class: <code>misc</code> .

The symbols for dc or ac voltages or currents can be useful to specify the waveforms in parts of the circuit, or to add to existing symbols.



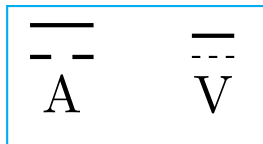
```
\begin{circuitikz}[european]
\draw (0,1.5) to[generic, name=a] ++(2,0);
\node [ac symbol] at (a.center) {};
\draw (0,0) node[dc symbol]{\qty{4}{V}};
\draw (1,0) node[dc symbol, dc symbol segments=1]{\qty{1}{\uA}};
\draw (2,0) node[dc symbol, dc symbol segments=4]{$V_p$};
\end{circuitikz}
```

4.8.4 Customization of ac/dc symbols.

You can change the width and height of the symbols with `dc symbol/width`, `dc symbol/height`, `ac symbol/width`, `ac symbol/height` (at the `circuitikz` level, the default is 0.3 for the widths and 0.15 for both symbols). The node text is positioned above the symbol, at a distance controlled by the keys `dc symbol/text offset` (and the similar `ac...`; default 6pt).

As shown in the example above, the `dc symbols` has an additional options. By default the lower line of the symbol has two segments, but you can change this with the `circuitikz` key `dc symbol/segments` (with also the direct key, at `TikZ` level, `dc symbol segments` (default 2)) with any value greater than 0.

Additionally, you can change the relative thickness of the bottom line of the `dc symbol` with the key `dc symbols/relative thickness` (default 1).



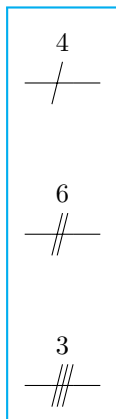
```
\begin{tikzpicture}[scale=2, transform shape]
\tikzset{my dc symbol/.style={
  dc symbol,
  circuitikz/misc/thickness=4,
  circuitikz/dc symbol/height=0.1,
  circuitikz/dc symbol/width=0.2,
  circuitikz/dc symbol/relative thickness=.5,
  circuitikz/dc symbol/segments=3
}}
\node (A) at (0,0) {A}; \node (V) at (1,0) {V};
% standard and customized
\node [dc symbol, above,
  circuitikz/misc/thickness=4] at (A.north) {};
\node [my dc symbol, above] at (V.north) {};
\end{tikzpicture}
```

4.9 Multiple wires (buses)

These are simple drawings to indicate multiple wires.

	multiwire : Single line multiple wires, type: <code>path-style</code> , nodename: <code>multiwireshape</code> . Aliases: <code>multiwire</code> . Class: default.
	bmultiwire : Double line multiple wires, type: <code>path-style</code> , nodename: <code>bmultiwireshape</code> . Aliases: <code>bmultiwire</code> . Class: default.
	tmultiwire : Triple line multiple wires ⁶⁴ , type: <code>path-style</code> , nodename: <code>tmultiwireshape</code> . Aliases: <code>tmultiwire</code> . Class: default.

⁶⁴added by offline



```
\begin{circuitikz}
  \draw (0,0) to[multiwire=4] ++(1,0);
  \draw (0,-2) to[bmultiwire=6] ++(1,0);
  \draw (0,-4) to[tmultiwire=3] ++(1,0);
\end{circuitikz}
```

4.10 Crossings

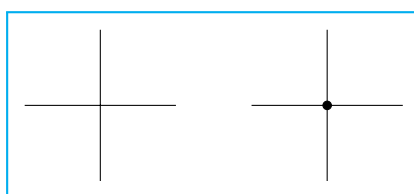
Path style:

	crossing: Jumper style non-contact crossing, type: path-style, nodename: crossingshape. Aliases: xing. Class: default.
--	---

Node style:

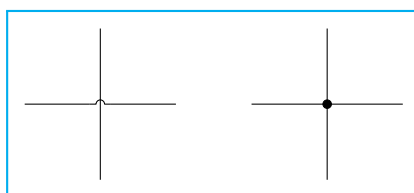
	Jumper-style crossing node, type: node (node[jump crossing]{}). No class.
	Plain style crossing node, type: node (node[plain crossing]{}). No class.

All circuit-drawing standards agree that to show a crossing without electric contact, a simple crossing of the wires suffices; the electrical contact must be explicitly marked with a filled dot.



```
\begin{circuitikz}[]
\draw(1,-1) to[short] (1,1)
      (0,0) to[short] (2,0);
\draw(4,-1) to[short] (4,1)
      (3,0) to[short] (5,0)
      (4,0) node[circ]{};
\end{circuitikz}
```

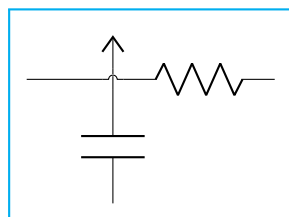
However, sometimes it is advisable to mark the non-contact situation more explicitly. To this end, you can use a path-style component called **crossing**:



```
\begin{circuitikz}[]
\draw(1,-1) to[short] (1,1) (0,0) to[crossing] (2,0);
\draw(4,-1) to[short] (4,1) (3,0) to[short] (5,0)
      (4,0) node[circ]{};
\end{circuitikz}
```

That should suffice most of the time; the only problem is that the crossing jumper will be put in the center of the subpath where the `to[crossing]` is issued, so sometimes a bit of trial and error is needed to position it.

For a more powerful (and elegant) way you can use the crossing nodes:



```
\begin{circuitikz}[]
  \node at (1,1)[jump crossing](X){};
  \draw (X.west) -- ++(-1,0);
  \draw (X.east) to[R] ++(2,0);
  \draw (X.north) node[vcc]{};
  \draw (X.south) to[C] ++(0,-1.5);
\end{circuitikz}
```

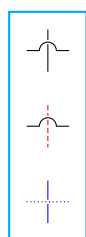
Notice that the **plain crossing** and the **jump crossing** have a small gap in the straight wire, to enhance the effect of crossing (as a kind of shadow).

4.10.1 Crossing customization

The size of the crossing elements can be changed with the key `bipoles/crossing/size` (default 0.2).

While the horizontal line will be drawn with the current path values, you can change the style of the vertical line⁶⁵ in a similar way to the one used for transistor's bodydiodes, by setting keys with the `\ctikzset` command under the **crossing vertical** hierarchy. The available keys are:

parameter	default	description
<code>relative thickness</code>	1.0	multiply the default thickness (which is the same of the choke component).
<code>color</code>	<code>default</code>	stroke color: default is the same as the component.
<code>dash</code>	<code>default</code>	dash pattern: default means not to change the setting for the component; none means unbroken line; every other input is a dash pattern. ⁶⁶



```
\begin{circuitikz}[every node/.append style={scale=2}]
  \draw (0,2) node[jump crossing](A){};
  \begin{scope}
    \ctikzset{crossing vertical/.cd, color=red, dash={{2pt}{1pt}}}
    \draw (0,1) node[jump crossing](B){};
  \end{scope}
  \ctikzset{crossing vertical/dash=none}
  \draw[densely dotted, blue] (0,0) node[plain crossing](B){};
\end{circuitikz}
```

⁶⁵Suggested by [user lkjell on GitHub](#), implemented in v1.6.2.

⁶⁶Follows the syntax of the pattern sequence `\pgfsetdash` — see TikZ manual [for details](#); phase is always zero. Basically you pass pairs of dash-length – blank-length dimensions, see the examples.

4.11 Arrows (fake and real)

The main arrow shapes in CircuiTikZ are really shapes, used as pseudo-arrows in lot of places in the packages (for transistors, flows, currents, and so on). The first three arrows are magnified by a factor 3 in the boxes below; for the `trarrow`, the anchor `tip` is exactly on the tip and `btip` is slightly receded.

	Arrow for current and voltage, type: node (<code>node[curarrow]{}</code>). No class.
	Arrow that is anchored at its tip, useful for block diagrams., type: node (<code>node[inputarrow]{}</code>). No class.
	Arrow the same size of <code>curarrow</code> but only filled., type: node (<code>node[trarrow]{}</code>). No class.
	Arrow used for the flows, type: node (<code>node[flowarrow]{I_p}</code>). No class.

4.11.1 Arrows size

You can use the parameter `current arrow scale` to change the size of the arrows in various components and indicators; the normal value is 16, higher numbers give smaller arrows and so on. You need to use `circuitikz/current arrow scale` if you use it into a node.

```

\begin{circuitikz}
  \draw (0,0) to[R, i=f] ++(2,0) node[npn, anchor=B]{};
  \draw (0,-2) to[R, f=f, current arrow scale=8] ++(2,0)
    node[pnp, anchor=B, circuitikz/current arrow scale
    =8]{};
  \draw (0,-4) to[R, f=f, current arrow scale=24] ++(2,0)
    node[nigt, anchor=B]{};
\end{circuitikz}

```

Moreover, you have the arrow tip `latexslim` which is an arrow similar to the old (in deprecated `arrows` library) `latex'` element:

```

\begin{circuitikz}[american,]
  \draw [latexslim-latexslim] (0,0) -- (1,0);
\end{circuitikz}

```

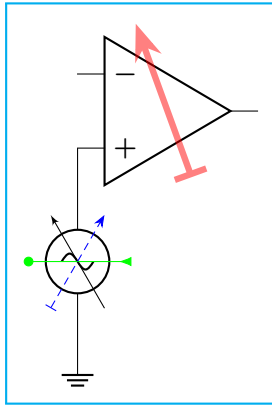
4.11.2 Generic Tunable Arrows

The basic passive components (resistors, capacitors and inductors) come with a “tunable version” (see for example 4.2.3.3) that conveys the information that their value is adjustable. For generic components you can obtain a similar effect with the extra macro `\ctikztunablearrow`, introduced in version 1.4.1. The macro should be called as:

```
\ctikztunablearrow[extra options]{thickness}{length}{angle}{name}
```

where *extra options* is an optional argument with generic TikZ keys, *thickness* is the relative thickness (referred to the current line width when the macro is invoked), *length* is the length of the arrow with respect to the diagonal size of the component, *angle* is the inclination with respect to the normal direction of the component⁶⁷, and finally *name* is the reference name of the bipole or node.

The arrows are the ones set with the keys `tunable start arrow` and `tunable end arrow` (to maintain coherency across the circuit), but you can override them in the *extra options* argument as shown in the following example.



```
\begin{circuitikz}
  \draw (0,0) node[tlground]{} to[sV, name=A] ++(0,3)
    node[op amp, anchor=+] (B){};
  \ctikztunablearrow{1}{1.2}{30}{A}
  \ctikzset{tunable start arrow={Bar},
    tunable end arrow={Stealth}}
  \ctikztunablearrow[color=green,
    {Latex[reversed]}-Circle}{1}{1.2}{90}{A}
  \ctikztunablearrow[color=blue, densely dashed]{1}{1.2}{-30}{A}
}
\begin{scope}[transparency group, opacity=0.5]
  \ctikztunablearrow[red, shorten <=3mm]{6}{0.8}{110}{B}
\end{scope}
\end{circuitikz}
```

Notice also the need to force a transparency group if you want a semitransparent arrow.

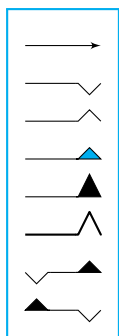
4.11.3 Arrow tips

In addition to the pseudo-arrows, CircuiTikZ also adds a couple of “real” arrow tips. The package automatically loads the `arrows.meta` TikZ library but *not* the deprecated `arrows` library; in the first versions of the package it used quite a lot the `latex` tip, which is not available anymore. To maintain the backward compatibility, the ‘`latexslim`’ tip has been added, and used by default in several components. This is an old-style arrow tip, with no customization possible.

The other tip is `Jack Tap`⁶⁸, which is mainly used to build jack connectors (see section 4.13.3). This is a new-style arrow tip, and accepts the parameter `length` (default 0.3 cm), `width` (default 0.15 cm), and the boolean `swap`.

⁶⁷which is the left-to-right direction of the component when shown in the component box in this manual.

⁶⁸Added after a suggestion from [Anisio Rogerio Braga](#) on GitHub



```
\begin{circuitikz}
\draw (0,.25) (0,0) edge[-latexslim] ++(1,0)
++(0, -0.5) edge[-{Jack Tap[swap]}] ++(1,0)
++(0, -0.5) edge[-Jack Tap] ++(1,0)
++(0, -0.5) edge[-{Jack Tap[fill=cyan]}] ++(1,0)
++(0, -0.5) edge[-{Jack Tap[width=.3cm,fill]}] ++(1,0)
++(0, -0.5) edge[-{Jack Tap[width=.3cm,line join=round,
line cap=round]}], thick] ++(1,0)
++(0, -0.5) edge[~^f] ++(1,0)
++(0, -0.5) edge[vf-v] ++(1,0);
\end{circuitikz}
```

You can also have a filled version, by adding the key `fill` (without arguments⁶⁹) or `fill=color` if you want a color different from the stroke ones, and they accept the `line join` and `line cap` as most of the standard TikZ arrows. As you can see, the normal and swapped Jack Tap tips have the shorthands `v` and `^` (and `vf` and `^f` for their filled counterparts). Notice that the tips are automatically reversed when they are at the *start* of the path.

4.12 Terminal shapes

These are the so-called “bipole nodes” shapes, or poles (see section 6.1). These nodes are always filled; the “open” versions (starting with an `o`) are by default filled with the color specified by the key `open nodes fill` (by default `white`), but you can override locally it with the `fill` parameter.

•	Connected terminal, type: node (<code>node[circ]{}</code>). No class.
◦	Unconnected terminal, type: node (<code>node[ocirc]{}</code>). No class.
◆	Diamond-square terminal, type: node (<code>node[diamondpole]{}</code>). No class.
◇	Open diamond-square terminal, type: node (<code>node[odiamondpole]{}</code>). No class.
■	Square-shape terminal, type: node (<code>node[squarepole]{}</code>). No class.
□	Open square-shape terminal, type: node (<code>node[osquarepole]{}</code>). No class.

This is not a pole, but it’s used to “fill” nasty corners (look closer, and see 6.4).

.	Filling square with line width size, type: node (<code>node[rectjoinfill]{}</code>). No class.
---	--

Since version 0.9.0, “bipole nodes” shapes have all the standard geographical anchors, so you can do things like these:



```
\begin{circuitikz}[american,]
\draw (0,-1) node[draw](R){R};
\draw (R.east) node[ocirc, right]{};
\end{circuitikz}
```

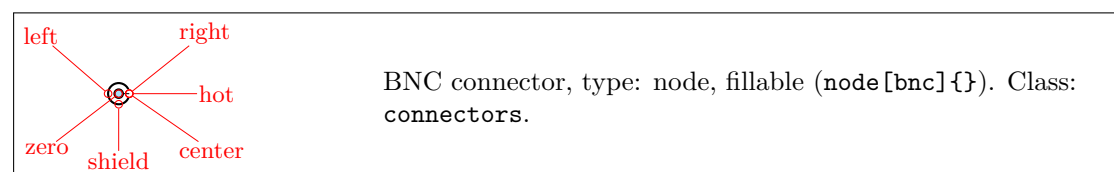
⁶⁹This usage of the `fill` key in arrow tips will be added to TikZ in version 3.1.11, see [this PR by Henri Menke](#); CircuiTikZ will add it to older versions.

The size of the poles is controlled by the key `nodes width` (default 0.04, relative to the basic length). Be sure to see section 6.1 for more usage and configurability.

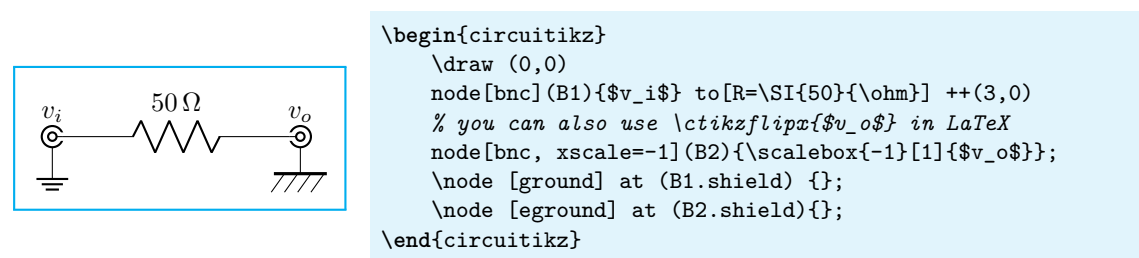
4.13 Connectors

Connectors have a class by themselves (`connectors`), so you can use the `scale`, `fill` and `thickness` properties as usual.

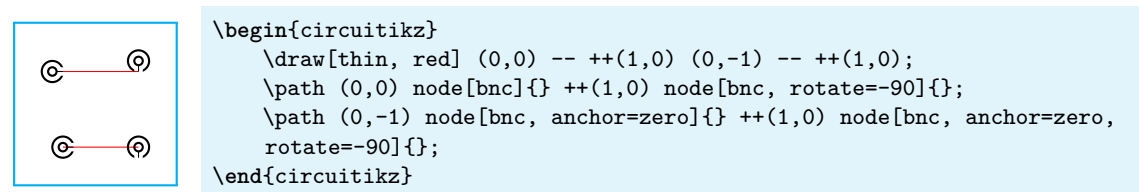
4.13.1 BNC connector/terminal



The BNC connector is defined so that you can easily connect it as input or output (but remember that you need to flip the text if you flip the component):

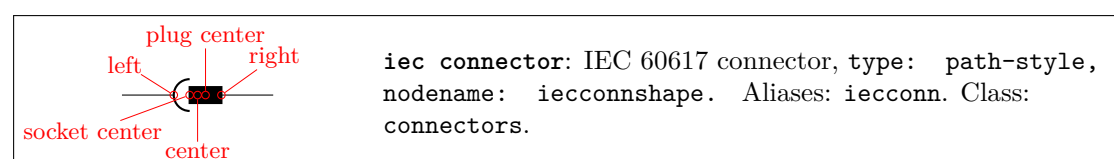


It also has a `zero` anchor if you need to rotate it about its real center.

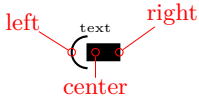
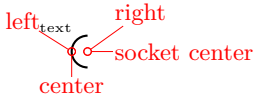
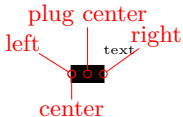
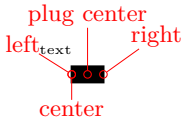
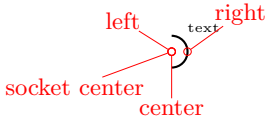


4.13.2 IEC 60617 socket-plug connectors

Plug and socket connectors (modeled on the IEC60617 standard) are available⁷⁰ both in path-style form and, with separated but matching shapes for plug and socket, in node-style. There are two differently oriented shapes for each type to ease the construction of “split” connections (see the examples below). **Notice** that the elements in the following table are scaled by a factor of 1.5, to better show the position of the anchors.



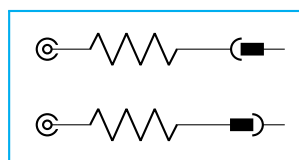
⁷⁰Since v1.5.0; thanks to Alexander Sauter for suggesting them and [helping in the design](#).

	IEC 60617 connector, type: node (node[ieconnshape]{\tiny text}). Class: connectors.
	IEC 60617 female socket, left side, type: node (node[iecsocketL]{\tiny text}). Class: connectors.
	IEC 60617 male plug, right side, type: node (node[iecplugR]{\tiny text}). Class: connectors.
	IEC 60617 male plug, left side, type: node (node[iecplugL]{\tiny text}). Class: connectors.
	IEC 60617 female socket, right side, type: node (node[iecsocketR]{\tiny text}). Class: connectors.

The **center** anchors (as well as the text position) of the split elements of the connectors are on the side of the component (similar to what happens with grounds and supply voltage arrows) to ease the most common use.

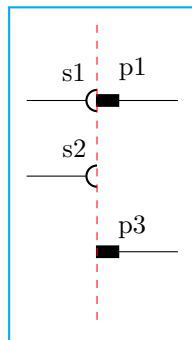
Also, the text for the plug nodes is raised to the same level of the text in the sockets, and it will ignore descendants, so that the two text lines up when the two components are put side by side.

The **plug center** anchor always point to the center of the rectangular plug shape, and the **socket center** to the center of the semicircle in the sockets.



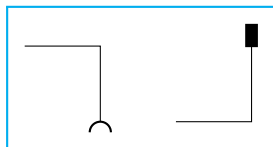
```
\begin{circuitikz}
\draw (0,1) node[bnc]{} to[R] ++(2,0)
to[iec connector] ++(1,0);
\draw (0,0) node[bnc]{} to[R] ++(2,0)
to[iec connector, invert] ++(1,0);
\end{circuitikz}
```

Aligning “disconnected” plugs and sockets is reasonably easy:



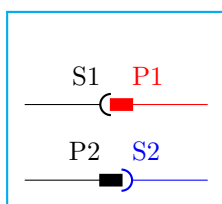
```
\begin{circuitikz}
\draw (0,2) to[iec connector, name=C1] ++(2,0) coordinate(stop)
(C1.nw) node[above left, inner xsep=0pt]{s1}
(C1.ne) node[above right, inner xsep=0pt]{p1};
\draw (0,1) coordinate(tmp) -- (tmp-|C1.left)
node[iecsocketL](S2){s2};
\draw (0,0) coordinate(tmp) (tmp-|C1.socket center)
node[iecplugR](P3){p3} (P3.right) -- (tmp-|stop);
\draw[red, dashed] ([yshift=1cm]C1.socket center) -- ++(0,-4);
\end{circuitikz}
```

You can choose the best shape when rotating them, to simplify the positioning (shape rotates around the `center` anchor).



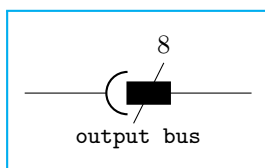
```
\begin{circuitikz}
\draw (0,0) -| ++(1,-1) node[iecssocketL, rotate=-90]{}
(2,-1) -| ++(1,1) node[iecpugR, rotate=90]{};
\end{circuitikz}
```

Choosing the proper left/right shape results in easily build “mixed” connectors; you can use the node text position properties to have lined-up labels, but remember that the text is outside the bounding box:



```
\begin{circuitikz}[]
\path (0,2); % for the bounding box, text is not accounted for
\draw (0,1)--++(1,0) node[iecssocketL](s1){S1};
\draw [color=red](s1.e) node[iecpugR](p1){P1} (p1.e)--++(1,0);
\draw (0,0) -- ++(1,0) node[iecpugL](p2){P2};
\draw [color=blue](p2.e) node[iecssocketR](s2){S2} (s2.e)--++(1,0);
;
\end{circuitikz}
```

You can use the plug `center` anchor to add the IEC “multiplier”:

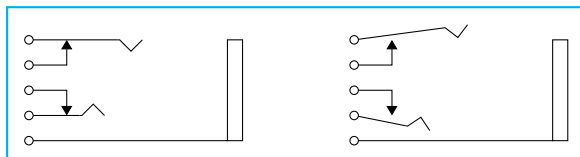


```
\begin{circuitikz}[]
\draw (0,0) to[iec connector, connectors/scale=2, name=A,
a={\small\ttfamily output bus}] ++(3,0);
\draw (A.plug center) ++(-.2,-.4) -- ++(.4,.8) node[above]{8};
\end{circuitikz}
```

4.13.3 Jack connectors

There are *lots* of different jack connectors symbols — see the [discussion here](#) for examples. So instead of creating a monster component, it has been decided to add elements to simplify the drawing of such connectors. The first (and for now only) such element is the Jack Tap arrow tip (see section 4.11.3) with its shorthands `v` and `^`.

For example, an audio jack can be drawn like this:

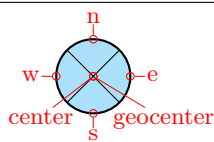


```
% drawing based on one by Anisio Rogerio Braga
% https://github.com/circuitikz/circuitikz/issues/806
\newcommand\dx{1.5}\newcommand\dy{1}
\begin{circuitikz}
\draw[-v] (0,\dy/3*4) to[short, o-] ++(\dx,0);
\draw[-Triangle] (0,\dy) node[ocirc]{} -| ++(\dx/3,\dy/3);
\draw[-Triangle] (0,\dy/3*2) node[ocirc]{} -| ++(\dx/3,-\dy/3);
\draw[-^] (0,\dy/3) to[short, o-] ++(2*\dx/3,0);
\draw (0,0.0) to[short, o-] ++(1.75*\dx,0) rectangle ++(0.2,4*\dy/3);
\end{circuitikz}\quad\quad
```

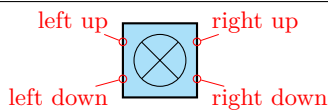
```
%---% audio jack with an inserted plug
\begin{circuitikz}
  \draw[-v] (0,\dy/3*4) to[short, o-] ++(\dx,0.2);
  \draw[-Triangle] (0,\dy) node[ocirc]{} -| ++(\dx/3,\dy/3);
  \draw[-Triangle] (0,\dy/3*2) node[ocirc]{} -| ++(\dx/3,-\dy/3);
  \draw[-^] (0,\dy/3) to[short, o-] ++(2*\dx/3,-0.2);
  \draw (0,0.0) to[short, o-] ++(1.75*\dx,0) rectangle ++(0.2,4*\dy/3);
\end{circuitikz}
```

4.14 Block diagram components

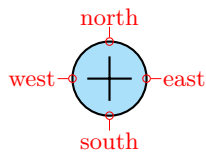
Contributed by Stefan Erhardt.



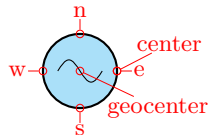
mixer, type: node, fillable (`node[mixer]{}).` Class: **blocks**.



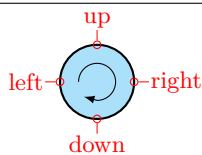
mixer, boxed, type: node, fillable (`node[mixer, boxed]{}).` Class: **blocks**.



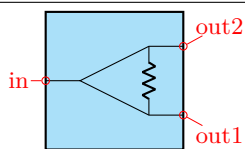
adder, type: node, fillable (`node[adder]{}).` Class: **blocks**.



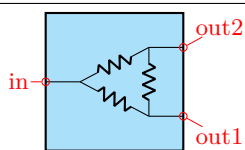
oscillator, type: node, fillable (`node[oscillator]{}).` Class: **blocks**.



circulator, type: node, fillable (`node[circulator]{}).` Class: **blocks**.

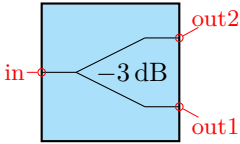
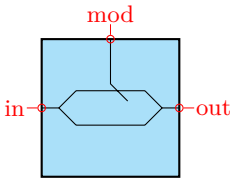
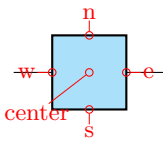
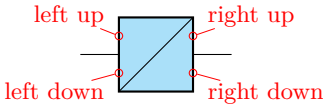
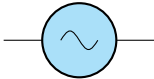
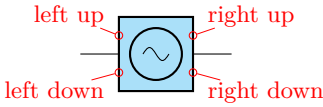
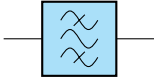
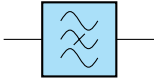
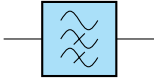
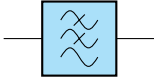
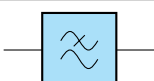


wilkinson divider, type: node, fillable (`node[wilkinson]{}).` Class: **blocks**.



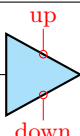
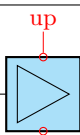
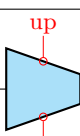
resistive splitter⁷¹, type: node, fillable (`node[splitter]{}).` Class: **blocks**.

⁷¹added by matthuszagh

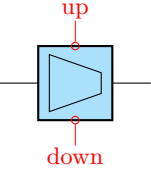
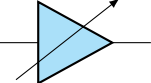
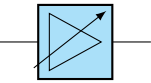
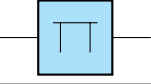
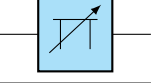
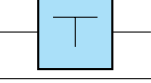
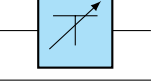
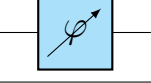
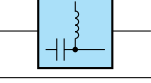
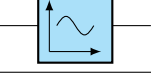
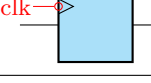
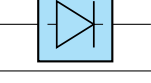
	generic splitter ⁷² , type: node, fillable (node[genericsplitter]{ $\SI{-3}{\decibel}$ }). Class: blocks.
	Mach Zehnder Modulator ⁷³ , type: node, fillable (node[mzm]{}). Class: blocks.
	twoport : generic two port (use t=... to specify text), type: path-style, fillable, nodename: twoportshape. Class: blocks.
	twoportsplit : generic two port split (use t1=... and t2=... to specify text), type: path-style, fillable, nodename: twoportsplitshape. Class: blocks.
	vco : vco, type: path-style, fillable, nodename: vcoshape. Class: blocks.
	vco,box : vco,box, type: path-style, fillable, nodename: vco,boxshape. Class: blocks.
	bandpass : bandpass, type: path-style, fillable, nodename: bandpassshape. Class: blocks.
	bandstop : bandstop, type: path-style, fillable, nodename: bandstopshape. Class: blocks.
	highpass : highpass, type: path-style, fillable, nodename: highpassshape. Class: blocks.
	lowpass : lowpass, type: path-style, fillable, nodename: lowpassshape. Class: blocks.
	highpass2 : simplified highpass (with only 2 waves), type: path-style, fillable, nodename: highpass2shape. Class: blocks.
	lowpass2 : simplified lowpass (with only 2 waves), type: path-style, fillable, nodename: lowpass2shape. Class: blocks.

⁷²added by frankplow

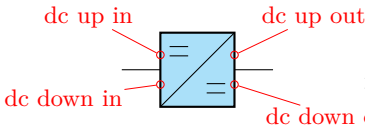
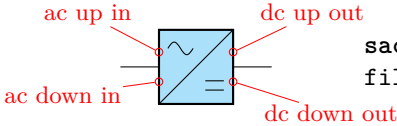
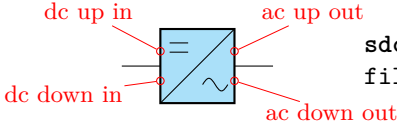
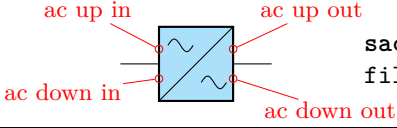
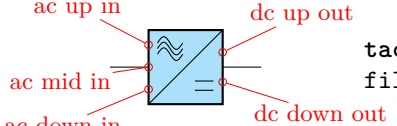
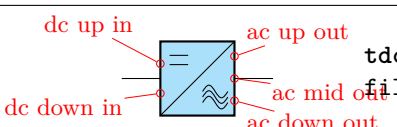
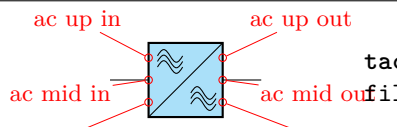
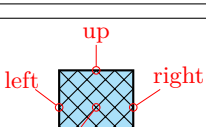
⁷³added by d11chb

	allpass: allpass, type: path-style, fillable, nodename: allpassshape. Class: blocks.
	vallpass: variable allpass ⁷⁴ , type: path-style, fillable, nodename: vallpassshape. Class: blocks.
	bgenerator: Block generator (use t=... to specify text), type: path-style, fillable, nodename: bgeneratorshape. Class: blocks.
	qgenerator: Crystal generator (use t=... to specify text), type: path-style, fillable, nodename: qgeneratorshape. Class: blocks.
	cgenerator: Clock generator, type: path-style, fillable, nodename: cgeneratorshape. Class: blocks.
	ngenerator: Generic generator (use t=... to specify text), type: path-style, fillable, nodename: ngeneratorshape. Class: blocks.
	adc: A/D converter, type: path-style, fillable, nodename: adcshape. Class: blocks.
	dac: D/A converter, type: path-style, fillable, nodename: dacshape. Class: blocks.
	dsp: DSP, type: path-style, fillable, nodename: dspshape. Class: blocks.
	fft: FFT, type: path-style, fillable, nodename: fftshape. Class: blocks.
	amp: amplifier; use t=... to add a text, type: path-style, fillable, nodename: ampshape. Class: blocks.
	amp, boxed: amplifier, boxed; use t=... to add a text, type: path-style, fillable, nodename: amp, boxedshape. Class: blocks.
	iamp: instrumentation amplifier; use t=... to add a text, type: path-style, fillable, nodename: iampshape. Class: blocks.

⁷⁴This block, as well as bgenerator, cgenerator, qgenerator, ngenerator, biast, sinetable, register, harmonics, and the TV/Radio system blocks have been contributed by [Dr. Matthias Jung](#).

	iamp, boxed: instrumentation amplifier, boxed; use <code>t=...</code> to add a text, type: <code>path-style</code> , fillable, nodename: <code>iamp, boxedshape</code> . Class: <code>blocks</code> .
	vamp: VGA (variable gain amplifier), type: <code>path-style</code> , fillable, nodename: <code>vampshape</code> . Class: <code>blocks</code> .
	vamp, boxed: VGA, boxed, type: <code>path-style</code> , fillable, nodename: <code>vamp, boxedshape</code> . Class: <code>blocks</code> .
	piattenuator: π attenuator, type: <code>path-style</code> , fillable, nodename: <code>piattenuatorshape</code> . Class: <code>blocks</code> .
	vpiattenuator: var. π attenuator, type: <code>path-style</code> , fillable, nodename: <code>vpiattenuatorshape</code> . Class: <code>blocks</code> .
	tattenuator: T attenuator, type: <code>path-style</code> , fillable, nodename: <code>tattenuatorshape</code> . Class: <code>blocks</code> .
	vtattenuator: var. T attenuator, type: <code>path-style</code> , fillable, nodename: <code>vtattenuatorshape</code> . Class: <code>blocks</code> .
	phaseshifter: phase shifter, type: <code>path-style</code> , fillable, nodename: <code>phaseshiftershape</code> . Class: <code>blocks</code> .
	vphaseshifter: var. phase shifter, type: <code>path-style</code> , fillable, nodename: <code>vphaseshiftershape</code> . Class: <code>blocks</code> .
	biast: BIAS-T circuit as block, type: <code>path-style</code> , fillable, nodename: <code>biastshape</code> . Class: <code>blocks</code> .
	sinetable: Sine lookup table, type: <code>path-style</code> , fillable, nodename: <code>sinetablespace</code> . Class: <code>blocks</code> .
	register: Register (use <code>t=...</code> to specify text), type: <code>path-style</code> , fillable, nodename: <code>registershape</code> . Class: <code>blocks</code> .
	detector: detector, type: <code>path-style</code> , fillable, nodename: <code>detectorshape</code> . Class: <code>blocks</code> .
	saturation: Saturation ⁷⁵ , type: <code>path-style</code> , fillable, nodename: <code>saturationshape</code> . Class: <code>blocks</code> .

⁷⁵Contributed by [P. Sacco <paul.sacco@estaca.eu>](mailto:paul.sacco@estaca.eu)

	sigmoid: Sigmoid, type: path-style, fillable, nodename: sigmoidshape. Class: blocks.
	allornothing: Comparison, all-or-nothing, type: path-style, fillable, nodename: allornothingshape. Class: blocks.
	fiber: Optical Fiber, type: path-style, fillable, nodename: fibershape. Class: blocks.
	sdc: single wire DC/DC converter ⁷⁶ , type: path-style, fillable, nodename: sdcshape. Class: blocks.
	sac: single phase AC/DC converter, type: path-style, fillable, nodename: sacshape. Class: blocks.
	sdac: single phase DC/AC converter, type: path-style, fillable, nodename: sdacshape. Class: blocks.
	sacac: single phase AC/AC converter, type: path-style, fillable, nodename: sacacshape. Class: blocks.
	tac: three phases AC/DC converter, type: path-style, fillable, nodename: tacshape. Class: blocks.
	tdc: three phases AC/DC converter, type: path-style, fillable, nodename: tdcshape. Class: blocks.
	tacac: three phases AC/DC converter, type: path-style, fillable, nodename: tacacshape. Class: blocks.
	gridnode ⁷⁷ , type: node, fillable (node[gridnode]{}). Class: blocks.

⁷⁶the converter blocks added by offline

⁷⁷added by offline

Filters blocks have an alternative, Bode-diagram-like appearance. Their names end in **p**, as a mnemonic for “plot”. By default, they have a “fake” semi-logarithmic grid to convey the hint that the diagram is a frequency response one, to separate them from similar drawings as, for example, saturation. The grid can be removed or customized, see section [4.14.2.7](#).

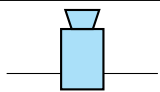
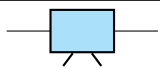
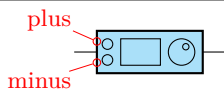
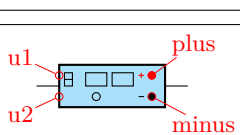
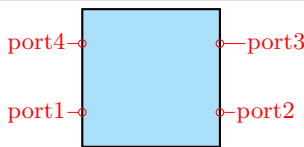
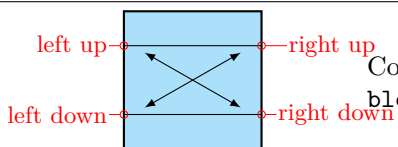
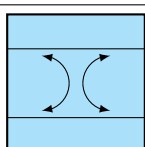
	lowpassp : lowpass, plot style, type: <code>path-style</code> , fillable, nodename: <code>lowpasspshape</code> . Class: blocks.
	highpassp : highpass, plot style, type: <code>path-style</code> , fillable, nodename: <code>highpasspshape</code> . Class: blocks.
	bandpassp : bandpass, plot style, type: <code>path-style</code> , fillable, nodename: <code>bandpasspshape</code> . Class: blocks.
	bandstopp : bandstop, plot style, type: <code>path-style</code> , fillable, nodename: <code>bandstoppshape</code> . Class: blocks.
	resonantp : resonant, plot style, type: <code>path-style</code> , fillable, nodename: <code>resonantpshape</code> . Class: blocks.
	notchp : notch, plot style, type: <code>path-style</code> , fillable, nodename: <code>notchpshape</code> . Class: blocks.

If you add the `ideal filter` option, the filters are changed like this:

	lowpassp, ideal filter : lowpass, plot style, ideal filter, type: <code>path-style</code> , fillable, nodename: <code>lowpassp</code> , ideal filtershape. Class: blocks.
	highpassp, ideal filter : highpass, plot style, ideal filter, type: <code>path-style</code> , fillable, nodename: <code>highpassp</code> , ideal filtershape. Class: blocks.
	bandpassp, ideal filter : bandpass, plot style, ideal filter, type: <code>path-style</code> , fillable, nodename: <code>bandpassp</code> , ideal filtershape. Class: blocks.
	bandstopp, ideal filter : bandstop, plot style, ideal filter, type: <code>path-style</code> , fillable, nodename: <code>bandstopp</code> , ideal filtershape. Class: blocks.
	resonantp, ideal filter : resonant, plot style, ideal filter, type: <code>path-style</code> , fillable, nodename: <code>resonantp</code> , ideal filtershape. Class: blocks.
	notchp, ideal filter : notch, plot style, ideal filter, type: <code>path-style</code> , fillable, nodename: <code>notchp</code> , ideal filtershape. Class: blocks.

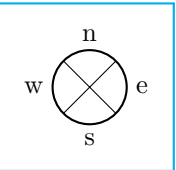
Another difference when using `ideal filter` is that now the grid is linear, and the angles are always sharp.

Blocks to draw TV/radio system schematics, contributed by [Dr. Matthias Jung](#).

	camera: Videocamera, type: path-style, fillable, nodename: camerashape. Class: misc.
	tvset: Television (TV), type: path-style, fillable, nodename: tvsetshape. Class: blocks.
	trx: Radio transceiver (HAM-Radio), type: path-style, fillable, nodename: trxshape. Class: blocks.
	swr: Standing Wave Ratio (SWR) meter, type: path-style, fillable, nodename: swrshape. Class: blocks.
	power: Power Supply, type: path-style, fillable, nodename: powershape. Class: blocks.
	Generic fourport, type: node, fillable (node[<code>fourport</code>]{}). Class: blocks.
	Coupler ⁷⁸ , type: node, fillable (node[<code>coupler</code>]{}). Class: blocks.
	Coupler with rounded arrows, type: node, fillable (node[<code>coupler2</code>]{}). Class: blocks.

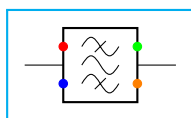
4.14.1 Blocks anchors

The ports of the mixer, adder, oscillator and circulator can be addressed with **west**, **south**, **east**, **north**; the equivalent **left**, **down**, **right**, **up**; or the shorter **w**, **s**, **e**, **n** ones:

	<pre> \begin{circuitikz} \draw (0,0) node[mixer] (mix) {} (mix.w) node[left] {w} (mix.s) node[below] {s} (mix.e) node[right] {e} (mix.n) node[above] {n} ;\end{circuitikz} </pre>
---	---

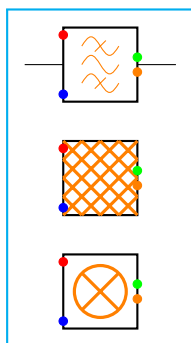
⁷⁸Notice that the **left/right up/down** anchors coincide with **port1...4** only for the default values of block [**left/right**] anchors pos.

In addition, since v1.6.0, most blocks also have the `left up`, `left down`, `right up` and `right down` anchors:



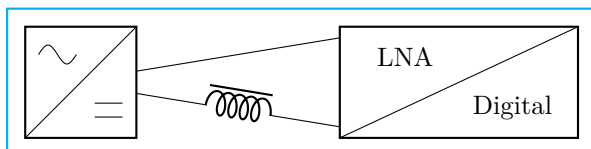
```
\begin{circuitikz} \draw
  (0,0) to[bandpass, name=bp] ++(2,0)
  (bp.left up) node[circ, red]{}
  (bp.left down) node[circ, blue]{}
  (bp.right up) node[circ, green]{}
  (bp.right down) node[circ, orange]{}
;\end{circuitikz}
```

Since 1.6.7 you can change the relative position of the lateral generic anchors⁷⁹ using the keys `block left anchors pos` and `block right anchors pos` (default 0.5, as in previous releases) which set the position as a fraction of the top to center segment. Since 1.8.4 the effect of this keys is also extended to node-type blocks, which is especially useful for the “boxed” versions; you can also use the inner styling here.



```
\begin{circuitikz}
  \ctikzset{block left anchors pos=0.8, block right anchors pos=0.2}
  \ctikzset{blocks/inner thickness=3, blocks/inner color=orange}
  \draw (0,3) to[bandpass, name=bp] ++(2,0)
  (1,1.5) node[gridnode](g){} (1,0) node[mixer, boxed](bm){};
  \foreach \n in {bp, bm, g} {\path
    (\n.left up) node[circ, red]{}
    (\n.left down) node[circ, blue]{}
    (\n.right up) node[circ, green]{}
    (\n.right down) node[circ, orange]{};};
\end{circuitikz}
```

To set both left and right at the same value, you can use the key `block lateral anchors pos`, which set both to the same number. You can use those anchors to build “mixed-type” circuits, using the node-shapes (the following drawing is a bit silly, but shows that the anchor position can be changed locally):

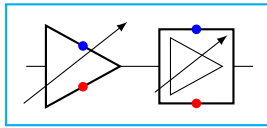


```
\begin{tikzpicture}[
  big/.style={circuitikz/blocks/scale=1.5},
  long/.style={circuitikz/bipoles/twoportsplit/width=1.5}]
  \ctikzset{block lateral anchors pos=0.8}
  \path (0,0) node[sacdcshape, big, circuitikz/block right anchors pos=0.2](A){}
    (5,0) node[twoportsplitshape, big, long, t1=LNA, t2=Digital](B){};
  \draw (A.right up) -- (B.left up) (A.right down) to[cute choke] (B.left down);
\end{tikzpicture}
```

Notice also from the previous example that the generic blocks (`twoport` and `twoportsplit`) can be made “longer” by setting different `width` and `height` (the other blocks are square, and just use the `width` key for both dimensions).

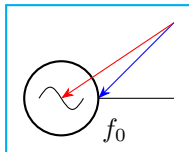
Also, for `amp` and `vamp`, the `up` and `down` anchors follow the shape when they are not boxed.

⁷⁹Suggested by [user @sputeanus](#) on GitHub



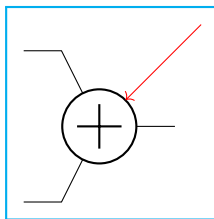
```
\begin{tikzpicture}
\draw (0,0) to[vamp, name=a] ++(1.5,0)
to [vamp, boxed, name=ab] ++(1.5,0);
\path (a.up) node[circ, blue]{} (ab.up) node[circ, blue]{};
\path (a.down) node[circ, red]{} (ab.down) node[circ, red]{};
\end{tikzpicture}
```

The `oscillator` has a displaced `center` anchor, to simplify the task of putting it at the left side of a circuit; it also has a special position for the node text. The four round elements (mixer, circulator, adder, and the oscillator) have a `geocenter` anchor which always corresponds to the center of the circle.



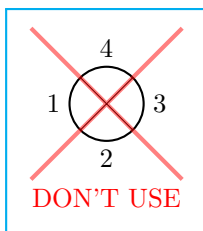
```
\begin{tikzpicture}[>=Stealth]
\draw (0,0) node[oscillator] (0) {\$f_0\$} -- ++(1,0);
\draw[blue, ->] (1,1) -- (0.center);
\draw[red, ->] (1,1) -- (0.geocenter);
\end{tikzpicture}
```

Moreover, they have proper border anchors since version 1.2.3 (and fixed for boxed elements in 1.5.0), so you can do things like this:



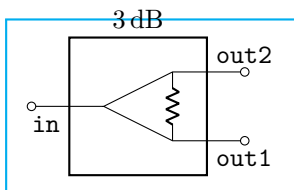
```
\begin{circuitikz}
\draw (0,0) node[adder] (mix) {}
(-1,1) -- ++(0.5,0) -- (mix)
(-1,-1) -- ++(0.5,0) -- (mix) -- ++(1,0);
\draw [red, <-] (mix.45) -- ++(1,1);
\end{circuitikz}
```

Those components have also **deprecated** anchors named 1, 2, 3, 4; they are better not used because they can conflict with the border anchor. They still work for backward compatibility, but could be removed in a future release.



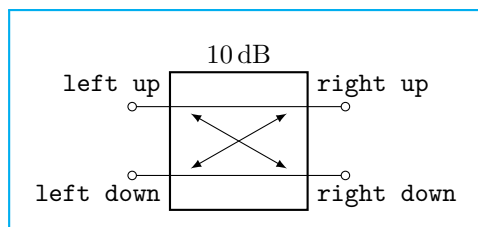
```
\begin{circuitikz} \draw
(0,0) node[mixer] (mix) {}
(mix.1) node[left] {1} (mix.2) node[below] {2}
(mix.3) node[right] {3} (mix.4) node[above] {4};
\draw [ultra thick, red, opacity=0.5]
(-1,-1)--(1,1) (-1,1)--(1,-1);
\node [red, below] at (0,-1) {DON'T USE};
\end{circuitikz}
```

The Wilkinson divider has (notice that the node text is outside the bounding box, similarly to what happens for transistors!):



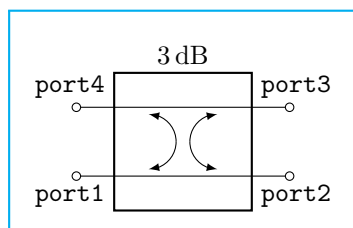
```
\begin{circuitikz} \draw
(0,0) node[wilkinson] (w) {\SI{3}{dB}}
(w.in) to[short,-o] ++(-0.5,0)
(w.out1) to[short,-o] ++(0.5,0)
(w.out2) to[short,-o] ++(0.5,0)
(w.in) node[below left] {\texttt{in}}
(w.out1) node[below right] {\texttt{out1}}
(w.out2) node[above right] {\texttt{out2}}
;
\end{circuitikz}
```

The couplers have:



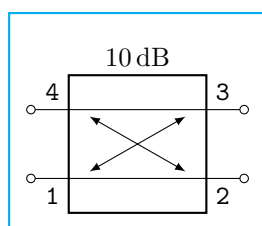
```
\begin{circuitikz} \draw (0,1.5) %bounding box
(0,0) node[coupler] (c) {\SI{10}{dB}}
(c.left down) to[short,-o] ++(-0.5,0)
(c.right down) to[short,-o] ++(0.5,0)
(c.right up) to[short,-o] ++(0.5,0)
(c.left up) to[short,-o] ++(-0.5,0)
(c.left down) node[below left] {\texttt{left
down}}
(c.right down) node[below right] {\texttt{right
down}}
(c.right up) node[above right] {\texttt{right
up}}
(c.left up) node[above left] {\texttt{left up}}
;
\end{circuitikz}
```

Or you can also use port1 to port4 if you prefer:



```
\begin{circuitikz} \draw (0,1.5) %bounding box
(0,0) node[coupler2] (c) {\SI{3}{dB}}
(c.port1) to[short,-o] ++(-0.5,0)
(c.port2) to[short,-o] ++(0.5,0)
(c.port3) to[short,-o] ++(0.5,0)
(c.port4) to[short,-o] ++(-0.5,0)
(c.port1) node[below left] {\texttt{port1}}
(c.port2) node[below right] {\texttt{port2}}
(c.port3) node[above right] {\texttt{port3}}
(c.port4) node[above left] {\texttt{port4}}
;
\end{circuitikz}
```

Also they have the simpler 1, 2, 3, 4 anchors, and although they have no border anchors (for now), it is better not to use them.

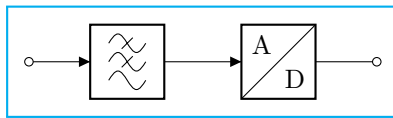


```
\begin{circuitikz} \draw(0,1.5) %bounding box
(0,0) node[coupler] (c) {\SI{10}{dB}}
(c.1) to[short,-o] ++(-0.5,0)
(c.2) to[short,-o] ++(0.5,0)
(c.3) to[short,-o] ++(0.5,0)
(c.4) to[short,-o] ++(-0.5,0)
(c.1) node[below left] {\texttt{1}}
(c.2) node[below right] {\texttt{2}}
(c.3) node[above right] {\texttt{3}}
(c.4) node[above left] {\texttt{4}}
;
\end{circuitikz}
```

4.14.2 Blocks customization

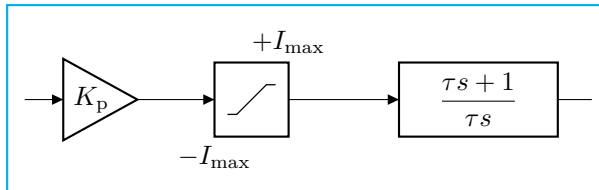
You can change the scale of all the block elements by setting the key `blocks/scale` to something different from the default 1.0.

With the option `>` you can draw an arrow to the input of the block diagram symbols.



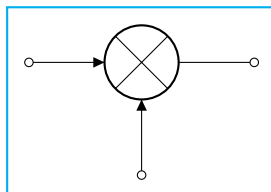
```
\begin{circuitikz} \draw
(0,0) to[short,o-] ++(0.3,0)
to[lowpass,>] ++(2,0)
to[adc,>] ++(2,0)
to[short,-o] ++(0.3,0);
\end{circuitikz}
```

You can also change the width for the generic block:



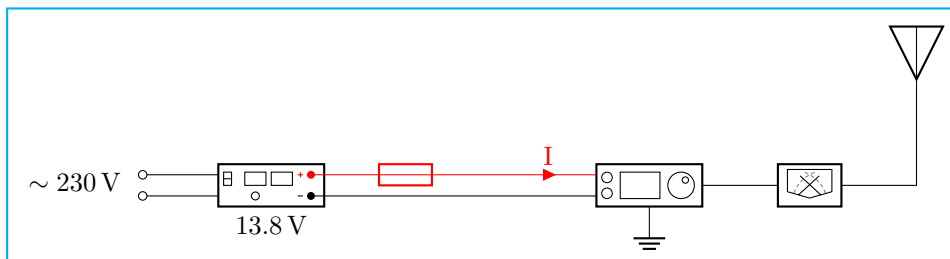
```
\begin{circuitikz}[european]
\draw (0,0) to[amp, >, t=$K_{\mathrm{p}}$] ++(2,0)
to[saturation, >, name=sat] ++(2,0) -- ++(0.5,0)
to[twoport, t=$\displaystyle\frac{\tau s + 1}{\tau s}$, >,
bipoles/twoport/width=1.5] ++(3,0);
\node[above] at (sat.ne) {$+I_{\mathrm{max}}$};
\node[below] at (sat.sw) {$-I_{\mathrm{max}}$};
\end{circuitikz}
```

4.14.2.1 Multi ports Since inputs and outputs can vary, input arrows can be placed as nodes. Note that you have to rotate the arrow on your own:



```
\begin{circuitikz} \draw
(0,0) node[mixer] (m) {}
(m.w) to[short,-o] ++(-1,0)
(m.s) to[short,-o] ++(0,-1)
(m.e) to[short,-o] ++(1,0)
(m.w) node[inputarrow] {}
(m.s) node[inputarrow,rotate=90] {};
\end{circuitikz}
```

You can use additional anchors to mix different kinds of elements.



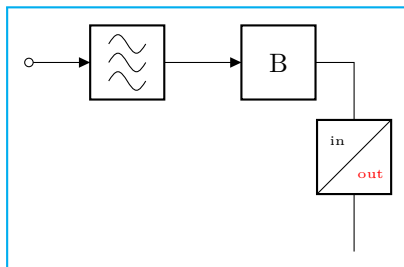
```
% Author: Prof. Dr. Matthias Jung Year: 2023
\begin{tikzpicture}
%\useasboundingbox (-3.75,-1.0) rectangle (10.5,3);
\node[draw, powershape] at (0,0) (supply) {};
\draw(supply.south) node[anchor=north]{\qty{13,8}{V}};
```

```

\draw[draw, trxshape] at (5,0) (trx) {};
\draw(trx.east) -- ++(1,0) node[swrshape, anchor=west] (swr){};
\draw(swr.east) -- ++(1,0) node[antenna]{};
\draw(trx.south) node[ground]{};
\draw[red](supply.plus) to [fuse, color=red] ++(2.5,0)
to [short, i={I}, color=red] (trx.plus);
\draw(trx.minus) to [short, i={}] (supply.minus);
\draw(supply.u1) to [short, -o] ++(-1,0) coordinate(c1);
\draw(supply.u2) to [short, -o] ++(-1,0) coordinate(c2);
\draw(c1) to [open, l_={$\sim\text{qty}{230}\{\text{volt}\}$}] (c2);
\end{tikzpicture}

```

4.14.2.2 Labels and custom two-port boxes You can use the keys `t`, `t1`, `t2` (shorthands for `text`, `text in`, `text out`) to fill the generic blocks:

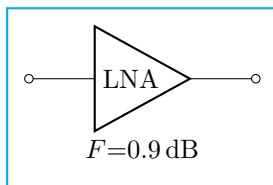


```

\begin{circuitikz} \draw
(0,0) to[short,o-] ++(0.3,0)
to[allpass,>] ++(2,0)
to[twoport,>,t={B}] ++(2,0)
to[twoportsplit,t1={\tiny in},
t2={\tiny\color{red}out}] ++(0,-2.5);
\end{circuitikz}

```

Some two-ports have the option to place a normal label (`l=`) and a inner label (`t=`).

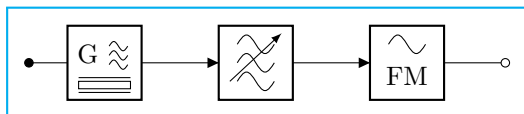


```

\begin{circuitikz}
\ctikzset{bipoles/amp/width=0.9}
\draw (0,0) to[amp,t=LNA,l={$F=0.9\,\text{dB}$},o-o] ++(3,0);
\end{circuitikz}

```

Each block determines where the text is added:



```

\begin{circuitikz} \draw
(0,0) to[qgenerator,t=G,*-] ++(2,0)
to[vallpass,>] ++(2,0)
to[negenerator,t=FM,>] ++(2,0)
to[short,-o] ++(0.3,0);
\end{circuitikz}

```

For the `twoportsplit` block, you can change how the text is positioned using the key (shared with amplifiers and logic ports, so better use it locally) `component text` (default value `center`), which gives the standard positioning (text centered in each one of the triangles in which the block is divided). If you set it to the value `border`, the text will be positioned at the corner (upper left or bottom right, depending on orientation) with a distance set by `border text distance` (default 2pt). This option makes it easy to define additional blocks, like these [suggested by Matthias Jung](#):

```

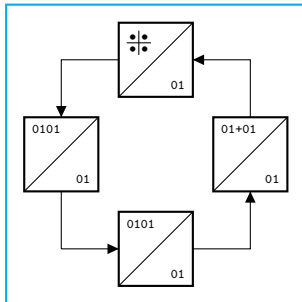
% define a macro to draw a "safe" four-dot symbol
% basic LaTeX pictures are usable inside TikZ nodes,
% no nesting problems.
\newcommand{\fourdots}{%
\begin{group}

```

```

\setlength{\unitlength}{0.8mm}%
\begin{picture}(4,4)(-2,-2)
  \put(-2,0){\line(1,0){4}}
  \put(0,-2){\line(0,1){4}}
  \multiput(-1,1)(2,0){2}{\circle*{1}}
  \multiput(-1,-1)(2,0){2}{\circle*{1}}
\end{picture}%
\endgroup
}
% define the new blocks
\tikzset{%
  tinysplits/.style 2 args={twoportsplit,
    component text=border, border text distance=3pt,
    t1={\tiny\ttfamily #1}, t2={\tiny\ttfamily #2}},
  source coder/.style={tinysplits={0101}{01}},
  channel coder/.style={tinysplits={01}{01+01}},
  mapper/.style={tinysplits={01}{\fourdots}},
}

```

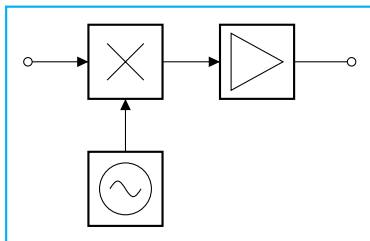


```

\begin{circuitikz}
\draw (0,0)
  to[source coder, >] ++(2.5,0)
  to[channel coder, >] ++(0,2.5)
  to[mapper, >] ++(-2.5,0)
  to[source coder,>] ++(0,-2.5)
  ;
\end{circuitikz}

```

4.14.2.3 Box option Several devices have the possibility to add a box around them with the `box` or `boxed` option. The inner symbol scales down to fit inside the box. For the “circled” devices (mixer, adder, oscillator and circulator) the inner circle is normally drawn, unless you use the `box only` or `boxed only` option.⁸⁰



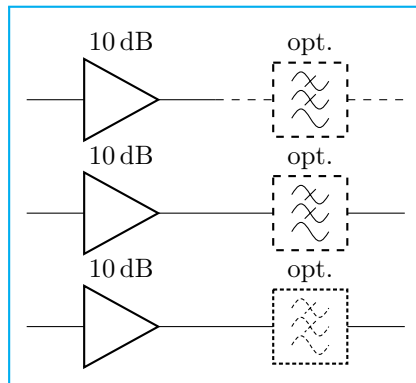
```

\begin{circuitikz} \draw
(0,0) node[mixer, box only, anchor=east](m){}
  to[amp, boxed, >, -o] ++(2.5,0)
(m.west) node[inputarrow]{} to[short,-o] ++(-0.8,0)
(m.south) node[inputarrow,rotate=90]{} --
  ++(0,-0.7) node[oscillator, box, anchor=north]{};
\end{circuitikz}

```

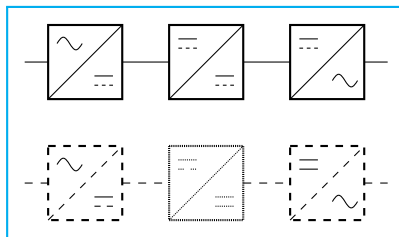
4.14.2.4 Dash optional parts To show that a device is optional, you can dash it. The inner symbol will be kept with solid lines, unless you set the key `inner blocks dashed` to true. Moreover, the key `dashed blocks pattern` (default `{\1mm}{1mm}`), be careful with the number of braces!

⁸⁰Since 1.5.0, suggested by [GitHub user myzinsky](#)



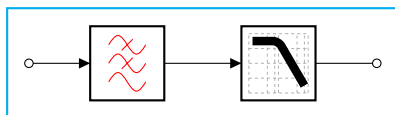
```
\begin{circuitikz}
\draw (0,1.5) to[amp,l=\SI{10}{dB}] ++(2.5,0);
\draw[dashed] (2.5,1.5) to[lowpass,l=opt.] ++(2.5,0);
% or just the block
\draw (0,0) to[amp,l=\SI{10}{dB}] ++(2.5,0)
      to[lowpass,l=opt., dashed] ++(2.5,0);
% or everything
\ctikzset{inner blocks dashed,
          dashed blocks pattern={1.5pt}{1pt}},
}
\draw (0,-1.5) to[amp,l=\SI{10}{dB}] ++(2.5,0)
      to[lowpass,l=opt., dashed] ++(2.5,0);
\end{circuitikz}
```

4.14.2.5 Dashing the DC symbol in blocks. The symbol for the DC side can be different across countries,⁸¹ with different kind of dashing on the bottom line. Moreover, sometimes the dashing is used to convey different meanings (like rectified sinusoidal or stabilized DC). You can change the general style for the DC symbol using the key `blocks dc segments`; using 1 (default) will use a continuous line; using 2 you will have the international-styled symbol, and with 3 the English one (you can use higher numbers; the only restriction is that it must be strictly greater than 0). You can also change the input and output part separately with the keys `blocks dc in segments` and `blocks dc out segments` (see the following example). The `inner blocks dashed` can have strange interaction with these ones.



```
\begin{tikzpicture}[scale=0.8]
\ctikzset{blocks dc segments=3}
\draw (0,2) to[sacdc] ++(2,0) to[sdcac]
      ++(2,0) to[sdcac] ++(2,0);
\ctikzset{blocks dc segments=1}
\draw[dashed] (0,0) to[sacdc, blocks dc out segments=2]
      ++(2,0) to[sdcac, blocks dc in segments=2,
        inner blocks dashed,
        dashed blocks pattern={0.4pt}{0.4pt}]
      ++(2,0) to[sdcac] ++(2,0);
\end{tikzpicture}
```

4.14.2.6 Blocks inner drawing. Almost all of the inner block drawing style can be customized, without affecting the external box, with the keys `blocks/inner color` (default `default`, which means “don’t change color”) and `blocks/inner thickness` (default 1; this is relative to the class-specified thickness).

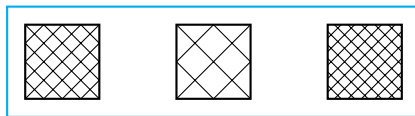


```
\begin{circuitikz} \draw
(0,0) to[short,o-] ++(0.3,0)
to[lowpass,>, blocks/inner color=red ] ++(2,0)
to[lowpassp,>, blocks/inner thickness=4,
  blocks/filter grid/thickness=0.25] ++(2,0)
to[short,-o] ++(0.3,0);
\end{circuitikz}
```

The key `blocks/gridnode/gridlines` controls the number of lines in the `gridnode` block, defined as parallel lines in one diagonal. This can be customised using the key, which accepts any positive integer (default is 7).⁸²

⁸¹Head-up from [user @dbstf on GitHub](#)

⁸²Contributed by [Jakob Leide on GitHub](#)



```
\begin{circuitikz}
  \node[gridnode] at (0,0) {};
  \node[gridnode,gridlines=3] at (2,0) {};
  \ctikzset{blocks/gridnode/gridlines=10}
  \node[gridnode] at (4,0) {};
\end{circuitikz}
```

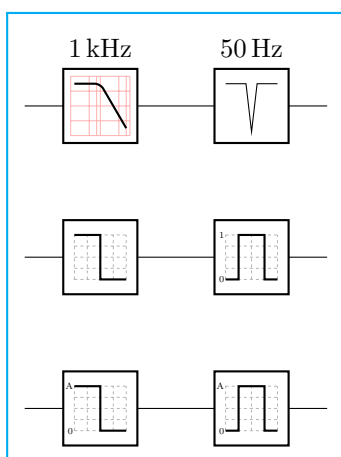
4.14.2.7 Plot-type filter blocks customization. In the “plot”-type filter blocks, you can change the appearance of the grid using the following parameters (with `\ctikzset`, under the family `blocks/filter grid/`)

parameter	default	description
<code>thickness</code>	0.5	line thickness, this is relative to the <code>inner thickness</code> setting for the block
<code>color</code>	default	stroke color: <code>default</code> is the same as the component color (<code>inner color</code> if it's set)
<code>dash</code>	<code>{{0.04cm}{0.04cm}}</code>	dash pattern: none means solid line, default means keep the global pattern ⁸³
<code>opacity</code>	0.3	stroke opacity for the grid. Using 0 will suppress the drawing of the grid completely.

Additionally, the `blocks/filter plot/relative thickness` (default 2) will change the relative thickness (with respect to the class thickness, or `inner thickness` if set) of the “frequency response” sketch line.

Finally, the plots are by default rounded⁸⁴, but you can choose to have them with sharp edges setting the key `blocks/filter plot/rounding` to 0 (the default value is 0.15).

Setting the key `ideal filter numbers` a small 0 and 1 are added at the left side of the grid. The two numbers can be customized using the keys `ideal filter number low` and `ideal filter number high` (by default set to 0 and 1 respectively), but notice that there is very little space there.



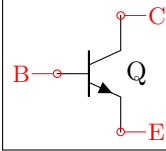
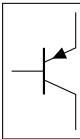
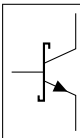
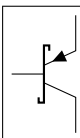
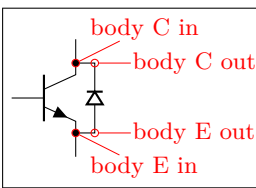
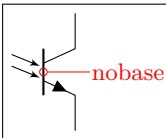
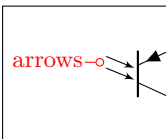
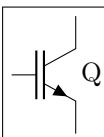
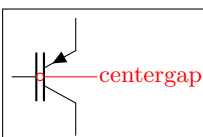
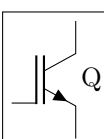
```
\begin{circuitikz}
  \draw (0,0) to[lowpass=\qty{1}{\kHz},
    blocks/filter grid/color=red,
    blocks/filter grid/dash=none,
    blocks/filter grid/thickness=1] ++(2,0)
    to[notchp=\qty{50}{\hertz},
    blocks/filter grid/opacity=0,
    blocks/filter plot/relative thickness=1,
    blocks/filter plot/rounding=0] ++(2,0);
  \draw (0,-2) to[lowpass, ideal filter] ++(2,0)
    to[bandpass, ideal filter,
    ideal filter numbers] ++(2,0);
  \ctikzset{ideal filter numbers, % set as default
    ideal filter number high=A}
  \draw (0,-4) to[lowpass, ideal filter] ++(2,0)
    to[bandpass, ideal filter] ++(2,0);
\end{circuitikz}
```

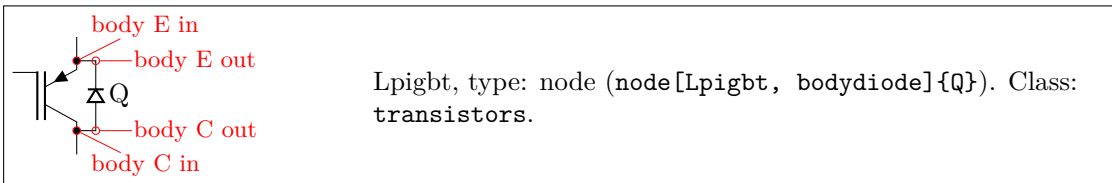
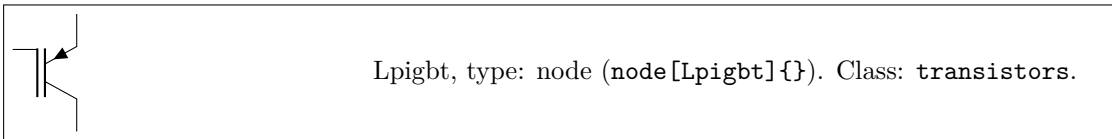
⁸³Follows the syntax of the pattern sequence `\pgfsetdash` — see TikZ manual for details; phase is always zero. Basically you pass pairs of dash-length – blank-length dimensions, see the examples. Notice that the dimensions are not exactly cm: the length are scaled so that 1 cm is really the block width.

⁸⁴This, and the hint for the new filter shapes, were suggested by @cis on the TeX-SX chat. Ideal filters ignore the roundness parameter.

4.15 Transistors

4.15.1 Standard bipolar transistors

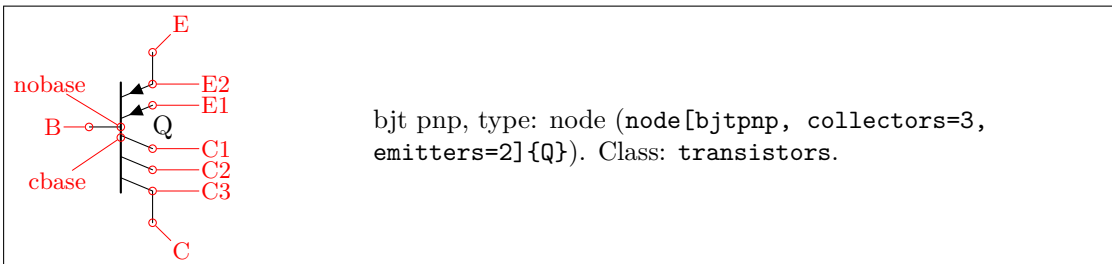
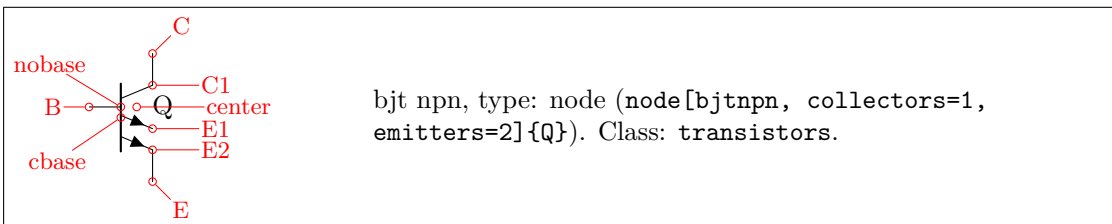
	nnp, type: node (node[nnp]{Q}). Class: transistors.
	pnp, type: node (node[pnp]{}). Class: transistors.
	schottky npn, type: node (node[nnp, schottky base]{}). Class: transistors.
	schottky pnp, type: node (node[pnp, schottky base]{}). Class: transistors.
	nnp, type: node (node[nnp, bodydiode]{}). Class: transistors.
	photo npn, type: node (node[nnp,photo]{}). Class: transistors.
	photo pnp, type: node (node[pnp,photo]{}). Class: transistors.
	nigbt, type: node (node[nigbt]{Q}). Class: transistors.
	pigbt, type: node (node[pigbt]{}). Class: transistors.
	Lnight, type: node (node[Lnight]{Q}). Class: transistors.



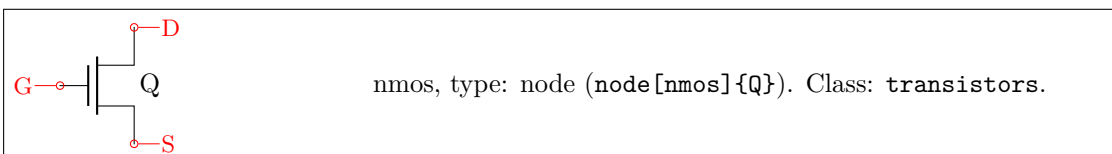
4.15.2 Multi-terminal bipolar transistors

In addition to the standard BJTs transistors, since version 0.9.6 the **bjtnpn** and **bjtpnp** are also available; these are devices where you can have more collectors and emitters (on the other hand, they have no **photo** nor **bodydiode** options — they are silently ignored).

Basically they are the same as the normal **npn** and **pnp**, and they (by default) have similar sizes; the options **collectors** and **emitters** will change the number of the relative terminals. The base terminal is connected midway from the collector and the emitter, *not* on the center of the base; a **cbase** anchor is available if you prefer to use it. The label of the component (the text) is set on the right side, vertically centered around the base terminal. They will accept the **schottky base** key.



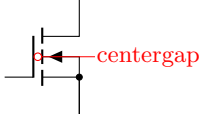
4.15.3 Field-effect transistors



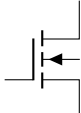
	pmos, type: node (node[pmos]{}). Class: transistors.
	nmos depletion, type: node (node[nmosd]{Q}). Class: transistors.
	pmos depletion, type: node (node[pmosd]{}). Class: transistors.
	hemt, type: node (node[hemt]{}). Class: transistors.
	hemt without base terminal, type: node (node[hemt, nobase]{Q}). Class: transistors.
	Gallium Nitride hemt (a “styled” hemt, see 4.15.5.7), type: node (node[GaN hemt]{Q}). Class: transistors.

NFETs and PFETs have been incorporated based on code provided by Clemens Helfmeier and Theodor Borsche. Use the package options `fetsolderdot`/`nofetsolderdot` to enable/disable solderdot at some fet-transistors. Additionally, the `solderdot` option can be enabled/disabled for single transistors with the option `solderdot` and `nosolderdot`, respectively (You can adjust the size of the solder dot, see section 4.15.5.14).

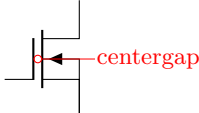
	nfet, type: node (node[nfet]{Q}). Class: transistors.
	nfet depletion, type: node (node[nfetd]{Q}). Class: transistors.
	nigfete, type: node (node[nigfete]{Q}). Class: transistors.



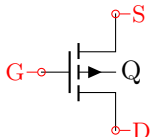
nigfete, type: node (node[nigfete,solderdot]{}). Class: transistors.



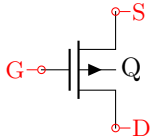
nigfetebulk, type: node (node[nigfetebulk]{}). Class: transistors.



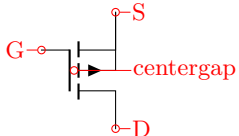
nigfetd, type: node (node[nigfetd]{}). Class: transistors.



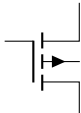
pfet, type: node (node[pfet]{Q}). Class: transistors.



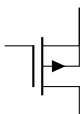
pfet depletion, type: node (node[pfetd]{Q}). Class: transistors.



pigfete, type: node (node[pigfete]{}). Class: transistors.

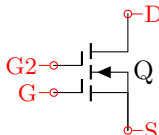


pigfetebulk, type: node (node[pigfetebulk]{}). Class: transistors.

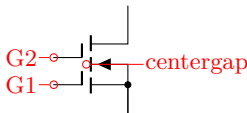


pigfetd, type: node (node[pigfetd]{}). Class: transistors.

Since version 1.6.0, you can add the `doublegate` option to the `*igfet*` family of devices to have a double gate MOS — the additional gate is called `G2` or `gate2` (the plain `G` is where it will be without the `doublegate` option).



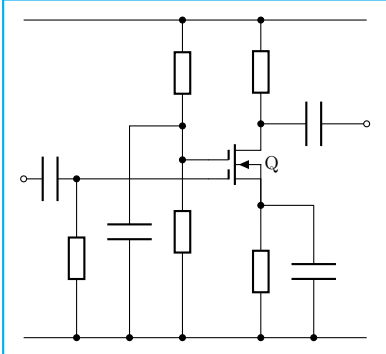
nigfete, doublegate, type: node (node[nigfete,doublegate]{Q}). Class: transistors.



nigfete, doublegate, type: node (node[nigfete,solderdot,doublegate]{}). Class: transistors.

	nignfetebulk, doublegate, type: node (node[nignfetebulk, doublegate]{}). Class: transistors.
	nignfetd, doublegate, type: node (node[nignfetd, doublegate]{}). Class: transistors.
	pigfete, doublegate, type: node (node[pigfete, doublegate]{}). Class: transistors.
	pigfetebulk, doublegate, type: node (node[pigfetebulk, doublegate]{}). Class: transistors.
	pigfetd, doublegate, type: node (node[pigfetd, doublegate]{}). Class: transistors.

You can use the double-gated transistor for example like this:⁸⁵



```

\begin{circuitikz}[european, scale=0.7, transform shape,
circuitikz/resistors/scale=0.7]
\draw (0,0) to[C, o-] ++(1,0) coordinate(in)
to[R, -] ++(0,-3) coordinate(GND)
++(1,0) to[C, *-] ++(0,4) -- ++(1,0) coordinate(div)
to[R, *-] ++(0,2) coordinate(vdd)
(div) to[R, -] (div|-GND)
(in) -- ++(2.5,0) node[nignfetd, doublegate, anchor=G1](Q){Q}
(Q.G2) to[short, -] (Q.G2|-div)
(Q.D) to[R, -] (Q.D|-vdd)
(Q.D) to[C, *-o] ++(2,0) coordinate(out)
(Q.S) to[R, *-] (Q.S|-GND) (Q.S) -- ++(1,0) coordinate(Sd)
(Sd) to[C, -] (Sd|-GND);
\draw (0,0|-GND) -- (out|-GND) (0,0|-vdd) -- (out|-vdd);
\end{circuitikz}

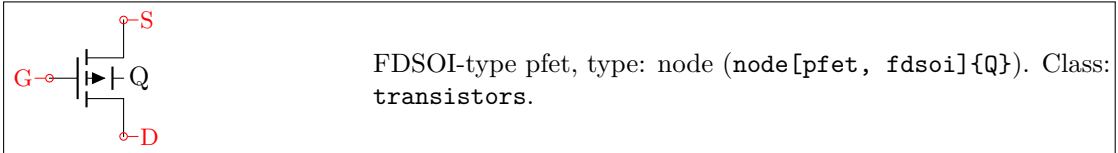
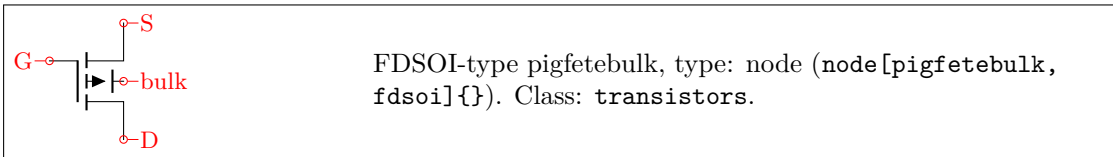
```

Since version v1.8.1 the `fdsoi` flag has been added to draw back gates for `FDSOI` types transistors.⁸⁶ The flag acts on all the FET transistors.

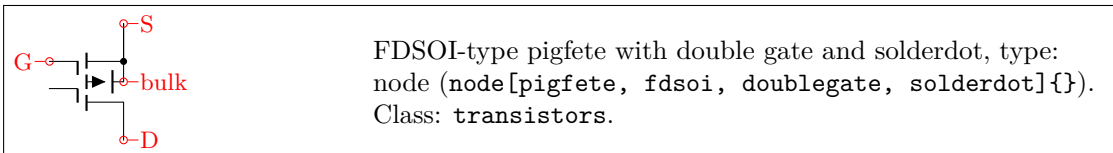
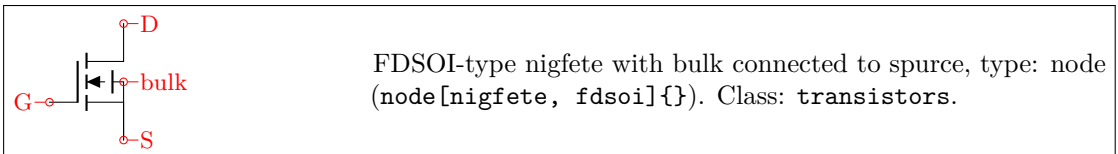
	FDSOI-type nignfetebulk, type: node (node[nignfetebulk, fdsoi]{}). Class: transistors.
	FDSOI-type nfet, type: node (node[nfet, fdsoi]{Q}). Class: transistors.

⁸⁵Found at [this page on electronics notes](#).

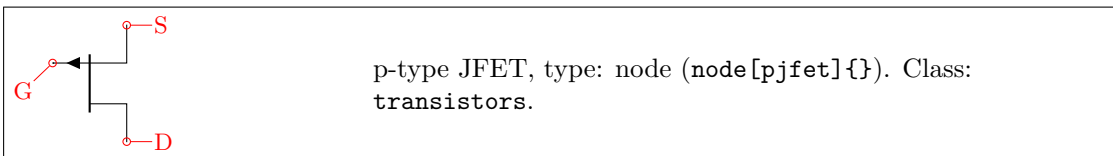
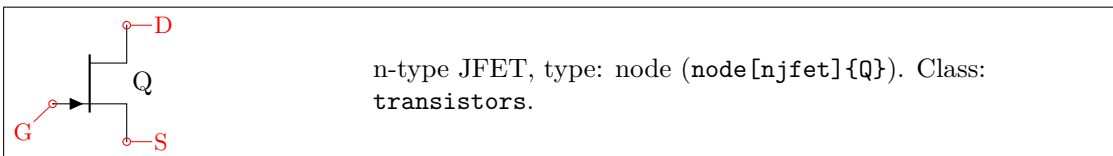
⁸⁶Suggested via email by Raphael Nägele <raphael.naegele@int.uni-stuttgart.de>; see also [this issue on GitHub](#).



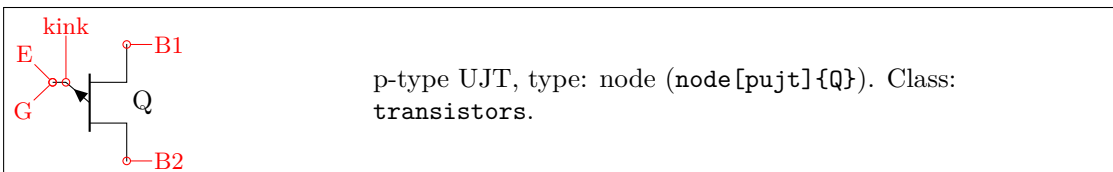
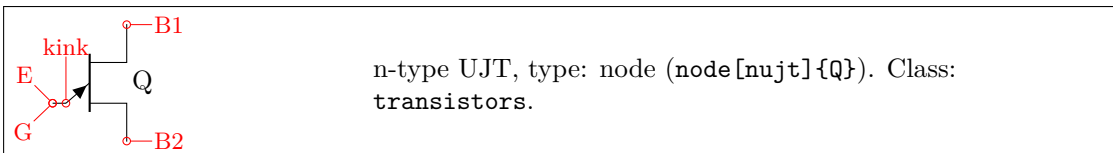
and also with other FET-based shapes with bulk connection:



JFET are also available⁸⁷, both n-type and p-type.

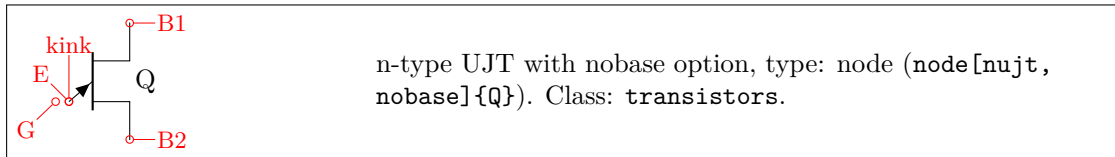


UJT transistors⁸⁸ have a different anchor names although **most** of the others, like D and G, also work (the exception is E and **emitter**!). Notice that if used with **nobase**, the anchor E follows the wire, while G is fixed (as is kink).

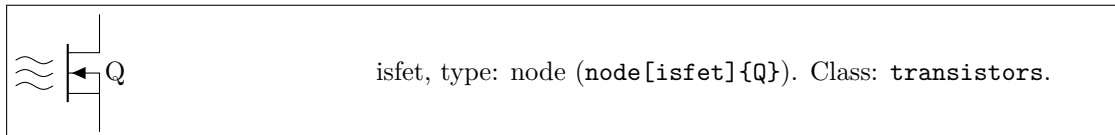


⁸⁷based on code provided by Danilo Piazzalunga

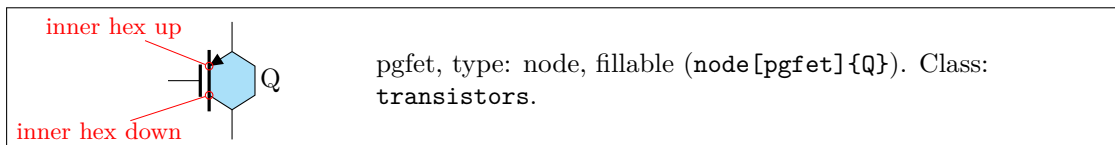
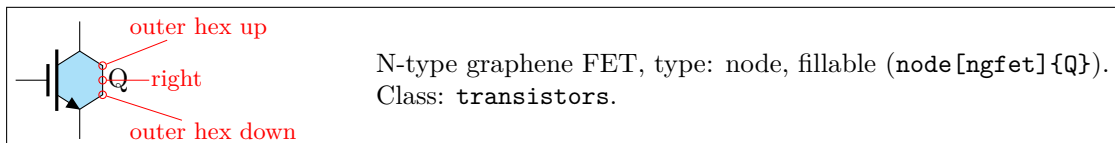
⁸⁸suggued by [user JetherReis on GitHub](#).



ISFET:



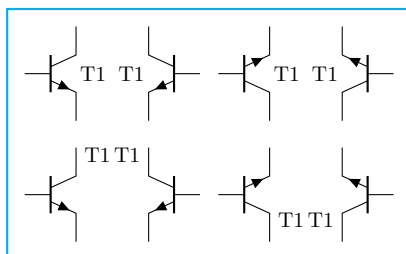
GRAPHENE FET have been added in version 1.3.2⁸⁹. They look better if you set `transistors/arrow pos=end` and `transistor/thickness=3` or higher for them, so they are plotted with this option here.



4.15.4 Transistor texts (labels)

In versions before 0.9.7, transistors text (the node text) was positioned near the collector terminal; since version 0.9.7 the default has been changed to a more natural position near the center of the device, similar to the multi-terminal transistors. You can revert to the old behavior locally with the key `legacy transistors text`, or globally by setting the package option `legacytransistorstext`.

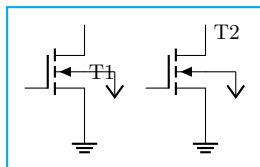
Notice the use of the utility functions `\ctikzflip{x,y,xy}` as explained in section 3.2.1.



```
\begin{circuitikz}[scale=0.8, transform shape]
  \draw (0,0) node [npn]{T1}
  ++(1.2,0) node [npn, xscale=-1]{\ctikzflipx{T1}}
  ++(2,0) node [npn, yscale=-1]{\ctikzflipy{T1}}
  ++(1.2,0) node [npn, scale=-1]{\ctikzflipxy{T1}};
  \ctikzset{legacy transistors text}
  \draw (0,-2) node [npn]{T1}
  ++(1.2,0) node [npn, xscale=-1]{\ctikzflipx{T1}}
  ++(2,0) node [npn, yscale=-1]{\ctikzflipy{T1}}
  ++(1.2,0) node [npn, scale=-1]{\ctikzflipxy{T1}};
\end{circuitikz}
```

⁸⁹added by Romano Giannetti after a suggestion by Cees Keyer.

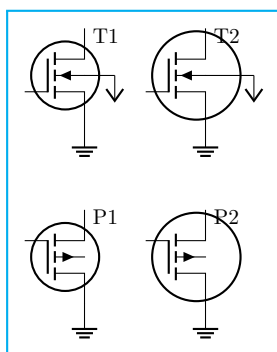
There is a situation where using the legacy position makes sense: if you have a transistor with a bulk pin, and you need to connect that pin to something outside the component:



```
\begin{circuitikz}[scale=0.8, transform shape]
\draw (0,0) node [nigfetebulk](T1){T1} (T1.S) node[ground]{}
(T1.bulk) -- ++(0.5, 0) node[vee]{};
\draw (2,0) node [nigfetebulk, legacy transistors text](T2){T2}

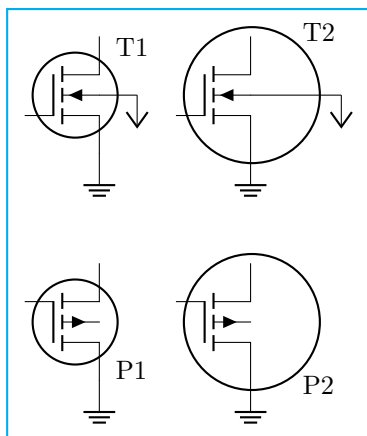
(T2.S) node[ground]{}
(T2.bulk) -- ++(0.5, 0) node[vee]{};
\end{circuitikz}
```

You can use the `legacy transistors text` to position the label “up”, near the drain (or source, whichever is up) of the transistor — but then we have a problem, if you use, for example, a transistor circle (it works by chance with the standard sizes, but if you change the size of the circle, you’ll have problems:)

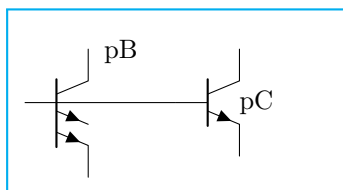


```
\begin{circuitikz}[scale=0.8, transform shape]
\ctikzset{legacy transistors text, tr circle}
\draw (0,0) node [nigfetebulk](T1){T1} (T1.S) node[ground]{}
(T1.bulk) -- ++(0.5, 0) node[vee]{}
(0,-3) node [pigfetebulk](P1){P1} (P1.D) node[ground]{};
\ctikzset{transistor circle/scale circle radius=1.3}
\draw (2,0) node [nigfetebulk](T2){T2}
(2,-3) node [pigfetebulk](P2){P2} (P2.D) node[ground]{}
(T2.S) node[ground]{}
(T2.bulk) -- ++(0.8, 0) node[vee]{};
\end{circuitikz}
```

Since version 1.8.2 you can solve this with `component text=up` (or `down`, default `center`). This flag will move the text label of the transistors near the upper (or lower, respectively) terminal, a bit offset to the right. The value of this flag is taken into account only for transistors (at least, for now).



```
\begin{circuitikz}
\ctikzset{component text=up, tr circle}
\draw (0,0) node [nigfetebulk](T1){T1}
(T1.S) node[ground]{}
(T1.bulk) -- ++(0.5, 0) node[vee]{}
(0,-3) node [pigfetebulk, component text=down](P1){P1}
(P1.D) node[ground]{};
\ctikzset{transistor circle/scale circle radius=1.6}
\draw (2,0) node [nigfetebulk](T2){T2}
(2,-3) node [pigfetebulk, component text=down](P2){P2}
(P2.D) node[ground]{}
(T2.S) node[ground]{}
(T2.bulk) -- ++(1.2, 0) node[vee]{};
\end{circuitikz}
```

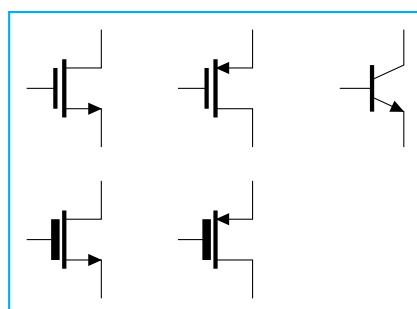


```
\begin{circuitikz}
\draw (0,0) node[bjtnpn, emitters=2,
component text=up](B){pB} ++(2,0)
node[bjtnpn](C){pC} (B.nobase) -- (C.base)
++(2,1); %adjust bounding box
\end{circuitikz}
```

4.15.5 Transistors customization

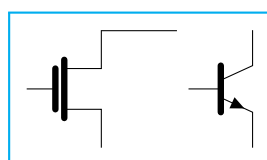
4.15.5.1 Size. You can change the scale of all the transistors (in scope) by setting the key `transistors/scale` (default 1.0). The size of the arrows (if any) is controlled by the same parameters as `currarrow` (see section 4.11.1) and the dots on P-type transistors (if any) are the same as the nodes/poles (see section 6.1).

4.15.5.2 Thickness. You can change the thickness of the gate and base (channel) segments with the key `transistors/thickness`; additionally, since v1.8.6, you can use the key `transistors/gate thickness` which will be interpreted as the *relative* thickness of the gate segment⁹⁰.



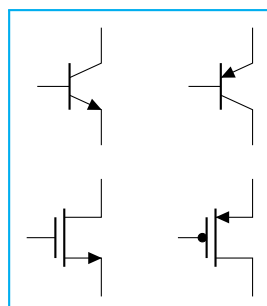
```
\begin{tikzpicture}[]
  \ctikzset{tripoles/.cd, mos style=arrows,
    pmos style/nocircle}
  \ctikzset{transistors/.cd, thickness=4,
    arrow pos=end}
  \draw (0,0) node[nmos]{} ++(2,0) node[pmos]{}
    ++(2,0) node[npn]{};
  \ctikzset{transistors/gate thickness=2}
  \draw (0,-2) node[nmos]{} ++(2,0) node[pmos]{};
\end{tikzpicture}
```

4.15.5.3 Rounded caps for gate and base. You can notice that with very thick segments the drawing is quite harsh. If you prefer to use rounded caps for the lines, you can use the boolean key `transistors/rounded caps` (default false):



```
\begin{tikzpicture}[]
  \ctikzset{transistors/.cd, thickness=6, rounded caps=true}
  \draw (0,-2) node[nmos](N){} ++(2,0) node[npn]{};
  \draw (N.C) -- ++(1,0);
\end{tikzpicture}
```

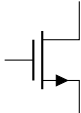
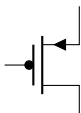
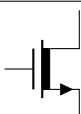
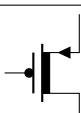
4.15.5.4 Arrows. The default position of the arrows in transistors is somewhat in the middle of the terminal; if you prefer you can move them to the end with the style key `transistors/arrow pos=end` (the default value is `legacy`).



```
\begin{circuitikz}
  \ctikzset{tripoles/mos style=arrows}
  \ctikzset{transistors/arrow pos=end}
  \draw (0,0) node[npn, ](npn){};
  \draw (2,0) node[pnp, ](pnp){};
  \draw (0,-2) node[nmos, ](nmos){};
  \draw (2,-2) node[pmos, ](pmos){};
\end{circuitikz}
```

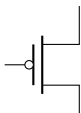
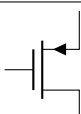
⁹⁰Suggested by [Michael Köfinger et al.](#)

If the option `arrowmos` is used (or after the command `\ctikzset{tripoles/mos style/arrows}` is given), this is the output:

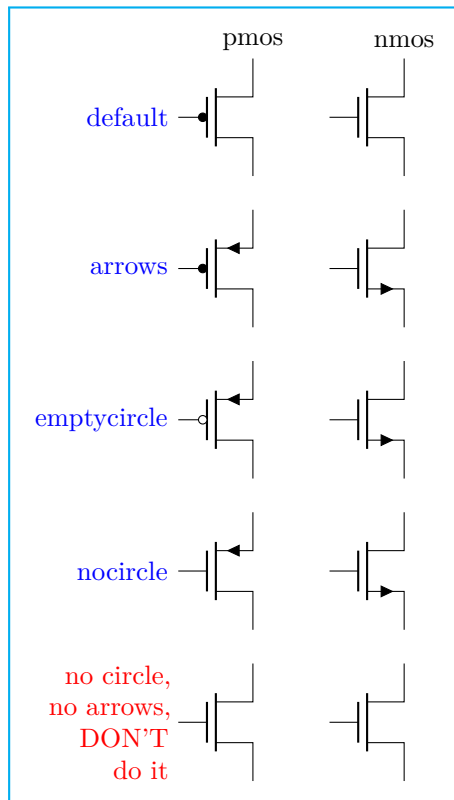
	<code>nmos, type: node (node[nmos]{}). Class: transistors.</code>
	<code>pmos, type: node (node[pmos]{}). Class: transistors.</code>
	<code>nmos depletion, type: node (node[nmosd]{}). Class: transistors.</code>
	<code>pmos depletion, type: node (node[pmosd]{}). Class: transistors.</code>

You can go back to the no-arrows mos with `noarrowmos` locally or with `\ctikzset{tripoles/mos style/no arrows}`.

To draw the PMOS circle non-solid, use the option `emptycircle` or the command `\ctikzset{tripoles/pmos style/emptycircle}`. To remove the dot completely (only useful if you have `arrowmos` enabled, otherwise there will be no difference between P-MOS and N-MOS), you can use the option `nocircle` or `\ctikzset{tripoles/pmos style/nocircle}`.

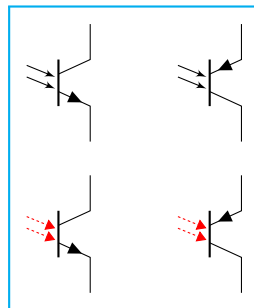
	<code>pmos, type: node (node[pmos,emptycircle]{}). Class: transistors.</code>
	<code>pmos, type: node (node[pmos,nocircle,arrowmos]{}). Class: transistors.</code>

This example show the combined effects of the arrows and gate-circle options:



```
\begin{circuitikz}[
  info/.style={left=1cm, blue, text width=5em,
    align=right},]
\draw (0,1) node{pmos} (2,1) node{nmos};
\draw (0,0) node[info]{default} node[pmos]{}
(2,0) node[nmos]{};
\ctikzset{tripoles/mos style/arrows}
\draw (0,-2) node[info]{arrows} node[pmos]{}
(2,-2) node[nmos]{};
\ctikzset{tripoles/pmos style/emptycircle}
\draw (0,-4) node[info]{emptycircle} node[pmos]{}
(2,-4) node[nmos]{};
\ctikzset{tripoles/pmos style/nocircle}
\draw (0,-6) node[info]{nocircle} node[pmos]{}
(2,-6) node[nmos]{};
\ctikzset{tripoles/mos style/no arrows}
\draw (0,-8) node[info, red]{no circle, no
  arrows, DON'T do it}
node[pmos]{} (2,-8) node[nmos]{};
\end{circuitikz}
```

You can also change⁹¹ the type of the arrow for the “light rays” of the phototransistors with the generic arrows options shown in 4.2.3.3, using the name `opto`, like in the following (overdone) example. Also the `opto arrows` styling options (see section 4.4.3.1).



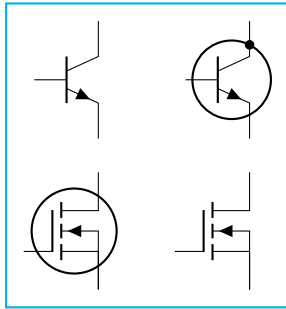
```
\begin{tikzpicture}
\draw (0,2) node[npn, photo]{} ++(2,0) node[pnp, photo]{};
\ctikzset{opto end arrow={Triangle[angle'=60]}}
\ctikzset{opto arrows/.cd, color=red, dash={1pt}{1pt}}
\draw (0,0) node[npn, photo]{} ++(2,0) node[pnp, photo]{};
\end{tikzpicture}
```

4.15.5.5 Circles. Since 1.2.6, you can add a circle⁹² to most of the transistor shapes — with the exception of multi-terminal (`bjtnpn` and `bjtpnp`, where it would be awkward anyway) and graphene FETs. The circle is intended in some case as the component’s housing, and used to distinguish discrete components from integrated ones.

To add the circle to a single transistor, you use the `tr circle` keys in the node; if you want all of your transistors with a circle, you can set the property `tr circle` with a `\ctikzset` command (it will respect normal grouping, of course); in that case, you can use `tr circle=false` to locally disable them.

⁹¹Thanks to the idea by [Dr. Matthias Jung on GitHub](#).

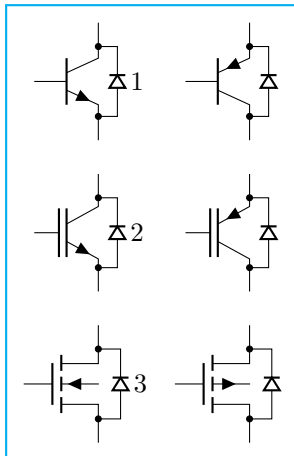
⁹²Suggested by [Dr. Matthias Jung on GitHub](#)



```
\begin{circuitikz}[]
  \draw (0,2) node[npn]{} (2,2) node[npn, tr circle](Q){};
  % collector connected to housing
  \node [circ] at (Q.circle C){};
  \ctikzset{tr circle=true} % or \ctikzset{tr circle} alone
  \draw (0,0) node[nigfete]{}
    (2,0) node[nigfete, tr circle=false]{};
\end{circuitikz}
```

You can tweak the appearance of transistor's circles and even draw it partially; see [4.15.7](#) for details.

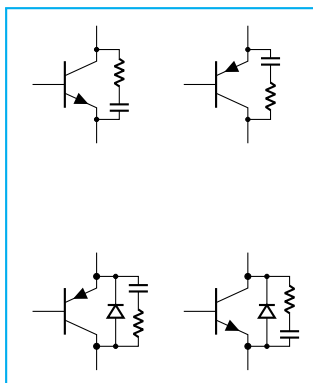
4.15.5.6 Body diodes and similar things. For all transistors (minus `bjtnpn` and `bjtpnp`) a body diode (or freewheeling or flyback diode) can automatically be drawn. Just use the global option `bodydiode`, or for single transistors, the TikZ-option `bodydiode`. As you can see in the next example, the text for the diode is moved if a `bodydiode` is present (but beware, if you change a lot the relative dimension of components, it may become misplaced):



```
\begin{circuitikz}
  \draw (0,0) node[npn,bodydiode] (npn){1}
    ++(2,0) node[pnp,bodydiode] (pnp){};
  \draw (0,-2) node[nigbt,bodydiode] (nigbt){2}
    ++(2,0) node[pigbt,bodydiode] (pnp){};
  \draw (0,-4) node[nfet,bodydiode] (npn){3}
    ++(2,0) node[pfet,bodydiode] (npn){};
\end{circuitikz}
```

You can tweak the appearance of transistor's bodydiodes; see [4.15.8](#).

For more complex snubs or protections, you can use the `body ...` anchors to add more or different things to the transistors in addition (or instead) of the flyback diode.

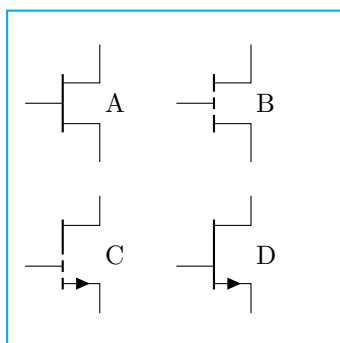


```
\def\snubb#1#2{% add a snubber to a transistor
  \draw (#1.body C #2) to[short, *, nodes width=0.02]
    ++(0.3,0) coordinate(tmp) to [R, resistors/scale=0.3]
    % 2/3 space for R, 1/3 for C
    ($ (tmp)!0.66!(tmp|-#1.body E #2)$)
    to [C, capacitors/scale=0.3] (tmp|-#1.body E #2)
    to [short, *, nodes width=0.02] (#1.body E #2);
}
\begin{circuitikz}
  \node[npn] (Q1) at(0,0) {};
  \node[pnp] (Q2) at(2,0) {};
  \node[pnp, bodydiode] (Q3) at(0,-3) {};
  \node[npn, bodydiode] (Q4) at(2,-3) {};
  \snubb{Q1}{in} \snubb{Q2}{in}
  \snubb{Q3}{out} \snubb{Q4}{out}
\end{circuitikz}
```

4.15.5.7 HEMT customization. Since v1.6.1 the shape of the `hemt` transistor can be customized.⁹³ There are several keys under the hierarchy `tripoles/hemt` that can be used to change the appearance; some of them are common to other transistors and some are specific.

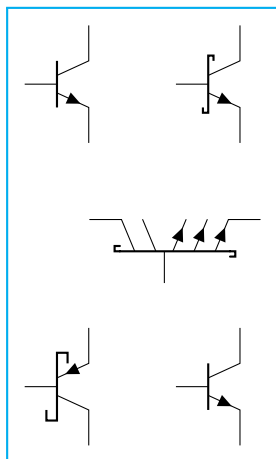
The main ones are `.../split gate` (boolean, default `false`) that will create a “split” gate, which sometimes is used to convey that the device is enhancement-type; `source arrow` (default 0), to add an arrow on the source terminal (1 for a right-facing one, -1 the other way around; the arrow will obey `arrow pos=end` if issued, but otherwise the position is fixed); `gate asym` (default 0.0) which displaces the gate asymmetrically. For example, the GaN `hemt` component is really a styled `hemt` (predefined), with the definition:

```
\tikzset{GaN hemt/.style={hemt,
    circuitikz/tripoles/hemt/base height=0.6,% length of the "base" vertical bar
    circuitikz/tripoles/hemt/gate height=0.5,% distance of the S/D terminals
    circuitikz/tripoles/hemt/bodydiode conn=0.85,% attachment point of body diode
    circuitikz/tripoles/hemt/gate asym=-0.1,% slightly down
    circuitikz/tripoles/hemt/split gate=true,% split gate
    circuitikz/tripoles/hemt/source arrow=1,% right-facing arrow
},
}
```



```
\begin{circuitikz}[]
    \path (-1,-1) rectangle (3,3);% bounding box
    \node [hemt] at (0,2) {A};
    \node [hemt,
        circuitikz/tripoles/hemt/split gate=true
    ] at (2,2) {B};
    \node [GaN hemt] at (0,0) {C};
    \node [GaN hemt,
        circuitikz/tripoles/hemt/split gate=false
    ] at (2,0) {D};
\end{circuitikz}
```

4.15.5.8 Schottky transistors. Transistors are mutated into Schottky transistors are generated by adding the `schottky base` key (there is also a `no schottky base` key that can be used if you use the other one as a default). You can change the size of the Schottky “hook” changing the parameter `tripoles/schottky base size` with `\ctikzset{}` (default 0.05; the unit is the standard resistor length, scaled if needed.)

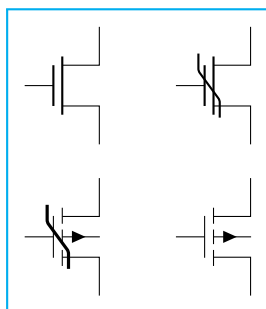


```
\begin{circuitikz}
    \draw (0,4) node[npn]{}
        ++(2,0) node[npn, schottky base]{};
    \draw (1,2) node[bjtnpn, collectors=2, emitters=3,
        schottky base, rotate=90]{};
    \tikzset{schottky base}
    \ctikzset{tripoles/schottky base size=0.1}
    \draw (0,0) node[pnp]{}
        ++(2,0) node[npn, no schottky base]{};
\end{circuitikz}
```

⁹³After a suggestion from [user @epsilon-phi on GitHub](#).

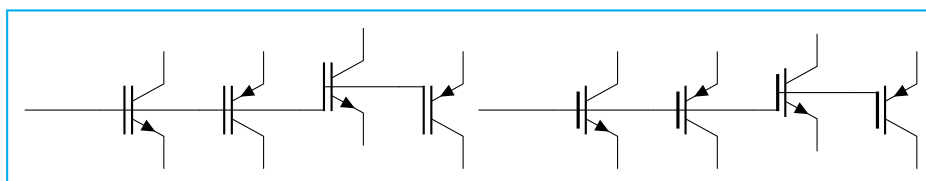
4.15.5.9 Ferroelectric transistors You can add the `ferroel` ferroelectric modifier⁹⁴ to the `*mos` and `*fet` transistor types. Similarly to the Schottky bipolar transistors, you activate it by adding the `ferroel gate` key (there is also a `no ferroel base` key that can be used if you use the other one as a default).

The mark will follow the `transistors` class thickness, but you can adjust it independently using the class parameter `modifier thickness` as in passive components — this value is relative to the class' thickness.



```
\begin{circuitikz}
  \draw (0,2) node[nmos]{}
    ++(2,0) node[nmos, ferroel gate]{};
  \ctikzset{ferroel gate} % by default from now on
  \ctikzset{transistors/.cd, % class properties
    thickness=1, modifier thickness=3}
  \draw (0,0) node[pfet]{}
    ++(2,0) node[pfet, no ferroel gate]{};
\end{circuitikz}
```

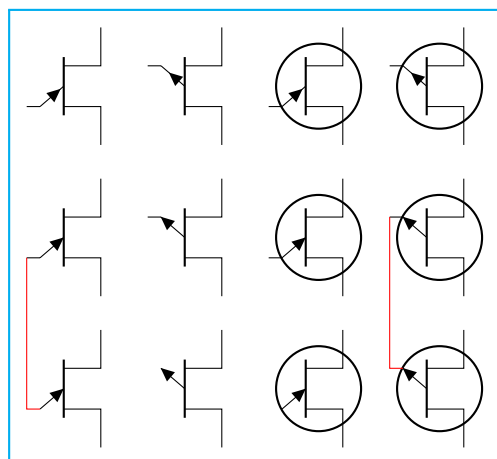
4.15.5.10 IGBT outer base. Normally, in bipolar IGBTs the outer base is the same size (height) of the inner one, and of the same thickness (which will depend on the class thickness value). You can change this by setting (via `\ctikzset`) the keys `tripoles/igbt/outer base height` (default 0.4, the same as `base height`), and `tripoles/igbt/outer base thickness` (default 1.0), which will be relative to the class thickness.



```
\begin{circuitikz}
  \draw (0,0)
    -- ++(1,0) node[nigbt, anchor=B] (B){} (B.nobase)
    -- ++(1,0) node[pigbt, anchor=B] (B){} (B.nobase)
    -- ++(1,0) node[Lnigbt, anchor=B] (B){} (B.nobase)
    -- ++(1,0) node[Lpigbt, anchor=B] (B){} (B.nobase)
    ;
  \ctikzset{tripoles/igbt/outer base height=0.3}
  \ctikzset{tripoles/igbt/outer base thickness=1.5}
  \draw (6,0)
    -- ++(1,0) node[nigbt, anchor=B] (B){} (B.nobase)
    -- ++(1,0) node[pigbt, anchor=B] (B){} (B.nobase)
    -- ++(1,0) node[Lnigbt, anchor=B] (B){} (B.nobase)
    -- ++(1,0) node[Lpigbt, anchor=B] (B){} (B.nobase)
    ;
\end{circuitikz}
```

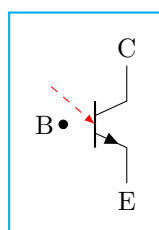
⁹⁴suggested by [Mayeul Cantan](#)

4.15.5.11 UJT transistors. They look better if you use `transistors/arrow pos=end`, especially if you use them with `tr circle`. If you use the key `nobase` with UJTs, the horizontal part of the controlling terminal is not drawn; notice that this *will* move the E or emitter anchor, but not the generic ones like G.



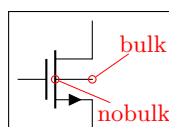
```
\begin{circuitikz}[scale=0.8]
  \draw (0,5) node[nujt]{} ++(2,0) node[pujt]{}
    ++(2,0) node[nujt, tr circle]{} ++(2,0)
    node[pujt, tr circle]{};
  \ctikzset{transistors/arrow pos=end}
  \draw (0,2.5) node[nujt](A){} ++(2,0) node[pujt]
    {}
    ++(2,0) node[nujt, tr circle]{} ++(2,0)
    node[pujt, tr circle](C){};
  \draw (0,0) node[nujt, nobase](B){} ++(2,0)
    node[pujt, nobase]{} ++(2,0)
    node[nujt, tr circle, nobase]{} ++(2,0)
    node[pujt, tr circle, nobase](D){};
  % "E" anchor follows the nobase option:
  \draw[red] (A.E) |- (B.E) (C.E) |- (D.E);
\end{circuitikz}
```

4.15.5.12 Base/Gate terminal. The Base/Gate connection of all transistors can be disabled by the options `nogate` or `nobase`, respectively. The Base/Gate anchors are floating, but there is an additional anchor `nogate/nobase`, which can be used to point to the unconnected base:

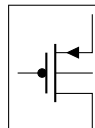


```
\begin{circuitikz}
  \draw (2,0) node[npn,nobase](npn){};
  \draw (npn.E) node[below]{E};
  \draw (npn.C) node[above]{C};
  \draw (npn.B) node[circ]{} node[left]{B};
  \draw[dashed,red,-latex] (1,0.5)--(npn.nobase);
\end{circuitikz}
```

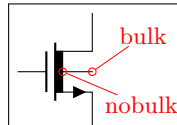
4.15.5.13 Bulk terminals. You can add a bulk terminal⁹⁵ to `nmos` and `pmos` using the key `bulk` in the node (and `nobulk` if you set the bulk terminal by default); additional anchors `bulk` and `nobulk` are added (in the next example, `tripoles/mos style/arrows` is enacted, too):



`nmos`, type: node (`node[nmos, bulk]{}).` Class: `transistors`.

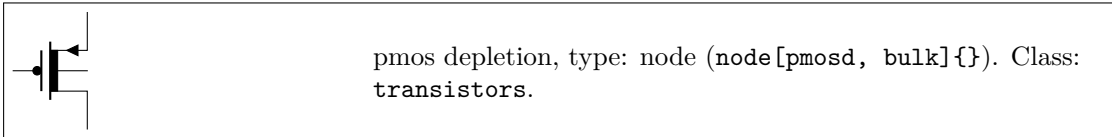


`pmos`, type: node (`node[pmos, bulk]{}).` Class: `transistors`.

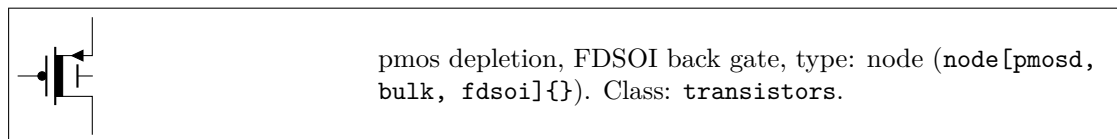
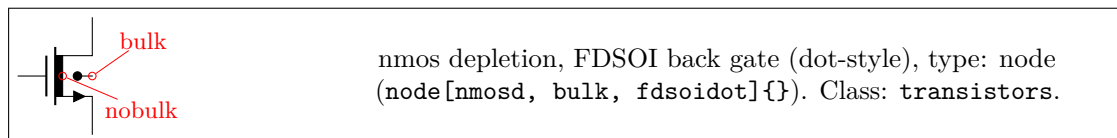
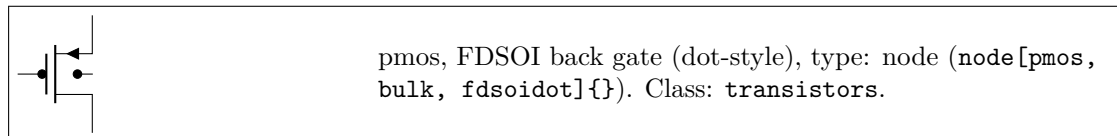
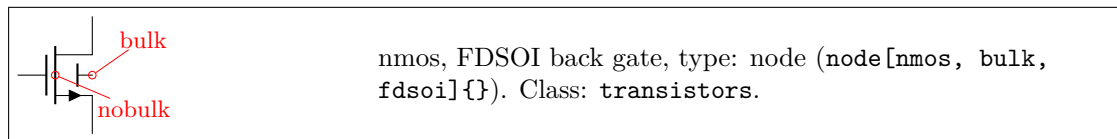


`nmos depletion`, type: node (`node[nmosd, bulk]{}).` Class: `transistors`.

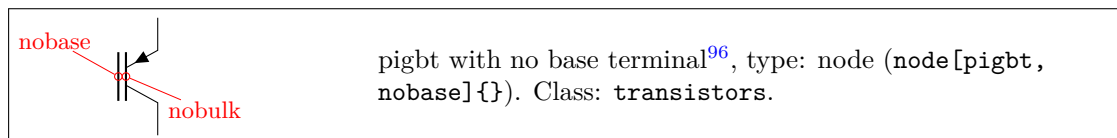
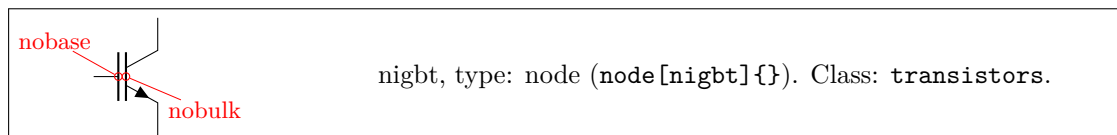
⁹⁵Thanks to Burak Kelleci <kellecib@hotmail.com>.



When adding the bulk connection, since v1.8.1, you can also add FDSOI-type back gates. In this case, there are two possible shapes for the back gate, as seen in literature. One is the one seen for the full symbol above, and the other one is a dot, which is activated by the `fdsoidot` flag.

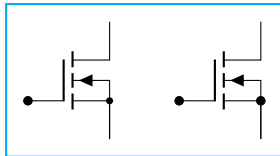


Also, IGBTs have anchors to access the “bulk” equivalent connections.



⁹⁶Since v1.4.4, noticed by [user hinata exc on Stack Exchange](#).

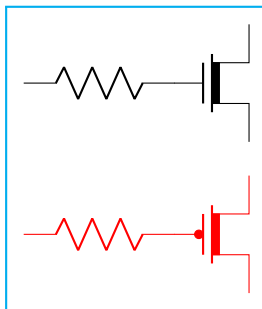
4.15.5.14 Solder dots. Solder dots are scaled-down⁹⁷ version of the normal `circ` connection dot (*pole*). This is to convey the information that the connection is *internal* to the device, and not controllable from the outside. By default, they are scaled as the bodydiode connection dots (see section 4.15.8), but you can change them with the command `\ctikzset{transistor solderdot scale=x}`, where x is the scale with respect to the normal `circ` node (default: 0.7).



```
\begin{tikzpicture}[]
  \draw (0,0) to[short,*-] ++(.1,0)
    node[nigfete, solderdot, anchor=G]{};
  \ctikzset{transistor solderdot scale=1}
  \draw (2,0) to[short,*-] ++(.1,0)
    node[nigfete, solderdot, anchor=G]{};
\end{tikzpicture}
```

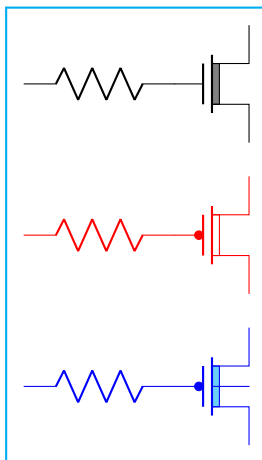
4.15.5.15 Simplified symbols for depletion-mode MOSFETs . The `nmosd`, `pmosd` (simplified) symbols for depletion-mode MOSFET (introduced in 1.2.4) behave exactly like the normal (without the final `d`) ones.

By default, the thick bar (indicating the pre-formed channel) is filled with the same color as the drawing:



```
\begin{circuitikz}[ ]
  \draw (0,2) to[R] ++(2,0) node[nmosd, anchor=G]{};
  \draw[color=red] (0,0) to[R] ++(2,0) node[pmosd, anchor=G]{};
\end{circuitikz}
```

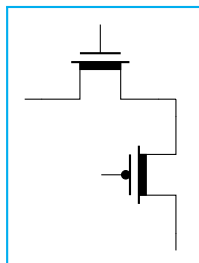
You can change this behavior by setting the key `tripoles/nmosd/depletion color` (default value `default`, which means “use the draw color”) to the color you want; using `none` will lead to an unfilled channel (note that in this case the color does not change automatically with the path!):



```
\begin{circuitikz}[ ]
  \ctikzset{tripoles/nmosd/depletion color=gray}
  \draw (0,2) to[R] ++(2,0) node[nmosd, anchor=G]{};
  \ctikzset{tripoles/pmosd/depletion color=none}
  \draw[color=red] (0,0) to[R] ++(2,0)
    node[pmosd, anchor=G]{};
  \ctikzset{tripoles/pmosd/depletion color=
    {cyan!50!white}}
  \draw[color=blue] (0,-2) to[R] ++(2,0)
    node[pmosd, anchor=G, bulk]{};
\end{circuitikz}
```

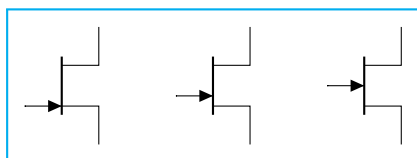
⁹⁷Since v1.6.3, suggested by [user Hartomes on TeX StackExchange](#); previously, the default scale was 1.0, which created a clash with body diodes.

Obviously you have the equivalent `tripoles/pmosd/depletion` color for type-P transistors. They also have path-style syntax, as the other transistors.



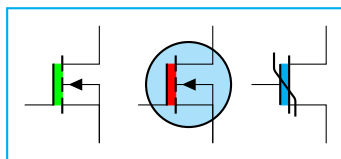
```
\begin{circuitikz}[ ]
  \draw (0,0) to[Tnmosd] ++(2,0)
    to[Tpmosd, invert] ++(0,-2)
  ;
\end{circuitikz}
```

4.15.5.16 JFET gate position. Since version 1.8.6⁹⁸, you can change the vertical position of the gate terminal in the junction transistor `njfet` and `pjfet` using the key `transistors/jfet gate height` (default 1.0, which means that the gate arrow is at the same height as the source terminal). In the following example you can see its use and effect (notice that when it is set to zero, there is complete symmetry between drain and source terminal; this is physically right for ideal/simplified devices, but not always for real ones).



```
\begin{tikzpicture}
  \ctikzset{transistors/arrow pos=end}
  \tikzset{jfet pos/.style={
    circuitikz/transistors/jfet gate height=#1}}
  \draw (0,0) node[njfet]{}
    ++(2,0) node[njfet, jfet pos=0.5]{}
    ++(2,0) node[njfet, jfet pos=0]{};
\end{tikzpicture}
```

4.15.5.17 Gate/Base gap coloring. You can color the space representing the gate capacitor or the insulated base by using the key `tr gap fill` (default `none`, which means nothing is drawn there). This fill is done *after* any circle fill but before any additional modifier (see the example below). You can use it locally or set it globally (normal scoping works, as ever).

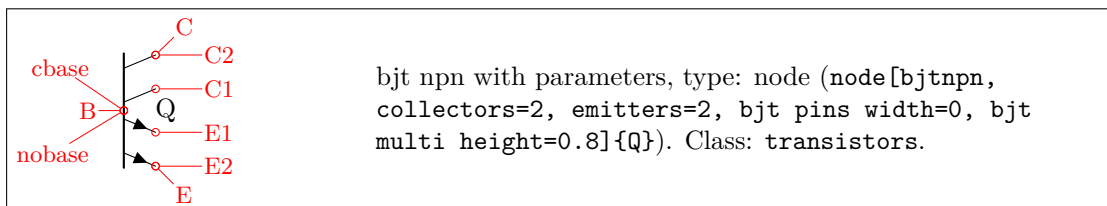


```
\begin{tikzpicture}
  \node[nigfete, tr gap fill=green] at(0,0){};
  \node[nigfete, tr gap fill=red, tr circle,
    fill=cyan!30] at(1.5,0){};
  \node[nmos, tr gap fill=cyan, ferroel gate](A)
    at(3,0){};
\end{tikzpicture}
```

4.15.6 Multiple terminal transistors customization

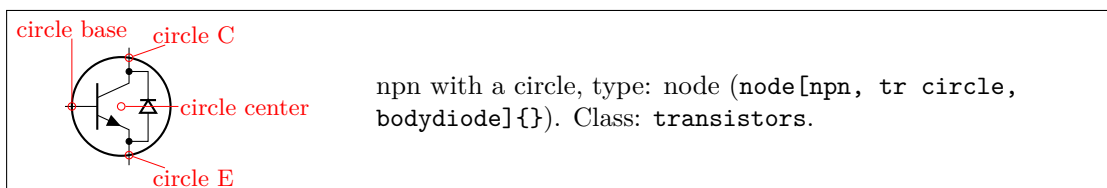
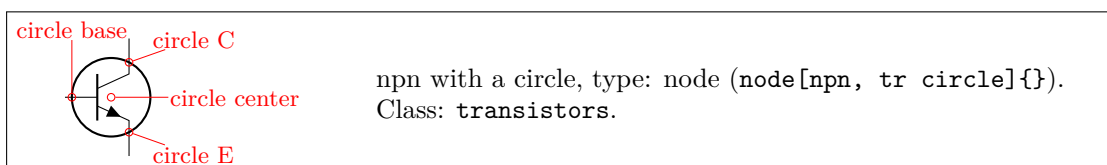
You can create completely “bare” transistors (without the connection leads to the B, C y E terminals), by changing the parameter `tripoles/bjt/pins width` (default 0.3; it is expressed as a fraction of the basic (scaled) length) or using the style `bjt pins width`; and you can change the distance between multiple collectors/emitters setting with `\ctikzset{}` the parameter `tripoles/bjt/multi height` (default 0.5) or the style `bjt multi height`.

⁹⁸the change was triggered by the question by user Vector on [TeX StackExchange](#).

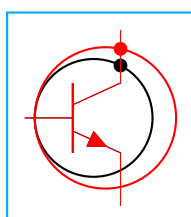


4.15.7 Transistor circle customization

4.15.7.1 Position and size. You can see in the following diagram where the circle is positioned — when there is no bodydiode, it will pass through the anchors for the body diode and near the base connection. The dimension of the circle is bigger when the bodydiode is in, to encompass it. The anchors are present even if there is no circle, so you can use them to draw different kinds of circles (say, encompassing two transistors) in a coherent way.



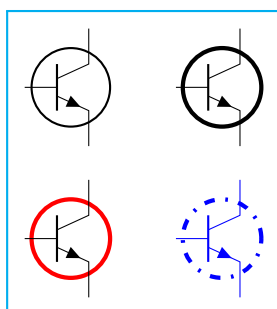
The position of the circle on collector and emitter by default is the one shown above; the position along the base can be adjusted in most transistors using the `\ctikzset` parameter `transistor circle/default base in` (by default 0.9); `njfet` and `pjfet` use `transistor circle/njfet base in` (default 1.05; the same for `pjfet`) and, finally, `isfet` uses `transistor circle/isfet base in` (default 0.65). You can change the resulting size of the circle by setting to something different to 1.0 the parameter `transistor circle/scale circle radius` — that will move the anchors too; for example:



```
\begin{circuitikz}[scale=1.5, transform shape]
  \draw (0,0) node[npn, tr circle](Q1){};
  \node [circ] at (Q1.circle C){};
  \ctikzset{transistor circle/scale circle radius=1.2}
  \draw[color=red] (0,0) node[npn, tr circle](Q2){};
  \node [circ, color=red] at (Q2.circle C){};
\end{circuitikz}
```

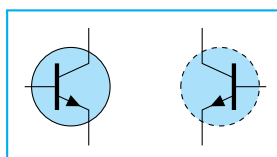
4.15.7.2 Line and color. Normally the circle follows the style of the component — the line thickness is fixed by the class element `transistors/thickness` and the color is the same as the component color. You can change, if you need, all of these things using the parameters of the following table (the parameters are under the `\ctikzset` category root `transistor circle/`).

parameter	default	description
relative thickness	1.0	multiply the class thickness
color	default	stroke color: default is the same as the component
dash	default	dash pattern: none means solid line, default means keep the global pattern ⁹⁹
partial borders	none	draw only part of the circle border: none means draw all
partial border dash	{{2pt}}{2pt}}	dash pattern used in partial borders



```
\begin{circuitikz}[]
  \draw (0,2) node[npn, tr circle](Q1){};
  \ctikzset{transistor circle/relative thickness=2}
  \draw (2,2) node[npn, tr circle](Q1){};
  \ctikzset{transistor circle/color=red}
  \draw (0,0) node[npn, tr circle](Q1){};
  \ctikzset{transistor circle/color=default}
  \ctikzset{transistor circle/dash={{4pt}}{4pt}}{1pt}{4pt}}
  \draw[color=blue] (2,0) node[npn, tr circle](Q1){};
\end{circuitikz}
```

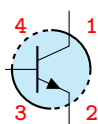
Finally, using the class style you can do quite interesting things.



```
\begin{circuitikz}[]
  \ctikzset{transistors/thickness=4, transistors/fill=cyan!30,
    transistor circle/relative thickness=0.25,}
  \draw (0,0) node[npn, tr circle](Q1){};
  \ctikzset{transistor circle/dash={{2pt}}{2pt}}
  \draw (1.5,0) node[npn, tr circle, xscale=-1](Q2){};
\end{circuitikz}
```

4.15.7.3 Partially drawn circle borders In some circuits, transistors are drawn with partial or dashed border (to convey the meaning of several active components encased in the same physical package, or to signify thermal contact). To achieve this effect, you can use the **transistor circle/partial border**¹⁰⁰ key (default **none**). This key can be set to **none**, or must be a sequence of **exactly** 4 numbers, that can have value 0, 1, or 2. Each number defines the style of a part of the border to be not drawn, solid or dashed respectively.

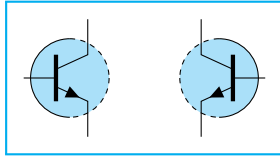
The part of the border are numbered from 1 to 4 as shown below:



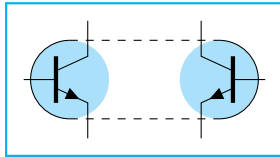
The dashed line pattern can be changed by setting the key **transistor circle/partial border dash** (default {{2pt}}{2pt}). Be careful with the extra set of braces here.

⁹⁹Follows the syntax of the pattern sequence `\pgfsetdash` — see TikZ manual [for details](#); phase is always zero. Basically you pass pairs of dash-length – blank-length dimensions, see the examples.

¹⁰⁰Suggested by [Jether Fernandes Reis](#) for tubes, implemented by Romano in v1.5.2.



```
\begin{circuitikz}[]
  \ctikzset{transistors/thickness=4, transistors/fill=cyan!30,
    transistor circle/relative thickness=0.25,
    transistor circle/partial borders=2211}
  \draw (0,0) node[npn, tr circle](Q1){};
  \ctikzset{transistor circle/dash={{2pt}{2pt}}}%
  \draw (1.5,0) node[npn, tr circle, xscale=-1](Q2){};
\end{circuitikz}
```



```
\begin{circuitikz}[]
  \ctikzset{transistors/thickness=4, transistors/fill=cyan!30,
    transistor circle/relative thickness=0.25,
    transistor circle/partial borders=0011}
  \draw (0,0) node[npn, tr circle](Q1){};
  \ctikzset{transistor circle/dash={{2pt}{2pt}}}%
  \draw (1.5,0) node[npn, tr circle, xscale=-1](Q2){};
  \draw[dashed] (Q1.circle top) -- (Q2.circle top);
  \draw[dashed] (Q1.circle bottom) -- (Q2.circle bottom);
\end{circuitikz}
```

4.15.8 Transistor bodydiode customization

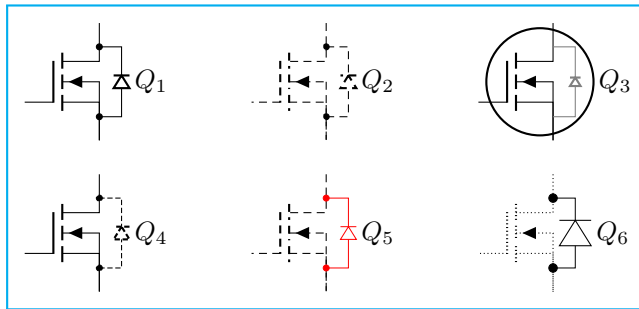
You can change the style of the bodydiode¹⁰¹ in a similar way to the one used for circles (albeit with less options), by setting keys with the `\ctikzset` command (or, like in the following example, directly in the node instantiation) under the `transistor bodydiode/` hierarchy. The available keys are:

parameter	default	description
<code>relative thickness</code>	1.0	multiply the class thickness
<code>color</code>	default	stroke color: default is the same as the component
<code>dash</code>	default	dash pattern: default means not to change the setting for the component; none means unbroken line; every other input is a dash pattern ¹⁰²
<code>scale</code>	0.3	scale of the diode, with respect to the basic (not scaled) diode dimension.
<code>dot scale</code>	0.7	scale of bodydiode connections dots, with respect to the <code>circ</code> pole. Use zero to remove them (useful if you have circles around the transistor).

The following is a quite extensive example. Obviously, a good strategy in this case is to define styles for the options, or, better, set the option in your style file or in the preamble (for coherency).

¹⁰¹Suggested by [user Alex Ghilas on TeX.SX](#), implemented in v1.5.4; `scale` suggested by [user @sputeanus on GitHub](#) and added in version 1.6.2.

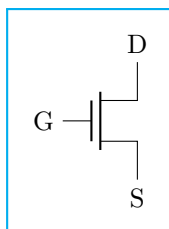
¹⁰²Follows the syntax of the pattern sequence `\pgfsetdash` — see TikZ manual [for details](#); phase is always zero. Basically you pass pairs of dash-length – blank-length dimensions, see the examples.



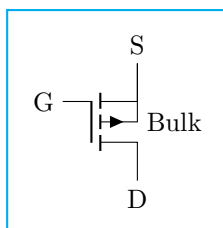
```
\begin{tikzpicture}[red solid thin bodydiode/.style={bodydiode,
    circuitikz/transistor bodydiode/dash=none,
    circuitikz/transistor bodydiode/color=red,
    circuitikz/transistor bodydiode/relative thickness=0.3}]
\draw (0,0) node (mosfet1) [nigfete,anchor=D,bodydiode] {$Q_1$};
\draw[densely dashed] (3,0) node (mosfet1) [nigfete,anchor=D,bodydiode] {$Q_2$};
\draw (6,0) node (mosfet1) [nigfete,anchor=D,bodydiode, tr circle,
    circuitikz/transistor bodydiode/color=gray,
    circuitikz/transistor bodydiode/scale=0.2,
    circuitikz/transistor bodydiode/dot scale=0] {$Q_3$};
\draw (0,-2) node (mosfet1) [nigfete,anchor=D,bodydiode,
    circuitikz/transistor bodydiode/dash={2pt}{1pt}] {$Q_4$};
\draw[densely dashed] (3,-2) node (mosfet1) [nigfete,anchor=D,
    red solid thin bodydiode] {$Q_5$};
\ctikzset{transistor bodydiode/relative thickness=.5,
    transistor bodydiode/scale=0.6}% from now on, in scope
\draw[densely dotted] (6,-2) node (mosfet1) [nigfete,anchor=D,bodydiode,
    circuitikz/transistor bodydiode/dash=none,
    circuitikz/transistor bodydiode/dot scale=1] {$Q_6$};
\path (7,0); %% adjust bounding box (node text is outside it!)
\end{tikzpicture}
```

4.15.9 Transistors anchors

For NMOS, PMOS, NFET, NIGFETE, NIGFETD, PFET, PIGFETE, and PIGFETD transistors one has base, gate, source and drain anchors (which can be abbreviated with B, G, S and D):

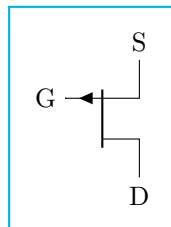


```
\begin{circuitikz} \draw
(0,0) node[nmos] (mos) {}
(mos.gate) node[anchor=east] {G}
(mos.drain) node[anchor=south] {D}
(mos.source) node[anchor=north] {S}
;\end{circuitikz}
```



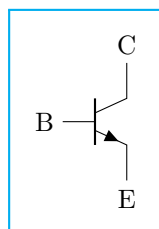
```
\begin{circuitikz} \draw
(0,0) node[pigfete] (pigfete) {}
(pigfete.G) node[anchor=east] {G}
(pigfete.D) node[anchor=north] {D}
(pigfete.S) node[anchor=south] {S}
(pigfete.bulk) node[anchor=west] {Bulk}
;\end{circuitikz}
```

Similarly NJFET and PJFET have **gate**, **source** and **drain** anchors (which can be abbreviated with G, S and D):

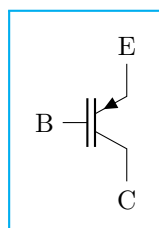


```
\begin{circuitikz} \draw
(0,0) node[pjfet] (pjfet) {}
(pjfet.G) node[anchor=east] {G}
(pjfet.D) node[anchor=north] {D}
(pjfet.S) node[anchor=south] {S}
;\end{circuitikz}
```

For NPN, PNP, NIGBT and PIGBT transistors, the anchors are **base**, **emitter** and **collector** anchors (which can be abbreviated with B, E and C):



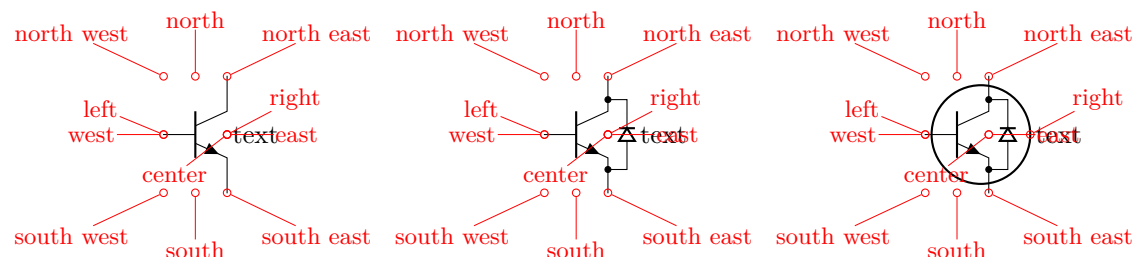
```
\begin{circuitikz} \draw
(0,0) node[npn] (npn) {}
(npn.base) node[anchor=east] {B}
(npn.collector) node[anchor=south] {C}
(npn.emitter) node[anchor=north] {E}
;\end{circuitikz}
```



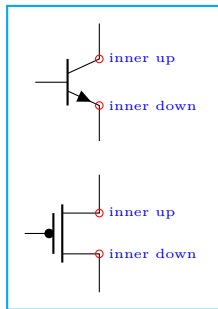
```
\begin{circuitikz} \draw
(0,0) node[pigbt] (pigbt) {}
(pigbt.B) node[anchor=east] {B}
(pigbt.C) node[anchor=north] {C}
(pigbt.E) node[anchor=south] {E}
;\end{circuitikz}
```

Notice that the geographical anchors of transistors are *not* affected by either the bodydiode and the circle options; the label text is also outside of them. This is to permit aligning the components independently from those features. On the other hand, this can sometimes create problems because that element is outside the bounding box automatically calculated by TikZ.

The exception is the **right** anchor which, when a circle is present, indicates the edge of the circle itself (since v1.3.2)

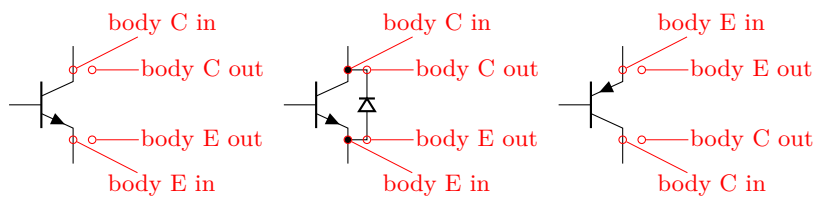


All transistors, except the multi-terminal **bjtnpn** and **bjtpnp**, (since 0.9.6) have internal nodes on the terminal corners, called **inner up** and **inner down**; you do not normally need them, but they are here for special applications:

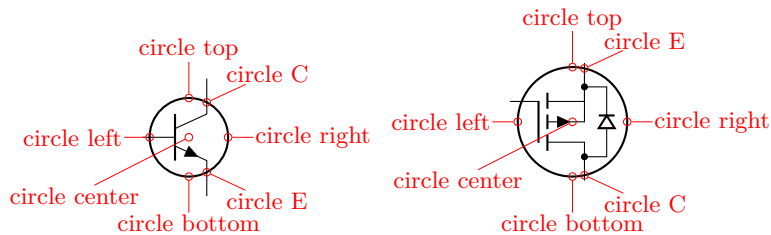


```
\begin{circuitikz}
\node [npn] (A) at(0,2) {};
\node [pmos] (B) at(0,0) {};
\foreach \e in {A, B}
  \foreach \a in {inner up, inner down} {
    \node[red, circle, inner sep=1pt, draw]
      at (\e.\a) {};
    \node [right, font=\tiny, blue]
      at (\e.\a) {\a};
  }
\end{circuitikz}
```

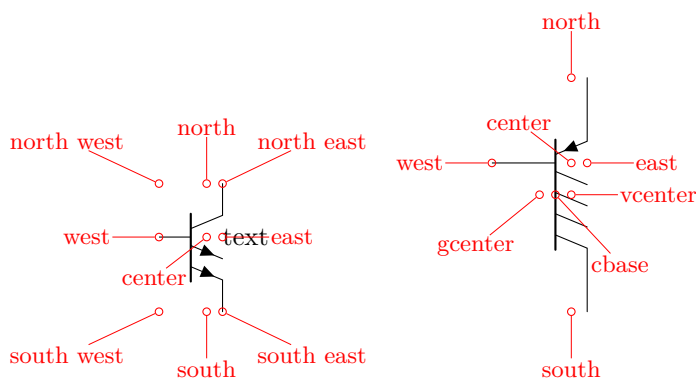
Additionally, you can access the position for the flyback diodes and possibly snubbers as shown in 4.15.5.6.



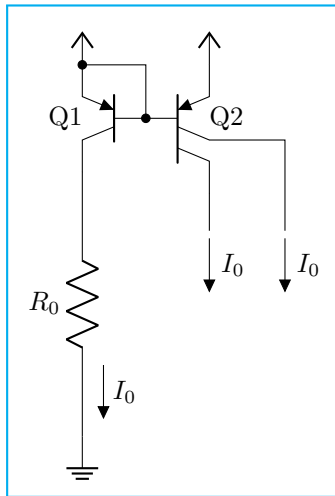
Transistor circles also have several anchors on them:



The multi-terminal transistors have all the geographical anchors; note though that the **center** anchor is not the geometrical center of the component, but the logical one (at the same height as the base). The additional anchors **vcenter** (vertical geometric center of the collector-emitter zone) and **gcenter** (graphical center) are provided, as shown in the following picture. They have no bodydiode anchors nor **inner up/down** ones.

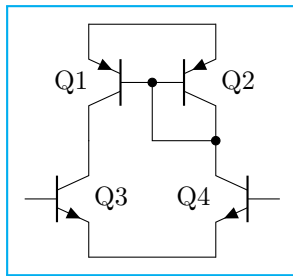


A complete example of multiple terminal transistor application is the following PNP double current mirror circuit.



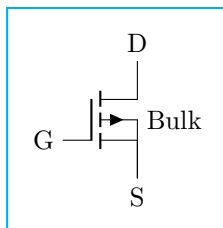
```
\begin{circuitikz}
\ctikzset{transistors/arrow pos=end}
\draw (0,0) node[bjtnpn, xscale=-1] (Q1){%
\scalebox{-1}[1]{Q1}};
\draw (Q1.B) node[bjtnpn, anchor=B, collectors=2]
(Q2){Q2} (Q1.B) node[circ]{};
\draw (Q1.E) node[circ]{} node[vcc]{} (Q2.E)
node[vcc]{} (Q1.E) -| (Q1.B);
\draw (Q1.C) to[R, l_=$R_0$, f=$I_0$] ++(0,-3.5)
node[ground] (GND){};
\draw (Q2.C) -- ++(0,-0.5) coordinate(a);
\draw (Q2.C1) -- ++(1,0) coordinate(b) -- (b|-a);
\draw (a) ++(0,-0.1) node[flowarrow, rotate=-90,
anchor=west]{\rotatebox{90}{$I_0$}};
\draw (b|-a) ++(0,-0.1) node[flowarrow, rotate=-90,
anchor=west]{\rotatebox{90}{$I_0$}};
\path (b) ++(0.5,0); % bounding box adjust
\end{circuitikz}
```

Here is one composite example (please notice that the `xscale=-1` style would also reflect the label of the transistors, so here a new node is added and its text is used, instead of that of `pnp1`):



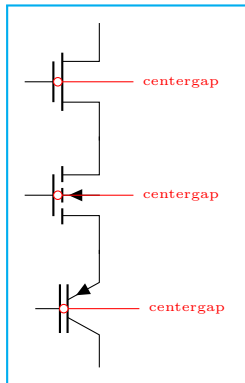
```
\begin{circuitikz} []\draw
(0,0) node[pnp] (pnp2) {Q2}
(pnp2.B) node[pnp, xscale=-1, anchor=B] (pnp1) {}
(pnp1) node[left, inner sep=0pt] {Q1}
(pnp1.C) node[npn, anchor=C] (npn1) {Q3}
(pnp2.C) node[npn, xscale=-1, anchor=C] (npn2)
{\scalebox{-1}[1]{Q4}}
(pnp1.E) -- (pnp2.E) (npn1.E) -- (npn2.E)
(pnp1.B) node[circ] {} |- (pnp2.C) node[circ] {}
;\end{circuitikz}
```

Notice that the text labels of transistors are outside the bounding box of the component (that is, the set of geographical anchors). If it is a problem, use a separate text node to set the transistor's label. Of course, transistors like other components can be reflected vertically:



```
\begin{circuitikz} \draw
(0,0) node[pigfete, yscale=-1] (pigfete) {}
(pigfete.bulk) node[anchor=west] {Bulk}
(pigfete.G) node[anchor=east] {G}
(pigfete.D) node[anchor=south] {D}
(pigfete.S) node[anchor=north] {S}
;\end{circuitikz}
```

Finally, double-gated components (MOSes, FETs, IGBTs) have an extra anchor `centergap` positioned in the middle of the “gate capacitor” or base.

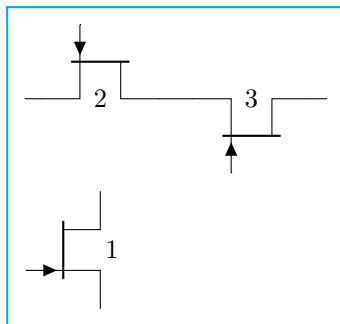


```
\begin{circuitikz}
  \node [nmos](A) at (0,3) {};
  \node [nfet](B) at (0,1.5) {};
  \node [pigbt](C) at (0,0) {};
  \foreach \myn in {A, B, C}
    \draw[color=red] (\myn.centergap)
      node[ocirc]{} -- ++(1,0)
      node[right, font=\tiny]{centergap};
\end{circuitikz}
```

For UJT transistors anchors, see section [4.15.5.11](#).

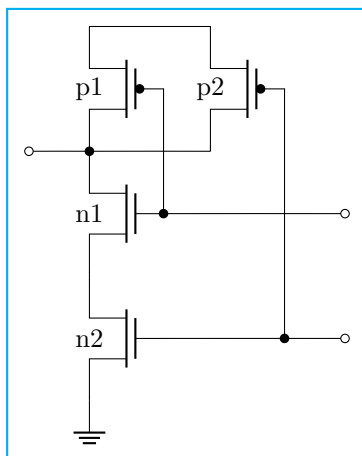
4.15.10 Transistor paths

For syntactical convenience standard transistors (not multi-terminal ones) can be placed using the normal path notation used for bipoles. The transistor type can be specified by simply adding a “T” (for transistor) in front of the node name of the transistor. It will be placed with the base/gate orthogonal to the direction of the path:



```
\begin{circuitikz} \draw
(0,0) node[njfet] {1}
(-1,2) to[Tnjfet=2] (1,2)
to[Tnjfet=3, mirror] (3,2)
;\end{circuitikz}
```

Access to the gate and/or base nodes can be gained by naming the transistors with the **n** or **name** path style:



```
\begin{circuitikz} \draw[yscale=1.1, xscale=.8]
(2,4.5) -- (0,4.5) to[Tpmos=p1, n=p1] (0,3)
to[Tnmos=n1, n=n1] (0,1.5)
to[Tnmos=n2, n=n2] (0,0) node[ground] {}
(2,4.5) to[Tpmos=p2, n=p2] (2,3) to[short, -*] (0,3)
(p1.G) -- (n1.G) to[short, *-o] ($(n1.G)+(3,0)$)
(n2.G) ++(2,0) node[circ] {} -| (p2.G)
(n2.G) to[short, -o] ($(n2.G)+(3,0)$)
(0,3) to[short, -o] (-1,3)
;\end{circuitikz}
```

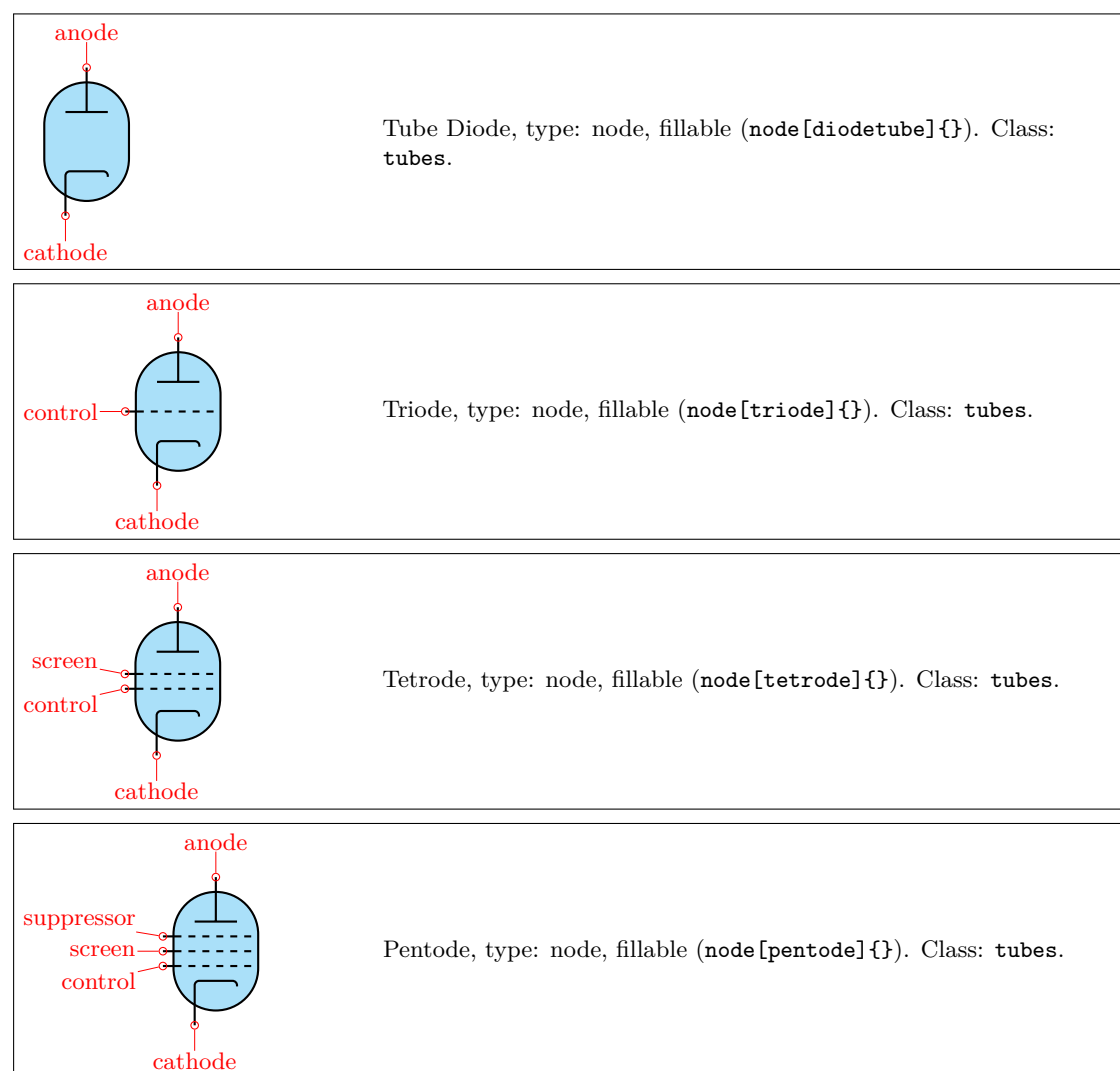
Transistors used in path are fully path-style components, so to flip and rotate them you should use **mirror** and **invert** as shown in section [3.1.4.2](#).

Transistor paths have the possibility to use the poles syntax (see section 6.1) but they have **no** voltage, current, flow, annotation options. Also, the positioning of the labels is very simple and is not foolproof for all rotations; if you need to control them more please name the node and position them by hand, or use the more natural node style for transistors.

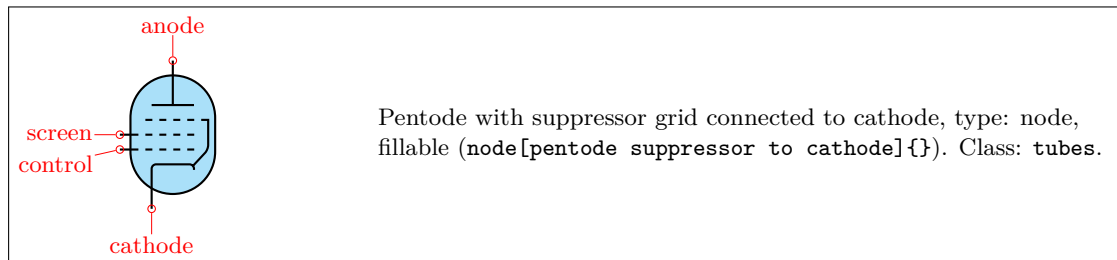
The **name** property is available also for bipoles; this is useful mostly for triac, potentiometer and thyristor (see 4.4.1).

4.16 Electronic Tubes

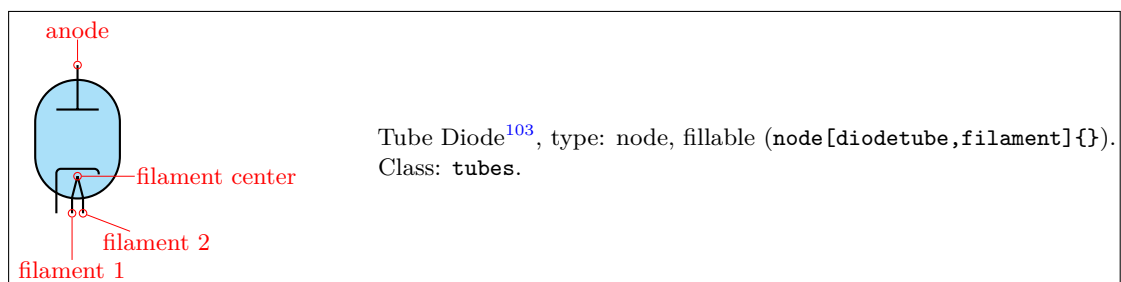
Electronic tubes, also known as vacuum tubes, control current flow between electrodes. They come in many different flavours. Contributed by J. op den Brouw (J.E.J.opdenBrouw@hhs.nl).



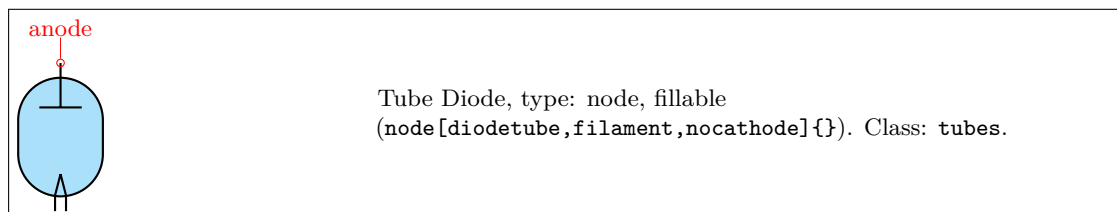
Some pentodes have the suppressor grid internally connected to the control grid, which saves a pin on the tube's housing.



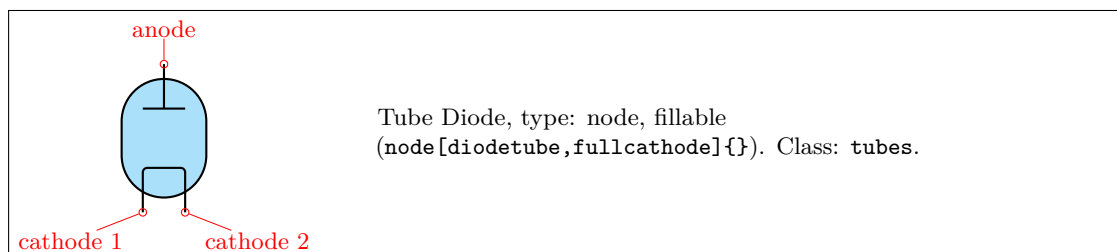
Note that the **diode****tube** is used as component name to avoid clashes with the semiconductor diode. Normally, the filament is not drawn. If you want a filament, put the **filament** option in the node description:



Sometimes, you don't want the cathode to be drawn (but you do want the filament). Use the **nocathode** option in the node description:



If you want a full cathode to be drawn, use the **fullcathode** option in the node description. You can then use the anchors **cathode 1** and **cathode 2**.



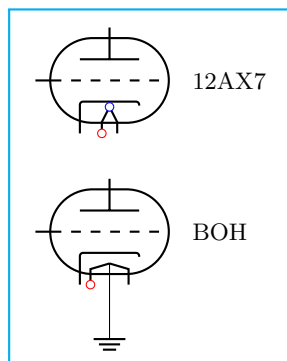
¹⁰³The **filament center** anchor has been added in v1.6.9 thanks to a suggestion by [user bogger33 on GitHub](#).

4.16.1 Tubes customization

The tubes can be scaled using the key `tubes/scale`, default 1.0. In addition, they are fully configurable, and the attributes are described below:

Key	Default value	Description
<code>tubes/scale</code>	1	scale factor
<code>tubes/width</code>	1	relative width
<code>tubes/height</code>	1.4	relative height
<code>tubes/tube radius</code>	0.40	radius of tube circle
<code>tubes/anode distance</code>	0.40	distance from center
<code>tubes/anode width</code>	0.40	width of an anode/plate
<code>tubes/grid protrusion</code>	0.25	distance from center
<code>tubes/grid dashes</code>	5	number of grid dashes
<code>tubes/grid separation</code>	0.2	separation between grids
<code>tubes/grid shift</code>	0.0	y shift of grids from center
<code>tubes/cathode distance</code>	0.40	distance from grid
<code>tubes/cathode width</code>	0.40	width of a cathode
<code>tubes/cathode corners</code>	0.06	corners of the cathode wire
<code>tubes/cathode right extend</code>	0.075	extension at the right side
<code>tubes/filament distance</code>	0.1	distance from cathode
<code>tubes/filament angle</code>	15	angle from the centerpoint

Conventionally, the model of the tube is indicated at the `east` anchor, and you can access filament anchors if you need them:



```
\ctikzset{tubes/width=1.4, tubes/height=1}
\begin{circuitikz}
  \draw (0,2) node[triode, filament] (Tri) {};
  \draw (Tri.east) node[right] {12AX7};
  \ctikzset{tubes/filament angle=40,
    tubes/filament distance=0.2}
  \draw (0,0) node[triode, filament] (Pent) {};
  \draw (Pent.east) node[right] {BOH};
  \path (Tri.filament 1) node[red,ocirc]{};
  \path (Pent.filament 1) node[red,ocirc]{};
  \path (Tri.filament center) node[blue,ocirc]{};
  \draw (Pent.filament center) -- ++(0,-1) node[tlground]{};
\end{circuitikz}
```

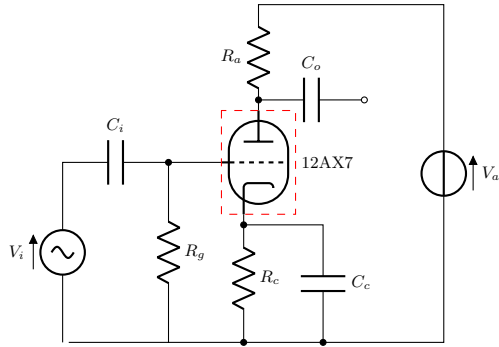
Example triode amplifier:

```
\begin{circuitikz}
\draw (0,0) node (start) {}
      to[sV=$V_i$] ++(0,2+\ctikzvalof{tubes/height})
      to[C=$C_i$] ++(2,0) coordinate(Rg)
      to[R=$R_g$] (Rg |- start)
(Rg)    to[short,*-] ++(1,0)
      node[triode,anchor=control] (Tri) {} ++(2,0)
(Tri.cathode) to[R=$R_c$,-*] (Tri.cathode |- start)
(Tri.anode)  to [R=$R_a$] ++(0,2)
              to [short] ++(3.5,0) node(Vatop) {}
              to [V<=$V_a$] (Vatop |- start)
              to [short] (start)
(Tri.anode)  ++(0,0.2) to[C=$C_o$,*-o] ++(2,0)
(Tri.cathode) ++(0,-0.2) to[short,*-] ++(1.5,0) node(Cctop) {}
```

```

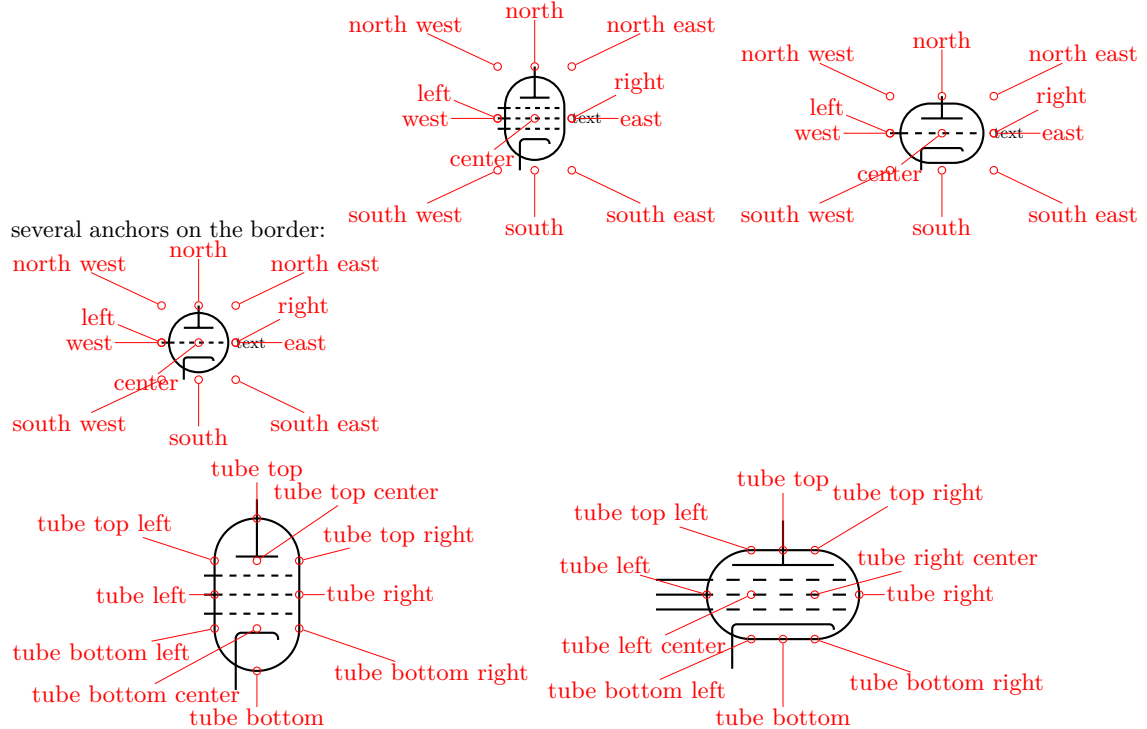
to[C=$C_c$,--] (start -| Cctop)
;
\draw[red,thin,dashed] (Tri.north west) rectangle (Tri.south east);
\draw (Tri.east) node[right] {12AX7};
\end{circuitikz}

```



4.16.2 Tubes anchors

Apart from the geographic anchors, which take into account the leads of the components, you have



4.16.3 Partially drawn tube borders

In some circuits, tubes are drawn with partial or dashed border (to convey the meaning of several active components encased in the same physical tube). To achieve this effect, you can use the **tubes/partial border**¹⁰⁴ key (default **none**). This key can be set to **none**, or must be a sequence of **exactly** 6 numbers, which can have value 0, 1, or 2. Each number defines the style of a part of the border to be not drawn, solid or dashed respectively.

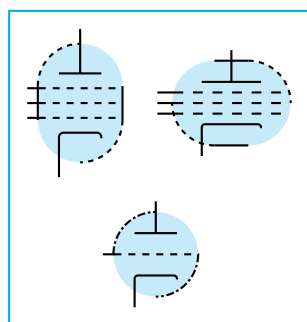
¹⁰⁴Suggested by [Jether Fernandes Reis](#), implemented by Romano in v1.5.2.

The part of the border are numbered from 1 to 6 as shown below:



(notice that the straight parts, if they exist, are numbered 2 and 5 in both tubes, vertical or horizontal).

The dashed line pattern can be changed by setting the key `tubes/partial border dash` (with default value `{{2pt}{2pt}}`).¹⁰⁵ Be careful with the extra set of braces here.

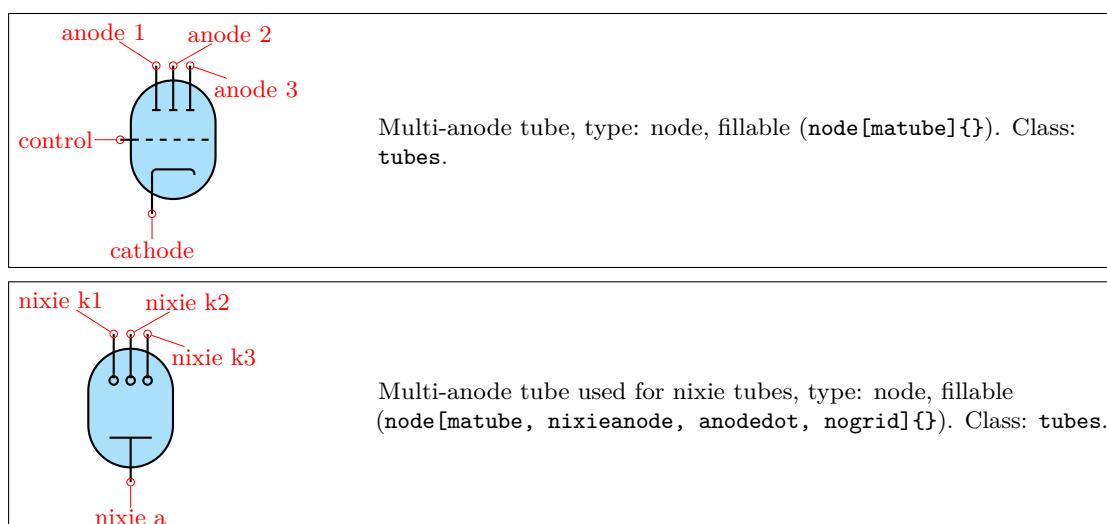


```
\begin{circuitikz}[circuitikz/tubes/fill=cyan!20,
  circuitikz/tubes/partial borders=012012]
\draw (0,0) node[pentode]{};
\draw (2,0) node[pentode,
  circuitikz/tubes/width=1.4,
  circuitikz/tubes/height=1]{};
\draw (1,-2) node[triode,
  circuitikz/tubes/height=1,
  circuitikz/tubes/partial border dash=
    {{3pt}{1pt}}]{};
\end{circuitikz}
```

4.16.4 Multi-anode tube

The multi-anode tube (`matube`) is a component thought to be tailored for several different usages,¹⁰⁶ as shown in the examples below.

The anchors for the multiple anodes have also alias names to ease the use in the case of implementing nixie tubes, because in that case the anode/cathode roles are swapped.



¹⁰⁵Follows the syntax of the pattern sequence `\pgfsetdash` — see TikZ manual [for details](#); phase is always zero. Basically you pass pairs of dash-length – blank-length dimensions, see the examples.

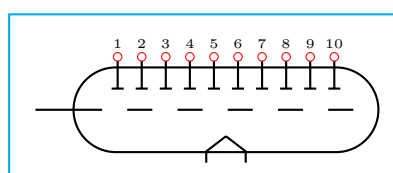
¹⁰⁶It was added in v1.6.8 after suggestions by user [bogger33](#) on GitHub [here](#) and [here](#).

Basically all the parameters available for triodes are available. The main difference is that the **anode width** parameter define the length of *all* the anodes; those tubes are normally used with a **width** parameter bigger than **height**, to have an elongated device.

The additional parameters/flags available only for **matubes** are the following.

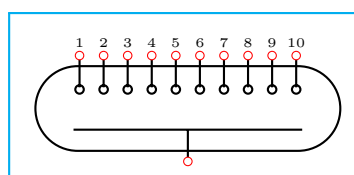
Key/Flag	Default value	Description
anodes	3	number of anodes
anodedot	false	substitute anodes bar for dots
nixieanode	false	substitute the cathode for the nixie-style anode
nogrid	false	suppresses the drawing of the grid

In the next example, we define a 10-anodes VFD tube:



```
\tikzset{vfd 10/.style={matube, filament, nocathode,
circuitikz/tubes/.cd,
width=3.6, height=1, anodes=10, anode width=0.6,
cathode width=0.1,
}}
\begin{circuitikz}[european]
\draw (0,0) node[vfd 10](A){};
\foreach \i in {1,...,10} \path (A.anode \i)
node[red, ocirc]{} node[above]{\tiny \i};
\end{circuitikz}
```

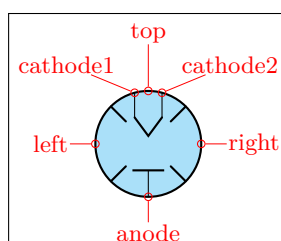
And a 10-cathodes nixie tube:



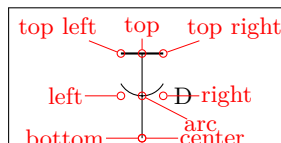
```
\tikzset{nixie/.style={matube, nogrid, nixieanode,
anodedot, circuitikz/tubes/.cd, cathode width=0.6,
width=3.6, height=1, anodes=10, anode width=0.6,
}}
\begin{circuitikz}[european]
\draw (0,0) node[nixie](A){};
\foreach \i in {1,...,10} \path (A.nixie k\i)
node[red, ocirc]{} node[above]{\tiny \i};
\path (A.nixie a) node[red, ocirc]{};
\end{circuitikz}
```

4.16.5 Other tubes-like components

The **magnetron** and **dynode** shapes will also scale with **tubes/scale**.

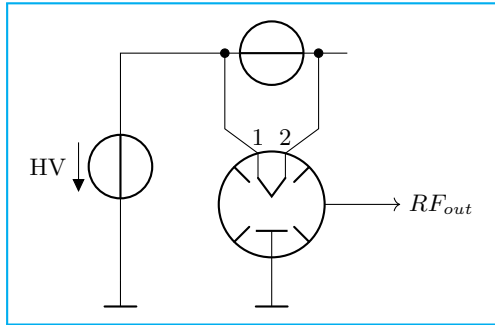


Magnetron, type: node, fillable (`node[magnetron]{}).` Class: **tubes**.



Dynode¹⁰⁷, type: node (`node[dynode]{D}.` Class: **tubes**.

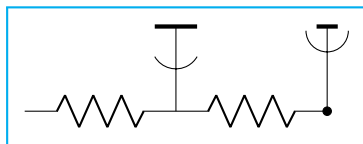
¹⁰⁷Suggested by the user [ferdymercury](#) on [GitHub](#).



```
\begin{circuitikz}
\draw (0,-2)node[rground](gnd){} to[voltage
source,v<={HV}]++(0,3)--++(1,0)to[V,n=DC
]++(2,0);
\draw (2,-1) node[magnetron,scale=1](magn){};
\draw (DC.left)++(-0.2,0)to [short,*-] ++(0,-1)
to [short] (magn.cathode1);
\draw (DC.right)++(0.2,0)to [short,*-] ++(0,-1)
to [short] (magn.cathode2);
\draw (magn.anode) to [short] (magn.anode|gnd)
node[rground]{};
\draw (magn.cathode1)node[above]{$1$};
\draw (magn.cathode2)node[above]{$2$};
\draw[->](magn.east) --++(1,0)node[right]{$RF_{out}$};
\end{circuitikz}
```

4.16.5.1 Dynode customization. The dynode element can be heavily customized. The parameters are the following (all of them under the `\ctikzset` family `monopoles/dynode`):

parameter	default	description
<code>width</code>	0.4	Total width (relative to the base length) measured at the arc width.
<code>height</code>	0.8	Total height (same units as width).
<code>arc angle</code>	30	Angle (from the horizontal, going down) where the arc starts. A value of 90 don't plot any arc, 0 plots a semicircle. To avoid artifacts, use a value between -60 and 90; the arc horizontal size is always equal to the <code>width</code> .
<code>arc pos</code>	0.5	Vertical position (relative to the height) of the arc center.
<code>top width</code>	1.0	Relative width of the top bar; a value of 1 means full width, 0 means no bar.



```
\begin{circuitikz}[american] \ctikzset{tubes/thickness=4}
\draw (0,0) to[R] (2,0) node[dynode]{} to[R,*] (4,0);
\ctikzset{monopoles/dynode/.cd,
arc angle=0, arc pos=0.7, top width=0.5}
\draw (4,0) node[dynode]{};
\end{circuitikz}
```

You can use styles and the parameters to create different types of electrodes:



```
\begin{circuitikz}[american] \ctikzset{tubes/thickness=4}
\tikzset{anode/.style={dynode,
circuitikz/monopoles/dynode/arc angle=90},
photocatode/.style={dynode,
circuitikz/monopoles/dynode/arc pos=1,
circuitikz/monopoles/dynode/top width=0},
}
\draw (0,0) node[dynode]{} (1,0) node[anode]{}
(2,0) node[photocatode]{};
\end{circuitikz}
```

4.17 RF components

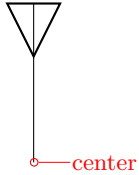
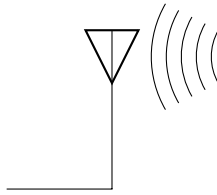
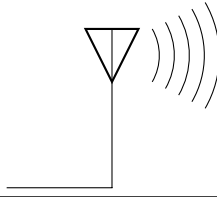
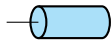
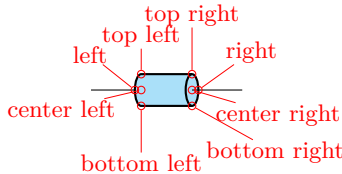
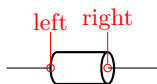
For the RF components, similarly to the grounds and supply rails, the **center** anchor is put on the connecting point of the symbol, so that you can use them directly in a **path** specification.

Notes that in the transmission and receiving antennas, the “waves” are outside the geographical anchors.

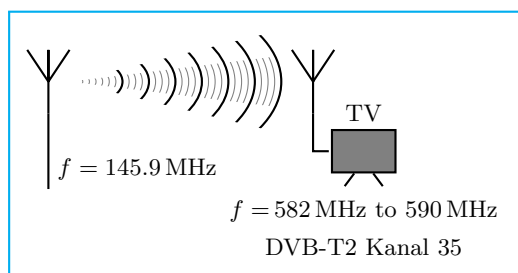
	Bare Antenna, type: node, fillable (<code>node[bareantenna]{A}</code>). Class: RF.
	DIN antenna ¹⁰⁸ , type: node, fillable (<code>node[dinantenna]{A}</code>). Class: RF.
	Bare TX Antenna, type: node, fillable (<code>node[bareTXantenna]{Tx}</code>). Class: RF.
	Bare RX Antenna, type: node, fillable (<code>node[bareRXantenna]{Rx}</code>). Class: RF.
	Waves, type: node (<code>node[waves]{}</code>). Class: RF.
	Harmonics which could be combined with waves, type: node (<code>node[harmonics]{}</code>). Class: RF.
	mstline : Microstrip transmission line ¹⁰⁹ , type: path-style, fillable, nodename: <code>mstlineshape</code> . Class: RF.
	Microstrip linear stub, type: node, fillable (<code>node[mslstub]{text}</code>). Class: RF.
	Microstrip port, type: node, fillable (<code>node[msport]{T}</code>). Class: RF.
	Microstrip radial stub, type: node, fillable (<code>node[msrstub]{}</code>). Class: RF.

¹⁰⁸Since 1.5.0, suggested by [GitHub user myzinsky](#)

¹⁰⁹These four components were suggested by [@tcpluss](#) on GitHub

	Legacy antenna (with tails), type: node (<code>node[antenna]{}</code>). Class: RF.
	Legacy receiving antenna (with tails), type: node (<code>node[rxantenna]{}</code>). Class: RF.
	Legacy transmitting antenna (with tails), type: node (<code>node[txantenna]{}</code>). Class: RF.
	Transmission line stub, type: node, fillable (<code>node[tlinestub]{}</code>). Class: RF.
	TL: Transmission line, type: path-style, fillable, nodename: <code>tlineshape</code> . Aliases: <code>transmission line</code> , <code>tline</code> . Class: RF.
	TL, <code>bipoles/tline/bare=true</code> : Transmission line without wires (notice that if you fill it, the fill will overwrite the exiting wire), type: path-style, nodename: <code>tlineshape</code> . Aliases: <code>transmission line</code> , <code>tline</code> . Class: RF.
	match, type: node (<code>node[match]{}</code>). Class: RF.

The **waves** and **harmonics** can help with RF block diagrams



```
% Author: Prof. Dr. Matthias Jung, DL9MJ % Year: 2023
\begin{circuitikz}[european]
  \draw(0,0) node [dinantenna](ant1){};
  \draw[thick](ant1.south) -- ++(0,-1)
    node[anchor=south west]({$f=\qty{145,9}{\mega\hertz}$});
  \draw(3.5,0) node [dinantenna](ant2){};
  \draw(3.75,-0.5) node[tvsetshape, anchor=west, fill=gray](tv){};
```



```

\draw[thick] (ant2) |- (tv);
\draw(tv.north) node[above](){TV};
%
\draw[gray](ant1.right) node[harmonics, xscale=2, anchor=left]({});
\draw[thick](ant1.right) node[waves, xscale=2, anchor=left]({});
%
\draw(tv.south) ++(0,-0.25)
    node[below](foo){$f=\$, \qtyrange{582}{590}{\mega\hertz}};
\draw(foo.south) node[below](){DVB-T2 Kanal 35};
\end{circuitikz}

```

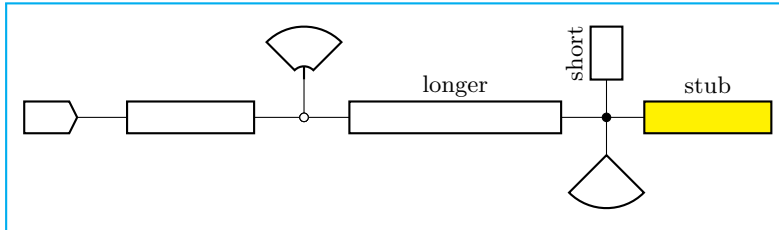
4.17.1 RF elements customization

The RF elements can be scaled using the key `RF/scale`, default 1.0.

4.17.2 Microstrip customization

The microstrip linear components' (`mstline`, `mslstub`, `msport`) heights can be changed by setting the parameter `bipoles/mstline/height` (for the three of them, default 0.3). The widths are specified in `bipoles/mstline/width` for the first two and by `monopoles/msport/width` for the port (defaults: 1.2, 0.5). Finally, you can change the size of the smaller circle (the connecting part) of `msrstub` with the parameter `monopoles/msrstub/radius` (default 0.25; using 0 will suppress the smaller circle¹¹⁰).

For the length parameter of the transmission line there is a shortcut in the form of the direct parameter `mstlinelen`.



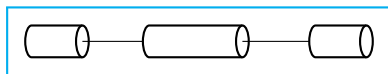
```

\begin{circuitikz}
\draw (0,0) node[msport, right, xscale=-1]{}
to[mstline, -o] ++(3,0) coordinate(there)
to[mstline, mstlinelen=2, l=longer, o-*] ++(4,0)
coordinate(here) -- ++(0.5,0) node[mslstub, fill=yellow]{stub}
(here) -- ++(0,0.5) node[mslstub, rotate=90, mstlinelen=0.5]{short};
\draw (there) to[short, o-] ++(0, 0.5) node[msrstub]{};
\draw (here) -- ++(0, -0.5) node[msrstub, yscale=-1,
circuitikz/monopoles/msrstub/radius=0]{};
\end{circuitikz}

```

The legacy `tline` can be used as in the following example. You can change the length with the key `bipoles/tline/width` (default 0.6). The “bare” version, which differs only for the small line on the visible ellipse, and activated with the boolean key `.../bare`, is useful as a substitute for `tlinestub` (with more flexibility).

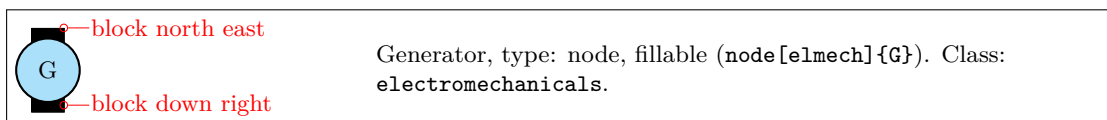
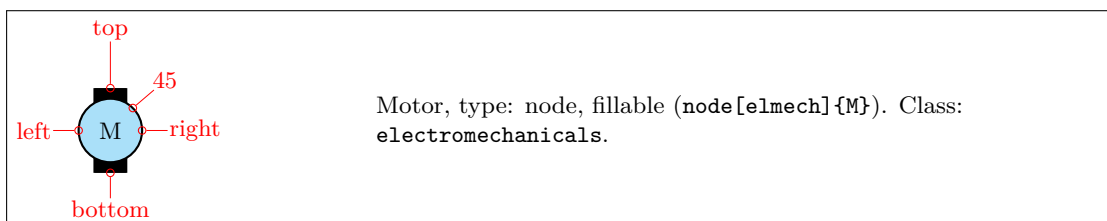
¹¹⁰suggested by [user cis on TeX.SX](#)



```
\begin{tikzpicture}[]
  \tikzset{bare t1/.style={tlineshape,
    circuitikz/bipoles/tline/bare=true}}
  \draw (0,0) node[bare t1](A){} (A.right)
    to[TL, bipoles/tline/width=1] ++(3,0)
    node[bare t1, anchor=left]{};
\end{tikzpicture}
```

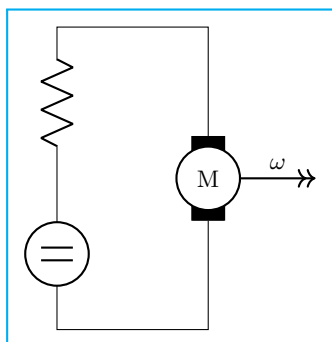
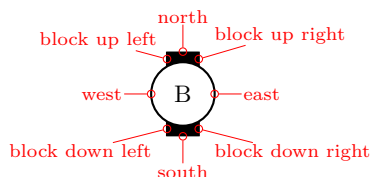
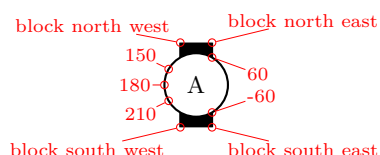
4.18 Electro-Mechanical Devices

The internal part of the motor and generator are, by default, filled with white (to avoid compatibility problems with older versions of the package).

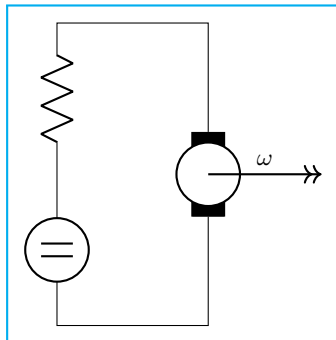


4.18.1 Electro-Mechanical Devices anchors

Apart from the standard geographical anchors, `elmech` has the border anchors (situated on the inner circle) and the following anchors on the “block”:

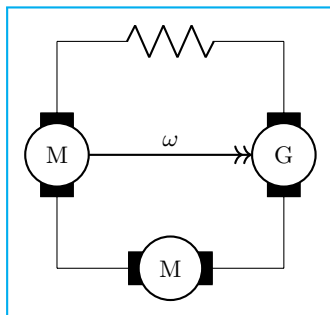


```
\begin{circuitikz}
\draw (2,0) node[elmech](motor){M};
\draw (motor.north) |- (0,2) to [R] ++(0,-2) to [dcvsource]
  ++(0,-2) -| (motor.bottom);
\draw[thick,->>] (motor.right) -- ++(1,0) node[midway,above]{$\omega$};
\end{circuitikz}
```



```
\begin{circuitikz}
\draw (2,0) node[elmech](motor){};
\draw (motor.north) |- (0,2) to [R] ++(0,-2) to [dcvsource]
++(0,-2) -| (motor.bottom);
\draw[thick,->](motor.center)--++(1.5,0)node[midway,above]
{\omega};
\end{circuitikz}
```

The symbols can also be used along a path, using the transistor-path-syntax (T in front of the shape name, see section 4.15.10). Don't forget to use parameter n to name the node and get access to the anchors:



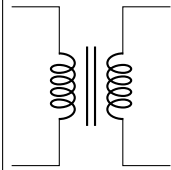
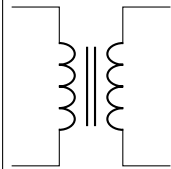
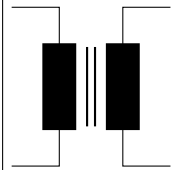
```
\begin{circuitikz}
\draw (0,0) to [Telmech=M,n=motor] ++(0,-3) to [Telmech=M]
++(3,0) to [Telmech=G,n=generator] ++(0,3) to [R] (0,0);
\draw[thick,->](motor.left)--(generator.left)node[midway,above]
{\omega};
\end{circuitikz}
```

4.19 Double bipoles (transformers)

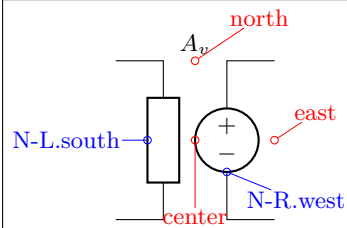
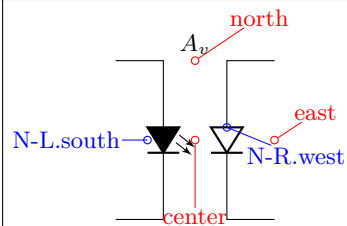
Transformers automatically use the inductor shape currently selected. These are the three possibilities:

	Transformer (cute inductor), type: node (node[transformer]{T_1}). Class: inductors.
	Transformer (american inductor), type: node (node[transformer]{}). Class: inductors.
	Transformer (european inductor), type: node (node[transformer]{}). Class: inductors.
	Gyrator, type: node, fillable (node[gyrator]{}). Class: inductors.

Transformers with core are also available:

	Transformer core (cute inductor), type: node (node[transformer core]{}). Class: inductors.
	Transformer core (american inductor), type: node (node[transformer core]{}). Class: inductors.
	Transformer core (european inductor), type: node (node[transformer core]{}). Class: inductors.

You can also build generic double bipoles¹¹¹ (although it's often better to use subcircuits in this case; see section 3.4).

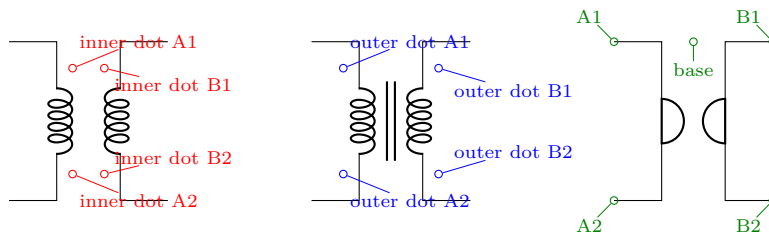
	Generic double bipole (configurable components), type: node (node[double bipole] (N){\$A_v\$}). Class: misc.
	Generic double bipole (this specific configuration is shown in section 4.19.4), type: node (node[double bipole] (N){\$A_v\$}). Class: misc.

4.19.1 Transformer anchors

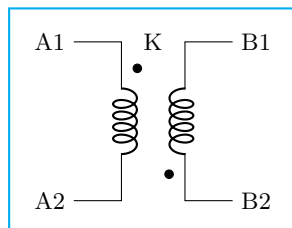
All the double bipoles/quadrupoles have the four anchors, two for each port. The first port, to the left, is port **A**, having the anchors **A1** (up) and **A2** (down); same for port **B**.

They also expose the **base** anchor, for labeling, and anchors for setting dots or signs to specify polarity. The set of anchors, to which you should add the standard “geographical” **north**, **north east**, etc. is here:

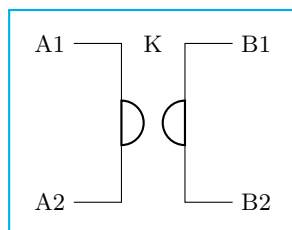
¹¹¹The idea of generic double bipoles was originated by user [erwindenboer on GitHub](#).



Also, the standard “geographical” **north**, **north east**, etc. are defined. A couple of examples follow:

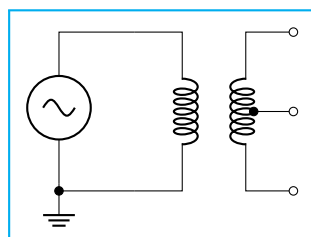
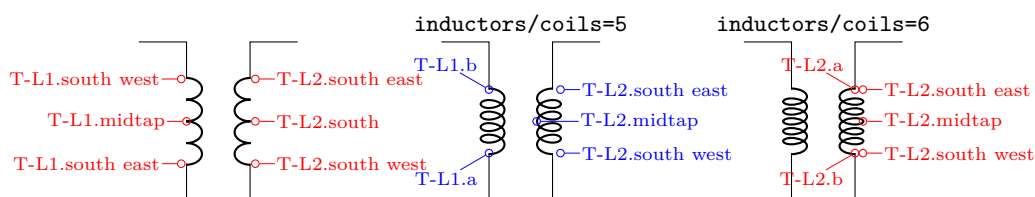


```
\begin{circuitikz} \draw
(0,0) node[transformer] (T) {}
(T.A1) node[anchor=east] {A1}
(T.A2) node[anchor=east] {A2}
(T.B1) node[anchor=west] {B1}
(T.B2) node[anchor=west] {B2}
(T.base) node{K}
(T.inner dot A1) node[circ]{}
(T.inner dot B2) node[circ]{}
;\end{circuitikz}
```



```
\begin{circuitikz} \draw
(0,0) node[gyrator] (G) {}
(G.A1) node[anchor=east] {A1}
(G.A2) node[anchor=east] {A2}
(G.B1) node[anchor=west] {B1}
(G.B2) node[anchor=west] {B2}
(G.base) node{K}
;\end{circuitikz}
```

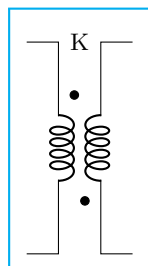
Moreover, you can access the two internal coils (inductances); if your transformer node is called **T**, they are named **T-L1** and **T-L2**. Notice that the two inductors are rotated (by -90 degrees the first, +90 degrees the second) so you have to be careful with the anchors. Also, the **midtap** anchor of the inductors can be on the external or internal side depending on the numbers of coils. Finally, the anchors **L1.a** and **L1.b** are marking the start and end of the coils.



```
\begin{circuitikz}
\draw (0,0) node[ground] (GND){} to [sV] ++(0,2) -- ++(1,0)
node[transformer, circuitikz/inductors/coils=6,
anchor=A1] (T){};
\draw (T.A2) to[short, -*] (T.A2-|GND);
\draw (T.L2.midtap) to[short, *-o] (T.B1 |- T.L2.midtap);
\node [ocirc] at (T.B1){}; \node [ocirc] at (T.B2){};
\end{circuitikz}
```

4.19.2 Transformers customization

Transformers are in the `inductors` class (also the gyrator...), so they scale with the key `inductors/scale`. You can change the aspect of a quadpole using the corresponding parameters `quadpoles/*/width` and `quadpoles/*/height` (substitute the star for `transformer`, `transformer core` or `gyrator`; default value is 1.5 for all). You have to be careful to not choose value that overlaps the components!



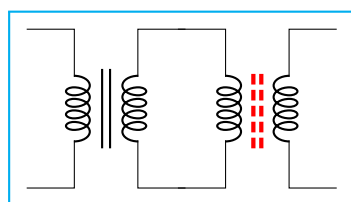
```
\begin{circuitikz}
\ctikzset{quadpoles/transformer/width=1,
quadpoles/transformer/height=2}
\draw (0,0) node[transformer] (T) {}
(T.base) node{K}
(T.inner dot A1) node[circ]{}
(T.inner dot B2) node[circ]{};
\end{circuitikz}
```

Transformers also inherit the `inductors/scale` (see 4.3.5) and similar parameters. It's your responsibility to set the aforementioned parameters if you change the scale or width of inductors.

Transformers core line distance is specified by the parameter `quadpoles/transformer core/core width` (default 0.05) and the thickness of the lines follows the choke one; in other words, you can set it changing `bipoles/cutechoke/cthick`.

You can change the style of the core lines¹¹² in a similar way to the one used for transistor's bodydiodes, by setting keys with the `\ctikzset` command under the `transformer core` hierarchy. The available keys are:

parameter	default	description
<code>relative thickness</code>	1.0	multiply the default thickness (which is the same of the <code>choke</code> component).
<code>color</code>	<code>default</code>	stroke color: <code>default</code> is the same as the component.
<code>dash</code>	<code>default</code>	dash pattern: <code>default</code> means not to change the setting for the component; <code>none</code> means unbroken line; every other input is a dash pattern. ¹¹³

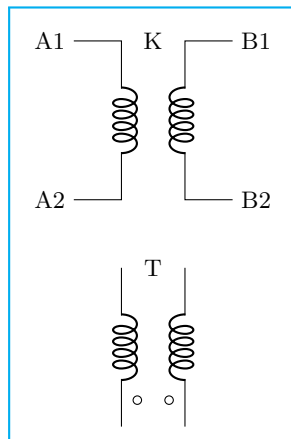


```
\begin{circuitikz}[]
\draw (0,0) node[transformer core](A){};
\ctikzset{transformer core/.cd, relative thickness=2,
color=red, dash={{4pt}{2pt}}}}
\draw (2,0) node[transformer core](B){};
\end{circuitikz}
```

Another very useful parameter is `quadpoles/*/inner` (default 0.4) that determine which part of the component is the “vertical” one. So, setting that parameter to 1 will eliminate the horizontal part of the component (obviously, to maintain the general aspect ratio you need to change the width also):

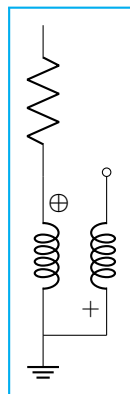
¹¹²Suggested by [user myzinsky on GitHub](#), implemented in v1.6.2.

¹¹³Follows the syntax of the pattern sequence `\pgfsetdash` — see TikZ manual [for details](#); phase is always zero. Basically you pass pairs of dash-length – blank-length dimensions, see the examples.



```
\begin{circuitikz}
\draw (0,0) node[transformer] (T) {}
(T.A1) node[anchor=east] {A1}
(T.A2) node[anchor=east] {A2}
(T.B1) node[anchor=west] {B1}
(T.B2) node[anchor=west] {B2}
(T.base) node{K} ;
\ctikzset{quadpoles/transformer/inner=1, quadpoles/transformer/
width=0.6}
\draw (0,-3) node[transformer] (P) {}
(P.base) node{T}
(P.inner dot A2) node[ocirc]{}
(P.inner dot B2) node[ocirc]{};
\end{circuitikz}
```

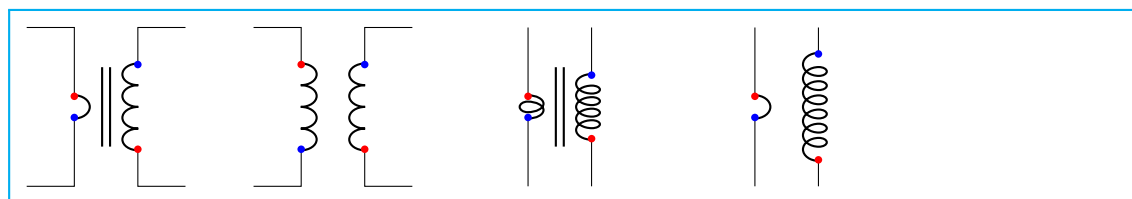
This can be useful if you want to put seamlessly something in series with either side of the component; for simplicity, you have a style setting `quadpoles` style to toggle between the standard shape of double bipoles (called `inward`, default) and the one without horizontal leads (called `inline`):



```
\begin{circuitikz}
\ctikzset{inductor=cute, quadpoles style=inline}
\draw
(0,0) to[R] ++(0,-2)
node[transformer, anchor=A1] (T){}
(T.A2) node[ground] (GND){}
(T.inner dot A1) node[font=\small\boldmath]{$\oplus$}
(T.inner dot B2) node[] {$+$}
(T.B1) node[above, ocirc]{}
(T.B2) -- (GND);
\end{circuitikz}
```

4.19.3 Styling transformer's coils independently

Since 0.9.6, you can tweak the style of each of the coils of the transformers by changing the value of the two styles `transformer L1` and `transformer L2`; the default for both are {}, that means inherit the inductors style in force.



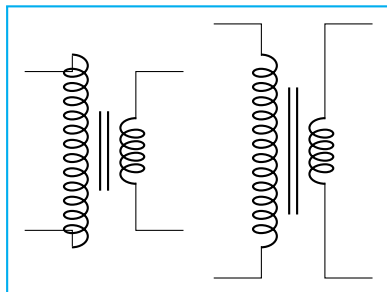
```
\begin{circuitikz}[american]
\begin{scope}
\ctikzset{transformer L1/.style={inductors/coils=1, inductors/width=0.2}}
\draw (0,0) node[transformer core] (T1){};
\end{scope}
\draw (3,0) node[transformer] (T2){};
\ctikzset{cute inductors, quadpoles style=inline}
\ctikzset{transformer L1/.style={inductors/coils=2, inductors/width=0.2}}
\draw (6,0) node[transformer core] (T3){};
```

```

\ctikzset{transformer L1/.style={american inductors, inductors/coils=1, inductors/
width=0.2}}
\ctikzset{transformer L2/.style={inductors/coils=7, inductors/width=1.0}}
\draw (9,0) node[transformer ](T4){};
\foreach \t in {T1, T2, T3, T4} {
  \foreach \l in {L1, L2} {
    \foreach \a/\c in {a/blue, b/red}
      \node [circle, fill=\c, inner sep=1pt] at (\t-\l.\a) {};
  }
}
\end{circuitikz}

```

Caveat: the size of the transformer is independent from the styles for L1 and L2, so they follow whatever the parameters for the inductances were before applying them. In other words, the size of the transformer could result too small if you are not careful.

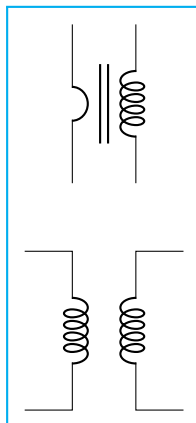


```

\begin{circuitikz}
  \ctikzset{transformer L1/.style={inductors/width
=1.8, inductors/coils=13}}
  % too small!
  \draw (0,0) node[transformer core](T1){};
  % adjust it
  \ctikzset{quadpoles/transformer core/height=2.4}
  \draw (2.5,0) node[transformer core](T1){};
\end{circuitikz}

```

You can obviously define a style for a “non-standard” transformer. For example, you can have a current transformer¹¹⁴ defined like this:



```

\begin{circuitikz}[
  TA core/.style={transformer core,
    % at tikz level, you have to use circuitikz/ explicitly
    circuitikz/quadpoles style=inline,
    circuitikz/transformer L1/.style={
      american inductors, inductors/coils=1,
      inductors/width=0.3},
    } ]
  \draw (0,0) node[TA core](T1){};
  % changes are local
  \draw (0,-3) node[transformer]{};
\end{circuitikz}

```

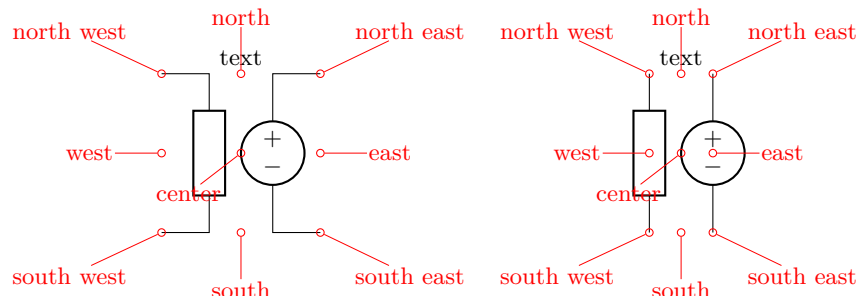
Remember that the default `pgfkeys` directory is `/tikz` for nodes and for the options of the environment, so you *have* to use the full path (with `circuitikz/`) there.

4.19.4 Generic double bipoles

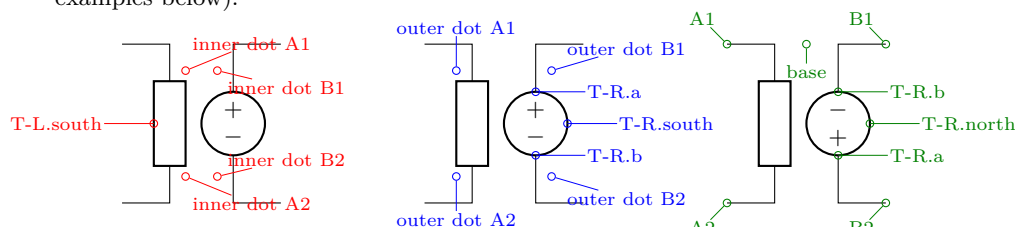
Generic double bipoles have more or less the same keys for size that the transformers (like `../width`, `../inner` etc.) using the component name `double bipole`. Also the anchors are similar, with the main difference that the “dot” anchors are fixed, so they do *not* adapt to the size of the component. Another important difference is that the class of the generic double bipole is `misc`, not `inductors` (which is reserved to transformers and, for an historical hiccup, to the gyrator).

¹¹⁴Suggested by Alex Pacini on [GitHub](#)

By default, the left component is a generic impedance, and the right one is an (American-style, it will not change automatically) voltage generator. You can use `quadpoles style=inner` as shown in the rightmost drawing below.



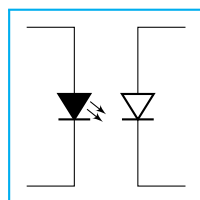
The other anchors behave similarly to the transistor's ones; you also have access to the internal components nodes by using `nodename-R` and `nodename-L` names for the right and left element, which is supposed to be T in the drawing below. Be wary that given that here you can (see later) reverse the direction of one or both of the elements, the rotation (and so the anchors) is not fixed (you can see that in the blue and green examples below).



Generic double bipoles are meant to be used through a style, choosing the left and right components. The keys that let you change the components are the following ones:

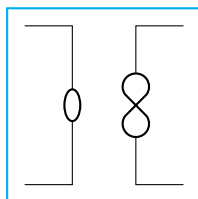
- `double bipole L`, `double bipole R`: the **nodename** of the component you want on the left and right side (default: `genericshape` and `vsourceshape`).
- `double bipole L invert`, `double bipole R invert`: controls the direction of the element inserted (default `false` for both; that means that the left bipole goes “down” and the right one “up”).
- `every double bipole L`, `every double bipole R`: a style that is enacted when drawing the component; by default it's void.

For example, the LED-diode double bipole at the start of the section could be obtained this way:



```
\begin{circuitikz}[
  led to D/.style={double bipole,
    % at tikz level, you have to use circuitikz/ explicitly
    circuitikz/double bipole L=fulllediodeshape,
    circuitikz/double bipole R=emptydiodeshape,
    circuitikz/every double bipole L/.style={diodes/scale=0.6},
    circuitikz/every double bipole R/.style={diodes/scale=0.6},
    circuitikz/double bipole R invert,
  },
]
\draw (0,0) node[led to D]{};
\end{circuitikz}
```

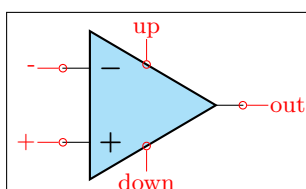
As a final example, and given that the addition of generic double bipole was stimulated by an issue opened by [user erwindenboer on GitHub](#) suggesting the addition of a nullor shape, the nullor can be obtained like this:



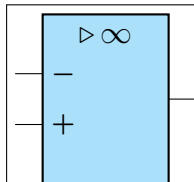
```
\begin{circuitikz}[
  nullor/.style={double bipole,
    % at tikz level, you have to use circuitikz/ explicitly
    circuitikz/double bipole L=nullatorshape,
    circuitikz/double bipole R=noratorshape,
    circuitikz/every double bipole L/.style={sources/scale=0.5},
  },
]
\draw (0,0) node[nullor](T1){};
\end{circuitikz}
```

although now adding currents and voltages is not as trivial as if the component is built with a subcircuit...

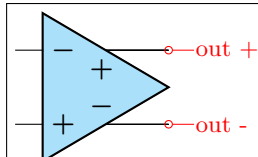
4.20 Amplifiers



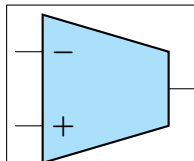
Operational amplifier, type: node, fillable (`node[op amp]{}`). Class: **amplifiers**.



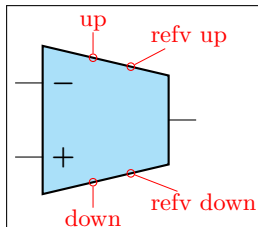
Operational amplifier compliant to DIN/EN 60617 standard, type: node, fillable (`node[en amp]{}`). Class: **amplifiers**.



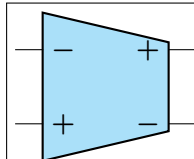
Fully differential operational amplifier¹¹⁵, type: node, fillable (`node[fd op amp]{}`). Class: **amplifiers**.



transconductance amplifier, type: node, fillable (`node[gm amp]{}`). Class: **amplifiers**.

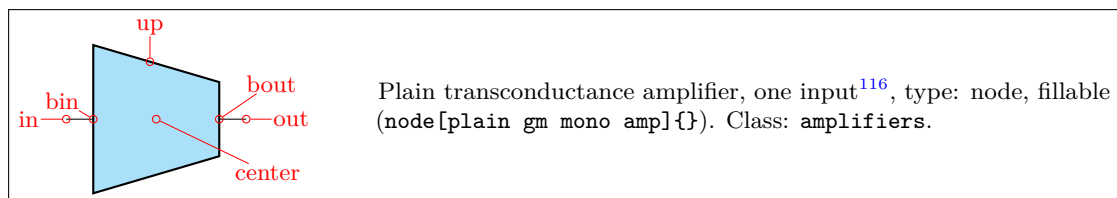
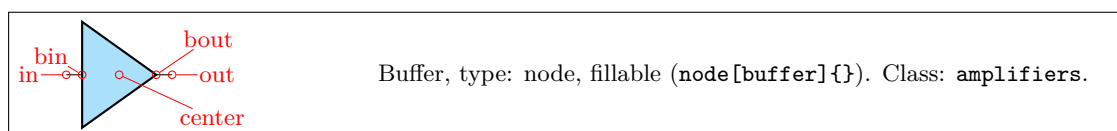
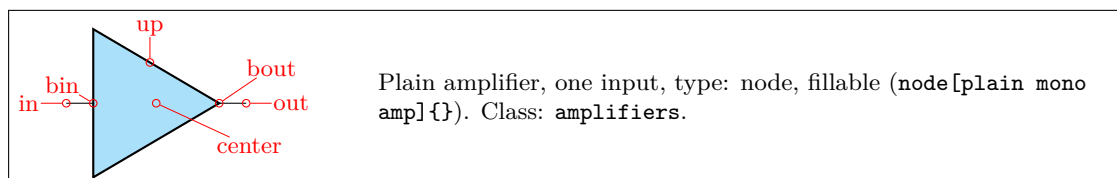
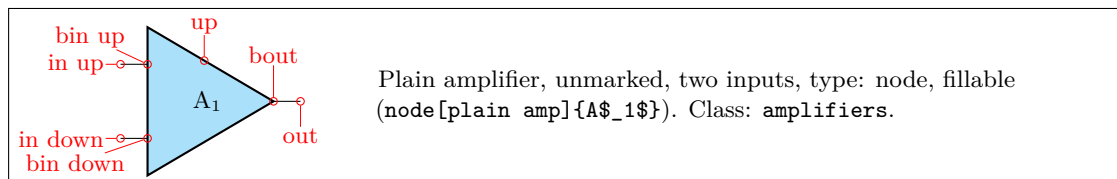
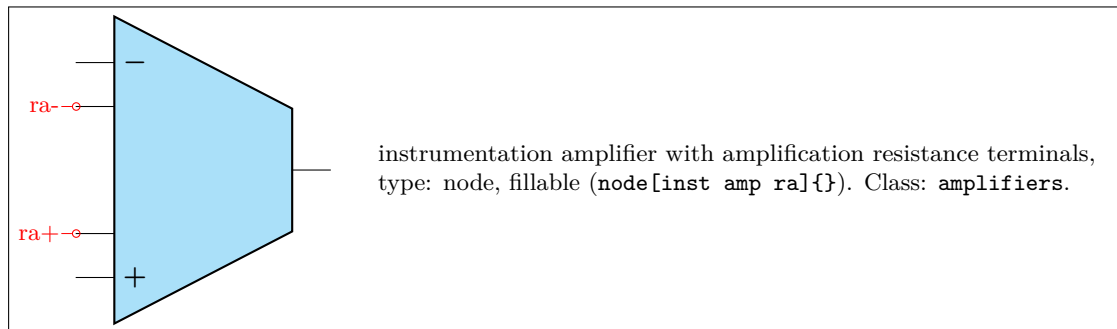


plain instrumentation amplifier, type: node, fillable (`node[inst amp]{}`). Class: **amplifiers**.



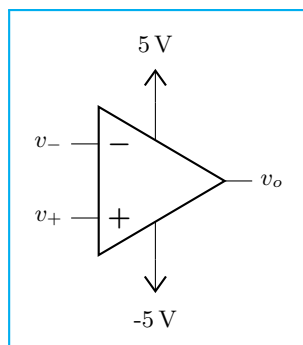
Fully differential instrumentation amplifier, type: node, fillable (`node[fd inst amp]{}`). Class: **amplifiers**.

¹¹⁵Contributed by Kristofer M. Monisit.



4.20.1 Amplifiers anchors

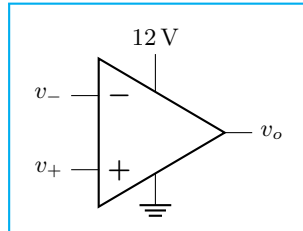
The op amp defines the inverting input (-), the non-inverting input (+) and the output (out) anchors:



```
\begin{circuitikz} \draw
(0,0) node[op amp] (opamp) {}
(opamp.+) node[left] {$v_+$}
(opamp.-) node[left] {$v_-$}
(opamp.out) node[right] {$v_o$}
(opamp.up) --++(0,0.5) node[vcc]{5\,\textnormal{V}}
(opamp.down) --++(0,-0.5) node[vee]{-5\,\textnormal{V}}
;\end{circuitikz}
```

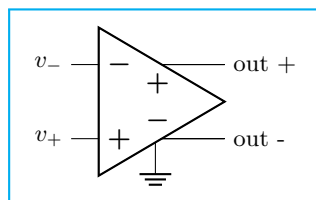
¹¹⁶Added by [Matthias Schweikardt](#).

There are also two more anchors defined, `up` and `down`, for the power supplies:



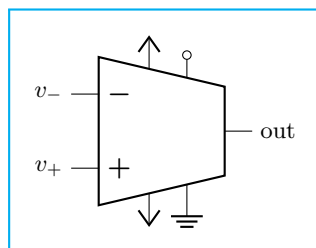
```
\begin{circuitikz} \draw
(0,0) node[op amp] (opamp) {}
(opamp.+) node[left] {$v_+$}
(opamp.-) node[left] {$v_-$}
(opamp.out) node[right] {$v_o$}
(opamp.down) node[ground] {}
(opamp.up) ++ (0,.5) node[above] {\SI{12}{\volt}}
-- (opamp.up)
;\end{circuitikz}
```

The fully differential op amp defines two outputs:



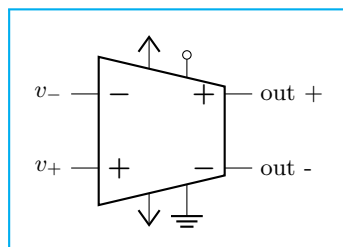
```
\begin{circuitikz} \draw
(0,0) node[fd op amp] (opamp) {}
(opamp.+) node[left] {$v_+$}
(opamp.-) node[left] {$v_-$}
(opamp.out +) node[right] {out +}
(opamp.out -) node[right] {out -}
(opamp.down) node[ground] {}
;\end{circuitikz}
```

The instrumentation amplifier `inst amp` defines also references (normally you use the `down`, unless you are flipping the component):



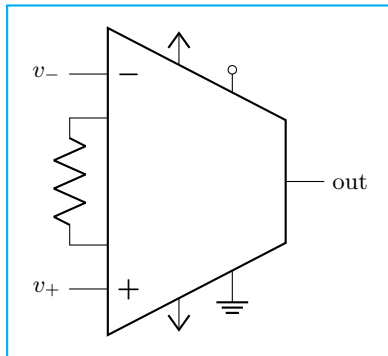
```
\begin{circuitikz} \draw
(0,0) node[inst amp] (opamp) {}
(opamp.+) node[left] {$v_+$}
(opamp.-) node[left] {$v_-$}
(opamp.out) node[right] {out}
(opamp.up) node[vcc] {}
(opamp.down) node[vee] {}
(opamp.refv down) node[ground] {}
(opamp.refv up) to[short, -o] ++(0,0.3)
;\end{circuitikz}
```

The fully differential instrumentation amplifier `inst amp` defines two outputs:



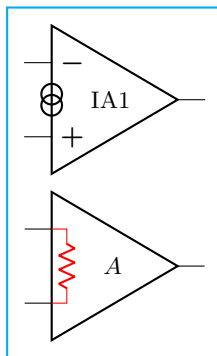
```
\begin{circuitikz} \draw
(0,0) node[fd inst amp] (opamp) {}
(opamp.+) node[left] {$v_+$}
(opamp.-) node[left] {$v_-$}
(opamp.out +) node[right] {out +}
(opamp.out -) node[right] {out -}
(opamp.up) node[vcc] {}
(opamp.down) node[vee] {}
(opamp.refv down) node[ground] {}
(opamp.refv up) to[short, -o] ++(0,0.3)
;\end{circuitikz}
```

The instrumentation amplifier with resistance terminals (`inst amp ra`) also defines terminals to add an amplification resistor:



```
\begin{circuitikz} \draw
(0,0) node[inst amp ra] (opamp) {}
(opamp.+) node[left] {$v_+$}
(opamp.-) node[left] {$v_-$}
(opamp.out) node[right] {out}
(opamp.up) node[vcc]{}
(opamp.down) node[vee] {}
(opamp.refv down) node[ground]{}
(opamp.refv up) to[short, -o] ++(0,0.3)
(opamp.ra-) to[R] (opamp.ra+)
;\end{circuitikz}
```

Amplifiers also have “border” anchors (just add **b**, without space, to the anchor, like **b+** or **bin up** and so on). These can be useful to add “internal components” or to modify the component. Also the **leftedge** anchor (on the border midway between input) is available.

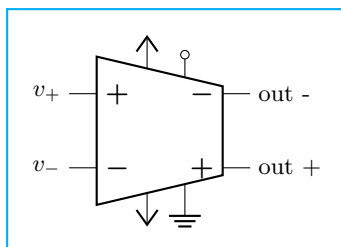


```
\begin{circuitikz}[]
\draw (0,2.2) node[op amp] (OA){IA1};
\node[oosourceshape, rotate=90, scale=0.5]
at (OA.leftedge) {};
\draw (0,0) node[plain amp] (A){$A$};
\draw [color=red] (A.bin up) -- ++(0.2,0)
coordinate (tmp)
to[R, resistors/scale=0.5]
(tmp|-A.bin down) -- (A.bin down);
\end{circuitikz}
```

4.20.2 Amplifiers customization

You can scale the amplifiers using the key **amplifiers/scale** and setting it to something different from 1.0. The font used for symbols will not scale, so it’s your responsibility to change it if the need arises.

4.20.2.1 Input polarity. All these amplifiers have the possibility to flip input and output (if needed) polarity. You can change polarity of the input with the **noinv input down** (default) or **noinv input up** key; and the output with **noinv output up** (default) or **noinv output down** key:



```
\begin{circuitikz} \draw
(0,0) node[fd inst amp,
noinv input up,
noinv output down] (opamp) {}
(opamp.+) node[left] {$v_+$}
(opamp.-) node[left] {$v_-$}
(opamp.out +) node[right] {out +}
(opamp.out -) node[right] {out -}
(opamp.up) node[vcc]{}
(opamp.down) node[vee] {}
(opamp.refv down) node[ground]{}
(opamp.refv up) to[short, -o] ++(0,0.3)
;\end{circuitikz}
```

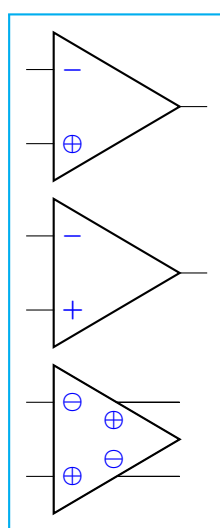
When you use the **noinv input/output ...** keys the anchors (+, -, out +, out -) will change with the effective position of the terminals. You also have the anchors **in up**, **in down**, **out up**, **out down** that will not change with the positive or negative sign.

4.20.2.2 Input and output pins symbols. You can change the symbols “+” or “−” appearing in the amplifiers if you want, both globally and on component-by-component basis. The plus and minus symbols can be changed with `\ctikzset` of the keys `amplifiers/plus` and `amplifiers/minus` (which defaults to the math mode plus or minus cited before), or using the styles `amp plus` and `amp minus`.

The font used is set in several keys, but you can change it globally with `\tikzset{amp symbol font}`, which has a default of 10-point (in L^AT_EX, and the corresponding one in ConT_EXt). You can change it for example with

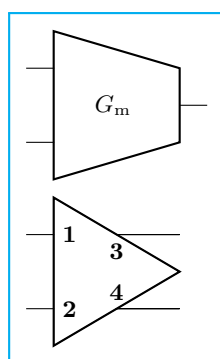
```
\tikzset{amp symbol font={%
\color{blue}\fontsize{12}{12}\selectfont\boldmath}}
```

to have plus and minus symbols that are bigger and blue.



```
\begin{circuitikz}[]
% change in this circuit only
\tikzset{amp symbol font={\color{blue}\small\boldmath}}
% local change
\draw (0,2.2) node[op amp, amp plus={\oplus}]{};
\draw (0,0) node[op amp]{};
% from now on...
\ctikzset{amplifiers/plus={\oplus}}
\ctikzset{amplifiers/minus={\ominus}}
\draw (0,-2.2) node[fd op amp]{};
\end{circuitikz}
```

You can remove the symbols by setting them to the null symbol `{}`. If you want different symbols for input and output you can use a null symbol and put them manually using the border anchors.



```
\begin{circuitikz}[]
% unset amplifier symbols
\ctikzset{amplifiers/plus={}}
\ctikzset{amplifiers/minus={}}

% place a transconductor without symbols
\draw (0,2.2) node[gm amp] (A) {\mathit{G}_m};

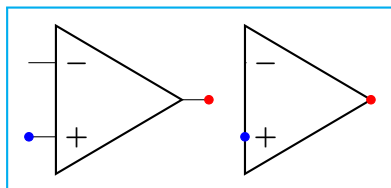
% fully differential op amp with 1, 2, 3 and 4 as symbols
\draw (0,0) node[fd op amp] (A) {};
\node [font=\small\bfseries, right] at(A.bin up) {1};
\node [font=\small\bfseries, right] at(A.bin down) {2};
\node [font=\small\bfseries, below] at(A.bout up) {3};
\node [font=\small\bfseries, above] at(A.bout down) {4};
\end{circuitikz}
```

4.20.2.3 Input and output pins length. The length of the wires that extends outside the main amplifier shape is not easily changed globally. You can use a trick¹¹⁷ though if you want to remove them

¹¹⁷See the discussion with [user @erwindenboer on GitHub](#); notice that this method is using internal keys and can stop working in the future.

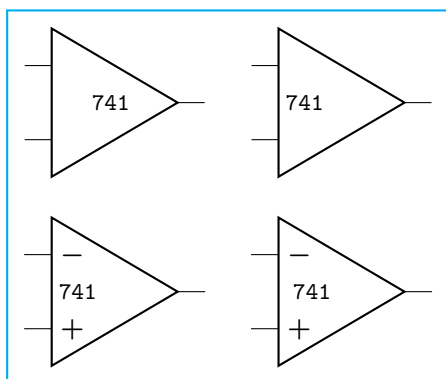
completely: the size of an amplifier (included the pins) is set by the `circuitikz` key `tripoles/amplifier style/width` and the size of the body of the amplifier, relative to it, is set by the key `tripoles/amplifier style/port width`. Making the latter equal to one will set the length of the pin to zero; if you want to maintain the same aspect ratio of the shape you need to compensate with the width.

For example, for the normal operational amplifier the key `tripoles/op amp/width` defaults to 1.7 and `tripoles/op amp/port width` is 0.7 (you need to peek that values in the source file `pgfcirctripoles.tex`). So you can do this:



```
\begin{tikzpicture}[]
\draw (0,0) node[op amp](A){};
\ctikzset{tripoles/op amp/port width=1,
tripoles/op amp/width=1.19, % 1.7*0.7
}
\draw (2.5,0) node[op amp](B){};
\draw
(A.out) node[red, circ]{} (A.+) node[blue, circ]{}
(B.out) node[red, circ]{} (B.+) node[blue, circ]{};
\end{tikzpicture}
```

4.20.2.4 Main amplifier label. The amplifier label (given as the text of the node) is normally more or less centered in the shape (in the case of the triangular shape, it is shifted a bit to the left to *seem* visually centered); since version 1.1.0 you can move it at the left side plus a fixed offset setting the key `component text` or the style with the same name to `left`; by default the key is `center`. You can change the offset with the key `left text distance` (default 0.3em; you must use a length here). These parameters are shared with IEEE-style logic ports.

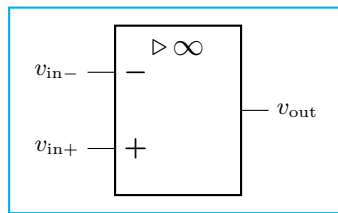


```
\begin{circuitikz}[]
\draw (0,2.5) node[plain amp]{\texttt{741}};
\draw (3,2.5)
node[plain amp, component text=left]
{\texttt{741}};
\ctikzset{component text=left}
\draw (0,0) node[op amp]{\texttt{741}};
\ctikzset{left text distance=0.6em}
\draw (3,0) node[op amp]{\texttt{741}};
\end{circuitikz}
```

These keys are also used for the positioning of the labels in the label positioning of IEEE logic gates (see 4.22.2).

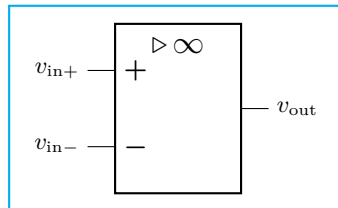
4.20.2.5 European-style amplifier customization. Thanks to the suggestions from David Rouvel (david.rouvel@iphc.cnrs.fr) there are several possible customization for the European-style amplifiers. Since 0.9.0, the default appearance of the symbol has changed to be more in line with the standard; notice that to have a bigger triangle by default we should require more packages, and I fear ConTeXt compatibility; but see later on how to change it. Notice that the font used for the symbol is defined in `tripoles/en amp/font2` and that the font used for the + and - symbols is `tripoles/en amp/font`.

You can change the distances of the inputs, using `tripoles/en amp/input height` (default 0.3):



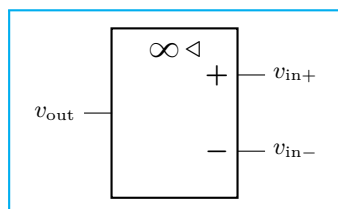
```
\begin{circuitikz}
\ctikzset{tripoles/en amp/input height=0.45}
\draw (0,0)node[en amp](E){}
(E.out) node[right] {$v_{\mathrm{out}}$}
(E.-) node[left] {$v_{\mathrm{in}}-$}
(E.+) node[left] {$v_{\mathrm{in}}+$};
\end{circuitikz}
```

and of course the key `noinv input up` is fully functional:



```
\begin{circuitikz}
\ctikzset{tripoles/en amp/input height=0.45}
\draw (0,0)node[en amp, noinv input up](E){}
(E.out) node[right] {$v_{\mathrm{out}}$}
(E.-) node[left] {$v_{\mathrm{in}}-$}
(E.+) node[left] {$v_{\mathrm{in}}+$};
\end{circuitikz}
```

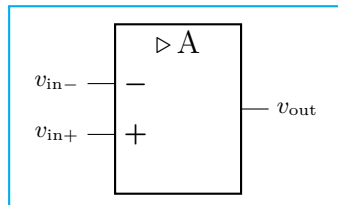
To flip the amplifier in the horizontal direction, you can use `xscale=-1` as usual:



```
\begin{circuitikz}
\ctikzset{tripoles/en amp/input height=0.45}
\draw (0,0)node[en amp, xscale=-1, noinv input up](E){}
(E.out) node[left] {$v_{\mathrm{out}}$}
(E.-) node[right] {$v_{\mathrm{in}}-$}
(E.+) node[right] {$v_{\mathrm{in}}+$};
\end{circuitikz}
```

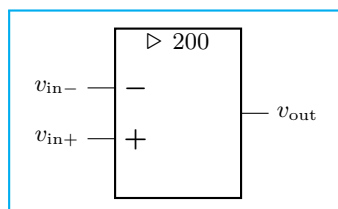
Notice that the label is fully mirrored, so check below for the generic way to change this.

You can use the new key `en amp text A` to change the infinity symbol with an A:



```
\begin{circuitikz}
\draw (0,0)node[en amp, en amp text A](E){}
(E.out) node[right] {$v_{\mathrm{out}}$}
(E.-) node[left] {$v_{\mathrm{in}}-$}
(E.+) node[left] {$v_{\mathrm{in}}+$} ;
\end{circuitikz}
```

And if you want, you can completely change the text using the key `en amp text=`, which by default is `$$\mathstrut{\triangleright}\,\,\{\infty\}$`:

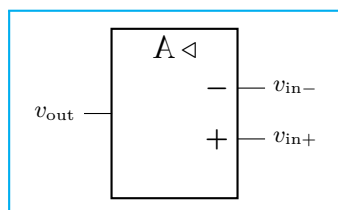


```
\begin{circuitikz}
\draw (0,0)node[en amp, en amp text={%
$\triangleright$ \small 200}](E){}
(E.out) node[right] {$v_{\mathrm{out}}$}
(E.-) node[left] {$v_{\mathrm{in}}-$}
(E.+) node[left] {$v_{\mathrm{in}}+$} ;
\end{circuitikz}
```

Notice two things here: the first, that `\triangleright` is enclosed in braces to remove the default spacing it has as a binary operator, and that `en amp text A` is simply a shortcut for

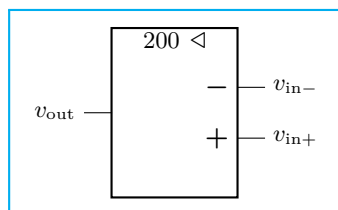
```
en amp text={$\mathstrut{\triangleright}\,\,\mathrm{A}$}
```


To combine flipping with a generic label you just do:



```
\begin{circuitikz}
\draw (0,0)node[en amp, xscale=-1, en amp text A](E){}
(E.out) node[left] {$v_{\mathrm{out}}$}
(E.-) node[right] {$v_{\mathrm{in}-}$}
(E.+) node[right] {$v_{\mathrm{in}+}$} ;
\end{circuitikz}
```

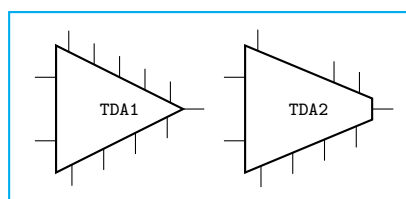
But notice that the “A” is also flipped by the `xscale` parameter. So the solution in this case is to use `scalebox`, like this:



```
\begin{circuitikz}
\draw (0,0)node[en amp, xscale=-1, en amp text={%
$\{\mathrm{triangleright}\}$ \scalebox{-1}[1]{\small 200}}](E){}
(E.out) node[left] {$v_{\mathrm{out}}$}
(E.-) node[right] {$v_{\mathrm{in}-}$}
(E.+) node[right] {$v_{\mathrm{in}+}$} ;
\end{circuitikz}
```

4.20.3 Designing your own amplifier

If you need a different kind of amplifier, you can use the `muxdemux` (see section 4.24) shape for defining one that suits your needs (you need version 1.0.0 for this to work, and 1.3.8 for the `draw only...` option).



```
\tikzset{tdax/.style={muxdemux,
muxdemux def={NL=2, Lh=3, NR=1, Rh=0,
NB=4, NT=5}, font=\scriptsize\ttfamily}}
\begin{circuitikz}
\draw (0,0) node[tdax](A){TDA1};
\draw (2.5,0) node[tdax, muxdemux def={Rh=0.5},
draw only top pins={1,4-5}]{TDA2};
\end{circuitikz}
```

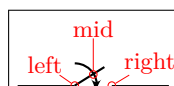
4.21 Switches, buttons and jumpers

Switches and buttons come in to-style (the simple ones and the pushbuttons), and as nodes.

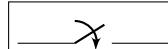
The switches can be scaled with the key `switches/scale` (default 1.0). Notice that scaling the switches will not scale the poles, which are controlled with their own parameters (see section 4.12).

4.21.1 Traditional switches

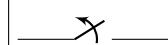
These are all of the to-style type:



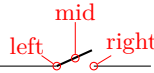
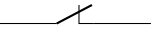
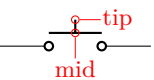
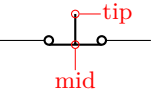
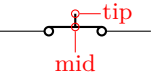
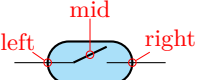
switch: Switch, type: `path-style`, nodename: `cspstshape`.
Aliases: `spst`. Class: `switches`.



closing switch: Closing switch, type: `path-style`, nodename: `cspstshape`. Aliases: `cspst`. Class: `switches`.



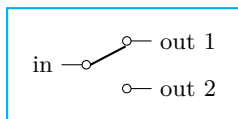
opening switch: Opening switch, type: `path-style`, nodename: `ospstshape`. Aliases: `ospst`. Class: `switches`.

	normal open switch: Normally open switch, type: <code>path-style</code> , nodename: <code>nosshape</code> . Aliases: <code>nos</code> . Class: <code>switches</code> .
	normal closed switch: Normally closed switch, type: <code>path-style</code> , nodename: <code>ncsshape</code> . Aliases: <code>ncs</code> . Class: <code>switches</code> .
	opening normal closed switch: Opening normally closed switch, type: <code>path-style</code> , nodename: <code>oncsshape</code> . Aliases: <code>oncs</code> . Class: <code>switches</code> .
	closing normal closed switch: Closing normally closed switch, type: <code>path-style</code> , nodename: <code>cncsshape</code> . Aliases: <code>cncs</code> . Class: <code>switches</code> .
	opening normal open switch: Opening normally open switch, type: <code>path-style</code> , nodename: <code>onosshape</code> . Aliases: <code>onos</code> . Class: <code>switches</code> .
	closing normal open switch: Closing normally open switch ¹¹⁸ , type: <code>path-style</code> , nodename: <code>cnosshape</code> . Aliases: <code>cnos</code> . Class: <code>switches</code> .
	push button: Normally open push button, type: <code>path-style</code> , nodename: <code>pushbuttonshape</code> . Aliases: <code>normally open push button</code> , <code>nopb</code> . Class: <code>switches</code> .
	normally closed push button: Normally closed push button, type: <code>path-style</code> , nodename: <code>ncpushbuttonshape</code> . Aliases: <code>ncpb</code> . Class: <code>switches</code> .
	normally open push button closed: Normally open push button (in closed position), type: <code>path-style</code> , nodename: <code>pushbuttoncshape</code> . Aliases: <code>nopbc</code> . Class: <code>switches</code> .
	normally closed push button open: Normally closed push button (in open position), type: <code>path-style</code> , nodename: <code>ncpushbuttonoshape</code> . Aliases: <code>ncpbo</code> . Class: <code>switches</code> .
	toggle switch: Toggle switch, type: <code>path-style</code> , nodename: <code>toggleswitchshape</code> . Class: <code>default</code> .
	reed: Reed switch, type: <code>path-style</code> , fillable, nodename: <code>reedshape</code> . Class: <code>switches</code> .

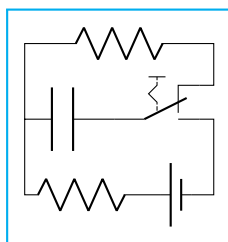
while this is a node-style component:

	spdt, type: <code>node (node[spdt]{})</code> . Class: <code>switches</code> .
---	--

¹¹⁸These last four were contributed by [Jakob Leide](#)



```
\begin{circuitikz} \draw
(0,0) node[spdt] (Sw) {}
(Sw.in) node[left] {in}
(Sw.out 1) node[right] {out 1}
(Sw.out 2) node[right] {out 2}
;\end{circuitikz}
```



```
\begin{circuitikz} \draw
(0,0) to[C] (1,0) to[toggle switch , n=Sw] (2.5,0)
-- (2.5,-1) to[battery1] (1.5,-1) to[R] (0,-1) -| (0,0)
(Sw.out 2) -| (2.5, 1) to[R] (0,1) -- (0,0)
;\end{circuitikz}
```

4.21.2 Cute switches

These switches have been introduced after version 0.9.0, and they come in also in to-style and in node-style, but they are size-matched so that they can be used together in a seamless way.

The path element (to-style) are:

	cute closed switch: Cute closed switch, type: path-style, nodename: cuteclosedswitchshape. Aliases: ccsw. Class: switches.
	cute open switch: Cute open switch, type: path-style, name=B, nodename: cuteopenswitchshape. Aliases: cosw. Class: switches.
	cute closing switch: Cute closing switch, type: path-style, nodename: cuteclosingswitchshape. Aliases: ccgsw. Class: switches.
	cute opening switch: Cute opening switch, type: path-style, nodename: cuteopeningswitchshape. Aliases: cogsw. Class: switches.

while the node-style components are the single-pole, double-throw (spdt) ones:

	Cute spdt up, type: node (node[cute spdt up]{}). Class: switches.
	Cute spdt mid, type: node (node[cute spdt mid]{}). Class: switches.
	Cute spdt down, type: node (node[cute spdt down]{}). Class: switches.
	Cute spdt up with arrow, type: node (node[cute spdt up arrow]{}). Class: switches.
	Cute spdt mid with arrow, type: node (node[cute spdt mid arrow]{}). Class: switches.



Cute spdt down with arrow, type: `node (node[cute spdt down arrow]{}).` Class: **switches**.

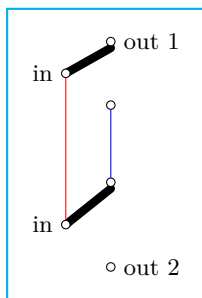
4.21.2.1 Cute switches anchors

The nodes-style switches have the following anchors:



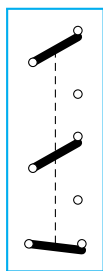
Please notice the position of the normal anchors at the border of the `ocirc` shape for the cute switches; they are thought to be compatible with an horizontal wire going out. Additionally, you have the `cin`, `cout 1` y `cout 2` which are anchors on the center of the contacts.

For more complex situations, the contact nodes are available¹¹⁹ using the syntax *name of the node*-`in`, `...-out 1` and `...-out 2`, with all their anchors.



```
\begin{circuitikz}
\draw (0,0) node[cute spdt up] (S1) {}
(S1.in) node[left] {in}
(S1.out 1) node[right] {out 1};
\draw (0,-2) node[cute spdt up,
/tikz/circuitikz/bipoles/cuteswitch/height=0.8] (S2) {}
(S2.in) node[left] {in}
(S2.out 2) node[right] {out 2};
\draw [red] (S1-in.s) -- (S2-in.n);
\draw [blue] (S1-out 2.s) -- (S2-out 1.n);
\end{circuitikz}
```

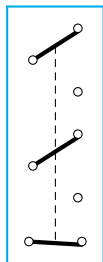
The `mid` anchor in the cute switches (both path- and node-style) can be used to combine switches to get more complex configurations:



```
\begin{circuitikz}
\draw (0,1.4) node[cute spdt up] (S1){};
\draw (0,0) node[cute spdt up] (S2){};
\draw (0,-1) node[cuteclosedswitchshape, yscale=-1] (S3){};
\draw [densely dashed] (S1.mid)--(S2.mid)--(S3.mid);
\end{circuitikz}
```

4.21.2.2 Cute switches customization

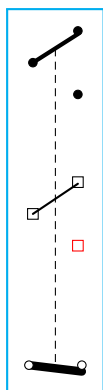
You can use the key `bipoles/cuteswitch/thickness` to decide the thickness of the switch lever. The units are the diameter of the `ocirc` connector, and the default is 1.



```
\begin{circuitikz}
\ctikzset{bipoles/cuteswitch/thickness=0.5}
\draw (0,1.4) node[cute spdt up] (S1){};
\draw (0,0) node[cute spdt up] (S2){};
\draw (0,-1) node[cuteclosedswitchshape, yscale=-1] (S3){};
\draw [densely dashed] (S1.mid)--(S2.mid)--(S3.mid);
\end{circuitikz}
```

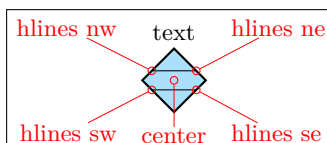
¹¹⁹Thanks to @marmot on tex.stackexchange.com.

Finally, the switches are normally drawn using the `ocirc` shape, but you can change it, as in the following example, with the key `bipoles/cuteswitch/shape`. Be careful that the shape is used with its defaults (which can lead to strange results), and that the standard anchors will be correct only for `circ` and `ocirc` shapes, so you have to use the internal node syntax to connect it.

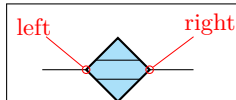


```
\begin{circuitikz}
\begin{scope}
\ctikzset{bipoles/cuteswitch/thickness=0.5,
bipoles/cuteswitch/shape=circ}
\draw (0,2) node[cute spdt up] (S1){};
\ctikzset{bipoles/cuteswitch/thickness=0.25,
bipoles/cuteswitch/shape=emptyshape}
\draw (0,0) node[cute spdt up] (S2){};
\draw (S2.cin) node[draw, inner sep=2pt]{};
\draw (S2.cout 1) node[draw, inner sep=2pt]{};
\draw (S2.cout 2) node[draw=red, inner sep=2pt]{};
\end{scope}
\draw (0,-2) node[cuteclosedswitchshape, yscale=-1] (S3){};
\draw [densely dashed] (S1.mid)--(S2.mid)--(S3.mid);
\end{circuitikz}
```

4.21.3 Proximity switches



proximeter, type: node, fillable (`node[proximeter]{text}`). Class: switches.

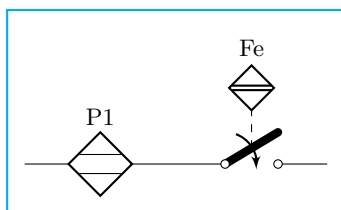


`inline proximeter`: proximeter switch, inline, type: path-style, fillable, nodename: proximeter. Class: switches.

The `proximeter` shape¹²⁰ can be used as a dipole with the `inline proximeter` variant.

It has been assigned to the `switches` class; you can adjust the (relative) thickness of the inside horizontal lines with the key `proximeter/hlines thickness` (default 0.5) and their vertical position with `proximeter/hlines position` (default 0.3). You can also change the default size of *all* proximeter symbols by changing `proximeter/width` (only safe at picture level; better set in the preamble if you need to change it). The default value is 0.3).

Notice in the following example that, as ever for node-type shape, the text is not included in the bounding box:

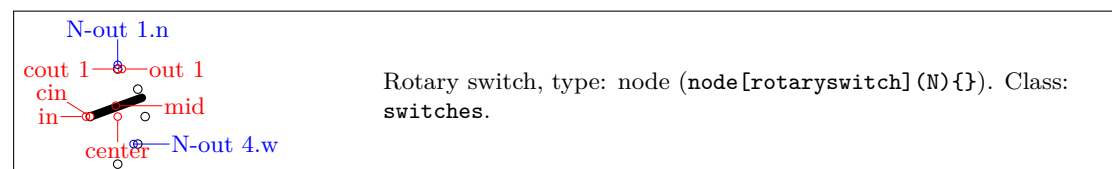


```
\begin{tikzpicture}
\tikzset{small up proxi/.style={proximeter, solid,
circuitikz/switches/scale=0.707,
circuitikz/proximeter/hlines thickness=1,
circuitikz/proximeter/hlines position=0.1}}
\draw (0,0) to[inline proximeter, l=P1] ++(2,0)
to[ccgsw, name=P2] ++(2,0);
\draw[dashed] (P2.mid) -- ++(0,0.5)
node[small up proxi, above] (P2p){Fe}
(P2p.north) ++ (0,0.5); % extend bounding box
\end{tikzpicture}
```

¹²⁰Suggested by Anisio Rogerio Braga, implemented in v1.5.2; see also [here](#).

4.21.4 Rotary switches

Rotary switches are a kind of generic multipole switches; they are implemented as a strongly customizable element (and a couple of styles to simplify its usage). The basic element is the following one, and it has the same basic anchors of the cute switches, included the access to internal nodes (shown in blue here).

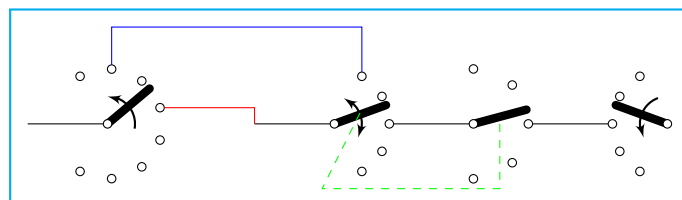


Notice that the name of the shape is `rotaryswitch`, no spaces. The default rotary switch component has 5 channels (this is set in the parameter `multipoles/rotary/channels`), spanning from -60° to 60° (parameter `multipoles/rotary/angle`) and with the wiper at 20° (parameter `multipoles/rotary/wiper`). Moreover, there are by default no arrows on the wiper; if needed, you can change this default setting the parameter `multipoles/rotary/arrow` which can assume the values `none`, `cw` (clockwise), `ccw` (counterclockwise) or `both`.

To simplify the usage of the component, a series of styles are defined: `rotary switch=<channels> in <angle> wiper <wiper angle>` (notice the space in the name of the style!). Using `rotary switch` without parameters will generate a default switch.

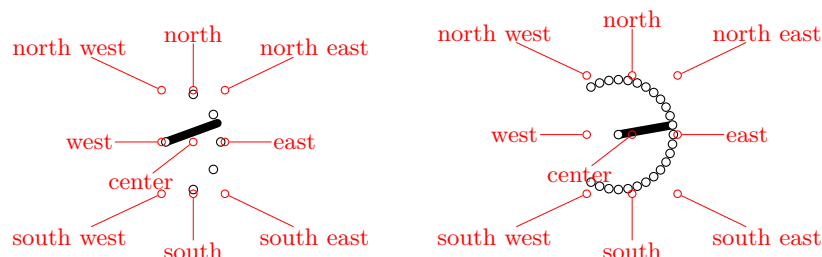
To add arrows, you can use the styles `rotary switch -` (no arrow, whatever the default), `rotary switch <-` (counterclockwise arrow), `rotary switch ->` (clockwise) and `rotary switch <->` (both).

Notice that the defaults of the styles are the same as the default values of the parameters, but that if you change globally the defaults using the keys mentioned above, you only change the defaults for the “bare” component `rotaryswitch`, not for the styles.

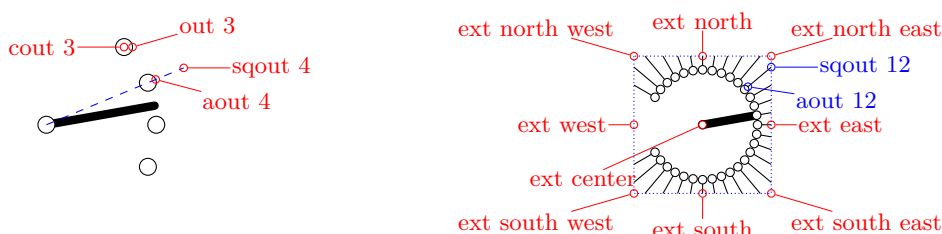


```
\begin{circuitikz}
\ctikzset{multipoles/rotary/arrow=both}
\draw (0,0) -- ++(1,0) node[rotary switch <-=8 in 120 wiper 40, anchor=in](A){};
\draw (3,0) -- ++(1,0) node[rotary switch, anchor=in](B){}; % default values
\draw[red] (A.out 4) -| (3,0);
\draw[blue] (A-out 2.n) -- ++(0,0.5) -| (B-out 1.n);
\draw (B.out 3) -- ++(1,0) node[rotary switch -=5 in 90 wiper 15, anchor=in](C){};
\draw (C.out 3) -- ++(1,0) node[rotary switch ->, xscale=-1, anchor=out 3](D){};
\draw[green, dashed] (B.mid) -- ++(-.5,-1) -| (C.mid);
\end{circuitikz}
```

4.21.4.1 Rotary switch anchors Rotary switches anchors are basically the same as the cute switches, including access (with the `<node name>--<anchor name>` notation) to the internal connection nodes. The geographical anchors work as expected, marking the limits of the component.



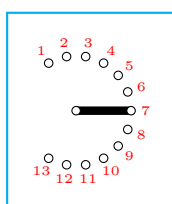
In addition to the anchors they have in common with the cute switches, the rotary switch has the so called “angled” anchors and the “external square anchors”. *Angled anchors*, called `aout 1`, `aout 2` and so forth, are anchors placed on the output poles at the same angle as the imaginary lines coming from the input pole; *square anchors*, called `sqout 1`... , are located on an imaginary square surrounding the rotary switch on the same line.



The code for the diagram at the left, above, without the markings for the anchors, is:

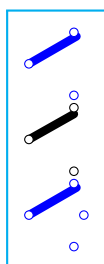
```
\begin{circuitikz}
  \draw (8,0) node[rotary switch --=31 in 150 wiper 10] (D){};
  \foreach \i in {1,...,31} \draw (D.sqout \i) -- (D.aout \i);
  \draw[blue, densely dotted] (D.ext north west) rectangle (D.ext south east);
\end{circuitikz}
```

One possible application for the angled and the “on square” anchors is that you can use them to move radially from the output poles, for example for adding numbers:



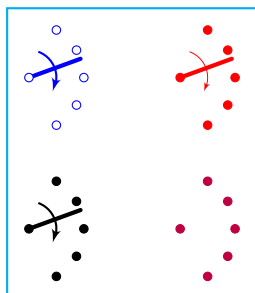
```
\begin{circuitikz}
\draw (0,0) node[rotary switch=13 in 120 wiper 0] (S){};
\foreach \i in {1,...,13} % requires "calc"
  \path ($(S.aout \i)!1ex!(S.sqout \i)$)
    node[font=\tiny\color{red}]{\i};
\end{circuitikz}
```

Finally, notice that the value of width for the rotary switches is taken from the one for the “cute switches” which in turn is taken from the width of traditional `spdt` switch, so that they match (notice that the “center” anchor is better centered in the rotary switch, so you have to explicitly align them).



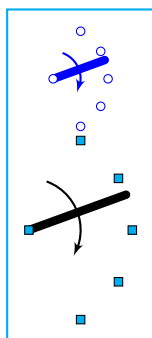
```
\begin{circuitikz}
  \draw (0,0) node[color=blue, rotary switch=2 in 35 wiper 30,
    anchor=in] (R){};
  \draw (0,-1) node[cute spdt up, anchor=in] (C){};
  \draw (0,-2) node[color=blue, rotary switch=3 in 35 wiper 30,
    anchor=in] (R){};
\end{circuitikz}
```

4.21.4.2 Rotary switch customization Apart from the basic customization seen above (number of channels, etc.) you can change, as in the cute switches, the shape used by the connection points with the parameter `multipoles/rotary/shape`, and the thickness of the wiper with `multipoles/rotary/thickness`; if you specify a negative value here, the wiper is not drawn at all¹²¹ (but the anchors, and the additional arrow if you specify it, are still there). The optional arrow has thickness equal to the standard bipole thickness `bipoles/thickness` (default 2).



```
\begin{circuitikz}
  \ctikzset{multipoles/rotary/thickness=0.5}
  \draw (0,2) node[rotary switch ->, color=blue] (S1){};
  \ctikzset{multipoles/rotary/shape=circ}
  \draw (0,0) node[rotary switch ->] (S2){};
  \ctikzset{bipoles/thickness=0.5}
  \draw (2,2) node[rotary switch ->, color=red] (S3){};
  \ctikzset{multipoles/rotary/thickness=-1}
  \draw (2,0) node[rotary switch, color=purple] (S4){};
\end{circuitikz}
```

Finally, the size can be changed using the parameter `tripoles/spdt/width` (default 0.85).

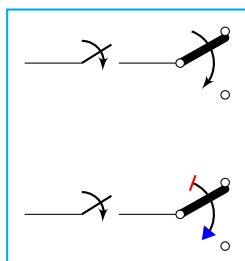


```
\begin{circuitikz}
  \draw (0,2) node[rotary switch ->, color=blue] (S1){};
  \ctikzset{tripoles/spdt/width=1.6, fill=cyan,
    multipoles/rotary/shape=osquarepole}
  \draw (0,0) node[rotary switch ->] (S2){};
\end{circuitikz}
```

4.21.5 Switch arrows

You can change the arrow tips used in all switches (traditional and “cute”) with the key `switch end arrow` (by default the key is the word “default” to obtain the default arrow, which is `latexslim`). Also you can change the start arrow with the corresponding `switchable start arrow` or `wiper start arrow` (the default value “default” is equivalent to {}, which means no arrow). The keys are settable with `\ctikzset` as with `\tikzset` (to ease their usage in nodes).

You can change that globally or locally, as ever. The tip specification is the one you can find in the *TikZ* manual (“[Arrow Tip Specifications](#)”).

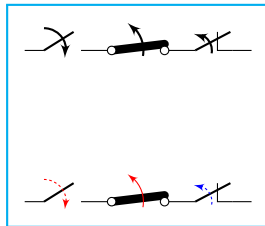


```
\begin{circuitikz}
  \draw (0,2) to[cspst] ++(2,0)
    node[cute spdt up arrow, anchor=in]{};
  \draw (0,0) to[cspst] ++(2,0)
    node[cute spdt up arrow, anchor=in,
    switch start arrow={Bar[red]},
    switch end arrow={Triangle[blue]}]{};
\end{circuitikz}
```

You can also have the option to change the color, relative thickness, and dash pattern by setting keys with the `\ctikzset` command under the `switch arrows customization` hierarchy. The available keys are:

¹²¹Suggested by users [kabenjuk](#) and [cis](#) on TeX-SX.com; see [this question&answer for an application](#).

parameter	default	description
<code>relative thickness</code>	1.0	multiply the class thickness
<code>color</code>	default	stroke color: default is the same as the component
<code>dash</code>	default	dash pattern: default means not to change the setting for the component; none means unbroken line; every other input is a dash pattern. ¹²²

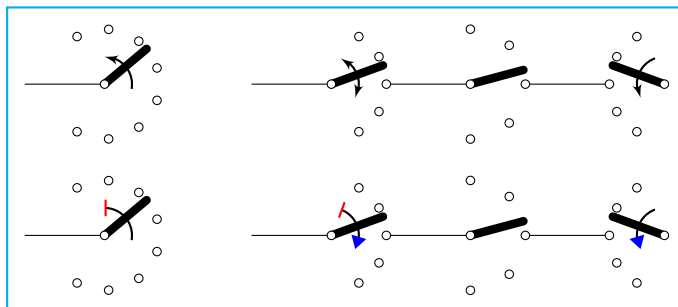


```

\begin{circuitikz}
\draw (0,2) to[spst] ++(1,0) to[cogsw]
      ++(1,0) to[oncs] ++(1,0);
\ctikzset{switch arrows/color=red}
\ctikzset{switch arrows/relative thickness=0.5}
\ctikzset{switch arrows/dash={{1pt}{1pt}}}}
\draw (0,0) to[spst] ++(1,0) to[cogsw, switch arrows/dash=none]
      ++(1,0) to[oncs, switch arrows/color=blue] ++(1,0);
\end{circuitikz}

```

4.21.5.1 Rotary switch arrows. You can change the rotary switch arrow shape in the same way as you change the ones in regular switches. Notice however that if you set either `switch end arrow` or `switch start arrow` they will be followed only if you have set both arrows with `<->` or equivalent, otherwise just one will be used.



```

\begin{circuitikz}
\ctikzset{multipoles/rotary/arrow=both}
\draw (0,0) -- ++(1,0) node[rotary switch <-=8 in 120 wiper 40, anchor=in] (A){};
\draw (3,0) -- ++(1,0) node[rotary switch, anchor=in] (B){}; % default values
\draw (B.out 3) -- ++(1,0) node[rotary switch -=5 in 90 wiper 15, anchor=in] (C){};
\draw (C.out 3) -- ++(1,0) node[rotary switch ->, xscale=-1, anchor=out 3] (D){};
\ctikzset{switch end arrow={Triangle[blue]}}
\ctikzset{switch start arrow={Bar[red]}}
\begin{scope}[yshift=-2cm]
\draw (0,0) -- ++(1,0) node[rotary switch <-=8 in 120 wiper 40, anchor=in] (A){};
\draw (3,0) -- ++(1,0) node[rotary switch, anchor=in] (B){}; % default values
\draw (B.out 3) -- ++(1,0) node[rotary switch -=5 in 90 wiper 15, anchor=in] (C){};
\draw (C.out 3) -- ++(1,0) node[rotary switch ->, xscale=-1, anchor=out 3] (D){};
\end{scope}
\end{circuitikz}

```

¹²²Follows the syntax of the pattern sequence `\pgfsetdash` — see TikZ manual [for details](#); phase is always zero. Basically you pass pairs of dash-length – blank-length dimensions, see the examples.

4.21.6 Jumpers

You can think of jumpers like a kind of switches (they have the same function, just the way of operating them is different). CircuiTikZ has two types of jumper symbols available, the simple ones and the three pins (or two-ways) ones.

4.21.6.1 Simple jumpers. These are the most common ones. They come in three variations, bare, open and closed.

	bare jumper: Bare jumper, type: <code>path-style</code>, fillable, name=B, nodename: <code>bjumpershape</code>. Class: <code>switches</code>.
	open jumper: Open jumper, type: <code>path-style</code>, fillable, name=B, nodename: <code>ojumpershape</code>. Class: <code>switches</code>.
	closed jumper: Closed jumper, type: <code>path-style</code>, fillable, name=B, nodename: <code>cjumpershape</code>. Class: <code>switches</code>.

The `top arc` anchor can be used to locate the position of the top of the wire (when present). In bare jumper, the anchor is located in the middle of the connectors gap.

```
\begin{circuitikz}[scale=0.8]
\draw (0,0) to[open jumper, l=J1] ++(2,0)
to[closed jumper, l_=J2, name=J2] ++(2,0)
to[bare jumper=J3] ++(2,0);
\draw [dashed] (J2.top arc) -- ++(0,0.5)
node[above] {\tiny open to enable};
\end{circuitikz}
```

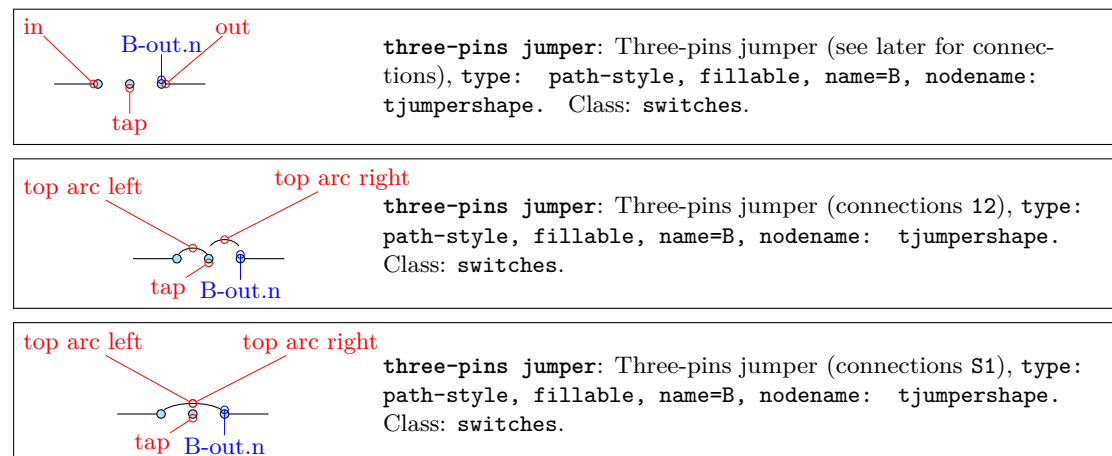
Similarly to switches, you have access to the subnodes representing the contacts, to be able to draw wires at different angles.

The kind of poles used in the diagram can be changed with the `\ctikzset` key `bipoles/jumpers/shape`.

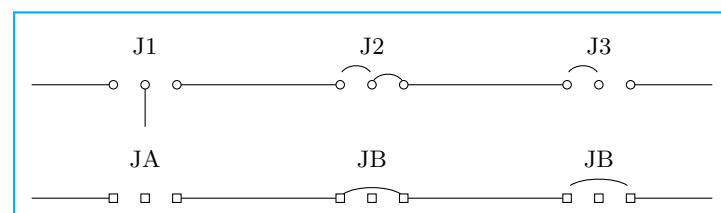
```
\begin{circuitikz}[scale=0.8]
\draw (0,0) to[open jumper, l=J1, name=J1] ++(2,0)
to[closed jumper, l_=J2, name=J2,
bipoles/jumper/shape=odiamondpole] ++(2,0);
\draw [red] (J2-in.-135) -- ++(-135:1)
node[font=\tiny, below]{marked \emph{hot}};
\end{circuitikz}
```

4.21.6.2 Two-ways (three-pins) jumpers. In this case, the symbol represent two-ways jumpers (normally, three pins that can be connected in a couple of ways).

To maintain flexibility, every possible combination of bare, open or closed is available; but to avoid having to define too much different bipoles, a different approach is used here. You have to specify the style using a different key, namely `tjumper connections`.



The option is used as shown in the following example, or by setting the key using `\ctikzset`. The value **must** be two characters, either two numbers (where 0 means “bare”, 1 means “open”, and 2 means “closed”) or the letter **S** (for “span”) and one number. In the latter case, the arc will connect the first and last pole¹²³

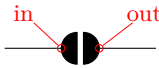
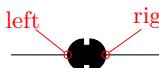
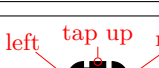
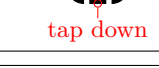
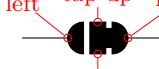


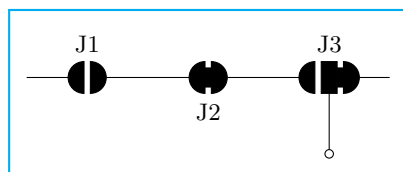
```
\begin{circuitikz}
  \draw (0,1.5) to[three-pins jumper, l=J1, name=T] ++(3,0)
  to[three-pins jumper, tjumper connections=21, l=J2] ++(3,0)
  to[three-pins jumper, tjumper connections=20, l=J3] ++(3,0);
  \draw (T.tap) -- ++(0,-0.5);
  \ctikzset{bipoles/jumper/shape=osquarepole}
  \draw (0,0) to[three-pins jumper, l=JA, name=T] ++(3,0)
  to[three-pins jumper, tjumper connections=S1, l=JB] ++(3,0)
  to[three-pins jumper, tjumper connections=S2, l=JB] ++(3,0);
\end{circuitikz}
```

4.21.7 Solder jumpers.

Solder jumpers are basically jumpers that can be closed or opened on the printed circuit board. Although electrically they behave exactly as jumpers, these are thought to change configurations in a more stable way (to change them from their default connection you have to use a cutter and/or a soldering iron).

¹²³Although really I never saw an example of this use... You never know.

	open solder jumper: Open solder jumper, type: path-style, nodename: osjumpershape. Class: switches.
	closed solder jumper: Closed solder jumper, type: path-style, nodename: csjumpershape. Class: switches.
	open double solder jumper: Open double solder jumper, type: path-style, nodename: odsjumpershape. Class: switches.
	left double solder jumper: Left double solder jumper, type: path-style, nodename: ldsjumpershape. Class: switches.
	right double solder jumper: Right double solder jumper, type: path-style, nodename: rdsjumpershape. Class: switches.
	closed double solder jumper: Closed double solder jumper, type: path-style, nodename: cdsjumpershape. Class: switches.

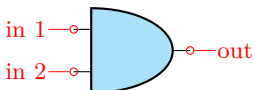
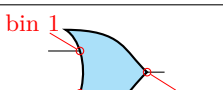
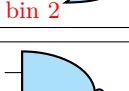


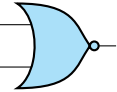
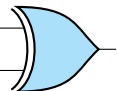
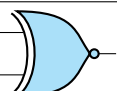
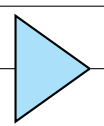
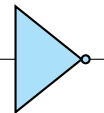
```
\begin{circuitikz}[scale=0.8]
\draw (0,0) to[open solder jumper, l=J1] ++(2,0)
to[closed solder jumper, l_=J2, name=J2]
++(2,0)
to[right double solder jumper, l=J3,
name=J3] ++(2,0);
\draw (J3.tap down) -- ++(0,-1) node[ocirc]{};
\end{circuitikz}
```

4.22 Logic gates

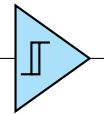
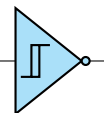
Logic gates, with two or more input, are supported. Albeit in principle these components are multipoles, the are considered tripoles here, for historical reasons (when they just had two inputs).

4.22.1 American Logic gates

	American AND port, type: node, fillable (node[american and port]{}). Class: logic ports.
	American OR port, type: node, fillable (node[american or port]{}). Class: logic ports.
	American NAND port, type: node, fillable (node[american nand port]{}). Class: logic ports.

	American NOR port, type: node, fillable (<code>node[american nor port]{}</code>). Class: logic ports.
	American XOR port, type: node, fillable (<code>node[american xor port]{}</code>). Class: logic ports.
	American XNOR port, type: node, fillable (<code>node[american xnor port]{}</code>). Class: logic ports.
	American BUFFER port, type: node, fillable (<code>node[american buffer port]{}</code>). Class: logic ports.
	American NOT port, type: node, fillable (<code>node[american not port]{}</code>). Class: logic ports.

There is no “european” version of the following symbols; for now they are used both in **american** and **european** styles, but it may change in the future.

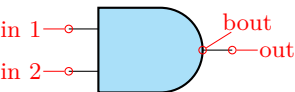
	Non-Inverting Schmitt trigger, type: node, fillable (<code>node[schmitt]{}</code>). Class: logic ports.
	Inverting Schmitt trigger, type: node, fillable (<code>node[invschmitt]{}</code>). Class: logic ports.

4.22.2 IEEE logic gates

In addition to the legacy ports, since release 1.1.0, logic ports following the recommended geometry of distinctive-shape symbols in IEEE Std 91a-1991 Annex A (Recommended symbol proportions) are also available¹²⁴.

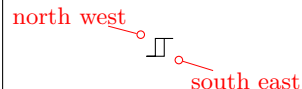
These ports are completely independent from the legacy set (either **american** or **european**); they are not enabled by default because the relative size of the ports is very different from the legacy ones, and that will disrupt every schematic (especially if drawn with absolute coordinate). If you want to use them as default, you can use the command `\ctikzset{logic ports=ieee}` and by default the shapes **and**, **port**, **or**, **port** and so on will be the IEEE standard ones.

The transmission gate (also known as “bowtie”) components are not described in the IEEE standard, so they are simply inspired by the other IEEE ports — this is why their name is prefixed by **ieee** and not by **ieeestd**. They are aliased to **tgate** and **double tgate** though, and it is recommended to use those names (maybe in the future there will be **american ports** and/or **european ports** versions available).

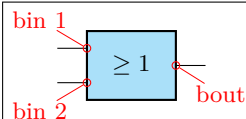
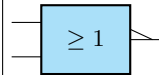
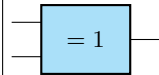
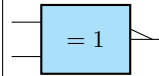
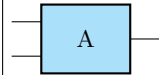
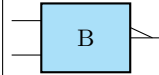
	IEEE standard “and” port, type: node, fillable (<code>node[ieeestd and port]{}</code>). Class: logic ports.
---	---

¹²⁴Thanks to Jason for proposing it and digging out the info, see this [GitHub issue](#).

	IEEE standard “nand” port, type: node, fillable (<code>node[ieeestd nand port]{}</code>). Class: logic ports.
	IEEE standard “or” port, type: node, fillable (<code>node[ieeestd or port]{}</code>). Class: logic ports.
	IEEE standard “nor” port, type: node, fillable (<code>node[ieeestd nor port](N){}</code>). Class: logic ports.
	IEEE standard “xor” port, type: node, fillable (<code>node[ieeestd xor port]{}</code>). Class: logic ports.
	IEEE standard “xnor” port, type: node, fillable (<code>node[ieeestd xnor port]{}</code>). Class: logic ports.
	IEEE standard buffer port, type: node, fillable (<code>node[ieeestd buffer port]{}</code>). Class: logic ports.
	IEEE standard “not” port, type: node, fillable (<code>node[ieeestd not port]{}</code>). Class: logic ports.
	Schmitt port matched to IEEE standard ports, type: node, fillable (<code>node[ieeestd schmitt port]{}</code>). Class: logic ports.
	Inverting Schmitt port matched to IEEE standard ports, type: node, fillable (<code>node[ieeestd invschmitt port]{}</code>). Class: logic ports.
	IEEE style transmission gate, type: node, fillable (<code>node[ieee tgate]{}</code>). Class: logic ports.
	IEEE style double transmission gate, type: node, fillable (<code>node[ieee double tgate]{}</code>). Class: logic ports.
	Inverting dot for IEEE ports, type: node, fillable (<code>node[notcirc]{}</code>). Class: logic ports.

	Schmitt symbol to add to input pins if needed, type: node, fillable (node[schmitt symbol]{}). Class: logic ports.
---	---

4.22.3 European Logic gates

	European AND port, type: node, fillable (node[european and port]{}). Class: logic ports.
	European OR port, type: node, fillable (node[european or port]{}). Class: logic ports.
	European NAND port, type: node, fillable (node[european nand port]{}). Class: logic ports.
	European NOR port, type: node, fillable (node[european nor port]{}). Class: logic ports.
	European XOR port, type: node, fillable (node[european xor port]{}). Class: logic ports.
	European XNOR port, type: node, fillable (node[european xnor port]{}). Class: logic ports.
	European BUFFER port, type: node, fillable (node[european buffer port]{}). Class: logic ports.
	European NOT port, type: node, fillable (node[european not port]{}). Class: logic ports.
	European blank port, type: node, fillable (node[european blank port]{A}). Class: logic ports.
	European blank not port, type: node, fillable (node[european blank not port]{B}). Class: logic ports.

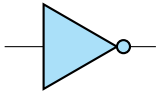
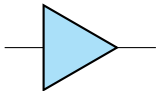
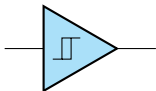
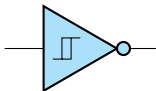
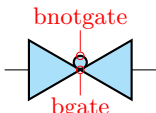
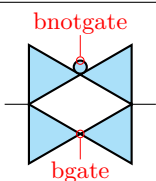
If (default behaviour) `americanports` option is active (or the style `american ports` is used), the shorthands `and port`, `or port`, `buffer port`, `nand port`, `nor port`, `not port`, `xor port`, `xnor port`, `schmitt port` and `invschmitt port` are equivalent to the american version of the respective logic port.

If otherwise `europeanports` option is active (or the style `european ports` is used), the shorthands `and port`, `or port`, `buffer port`, `nand port`, `nor port`, `not port`, `xor port`, `xnor port` are equivalent to the european version of the respective logic port; `schmitt port` and `invschmitt port` are the same as in `american ports` style.

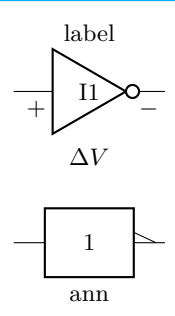
Finally, for version 1.1.0 and up, you can use the style `ieee ports` to set the shorthands to the set of `ieeestd` ports. (There is no global option for this).

4.22.4 Path-style logic ports

The one-input, one-output ports have a handy path-style equivalent; they are the following:

	<code>inline not:</code> “not” logic port, type: <code>path-style</code> , fillable, nodename: <code>not port</code> . Class: <code>logic ports</code> .
	<code>inline buffer:</code> “buffer” logic port, type: <code>path-style</code> , fillable, nodename: <code>buffer port</code> . Class: <code>logic ports</code> .
	<code>inline schmitt:</code> Schmitt logic port, type: <code>path-style</code> , fillable, nodename: <code>schmitt port</code> . Class: <code>logic ports</code> .
	<code>inline invschmitt:</code> Inverting Schmitt logic port, type: <code>path-style</code> , fillable, nodename: <code>invschmitt port</code> . Class: <code>logic ports</code> .
	<code>inline tgate:</code> transmission gate, type: <code>path-style</code> , fillable, nodename: <code>tgate</code> . Class: <code>logic ports</code> .
	<code>inline double tgate:</code> double transmission gate, type: <code>path-style</code> , fillable, nodename: <code>double tgate</code> . Class: <code>logic ports</code> .

Those ports follow the current selected style, although you can change it on the fly (even if it does not have a lot of sense); you can apply labels, annotations and (again, not a lot of sense) voltages to them. The assigned value is typeset as if it were the main text of the node.

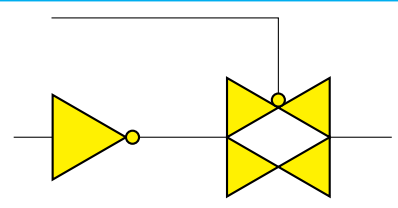


```

\begin{circuitikz}[american]
\ctikzset{logic ports=ieee}
\draw (0,0) to[inline not=I1, l=label, v=$\Delta V$] ++(2,0);
\draw (0,-2) to[inline not, a=ann, european ports] ++(2,0);
\end{circuitikz}

```

Notice that in the inline version the leading pins are not drawn, so in the case of the transmission gates you have to use the border pins to connect the gates.



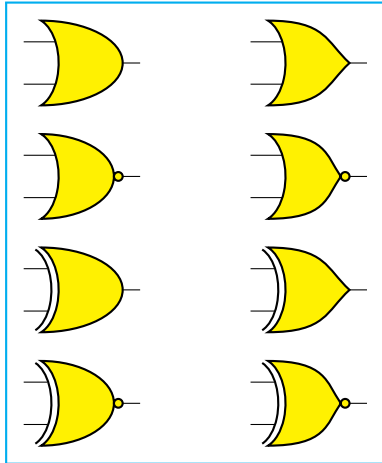
```

\begin{circuitikz}[ ]
\ctikzset{logic ports=ieee,
logic ports/fill=yellow}
\draw (0,0) to[inline not] ++(2,0)
to[inline double tgate, name=P] ++(3,0)
(P.bnotgate) |- ++(-3,1);
\end{circuitikz}

```


4.22.5 American ports usage

Since version 1.0.0, the default shape of the family of american “or” ports has changed to a more “pointy” one, for better distinguish them from the “and”-type ports. You can still go back to the previous aspect with the key `american or shape` that can be set to `pointy` or `roundy`. The `legacy` style will enact the old, roundy style also.

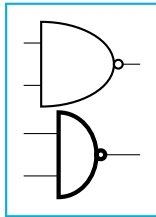


```
\begin{circuitikz}[
  american
  % legacy shapes
  \ctikzset{american or shape=roundy}
  \ctikzset{logic ports/fill=yellow}
  \node [or port](O1) at (0,0) {};
  \node [nor port](O2) at (0,-1.5) {};
  \node [xor port](O3) at (0,-3) {};
  \node [xnor port](O4) at (0,-4.5) {};
  \begin{scope}[xshift=3cm]
    % new shapes
    \ctikzset{american or shape=pointy}
    \node [or port](O1) at (0,0) {};
    \node [nor port](O2) at (0,-1.5) {};
    \node [xor port](O3) at (0,-3) {};
    \node [xnor port](O4) at (0,-4.5) {};
  \end{scope}
\end{circuitikz}
```

4.22.5.1 American logic port customization Logic port class is called `logic ports`, so you can scale them all with `logic ports/scale` (default 1.0).

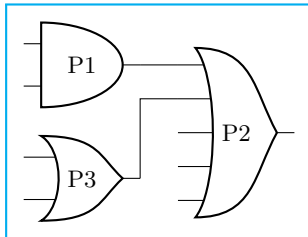
As for most components, you can change the width and height of the ports; the thickness is given by the parameter `tripoles/thickness` (default 2).

It is possible to change height and width of the logic ports using the parameters `tripoles/american type port/` plus width or height:



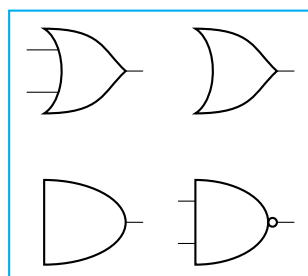
```
\tikz \draw (0,0) node[nand port] {}; \par
\ctikzset{tripoles/american nand port/input height=.2}
\ctikzset{tripoles/american nand port/port width=.4}
\ctikzset{tripoles/thickness=4}
\tikz \draw (0,0) node[nand port] {};
```

This is especially useful if you have ports with more than two inputs, which are instantiated with the parameter `number inputs` :



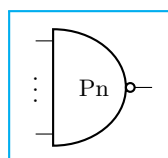
```
\begin{circuitikz}
\draw (0,3) node[american and port] (A) {P1};
\begin{scope}
  \ctikzset{tripoles/american or port/height=1.6}
  \draw (A.out) -- ++(0.5,0)
    node[american or port,
      number inputs=5,
      anchor=in 1] (B) {P2};
\end{scope}
\draw (0,1.5) node[american or port] (C) {P3};
\draw (C.out) |- (B.in 2);
\end{circuitikz}
```

You can suppress the drawing of the logic ports input leads by using the boolean key `logic ports draw input leads` (default `true`) or, locally, with the style `no input leads` (that can be reverted with `input leads`), like in the following example. The anchors do not change and you have to take responsibility to make the connection to the “border”-anchors.



```
\begin{circuitikz}
  \node [or port](O1) at (0,2) {};
  \node [or port, no input leads](O1) at (2,2) {};
  \ctikzset{logic ports draw input leads=false}
  \node [and port](A1) at (0,0) {};
  \node [and port, input leads](A1) at (2,0) {};
\end{circuitikz}
```

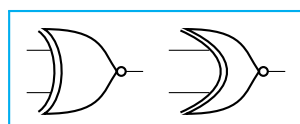
This is useful if you need to draw a generic port, like the one following here:



```
\begin{circuitikz}
  \ctikzset{tripoles/american nand port/height=1.6}
  \draw (0,0)
    node[american nand port,
    circuitikz/tripoles/american nand port/height=1.1,
    number inputs=5, no input leads,
    ] (B) {Pn};
  \draw (B.in 1) -- (B.bin 1) (B.in 5) -- (B.bin 5);
  \node[rotate=90] at (B.in 3) {\dots};
\end{circuitikz}
```

In an analogous manner, there is a setting `logic ports draw output leads` (and a corresponding style `no output leads`) that suppresses the drawing of the output lead. A shortcut boolean key `logic ports draw leads` will suppress or enable all leads (the corresponding styles are `no leads` and `all leads`).

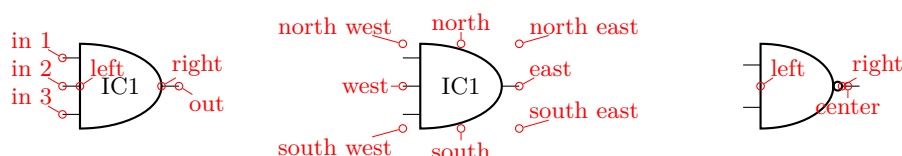
You can tweak the appearance of american “or” family (`or`, `nor`, `xor` and `xnor`) ports, too, with the parameters `inner` (how much the base circle goes “into” the shape, default 0.3) and `angle` (the angle at which the base starts, default 70).



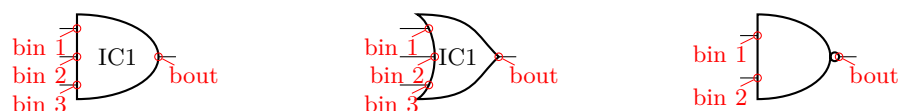
```
\tikz \draw (0,0) node[xnor port] {};
\ctikzset{tripoles/american xnor port/inner=.7}
\ctikzset{tripoles/american xnor port/angle=40}
\tikz \draw (0,0) node[xnor port] {};
```

4.22.5.2 American logic port anchors

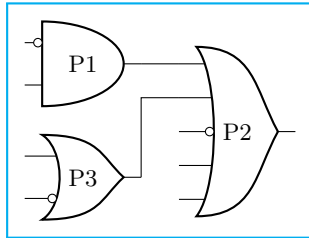
These are the anchors for logic ports:



You also have “border pin anchors”:

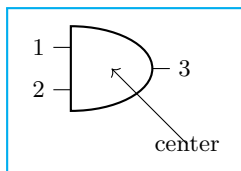


These anchors are especially useful if you want to negate inputs:

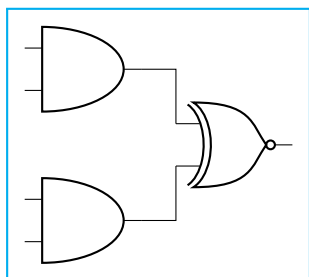


```
\begin{circuitikz}
\draw (0,3) node[american and port] (A) {P1};
\node at (A.bin 1) [ocirc, left]{} ;
\begin{scope}
\tikzset{tripoles/american or port/height=1.6}
\draw (A.out) -- ++(0.5,0) node[american or port,
number inputs=5, anchor=in 1] (B) {P2};
\node at (B.bin 3) [ocirc, left]{} ;
\end{scope}
\draw (0,1.5) node[american or port] (C) {P3};
\node at (C.bin 2) [ocirc, left]{} ;
\draw (C.out) |- (B.in 2);
\end{circuitikz}
```

As you can see, the `center` anchor is (for historic reasons) not in the center at all. You can fix this with the command `\ctikzset{logic ports origin=center}`:

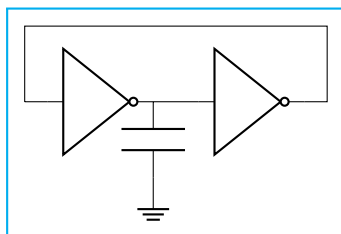


```
\begin{circuitikz}
\ctikzset{logic ports origin=center}
\draw (0,0) node[and port] (myand) {}
(myand.in 1) node[anchor=east] {1}
(myand.in 2) node[anchor=east] {2}
(myand.out) node[anchor=west] {3};
\draw[<-] (myand.center) -- ++(1,-1)
node{center};
\end{circuitikz}
```



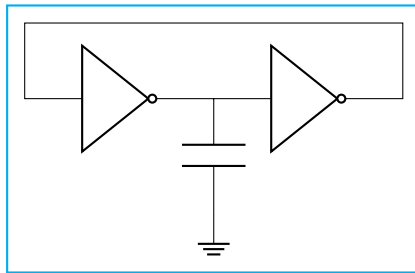
```
\begin{circuitikz} \draw
(0,2) node[and port] (myand1) {}
(0,0) node[and port] (myand2) {}
(2,1) node[xnor port] (myxnor) {}
(myand1.out) -| (myxnor.in 1)
(myand2.out) -| (myxnor.in 2)
;\end{circuitikz}
```

In the case of NOT, there are only `in` and `out` (although for compatibility reasons `in 1` is still defined and equal to `in`):



```
\begin{circuitikz} \draw
(1,0) node[not port] (not1) {}
(3,0) node[not port] (not2) {}
(0,0) -- (not1.in)
(not2.in) -- (not1.out)
++(0,-1) node[ground] {} to[C] (not1.out)
(not2.out) -| (4,1) -| (0,0)
;\end{circuitikz}
```

This last circuit could be drawn also (and probably in a more natural manner) using the path-style components:

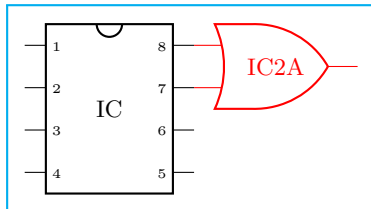


```
\begin{circuitikz}[american]
\draw (0,0) node[ground]{} to[C] ++(0,1.5)
coordinate(c)
to[inline not] ++(2.5,0) -- ++(0,1)
-| ++(-5,-1)
to[inline not] (c);
\end{circuitikz}
```

4.22.6 IEEE logic gates usage.

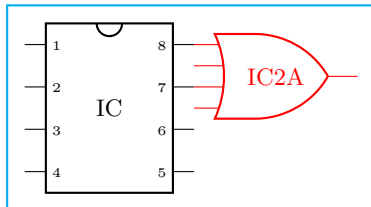
The rest of this section will assume you have issued the command `\ctikzset{logic ports=ieee}`, so that the short form of the names is used.

IEEE standard logic gates have a basic difference with the legacy ones: the proportions of their shapes do not change when you change the size, so you can't have a "tall" port or a "squatty" one. The two-inputs gates, by default, have their default size designed so that they match the chips component (see 4.25).



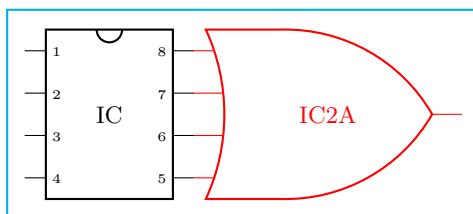
```
\begin{circuitikz}
\draw (0,0) node[dipchip](C){IC} (C.pin 8)
node[or port, anchor=in 1,
color=red] (A){IC2A};
\end{circuitikz}
```

If you need, say, a 4-inputs port, the port will look like this:



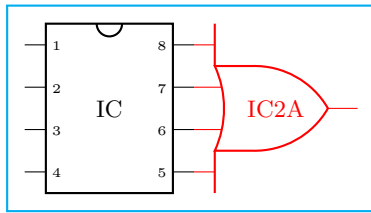
```
\begin{circuitikz}
\draw (0,0) node[dipchip](C){IC} (C.pin 8)
node[or port, anchor=in 1, number inputs=4,
color=red] (A){IC2A};
\end{circuitikz}
```

...and in this case it is clear that it does not match. With standard ports, there are two possibilities. The first one is to scale the port; if you set the port height so that it has the same size (see "IEEE logic gates customization" below for details) as the number of ports, they will match again.



```
\begin{circuitikz}
\draw (0,0) node[dipchip](C){IC} (C.pin 8)
node[or port, anchor=in 1,
number inputs=4,
circuitikz/ieeestd ports/height=4,
color=red] (A){IC2A};
\end{circuitikz}
```

But then the size of the port is quite "unusual". The solution in technical literature is to use what we can call a "rack" for the inputs; basically, only a certain number of pins are kept on the port, and the others are put on an extended input line.

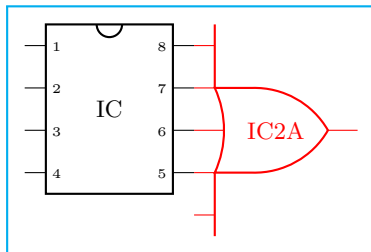


```
\begin{circuitikz}
  \draw (0,0) node[dipchip](C){IC} (C.pin 8)
    node[or port, anchor=in 1,
      number inputs=4,
      inner inputs=2,
      color=red](A){IC2A};
\end{circuitikz}
```

When using the `inner inputs` key, keep in mind the rule of thumbs:

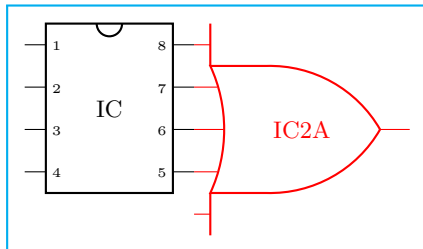
- the distance between the pins is matched with the chip ones when the `inner inputs` match the `/ieeestd ports/height` key;
- when the number of pins in the rack is odd, the result is often quite ugly, so try to avoid it.

For example, look at the following example; given that we are asking an odd number of pins on the rack, some of the inputs are drawn on the port's border, resulting in a less-than-ideal diagram.



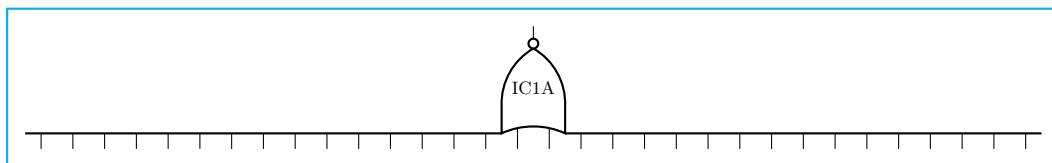
```
\begin{circuitikz}
  \draw (0,0) node[dipchip](C){IC} (C.pin 8)
    node[or port, anchor=in 1,
      number inputs=5,
      inner inputs=2,
      color=red](A){IC2A};
\end{circuitikz}
```

In this case, if you don't like the solution, the better approach is to let the gate grow a bit.



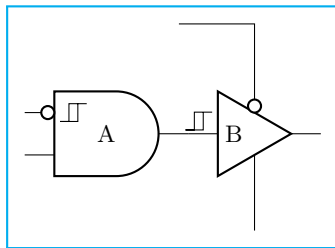
```
\begin{circuitikz}
  \draw (0,0) node[dipchip](C){IC} (C.pin 8)
    node[or port, anchor=in 1,
      number inputs=5,
      inner inputs=3,
      circuitikz/ieeestd ports/height=3,
      color=red](A){IC2A};
\end{circuitikz}
```

The good thing about the rack mechanism is that you can have quite big ports without problems.



```
\begin{circuitikz}[scale=0.75, transform shape]
  \draw node[nor port, number inputs=32, inner inputs=2,
    rotate=90](A){\rotatebox{-90}{IC1A}};
\end{circuitikz}
```

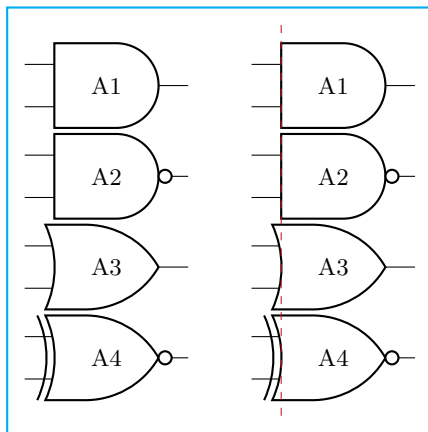
You can use the additional elements (the `notcirc` and the `schmitt` symbol to obtain circuits like the following ones (well, a bit of a mix of conventions, but...):



```
\begin{circuitikz}
\draw (0,0) node[and port] (A){A} (A.out)
node[buffer port, anchor=in,
component text=left] (B){B} (B.bin)
node[schmitt symbol, above left]{}
(A.bin 1) node[schmitt symbol, right]{};
\node [notcirc, left] at (A.bin 1) {};
\node [notcirc, above] (C) at (B.up) {};
\draw (C.north) |- ++(-1,1) (B.down) --++(0,-1);
\end{circuitikz}
```

Notice the key `component text=left` that moves the label near to the left border of the component. There is also a `\ctikzset{component text=left}` if you prefer to have it as a default for all the IEEE ports.¹²⁵

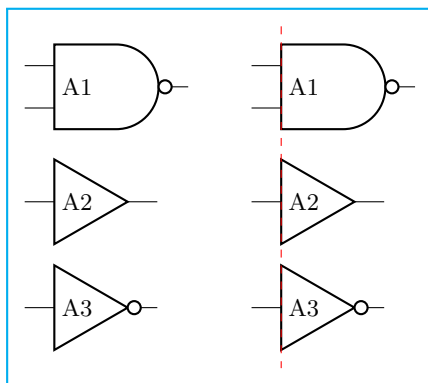
4.22.6.1 Stacking and aligning IEEE standard gates. The standard gates are designed so that they stack up nicely when positioned using the external leads as anchors. Notice that the ports **do** have different sizes, but the leads lengths are designed to counter the differences.



```
\begin{circuitikz}
\draw
(0,0) node[and port, anchor=in 1]{A1}
(0,-1.2) node[nand port, anchor=in 1]{A2}
(0,-2.4) node[or port, anchor=in 1]{A3}
(0,-3.6) node[xnor port, anchor=in 1]{A4};
\draw
(3,0) node[and port, anchor=in 1] (A1){A1}
(3,-1.2) node[nand port, anchor=in 1]{A2}
(3,-2.4) node[or port, anchor=in 1]{A3}
(3,-3.6) node[xnor port, anchor=in 1] (A4){A4};
\draw[red, dashed] ([yshift=0.8cm]A1.body left)
-- ([yshift=-0.8cm]A4.body left);
\end{circuitikz}
```

The length of the external leads can be changed by the user, but notice that if you use a too small value you can jeopardize that property.

The single input ports (`not port`, `buffer port` and their Schmitt equivalent) are smaller than the six standard ports, so they are not kept aligned by default; they just have the same distance at the input side. For the not ports, the `left` position of the text results often in a better look (the centered text in the triangle seems to be much more at the right).



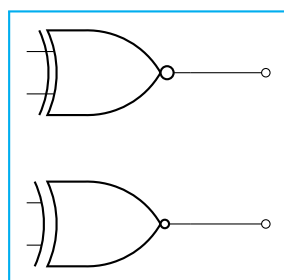
```
\begin{circuitikz}
\ctikzset{component text=left}
\draw (0,0) node[nand port, anchor=in 1]{A1}
(0,-1.8) node[buffer port, anchor=in 1]{A2}
(0,-3.2) node[not port, anchor=in 1]{A3};
\draw (3,0) node[nand port, anchor=in 1] (A1){A1}
(3,-1.8) node[buffer port, anchor=in 1]{A2}
(3,-3.2) node[not port, anchor=in 1] (A3){A3};
\draw[red, dashed] ([yshift=0.8cm]A1.body left)
-- ([yshift=-0.8cm]A3.body left);
\end{circuitikz}
```

¹²⁵You can use the same key with amplifiers, too.

4.22.6.2 IEEE standard ports customization There are several parameters that can be used to customize the IEEE standard ports, although less than the ones in the legacy american ones — the basic shape is set to follow the IEEE recommendation. The basic parameters are shown in the following table, and they can be set via `\ctikzset{ieeestd ports/...}`

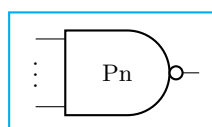
key	default	description
<code>baselen</code>	0.4	the basic length for every dimension, as a fraction of the (scaled) resistor length
<code>height</code>	2	the height of the port, in term of <code>baselen</code> . Pin distance is given by this parameter divided by the inner pins.
<code>pin length</code>	0.7	length of the external pin leads that are drawn with the port. This length is always calculated starting from the inner body of the shape.
<code>not radius</code>	0.154	radius of the “not circle” added to the negated-output ports. The default value is the IEEE recommended one.
<code>xor bar distance</code>	0.192	distance of the detached input shape in <code>xor</code> and <code>xnor</code> ports. The default value is the IEEE recommended one.
<code>xor leads in</code>	1	If set to 0, there will be no leads drawn between the detached input line and the body in the <code>xor</code> and <code>xnor</code> ports. IEEE recommends 1 here.
<code>schmitt symbol size</code>	0.3	Size of the small Schmitt symbol to use near input leads.

For example, using a `not radius` of 0.1 will give a “not ball” of the same size of a connecting pole, as it is in the legacy ports.



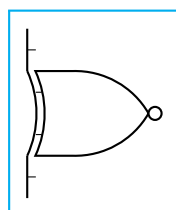
```
\begin{circuitikz}
\draw (0,2) node[xnor port](P){}
(P.out) to[short, -o] ++(1,0);
\ctikzset{ieeestd ports/.cd, not radius=0.1,
xor bar distance=0.3, xor leads in=0}
\draw (0,0) node[xnor port](P){}
(P.out) to[short, -o] ++(1,0);
\end{circuitikz}
```

In addition to the specific parameters, you can also apply to these ports the boolean style `no input leads` as in legacy ones (this simply *does not draw* the input leads, but the anchors stays where they should):



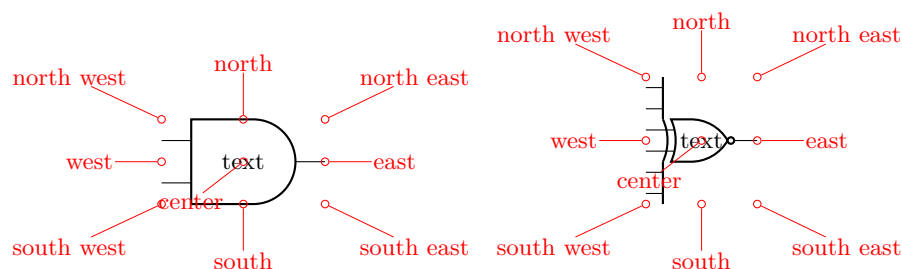
```
\begin{circuitikz}
\draw (0,0) node[nand port,
number inputs=5, no input leads,](B){Pn};
\draw (B.in 1) -- (B.bin 1) (B.in 5) -- (B.bin 5);
\node[rotate=90] at (B.in 3) {\dots};
\end{circuitikz}
```

Changing the leads length must be done with a bit of care, because if the length is shorter than the port left or right extrusions strange things can happen (yes, a 4-inputs xnor gates is not so well defined... but it's a nice example to show):

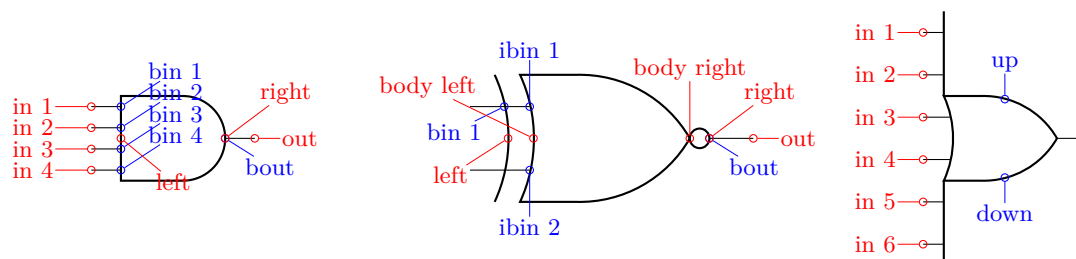


```
\begin{circuitikz}
\ctikzset{ieeestd ports/pin length=0.2}
\draw (0,0) node[xnor port,
number inputs=4, inner inputs=2](B){};
\end{circuitikz}
```

4.22.6.3 IEEE standard ports anchors Geographical anchors define the rectangular space that the port is using, included the leads if presents.



Most of the anchors can be seen in the following diagram:



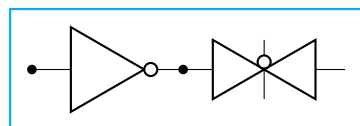
The inputs anchor are *in number* (on the tip of the lead) and *bin number* (border inputs) on the component's border (useful if you draw the ports with **no inut leads**). Additionally, you have *ibin number* (inner border inputs) for the *x*-type ports. The anchor named *left* is where a central border input would be.

In one-input ports (**not port**, the buffer, and Schmitt-type ports) you can use plain *in* or *in 1* indifferently. On the output, *out* is on the tip of the lead, and *bout* on the rightmost border (so, if there is a negation circle, it is on it); *right* is the same as *bout*.

The main body of the port is marked with *body left* and *body right* anchors (as seen in the middle port in the diagram above); you also have an *up* and *down* anchors centered on the body (you can use them as enable signals or similar things).

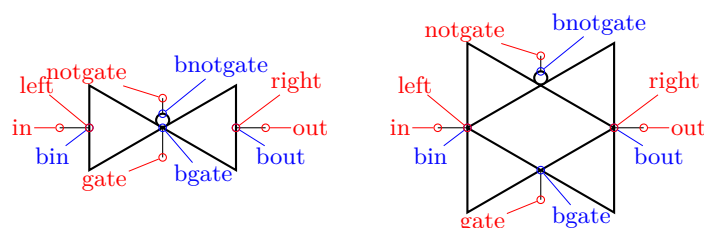
Finally, the internal *notcirc* node used for the output negation is accessible with the name *nodename-not*, where *nodename* is the name given to the logic port node.

4.22.6.4 Transmission gate symbols. The *tgate* and *double tgate* components are available since 1.2.4 but only in the IEEE style. An additional parameter *tgate scale* (default 0.7; if you set this to 1 the triangles will have the same size as a *ieeestd buffer port*) select the relative scale of the components.



```
\begin{circuitikz}
\ctikzset{logic ports=ieee}
\draw (0,0) to[inline not, **] ++(2,0)
node[tgate, anchor=in]{};
\end{circuitikz}
```

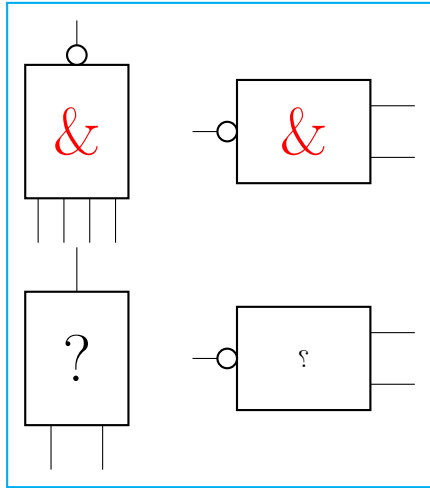
The anchors for the *tgate*'s control point are called *gate* and *notgate* (and the corresponding *bgate* and *bnotgate* for the border anchors).



4.22.7 European logic port usage

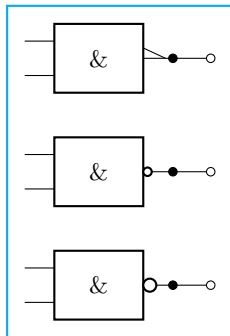
European logic port are in the same class as american and IEEE-style ones, and they obey the same class modifier. Moreover, you can use the `no inputs pin` as in the other logic ports to suppress input pins.

The standard text inside the port does not rotate (not flip) with the component¹²⁶, but you can change the font (and color and so on) with the key `european ports font` (default nothing, which means it uses the standard font and color). For more complex customization, you can use the two “blank” European ports, and add the text you want on them.



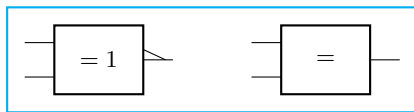
```
\begin{tikzpicture}
\ctikzset{
  logic ports=european,
  logic ports origin=center,
  logic ports/scale=1.5,
  tripoles/european not symbol=ieee circle,
}
% Draw the Nand with big AND symbol
\ctikzset{european ports font=\Huge\color{red}}
\draw(0,0) node [nand port, rotate=90,
  number inputs=4]{};
\draw(3,0) node [nand port, xscale=-1]{};
% the un-rotation is not automatic for node text!
\draw(0,-3) node [blank port, rotate=90,
  ]{\rotatexbox{-90}{\Huge ?}};
\draw(3,-3) node [blank not port, xscale=-1]{};
\end{tikzpicture}
```

4.22.7.1 European logic port customization Normally the European-style logic port with inverted output are marked with a small triangle; you can change it with the key `tripoles/european not symbol`; its default is `triangle` but you can set it to `circle` like in the following example. As you can see, the circle size is the same as the circuit poles; if you prefer the size used in the IEEE standard ports, you can use set it to `ieee circle`.



```
\begin{circuitikz}[european]
\draw (0,3) node[nand port](A){}
  (A.out) to[short, *-o] ++(0.5,0);
\ctikzset{tripoles/european not symbol=circle}
\draw (0,1.5) node[nand port](A){}
  (A.out) to[short, *-o] ++(0.5,0);
\ctikzset{tripoles/european not symbol=ieee circle}
\draw (0,0) node[european nand port](A){}
  (A.out) to[short, *-o] ++(0.5,0);
\end{circuitikz}
```

In some standard, the `xnor` port is different — without the negation at the end and with just an `=` sign.¹²⁷ You can switch to this if you like, with the key `european xnor style` that can be `default` or `direct`.

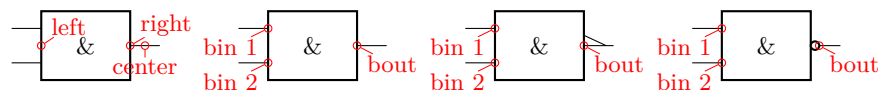


```
\begin{circuitikz}[european]
\draw (0,0) node[xnor port]{};
\ctikzset{european xnor style=direct}
\draw (3,0) node[xnor port]{};
\end{circuitikz}
```

¹²⁶since 1.6.4, thanks to a suggestion by [user @sputeanus on GitHub](#).

¹²⁷Suggested by user [Schlepptop on GitHub](#).

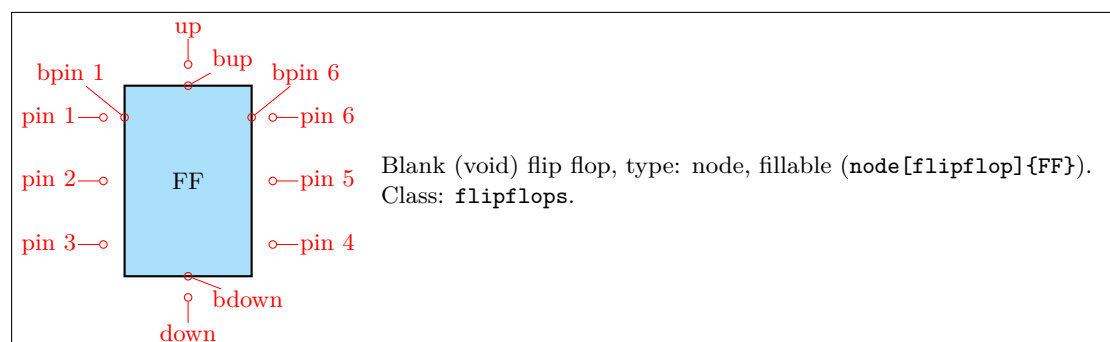
4.22.7.2 European logic port anchors The anchors are basically the same as in the american-style ports.



4.23 Flip-flops

Flip-flops (available since version 1.0.0) are an hybrid between the logic ports and the chips. They have a class by themselves (**flipflops**) but the default parameters are set at the same values as the logic gates one.

The default flip flop is empty: it is just a rectangular box like a blank **dipchip** with 6 pins.



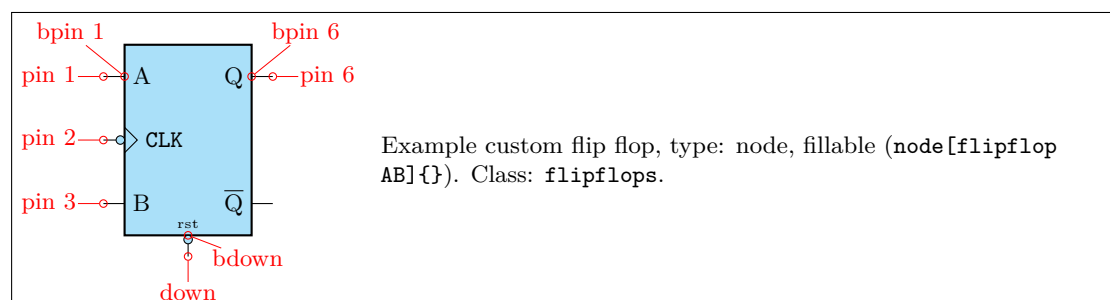
As you can see, in a void flip flop no external pins are drawn: you have to define the meaning of each of them to see them. To define a specific flip-flop, you have to set a series of keys under the `\ctikzset` directory `multipoles/flipflop/`, corresponding to pins 1...6, u for “up” and d for “down”:

- a *text* value `t0`, `t1`, ... `t6`, and `tu` and `td` (the last ones for up and down) which will set a label on the pin;
- a *clock wedge* flag (`c0`, ... `c6`, `cu`, `cd`), with value 0 or 1, which will draw a triangle shape on the border of the corresponding pin;
- a *negation* flag (`n0`, ... `n6`, `nu`, `nd`), with value 0 or 1, which will put an `ocirc` shape on the outer border of the corresponding pin.

To set all these keys, an auxiliary style `flipflop def` is defined, so that you can do the following thing:

```
\tikzset{flipflop AB/.style={flipflop,
  flipflop def={t1=A, t3=B, t6=Q, t4={\ctikztextnot{Q}},
    td=rst, nd=1, c2=1, n2=1, t2={\texttt{CLK}}},
}}
```

to obtain:



`\ctikztextnot{}` is a small utility macro to set a overbar to a text, like $\overline{\text{RST}}$ (created by the command `\ctikztextnot{RST}`).

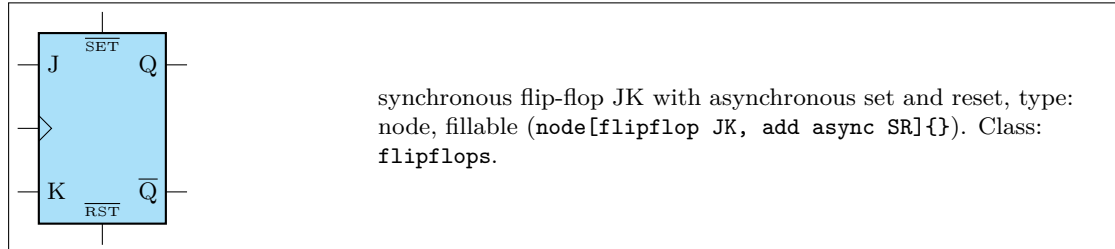
By default, the following flip-flops are defined, as well as a support shape for the clock wedge:

	D-type latch, type: node, fillable (<code>node[latch]{}</code>). Class: <code>flipflops</code>.
	flip-flop SR, type: node, fillable (<code>node[flipflop SR]{}</code>). Class: <code>flipflops</code>.
	Edge-triggered synchronous flip-flop D, type: node, fillable (<code>node[flipflop D]{}</code>). Class: <code>flipflops</code>.
	Edge-triggered synchronous flip-flop T, type: node, fillable (<code>node[flipflop T]{}</code>). Class: <code>flipflops</code>.
	Edge-triggered synchronous flip-flop JK, type: node, fillable (<code>node[flipflop JK]{}</code>). Class: <code>flipflops</code>.
	clock wedge shape, type: node (<code>node[clockwedge]{text}</code>). Class: <code>flipflops</code>.

If you prefer that the negated output is labelled $\overline{Q}$ and a dot indicating negation is shown, you can add the `dot on notQ` key:

	synchronous flip-flop JK with asynchronous set and reset, type: node, fillable (<code>node[flipflop JK, dot on notQ]{}</code>). Class: <code>flipflops</code>.
--	---

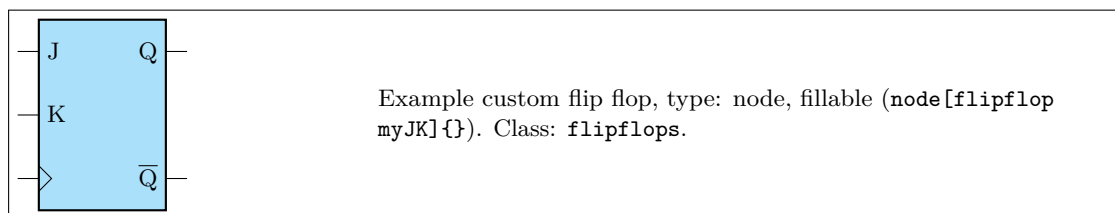
You can also add “vertical” asynchronous set and reset (active low) adding the style `add async SR` to all of them:



4.23.1 Custom flip-flops

If you like different pin distributions, you can easily define different flip-flops to your taste. For example, somebody likes the clock pin on the bottom pin:

```
\tikzset{flipflop myJK/.style={flipflop,
    flipflop def={t1=J, t2=K, t6=Q, t4={\ctikztextnot{Q}}, c3=1}}
}
```

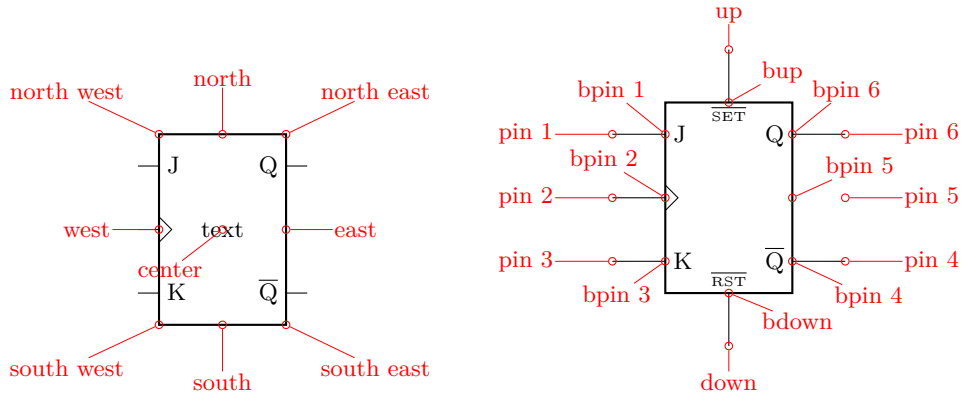


The standard definition of the default flip-flops are the following (in the file `pgfcircmultipoles.tex`):

```
\tikzset{
  % async
  latch/.style={flipflop, flipflop def={t1=D, t6=Q, t3=CLK, t4={\ctikztextnot{Q}}}},
  flipflop SR/.style={flipflop, flipflop def={t1=S, t3=R, t6=Q, t4={\ctikztextnot{Q}}}},
  % sync
  flipflop D/.style={flipflop, flipflop def={t1=D, t6=Q, c3=1, t4={\ctikztextnot{Q}}}},
  flipflop T/.style={flipflop, flipflop def={t1=T, t6=Q, c3=1, t4={\ctikztextnot{Q}}}},
  flipflop JK/.style={flipflop,
    flipflop def={t1=J, t3=K, c2=1, t6=Q, t4={\ctikztextnot{Q}}}},
  % additional features
  add async SR/.style={flipflop def={%
    tu={\ctikztextnot{SET}}, td={\ctikztextnot{RST}}}},
  dot on notQ/.style={flipflop def={t4={Q}, n4=1}},
}
```

4.23.2 Flip-flops anchors

Flip-flops have all the standard geometrical anchors, although it should be noticed that the external pins are *outside* them. The pins are accessed by the number 1 to 6 for the lateral ones (like in DIP chips), and with the `up` and `down` anchors for the top and bottom one. All the pins have the “border” variant (add a `b` in front of them, no spaces).



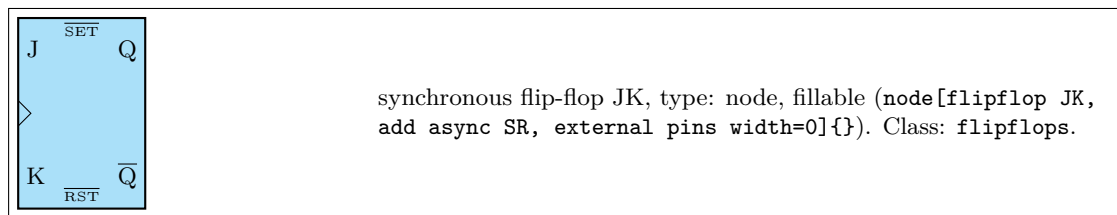
If you have negated pins, you can access the `ocirc` shapes with the name as `<nodename>-N<pin number>`, and all the respective anchors (for example — `myFFnode-N4.west`).

4.23.3 Flip-flops customization

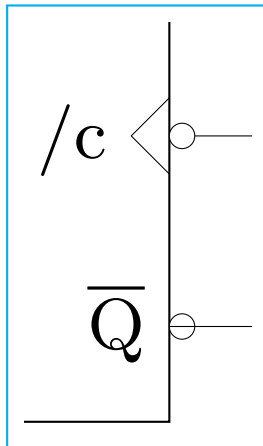
Flip-flop’s size is controlled by the class parameters (like `flipflops/scale`) and the specific `\ctikzset` keys `multipoles/flipflop/width` and `multipoles/flipflop/pin spacing`. Class parameters are also used for line thickness and fill color. The default values are matched with the logic ports ones.

The fonts used for the pins 1...6 is set by the key `multipoles/flipflop/font` (by default `\small` in $\text{\LaTeX}$ and the equivalent in other formats) and the font used for pins `u` and `d` is `multipoles/flipflop/fontud` (`\tiny` by default). You can change it globally or specifically for each flip flop.

As in chips, you can change the length of the external pin with the key `external pins width`; you can for example have a pinless flip-flop like this:

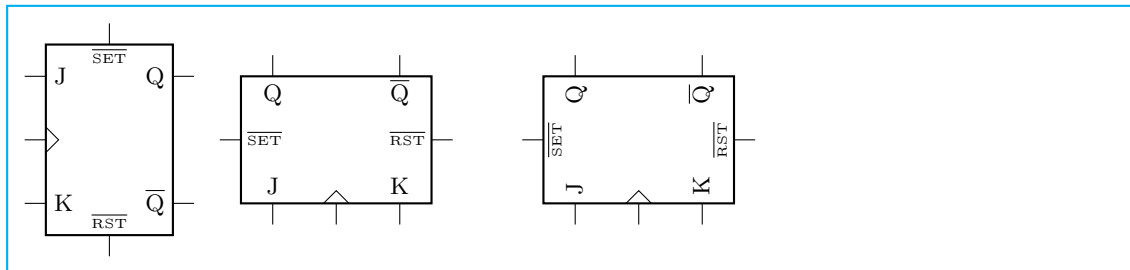


Notice however that negated pins when the pins width is zero has to be handled with care. As explained in the poles sections, the `ocirc` shape is drawn at the end of the shape to cancel out the wires below; so if you use a pinless flipflop when you make the connection you should take care of connecting the symbol correctly. To this end, the shapes of the negation circles are made available as `<nodename>-N<pin number>`, as you can see in the next (contrived) example.



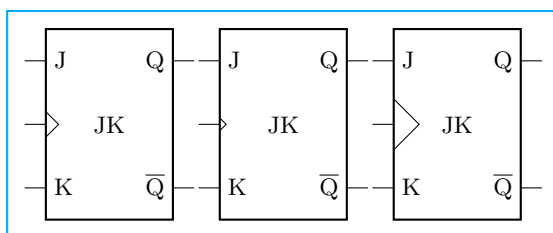
```
\begin{circuitikz}[scale=3, transform shape]
  \clip (0.2,0.5) rectangle (1.2,-1.3);
  \node [flipflop JK,
    flipflop def={n5=1,n4=1,t5={/c},c5=1},
    external pins width=0,
  ](A){};
  \draw (A-N5.east) -- ++(1,0); % correct
  \draw (A.pin 4) -- ++(1,0);  % wrong
\end{circuitikz}
```

Normally the symbols on the flip-flop are un-rotated when you rotate the symbol, but as in case of chips, you can avoid it.



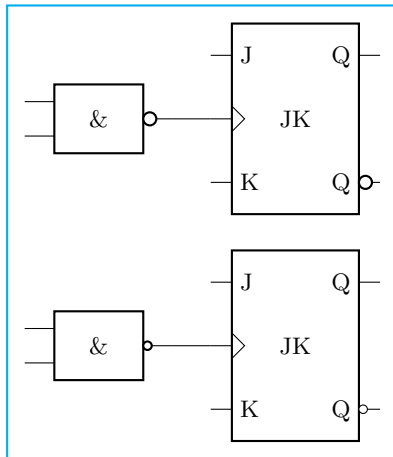
```
\begin{tikzpicture}
  \draw (0,0) node[flipflop JK, add async SR]{};
  \draw (3,0) node[flipflop JK, add async SR, rotate=90]{};
  \draw (7,0) node[flipflop JK, add async SR, rotate=90, rotated numbers]{};
\end{tikzpicture}
```

You can also change the size of the wedge, with the key `multipoles/flipflop/clock wedge size` (default value 0.2).



```
\begin{circuitikz}[]
  \draw (0,0) node[flipflop JK]{JK};
  \ctikzset{multipoles/flipflop/clock
    wedge size=0.1}
  \draw (2.3,0) node[flipflop JK]{JK};
  \ctikzset{multipoles/flipflop/clock
    wedge size=0.4}
  \draw (4.6,0) node[flipflop JK]{JK};
\end{circuitikz}
```

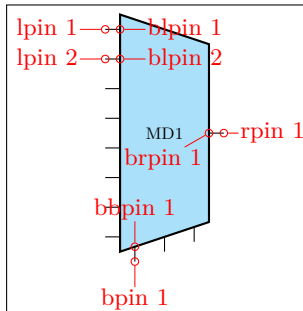
Flip-flops “not circles” follows the current logic port setting (either if you choose `ieee ports`, or if you are using `european ports` with `european not symbol` set to `circle` or `ieee circle`).



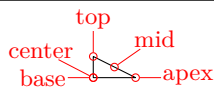
```
\begin{circuitikz}[]
\ctikzset{logic ports=european,
tripoles/european not symbol=ieee circle}
\draw (0,0) node[nand port](A){}
(A.out) to[short] ++(0.5,0)
node[flipflop JK, dot on notQ, anchor=pin 2]{JK};
\ctikzset{logic ports=european,
tripoles/european not symbol=circle}
\draw (0,-3) node[nand port](A){}
(A.out) to[short] ++(0.5,0)
node[flipflop JK, dot on notQ, anchor=pin 2]{JK};
\end{circuitikz}
```

4.24 Multiplexer and de-multiplexer

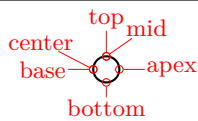
The shape used for muxes and de-muxes is probably the most configurable shape of the package; it has been added by Romano in v1.0.0. The basic shape is a multiplexer with 8 input pin, one output pin, and three control pins ($2^3 \rightarrow 1$ multiplexer). The pins are not named as input or output pins (see below for a full description for anchors) for reasons that will be clear later.



mux-demux, type: node, fillable (`node[muxdemux]{MD1}`). Class: `muxdemuxes`.



Inversion marker for European logic symbols¹²⁸, type: node (`node[wedgeinv, scale=2]{}`). Class: `muxdemuxes`.



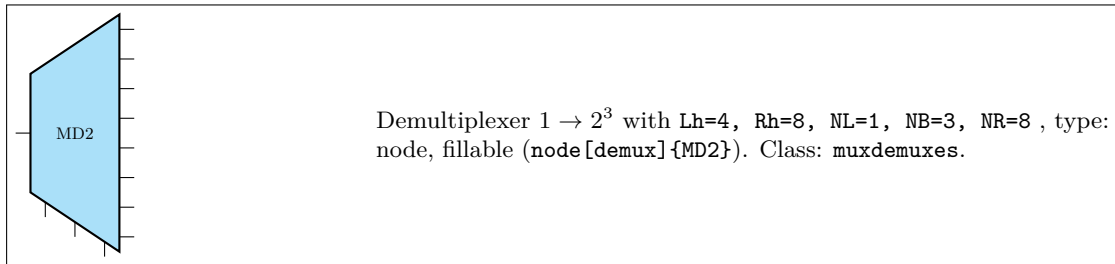
Circle-shaped inversion marker, type: node (`node[circleinv, scale=2]{}`). Class: `muxdemuxes`.

You can define a custom shape for the `muxdemuxes` using an interface similar to the one used in flip-flops; for example:

```
\tikzset{demux/.style={muxdemux, muxdemux def={Lh=4, Rh=8, NL=1, NB=3, NR=8}}}
```

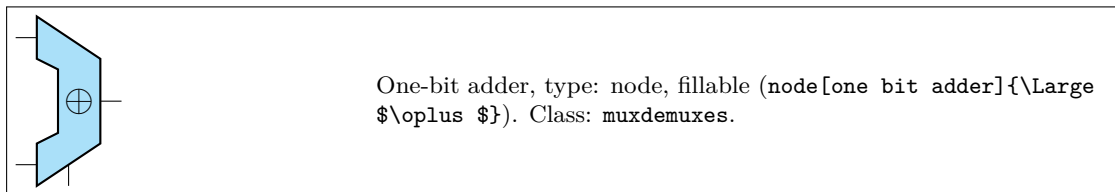
will generate the following shape (the definition above is already defined in the package):

¹²⁸Thanks for the contribution by [yashpalgoyal1304](#) on GitHub.



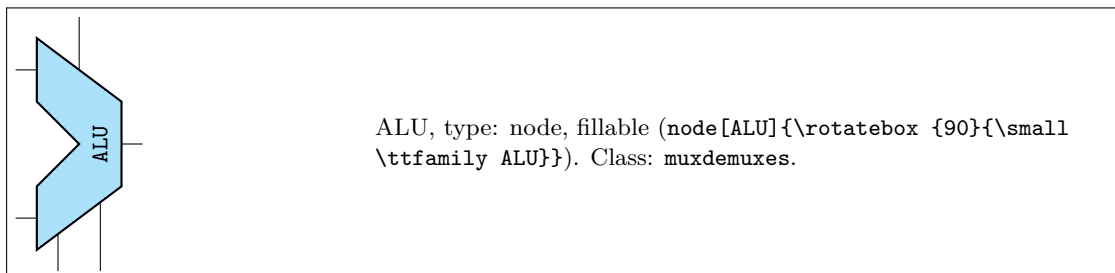
The shape can also be defined with an inset. For example it can be used like this to define a 1-bit adder (also already available):

```
\tikzset{one bit adder/.style={muxdemux,
  muxdemux def={Lh=4, NL=2, Rh=2, NR=1, NB=1, w=1.5,
  inset w=0.5, inset Lh=2, inset Rh=1.5}}}
```



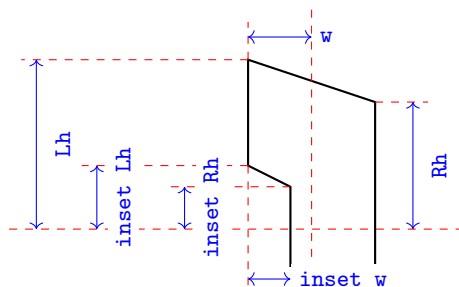
Or a Arithmetic Logic Unit (again, already defined by default):

```
\tikzset{ALU/.style={muxdemux,
  muxdemux def={Lh=5, NL=2, Rh=2, NR=1, NB=2, NT=1, w=2,
  inset w=1, inset Lh=2, inset Rh=0, square pins=1}}}
```



4.24.1 Mux-Demux: design your own shape

In designing the shape there are several parameters to be taken into account. In the diagram on the right they are shown in a (hopefully) practical way. The parameter can be set in a node or in a style using the `muxdemux def` key as shown above, or set with `\ctikzset` as `multipoles/muxdemux/Lh` keys and so on.



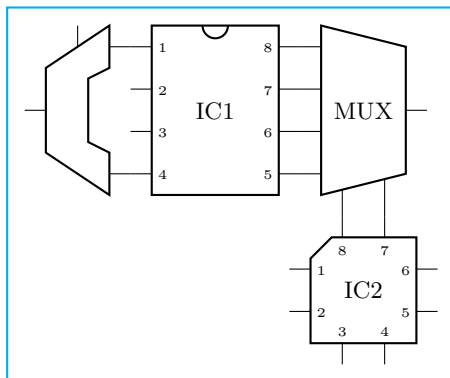
The default values are $L_h = 8$, $R_h = 6$, $w = 3$ and no inset: `inset Lh = inset Rh = inset w = 0`. In addition, you can set the following parameters:

NL, NR, NB, NT : number of pins relatively on the left, right, bottom and top side (default 8, 1, 3, 0).

When an inset is active (in other words, when $L_h > 0$) the pins are positioned on the top and bottom part, not in the inset; the exception is when the number of left pins is odd, in which case you have one pin set on the center of the inset. If you do not want a pin in one side, use 0 as number of pins.

square pins : set to 0 (default) if you want the square pins to stick out following the slope of the bottom or top side, 1 if you want them to stick out in a square way (see the example above for the ALU).

All the distances are multiple of `multipoles/muxdemux/base len` (default 0.4, to be set with `\ctikzset`), which is relative to the basic length. That value has been chosen so that, if you have a number of pins which is equal to the effective distance where they are spread (which is L_h without inset, $L_h - (\text{inset } L_h)$ with an inset), then the distance is the same as the default pin distance in chips, as shown in the next circuit. In the same drawing you can see the effect of `square pins` parameters (without it, the rightmost bottom lead of the `mux 4by2` shape will not connect with the below one).



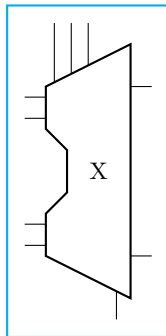
```
\begin{circuitikz}
  \tikzset{mux 4by2/.style={muxdemux,
    muxdemux def={Lh=4, NL=4, Rh=3,
      NB=2, w=2, square pins=1}}}
  \node [dipchip, num pins=8] (A) at (0,0) {IC1};
  \node [one bit adder, scale=-1, anchor=lpin 2]
    at (A.pin 1){};
  \node [mux 4by2, anchor=lpin 1] (B)
    at (A.pin 8){MUX};
  \node [qfpchip, num pins=8, anchor=pin 8] at
    (B.bpin 1) {IC2};
\end{circuitikz}
```

4.24.2 Mux-Demux customization

Mux-demuxes have the normal parameters of their class (`muxdemuxes`): you can scale them with the `\ctikzset` key `muxdemuxes/scale`, control the border thickness with `muxdemuxes/thickness` and the default fill color with `muxdemuxes/fill` — they are set, by default, at the same values than `logic ports`.

External pins' length is controlled by the key `multipoles/external pins width` (default 0.2) or by the style `external pins width`. The parameter `multipoles/external pins thickness` is also respected. like in chips. In addition, like in logic ports, you can suppress the drawing of the leads by using the boolean key `logic ports draw input leads` (default `true`) or, locally, with the style `no input leads` (that can be reverted with `input leads`). The main difference between setting `external pins width` to 0 or using `no input lead` is that in the first case the normal pin anchors and the border anchors will coincide, and in the second case they will not move and stay where they should have been if the leads were drawn.

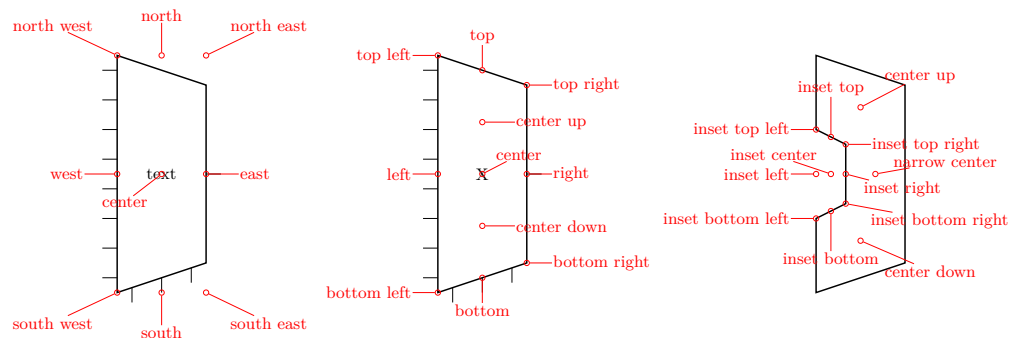
You can draw only selected pins and leave out the rest by setting the keys `multipoles/draw only side pins` and the corresponding style `draw only side pins` where `side` can be `left`, `right`, `top`, `bottom`. Those key accept a comma-separated list of pin numbers or ranges of pin numbers (a range is given as `<start> - <end>`, ends are inclusive). The numbers will not be expanded in any way, except those given as ends of ranges. A special value (and the initial one) is `all`, in which case all pins are drawn. The anchors will be adjusted, such that each `xpin n` will be placed at the end of the pins which are drawn, and coincide with the `bpin n` anchors for the suppressed pins.



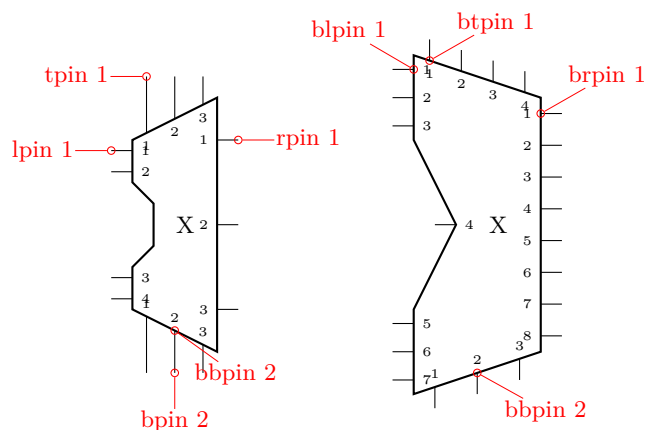
```
\begin{circuitikz}
\node [muxdemux, muxdemux def={NL=4, NR=3, NT=5, NB=3, w=2,
inset w=0.5, Lh=4, inset Lh=2.0, inset Rh=1.0,
square pins=1},
draw only right pins={1,3},
draw only top pins={1-3},
draw only bottom pins={3}](C) at (0,0) {X};
\end{circuitikz}
```

4.24.3 Mux-Demux anchors

Mux-demuxes have a plethora of anchors. As in the case of chips, the geographic anchors mark the rectangle occupied by the component, without taking into account the pin leads.



The pins anchors are named **lpin**, **rpin**, **bpin** and **tpin** for the left, right, bottom and top pin respectively, and points to the “external” pin. The border pins are named the same, with a **b** added in front: **blpin**, **brpin**, **bbpin** and **btpin**. The following graph will show the numbering and position of the pin anchors.



The code that implemented the printing of the numbers (which in **muxdemuxes**, differently from chips, are never printed automatically) in the last graph is the following one.

```
\begin{circuitikz}
\node [muxdemux, muxdemux def={NL=4, NR=3, NT=3, NB=3, w=2, inset w=0.5,
Lh=4, inset Lh=2.0, inset Rh=1.0, square pins=1}](C) at (0,0) {X};
\node [muxdemux, muxdemux def={NL=7, NR=8, NT=4, inset w=1.0,
inset Lh=4.0, inset Rh=0.0}](D) at (4,0) {X};
\foreach \myn/\NL/\NR/\NB/\NT in {C/4/3/3/3,D/7/8/3/4} {
```

```

\foreach \myp in {1,...,\NL} \node[right, font=\tiny] at (\myn.blpin \myp){\myp};
\foreach \myp in {1,...,\NR} \node[left, font=\tiny] at (\myn.brpin \myp) {\myp};
\foreach \myp in {1,...,\NB} \node[above, font=\tiny] at (\myn.bbpin \myp){\myp};
\foreach \myp in {1,...,\NT} \node[below, font=\tiny] at (\myn.btpin \myp){\myp};
}
\end{circuitikz}

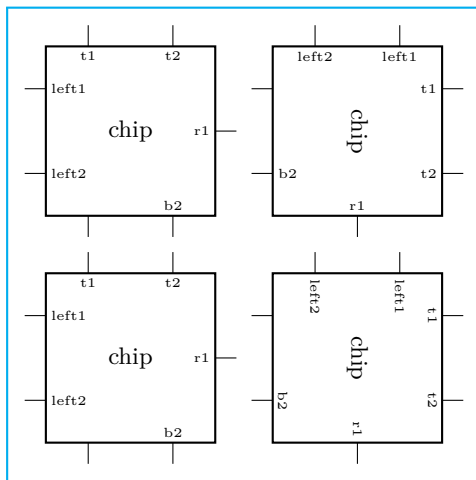
```

4.24.4 Adding labels to the pins

In `muxdemuxes`, there is no automatic labelling of pins with numbers as in chips; there is no simple standard enumeration possible. But since **v1.6.5** it is possible to associate a name to the pins that will be printed with the shape; that names are called *pin labels*. Pin labels are counter-rotated by default if the shape is rotated, as in chip pin numbers (see 4.25.3), but you can avoid it with the key `rotated numbers` (the default is `straight numbers`). Additionally, you can set also *border labels* on the four borders (more or less, see later); that are normally not counter-rotated *unless* the result would be upside-down (or if you use `straight numbers`, of course), and clock and negation symbols.

All of these labels and symbols are added by specifying them in a `muxdemux label={...}` clause. Notice that the key specified there are not checked for validity; if you misspell any of that, it will be simply ignored.

4.24.4.1 Inner pin labels will be printed in the inside of the shape, with the font specified in the `\ctikzset` key `muxdemux/inner label font` (default is `\tiny` in $\text{\LaTeX}$, other engine can have it different — better set it in case of doubt) and with a padding setting with the keys `muxdemux/inner label xsep` and `muxdemux/inner label ysep` (respectively for horizontal and vertical shifts; default for both 2pt). You can also use the key `muxdemux/inner label sep` to set both at the same time.

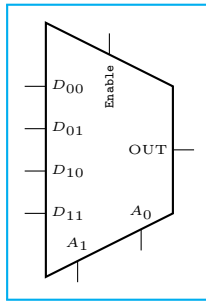


```

\begin{circuitikz}
\tikzset{myICw1/.style={muxdemux,
muxdemux def={Lh=4, Rh=4, w=4,
NR=1, NL=2, NB=2, NT=2,},
muxdemux label={L1=left1, L2=left2,
R1=r1, B2=b2, T1=t1, T2=t2},
}
}
\draw (0, 0) node[myICw1]{chip} ++(3,0)
node[myICw1, rotate=-90]{chip};
\draw (0, -3) node[myICw1]{chip} ++(3,0)
node[myICw1, rotate=-90,
rotated numbers]{chip};
\end{circuitikz}

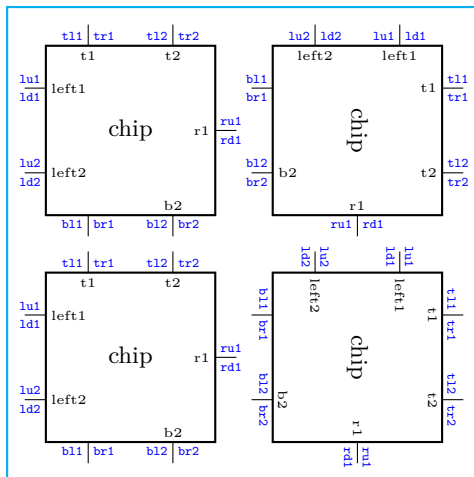
```

As you can see, the syntax is to add a `muxdemux label` to the specification; the labels are set using one of the letter L, R, B, and T for respectively left, right, bottom and top labels. You can define all the labels, none (which will give the default behavior of no-labels as it was before **v1.6.5**), or any number you wish. If you want some specific label rotated in a different way, you have to do it manually, as shown in the following example.



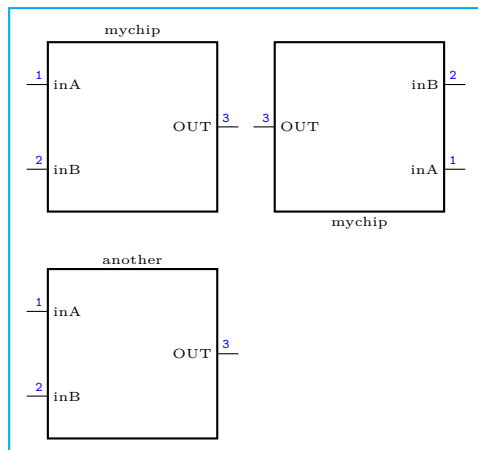
```
\begin{circuitikz}
\tikzset{mux 4by2 w1/.style={muxdemux,
  muxdemux def={Lh=6, NL=6, Rh=3, NB=2, w=3, NT=1},
  draw only left pins={2-5},
  muxdemux label={B1=$A_1$, B2=$A_0$, R1=OUT,
    L2=$D_{00}$, L3=$D_{01}$, L4=$D_{10}$, L5=$D_{11}$,
    T1=\rotatebox{90}{\texttt{Enable}}}},
  circuitikz/muxdemux/inner label ysep=4pt}}
\node [mux 4by2 w1]{};
\end{circuitikz}
```

4.24.4.2 Outer pin labels will be printed on the outside of the pin position — in the case of left and right pins, either above (“up”, identified by LU and RU labels), or below (“down”, LD and RD labels); in the case of top and bottom pin, either at the left (TL and BL) or at the right (TR and BR). The font is specified in the key `muxdemux/outer label font` (default `\tiny`) and the padding with the corresponding `muxdemux/outer label xsep` and `muxdemux/outer label ysep` (default for both 2pt), or `muxdemux/outer label sep` to set both at the same time.



```
\begin{circuitikz}
\ctikzset{muxdemux/outer label
  font={\tiny\ttfamily\color{blue}}}
\tikzset{myICw1/.style={muxdemux,
  muxdemux def={Lh=4, Rh=4, w=4,
    NR=1, NL=2, NB=2, NT=2},
  muxdemux label={L1=left1, L2=left2,
    R1=r1, B2=b2, T1=t1, T2=t2,
    LU1=lu1, LU2=lu2, LD1=ld1, LD2=ld2,
    BR1=br1, BL1=bl1, BR2=br2, BL2=bl2,
    RU1=ru1, RD1=rd1, TR2=tr2, TL2=tl2,
    TR1=tr1, TL1=tl1}},}
}
\draw (0, 0) node[myICw1]{chip} ++(3,0)
  node[myICw1, rotate=-90]{chip};
\draw (0, -3) node[myICw1]{chip} ++(3,0)
  node[myICw1, rotate=-90,
    rotated numbers]{chip};
\end{circuitikz}
```

4.24.4.3 Border labels are drawn *before* the other labels along the external border (to be exact: in north, south, east, and west position) of the component. You set them with the key N, S, W, and E for the outer position, and Ni, Si, Wi, and Ei for the inner ones. The font is specified in the key `muxdemux/border label font` (default `\tiny`) and the padding with the corresponding `muxdemux/border label xsep` and `muxdemux/border label ysep` (default for both 2pt), or `muxdemux/border label sep` to set both at the same time.



```
\begin{circuitikz}
\ctikzset{muxdemux/outer label
  font={\tiny\ttfamily\color{blue}}}
\tikzset{myICw1/.style={muxdemux,
  muxdemux def={Lh=4, Rh=4, w=4,
    NR=1, NL=2, NB=0, NT=0,},
  muxdemux label={L1=inA, L2=inB,
    R1=OUT, RU1=3, LU1=1, LU2=2,
    N=mychip},}
}
\draw (0, 0) node[myICw1]{} ++(3,0)
  node[myICw1, rotate=180]{};
\draw (0, -3) node[myICw1,
  muxdemux label={N=another}]{};
\end{circuitikz}
```

As you can see, you can locally change any label in a specific instance.

4.24.4.4 Clock and negation symbols are not exactly labels, but they can be added with the same mechanism. There are four symbols available:

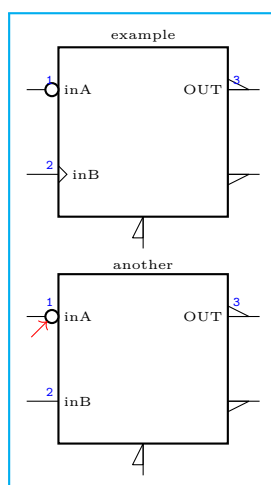
clock wedge: activated by the key `c` followed by the position (for example, `cL2` will set the clock wedge on the second left pin); its size can be changed with the key `muxdemux/clock wedge size` (default 0.2, relative to `muxdemux/base len`);

not ball: activated by the key `n` followed by the position (for example, `nR1` will set the negation circle on the first right pin); the type and shape of the ball will depend on the logic port negation in use (basically it will use a `circleinv` shape unless you are using european ports with the small `ocirc` negation symbol);

wedge in: a wedge negation *entering* the component (with the point on the border), activated by the key `wi` followed by the position;

wedge out: a wedge negation *exiting* the component (with the base on the border), activated by the key `wo` followed by the position.

The value of the key should be 0 or 1 for clocks and not circles; zero means “do not draw” and is used to override a previously specified element. In the case of wedge-style not, the -1 value flips the side of the triangle (see the following example).



```
\begin{circuitikz} \ctikzset{muxdemux/outer label
  font={\tiny\ttfamily\color{blue}}}
\ctikzset{logic ports=ieee, multipoles/external pins width=0.3}
\tikzset{myICw1/.style={muxdemux,
  muxdemux def={Lh=4, Rh=4, w=4, NR=2, NL=2, NB=1, NT=0,},
  muxdemux label={L1=inA, L2=inB, R1=OUT, RU1=3, LU1=1, LU2=2,
    N=example, nL1=1, woR1=1, woR2=-1, wiB1=1, cL2=1},}
}
\draw (0, 0) node[myICw1]{};
\draw (0, -3) node[myICw1,
  muxdemux label={N=another, LU1={1\strut},
    RU1={3\strut}, cL2=0}](A){};
\draw[red, <-] (A-nL1.-135) -- ++(-135:0.3);
\end{circuitikz}
```

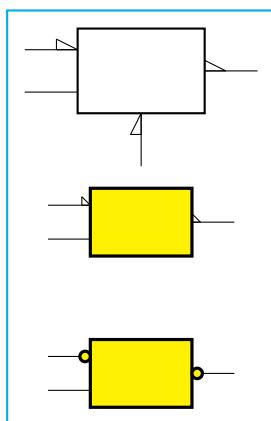
As you can see, the label position is not affected (with the exception of the clock wedge, that displaces the inner label), so you have to manually take care of not having overruns; you can change the `xsep` or `ysep` or, as shown in the example, modify the `heigh` and `depth` of the affected labels.

You have also to take care of the effect shown for the flip-flops in section 4.23.3, to avoid overrun the not circles when connecting wires. The “not” elements are named after the instance as `<nodename>-<activating key>` to give access to the border (show also in the example, although in bit of forced way...look at the red arrow).

4.24.5 Manually adding wedges or circular inversion markers

If the standard “internal” negation symbols are not sufficiently configurable for your application, you can add them manually. To add the wedge symbol, the `wedgeinv` shape will nicely do. It’ll scale with the `muxdemuxes` class, and the length and height can be changed with the keys `wedge inversion mark/width` (default 0.2) and `height` (default 0.1), with the same units that are used for the `external pins width` and similar keys.

Similarly, there is also a `circleinv` shape, which is basically the same as the `notcirc` (see 4.22.2) one, but that scales with the `muxdemuxes` class and that has the default anchor at its left, similarly to `wedgeinv`. This one will be filled if the class says so, contrary to the wedge-like shapes that are always open.

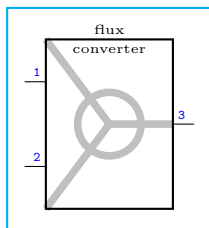


```
\begin{tikzpicture}[]
  \path (0,4) node[muxdemux, muxdemux def={NL=2,Lh=2,Rh=2,NB=1},
    external pins width = 0.5](mdemux){};
  \draw (mdemux.blpin 1) node[wedgeinv,anchor=apex]{};
  \draw (mdemux.brpin 1) node[wedgeinv]{};
  \draw (mdemux.bbpin 1) node[wedgeinv, anchor=apex,
    rotate=90]{};
  \ctikzset{wedge inversion mark/.cd, width=0.1}
  \ctikzset{muxdemuxes/.cd, fill=yellow, thickness=3, scale=0.8}

  \path (0,2) node[muxdemux, muxdemux def={NL=2,Lh=2,Rh=2,NB=0},
    external pins width = 0.5](mdemux){};
  \draw (mdemux.blpin 1) node[wedgeinv,anchor=apex]{};
  \draw (mdemux.brpin 1) node[wedgeinv]{};
  \path (0,0) node[muxdemux, muxdemux def={NL=2,Lh=2,Rh=2,NB=0},
    external pins width = 0.5](mdemux){};
  \draw (mdemux.blpin 1) node[circleinv,anchor=apex]{};
  \draw (mdemux.brpin 1) node[circleinv]{};
\end{tikzpicture}
```

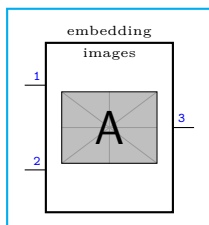
4.24.6 Adding a background drawing to the muxdemux

In a similar way to the oscilloscope waveform, you can add a drawing to the `muxdemux` component, provided you use basic-level `pgf` commands. You have to define a `.code` key named `bgpicture` in the `muxdemux def` definitions, as in the following example. The coordinate system is changed so that the horizontal axis of the shape is mapped between -1 cm and 1 cm and the origin at the center; the vertical extent will then depend on the form factor of the `muxdemux`: for example, the vertical coordinate of the top left border will be $1\text{ cm} \cdot Lh/w$.



```
\begin{circuitikz}
\ctikzset{muxdemux/outer label font={\tiny\ttfamily\color{blue}}}
\tikzset{myICw1/.style={muxdemux,
muxdemux def={Lh=4, Rh=4, w=3, NR=1, NL=2, NB=0, NT=0,
bgpicture/.code={%
\pgfsetcolor{gray!50}\pgfsetlinewidth{1mm}
\pgfpathmoveto{\pgfpoint{-1cm}{1cm*4/3}} \pgfpathlineto{\pgfpointorigin}
\pgfpathmoveto{\pgfpoint{-1cm}{-1cm*4/3}} \pgfpathlineto{\pgfpointorigin}
\pgfpathmoveto{\pgfpoint{1cm}{0cm}} \pgfpathlineto{\pgfpointorigin}
\pgfpathcircle{\pgfpointorigin}{0.5cm}
\pgfusepath{draw}
},
},
muxdemux label={RU1=3, LU1=1, LU2=2, N=flux, Ni=converter},}
}
\draw (0, 0) node[myICw1]{};
\end{circuitikz}
```

You can even embed images:



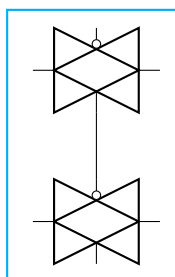
```
\begin{circuitikz}
\ctikzset{muxdemux/outer label font={\tiny\ttfamily\color{blue}}}
\tikzset{myICw1/.style={muxdemux,
muxdemux def={Lh=4, Rh=4, w=3, NR=1, NL=2, NB=0, NT=0,
bgpicture/.code={%
\pgfdeclareimage[width=1.5cm]{myimg}{example-image-a}
\pgftext{\pgfuseimage{myimg}}
},
},
muxdemux label={RU1=3, LU1=1, LU2=2, N=embedding, Ni=images},}
}
\draw (0, 0) node[myICw1]{};
\end{circuitikz}
```

For more complex things, consider using a normal macro that draws on the background layer and that use the geographical coordinates of the node to locate it.

4.24.7 Mux-Demux special usage

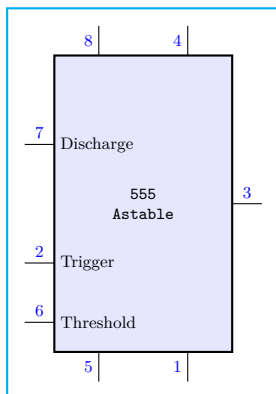
You can use these shapes to draw a lot of symbols that are unavailable; using a bit of L^AT_EX command trickery you can use them quite naturally too... Examples with personalized amplifier shapes are listed in section 4.20.3.

As an additional example, this was used before the introduction of the `double tgate` symbol in 1.2.4 (see 4.22.6.4):



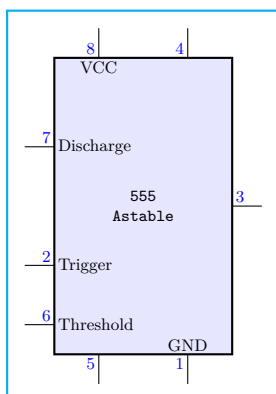
```
\def\tgate#1{
node[simple triangle, anchor=left, no input leads](#1-LR){}
(#1-LR.right) node[simple triangle, xscale=-1,
anchor=left](#1-RL){}
([yshift=.5ex]#1-RL.btpin 1) node[ocirc]{}
\begin{circuitikz}[
simple triangle/.style={muxdemux, muxdemux def={
NL=1, NR=1, NB=1, NT=1, w=2, Lh=2, Rh=0,
}}]
\draw (0,0) \tgate{A} (0,-2) \tgate{B};
\draw (A-RL.bpin 1) -- (B-RL.tpin 1);
\end{circuitikz}
```

Finally, you can play with them to create chips that have generic numbers of pins on the four sides, as in the following example (asked on [TeX.Stackexchange](https://tex.stackexchange.com); notice however that this example has been made *before* the option for labels existed; it could be quite streamlined now, as shown later):



```
\begin{tikzpicture}[scale=0.7, transform shape]
\tikzset{ic555/.style={muxdemux,
muxdemux def={Lh=10, NL=5, Rh=10, NR=5, NB=2, w=6, NT=2,
square pins=1},
no input leads, external pins width=0.4,
circuitikz/muxdemuxes/fill=blue!10}
}
\node [ic555, font=\small\ttfamily,align=center](A) at (0,0) {555\\Astable};
% left pins
\foreach \rawpin/\npin/\label in {2/7/Discharge, 4/2/Trigger, 5/6/Threshold}{
\draw (A.lpin \rawpin) -- (A.blpin \rawpin)
node[midway, blue, font=\small, above]{\npin}
node[right, font=\small]{\label};
}
% top pins
\foreach \rawpin/\npin in {1/8, 2/4} {
\draw (A.tpin \rawpin) -- (A.btpin \rawpin)
node[midway, blue, font=\small, left]{\npin};
}
% bottom pins
\foreach \rawpin/\npin in {1/5, 2/1} {
\draw (A.bpin \rawpin) -- (A.bbpin \rawpin)
node[midway, blue, font=\small, left]{\npin};
}
% finally, right
\draw (A.rpin 3) -- (A.brpin 3)
node[midway, blue, font=\small, above]{3};
\end{tikzpicture}
```

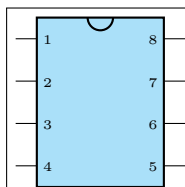
In a version of CircuiTikZ better or equal to v1.6.5, you can do this:



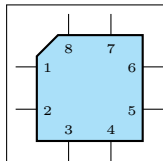
```
\begin{tikzpicture}[scale=0.7, transform shape]
\ctikzset{muxdemux/inner label font=\small}
\ctikzset{muxdemux/outer label font={\small\color{blue}}}
\tikzset{ic555/.style={muxdemux,
muxdemux def={Lh=10, NL=5, Rh=10, NR=1, NB=2, w=6, NT=2},
muxdemux label={L2=Discharge, L4=Trigger, L5=Threshold,
T1=VCC, B2=GND,
LU2=7, LU4=2, LU5=6, TL1=8, TL2=4, RU1=3, BL1=5, BL2=1},
external pins width=0.4,
draw only left pins={2,4,5},
circuitikz/muxdemuxes/fill=blue!10}
}
\node [ic555, font=\small\ttfamily,align=center](A)
at (0,0) {555\\Astable};
\end{tikzpicture}
```

4.25 Chips (integrated circuits)

CircuiTikZ supports two types of variable-pin chips: DIP (Dual-in-Line Package) and QFP (Quad-Flat Package).



Dual-in-Line Package chip, type: node, fillable (`node[dipchip]{}`).
Class: `chips`.



Quad-Flat Package chip, type: node, fillable (`node[qfpchip]{}`).
Class: `chips`.

4.25.1 DIP and QFP chips customization

You can scale chips with the key `chips/scale`. As ever, that will **not** scale text size of the labels, when they are printed.

The line thickness of the main shape is controlled by `multipoles/thickness` (default 2) and the one of the external pins/pads with `multipoles/external pins thickness` (default 1).

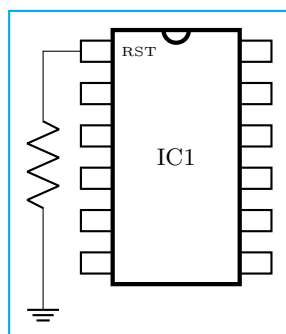
You can customize the DIP chip with the key `multipoles/dipchip/width` (with a default of 1.2) and the key `multipoles/dipchip/pin spacing` (default 0.4) that are expressed in fraction of basic lengths (see section 3.1.4). The height of the chip will be equal to half the numbers of pins multiplied by the spacing, plus one spacing for the borders.

For the QFP chips, you can only chose the pin spacing with `multipoles/qfpchip/pin spacing` key.

The number of pins is settable with the key `num pins`. **Please notice** that the number of pins **must** be *even* for `dipchips` and *multiple of 4* for `qfpchips`, otherwise havoc will ensue.

The pins of the chip can be “hidden” (that is, just a spot in the border, optionally marked with a number) or “stick out” with a thin lead by setting `multipoles/external pins width` greater than 0 (default value is 0.2, so you’ll have leads as shown above). Moreover, you can transform the thin lead into a pad by setting the key `multipoles/external pad fraction` to something different from 0 (default is 0); the value expresses the fraction of the pin spacing space that the pad will use on both sides of the pin.

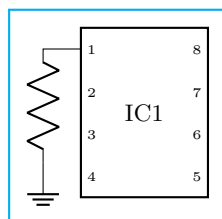
You can, if you want, avoid printing the numbers of the pin with `hide numbers` (default `show numbers`) if you prefer positioning them yourself (see the next section for the anchors you can use).



```
\begin{circuitikz}
\ctikzset{multipoles/thickness=4}
\ctikzset{multipoles/external pins thickness=2}
\draw (0,0) node[dipchip,
num pins=12,
hide numbers,
external pins width=0.3,
external pad fraction=4 ](C){IC1};
\draw (C.pin 1) -- ++(-0.5,0) to[R]
++(0,-3) node[ground]{};
\node [right, font=\tiny]
at (C.bpin 1) {RST};
\end{circuitikz}
```

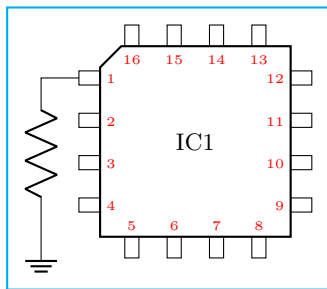
Also, you can suppress the drawing of the pins, by using the style `no input leads` (that can be reverted with `input leads`). The main difference between setting `external pins width` to 0 or using `no input lead` is that in the first case the normal pin anchors and the border anchors will coincide, and in the second case they will not move and stay where they should have been if the leads were drawn.

For special use you can suppress the orientation mark with the key `no topmark` (default `topmark`).



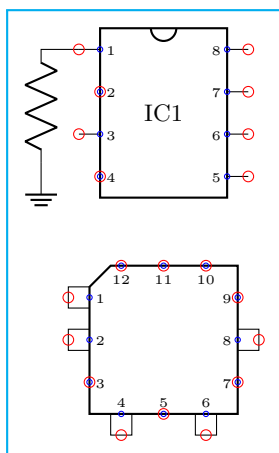
```
\begin{circuitikz}
\draw (0,0) node[dipchip,
num pins=8, no topmark,
external pins width=0.0](C){IC1};
\draw (C.pin 1) -- ++(-0.5,0) to[R]
++(0,-1.5) node[ground]{};
\end{circuitikz}
```

The font used for the pins is adjustable with the key `multipoles/font` (default `\tiny`)



```
\begin{circuitikz}
  \ctikzset{multipoles/font={\color{red}\tiny}}
  \draw (0,0) node[qfpchip,
    num pins=16,
    external pad fraction=6](C){IC1};
  \draw (C.pin 1) -- ++(-0.5,0) to[R]
    ++(0,-2) node[ground]{};
\end{circuitikz}
```

You can draw only selected pins and leave out the rest by setting `multipoles/draw only pins`¹²⁹. This key accepts a comma-separated list of pin numbers or ranges of pin numbers (a range is given as `(start) - (end)`, ends are inclusive). The numbers will not be expanded in any way, except those given as ends of ranges. A special value (and the initial one) is `all`, in which case all pins are drawn. The anchors will be adjusted, such that each `pin n` will be placed at the end of the pins which are drawn, and coincide with the `bpin n` anchors for the suppressed pins.



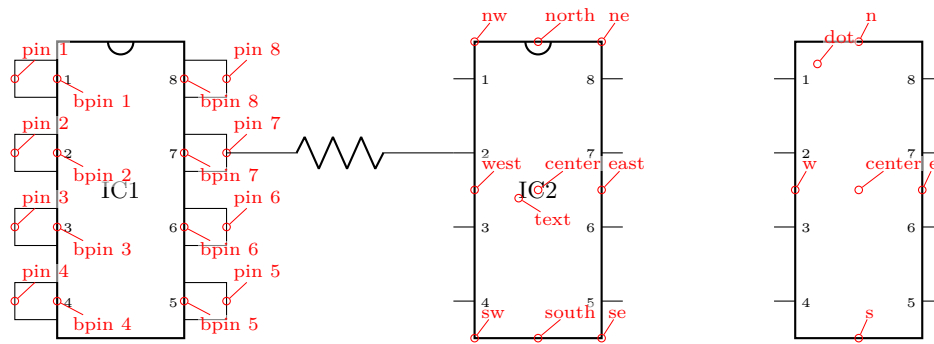
```
\begin{circuitikz}
  \draw (0,3) node[dipchip,
    num pins=8,
    draw only pins={1, 3, 5-8}](C){IC1};
  \draw (C.pin 1) -- ++(-0.5,0) to[R]
    ++(0,-1.5) node[ground]{};
  \foreach \x in {1,...,8} {
    \draw[red] (C.pin \x) circle[radius=2pt];
    \draw[blue] (C.bpin \x) circle[radius=1pt];
  }
  \draw (0, 0) node[qfpchip, draw only pins={1-2, 6, 8, 4},
    external pad fraction=4, num pins=12](Q){};
  \foreach \x in {1,...,12} {
    \draw[red] (Q.pin \x) circle[radius=2pt];
    \draw[blue] (Q.bpin \x) circle[radius=1pt];
  }
\end{circuitikz}
```

4.25.2 Chip anchors

Chips have anchors on pins and global anchors for the main shape. The pin anchors to be used to connect wires to the chip are called `pin 1`, `pin 2`, ..., with just one space between `pin` and the number. Border pin anchors (`bpin 1...`) are always on the box border, and can be used to add numbers or whatever markings are needed. Obviously, in case of `multipoles/external pins width` equal to zero, border and normal pin anchors will coincide.

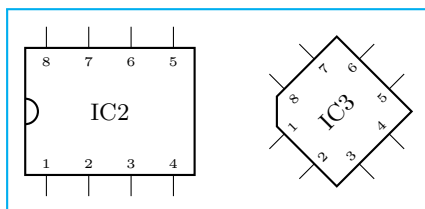
Additionally, you have geometrical anchors on the chip “box”, see the following figure. The nodes are available with the full name (like `north`) and with the short abbreviations `n`, `nw`, `w`, ... The `dot` anchor is useful to add a personalized marker if you use the `no topmark` key.

¹²⁹Added by Jonathan P. Spratte in v1.3.8



4.25.3 Chip rotation

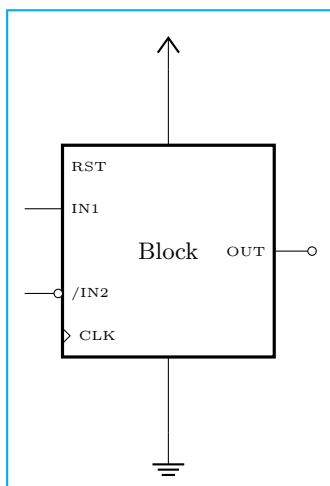
You can rotate chips, and normally the pin numbers are kept straight (option **straight numbers**, which is the default), but you can rotate them if you like with **rotated numbers**. Notice that the main label has to be (counter-) rotated manually in this case.



```
\begin{circuitikz}
\draw (0,0) node[dipchip,
rotate=90]{%
\rotatebox{-90}{IC2}};
\draw (3,0) node[qfpchip,
rotated numbers,
rotate=45]{IC3};
\end{circuitikz}
```

4.25.4 Chip special usage

You can use chips to have special, personalized blocks. Look at the following example, which is easily put into a macro.



```
\begin{circuitikz}
\ctikzset{multipoles/thickness=3}
\ctikzset{multipoles/dipchip/width=2}
\draw (0,0) node[dipchip,
num pins=10, hide numbers, no topmark,
external pins width=0](C){Block};
\node [right, font=\tiny] at (C.bpin 1) {RST};
\node [right, font=\tiny] at (C.bpin 2) {IN1};
\node [right, font=\tiny] at (C.bpin 4) {/IN2};
\node [left, font=\tiny] at (C.bpin 8) {OUT};
\draw (C.bpin 2) -- ++(-0.5,0) coordinate(extpin);
\node [ocirc, anchor=0](notin2) at (C.bpin 4) {};
\draw (notin2.180) -- (C.bpin 4 -| extpin);
\draw (C.bpin 8) to[short,-o] ++(0.5,0);
\draw (C.bpin 5) ++(0,0.1) -- ++(0.1,-0.1)
node[right, font=\tiny]{CLK} -- ++(-0.1,-0.1);
\draw (C.n) -- ++(0,1) node[vcc]{};
\draw (C.s) -- ++(0,-1) node[ground]{};
\end{circuitikz}
```

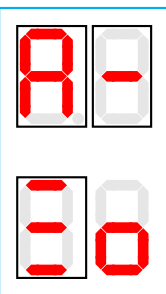
4.26 Seven segment displays



Seven segment display, type: node, fillable (`node[bare7seg]{}`).
Class: `displays`.

The seven-segment display lets you show values as if they were displayed in a classical seven-segment display.¹³⁰

The main “bare” component is the one shown above, but for simplicity a couple of style interfaces are defined:



```
\begin{circuitikz}
  \draw (0,0) node[seven segment val=A dot off box on]{};
  \draw (1,0) node[seven segment val=- dot none box on]{};
  \draw (0,-2) node[seven segment bits=1001001 dot empty box on]{};
  \draw (1,-2) node[seven segment bits=0011101 dot none box off]{};
\end{circuitikz}
```

There are two main configuration methods. The first one is `seven segment val`, which will take an hexadecimal number or value and display it: the possible values are 0, ..., 15, plus A, B, C, D, E, F (or lowercase) and the symbol - (minus).

The other interface is `seven segment bits`, where you specify seven bits saying which segment must be on (please never specify a different number of bits, it will throw a very obscure error); you can see in the anchors the name of each segment.

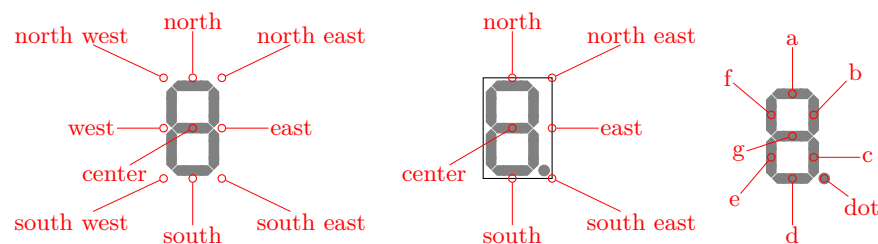
The option `dot` specifies if you want a decimal dot or not. The key `none` will remove the dot and the space it would take; `empty` will not show the dot at all but reserve the space, and `on` or `off` will show the dot in the corresponding state.

The option `box` (can be `on` or `off`) simply toggles the drawing of the external box. You can separate it from the display with the key `seven seg/box sep` (default 1pt), and it will use the thickness specified in `multipoles/thickness` (The same as the chips).

You can use these option with the “bare” object `bare7seg` and the keys `seven seg/bits` (default 0000000), `seven seg/dot` (default `none`) and `seven seg/box` (default `off`); there is no option equivalent to the `val` interface.

4.26.1 Seven segments anchors

These are the anchors for the seven-segment displays; notice that when the `dot` parameter is not `none`, the cell is a bit wider at the right side.



¹³⁰This component has been loosely inspired by the package `SevenSeg` by Germain Gondor, 2009, see TExexample.net.

4.26.2 Seven segments customization

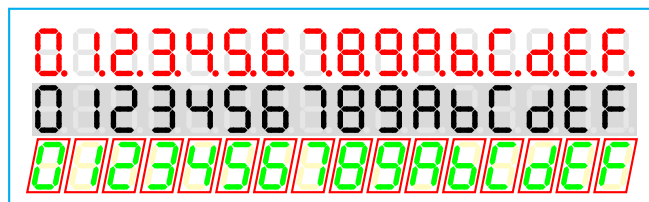
You can scale the seven segment display with the key `displays/scale`. This will scale the size of the digit, but not the absolute sizes shown below — if you want them to scale, you have to do it manually.

You can change several parameters to adjust the displays:

```
\ctikzset{seven seg/width/.initial=0.4}% relative to \pgf@circ@Rlen (scalable)
\ctikzset{seven seg/thickness/.initial=4pt}% segment thickness (not scaled)
\ctikzset{seven seg/segment sep/.initial=0.2pt}% gap between segments (not scaled)
\ctikzset{seven seg/box sep/.initial=1pt}% external box gap (not scaled)
\ctikzset{seven seg/color on/.initial=red}% color for segment "on"
\ctikzset{seven seg/color off/.initial=gray!20!white} % ...and "off"
```

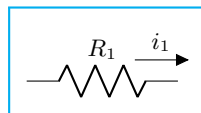
A couple of examples are shown below.

```
\begin{circuitikz}[scale=0.5]
\ctikzset{seven seg/width=0.2, seven seg/thickness=2pt}
\foreach \i in {0,...,15} \path (\i,0)
  node[seven segment val=\i dot on box off]{};
\ctikzset{seven seg/color on=black}
\foreach \i in {0,...,15} \path (\i,-1.5)
  node[seven segment val=\i dot off box off, fill=gray!30!white]{};
\ctikzset{seven seg/color on=green, seven seg/color off=yellow!30}
\foreach \i in {0,...,15} \path [color=red] (\i,-3)
  node[seven segment val=\i dot none box on, xslant=0.2]{};
\end{circuitikz}
```

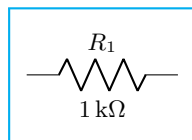


5 Labels, voltages and currents

You can add “decorations” to the path-style components; there are basically five types of them: labels, annotations, voltages, currents, and flows. Let’s see an example of all of them. . .



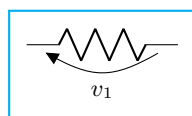
```
\begin{circuitikz}
  \draw (0,0) to[R, l=$R_1$, f=$i_1$] (2,0);
\end{circuitikz}
```



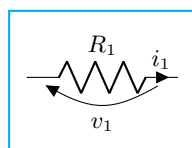
```
\begin{circuitikz}
  \draw (0,0) to[R=$R_1$, a=\SI{1}{\kohm}] (2,0);
\end{circuitikz}
```



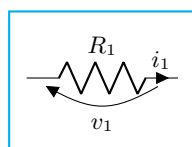
```
\begin{circuitikz}
  \draw (0,0) to[R, i=$i_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, v=$v_1$] (2,0);
\end{circuitikz}
```

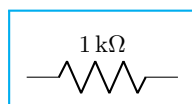


```
\begin{circuitikz}
  \draw (0,0) to[R=$R_1$, i=$i_1$, v=$v_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R=$R_1$, i=$i_1$, v=$v_1$] (2,0);
\end{circuitikz}
```

Long names/styles for the bipoles can be used, of course, and there is a special syntax (that works only in simple cases, and only with $\text{\LaTeX}$ — use it with caution!) if you load the package with the `siunitx` options:



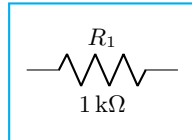
```
\begin{circuitikz}\draw
  (0,0) to[resistor=1<\kilo\ohm>] (2,0);
\end{circuitikz}
```

5.1 Labels and Annotations

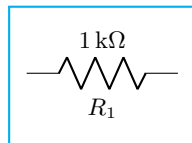
Since Version 0.7, beside the original label (l) option, there is a new option to place a second label, called annotation (a) at each bipole.

5.1.1 Label and annotation position

When drawing a component left-to-right, the label l is by default above the component, and the annotation a is by default below it. The position of annotations and labels can be adjusted adding the characters $_$ or $\wedge$ to the key.

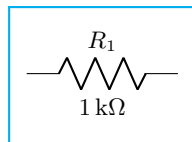


```
\begin{circuitikz}
\draw (0,0) to[R, l=$R_1$,a=1<\kilo\ohm>] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
\draw (0,0) to[R, l_=$R_1$,a^=1<\kilo\ohm>] (2,0);
\end{circuitikz}
```

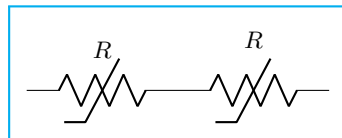
For passive components, you can use `component type=text` as a shortcut for `component type`, `l=text`:



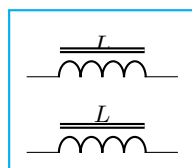
```
\begin{circuitikz}
\draw (0,0) to[R=$R_1$,a=1<\kilo\ohm>] (2,0);
\end{circuitikz}
```

Notice though that in active component (sources of either voltage or current) the shortcut will set the voltage (v) or current (i) property.

5.1.1.1 Adjust label and annotation position. Normally the package will guess a good position for the label or annotation; if you do not like it, you can add¹³¹ (or remove, with negative values) distance using the `\ctikzset` keys `label distance` and `annotation distance`.



```
\begin{circuitikz}
\draw (0,0) to[sR, l=$R$, label distance=-4pt] (2,0)
to [sR, l=$R$] (4,0);
\end{circuitikz}
```

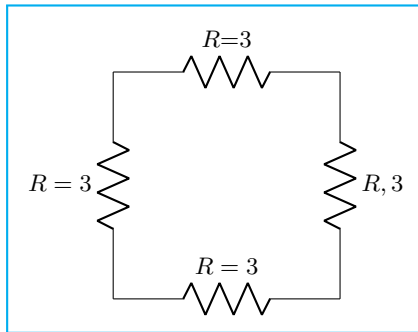


```
\begin{circuitikz}[american]
\ctikzset{bipoles/inductors/core distance=4pt}
\draw (0,1) to[L=$L$, name=myL] ++(2,0);
\draw[thick, double] (myL.core west) -- (myL.core east);
\draw (0,0) to[L=$L$, name=myL, label distance=2pt] ++(2,0);
\draw[thick, double] (myL.core west) -- (myL.core east);
\end{circuitikz}
```

5.1.2 Special symbols in labels and annotations.

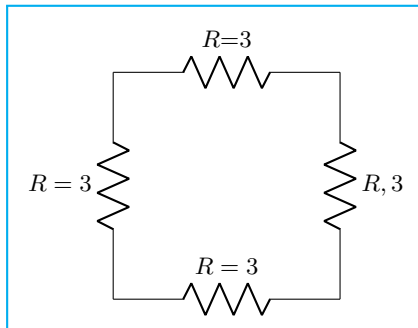
When TikZ processes the options, there will be problems if the label (or annotation, voltage, or current) contains one of the characters `=` (equal) or `,` (comma) — because the parser search for those two characters to delimit the arguments, giving unexpected errors and wrong output. These two characters can be protected from the option parser using an extra set of braces.

¹³¹Since version 1.3.3



```
\begin{circuitikz}
% the following will fail:
% \draw (0,0) to[R, l=$R=3$] (3,0);
\draw (0,0) to[R, l={ $R=3$}] (3,0);
\draw (0,0) to[R={ $R=3$}] (0,3);
\draw (3,3) to[R={ $R,3$}] (3,0);
% this works, but it has wrong spacing
\draw (0,3) to[R, l=$R{=}3$] (3,3);
\end{circuitikz}
```

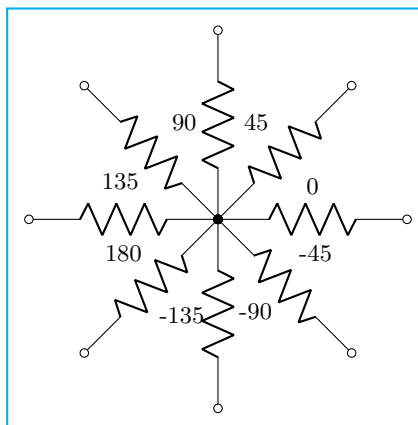
Caveat: up to version 1.2.7, due to the way in which CircuitikZ used to process the options, even that was not sufficient, so you must protect that tokens even more, for example using an `\mbox` command, or redefining the characters with a `\TeX \def`:



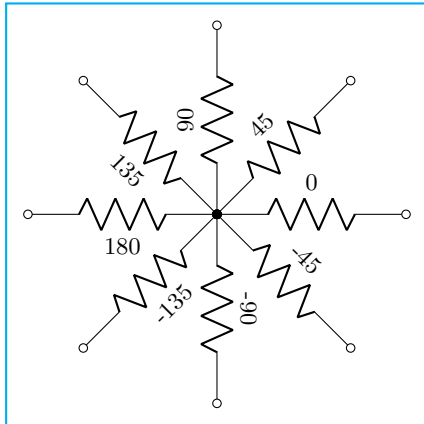
```
\begin{circuitikz}
\def\eq{=}
% the following will fail up to 1.2.7:
% \draw (0,0) to[R, l={ $R=3$}] (3,0);
\draw (0,0) to[R, l=\mbox{$R=3$}] (3,0);
\draw (0,0) to[R, l=$R\eq3$] (0,3);
\draw (3,3) to[R, l=\mbox{$R,3$}] (3,0);
% this works, but it has wrong spacing
\draw (0,3) to[R, l=$R{=}3$] (3,3);
\end{circuitikz}
```

5.1.3 Labels and annotation orientation.

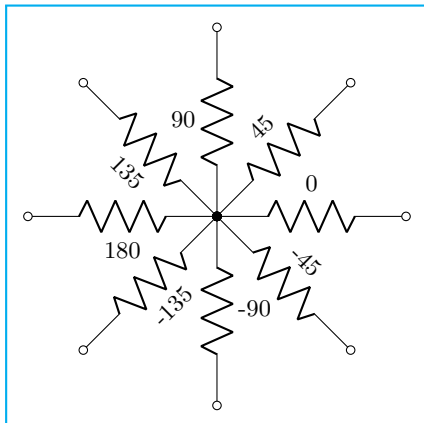
The default orientation of labels is controlled by the global options `smartlabels`, `rotatelabels` and `straightlabels` (or the corresponding `label/align` keys). Here are examples to see the differences:



```
\begin{circuitikz}
\ctikzset{label/align = straight}
\def\DIR{0,45,90,135,180,-90,-45,-135}
\foreach \i in \DIR {
\draw (0,0) to[R=\i, *-o] (\i:2.5);
}
\end{circuitikz}
```

```
\begin{circuitikz}
\ctikzset{label/align = rotate}
\def\DIR{0,45,90,135,180,-90,-45,-135}
\foreach \i in \DIR {
  \draw (0,0) to[R=\i, *-o] (\i:2.5);
}
\end{circuitikz}
```



```
\begin{circuitikz}
\ctikzset{label/align = smart}
\def\DIR{0,45,90,135,180,-90,-45,-135}
\foreach \i in \DIR {
  \draw (0,0) to[R=\i, *-o] (\i:2.5);
}
\end{circuitikz}
```

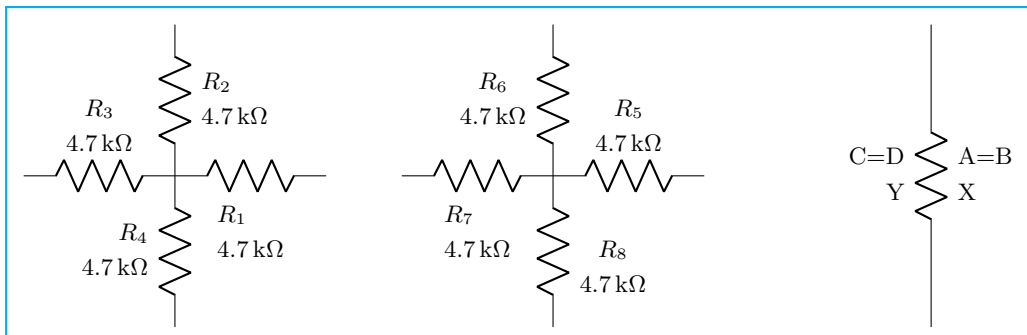
5.1.4 Stacked (two lines) labels.

When using `circuitikz` in LaTeX, you can use stacked (two lines) labels. The example should be self-explanatory: the two lines are specified as `l2=`*line1* and *line2*. You can use the keys `l2 halign` to control horizontal position (left, center, right) and `l2 valign` to control the vertical one (bottom, center, top). The default values for alignments are thought for vertical components (where the stacked labels are more natural), in other positions you have to force them.

Notice that you **can't** use the compact `<...>` notation for `siunitx` with stacked labels. Before v1.3.6 the label was ignored, but that has been converted into an error.

Since v1.3.6 you have the same possibility with the `annotation` (just use `a2=...`, `a2_=...`, `a2 valign` and so on. Notice that the default position for stacked annotation is `a2 halign=1`.

The `l2` and `a2` will only work in LaTeX because they use a `tabular` environment in their implementation. For plain TeX and ConTeXt you have to use `l` and `a` and build the stack of labels externally.



```

\begin{circuitikz}[american]
%
% the default for l2 is: l2 halign=l, l2 valign=c. DO NOT USE the <...> notation
%
\draw (0,0) to[R, l2_=$R_1$ and \SI{4.7}{k\ohm}, , l2 valign=t] ++(2,0);
\draw (0,0) to[R, l2_=$R_2$ and \SI{4.7}{k\ohm}, , ] ++(0,2);
\draw (0,0) to[R, l2_=$R_3$ and \SI{4.7}{k\ohm}, l2 halign=c, l2 valign=b] ++(-2,0);
\draw (0,0) to[R, l2_=$R_4$ and \SI{4.7}{k\ohm}, l2 halign=r, l2 valign=c] ++(0,-2);
\draw (5,0) to[R, l2_=$R_5$ and \SI{4.7}{k\ohm}, l2 halign=c, l2 valign=b] ++(2,0);
\draw (5,0) to[R, l2_=$R_6$ and \SI{4.7}{k\ohm}, l2 halign=c, ] ++(0,2);
\draw (5,0) to[R, l2_=$R_7$ and \SI{4.7}{k\ohm}, , l2 valign=t] ++(-2,0);
\draw (5,0) to[R, l2_=$R_8$ and \SI{4.7}{k\ohm}, l2 halign=c, l2 valign=t] ++(0,-2);
\draw (10,2) to[R, l2={A=B} and X, a2={C=D} and Y] ++(0,-4);
\end{circuitikz}

```

For extra options about labels and annotations, please refer to section [5.6](#)

5.2 Currents and voltages

The default direction/sign for currents and voltages in the components is, unfortunately, not standard, and can change across country and sometime across different authors. This unfortunate situation created a bit of confusion in `circuitikz` across the versions, with several incompatible changes starting from version 0.5. From version 0.9.0 onward, the maintainers agreed a new policy for the directions of bipoles' voltages and currents, depending on 4 different possible options:

- **oldvoltagedirection**, or the key style **voltage dir=old**: Use old way of voltage direction having a difference between european and american direction, with wrong default labeling for batteries (it was the default before version 0.5);
- **nooldvoltagedirection**, or the key style **voltage dir=noold**: The standard from version 0.5 onward, utilize the (German?) standard of voltage arrows in the direction of electric fields (without fixing batteries);
- **RPvoltages** (meaning Rising Potential voltages), or the key style **voltage dir=RP**: the arrow is in direction of rising potential, like in **oldvoltagedirection**, but batteries and current sources are fixed so that they follow the passive/active standard: the default direction of **v** and **i** are chosen so that, when both values are positive:
 - in passive component, the element is *dissipating power*;
 - in active components (generators), the element is *generating power*.
- **EFvoltages** (meaning Electric Field voltages), or the key style **voltage dir=EF**: the arrow is in the direction of the electric field, like in **nooldvoltagedirection**, but batteries are fixed;

Notice that the four styles are designed to be used at the environment level: that is, you should use them at the start of your environment as in `\begin{circuitikz}[voltage dir=old]` ... and not as a key for single components, in which case the behavior is not guaranteed.

The standard direction of currents, flows and voltages are changed by these options; notice that the default drops in case of passive and active elements is normally different. Take care that in the case of **noold** and **EFvoltages** also the currents can switch directions. It is much easier to understand the several behaviors by looking at the following examples, that have been generated by the code:

```

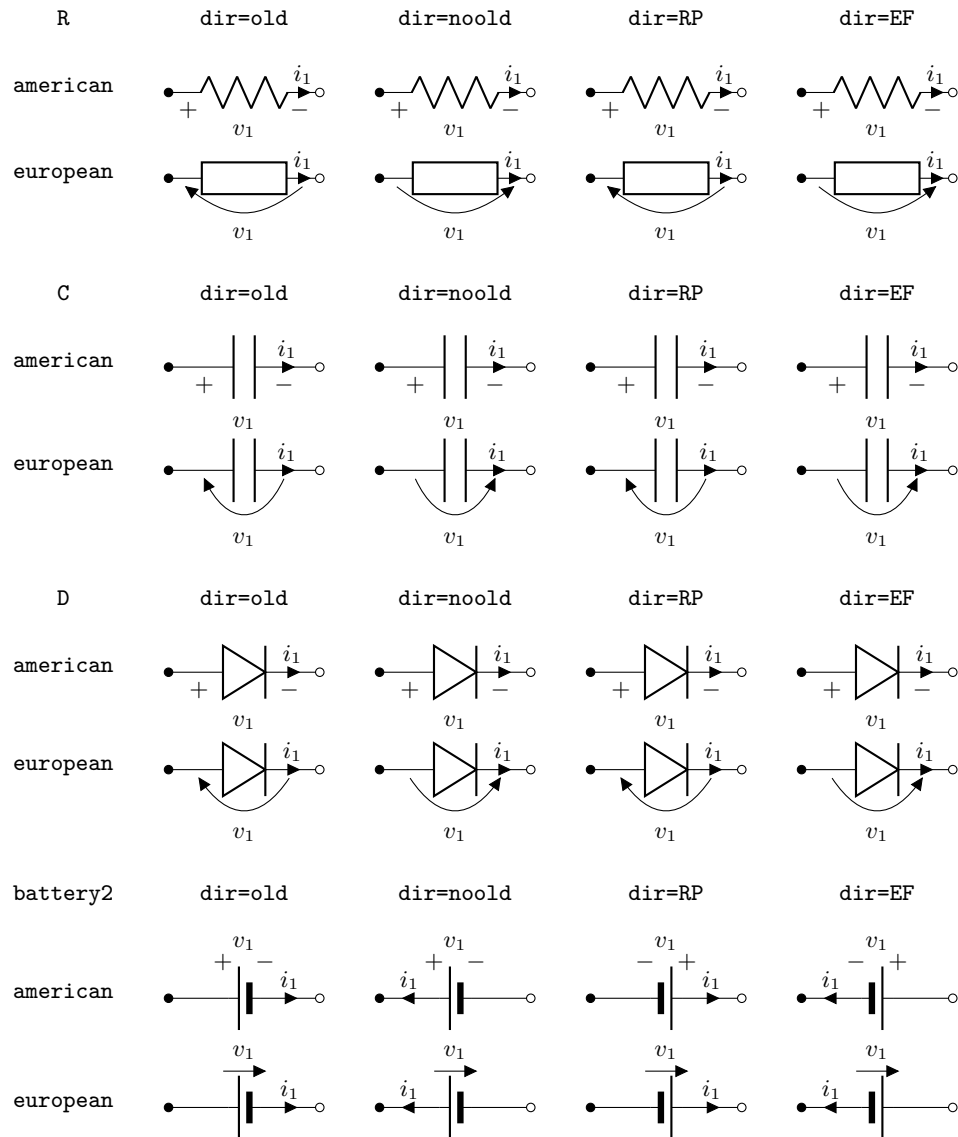
\newcommand{\VoltageDirPic}[3]{%
  \begin{tikzpicture}[#2, voltage dir=#3, baseline]
    \draw (0,0) to[#1, *-o, v=$v_1$, i=$i_1$] (2,0);
  \end{tikzpicture}%
}
\ifbool{warpingprint}{\StartDefiningTabulars}% for lwrap
\newcommand{\VoltageDirRow}[1]{%
  \noindent\ttfamily

```

```

\begin{tabular}{@{}c@{\quad}c@{\quad}c@{\quad}c@{\quad}c@{}}
#1 & dir=old & dir=noold & dir=RP & dir=EF \\
american & \VoltageDirPic{#1}{american}{old} \\
& \VoltageDirPic{#1}{american}{noold} \\
& \VoltageDirPic{#1}{american}{RP} \\
& \VoltageDirPic{#1}{american}{EF} \\
european & \VoltageDirPic{#1}{european}{old} \\
& \VoltageDirPic{#1}{european}{noold} \\
& \VoltageDirPic{#1}{european}{RP} \\
& \VoltageDirPic{#1}{european}{EF} \\
\end{tabular}%
\par\medskip
}
\ifbool{warpingprint}{\StopDefiningTabulars}% for lwrap
\foreach\element in {R, C, D, battery2, V, I, sV, cV, cI}{%
  \VoltageDirRow{\element}%
}

```



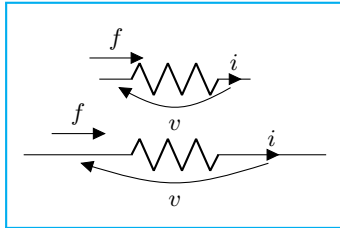
V	dir=old	dir=noold	dir=RP	dir=EF
american				
european				
I	dir=old	dir=noold	dir=RP	dir=EF
american				
european				
sV	dir=old	dir=noold	dir=RP	dir=EF
american				
european				
cV	dir=old	dir=noold	dir=RP	dir=EF
american				
european				
cI	dir=old	dir=noold	dir=RP	dir=EF
american				
european				

Obviously, you normally use just one between current and flows, but anyway you can change the direction of the voltages, currents and flows using the complete keys $i_>$, $i\^<$, $i>_$, $i\^>$, as shown in the following examples.

This manual has been typeset with the option `RPvoltages`.

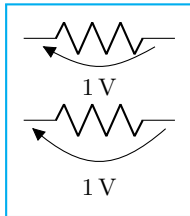
5.2.1 Common properties of voltages and currents

Currents, voltages and flows (see later) are positioned along, or across, the part of the wires that connect the inner component to the rest of the circuit. So, changing the length of the connection (the coordinates that embrace the `to[...]` command) will change the position of the components.



```
\begin{circuitikz}
  \draw (-1,1) to[R, v=$v$, i=$i$, f>=$f$] (1,1);
  \draw (-2,0) to[R, v=$v$, i=$i$, f>=$f$] (2,0);
\end{circuitikz}
```

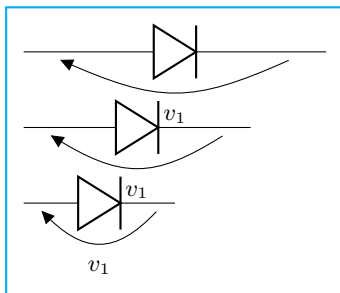
However, you can override the properties `voltage/distance from node` (default 0.5: how distant from the initial and final points of the path the arrow starts and ends or the plus and minus symbols are drawn) and `voltage/bump b` (how high the bump of the arrow is — how curved it is, default 1.5), and also `voltage/european label distance` (how distant from the normal position the voltage label will be, default 1.4) on a per-component basis, in order to fine-tune the voltages:



```
\tikz \draw (0,0) to[R, v=1<\volt>] (2,0); \par
\ctikzset{voltage/distance from node=.1}
\ctikzset{voltage/bump b=2.5}
\tikz \draw (0,0) to[R, v=1<\volt>] (2,0);
```

You can also use a global `ctikzset` on the key `voltage/distance from node` (and similar) that will act as a default value. Notice however that the specific component value **overrides** the global one, and several components have pre-defined overrides, so they will ignore the default value. The components that have out of the box predefined overrides for `distance from node` are `generic`, `ageneric`, `fullgeneric` and `memristor` (set to 0.4), and the ones that have it for `bump b` are `generic`, `ageneric`, `fullgeneric`, `memristor`, `tline`, `varistor`, `photoresistor`, `thermistor`, `thermistorntc`, `thermistorptc`, `ccapacitor`, `emptyzddiode`, `fullzddiode`, `emptythyristor`, `fullthyristor`, `emptytriac` and `fulltriac`, with several values (you can look at them in the file `pgfcirc.defines.tex`).¹³²

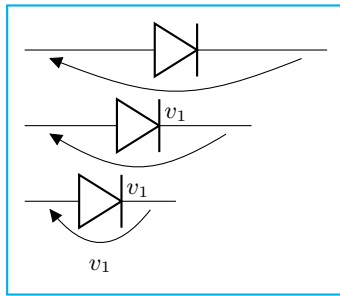
Notice also that normally `distance from node` is a relative displacement, computed on the node-component wire. So that this will put the start and stop point 1/4 of the way between node and component:



```
\begin{circuitikz}
  \ctikzset{voltage/distance from node=0.25}
  \draw (0, 2) to[D, v=$v_1$] ++(4,0);
  \draw (0, 1) to[D, v=$v_1$] ++(3,0);
  \draw (0, 0) to[D, v=$v_1$] ++(2,0);
\end{circuitikz}
```

The value of `distance from node` can also be an absolute distance; in that case is measured from the start of the connection toward the component on the left (and symmetrically on the right), so this will put the start and end point to 0.25 cm from the start of the node:

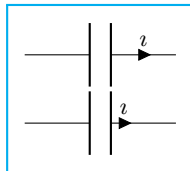
¹³²To achieve consistent results of `voltage/distance from node=0` refer to this solution by Jakob Leide on tex.stackexchange.com.



```
\begin{circuitikz}
  \ctikzset{voltage/distance from node=0.25cm}
  \draw (0, 2) to[D, v=$v_1$] ++(4,0);
  \draw (0, 1) to[D, v=$v_1$] ++(3,0);
  \draw (0, 0) to[D, v=$v_1$] ++(2,0);
\end{circuitikz}
```

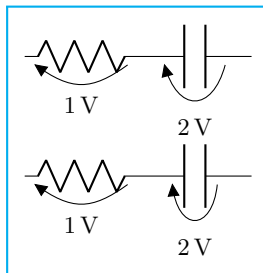
There is currently no way to specify the position at a fixed distance from the component (as opposed as from the node).

The same concept as **distance from node** applies to the key **current/distance** for the position of the current's arrow (and to **flow/distance** for the flow arrow position):



```
\tikz \draw (0,0) to[C, i=$\imath$] (2,0); \par
\ctikzset{current/distance = .2}
\tikz \draw (0,0) to[C, i=$\imath$] (2,0);
```

If you want to change those parameters by defining a component-specific key you have to use the internal name of the component (in the component list, is the **nodename** without the terminal “**shape**” part):



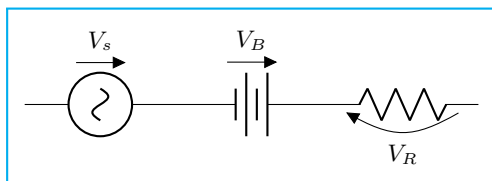
```
\tikz \draw (0,0) to[R, v=1<\volt>] (1.5,0)
to[C, v=2<\volt>] (3,0); \par
\ctikzset{bipoles/capacitor/voltage/distance from node/.initial
=.7}
\tikz \draw (0,0) to[R, v=1<\volt>] (1.5,0)
to[C, v=2<\volt>] (3,0); \par
```

Note the **.initial**; you have to create such key the first time you use it. These kinds of adjustments are not guaranteed to work in future upgrades, though; if you have to create a key you are somehow touching the internal structure of the package; it's much safer to create a style.

One common request is to change the style of the arrows (both head and line) of these elements. Voltages, currents and flows are part of the same path of the component, so this is not possible in simple way; you have to draw your own with TikZ commands using the facilities explained in section 5.8.

5.2.2 Special treatment for generators

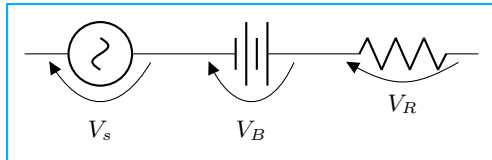
The “active” elements (sources and batteries, mainly) are treated differently from passive elements, in the sense that the default current and voltage direction and position could be different¹³³ following the chosen global voltage direction strategy (see section 5.2). If they change or not depend on both the element and the chosen **voltage dir** option.



```
\begin{tikzpicture}[]
  \draw (0,0) to[sV, v=$V_s$] ++(2,0)
to[battery, v=$V_B$] ++(2,0)
to[R, v=$V_R$] ++(2,0);
\end{tikzpicture}
```

¹³³This, in hindsight, has been a bad feature — and I'm partly responsible for it. But removing it would create *too small* variations in circuits, easily to go unnoticed, so it stays: nobody wants *wrong* circuits just by recompiling.

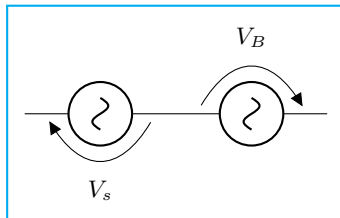
The consistency between symbols drawings and the default voltage and current directions are designed to work well *when this default is enabled*. If you want, though, you can override this behavior by “switching off” the source status of the component by setting the property `bipole/is voltage` to `false`:



```
\begin{tikzpicture}[]
  \draw (0,0) to[sV, bipole/is voltage=false,
    v=$V_s$] ++(2,0)
    to[battery, bipole/is voltage=false,
    v=$V_B$] ++(2,0)
    to[R, v=$V_R$] ++(2,0);
\end{tikzpicture}
```

When you do this, **be careful** that (as you can see) the direction of the plain `v=...` option will change (please notice that this does not mean that it is incorrect, given that the voltage and current direction are arbitrary; in the case above, if the battery is a 3 V one, $V_B = -3\text{ V}$ with the `RPvoltages` conventions).

Also, notice that there is an ordering problem in the `to[...]` options: you have to switch the `is voltage` property off **before** setting the voltage, otherwise you will have a mix of the source-type and passive positioning:

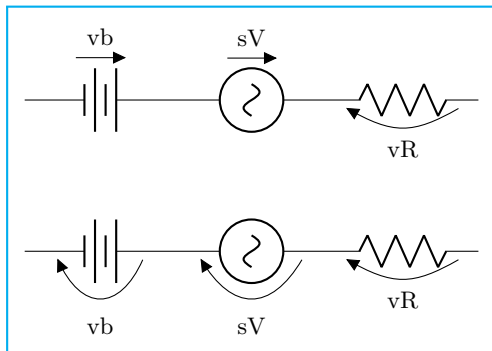


```
\begin{tikzpicture}[]
  % correct way
  \draw (0,0) to[sV, bipole/is voltage=false, v=$V_s$]
    ++(2,0)
  % wrong way, setting voltage before changing type
    to[sV=$V_B$, bipole/is voltage=false, ] ++(2,0);
\end{tikzpicture}
```

In the first `to[]` command, the voltage is set before changing the type (assigning a value to the name of the element is understood as a `v=...` command for voltage sources).

A similar switch is present for current generators, called `bipoles/is current`, acting in a very similar way.

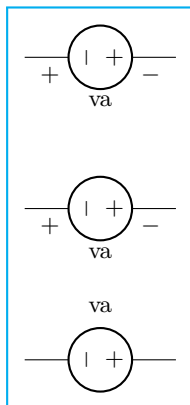
If you would prefer to switch to the `is voltage=false, is current=false` behavior by default, you can (since v1.4.4¹³⁴) by setting the option `bipole/override source vif` to `true`. This is *highly* experimental, so use with care.



```
\begin{tikzpicture}[]
  \draw (0,0) to [battery=vb] ++(2,0)
    to[sV=sV] ++(2,0) to[R, v=vR] ++(2,0);
  \ctikzset{bipole/override source vif=true}
  \draw (0,-2) to [battery=vb] ++(2,0)
    to[sV=sV] ++(2,0) to[R, v=vR] ++(2,0);
\end{tikzpicture}
```

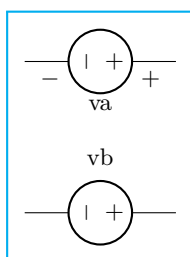
Notice that the option `override source vif` is “stronger” than the normal `is voltage`; so to locally re-set the behavior for just one source, you need to disable that *before* using a voltage designator.

¹³⁴Suggested by user [@judober](#) on GitHub.



```
\begin{tikzpicture}[american]
  % dangerous option ahead: USE WITH CARE
  \ctikzset{bipole/override source vif=true}
  % ugly output, you should really use V>=va
  \draw (0,6) to [V=va] ++(2,0);
  % not working!
  \draw (0,4) to [V=va, bipole/override source vif=false] ++(2,0);
  % ok, this one is working --- you need both settings!
  \draw (0,2) to [V,
    bipole/override source vif=false,
    bipole/is voltage=true,
    v=va] ++(2,0);
\end{tikzpicture}
```

Clearly, if you find yourself using the last component often, it is better to define a style, which will save you a lot of typing and help readability:



```
\tikzset{myV/.style={V, bipole/override source vif=false,
  bipole/is voltage=true, v={#1}}}%
\begin{tikzpicture}[american]
  % dangerous option ahead: USE WITH CARE
  \ctikzset{bipole/override source vif=true}
  \draw (0,0) to [V=va] ++(2,0);
  \draw (0,-2) to [myV=vb] ++(2,0);
\end{tikzpicture}
```

On the other way around, you could use styles to set `is voltage=false` only on the components you use and without using the global switch — which is the recommended way of doing it.

5.3 Currents

Inline (along the wire) currents are selected with `i_>`, `i^<`, `i>_`, `i>^`, and various combinations; the default position and direction is obtained with the simple key `i=...`

Basically, `^` and `_` control if the label is above or below the line (above and below **do** depend on the direction of the component path), and `<` and `>` the direction of the arrow; swapping them (from for example from `i^>` to `i>^`) will switch the side of the component where the symbol is drawn. See the following examples:



```
\begin{circuitikz}
  \draw (0,0) to[R, i^>=$i_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, i_>=$i_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, i^<=$i_1$] (2,0);
\end{circuitikz}
```




```
\begin{circuitikz}
  \draw (0,0) to[R, i_<=$i_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, i>^=$i_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, i>_=$i_1$] (2,0);
\end{circuitikz}
```

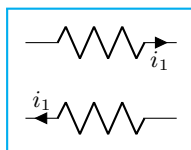


```
\begin{circuitikz}
  \draw (0,0) to[R, i<^=$i_1$] (2,0);
\end{circuitikz}
```



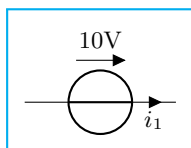
```
\begin{circuitikz}
  \draw (0,0) to[R, i<_=$i_1$] (2,0);
\end{circuitikz}
```

Also notice that the direction of the path is important:

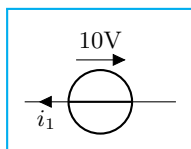


```
\begin{circuitikz}
  \draw (2,1) to[R, i<=$i_1$] (0,1);
  \draw (0,0) to[R, i<=$i_1$] (2,0);
\end{circuitikz}
```

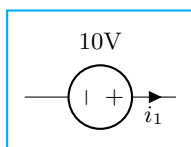
Default directions can change if the component is active or passive,¹³⁵ following the chosen global voltage direction strategy (see section 5.2).



```
\begin{circuitikz}
  \draw (0,0) to[V=10V, i_=$i_1$] (2,0);
\end{circuitikz}
```

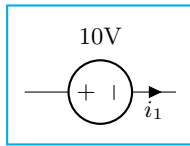


```
\begin{circuitikz}[voltage dir=EF]
  \draw (0,0) to[V=10V, i_=$i_1$] (2,0);
\end{circuitikz}
```



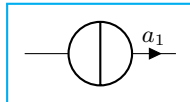
```
\begin{circuitikz}[american]
  \draw (0,0) to[V=10V, i_=$i_1$] (2,0);
\end{circuitikz}
```

¹³⁵This is better explained in section 5.2.2



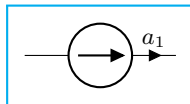
```
\begin{circuitikz}[american]
  \draw (0,0) to[V=10V,invert, i_=$i_1$] (2,0);
\end{circuitikz}
```

Current generators with the direct label (the one obtained by, for example, $I = \text{something}$) will treat it as a current:



```
\begin{circuitikz}
  \draw (0,0) to[I=$a_1$] (2,0);
\end{circuitikz}
```

If you use the option `americancurrent` or using the style `american currents` you can change the style of current generators.



```
\begin{circuitikz}[american currents]
  \draw (0,0) to[I=$a_1$] (2,0);
\end{circuitikz}
```

5.4 Flows

As an alternative for the current arrows, you can also use the following “flows”. They can also be used to indicate thermal or power flows. The syntax is pretty the same as for currents.



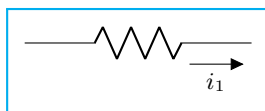
```
\begin{circuitikz}
  \draw (0,0) to[R, f=$i_1$] (3,0);
\end{circuitikz}
```



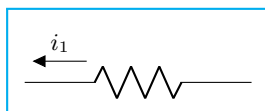
```
\begin{circuitikz}
  \draw (0,0) to[R, f<=$i_1$] (3,0);
\end{circuitikz}
```



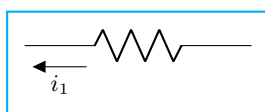
```
\begin{circuitikz}
  \draw (0,0) to[R, f_=$i_1$] (3,0);
\end{circuitikz}
```



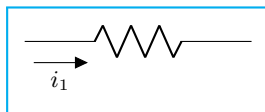
```
\begin{circuitikz}
  \draw (0,0) to[R, f_>=$i_1$] (3,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, f<^=$i_1$] (3,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, f<_=$i_1$] (3,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, f>_=$i_1$] (3,0);
\end{circuitikz}
```

5.5 Voltages

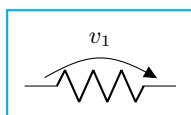
See the introduction at Currents and Voltages (section 5.2, page 225) for the default direction of the voltage and currents.

Voltages come in four different styles: European (with curved or straight arrows) and American (with signs that can stay near the wire or raised at the label level).

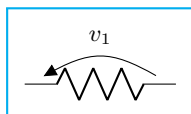
Direction and position of the symbols are controlled in the same way as for the currents (see section 5.3) with the `_<>` symbols.

5.5.1 European style

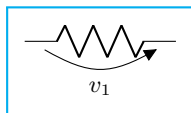
The default, with curved arrows. Use option `europeanvoltage` or style `european voltages`, or setting (even locally) `voltage=european`.



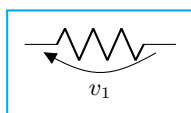
```
\begin{circuitikz}[european voltages]
  \draw (0,0) to[R, v^>=$v_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}[european voltages]
  \draw (0,0) to[R, v^<=$v_1$] (2,0);
\end{circuitikz}
```

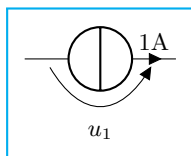


```
\begin{circuitikz}[european voltages]
  \draw (0,0) to[R, v_>=$v_1$] (2,0);
\end{circuitikz}
```

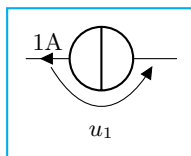


```
\begin{circuitikz}[european voltages]
  \draw (0,0) to[R, v_<=$v_1$] (2,0);
\end{circuitikz}
```

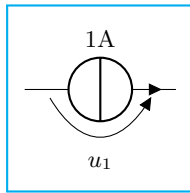
The default direction for active elements can change, depending on the global `voltage dir` setting, so be careful.



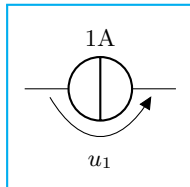
```
\begin{circuitikz}
  \draw (0,0) to[I=1A, v_=$u_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[I<=1A, v_=$u_1$] (2,0);
\end{circuitikz}
```

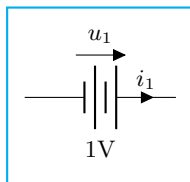


```
\begin{circuitikz}
  \draw (0,0) to[I=$$,l=1A, v_=$u_1$] (2,0);
\end{circuitikz}
```

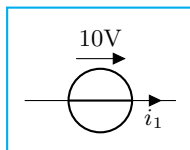


```
\begin{circuitikz}
  \draw (0,0) to[I,l=1A, v_=$u_1$] (2,0);
\end{circuitikz}
```

Moreover, for historical reasons, voltage generators have differently looking arrows (they are straight even in curved European style).

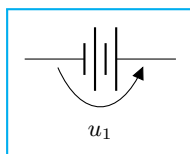


```
\begin{circuitikz}
  \draw (0,0) to[battery,l=1V, v=$u_1$, i=$i_1$] (2,0);
\end{circuitikz}
```



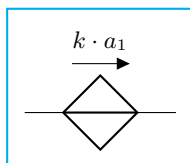
```
\begin{circuitikz}
  \draw (0,0) to[V=10V, i_=$i_1$] (2,0);
\end{circuitikz}
```

You can change this last thing by forcing “off” the status of “voltage generator” of the component; but now the normal (passive) rule will apply, so, again, be careful and read section 5.2.2.



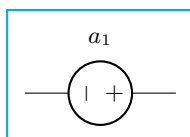
```
\begin{circuitikz}
  \draw (0,0) to[battery, bipole/is voltage=false,
    v>=$u_1$,] (2,0);
\end{circuitikz}
```

As for the currents, the direct label of voltage sources is passed as a voltage:



```
\begin{circuitikz}
  \draw (0,0) to[cV=$k\cdot a_1$] (2,0);
\end{circuitikz}
```

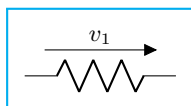
The following results from using the option `american voltages` or the style `american voltages`.



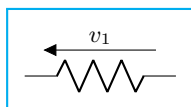
```
\begin{circuitikz}[american voltages]
  \draw (0,0) to[V=$a_1$] (2,0);
\end{circuitikz}
```

5.5.2 Straight European style

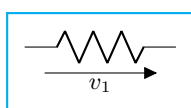
Using straight arrows. Use option `straightvoltages` or style `straight voltages`, or setting (even locally) `voltage=straight`.



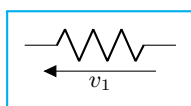
```
\begin{circuitikz}[straight voltages]
\draw (0,0) to[R, v^>=$v_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}[straight voltages]
\draw (0,0) to[R, v^<=$v_1$] (2,0);
\end{circuitikz}
```

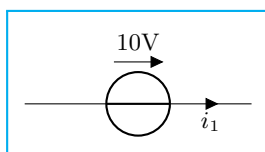


```
\begin{circuitikz}[straight voltages]
\draw (0,0) to[R, v_>=$v_1$] (2,0);
\end{circuitikz}
```

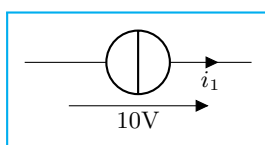


```
\begin{circuitikz}[straight voltages]
\draw (0,0) to[R, v_<=$v_1$] (2,0);
\end{circuitikz}
```

Again, voltage generators are treated differently:

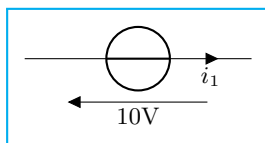


```
\begin{circuitikz}[straight voltages]
\draw (0,0) to[V=10V, i_=$i_1$] (3,0);
\end{circuitikz}
```



```
\begin{circuitikz}[straight voltages]
\draw (0,0) to[I, v=10V, i_=$i_1$] (3,0);
\end{circuitikz}
```

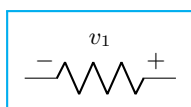
And you can override that with `bipole/is voltage` keeping into account that the default direction will be the one of passive components (see 5.2.2):



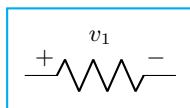
```
\begin{circuitikz}[straight voltages]
\draw (0,0) to[V, bipole/is voltage=false,
v=10V, i_=$i_1$] (3,0);
\end{circuitikz}
```

5.5.3 American style

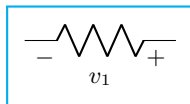
Use option `americanvoltage` or set `american voltages` or use the option `voltage=american`.



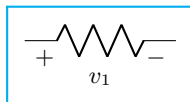
```
\begin{circuitikz}[american voltages]
\draw (0,0) to[R, v^>=$v_1$] (2,0);
\end{circuitikz}
```



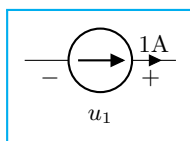
```
\begin{circuitikz}[american voltages]
\draw (0,0) to[R, v^<=$v_1$] (2,0);
\end{circuitikz}
```



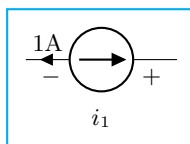
```
\begin{circuitikz}[american voltages]
\draw (0,0) to[R, v_>=$v_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}[american voltages]
\draw (0,0) to[R, v_<=$v_1$] (2,0);
\end{circuitikz}
```



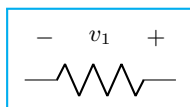
```
\begin{circuitikz}[american]
\draw (0,0) to[I=1A, v_=$u_1$] (2,0);
\end{circuitikz}
```



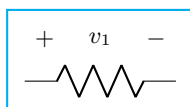
```
\begin{circuitikz}[american]
\draw (0,0) to[I<=1A, v_=$i_1$] (2,0);
\end{circuitikz}
```

5.5.4 Raised American style

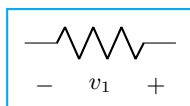
Since version 1.2.1, “raised” American voltages are available; to use them, set the style **raised voltages** or use the option **voltage=raised**. This is a version of the American-style voltage where the signs are raised to the level of the label. The label is centered between the two signs, and the position of the signs is calculated by supposing that the label itself will be pretty simple; if you have very big labels you will need to adjust the position with **voltage shift** and/or the **voltage/distance from node** properties (see section 5.2.1).



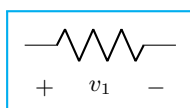
```
\begin{circuitikz}[raised voltages]
\draw (0,0) to[R, v^>=$v_1$] (2,0);
\end{circuitikz}
```



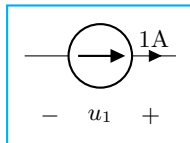
```
\begin{circuitikz}[raised voltages]
\draw (0,0) to[R, v^<=$v_1$] (2,0);
\end{circuitikz}
```



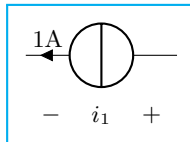
```
\begin{circuitikz}[raised voltages]
\draw (0,0) to[R, v_>=$v_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}[raised voltages]
\draw (0,0) to[R, v_<=$v_1$] (2,0);
\end{circuitikz}
```



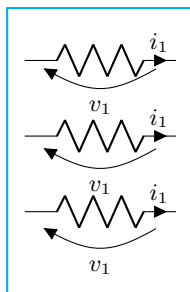
```
\begin{circuitikz}[american]
\ctikzset{voltage=raised}
\draw (0,0) to[I=1A, v_=$u_1$] (2,0);
\end{circuitikz}
```



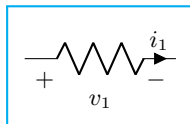
```
\begin{circuitikz}[raised voltages]
\draw (0,0) to[I<=1A, v_=$i_1$] (2,0);
\end{circuitikz}
```

5.5.5 Voltage position

It is possible to move the arrows and the plus or minus signs away from the component with the key `voltages shift` (default value is 0, which gives the standard position):

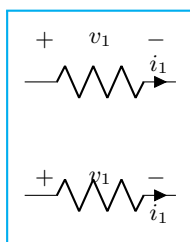


```
\begin{circuitikz}[]
\draw (0,0) to[R, v=$v_1$, i=$i_1$] (2,0);
\draw (0,-1) to[R, v=$v_1$, i=$i_1$,
voltage shift=0.5] (2,-1);
\draw (0,-2) to[R, v=$v_1$, i=$i_1$,
voltage shift=1.0] (2,-2);
\end{circuitikz}
```



```
\begin{circuitikz}[american voltages, voltage shift=0.5]
\draw (0,0) to[R, v=$v_1$, i=$i_1$] (2,0);
\end{circuitikz}
```

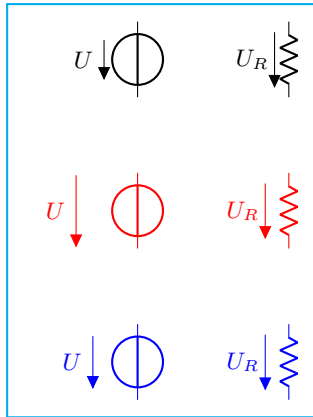
Negative values do work as expected:



```
\begin{circuitikz}[raised voltages]
\draw (0,1.5) to[R, v^=$v_1$, i=$i_1$] ++(2,0);
\draw (0,0) to[R, v^=$v_1$, i_=$i_1$,
voltage shift=-1.0] ++(2,0);
\end{circuitikz}
```

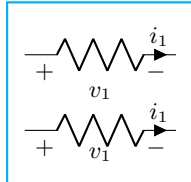
Unfortunately¹³⁶ the amount of shift given by `voltage shift` is not always the same between sources and passive bipoles, especially if the sizes of the component is very different from the default. Although this qualifies as a bug, and should be fixed in a more comprehensive way, a workaround is available with the key `voltage shift sources adjust` (default: 0.5). A smaller value is better for smaller components, as you can see in the example below.

¹³⁶see [this bug report](#).



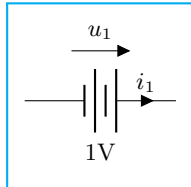
```
\newcommand{\example}[2] [] {\draw[#1] (#2)
    to [V_=$U$] ++(0, -1) (#2) ++(2,0)
    to [R,v=$U_R$] ++(0,-1);
}
\ctikzset{resistors/scale=0.55,inductors/scale=0.55,
    capacitors/scale=0.6,sources/scale=.8}
\begin{circuitikz}[circuitikz/voltage=straight,
    voltage dir=EF]
\example{0,4}
\ctikzset{voltage shift=2}
\example[color=red]{0,2}
\ctikzset{voltage shift sources adjust=0.2}
\example[color=blue]{0,0}
\end{circuitikz}
```

You can fine-tune the position of the + and - symbols and the label in independent way using `voltage/shift` (default 0.0 for the former and `voltage/american label distance` (the distance of the label from the lines of the symbols, default 1.4) for the latter.

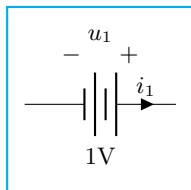


```
\begin{circuitikz}[american voltages]
\draw (0,1) to[R, v=$v_1$, i=$i_1$] ++(2,0);
% normally 1.4, make it tighter
\ctikzset{voltage/american label distance=0.5}
\draw (0,0) to[R, v=$v_1$, i=$i_1$] ++(2,0);
\end{circuitikz}
```

Notes that `american voltage` also affects batteries.

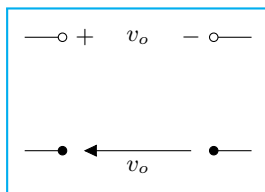


```
\begin{circuitikz}[voltage shift=0.5]
\draw (0,0) to[battery,l=1V, v=$u_1$, i=$i_1$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}[american voltages, voltage shift=0.5]
\draw (0,0) to[battery,l=1V, v=$u_1$, i=$i_1$] (2,0);
\end{circuitikz}
```

Additionally, the `open` component is treated differently; the voltage is placed in the middle of the open space¹³⁷:



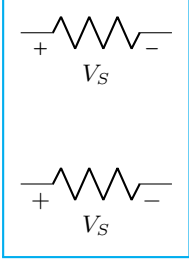
```
\begin{circuitikz}[american voltages]
\draw (0,1.5) -- ++(0.5,0)
    to[open, v=$v_o$, o-o] ++(2,0) -- ++(0.5,0);
\draw (0,0) -- ++(0.5,0)
    to[open, v=$v_o$, voltage=straight, *-] ++(2,0)
    -- ++(0.5,0);
\end{circuitikz}
```

If you want or need to maintain the old behavior for `open` voltage, you can set the key `open voltage position` to `legacy` (the default is the new behavior, which corresponds to the value `center`).

¹³⁷Since v1.1.2, thank to an [issue opened by user rhandley on GitHub](#).

5.5.6 American voltages customization

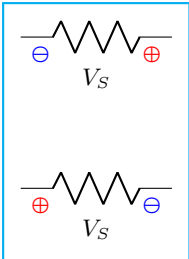
Since 0.9.0, you can change the font¹³⁸ used by the `american voltages` style, by setting to something different from nothing the key `voltage/american font` (default: nothing, using the current font) style:



```
\begin{circuitikz}[american]
\begin{scope}
\ctikzset{voltage/american font=\tiny\boldmath}
\draw (0,0) to[R,v=$V\_S$] ++(2,0);
\end{scope}
\draw (0,-2) to[R,v=$V\_S$] ++(2,0);
\end{circuitikz}
```

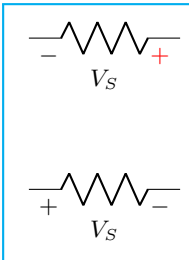
Also, if you want to change the symbols (sometimes just the $+$ sign is drawn, for example, or for highlighting something), using the keys `voltage/american plus` and `voltage/american minus` (default $++$ and $--$) style.

Notice that the definition of the minus sign is not simply $--$ because in most font (but not Computer Modern!) the size of the bounding box for the mathematical plus or minus are different. The `\vphantom` forces the vertical size of the minus sign to be the same as the plus sign.¹³⁹



```
\begin{circuitikz}[american]
\ctikzset{voltage/american font=\scriptsize\boldmath}
\ctikzset{voltage/american plus=\textcolor{red}{\oplus}}
\ctikzset{voltage/american minus=\textcolor{blue}{\ominus}}
\draw (0,0) to[R,v_>=$V\_S$] ++(2,0);
\draw (0,-2) to[R,v_<=$V\_S$] ++(2,0);
\end{circuitikz}
```

This could be especially useful if you define a style, to use like this:



```
\tikzset{red plus/.style={
circuitikz/voltage/american plus=\textcolor{red}{++},
}}
\begin{circuitikz}[american]
\draw (0,0) to[R,v_>=$V\_S$, red plus] ++(2,0);
\draw (0,-2) to[R,v_<=$V\_S$] ++(2,0);
\end{circuitikz}
```

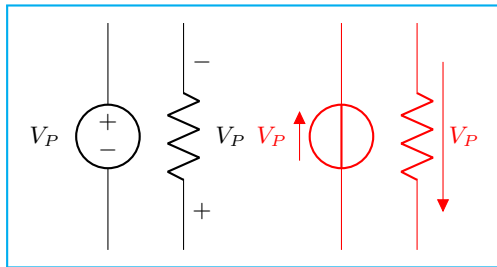
5.5.7 Combining different styles

Due to an historical hiccup, you need to be careful if you want to mix styles, like for example having `american` styled components and straight voltages (which are basically `european` style, at least in `CircuitikZ`). The problem is that the order of style parameters can change the output¹⁴⁰ as you can see in the following example, where in the red case the voltage generator shape reverted to the `european` one.

¹³⁸There was a bug before, noticed by the user [dzereb](#) on [tex.stackexchange.com](#) which made the symbols using different fonts in a basically random way. In the same page, user [campa](#) found the problem. Thanks!

¹³⁹Changed in v1.6.3, you can look at [this issue on GitHub](#) for more details.

¹⁴⁰thanks to Stack Exchange user [Mads P Olesen](#) for noticing.



```
\begin{circuitikz}[straight voltages, american]
  \draw (0,0) to [V, v=$V_P$] ++(0,3);
  \draw (1,0) to [R, v=$V_P$] ++(0,3);
\end{circuitikz}\color{red}%
\begin{circuitikz}[american, straight voltages]
  \draw (0,0) to [V, v=$V_P$] ++(0,3);
  \draw (1,0) to [R, v=$V_P$] ++(0,3);
\end{circuitikz}
```

This is arguably a bug, but fixing it (separating the voltage generator shapes from the voltage style) would break havoc with older circuits, so this will not be fixed for now.

5.6 Changing the style of labels, voltages, and other text ornaments

Since version 0.9.5, it is possible to change the style of bipole text ornaments (labels, annotations, voltages etc) by using the appropriate styles or keys. The basic style applied to the text is defined in the `/tikz/circuitikz` key directory and applied to every node that contains the text; you can also change them locally by using the `tikz` direct keys in local scopes.

For example, you can make all annotations small by using:

```
\ctikzset{bipole annotation style/.style={font=\small}}
```

And/or change (override) the setting in one specific bipole using:

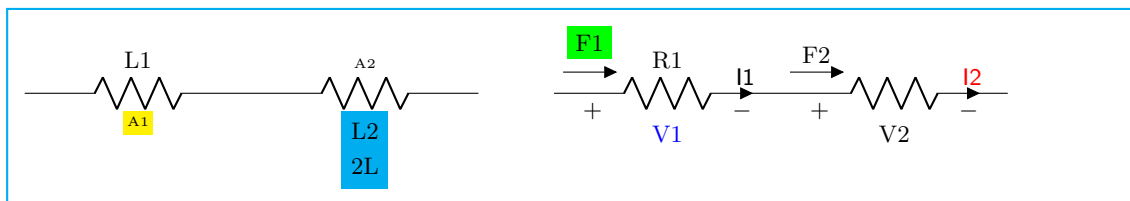
```
...to[bipole annotation style={color=red}, R, a={Red note}]...
```

where the annotation will be in normal font (it has been reset!) and red, or append to the style:

```
...to[bipole annotation append style={color=red}, R, a={Red small note}]...
```

Caveat: you have to put the style changing key at the start of the `to` arguments to have any effect¹⁴¹.

The available styles and commands are `bipole label style`, `bipole annotation style`, `bipole voltage style`, `bipole current style`, and `bipole flow style`. The following example shows a bit of everything.

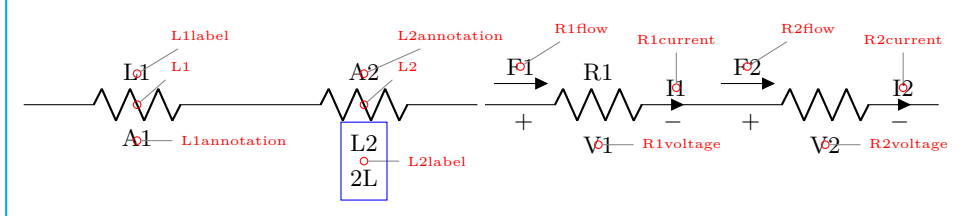


```
\begin{circuitikz}[american]
  \ctikzset{bipole annotation style/.style={font=\tiny}}
  \ctikzset{bipole current style/.style={font=\small\sffamily}}
  \draw (0,0) to [bipole annotation append style={fill=yellow}, R=L1, a=A1] ++(3,0)
    to [bipole label style={fill=cyan}, R, l2=L2 and 2L, a^=A2] ++(3,0);
  \draw (7,0) to [bipole voltage style={color=blue},
    bipole flow style={fill=green, outer sep=5pt},
    R=R1, v=V1, i=I1, f^=F1] ++(3,0)
    to [bipole current append style={color=red}, R, v<=V2, i^=I2, f^=F2] ++(3,0);
\end{circuitikz}
```

¹⁴¹No, I do not know why. Hints and fixes are welcome.

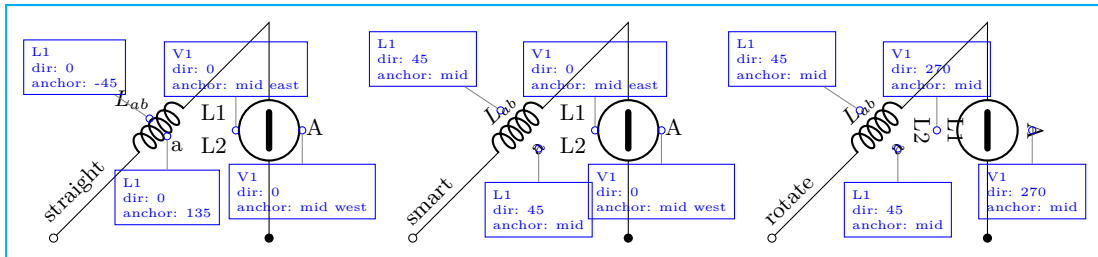
5.7 Accessing labels text nodes

Since 0.9.5, you can access all the labels nodes¹⁴² using special node names. So, if you use **name** to give a name to the bipole node, you can also access the following nodes: **namelabel** (notice: no space nor any other symbol between **name** and **label**!), **nameannotation**, **namevoltage**, **namecurrent** and **nameflow**. Notice that the node names are available only if the bipole has an anchor or an annotation, of course.



```
\newcommand{\marknode}[2][45]{%
  \node[circle, draw, red, inner sep=1pt,
    pin={\tiny\red #1:#2}] at (#2.center) {};
}
\begin{circuitikz}[american]
  \draw (0,0) to [R=L1, a=A1, name=L1] ++(3,0)
    to [R, l2=L2 and 2L, a^=A2, name=L2] ++(3,0);
  \marknode{L1} \marknode{L1label} \marknode{0}{L1annotation}
  \marknode{L2} \marknode{0}{L2label} \marknode{L2annotation}
  \draw[blue] (L2label.south west) rectangle (L2label.north east);
  \draw (6.1,0) to [R=R1, v=V1, i=I1, f^=F1, name=R1] ++(3,0)
    to [R, v=V2, i^=I2, f^=F2, name=R2] ++(3,0);
  \marknode{0}{R1voltage} \marknode{0}{R2voltage} \marknode{90}{R1current}
  \marknode{90}{R2current} \marknode{R1flow} \marknode{R2flow}
\end{circuitikz}
```

If you want to have more access to the label positioning algorithm, since 1.2.5 you can access the label rotation using the command `\ctikzgetdirection{nodename}` (where node name is for example `L1label` or `L2annotation`), and the anchor used for positioning the node as `\ctikzgetanchor{component label}{type}`, where *component label* is, for example, `L1` and *type* is either `label` or `annotation` (notice that the syntax is slightly different, for implementation reasons). Those values are available only if the dipole declares a `l` or a `key`; if you want them without any label you need to declare a blank one (like for example `l=`). The following example gives an idea of the values of those macros for the three types of label positioning strategies.



```
\newcommand{\marklabann}[3][45]{% [angle] {node label} {type: label or annotation}
  \node[circle, draw, blue, inner sep=1pt,
    pin={\tiny\blue #1:#2 \\\ dir: \ctikzgetdirection{#2#3} \\\
      anchor: \ctikzgetanchor{#2}{#3}}] at (#2#3.\ctikzgetanchor{#2}{#3}) {};
}
\begin{tikzpicture}[scale=0.95, transform shape]
```

¹⁴²The access to labels and annotations was present before, but not documented.

```

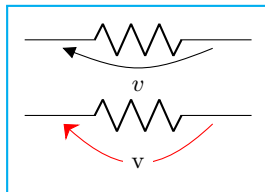
\foreach \style/\xdelta in {straight/0, smart/5, rotate/10} {
\begin{scope}[xshift=\xdelta cm]
\ctikzset{label/align = \style}
\draw (0,0) node[above right, rotate=45]{\style}
to[L, o-, l=$L_{ab}$, v, name=L1, a=a] ++(3,3)
to[ceV, -*, v, name=V1, l2=L1 and L2, a^=A] ++(0,-3);
\marklabann[135]{L1}{label}
\marklabann[-90]{L1}{annotation}
\marklabann[90]{V1}{label}
\marklabann[-90]{V1}{annotation}
\end{scope}}
\end{tikzpicture}

```

5.8 Advanced voltages, currents and flows

Since version 1.2.1¹⁴³, it is possible to access the anchors of the “ornaments” — voltage, current and flows, together with some additional information that makes it possible to personalize them. Normally, voltages and flow and currents are drawn into the path of the bipoles, so that it is not possible, for example, to change the line type or color of the arrows, or the type of arrows¹⁴⁴. Access to the anchors allows you to do all these things, and more.

For example, you can do something like this:

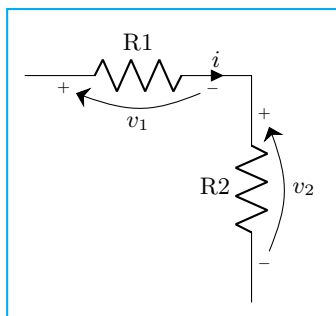


```

\begin{circuitikz}[]
\draw (0,1) to[R, v=$v$] ++(3,0);
\draw (0,0) to[R, v, name=R, voltage/bump b=3] ++(3,0);
\draw [thin, red, -{Stealth[width=8pt]}, ]
(R-Vfrom) .. controls (R-Vcont1) and (R-Vcont2).. (R-Vto)
node [black, pos=0.5, fill=white]{v};
\end{circuitikz}

```

Or, for example, to have a different voltage style; normally you would define a macro (see 5.5.6 to understand the `\vphantom`).



```

\begin{circuitikz}[voltage shift=0.5]
\def\eurVPM#1#2{% node, label
\draw [thin, -{Stealth[width=8pt]}, shorten >=5pt,
shorten <=5pt] (#1-Vfrom) node[font=\tiny]{$\vphantom{+}-}$}
.. controls (#1-Vcont1) and (#1-Vcont2)..
(#1-Vto) node[font=\tiny]{$+}$}
node[pos=0.5, anchor=\ctikzgetanchor{#1}{Vlab}]{#2};}
\draw (0,0) to[R=R1, name=R1, v, i=$i$] ++(3,0)
to[R, l=R2, v^, name=R2] ++(0,-3);
\eurVPM{R1}{v_1} \eurVPM{R2}{v_2}
\end{circuitikz}

```

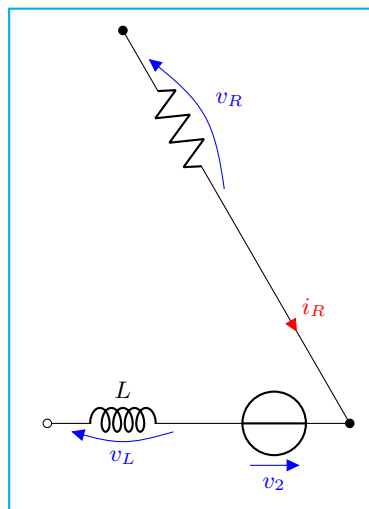
5.8.1 Using standard label with custom symbols

Since v1.4.1 you can also keep the voltage, current and flow labels and suppress the output of the symbols (arrows or plus/minus depending on the style) with the keys `no v symbols`, `no i symbols`, `no f symbols` (there are also the corresponding `v symbols`, `i symbols` and `f symbols` in case you want to switch the

¹⁴³some options have been added in v1.4.1

¹⁴⁴in regular voltages, the arrows are not real TikZ arrows, but the auxiliary arrow shapes of CircuiTikZ

behavior off/on globally). This for example simplify an often requested feature, like having all the current in one color and the voltages in another one, which is not possible natively because the arrows are part of the same path. One possible implementation of that is the following one:



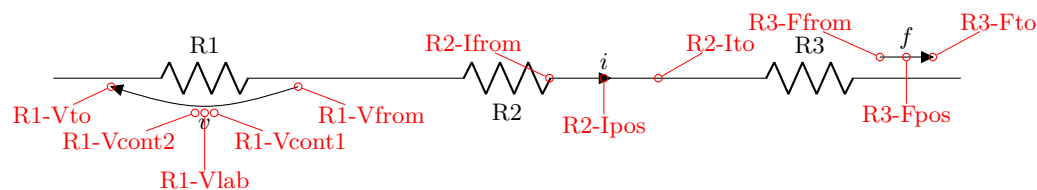
```
\newcommand{\iarronly}[1]{% name
  \node [curarrow, color=red, anchor=center,
    rotate=\ctikzgetdirection{#1-Iarrow}] at (#1-Ipos) {};
}
\newcommand{\varronly}[1]{% name
  \draw [color=blue] (#1-Vfrom) .. controls (#1-Vcont1)
    and (#1-Vcont2).. (#1-Vto) node [curarrow,
    sloped, anchor=tip, allow upside down,pos=1]{};
}
\begin{circuitikz}[]
  \ctikzset{!vi/.style={no v symbols, no i symbols}}
  \ctikzset{bipole voltage style/.style={color=blue},
    bipole current style/.style={color=red}}
  \draw (120:6) to[R, *-, name=R, v^=$v_R$, !vi]
    (120:3) to[short, i=$i_R$, name=SR, !vi] (0,0);
  \draw (180:4) to[L, o-, l=$L$, name=L2, v=$v_L$, !vi]
    (180:2) to[V, -*, name=V2, v_=$v_2$, !vi] (0:0);
  \iarronly{SR}\varronly{R}\varronly{L2}\varronly{V2}
\end{circuitikz}
```

5.8.2 Activating the anchors

You will have access to the anchors for voltages, currents and flows when, in the bipole, you have a **v**, **i**, **f** specification (one or more of them). If you have a **name** key, to give the bipole a name, they will be easily accessible. Without a **name** key, a temporary name is generated, to be used by the “automatic v-i-f” mechanism (see 5.8.5). Otherwise, the anchors and the associated functions are not defined. To suppress the normal output of the **v**, **i**, **f** keys, you can use such keys without any argument, like in the previous example; notice that the **_** and **^** modifiers work as expected.

The following line of resistors has been drawn with the following commands; it is used to show the name of the available anchors.

```
\draw (0,0) to[R=R1, v=$v$, name=R1] ++(4,0)
  to[R, l_=$R2$, i=$i$, name=R2] ++(4,0)
  to[R=R3, f=$f$, name=R3] ++(4,0);
```



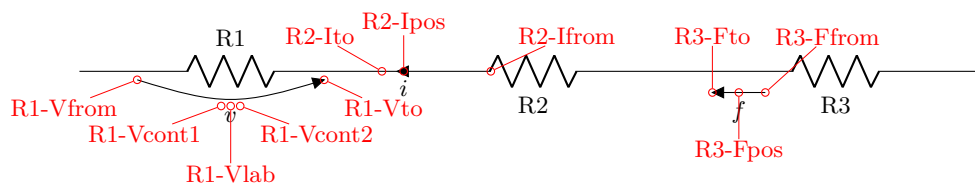
The meaning of the anchors is the following:

- **Vfrom** and **Vto** are the main points where the voltage information is given: start and end point of the arrow, or position of the **+** or **-** sign. This is the same for the **Ffrom** or **Fto** anchors for flows; for inline currents, the corresponding **Ifrom** and **Ito** mark the wire segment where the arrowhead is positioned (at the specified **current/distance** fraction. The direction of the arrow is available using the auxiliary macro `\ctikzgetdirection` (see below).
- **Vcont1** and **Vcont2** are the control points for the curved arrow (see the examples above); in the case of straight arrows or american-style voltages, they are set at the midpoint between **Vfrom** and **Vto**.

- **Vlab** is where the text label for the voltage is normally positioned. The anchor used for such label is available using the auxiliary macro `\ctikzgetanchor` (see below). Consider the default `inner sep=2pt` of voltage label nodes.
- **Ipos** and **Fpos** are the position for the arrowhead or the small flow arrow (which is a `curarrow` or `flowarrow` node normally) is positioned, respectively. The label is then added to the correct side of it using the anchor available via `\ctikzgetanchor` (see below, 5.8.3). In this case, the exact position of the label is not available if you do not position the element, for this there is no **Flab** or **Ilab** coordinate; you have to use the **Fpos** and **Ipos** coordinate with the corresponding **Ilab** and **Flab** anchors.

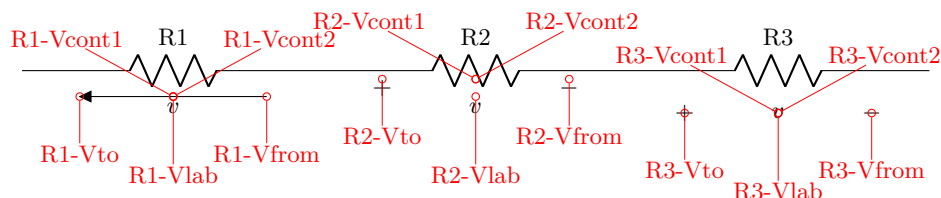
Changing the options of the elements will change the anchors accordingly:

```
\ctikzset{current/distance=0.2}
\draw (0,0) to[R=R1, v>=$v$, name=R1] ++(4,0)
      to[R, l_=R2, i<_=$i$, name=R2] ++(4,0)
      to[R, l_=R3, f<_=$f$, name=R3] ++(4,0);
```



Obviously, the anchors follow the voltage style you choose:

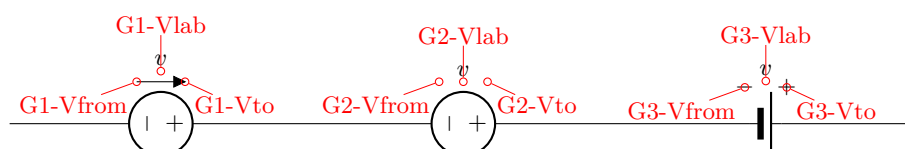
```
\draw (0,0) to[R=R1, v=$v$, name=R1, voltage=straight] ++(4,0)
      to[R=R2, v=$v$, name=R2, voltage=american] ++(4,0)
      to[R=R3, v=$v$, name=R3, voltage=raised] ++(4,0);
```



Notice the position of the control points, as well as the fact that the anchor available with `\ctikzgetanchor` is applied to **Vfrom** and **Vto** symbols, too.

Finally, as ever, generators are treated differently, but you have all your anchors too.

```
\ctikzset{american}
\draw (0,0) to[V=$v$, name=G1, voltage=european] ++(4,0)
      to[V=$v$, v=$v$, name=G2, voltage=american] ++(4,0)
      to[battery2, v=$v$, name=G3, voltage=raised] ++(4,0);
```

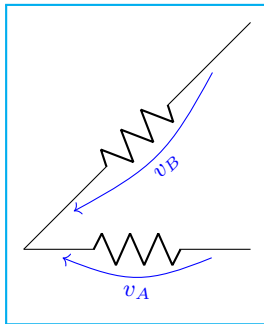


5.8.3 Auxiliary information

When the anchors are activated, there are additional macros that you can use:

- `\ctikzgetanchor{<name>}{<anchor>}`: *name* is the name of the bipole, and *anchor* can be `Vlab`, `Flab` or `Ilab`. This macro expands to the normal anchor position (something like `north`, `south` or `west`). Notice that if you have not activated the corresponding anchor, the content of this macro is not specified. It could be equivalent to `\relax` (basically, empty) or contains the anchor of a bipole with the same name from another drawing — it's a global macro like the coordinates.
- `\ctikzgetdirection{<name>}`: a number which is the direction of the *named* bipole.
- `\ctikzgetdirection{<name>-Iarrow}`: a number which is the direction of the current arrow requested for the *named* bipole; using `<name>-Farrow` you get the same information for flow arrows.

For example, you could like the voltage label oriented with the bipole:



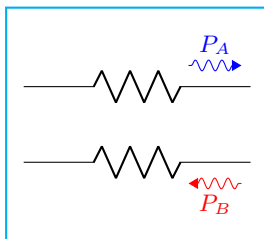
```
\begin{circuitikz}[]
  \def\myvv#1#2{%
    \draw [thin, blue, ->,]
      (#1-Vfrom) .. controls (#1-Vcont1) and (#1-Vcont2).. (#1-Vto)
    node [pos=0.5, below,
          rotate=\ctikzgetdirection{#1}] at (#1-Vlab) {#2}; }
  \draw (0,0) to[R, v, name=A] ++(3,0);
  \draw (0,0) to[R, v, name=B] ++(3,3);
  \myvv{A}{v_A}\myvv{B}{v_B}
\end{circuitikz}
```

Or you could use the anchor to substitute the flow with a fancy one and still position the label automatically; suppose you have the following definition in your preamble (see TikZ manual, “[Path decorations](#)”):

```
% requires \usetikzlibrary{decorations, decorations.pathmorphing}
\tikzset{%
  lray/.style={decorate, decoration={
    snake, amplitude=2pt,pre length=1pt,post length=2pt, segment length=5pt,},
    -Triangle,
  }}

```

You can then define a kind of “power flow” style:

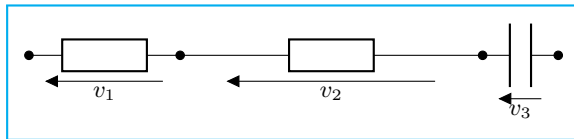


```
\begin{circuitikz}[]
  \newcommand\myff[3][blue]{% [opt: color] node label
    \draw [lray, #1, ] (#2-Ffrom) -- (#2-Fto)
    node [anchor=\ctikzgetanchor{#2}{Flab}, inner sep=4pt]
      at (#2-Fpos) {#3};}
  \draw (0,1) to[R, f, name=A] ++(3,0);
  \draw (0,0) to[R, f_<, name=B] ++(3,0);
  \myff{A}{P_A}\myff[red]{B}{P_B}
\end{circuitikz}
```

5.8.4 Fixed voltage arrows: an example of advanced voltage usage

An interesting application of the advanced voltage is to have fixed length straight voltage arrows.¹⁴⁵ The normal voltage arrows length depends not on the component length but on the node distance (this is the behavior since when the voltages were first introduced, so it can't be changed).

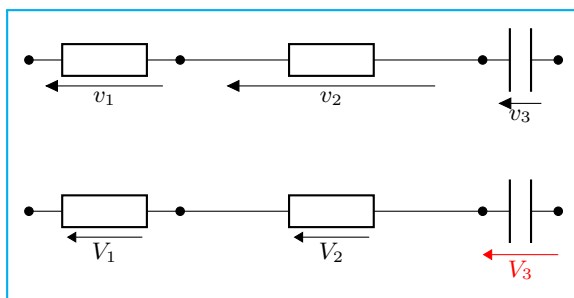
¹⁴⁵This was suggested by users Franklin and Zarko in [a question on tex.stackexchange.com](#)



```
\begin{circuitikz}[european,]
\ctikzset{voltage=straight}
\draw (0,0) to[R,v=$v_1$,*-*] ++(2,0) to[R, v<=$v_2$] ++(4,0) to[C, *-, v=$v_3$] ++(1,0);
\end{circuitikz}
```

Using the advanced voltage interface mechanism, you can for example design voltages that are of fixed lengths; in the example below the new `xparse` method for defining commands is used, so that we can have a couple of different optional arguments:

```
\NewDocumentCommand{\fixedvlen}{0{0.5cm} m m 0{}}{% [semilength]{node}{label}[extra options]
% get the center of the standard arrow
\coordinate (#2-Vcenter) at ($(#2-Vfrom)!0.5!(#2-Vto)$);
% draw an arrow of a fixed size around that center and on the same line
\draw[-Triangle, #4] ($(#2-Vcenter)!#1!(#2-Vfrom)$) -- ($(#2-Vcenter)!#1!(#2-Vto)$);
% position the label as in the normal voltages (inner sep of voltage labels is 2pt per default)
\node[anchor=\ctikzgetanchor{#2}{Vlab}, inner sep=2pt, #4] at (#2-Vlab) {#3};
}
```



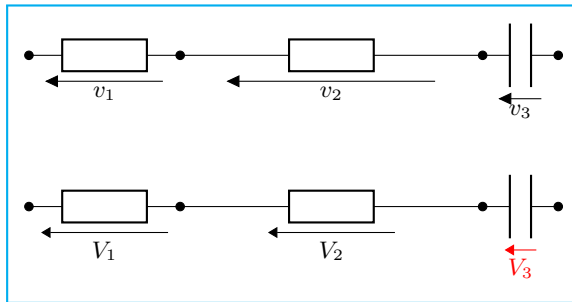
```
\begin{circuitikz}[european,]
\ctikzset{voltage=straight}
\draw (0,2) to[R,v=$v_1$,*-*] ++(2,0) to[R, v<=$v_2$] ++(4,0) to[C, *-, v=$v_3$] ++(1,0);
\draw (0,0) to[R,v=,name=v1,*-*] ++(2,0) to[R, v<, name=v2] ++(4,0) to[C, *-, v, name=v3] ++(1,0);
\fixedvlen{v1}{v_1$}
\fixedvlen{v2}{v_2$}
\fixedvlen{v3}{v_3$}[red]
\end{circuitikz}
```

Notice that with a coherent naming you can use a `\foreach` loop for the last three lines.

You can also notice that the arrow is not exactly the same as other arrows in the circuit; if you want them to be exactly the same, you can use a trick to get the default CircuiTikZ arrow size — please look at [this answer by Romano on tex.stackexchange.com](https://tex.stackexchange.com/a/111111).

Another possibility is to have the arrow length based on the length of the component; for example you can use this code:

```
\NewDocumentCommand{\compvlen}{0{1.5} m m 0{}}{% [relative length]{node}{label}[extra options]
% get the center of the standard arrow
\coordinate (#2-Vcenter) at ($(#2-Vfrom)!0.5!(#2-Vto)$);
% draw an arrow of a size proportional to the component length
% around that center and on the same line
% the component length is calculated using the let...in with the left and right anchors
% and multiplied by the relative length
\draw[-Triangle, #4] let \p1=(#2.left), \p2=(#2.right), \n1={0.5*#1*vecLen(\x2-\x1,\y2-\y1)}
in ($(#2-Vcenter)!#1!(#2-Vfrom)$) -- ($(#2-Vcenter)!#1!(#2-Vto)$);
% position the label as in the normal voltages
\node[anchor=\ctikzgetanchor{#2}{Vlab}, #4] at (#2-Vlab) {#3};
}
```

```
\begin{circuitikz}[european,]
  \ctikzset{voltage=straight}
  \draw (0,2) to[R,v=$v_1$,*-*] ++(2,0) to[R,v<=$v_2$] ++(4,0) to[C,*-*,v=$v_3$] ++(1,0);
  \draw (0,0) to[R,v=,name=v1,*-*] ++(2,0) to[R,v<=, name=v2] ++(4,0) to[C,*-*,v, name=v3] ++(1,0);
  \compvlen{v1}{$V_1$}
  \compvlen{v2}{$V_2$}
  \compvlen{v3}{$V_3$}[red]
\end{circuitikz}
```

5.8.5 Automatic advanced voltages, current and flows (experimental)

Since v1.8.5, a new mechanism has been added¹⁴⁶ to simplify the use of advanced voltages, currents, and flows in circuits. The idea is to avoid the manual invocation of the macros that draw the appropriate symbols and/or labels at the end of a path or picture.

*Please notice that this feature is **experimental** — at least for a couple of releases. The behavior, macro names, and restrictions can change in the next versions.*

The core addition consists of a new `\ctikzset` key, `vif queue add`, which takes four arguments: a macro *name* and three further arguments¹⁴⁷, which can contain arbitrary content. For example, if you use the following key in a `to` bipole:

```
to[R, ..., vif queue add={mymacro}{$\alpha$}{red}{foo}]
```

the effect of the key is to add a macro call of the form:

```
\mymacro{nodename}{$\alpha$}{red}{foo}
```

to an internal list of “macros to be executed”. The *nodename* is added automatically; if no explicit **name** key is given, a temporary name is generated.

It is important that the `vif queue add` key appears last, or at least after any `v`, `i`, `f`, or `name` key in the `to[...]` options. Moreover, its correct operation is guaranteed only when the `to` command places a CircuiTikZ bipole (in other words, it should not be expected to work with a basic TikZ `to`, such as `to[bend left, ...]`), **and** when the voltages, current and/or flow anchors are activated (that is, a `v`, `i` or `f` key is present).

In the case you want to delegate to your custom macro only the drawing of the symbols, but let the standard mechanism place the label, you can still use the `no v symbols` and similar (see 5.8.1). In that case you need to use an explicit `v=label` to set the value label; notice that this also sets the default “direction” of the symbols (that you can override with `v>=...`, `v^=...` or similar, of course). If you need to modify the label value without affecting a previously issued `v...=...` command, you can use the key `v label` (and the corresponding `i label`, `f label`) that changes *only* the value of the label, without any other action (not even activating the anchors, that is).

¹⁴⁶As with all the advanced v-i-f features, this was added by Romano; see the discussion [on GitHub](#).

¹⁴⁷The idea is that this way you can pass to the custom macro a label, options for the labels, and options for the drawing of the symbols; however they can be whatever the user decides.

The stored macro will be replayed automatically at the end of the environment, in the same order in which they were added to the queue, at least if you are using a recent enough version of L^AT_EX; if you are using an older version or a different format (like for example ConT_EXt) you need to add the key `use auto vif` in the options of the `tikzpicture` environment. If you forget it, a warning will be issued.

You also can “execute” the stored commands with the `\ctikzprocessvif` macro if you need to draw them before the end of the environment. This can be important if you define some name in your macros and you need to use them in another part of the circuit; by default all the stored macros are executed at the end of the environment. The call to `\ctikzprocessvif` “clear” the queue, so consecutive calls have no effect.

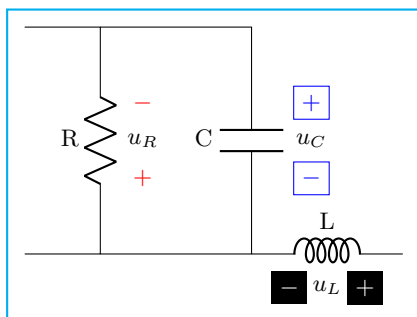
Naturally, you must define the macro(s) that will be called, such as the `\mymacro` shown above. The macro **must** have three arguments, like this:

```
\newcommand{\mymacro}[4]{% try not to add spurious spaces
  % #1 -> node name
  % #2 -> first user argument
  % #3 -> second user argument
  ... code here ...
}
```

There is little sense in using the `vif queue add` key explicitly; it is better hidden behind more specific user-level keys, as shown in the following examples. This also helps remove some arguments when they are not needed.

For example, let’s suppose you like the `raised` voltage notation, but you want the plus and minus signs at a fixed distance from the label, independently on the size of the branch where the component is, and moreover you want to be able to change the color or the style of the signs. We can setup the voltage like this:

```
%% The voltage label (the text) will be managed normally by v...={ }
%% We need just one parameter to specify the color/style of the signs; we
%% set the other argument to nothing.
\ctikzset{my v/.style = {
  voltage=raised, no v symbols, vif queue add={placesigns}{#1}{ }
}}
%% my V: standard labels, drawing the signs at fixed distance.
%% The macro ALWAYS receive 4 arguments. The first is the node name
%% (that is, the anchor prefix) and the second, in this case, the color/style.
\newcommand{\placesigns}[4]{% place the +- signs at a fixed distance
  \path ($(#1voltage.center)!0.5cm!(#1-Vto)$) node[#2]{$+$};
  \path ($(#1voltage.center)!0.5cm!(#1-Vfrom)$) node[#2]{$-$};
}
```



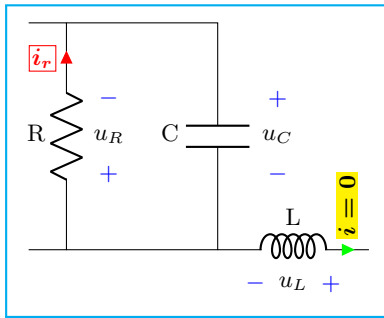
```
\begin{circuitikz}
  \draw (3,0) -- ++(1,0) coordinate(Rbot)
    to[R=R, name=vR, v={\$u_R\$}, my v={red}]
    ++(0,+3) -- ++(-1,0);
  \draw (Rbot) -- ++(2,0) coordinate(Cbot)
    to[C=C, name=vC, v>={\$u_C\$}, my v={draw,blue}]
    ++(0,+3) -- ++(-2,0);
  \draw (Cbot) to [L=L, v_>={\$u_L\$},
    my v={white,fill=black}] ++(2,0);
\end{circuitikz}
```

If you have an old L^AT_EX or are using ConT_EXt, you must add the `use auto vif` to the options of the environment; the option will not have any negative effect if it is not needed.

```
\begin{tikzpicture}[use auto vif]
... as before
\end{tikzpicture}
```

Suppose now we want to change the current indication so that we can typeset the label using `\boldmath`, changing the style of the label and the color of the arrow. For example, we could define the following:

```
% arguments: label, style for the arrow, style for the label
\tikzset{my i/.style n args = {3}{vif queue add={docurrent}{#1}{#2}{#3}}}
\newcommand{\docurrent}[4]{
  \node [curarrow, anchor=center, rotate=\ctikzgetdirection{#1-Iarrow},
    #3] at (#1-Ipos) {};
  \node [anchor=\ctikzgetanchor{#1}{Ilab}, outer sep=4pt, inner sep=1pt,
    #4] at (#1-Ipos) {\boldmath #2}; %the double {} are needed!!!
}
```

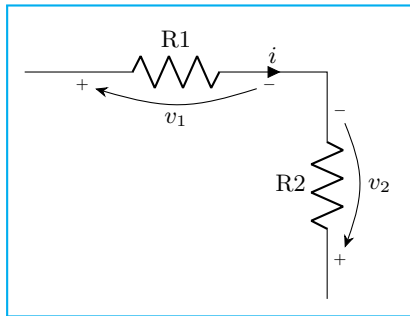


```
\begin{circuitikz}
% set a default for my v
\tikzset{my v/.default=blue}
\draw (3,0) -- ++(0.5,0) coordinate(Rbot)
to[R=R, name=vR, v={\$u_R\$}, my v,
i, my i={\$i_r\$}{red}{draw, color=red}]
++(0,+3) -- ++(-.5,0);
\draw (Rbot) -- ++(2,0) coordinate(Cbot)
to[C=C, name=vC, v>={\$u_C\$}, my v]
++(0,+3) -- ++(-2,0);
\draw (Cbot) to [L=L, v>={\$u_L\$}, my v, i,
my i={\rotatebox{90}{\$i=0\$}}{green}{fill=yellow}]
++(2,0);
\end{circuitikz}
```

5.8.6 Creating new voltage/current/flows commands

Sometimes, the need of using the syntax `v, my v=...` can result tedious; unfortunately, it is not possible to add the `v` to the key `my v` because it will reset the direction/position of the key. To avoid this problem, you can use the new command `\ctikzactivatevoltagedirections` (and their sibling `\ctikzactivatecurrentdirections` and `\ctikzactivateflowdirections`) which will create the full set of personalized voltage (or current or flow) commands with the appended directions. For example, this is how we can reimplement the “European arrow with signs” shown in the previous section:

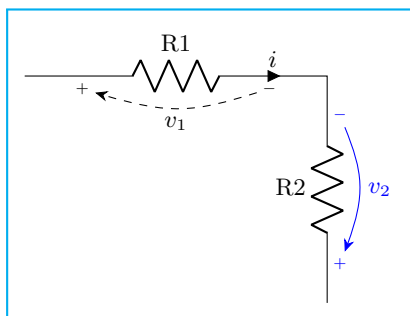
```
\newcommand\eurVPM[4]{% node, label, two ignored arguments
  \draw [thin, -{Stealth[width=4pt]}, shorten >=5pt, shorten <=5pt]
    (#1-Vfrom) node[font=\tiny]{\$ \vphantom{+}-\$}
    .. controls (#1-Vcont1) and (#1-Vcont2)..
    (#1-Vto) node[font=\tiny]{\$+ \$}
    node[pos=0.5, anchor=\ctikzgetanchor{#1}{Vlab}]{#2};
}
\tikzset{vpm/.style={european voltages, v, vif queue add={eurVPM}{#1}{}}}
% do magic with vpm
\ctikzactivatevoltagedirections{vpm}
```



```
\begin{circuitikz}[voltage shift=0.5]
\draw (0,0)
to[R=R1, name=R1, i=$i$, vpm=$v_1$] ++(4,0)
to[R, l_*=R2, vpm^>=$v_2$] ++(0,-3);
\end{circuitikz}
```

You can use this trick even with styles with more than one argument, but then you have to be sure to always add both argument *braced*, otherwise very strange errors will be generated.

```
\newcommand\eurVPMstyled[4]{% node, label, arrow style, one ignored arguments
\draw [thin, -{Stealth[width=4pt]}, shorten >=5pt, shorten <=5pt, #3]
(#1-Vfrom) node[font=\tiny]{$\vphantom{+}-}$}
.. controls (#1-Vcont1) and (#1-Vcont2)..
(#1-Vto) node[font=\tiny]{$+$}
node[pos=0.5, anchor=\ctikzgetanchor{#1}{Vlab}]{#2};
}
\ctikzset{vpms/.style 2 args={european voltages, v,
vif queue add={eurVPMstyled}{#1}{#2}{}}}
\ctikzactivatevoltagedirections{vpms}%
```

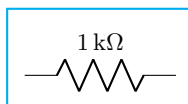


```
\begin{circuitikz}[voltage shift=0.5]
\draw (0,0)
to[R=R1, name=R1, i=$i$, vpms={v_1}{dashed}]
++(4,0)
to[R, l_*=R2, vpms^>={v_2}{blue}] ++(0,-3);
\end{circuitikz}
```

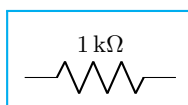
You can see a full example at [10.11](#).

5.9 Integration with siunitx

If the option `siunitx` is active¹⁴⁸, then the following are equivalent (this will **not** work in ConT_EXt, it has been disabled in upstream ConT_EXt, in favor of *its own units module*):



```
\begin{circuitikz}
\draw (0,0) to[R, l=1<\kilo\ohm>] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
\draw (0,0) to[R, l=$\SI{1}{\kilo\ohm}$] (2,0);
\end{circuitikz}
```

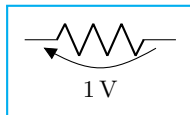
¹⁴⁸This option is still experimental — personally (Romano) I would advise using the normal `\SI{}{}` syntax, or the `\qty{}{}` one for `siunitx v3` and newer.



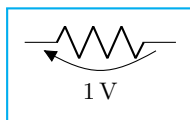
```
\begin{circuitikz}
  \draw (0,0) to[R, i=1<\milli\ampere>] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, i=\$SI{1}{\milli\ampere}\$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, v=1<\volt>] (2,0);
\end{circuitikz}
```

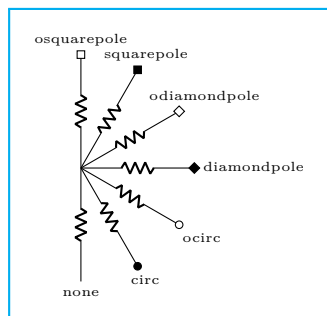


```
\begin{circuitikz}
  \draw (0,0) to[R, v=\$SI{1}{\volt}\$] (2,0);
\end{circuitikz}
```

6 Using bipoles in circuits

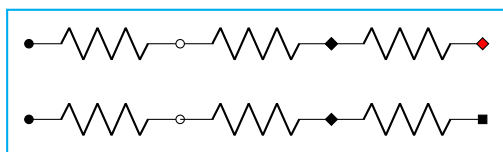
6.1 Nodes (also called poles)

You can add nodes to the bipoles, positioned at the coordinates surrounding the component. The general style to use is `bipole nodes={start}{stop}`, where `start` and `stop` are the nodes — to be chosen between `none`, `circ`, `ocirc`, `squarepole`, `osquarepole`, `diamondpole`, `odiamondpole` and `rectfill`¹⁴⁹ (see section 4.12).



```
\begin{circuitikz}
\ctikzset{bipoles/length=.5cm, nodes width=0.1}%small
components, big nodes
\foreach \a/\p [evaluate=\a as \b using (\a+180)] in
{-90/none, -60/circ, -30/ocirc, 0/diamondpole, 30/
odiamondpole, 60/squarepole, 90/osquarepole}
\draw (0,0) to[R, bipole nodes={none}{\p}] ++(\a:1.5)
node[font=\tiny, anchor=\b]{\p};
\end{circuitikz}
```

These bipole nodes are added *after* any single path is drawn, as every node in TikZ — this is the reason why they are always filled (with the main color the normal nodes, with white the open ones), in order to “hide” the wire below. You can override the fill color if you want; but notice that if you draw things in two different paths, you will have “strange” results; notice that in the second line of resistors the second wire is starting from the center of the white `ocirc` of the previous path.



```
\begin{circuitikz}
\draw (0,0) to[R, *-o] ++(2,0) to[R, -d] ++(2,0)
to[R, bipole nodes={diamondpole}{odiamondpole, fill=red}] ++(2,0);
\draw (0,-1) to[R, *-o] ++(2,0) ;
\draw (2,-1) to[R, -d] ++(2,0) to[R, bipole nodes={none}{squarepole}] ++(2,0);
\end{circuitikz}
```

You can define shortcuts for the `bipole nodes` you use most; for example if you want a shortcut for a bipole with open square node in red in the right side you can:



```
\begin{circuitikz}
\ctikzset{-s/.style = {bipole nodes={none}{osquarepole, fill=red}}}
\draw (0,0) to[R, -s] ++(2,0);
\end{circuitikz}
```

There are several predefined shorthand as the above; in the following pages you can see all of them.



```
\begin{circuitikz}
\draw (0,0) to[R, o-o] (2,0);
\end{circuitikz}
```

¹⁴⁹You can use other shapes too, but at your own risk... Moreover, notice that `none` is not really a node, just a special word used to say “do not put any node here”.



```
\begin{circuitikz}
  \draw (0,0) to[R, -o] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, o-] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, *-*] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, -*] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, *-] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, d-d] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, -d] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, d-] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, o-*] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, *-o] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, o-d] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, d-o] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[R, *-d] (2,0);
\end{circuitikz}
```



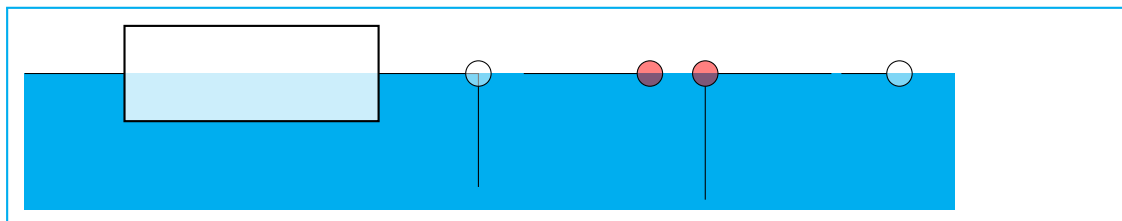
```
\begin{circuitikz}
  \draw (0,0) to[R, d-*] (2,0);
\end{circuitikz}
```

6.1.1 Transparent poles

“Open-poles” terminals (`ocirc`, `odiamondpole`, and `osquarepole`) are normally filled with the background color at full opacity. The reason is that TikZ, when stroking a path, places and draws the nodes *after* the lines are drawn; that way the poles “white-out” the underlying lines. Clearly this works if the wires and poles are written *in the same path command*, otherwise the explicit order is respected.

Anyway, *if you know what you are doing*, you can change it with the key `poles/open fill opacity` (with `\ctikzset`) or the style `open poles opacity`. Notice that you will have artifacts if you don’t use the border anchors of the poles to connect wires, and you need to do that by hand.

Notice that in poles, the opacity is *always* selected with these keys, and it overrides the opacity of the draw commands (when not set explicitly as if it is set to 1.0, i.e., full opaque). This is because you normally do not want unfilled poles!

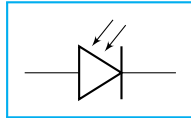


```
\begin{circuitikz}[scale=3, transform shape]
  \fill[cyan] (0,0) rectangle (4.1,-0.6);
  \ctikzset{open poles fill=red}
  \tikzset{open poles opacity=0.5}
  % automatic positioning when opacity is not 1.0 creates artifacts
  % note that the global fill opacity affects the "generic shape", but not the poles!
  % the fill color of the poles, instead, goes with the component
  \draw[fill opacity=0.8] (0,0) to[generic, fill=white, -o] ++(2,0) ---++(0,-0.5);
  % \draw (0,0) to[generic, fill=white, -o] ++(2,0) ---++(0,-0.5);
  % you have to use manual positioning
  \draw (2.2,0) -- ++(0.5,0) node[ocirc, anchor=180, fill opacity=0.5]{};
  \draw (3,0) node[ocirc, fill opacity=0.5] (B){} (B.0) ---++(0.5,0) (B.-90)
    ---++(0,-0.5);
  % maybe really useful only for terminals going out of the circuit...
  % notice that in node commands you can specify the opacity directly
  \draw (3.6,0) -- ++(0.2,0) node[ocirc, fill=white, fill opacity=0.5, anchor=180]{};
\end{circuitikz}
```

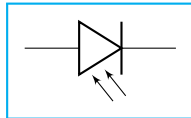
You also have the similar keys for the “full” poles (albeit they are probably not useful at all).

6.2 Mirroring and Inverting

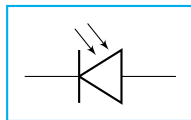
Bipole paths can also be mirrored and inverted (or reverted) to change the drawing direction.



```
\begin{circuitikz}
\draw (0,0) to[pD] (2,0);
\end{circuitikz}
```

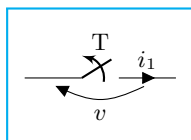


```
\begin{circuitikz}
\draw (0,0) to[pD, mirror] (2,0);
\end{circuitikz}
```

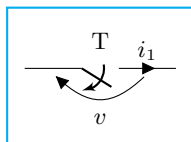


```
\begin{circuitikz}
\draw (0,0) to[pD, invert] (2,0);
\end{circuitikz}
```

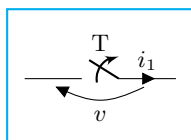
Placing labels, currents and voltages also works, please note, that mirroring and inverting does not influence the positioning of labels and voltages. Labels are by default above/right of the bipole and voltages below/left, respectively.



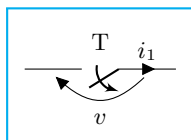
```
\begin{circuitikz}
\draw (0,0) to[ospst=T, i=$i_1$, v=$v$] (2,0);
\end{circuitikz}
```



```
\begin{circuitikz}
\draw (0,0) to[ospst=T, mirror, i=$i_1$, v=$v$] (2,0);
\end{circuitikz}
```

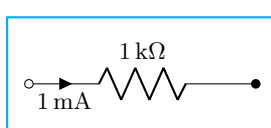


```
\begin{circuitikz}
\draw (0,0) to[ospst=T, invert, i=$i_1$, v=$v$] (2,0);
\end{circuitikz}
```

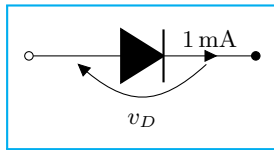


```
\begin{circuitikz}
\draw (0,0) to[ospst=T,mirror,invert, i=$i_1$, v=$v$] (2,0);
\end{circuitikz}
```

6.3 Putting them together



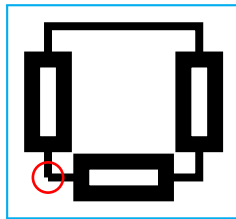
```
\begin{circuitikz}
\draw (0,0) to[R=1<\kilo\ohm>,
i>_1<\milli\ampere>, o-*] (3,0);
\end{circuitikz}
```



```
\begin{circuitikz}
  \draw (0,0) to[D*, v=$v_D$,
    i=1<\milli\ampere>, o-] (3,0);
\end{circuitikz}
```

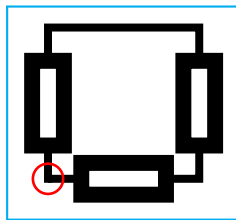
6.4 Line joins between Path Components

Line joins should be calculated correctly - if they are on the same path, and the path is not closed. For example, the following path is not closed correctly (`-cycle` does not work here!):



```
\begin{tikzpicture}[line width=3pt,european]
  \draw (0,0) to[R]++(2,0)to[R]++(0,2)
    --++(-2,0)to[R]++(0,-2);
  \draw[red,line width=1pt] circle(2mm);
\end{tikzpicture}
```

To correct the line ending, there are support shapes to fill the missing rectangle. They can be used like the support shapes (`*,o,d`) using a dot (`.`) on one or both ends of a component (have a look at the last resistor in this example):

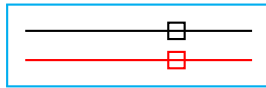


```
\begin{tikzpicture}[line width=3pt,european]
  \draw (0,0) to[R]++(2,0)to[R]++(0,2)
    --++(-2,0)to[R,-.]++(0,-2);
  \draw[red,line width=1pt] circle(2mm);
\end{tikzpicture}
```

7 Colors

Color support in CircuiTikZ has been quite limited up to version 1.5.1; from that one onward there has been an effort to make components' behavior more intuitive.

Part of the problem is how colors in paths are treated by TikZ itself; you can see part of the discussion on [this issue](#) and in [this question on TeX.SX](#) — many thanks to @muzimuzhi for helping there. Basically, nodes are drawn *after* the path is completed, and color is applied to the path at the end. Look at this code (pure TikZ, no CircuiTikZ here):



```
\tikz \draw[thick] (0,0) -- (1,0) {[color=red] -- (2,0) node[draw
]{} } --(3,0); \par
\tikz \draw[thick] (0,0) -- (1,0) [color=red] -- (2,0) node[draw
]{} --(3,0);
```

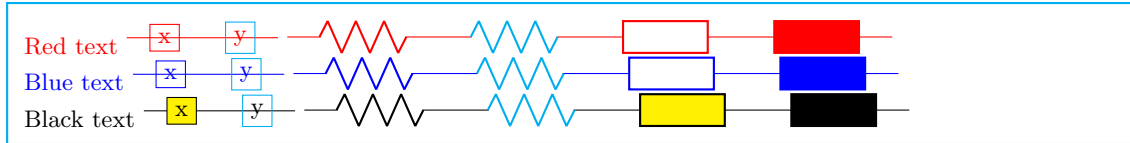
So the path is drawn with the last “effective” (in current group) color. The deferred behavior of the color properties is very difficult to track for CircuiTikZ, especially when the shorthand `color-name` (e.g., `red` instead of `color=red`) is used. CircuiTikZ will try to keep track of the colors specified by `color=...` and `fill=...`, but if you use the implicit way (`\draw[red]...`) it often fails.

If you're adventurous, you can try to add

```
\usepackage{regexpatch}\ctikzPatchImplicitColor
```

after loading CircuiTikZ, and it will try to patch the default commands to keep track of the “current color”; if it fails will give a warning like `patch failed, use only explicit color=...`. This is, unfortunately, not compatible with package `xpatch` (and much others, which load `xpatch`).¹⁵⁰

Before 1.5.0, CircuiTikZ used black as the default color. Now it tries to follow the current color, as TikZ does normally; but notice that there is a difference with the fill strategy:



```
\color{red}
Red text
\tikz \draw (0,0) -- node[draw] {x} (1,0) -- node[draw=cyan] {y} (2,0);
\tikz \draw (0,0) to[R] (2,0) to[R,color=cyan] (4,0) to [generic] (6,0) to [fullgeneric]
(8,0);

\color{blue}
Blue text
\tikz \draw (0,0) -- node[draw] {x} (1,0) -- node[draw=cyan] {y} (2,0);
\tikz \draw (0,0) to[R] (2,0) to[R,color=cyan] (4,0) to [generic] (6,0) to [fullgeneric]
(8,0);

\color{black}
Black text
\tikz \draw[fill=yellow] (0,0) -- node[draw, fill] {x} (1,0) -- node[draw=cyan] {y} (2,0)
;
\tikz \draw[fill=yellow] (0,0) to[R] (2,0) to[R,color=cyan] (4,0) to [generic] (6,0) to
[fullgeneric] (8,0);
```

¹⁵⁰version 1.5.0 loaded it unconditionally for LaTeX; please do not use it

CircuitikZ components that are fillable will inherit the `fill` property of the path (it is almost impossible to do otherwise) as if the `fill` flag was present. “Full”-type elements (for example, full diodes or similar) are filled with the draw color; elements with intrinsic labels (i.e., labels that are part of the shape, like signs on amplifiers and pin numbers in chips) are drawn with the “draw” colors.

Basically, you should have no problem if:

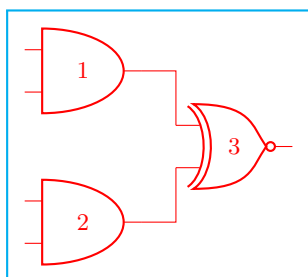
1. You stick to use styles (see 3.3.2) for filling your components, or using a direct `fill=...` option in the component node or `to` option;
2. do not try to change the color mid-path; sometimes it works (see the examples below), but it’s better to avoid it (using different paths is better);
3. when coloring whole circuits, it’s better to use the option `color=...` in your global picture options or in the `\draw` command (not just the color name as a shorthand);
4. forget about transparency.

Nevertheless, if you really need to do strange things with colors you can read on; you can do almost everything but there are several glitches to take into account.

Moreover, in some cases, also the engine you are using (as `pdflatex`, `xelatex`, and so on) can impact corner cases (or even not so corner, like in [american-style voltage source signs](#)).

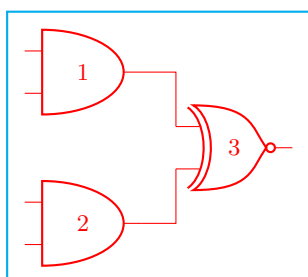
7.1 Shape colors

The color of the components is stored in the key `\circuitikzbasekey/color`. CircuitikZ tries to follow the color set in TikZ, although sometimes it fails. The following circuit will fail to draw the circuit in red if the patching of the inner commands of TikZ fails, like for example in ConTeXt.



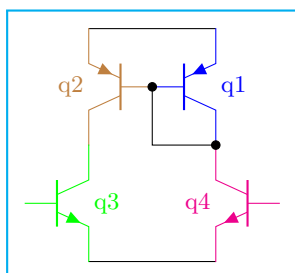
```
\begin{circuitikz} \draw[red]
(0,2) node[and port] (myand1){1}
(0,0) node[and port] (myand2){2}
(2,1) node[xnor port] (myxnor){3}
(myand1.out) -| (myxnor.in 1)
(myand2.out) -| (myxnor.in 2)
;\end{circuitikz}
```

If you see this problem, please do not use just the color name as a style, like `[red]`, but rather assign the style `[color=red]`.



```
\begin{circuitikz} \draw[color=red]
(0,2) node[and port] (myand1){1}
(0,0) node[and port] (myand2){2}
(2,1) node[xnor port] (myxnor){3}
(myand1.out) -| (myxnor.in 1)
(myand2.out) -| (myxnor.in 2)
;\end{circuitikz}
```

One can, of course, change the color directly in the component:

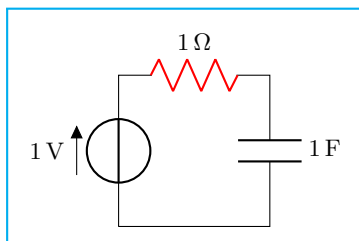


```

\begin{circuitikz} \draw
(0,0) node[pnp, color=blue] (pnp2){q1}
(pnp2.B) node[pnp, xscale=-1, anchor=B, color=brown] (pnp1){\ctikzflipx{q2}}
(pnp1.C) node[npn, anchor=C, color=green] (nnp1){q3}
(pnp2.C) node[npn, xscale=-1, anchor=C, color=magenta] (nnp2){\ctikzflipx{q4}}
(pnp1.E) -- (pnp2.E) (nnp1.E) -- (nnp2.E)
(pnp1.B) node[circ]{} |- (pnp2.C) node[circ]{}
;\end{circuitikz}

```

The all-in-one stream of bipoles poses some challenges, as only the actual body of the bipole, and not the connecting lines, will be rendered in the specified color (Notice that the following example uses the special **siunitx** shorthand; you can use it only in simple cases like here, in general using the full `\SI{}` or `\qty{}` commands).

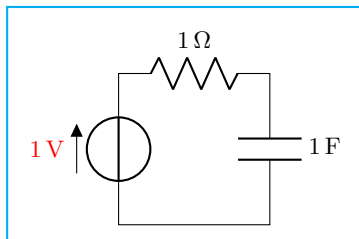


```

\begin{circuitikz} \draw
(0,0) to[V=1<\volt>] (0,2)
to[R=1<\ohm>, color=red] (2,2)
to[C=1<\farad>] (2,0) -- (0,0)
;\end{circuitikz}

```

The postponed application of colors creates a problem if you want to use arrows for voltages, because the “arrows” are partially part of the path, partially nodes:

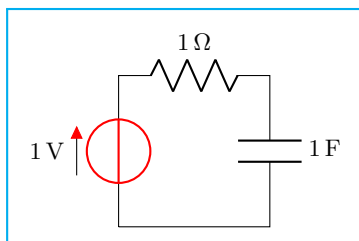


```

\begin{circuitikz} \draw
(0,0){[red] to[V=1<\volt>] (0,2) }
to[R=1<\ohm>] (2,2)
to[C=1<\farad>] (2,0) -- (0,0)
;\end{circuitikz}

```

...and that can become quite frustrating:

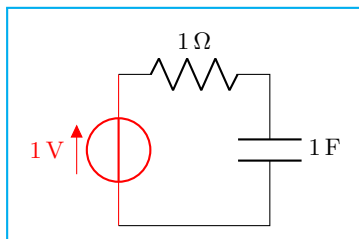


```

\begin{circuitikz} \draw
(0,0) to[V=1<\volt>, color=red] (0,2)
to[R=1<\ohm>] (2,2)
to[C=1<\farad>] (2,0) -- (0,0)
;\end{circuitikz}

```

In those cases, the only way out is to specify different paths (and/or using advanced voltages, see [5.8](#))



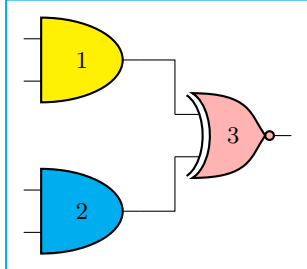
```

\begin{circuitikz} \draw[color=red]
(0,0) to[V=1<\volt>, color=red] (0,2);
\draw (0,2) to[R=1<\ohm>] (2,2)
to[C=1<\farad>] (2,0) -- (0,0)
;\end{circuitikz}

```

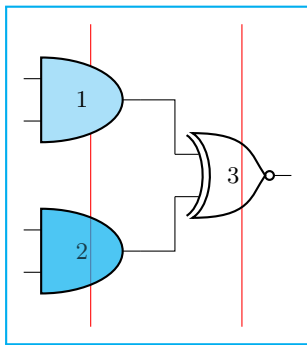
7.2 Fill colors

Since version 0.9.0, you can also fill most shapes with a color (the manual specifies which ones are fillable or not). The syntax is quite intuitive:



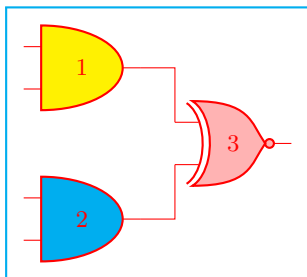
```
\begin{circuitikz} \draw
  (0,2) node[and port, fill=yellow](myand1){1}
  (0,0) node[and port, fill=cyan](myand2){2}
  (2,1) node[xnor port,fill=red!30!white](myxnor){3}
  (myand1.out) -| (myxnor.in 1)
  (myand2.out) -| (myxnor.in 2)
;\end{circuitikz}
```

This fill color will override any color defined by the style (see section 3.3.2). If you want to override a style fill color with no-fill for a specific component, you need to override the style — it's a bit unfortunate, but should be an exceptional thing anyway. Notice that in simple case `fill opacity` works, but don't count too much on it.

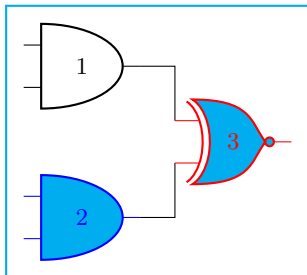


```
\begin{circuitikz}
  \ctikzset{logic ports/fill=cyan!30!white}
  \draw[red] (-0.5,3) -- (-0.5, -1);
  \draw[red] (1.5,3) -- (1.5, -1);
  \draw
  (0,2) node[and port](myand1){1}
  (0,0) node[and port, fill=cyan, fill opacity=0.7](myand2){2}
  (2,1) node[xnor port, circuitikz/logic ports/fill=none
    ](myxnor){3}
  (myand1.out) -| (myxnor.in 1)
  (myand2.out) -| (myxnor.in 2)
;\end{circuitikz}
```

You can combine shape colors with fill colors, too, but you should use the explicit `color` option style for this:



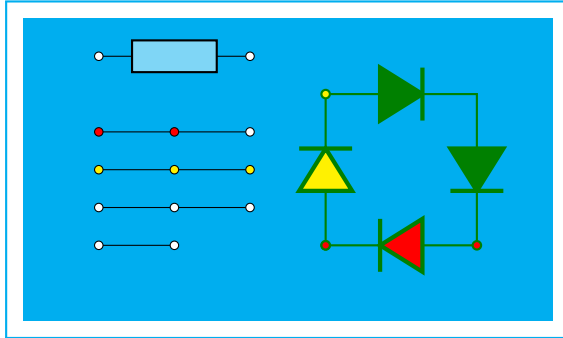
```
\begin{circuitikz} \draw[color=red]
  (0,2) node[and port, fill=yellow](myand1) {1}
  (0,0) node[and port, fill=cyan] (myand2){2}
  (2,1) node[xnor port,fill=red!30!white](myxnor){3}
  (myand1.out) -| (myxnor.in 1)
  (myand2.out) -| (myxnor.in 2)
;\end{circuitikz}
```



```
\begin{circuitikz} \draw
  (0,2) node[and port, color=black](myand1){1}
  (0,0) node[and port, color=blue, fill=cyan](myand2){2}
  (2,1) node[xnor port, color=red, fill=cyan](myxnor){3}
  (myand1.out) -| (myxnor.in 1)
  (myand2.out) -| (myxnor.in 2)
;\end{circuitikz}
```

7.2.1 Background colors different from white

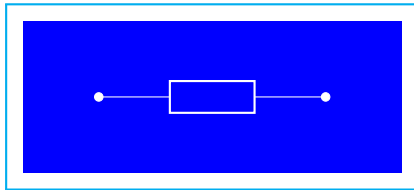
Notice also that the connection point is always filled, and the color *tries* to follow the color of the filling of the component (but look at section 6.1.1). Moreover, if you want to pass fill transparency down to path-style components, you *have* to put it into the options of the `\draw` command.



```
\begin{circuitikz}
  \fill[cyan] (0,3.0) rectangle (7,7);
  \draw [fill opacity=0.5] (1,6.5) to[generic, fill=white,o-o] ++(2,0);
  \draw (1,5.5) to[short, fill=red, o-o] ++(1,0) to[short, -o] ++(1,0);
  \draw[fill=yellow] (1,5) to[short, o-o] ++(1,0) to[short, -o] ++(1,0);
  \draw (1,4.5) to[short, o-o] ++(1,0) to[short, -o] ++(1,0);
  \draw (1,4) node[ocirc]{} -- ++(1,0) node[ocirc]{};
  \draw [thick, color=green!50!black] (4,4) to [D,o-o,fill=yellow] ++(0,2) to[D*, fill=yellow]
    ++(2,0) to[D*,fill=yellow] ++(0,-2) to[D, fill=red, o-o] ++(-2,0);
\end{circuitikz}
```

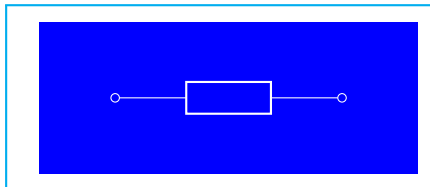
As you can see, the “black” components (as `D*`) follow the color of the line, not the fill.

Note, however, that if you choose a colored background, for example with the `\pagecolor{}` command or with other tricks, the nodes will be by default still filled with white.



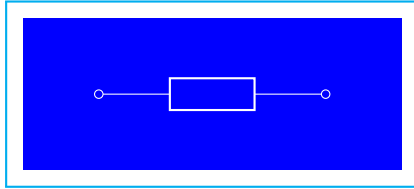
```
\begin{circuitikz}[european]
  \fill[color=blue] (-1,-1) rectangle (4,1);
  \draw[color=white] (0,0) to[R, o-o] ++(3,0);
\end{circuitikz}
```

You have two solutions for this. You can redefine the `o-o` (and the similar commands `-o`, `o-`, `*-o` and so on) with a blue filled “open” pole:



```
\tikzset{bcirc/.style={shape=ocirc, fill=blue}}
\ctikzset{o-o/.style ={
  \circuitikzbasekey/bipole/nodes/left=bcirc,
  \circuitikzbasekey/bipole/nodes/right=bcirc}}
\begin{circuitikz}[european]
  \fill[color=blue] (-1,-1) rectangle (4,1);
  \draw[color=white] (0,0) to[R, o-o] ++(3,0);
\end{circuitikz}
```

Also, since v1.2.3, you can set the key `open poles fill` (default: `white` which works for `ocirc`, `odiamondpole` and `osquarepole`):



```
\begin{circuitikz}[european]
  \ctikzset{open poles fill=blue}
  \fill[color=blue] (-1,-1) rectangle (4,1);
  \draw[color=white] (0,0) to[R, o-o] ++(3,0);
\end{circuitikz}
```


8 FAQ: Frequently asked questions

8.1 Using named nodes in circuits

Q: When I use a node to name a connection in the circuit, I have gaps in the wires! I am sure it used to work!

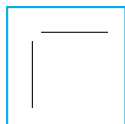
A: This is explained in 1.10. The fast answer is that in a hurry, use the 1.1.2 fallback point with:

```
\usepackage{circuitikz-1.1.2}
```

in your preamble.

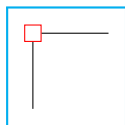
But really, your circuit definition is buggy, so the best thing to do is fix that; if you want to name a point in your circuit, you should use a **coordinate**, not a **node**.¹⁵¹ Here is a small tutorial on *why* you should change your circuit.

Nodes, in TikZ, normally have a non-zero size even when they are empty; moreover, connections are supposed to join the border of nodes. Please study the following (pure TikZ, not CircuiTikZ):



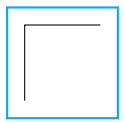
```
\begin{tikzpicture}
  \path (1,1) node (A){}; % empty node at (1,1)
  \draw (1,0) -- (A) -- (2,1); % surprise!
\end{tikzpicture}
```

The gap is there because the node has a non-zero size (more in detail, its **inner sep** is by default different from zero). You can see it easily if you draw the node shape:



```
\begin{tikzpicture}
  \path (1,1) node [draw=red] (A){};
  \draw (1,0) -- (A) -- (2,1);
\end{tikzpicture}
```

The problem is that when you want to name a coordinate, in the sense of a dimensionless point, you should use a **coordinate**, **not** a **node**!



```
\begin{tikzpicture}
  \path (1,1) coordinate (A); % give a name to (1,1)
  \draw (1,0) -- (A) -- (2,1); % now it's ok!
\end{tikzpicture}
```

Now, before version 1.2.1 (and since around 0.6), CircuiTikZ was detecting when a connection was between nodes and sort-of added a **node.center** movement to the path. That in turn generated the need of hacks to draw the correct joining of lines, because that kind of movement broke the continuity of the path, like in this example:



```
\begin{tikzpicture}[line width=4pt]
  \path (1,1) node (A){};
  \draw (1,0) -- (A.center) (A) (A.center) -- (2,1);
\end{tikzpicture}
```

You can see more examples and more reasonings on GitHub; start from the [issue detecting the join problem](#), then [look at the merged fix](#); you can follow several issues and discussions from there, but for example there are circuits that can't be drawn with the “hack” in, [like this one](#).

So finally it was decided¹⁵² to remove the change, to simplify the code and to make the package more maintainable.

¹⁵¹Yes, I understand from where the confusion arise — in circuit theory they are called nodes.

¹⁵²well, Romano decided, so you can blame him. *I do not think that workarounds to correct malformed circuits are really maintainable; just see the bunch of code removed by the patch! — Romano.*

8.2 Using dashed (or colored) wires in circuits

Q: How can I make part of the wires dashed (or colored)? This does not work:



```
\begin{circuitikz}
\draw (0,0) to[R] ++(2,0)
to[short, dashed, red] ++(1,0)
to [R] ++(2,0); % surprise!
\end{circuitikz}
```

Nor this one, which is even stranger:



```
\begin{circuitikz}
\draw (0,0) to[R] ++(2,0)
[dashed, red] -- ++(1,0)
to [R] ++(2,0); % surprise!
\end{circuitikz}
```

A: This is an effect on how TikZ builds and draws path. As explained in the TikZ manual,¹⁵³ most path options are globally valid for the whole path; color and dash/dot is one of this. You have two options in this case. The first one is to use two paths.



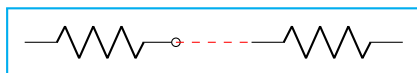
```
\begin{circuitikz}
\draw (0,0) to[R] ++(2,0) coordinate(a);
\draw [dashed, red] (a) -- ++(1,0) coordinate(b);
\draw (b) to [R] ++(2,0);
\end{circuitikz}
```

The other one is to use **edge** operations¹⁵⁴; be sure to read about it on the TikZ manual¹⁵⁵ — but basically this is similar to the **to** operation but it builds another path (added at the end of the current path, like nodes are). This means that it can use different options, and that it **does not** move the path coordinates. So, for example:



```
\begin{circuitikz}
\draw (0,0) to[R] ++(2,0)
edge[dashed, red] ++(1,0)
% we have to move the path position here!
++(1,0) to [R] ++(2,0);
\end{circuitikz}
```

The only problem with this approach is that the **edges** are added *after* the nodes, so it can create problems with nodes (look carefully!):



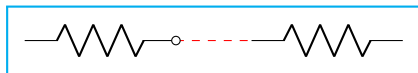
```
\begin{circuitikz}
\draw (0,0) to[R,-o] ++(2,0)
edge[dashed, red] ++(1,0)
++(1,0) to [R] ++(2,0);
\end{circuitikz}
```

¹⁵³in 3.1.5b, section 14, “syntax for path specification”

¹⁵⁴I took the idea from [this answer by @LaTeXdraw-com user on TeX.SE](#), thanks!

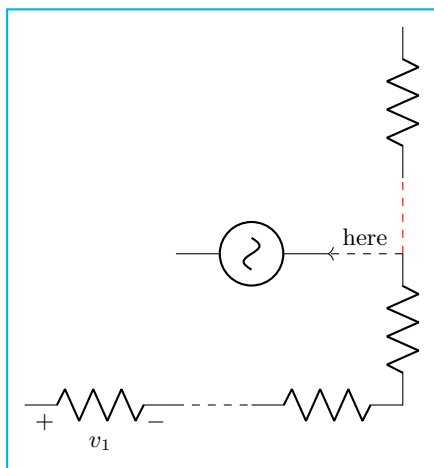
¹⁵⁵in 3.1.5b, section 17.12, “connecting nodes: use the edge operation”

So it's better, in this case, to add the nodes manually after the path (there is no perfect solution!):



```
\begin{circuitikz}
  \draw (0,0) to[R] ++(2,0) coordinate(a)
  edge[dashed, red] ++(1,0)
  ++(1,0) to [R] ++(2,0);
  \node [ocirc] at (a){};
\end{circuitikz}
```

A more complex example can be seen (look at the comments!) in the following circuit.



```
\begin{circuitikz}[american]
  \draw (0,0) to[R, v=$v_1$] ++(2,0)
  edge[dashed] ++(1,0)
  ++(1,0) to[R]
  ++(2,0) to [R] ++(0,2) coordinate(a)
  edge[red, dashed] ++(0,1)
  % several edges start from the same position
  edge[dashed, ->] node[above]{here} ++(-1,0)
  % notice that the path here is still
  % at coordinate (a)!
  ++(0,1) to[R] ++(0,2)
  (a) ++(-1,0) to[sV] ++(-2,0);
\end{circuitikz}
```

8.3 Errors when externalizing pictures

Q: When using `\tikzexternalize` I get the following error:

! Emergency stop.

A: The TikZ manual states:

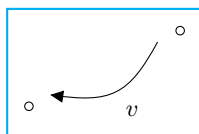
Furthermore, the library assumes that all $\text{\LaTeX}$ pictures are ended with `\end{tikzpicture}`.

Just substitute every occurrence of the environment `circuitikz` with `tikzpicture`. They are actually pretty much the same.

8.4 Labels, voltages and currents woes

Q: How do I draw the voltage between two nodes?

A: Between any two nodes there is an open circuit!



```
\begin{circuitikz} \draw
  node[ocirc] (A) at (0,0) {}
  node[ocirc] (B) at (2,1) {}
  (A) to[open, v=$v$] (B)
;\end{circuitikz}
```

Q: I cannot write `to[R = $R_1=12V$]` nor `to[ospst = open, 3s]`: I get errors.

A: It is a limitation of the parser, joined with a suboptimal processing by `CircuitikZ` (up to 1.2.7) of the passing of the argument of keys.

You should protect commas and equal signs like in `to[R = {\$R_1=12V$}]` or `to[ospst = {open, 3s}]`.

In versions up to 1.2.7, use for example `\mbox{}` or define `\def\eq{=}` and use `to[R = $R_1\eq 12V$]`, or try to protect commas and equal signs like `to[ospst = open{,} 3s]` or `ospst=\mbox{open, 3s}` instead; see caveat in section 5.1.

8.5 Global scaling and rotating

Q: I tried to change the direction of the y -axis with `yscale=-1`, but the circuit is completely messed up.

A: Yes, it's a known bug (or misfeature, or limitation). See section 1.8. Don't do that.

Q: I tried to put a diode in a `pic`, but it's coming out badly rotated.

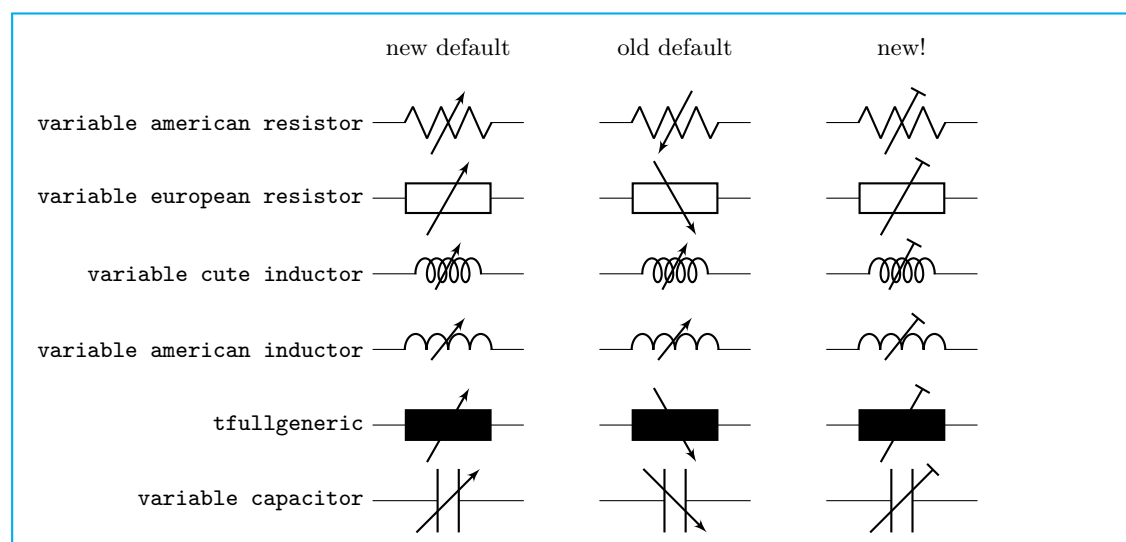
A: This is a bug that *should* have been fixed in v1.8.0. Previous versions of `CircuitikZ` are plainly not compatible with `pics`.

8.6 Tunable components

Q: The direction of the arrows in variable resistors or capacitors changed!

A: Yes, it changed in v1.3.3.

Version 1.3.3 fixes the direction of the arrows in tunable elements; before this version, they were more or less random, now the arrow goes from bottom left to top right. You have the option to go back to the old behavior with `\ctikzset{bipoles/fix tunable direction=false}`. As a compensation for the fuss, now the arrows are configurable.



```
\begin{circuitikz}[european]
  \draw (1,0) node{new default} (4,0) node{old default} (7,0) node{new!};
  \foreach [count=\i] \comp in
    {variable american resistor, variable european resistor,
      variable cute inductor, variable american inductor, tfullgeneric,
      variable capacitor} {
    \draw (0,-\i) node[left]{\texttt{\comp}} to[\comp, name=E] ++(2,0);
  }
```

```

\ctikzset{bipoles/fix tunable direction=false}
\draw (3,-\i) to[\comp, name=E] ++(2,0);
\ctikzset{bipoles/fix tunable direction=true, tunable end arrow={Bar}}
\draw (6,-\i) to[\comp, name=E] ++(2,0);
}
\end{circuitikz}

```

9 Defining new components

Per me si va ne la città dolente,
per me si va ne l'eterno dolore,
per me si va tra la perduta gente.
...

Lasciate ogne speranza, voi ch'intrate.¹⁵⁶

Big fat warning: this material is reserved for T_EX-hackers; do not delve into this if you have no familiarity with (at least) a bit of core T_EX programming and to the basic TikZ layer. You have been warned.

9.1 Suggested setup

Notice: the source code has been reorganized after release 1.2.7; if you are bound to use an older version check the corresponding manual.

The suggested way to start working on a new component is to use the utilities of the CircuiTikZ manual for checking and testing your device. Basically, find (or download) the source code of the last version of CircuiTikZ and find the file `ctikzmanutils.sty`; copy it in your directory and prepare a file like this:

```
\documentclass[a4paper, titlepage]{article}
\usepackage{a4wide}           %smaller borders
\usepackage[utf8]{inputenc}   %not needed since LaTeX 2019
\usepackage[T1]{fontenc}
\parindent=0pt
\parskip=4pt plus 6pt minus 2pt
\usepackage[siunitx, RPvoltages]{circuitikzgit}
\usepackage{ctikzmanutils}
\makeatletter
%% Test things here
% defines

% components

% paths
\makeatother

\begin{document}

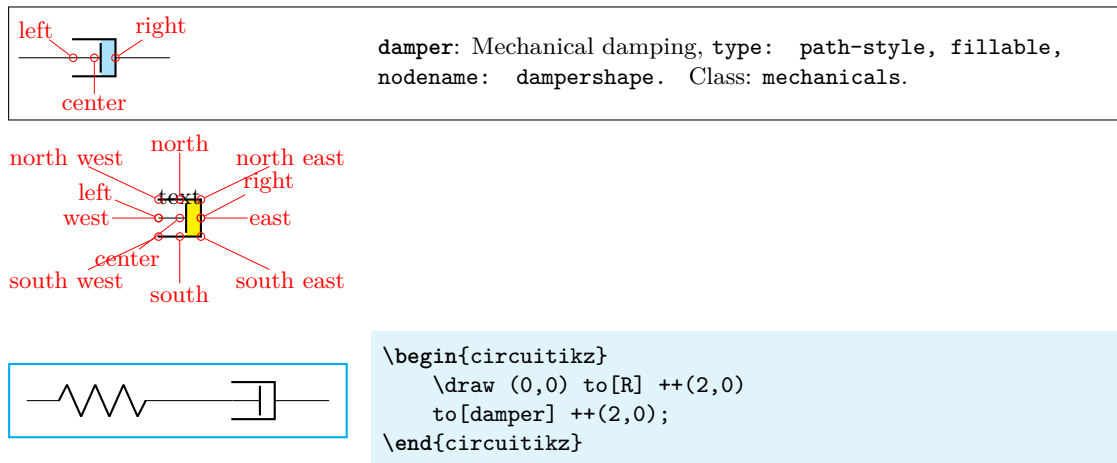
\circuitdescrip*{damper}{Mechanical damping}{}(left/135/0.2, right/45/0.2, center
/-90/0.3)

\geolrcoord{dampershape, fill=yellow}

\begin{ctikzExample}
\begin{circuitikz}
  \draw (0,0) to[R] ++(2,0)
    to[damper] ++(2,0);
\end{circuitikz}
\end{ctikzExample}
\end{document}
```

This will compile to something like this (in this case, we are using a couple of existing components to check everything is OK):

¹⁵⁶<https://classicsincontext.wordpress.com/2010/02/28/canto-iii-per-me-si-va-ne-la-citta-dolente/>



The command `circuitdescbip*` is used to show the component description (you can check the definition and the usage looking at `ctikzmanutils.sty` file, and the `\geolrcoord` is used to show the main anchors (geographical plus `left` and `right`) of the component).

From now on, you can add the new commands for the component between the `\makeatletter` and `\makeatother` commands and, modifying the example, check the results.

9.2 Path-style component

Let's define for example a path style component, like the one suggested by the user [@alex](#) on [TeX stackexchange site](#). The component will be a mix of the `damper` and the `spring` components already present.

The definitions of the components are in the files `pgfcircsomething.tex`; they are more or less distributed by the number of terminals, but there are exceptions (for example, switches are in `bipoles`, even if several of them are tripoles or more... `grep` is your friend here).

To define the new component we will look into (in this case) `pgfcircbipoles.tex`; at the start of the block where the components are defined, you can find the relevant definitions (sometime some of the definitions are in `pgfcirc.defines.tex`, for historical or dependencies reasons). The first step is to check if we can use the definition already existing for similar elements (for coherence of size) or if we need to define new ones; for this you have to check into the we find

```

\ctikzset{bipoles/spring/height/.initial=.5}
\ctikzset{bipoles/spring/width/.initial=.5}
\ctikzset{bipoles/damper/height/.initial=.35}
\ctikzset{bipoles/damper/length/.initial=.3}
\ctikzset{bipoles/damper/width/.initial=.4}

```

We will use them; at this stage you can decide to add other parameters if you need them. (Notice, however, than although flexibility is good, these parameters should be described in the manual, otherwise they're as good as a fixed number in the code).

After that we will copy, for example, the definition of the damper into our code, just changing the name:

```

%% mechanical resistor - damper
\pgfcircdeclarebipolescaled{mechanicals}
{
}
{
  \ctikzvalof{bipoles/damper/height}} % depth (under the path line)
{viscoe} % name
{
  \ctikzvalof{bipoles/damper/height}} % height (above the path line)
{
  \ctikzvalof{bipoles/damper/width}} % width
{

```

```

\pgfpathrectanglecorners{\pgfpoint{\ctikzvalof{bipoles/damper/length}\pgf@circ@res@right}{\pgf@circ@res@down}}{\pgfpoint{\pgf@circ@res@right}{\pgf@circ@res@up}}
\pgf@circ@maybefill

% line into the damper
\pgfpathmoveto{\pgfpoint{\pgf@circ@res@left}{\pgf@circ@res@zero}}
\pgfpathlineto{\pgfpoint{\ctikzvalof{bipoles/damper/length}\pgf@circ@res@right}{\pgf@circ@res@zero}}
\pgfusepath{stroke}

% damper box
\pgf@circ@setlinewidth{bipoles}{\pgfstartlinewidth}
\pgfpathmoveto{\pgfpoint{\pgf@circ@res@left}{\pgf@circ@res@down}}
\pgfpathlineto{\pgfpoint{\pgf@circ@res@right}{\pgf@circ@res@down}}
\pgfpathlineto{\pgfpoint{\pgf@circ@res@right}{\pgf@circ@res@up}}
\pgfpathlineto{\pgfpoint{\pgf@circ@res@left}{\pgf@circ@res@up}}

\pgfsetrectcap
\pgfsetmiterjoin
\pgfusepath{stroke}

% damper vertical element
\pgfpathmoveto{\pgfpoint{\ctikzvalof{bipoles/damper/length}\pgf@circ@res@right}{.8\pgf@circ@res@down}}
\pgfpathlineto{\pgfpoint{\ctikzvalof{bipoles/damper/length}\pgf@circ@res@right}{.8\pgf@circ@res@up}}
\pgfsetbuttcap
\pgfusepath{stroke}
}

```

This `\pgfcircdeclarebipolescaled` command will define a shape that is named `viscoeshape`, with all the correct geographical anchors based on the depth, height and width defined in the parameters: in this case we are reusing the ones of the `damper` shape. Moreover, the element is assigned to the class `mechanicals` for styling.

To be coherent with the styling, you should use (when needed) the length `\pgf@circ@scaled@Rlen` as the “basic” length for drawing, using the fill functions (they are defined at the start of the file `pgfcirc.defines.tex`) to fill and stroke — so that the operation will follow the style parameters and, finally, use the macro `\pgf@circ@setlinewidth` to set the line thickness: the first argument is the “legacy” class, if you do not want to assign one you can use the pseudo-legacy class `none`.

The anchors for the bipole (which then set the lengths `\pgf@circ@res@left`) are already scaled for your use. You can use these lengths (which defines, normally, the geographical anchors of the element) to draw your shapes.

This is not sufficient for using the element in a `to[]` path command; you need to “activate” it (the definition of the commands are normally in `pgfcircpath.tex`). In this case the component is simple — look at the definitions if you need to do more complex things.

```

\pgfcirc@activate@bipole@simple{1}{viscoe}

```

In the definition above, the `{1}` parameter means that using the component like `to[viscoe=A]` will be equivalent to `to[viscoe, 1=A]`; you can also use `v` or `i` or `f` if your component needs it. Now you can show it with:


```

\circuitdescbip*{viscoe}{Mechanical viscoelastic element}{}(left/135/0.2, right/45/0.2,
center/-90/0.3)

\geolrcoord{viscoeshape, fill=yellow}

\begin{ctikzExample}
\begin{circuitikz}
\draw (0,0) to[spring] ++(2,0)
to[viscoe] ++(2,0);
\end{circuitikz}
\end{ctikzExample}

```

Obviously, at first you just have a component that is the same as the one you copied with another name. It is now just a matter of modifying it so that it has the desired shape; in the example above you can already see the new symbol after the changes.

When doing the drawing in the main argument of the `\pgfcircdeclarebipolescaled`, things will be set up so that the lengths `\pgf@circ@res@right` and `\pgf@circ@res@up` are the x - y coordinates of the upper right corner, and `\pgf@circ@res@left` and `\pgf@circ@res@down` are the x - y coordinates of the lower left corner of your shape. The `center` coordinate is usually at $(0pt, 0pt)$.

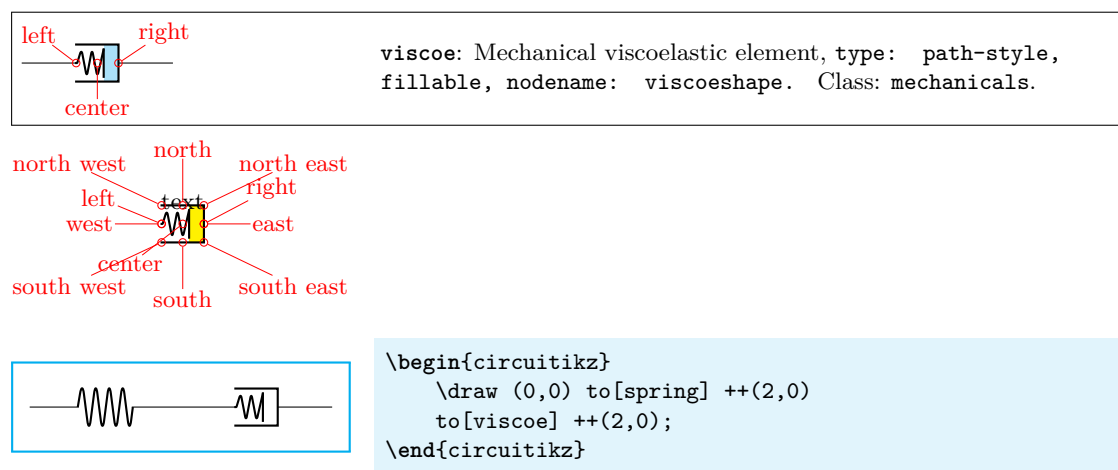
Looking at the implementation of the `spring` element, one possibility is changing the lines between lines 12 and 16 with:

```

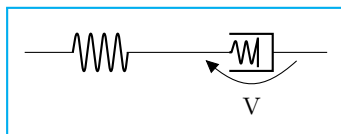
% spring into the damper
\pgfscope
\pgfpathmoveto{\pgfpoint{\pgf@circ@res@left}{\pgf@circ@res@zero}}
\pgf@circ@setlinewidth{bipoles}{\pgfstartlinewidth}
\pgfsetcornersarced{\pgfpoint{.25\pgf@circ@res@up}{.25\pgf@circ@res@up}}
\pgfpathlineto{\pgfpoint{.75\pgf@circ@res@left}{.75\pgf@circ@res@up}}
\pgfpathlineto{\pgfpoint{.5\pgf@circ@res@left}{-.75\pgf@circ@res@up}}
\pgfpathlineto{\pgfpoint{.25\pgf@circ@res@left}{.75\pgf@circ@res@up}}
\pgfpathlineto{\pgfpoint{0pt}{-.75\pgf@circ@res@up}}
\pgfpathlineto{\pgfpoint{\ctikzvalof{bipoles/damper/length}}{\pgf@circ@res@right}}
}{.75\pgf@circ@res@up}}
\pgfusepath{stroke}
\endpgfscope

```

which leads to:

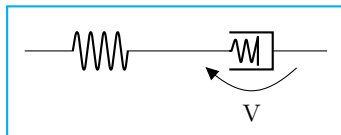


Now you can check if the voltage labels are correct for your new component:



```
\begin{circuitikz}[]
  \draw (0,0) to[spring] ++(2,0)
    to[viscoe, v=V] ++(2,0);
\end{circuitikz}
```

If you think they are too tight or too loose, you can use a (developer-only) key to adjust the distance:



```
\begin{circuitikz}
  \ctikzset{bipoles/viscoe/voltage/additional shift/.
    initial=1}
  \draw (0,0) to[spring] ++(2,0)
    to[viscoe, v=V] ++(2,0);
\end{circuitikz}
```

Notice that by default the key `bipoles/mybipole/voltage/additional shift` is not defined, so if you want to use it you must create it before (this is the meaning of the `.initial` here).

Now you can save all the code between the `\makeatletter` and `\makeatother` in a file and `\input{}` it for using your special component, or submit the component to the project (see below).

As a final note, notice that the `viscoe` element is already added to the standard package.

9.3 Node-style component

Adding a node-style component is much more straightforward. Just define it by following examples in, for example, `pgfcirctripoles.tex` or the other files; be careful that you should define all the geographical anchors of the shape if you want that the TikZ positioning options (like `left`, `above`, etc.) behave correctly with your component.

To have a scalable component, for example in the `transistors` class, you should use something like

```
\savedmacro{\ctikzclass}{\edef\ctikzclass{transistors}}
\saveddimen{\scaledRlen}{\pgfmathsetlength{\pgf@x}{\ctikzvalof{\ctikzclass/scale}
\pgf@circ@Rlen}}
```

at the start of anchors and macros definition, and use (for example, the exact code will change greatly depending on your component):

```
\savedanchor\northeast{% upper right
  \pgfmathsetlength{\pgf@circ@scaled@Rlen}{\ctikzvalof{\ctikzclass/scale}\pgf@circ@Rlen}
  \pgf@y=\pgf@circ@scaled@Rlen
  \pgf@y=0.5\pgf@y
  \pgf@x=0.3\pgf@y
}
```

in all the `savedanchors`.

Then, to draw your component, you should start with¹⁵⁷:

```
\pgf@circ@draw@component{%
  \pgf@circ@scaled@Rlen=\scaledRlen
  ...
}
```

¹⁵⁷Since v1.5.0; component defined with this mechanism will not be compatible with older CircuiTikZ.

and then use `\pgf@circ@scaled@Rlen` (or the anchors) as the default length while you draw it.

The special command `\pgf@circ@draw@component` will issue a `\behindforegroundpath` command, and take care of calling the start and end hooks for the component. Notice that, given that the component is drawn as `\behindforegroundpath`, you must take care to “use” the path you define here, issuing a `\pgfusepath` macro. The path itself is protected by a `pgfscope` (and so also by a `TEX` group), so local definitions will be reset after exiting.

9.3.1 The internal hook system

Since version v1.5.0, before starting the drawing of any component, `CircuitikZ` will check for the existence of three different hooks, in the following order (suppose that the shape name is *myname*, and it has a class *myclass*):

- `\ctikz@hook@start@draw@component@myname`
- `\ctikz@hook@start@draw@class@myclass`
- `\ctikz@hook@start@draw@default`

The first one that is defined in the current (or outer) scope is used, and the following ones are not used. These hooks can be used to set drawing parameters, or to reset them to a known state: `TikZ` normally inherits most of the drawing options, but that can lead to surprises (like unexpected arrows, etc.).

In the same way, before leaving `\pgf@circ@draw@component`, a set of similar hooks (with `end` instead of `start`) is tried, with the same logic.

The only predefined hook is `\ctikz@hook@start@draw@default`, which is set to the equivalent of:

```
\pgfsetshortenstart{+0pt}\pgfsetshortenend{+0pt}\pgfsetarrows{-}%  
\def\pgf@circ@reset@rounded{\pgfsetcornersarced{\pgfpointorigin}}%
```

which means that, by default, arrows parameters are reset to the default (no shorten, no arrows) and that corners are not rounded. If you want to override them, just define the appropriate hook for your component/class and the generic one will not be called.

No `...@end@draw@...` hook is defined by default.

9.3.2 Finishing your work

Once you have a satisfactory element, you should

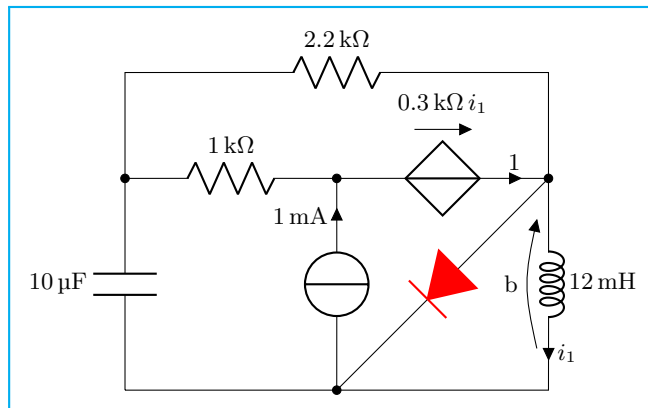
- Clean up your code;
- write a piece of documentation explaining its use, with an example;
- Propose the element for inclusion in the GitHub page of the project (you will have to license this as explained in that page, of course).

The best way of contributing is forking the project, adding your component in the correct files, modifying the manual and creating a pull request for the developers to merge. Anyway, if this is a problem, just open an issue and someone (when they have time...) will answer.

10 Examples

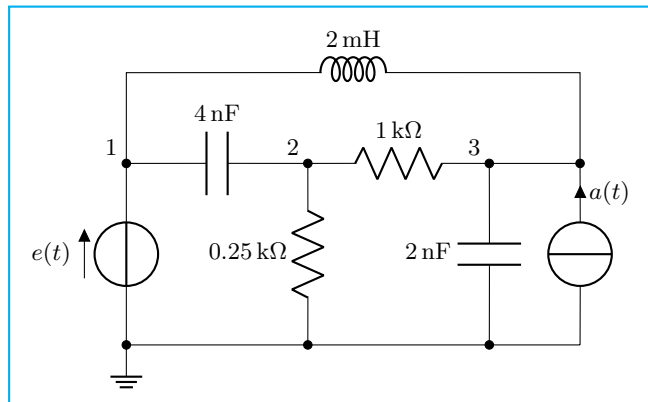
Here a series of examples, contributed by several people, is shown with their code.

10.1 A red diode



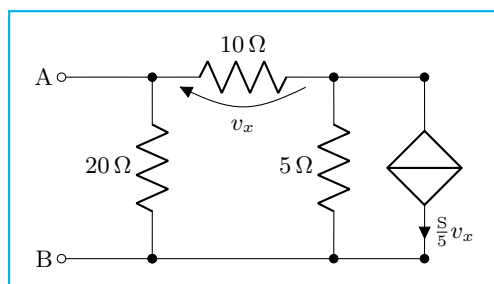
```
\begin{circuitikz}[scale=1.4]\draw
  (0,0) to[C, l=10<\micro\farad>] (0,2) -- (0,3)
    to[R, l=2.2<\kilo\ohm>] (4,3) -- (4,2)
    to[L, l=12<\milli\henry>, i=$i_1$,v=b] (4,0) -- (0,0)
  (4,2) to[D*, color=red] (2,0)
  (0,2) to[R, l=1<\kilo\ohm>, *-] (2,2)
    to[cV, i=1, -*-, v=$\SI{.3}{\kilo\ohm}\, i_1$] (4,2)
  (2,0) to[I, i=1<\milli\ampere>, -*-] (2,2)
;\end{circuitikz}
```

10.2 Using the (experimental) siunitx syntax



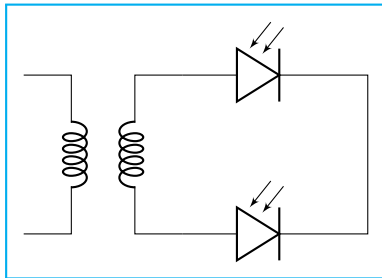
```
\begin{circuitikz}[scale=1.2]\draw
  (0,0) node[ground] {}
    to[V=$e(t)$, *-] (0,2) to[C=4<\nano\farad>] (2,2)
    to[R, l_=.25<\kilo\ohm>, *-] (2,0)
  (2,2) to[R=1<\kilo\ohm>] (4,2)
    to[C, l_=2<\nano\farad>, *-] (4,0)
  (5,0) to[I, i_=$a(t)$, *-] (5,2) -- (4,2)
  (0,0) -- (5,0)
  (0,2) -- (0,3) to[L, l=2<\milli\henry>] (5,3) -- (5,2)

  {[anchor=south east] (0,2) node {1} (2,2) node {2} (4,2) node {3}}
;
\end{circuitikz}
```



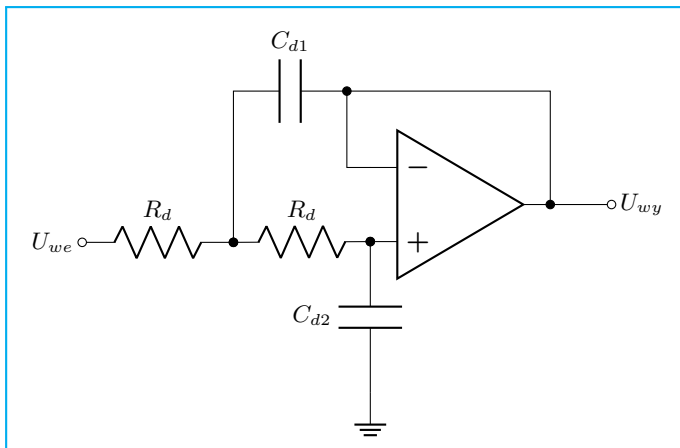
```
\begin{circuitikz}[scale=1.2]\draw
  (0,0) node[anchor=east] {B}
    to[short, o-*] (1,0)
    to[R=20<\ohm>, *-] (1,2)
    to[R=10<\ohm>, v=$v_x$] (3,2) -- (4,2)
    to[cI=$\frac{\si{siemens}}{5} v_x$, *-] (4,0) -- (3,0)
    to[R=5<\ohm>, *-] (3,2)
  (3,0) -- (1,0)
  (1,2) to[short, -o] (0,2) node[anchor=east]{A}
;\end{circuitikz}
```

10.3 Photodiodes



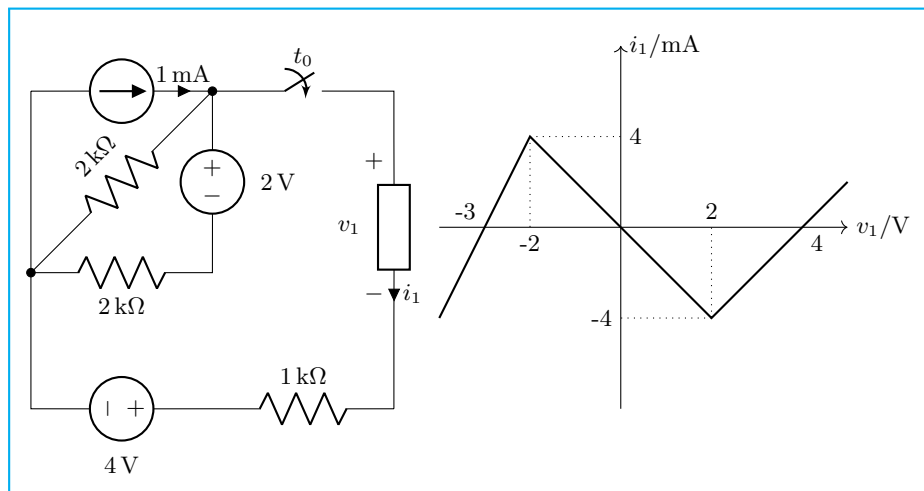
```
\begin{circuitikz}[scale=1]\draw
  (0,0) node[transformer] (T) {}
  (T.B2) to[pD] ($(T.B2)+(2,0)$) -| (3.5, -1)
  (T.B1) to[pD] ($(T.B1)+(2,0)$) -| (3.5, -1)
;\end{circuitikz}
```

10.4 A Sallen-Key cell



```
\begin{circuitikz}[scale=1]\draw
  (5,.5) node [op amp] (opamp) {}
  (0,0) node [left] {$U_{we}$} to [R, l=$R_d$, o-] (2,0)
  to [R, l=$R_d$, *-] (opamp.+)
  to [C, l=$C_{d2}$, *-] ($(opamp.+) + (0,-2)$) node [ground] {}
  (opamp.out) |- (3.5,2) to [C, l=$C_{d1}$, *-] (2,2) to [short] (2,0)
  (opamp.-) -| (3.5,2)
  (opamp.out) to [short, *-o] (7,.5) node [right] {$U_{wy}$}
;\end{circuitikz}
```

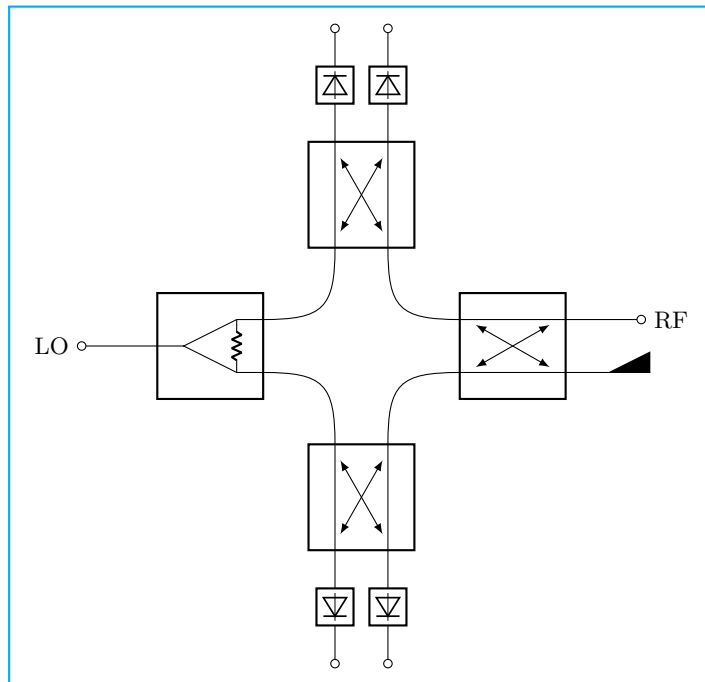
10.5 Mixing circuits and graphs



```
\begin{circuitikz}[scale=1.2, american]\draw
(0,2) to[I=1<\milli\ampere>] (2,2)
to[R, l_2=2<\kilo\ohm>, *-] (0,0)
to[R, l_2=2<\kilo\ohm>] (2,0)
to[V, v_2=2<\volt>] (2,2)
to[csfst, l=$t_0$] (4,2) -- (4,1.5)
to [generic, i=$i_1$, v=$v_1$] (4,-.5) -- (4,-1.5)
(0,2) -- (0,-1.5) to[V, v_4=4<\volt>] (2,-1.5)
to [R, l=1<\kilo\ohm>] (4,-1.5);

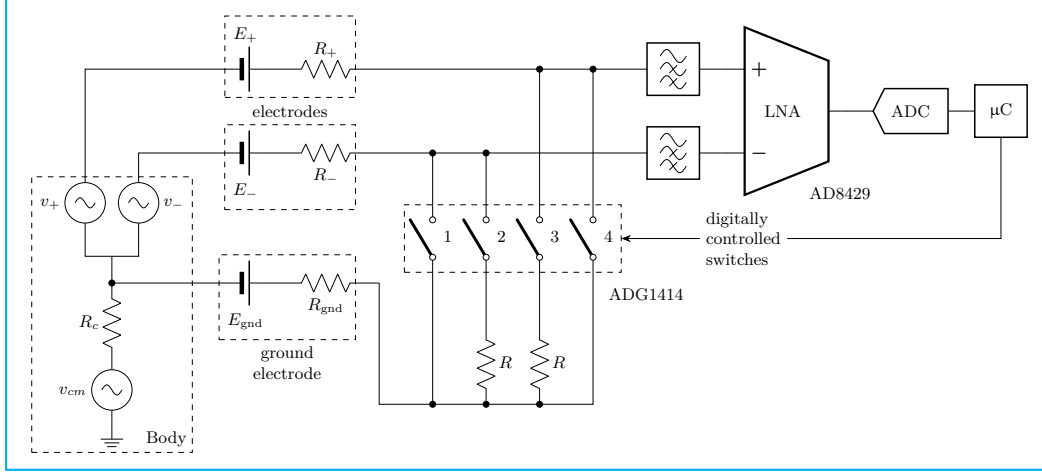
\begin{scope}[xshift=6.5cm, yshift=.5cm]
\draw [->] (-2,0) -- (2.5,0) node[anchor=west] {$v_1/\si{\volt}$};
\draw [->] (0,-2) -- (0,2) node[anchor=west] {$i_1/\si{\milli\ampere}$} ;
\draw (-1,0) node[anchor=north] {-2} (1,0) node[anchor=south] {2}
(0,1) node[anchor=west] {4} (0,-1) node[anchor=east] {-4}
(2,0) node[anchor=north west] {4}
(-1.5,0) node[anchor=south east] {-3};
\draw [thick] (-2,-1) -- (-1,1) -- (1,-1) -- (2,0) -- (2.5,.5);
\draw [dotted] (-1,1) -- (-1,0) (1,-1) -- (1,0)
(-1,1) -- (0,1) (1,-1) -- (0,-1);
\end{scope}
\end{circuitikz}
```

10.6 RF circuit



```
\begin{circuitikz}[scale=1]
  \ctikzset{bipoles/detector/width=.35}
  \ctikzset{quadpoles/coupler/width=1}
  \ctikzset{quadpoles/coupler/height=1}
  \ctikzset{tripoles/wilkinson/width=1}
  \ctikzset{tripoles/wilkinson/height=1}
  %\draw[help lines,red,thin,dotted] (0,-5) grid (5,5);
  \draw
    (-2,0) node[wilkinson](w1){}
    (2,0) node[coupler](c1){}
    (0,2) node[coupler,rotate=90](c2){}
    (0,-2) node[coupler,rotate=90](c3){}
    (w1.out1) .. controls ++(0.8,0) and ++(0,0.8) .. (c3.port3)
    (w1.out2) .. controls ++(0.8,0) and ++(0,-0.8) .. (c2.port4)
    (c1.port1) .. controls ++(-0.8,0) and ++(0,0.8) .. (c3.port2)
    (c1.port4) .. controls ++(-0.8,0) and ++(0,-0.8) .. (c2.port1)
    (w1.in) to[short,-o] ++(-1,0)
    (w1.in) node[left=30]{LO}
    (c1.port2) node[match,yscale=1]{}
    (c1.port3) to[short,-o] ++(1,0)
    (c1.port3) node[right=30]{RF}
    (c2.port3) to[detector,-o] ++(0,1.5)
    (c2.port2) to[detector,-o] ++(0,1.5)
    (c3.port1) to[detector,-o] ++(0,-1.5)
    (c3.port4) to[detector,-o] ++(0,-1.5)
  ;
\end{circuitikz}
```


10.7 A styled low noise input stage



```

\ctikzloadstyle{romano}
\scalebox{0.707}{%
\begin{circuitikz}[american, romano circuit style]
\ctikzset{bipoles/cuteswitch/thickness=0.5}
\draw (0,0) node[ground](GND0){} to[sV, l=$v_{cm}$] ++(0,1)
to [R, l=$R_c$, -*] ++(0,1.5) coordinate(vcm) --++(0,0.5) coordinate(diffc);
\draw (diffc) -| ++(-0.5, 0.5) to[sV, l=$v_+$, name=vplus] ++(0,1) --++(0,2)
-- ++(2.5,0) coordinate(skin+ a) to[battery2, l=$E_+$, name=eplus] ++(1,0)
to[R=$R_+$, name=rplus] ++(2,0) coordinate(skin+ b) -- ++(0.5,0)
-- ++(4,0) coordinate(hpin+) to[highpass] ++(2,0)
node[inst amp, anchor=+, noinv input up,
circuitikz/amplifiers/scale=1.6,
circuitikz/tripoles/inst amp/width=1](LNA){LNA}
(LNA.out);
\coordinate (skin- a) at (LNA.- -| skin+ a);
\draw (diffc) -| ++(0.5,0.5) to[sV, l=$v_-$, name=vminus] ++(0, 1) |- (skin- a);
\draw (skin- a) to[battery2, l=$E_-$, name=eminus] ++(1,0)
to[R, l=$R_-$, name=rminus] ++(2,0) coordinate(skin- b) -- ++(2.5,0)
-- (skin- b -| hpin+) to[highpass] (LNA.-);
\coordinate (gnd a) at (vcm -| skin+ a);
\draw (vcm) -- (gnd a) to[battery2, l=$E_{\mathrm{gnd}}$, name=egnd] ++(1,0)
to[R, l=$R_{\mathrm{gnd}}$, name=rgnd] ++(2,0) coordinate(gnd b);
% switch set
\def\swdown{-3.2}
\draw (skin- b) ++(1,0) coordinate(sw1) to[cosw, invert, mirror, l=1, *-, name=s1]
++(0,\swdown) to[short, -*] ++(0, -1.5);
\draw (sw1) ++(1,0) coordinate(sw2) to[cosw, invert, mirror, l=2, *-, name=s2] ++(0,\swdown)
to[R=$R$, -*] ++(0, -1.5);
\draw (sw2|-skin+ b) ++(1,0) coordinate(sw3) to[short, *-, name=s3] (sw3|-sw2) to[cosw,
invert, mirror, l=3,] ++(0,\swdown) to[R=$R$, -*] ++(0, -1.5);
\draw (sw3) ++(1,0) coordinate(sw4) to[short, *-, name=s4] (sw4|-sw2) to[cosw, invert, mirror,
l=4, name=s4] ++(0,\swdown) to[short] ++(0, -1.5) coordinate(endsw);
\draw (gnd b) |- (endsw) node[rectjoinfill]{};
% boxes
\node [rectangle, draw, dashed, fit=(GND0) (vplus) (vpluslabel) (vminuslabel)](body)
{};
\node [anchor=south east, align=center] at (body.south east) {Body} ;

```

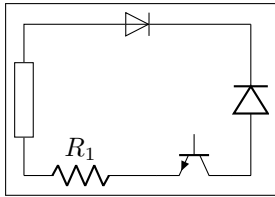
```

\node [rectangle, draw, dashed, fit=(rplus) (eplus) (epluslabel) (rpluslabel)](top)
{};
\node [rectangle, draw, dashed, fit=(eminus) (rminus) (eminuslabel) (rminuslabel)](
bot){};
\node [anchor=center, align=center] at ($(top.south)!0.5!(bot.north)$) {electrodes} ;

\node [rectangle, draw, dashed, fit=(egnd) (rgnd) (egndlabel) (rgndlabel)](gnd){};
\node [below, align=center] at (gnd.south) {ground\\ electrode} ;
\node [rectangle, draw, dashed, fit=(s1) (s4label), inner ysep=8pt](switches){};
% ADC and micro
\draw (LNA.out) -- ++(0.5,0) node[msport,circuitikz/RF/scale=2](ADC){ADC};
\draw (ADC.right) -- ++(0.5,0) node[twoportshape, anchor=left, t=$\upmu$C](uC){};
\draw (uC.south) -- (uC.south |- switches.east) -- ++(-4,0)
node[align=left, anchor=east](DCS){\small digitally\\ controlled\\ switches};
\draw[-Stealth] (DCS.west) -- (switches.east);
% components
\node [anchor=north west] at ([xshift=-10pt, yshift=-5pt]switches.south east) {ADG
1414};
\node [anchor=north west] at ([yshift=-5pt]LNA.refv down) {AD8429};
\end{circuitikz}
} % scalebox

```

10.8 An example with the compatibility option



```

\documentclass{standalone}

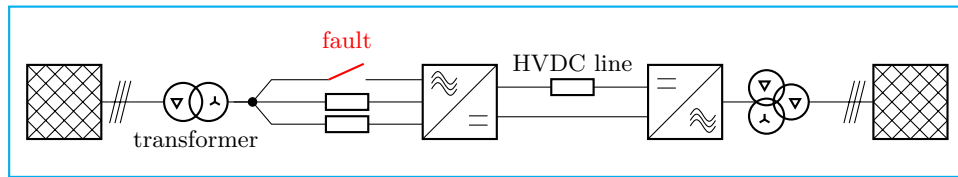
\usepackage{tikz}
\usetikzlibrary{circuits.ee.IEC}
\usetikzlibrary{positioning}

\usepackage[compatibility]{circuitikzgit}
\ctikzset{bipoles/length=.9cm}

\begin{document}
\begin{tikzpicture}[circuit ee IEC]
\draw (0,0) to [resistor={name=R}] (0,2)
to[diode={name=D}] (3,2);
\draw (0,0) to[*R=$R_1$] (1.5,0) to[*Tnpn] (3,0)
to[*D](3,2);
\end{tikzpicture}
\end{document}

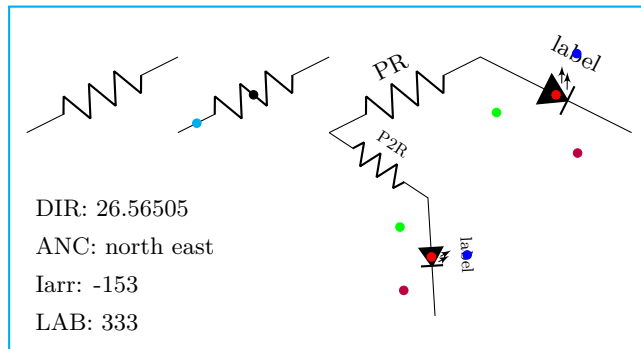
```

10.9 3-phases block schematic



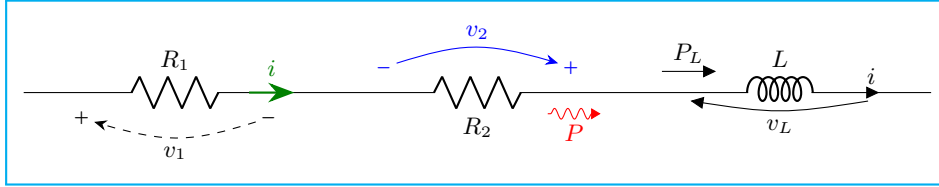
```
\begin{circuitikz}[smallR/.style={european resistor, resistors/scale=0.5}]
  \draw (0,0) node[tacdcshape, anchor=ac mid in](acdc){} to[smallR] ++(-2,0)
    -- coordinate(point) node[circ]({}) ++(-.5,0);
  \draw (acdc.ac up in)
    to[nos, invert, mirror, name=switch,color=red] ++(-2,0)
    -- (point);
  \draw (acdc.ac down in) to[smallR] ++(-2,0)
    -- (point)
    to[oosourcetrans,prim=wye,sec=delta,l=transformer] ++(-1.5,0)
    to[tmultiwire] ++(-.5,0)
    node[gridnode, anchor=right]{};
  \node[above=.3cm,color=red] at (switch) {fault};
  \draw (acdc.dc up out) to[smallR,l=HVDC line] ++(2,0 )
    node[tdcacshape, anchor=dc up in](dcac){};
  \draw (acdc.dc down out) -- (dcac.dc down in);
  \draw (dcac.right)
    to[oosource,prim=delta,sec=delta,tert=wye,invert] ++(1.5,0)
    to[tmultiwire] ++(.5,0) node[gridnode,anchor=left]{};
\end{circuitikz}
```

10.10 Using components in pics (since v1.8.0)



```
\tikzset{mypic/.pic = {
  \draw (0,0) coordinate(-in)
  % unnamed component
  to[R=#1] ++(2,1)
  % named component
  to[leD*, diodes/scale=0.6, led arrows from cathode,
    name=-led, l=label, v]
  ++(2,-1) coordinate(-out);
}
}
\begin{tikzpicture}
  \ctikzset{voltage=raised}
  \draw (0,0) to[R] ++(2,1);
  \begin{scope}[xshift=2cm,
    name prefix=inner-
  ]
    \draw (0,0) to[R, name=r, i<] ++(2,1);
  \end{scope}
  \draw (4,0) pic(P){mypic=PR};
  \node[circ] at (inner-r) {};
  \node[circ, cyan] at (inner-r-Ipos) {};
  \node[circ, red] at (P-led) {};
  \node[circ, blue] at (P-ledlabel) {};
  \node[circ, green] at (P-led-Vto) {};
  \node[circ, purple] at (P-led-Vfrom) {};
  \draw (4,0) pic[rotate=-60, scale=0.7, transform shape](P2){mypic=P2R};
  \node[circ, red] at (P2-led) {};
  \node[circ, blue] at (P2-ledlabel) {};
  \node[circ, green] at (P2-led-Vto) {};
  \node[circ, purple] at (P2-led-Vfrom) {};
  %% extra data
  \node[right] at (0, -1) {DIR: \ctikzgetdirection{inner-r}};
  \node[right] at (0, -1.5) {ANC: \ctikzgetanchor{P-led}{Vlab}};
  \node[right] at (0, -2) {Iarr: \ctikzgetdirection{inner-r-Iarrow}};
  \node[right] at (0, -2.5) {LAB: \ctikzgetdirection{P2-ledlabel}};
\end{tikzpicture}
```

10.11 Automatic user-defined voltages, currents and flows (since v1.8.5)



```

%% This can go to your preamble and/or a style file!
%% setup voltage drawing macro: european style, but adding "+" and "-"
\newcommand\eurVPMstyled[4]{% node, label, style, one ignored arguments
  \draw [thin, -{Stealth[width=4pt]}, shorten >=5pt, shorten <=5pt, #3]
    % NOTICE that boldmath requires double braces to work in nodes
    % see https://tex.stackexchange.com/q/487777/38080
    (#1-Vfrom) node[font=\tiny]{\boldmath$\vphantom{+}-}$}
    .. controls (#1-Vcont1) and (#1-Vcont2)..
    (#1-Vto) node[font=\tiny]{\boldmath$+$}
    node[pos=0.5, anchor=\ctikzgetanchor{#1}{Vlab}]{#2};
}
% voltage shift must go after the "european voltage, v" pair
\ctikzset{vpms/.style 2 args={european voltages, v, voltage shift=2,
  vif queue add={eurVPMstyled}{#1}{#2}{}}}
\ctikzactivatevoltagedirections{vpms}
% setup flow drawing macro: squiggle for power flux
%% requires \usetikzlibrary{decorations, decorations.pathmorphing}
\tikzset{ lray/.style={decorate, decoration={
  snake, amplitude=2pt, pre length=1pt, post length=2pt, segment length=5pt,},
  -Triangle}}
\newcommand\flowpow[4]{% node, label, color
  \draw [lray, #3 ] (#1-Ffrom) -- (#1-Fto)
    node [anchor=\ctikzgetanchor{#1}{Flab}, inner sep=4pt]
    at (#1-Fpos) {#2};}
\ctikzset{fpow/.style 2 args={f, vif queue add={flowpow}{#1}{#2}{}}}
\ctikzactivateflowdirections{fpow}
% setup current drawing macros: 6 mm with Stealth arrow on the line, dark green
\newcommand\inlinecurrent[4]{% node, label
  \draw [-{Stealth[scale=1.5]}, thick, color=green!50!black]
    ($(#1-Ipos)!3mm!(#1-Ifrom)$) -- ($(#1-Ipos)!3mm!(#1-Ito)$)
    node [midway, inner sep=6pt, anchor=\ctikzgetanchor{#1}{Ilab}] {#2};
}
\ctikzset{iline/.style = {i, vif queue add={inlinecurrent}{#1}{#2}{}}}
\ctikzactivatecurrentdirections{iline}
%% setup end
%
% Let's draw our circuit!
%
%% "use auto vif" only needed if using old (before 2020) LaTeX or Plain or ConTeXt
\begin{circuitikz}[use auto vif]
  \draw (0,0) to[R=$R_1$, iline={i}, vpms={$v_1$}{dashed}] ++(4,0)
    to[R, l_=$R_2$, vpms^>={$v_2$}{blue}, fpow={$P$}{red}] ++(4,0)
    to[L=$L$, i=$i$, v=$v_L$, f>=$P_L$] ++(4,0); % traditional still working!
\end{circuitikz}

```

11 Changelog and Release Notes

The major changes among the different CircuiTikZ versions are listed here. See <https://github.com/circuitikz/circuitikz/commits> for a full list of changes.

- Version 1.8.7 (unreleased)
Quite a bit of work on the [HTML version of the manual](#), and a new three-phase symbol.
 - New three-phase symbol `xdelta` (extended delta), [suggested by user sputeanus on GitHub](#)
 - Fix `nogate` anchor on some transistor ([without gates, but well...](#))
 - Minor fixes to the manual formatting: avoid overfull lines, change the voltage direction table code to play nicer with the html code, improve some example, and fix code snippet rendering.
- Version 1.8.6 (2026-05-24)
A shiny new style for the manual, with a new example code implementation by Jonathan P. Spratte, based on [the package enverb](#). The manual now avoid writing *hundreds* of auxiliary files, and compiles significantly faster. Additionally, now the manual can be compiled with `lwarp`, enabling the creation of a pure-HTML version. Additionally, a new option to simplify building custom blocks, thanks to a [suggestion by Matthias Jung](#)
 - New manual style, with new example code based on `enverb` (mainly by [Jonathan P. Spratte](#))
 - New option `border` for `component text` that applies to the `twoportsplit` component
 - New option `jfet gate height` to move the vertical position of JFET gate, triggered by [this question by Vector](#)
 - New options for relative gate thickness in FETs and MOSFETs, and rounded caps for transistors; suggested by [Michael Köfinger et al.](#)
 - Fixed a nasty bug with `oosourcetrans`, thanks to [user sputeanus](#) for spotting it.
- Version 1.8.5 (2026-02-4)
The main highlight of this version is a new mechanism for managing advanced (that is, user-defined) voltage, current, and flow elements. Additionally, a new set of shapes for ideal filter blocks has been added.
 - Add “automatic advanced voltages/currents/flows” ([by Romano](#)), marked as experimental for now.
 - Add an `ideal filter` option for plot-type filters, to draw ideal (square) filter shapes.
- Version 1.8.4 (2026-01-07)
The main highlights of this release are a new appearance (optional!) for blocks representing filters, some option to add style to the inner drawings of blocks (and some fixes for anchors too), and several new options.
 - Add a new set of filter blocks, add options for inner block drawings (by Romano)
 - Add the option to *not* draw the wiper in rotary switches, suggested by [@kabenyuk and @cis in this Q&A](#)
 - Add an option to change the aspect of `msrstub` (thanks to user cis, see [relevant chat on TeX.SX](#))
 - Add the singeneric (sine generic bipole) component (by [Jakob Leide](#))
 - Fix a problem with dc symbol dashes, see [this issue on GitHub](#)
 - Fix position of the left/right up/down anchors for node-type blocks (they did not obey the `pos` keys) and for amplifier-type blocks when boxed
 - Minor fixes in the manual (thanks [quark67!](#))
 - Make the value of `bipoles/length` usable by `\ctikzvalof` (by [Jonathan P. Spratte](#))
 - Add a key to customise the number of lines of the `gridnode` inner drawing (by [Jakob Leide](#))

- Version 1.8.3 (2025-11-23)

The main highlight of this version is the fix of the oo-type sources and transformers. They used parameters different from width and height to define the shape, and the anchors were not stable (and some of them were plain wrong). It also adds several capabilities to them (anchors, additional windings, and so on). Additionally, IGT thyristors and a couple of generic shapes (AC/DC symbols) have been added.

- Add anchors, additional winding, global scale switch and symbol rotation to the oo-type component, suggested by [Jakob Leide](#)
- Add IGCT thyristors by [Paul Sacco](#)
- Add ac/dc symbols (by Romano)
- Fix definition and stabilize anchors of oo- and ooo-type components.
- Fix encoding of the manual, removing some latin1 chars and converting to utf8. Why that was working is a mystery.

- Version 1.8.2 (2025-07-08)

Another significant change for Circuitikz’s internals: all the text anchors of the components are now stable, meaning they are meaningful after the component has been drawn. That was the case only for part of the components. It *shouldn’t* affect anything for standard usage, but according to the TikZ manual, this is “the way”. Additionally, new label positions for transistors, and a much better alphabetical index in the manual.

- Stabilize all the text anchors (problem [reported by user @JPWiedemann on GitHub](#), while coding the [Circuitikz GUI](#))
- Fix text anchors for blocks `oscillator` and `gridnode`, which were completely bogus
- Add `component text=up` or `...down` for transistor labels
- Documentation fixes for text anchors
- Better index entries (use *seealso*, hyperlink in every entry). Thanks to David Carlisle for the hint.

- Version 1.8.1 (2025-06-15)

Minor changes: better documentation on using `pics` for subcircuits, a new style of MOSFETs, completing the “plain” amplifiers.

- add Andrew Stacey’s [tikzmark library](#) to the manual
- add FDSOI MOSFETs, [suggested by Raphael Nägele](#)
- New component: `plain gm mono amp` (added by [Matthias Schweikardt](#))
- documentation enhancements

- Version 1.8.0 (2025-05-25)

The change that deserves a version level bump is applied to the path logic, which fixes a longstanding bug (or lack of feature), which enables the embedding of `circuitikz` paths into `pics` (among other things; see the related issue).

- Fix errors when path-style components are used in `pics` (and in some other place; fix by Romano, [see the related issue](#))
- Add rollback point for version 1.7.2
- Several fixes and additions to the manual

- Version 1.7.2 (2025-03-21)

A couple of new components (wiggly bulbs and a new “oo” type autotransformer), and several small corrections.

- Fix leaking filament in `bulb` bipole
- Fix small incoherence between width/height in oo-type sources

- New component: Wiggly bulb (suggested by [Sebastiano](#))
- New component: “zero-type” autotransformer (suggested by [Max Börjesson](#))
- Allow to set the index position for square instruments (by [Julien Labbé](#))
- Version 1.7.1 (2025-01-10)

Various new blocks have been added, and several fixes have been applied (the barrier one is slightly backward-incompatible. . .).

 - Added a flag to have German style TVS (suggested by [Dr. Matthias Jung](#), implemented by Romano)
 - Added many blocks all over the map (suggested by [Dr. Matthias Jung](#), with tweaks by Romano)
 - Fix for straight voltage on **open** bipoles (reported by [Oliver Wallscheid on GitHub](#))
 - Fix a very, very old bug about aliases for american/european sources
 - Fix **barrier** wire linewidth (issue [#833](#) by schtandard).
 - Fix stroke-type transorb (which did not work at all)
 - Reduce **barrier** and **openbarrier** default widths so no wire is drawn by default. This breaks backward-compatibility and changes the meaning of some associated keys, but the appearance with the default settings remains unchanged. See [#835](#) for rationale.
 - Documentation enhancement (example of chopper macro)
- Version 1.7.0 (2024-08-03)

There are no big changes here, but the change to the resistor code (maybe one of the most used by the package) well deserves a minor version bump. A couple of new components, and several minor fixes.

 - New component: new kind of current tap (suggested by [EEpchi and Jakob Leide on GitHub](#))
 - New arrow tip **Jack Tap** to help drawing jack connectors (suggested by [Anisio Rogerio Braga](#))
 - Change the drawing of the thermocouple (suggested by [Jakob Leide on GitHub](#))
 - Change and enhancement to the drawing of the American resistors (triggered by [Jakob Leide on GitHub](#)), fixing a long-standing small asymmetry that nobody noticed
 - Minor adjustment for joins in **viscoe** component
 - Minor additions (**rectjoinfill**) and fixes in documentation
- Version 1.6.9 (2024-05-25)

Several new components and a bug fix for a nasty long-standing bug about switching diode types.

 - Added a Relais-Shape (contributed by [Jakob Leide on GitHub](#))
 - Added a center tap anchor for tube filament (suggested by [user bogger33 on GitHub](#))
 - Added neon lamps (two versions, suggested by [user bogger33 on GitHub](#))
 - Added a configurable spark gap (suggested by [user bogger33 on GitHub](#))
 - Fix a long-standing problem when [\(locally\) switching diode type](#)
- Version 1.6.8 (2024-05-05)

Several new components, more anchors, a bit of documentation enhancement; maybe the biggest change is the new “flexible” tube.

 - Added **mid** anchor to all traditional switches
 - Added a slashed generic European-style resistor (thanks to [Jana](#))
 - Added a multi-anode tube for implementing nixies and vfd (thanks to [GitHub user nogger33](#))
 - Switch the default compiler to pdflatex (see <https://tex.stackexchange.com/q/709273/38080>)
 - Added a warning about color and engine in the documentation
 - Enhanced the documentation for instruments (thanks to [Github user mxxmxm](#))

- Version 1.6.7 (2024-02-09)
Several new blocks, more flexible generic anchors for blocks, and a new option to align the signs on american-style voltage sources.
 - Added **saturation** block (contributed by [P. Sacco <paul.sacco@estaca.eu>](#))
 - Added **iamp**, **sigmoid**, and **allornothing** blocks
 - Added optical fiber **fiber** (contributed by [Christopher Beck](#))
 - Now the position of the lateral anchors (**left up** and similar) of blocks is configurable (suggested by user “[sputeanus](#)” on [GitHub](#))
 - Now you can choose how the signs on american-style sources rotate when the source is not vertical (suggested by [jotagah](#) on [GitHub](#))
 - New section in the manual about related packages
- Version 1.6.6 (2023-12-09)
Several new components.
 - Added the symbol for metal-oxide varistor **mov**
 - Added another symbol for fuse (wiggly fuse **wfuse**)
- Version 1.6.5 (2023-10-29)
This version features an important overhaul of the **muxdemux** configurable component/shape, making it much more flexible and powerful, by adding configurable labels and negation and clock symbols to the pins. Also, a couple of minor fixes/workarounds.
 - Added optional and configurable inner, outer and border labels to the **muxdemux** shapes
 - Added optional clock wedge and negation signs to the pins of **muxdemux** shapes
 - Added the possibility to add a background drawing to **muxdemux** shapes
 - Fixed a [bug](#) with **straightvoltages** and **open**
 - Added an (ugly) workaround for a [voltage shift mismatch](#) for sources
- Version 1.6.4 (2023-10-10)
A bit of enhancement and fixes for the European-style logic ports, more switches (and a bit more configurability for them), more option for some sources.
 - The symbol in European logic ports is now rotation-invariant, and its font can be customized (suggested by user [@sputeanus](#) on [GitHub](#))
 - Added a couple of “blank” (no symbol) European logic ports
 - Added new “traditional” switches (contributed by [Jakob Leide](#) on [GitHub](#))
 - Added configurability (color, thickness, dash) to switch arrows
 - Added “eyw”-symbol (reverse star) for “oo”-type sources (contributed by [Jakob Leide](#) on [GitHub](#))
 - Added configurable open shape to the sinusoidal current source (contributed by [Maximilian Martin](#))
 - Documentation fixes
- Version 1.6.3 (2023-06-23)
The main change is that the definition of the “plus” and “minus” symbols used in several parts of the library has changed in order to achieve better alignment of voltages and amplifier symbols when using fonts different from Computer Modern. Additionally, internal connection dots in transistors are configurable and have a new default, and documentation has got several fixes and enhancements.
 - Change the definition of the “minus” symbol (see [this issue](#)) for details
 - Add documentation on how to contact the border of the source symbols (suggested by user [@Tipounk](#) on [GitHub](#))

- in transistors, solder dots and connection dots for body diodes [are now configurable](#)
 - Add anchors for the symbols on the oo-type sources, suggested by [user @lapreindl on GitHub](#); the symbols have been slightly changed and repositioned in the process
 - several documentation fixes
- Version 1.6.2 (2023-05-13)

Several more styling options for elements (body diodes, transformers, crossing), a clock wedge shape for logical circuits, and documentation updates for ConTeXt, mainly noticing the (upstream) elimination of the thin `siunitx` layer compatibility macros.

 - There is no `siunitx` support for ConTeXt, point to the `units` package
 - `context` compatibility can have glitches: please see [this issue](#)
 - Add styling of `transform core` lines (suggested by [user @myzinsky on GitHub](#))
 - Add `scale` to the bodydiode options (suggested by [user @sputeanus on GitHub](#))
 - Add styling of crossing vertical line (suggested by [user @lkjell on GitHub](#))
 - Add `clockwedge` shape (suggested by [user @Mario1159 on GitHub](#))
 - Version 1.6.1 (2023-02-11)

New components: solder jumpers; a couple of small but very useful inversion markers for logical circuits, especially targeted at the mux-demux family; a new inline microphone; a much more versatile hemt; a better legacy `tline`. More tweaks to converters blocks, and a lot of typo/grammar fixes in the manual.

 - Add configurable dashes to the dc symbols in converter blocks (suggested by [user @dbstf on GitHub](#))
 - Add solder jumpers (by Romano)
 - Add a shape to mark european-style inversion (suggested by [user yashpalgoyal1304 on GitHub](#)), adjust European-style logic port triangle inversion symbols to match
 - Add a tail-less mic (suggested by [Dr. Mathhias Jung](#)) and an option to change the thickness of the microphone’s bar
 - Enhance the `hemt` shape with a GaN-hemt as example (suggested by [user @epsilon-phi on GitHub](#))
 - Add anchors and a “bare” option to `tline` (suggested by [Dr. Mathhias Jung](#))
 - subcircuits are no more experimental
 - Correction of several typo/grammar errors in the documentation by [quark67](#)
 - Version 1.6.0 (2022-12-10)

The big change is the refactoring (and enhancement) of the block’s code. In addition, double gate MOSes, several fixes all over the map, and quite a lot of anchors were added into the mix.

 - Big change (mostly backward compatible, minus a couple of bug fixes) to the block’s code.
 - * Now `vco` can be boxed
 - * enabled more short-name geographical anchors
 - * generic blocks can be made rectangular
 - * mid-way lateral anchors for all blocks, as well as up/down
 - * renamed converters anchors (old ones retained for backward compatibility)
 - * new `ac/ac` blocks, both single- and three-phase
 - Added double gate MOS transistors (by Romano Giannetti)
 - Fix deformed shape for legacy `TL` component ([issue on GitHub](#))
 - Added several anchors on variable components, suggested by [Dr Matthias Jung](#)
 - Added `genericsplitter` component (by [frankplow](#))

- Fix - reshape `splitter` using `/tripoles/splitter/width` and `/tripoles/splitter/height` rather than `/tripoles/wilkinson/width` and `/tripoles/wilkinson/height`.
- Version 1.5.5 (2022-11-12)
New features for optoelectronic devices: a new component, arrow styling, and anchors.
 - Added styling of arrows on opto devices, thanks to a suggestion by [Dr Matthias Jung](#)
 - Added Light-Dependent resistor shape (by Romano)
 - Added `arrows` anchors to the opto-components
 - Documentation updates (rotating and flipping for path components)
- Version 1.5.4 (2022-09-09)
New components and enhancement for old ones in this version.
 - Added jumpers, inspired by a question [on TeX.stackexchange](#)
 - Added generic double bipoles, inspired by user [@erwinderboer](#) [on GitHub](#)
 - Added styling for the transistor bodydiode, suggested by user [Alex Ghilas on TeX.stackexchange](#)
 - Additions to the manual (how to remove pins on amplifiers)
- Version 1.5.3 (2022-07-02)
Minor release: fixes to the manual, and a new component (Shockley diodes).
 - Merging changes to fix the language in the manual (thanks to Charles B. Cameron, user [@cameroncb1](#) [on GitHub](#))
 - Added Shockley diode (suggested by [@dauph](#))
- Version 1.5.2 (2022-05-08)
Adding a couple of new component and a nice feature to transistors and tubes.
 - Added TVS diodes (transorb), suggested by [Anisio Rogerio Braga](#)
 - Added proximity switches, suggested by [Anisio Rogerio Braga](#)
 - Added partially drawn tube and transistor borders, suggested by [Jether Fernandes Reis](#)
- Version 1.5.1 (2022-04-26)
Bug fix release.
 - Do not load package `regexpatch` by default, thanks to [GitHub user alceu-git](#)
- Version 1.5.0 (2022-04-22)
In this version, several internal changes have been included in order to streamline and organize better the components and to change the management of color. The changes are pretty deep and subtle, so a bug or unexpected behaviour is always possible. You can use the 1.4.6 rollback point in case of trouble, but be sure to report any bug.
 - Added connectors shapes, and included the BNC into that class; thanks to [Alexander Sauter for suggesting them and helping in the design](#)
 - Added nullator and norator shapes, suggested by [user atticus-sullivan on GitHub](#)
 - Added buzzer and reversed buzzer bipoles, suggested by [user Michael.H](#)
 - Added “dot” anchors to inductances
 - Added “boxed only” option for some circular blocks, suggested by [user myzinsky](#)
 - Added DIN antenna shape, suggested by [user myzinsky](#)
 - Fixed block/input arrow connection, thanks to [Laurenz Preindl for reporting](#)
 - Fixed a problem with generic tunable arrows, noticed thanks to [this question on TeX.SX](#)

Internal changes:

- Added a generic drawing function for shapes, which are now drawn always in background
- Added a hook system to be able to change component drawing settings per-shape, per-class or globally
- All the 250+ shapes are now “protected” by possible external arrow and arced corners parameters
- Completely changed the management of the shapes’ color, thanks to [GitHub user muzimuzhi](#)
- Version 1.4.6 (2022-02-04)

A nasty bug fix and some hack to avoid that some global TikZ option spill into the shapes. To better solve that problem, some risky changes are due, so this release will be also a rollback point for compatibility reasons.

 - Fix bug with legacy transmission lines in **overlays** ([noticed by Benedikt Wilde](#))
 - Robustify some shapes: do not let arrows option pass to the inner drawing (see [here](#) and [here](#))
 - Add warning about global draw options in the manual
 - Fixes in documentation: hyperlink the index again, cite new recovery point, remove some legacy construct
 - Added 1.4.6 rollback point
- Version 1.4.5 (2021-12-06)

Important fix for ConTeXt users, thanks to @TeXnician for reporting.

 - Fixed an incompatibility introduced with subcircuits that made compilation in ConTeXt fail
 - Added `\ctikzflip[x][y]` utility macros for ConTeXt too
 - Fixed stray characters in some TikZ environment
- Version 1.4.4 (2021-10-31)

Normal maintenance release; minor bugs fixed, a new component and a new option. No Halloween component, sorry...

 - Added a laser diode component ([contributed by André Alves](#))
 - Add the **override source vif** option and better describe source’s voltage positioning in the manual
 - fix **nobase** option with IGBT family ([noticed by user hinata exc on Stack Eschange](#))
 - fix a problem with [legacy open voltage label position](#)
- Version 1.4.3 (2021-09-06)

Minor release, mainly a single bugfix.

 - added hidden anchors of **oosource** to the manual
 - fix a bug in anchors of **oosource** (they did not respect class scaling)
 - faster **use fpu reciprocal** (thanks to Henri Menke)
- Version 1.4.2 (2021-07-26)

This is a minor release, containing just a new component and a small set of fixes (mainly in the documentation).

 - add the **cpe** (constant phase element)
 - correct minor errors in the manual (capacitor’s fill, spaces) and the code.
- Version 1.4.1 (2021-07-14)

This version has an important bug fix for label positioning when once-relative style coordinates are used (the ones with a single +, like `+(1,1)`). Moreover, the possibility to have voltage, current and flow labels *without* the symbols (arrows, etc) has been added, which greatly simplify some kind of personalization of these elements.

 - Added the generic tunable macro

- Added `no v` symbols (and also for `i` and `f`), thanks to a [head-up by user judober on GitHub](#), see also [issue 448](#)
- Fixed [label position for +\(\) style coordinates](#)
- Version 1.4.0 (2021-07-06)

The main news is that *package rollback* for `circuitikz` has been implemented (LaTeX-only, of course). Additionally, a small but important change in the path (`to`) construction that should fix some warning from TikZ and give better line joins in wire corners.

 - bump version to 1.4.0
 - implement the version rollback: time travel to 0.4!
 - remove a wrong movement in the path construction (potentially dangerous)
- Version 1.3.9 (2021-06-27)

Bugfix release: `open poles opacity` was not working in most of the cases.

 - minor fixes to the manual
 - fix bug with `open poles opacity`; see [this question by Florian H.](#) for details.
- Version 1.3.8 (2021-06-15)

The big news of this release is the ability to selectively draw the pins of the integrated circuit and mux-demuxes symbols.

 - Add `draw only pins` feature to `dipchip` and `qfpchip`, thanks to [Jonathan P. Spratte](#), and a similar option to control the pins of `muxdemux`
 - Make `dipchip` and `qfpchip` respect `no input leads` option
 - Several corrections to the manual
- version 1.3.7 (2021-06-01)

Minor release, mainly documentation upgrades.

 - New options for the line thickness, rotation and size of symbols drawn in sources
 - New tutorial: drawing a circuit around an operational amplifier
 - Documentation fixes and small enhancements
- version 1.3.6 (2021-05-09)

Mainly a bugfix release; fixing a bug in the 12 stacked labels means that old constructs that were failing silently can give an error now. Sorry. To compensate, I added stacked annotation (for symmetry).

 - Added stacked annotations for symmetry with stacked labels.
 - Fixed a bug in the plotting of `inst amp ra` terminals.
 - Fixed a bug in managing stacked labels (`12=...`); possibly it will be mildly backward-incompatible (please see the manual about incompatible changes)
- Version 1.3.5 (2021-05-02)

Power electronics devices are the main characters in this release: PUT, GTOs, a new style for thyristors, and a photovoltaic module. Additionally, an **experimental** support for subcircuits has been added; it could change in the future. Fixed a nasty bug in rotary switches “in” anchor positioning in some cases.

 - Added support for creating and using sub-circuits
 - Added UJT transistors and GTO devices ([suggested by JetherReis](#))
 - Added (as an option) a different, more compact style for thyristor-type devices.
 - Added a photovoltaic module ([suggested by André Alves](#))
 - Added a DC/DC converter block for symmetry ([suggested by Pratched](#))

- Added the possibility to change the waveforms shown in the oscilloscope ([suggested by Mario Tafur](#))
- In the manual, separate the component usage chapter from the big component list
- Fix wrong rotary switch “in” anchors for switches with more than 180 degrees coverage ([see bug](#))
- Version 1.3.4 (2021-04-20)

New things, like configurable modifier thickness, ferroelectric devices, and several transistor tweaks. Importantly, a bug that hindered compatibility with the internal TikZ `circuits` library (introduced in 1.3.3) has been fixed.

 - Added separate configuration for the line thickness of resistors, capacitors, and inductors modifiers
 - Added ferroelectric capacitors and ferroelectric gate MOS/FETs ([suggested by Mayeul Cantan](#))
 - Added an option to fill the gate gap in MOSes, FETs and IGBTs with a color
 - Added a “centergap” anchor for transistors
 - Added the option “nogate” to the `hemt` symbol
 - Fixed a bug in thermistors not respecting their class line thickness
 - Fixes in the manual (copy and paste of snippets without numbers, correct old usage of `siunitx`, factor out repetitions in the preamble; [thanks to Ulrike Fischer](#).)
 - Fixed a bug introduced in 1.3.3 that would reduce compatibility with the `circuits` internal library; [reported by JetherReis](#))
- Version 1.3.3 (2021-04-04)

Several usability additions in this version, and one small fix that could change the look of your circuit (without affecting correctness). Some of the arrow shapes are now configurable. Do not use this version, there is a bug with the new “label distance” key.

 - Added options to fine-tune the position of labels and annotations
 - Added options to change arrow tips on variable resistors, inductors and capacitors as well as in potentiometers
 - Added options to change arrow tips on switches
 - Added anchors to inductance to add core lines
 - Fixed the default direction of tunable arrows (with an option to go back to the old ones)
- Version 1.3.2 (2021-03-14)
 - Added the simplified (2-waves) highpass and lowpass blocks
 - Added graphene FETs ([suggested by Cees Keyer](#))
 - Added left/right anchors to transistors
 - Fixed a [bug in flip-flops](#)
- Version 1.3.1 (2021-02-20)
 - Fixed a bug in “fuse” and “afuse” fill
 - Remove the voltage direction warning. Nobody really ever cared
 - Minor fixes and enhancements to the manual
- Version 1.3.0 (2021-01-19)
 - Fixed a long-standing problem with labels and similar decoration with equal signs and commas
 - Fixed a typo in the manual (thanks to @muzimuzhi on GitHub)
 - The Mother of All Code Refactoring: no real changes (modulo errors)
 - Added a rollback point to 1.2.7

- Version 1.2.7 (2020-12-27)
Bugfix release.
 - The recent temporary changes to TikZ to v3.1.8a revealed a problem in corner cases with `circuitikz` that should be fixed (thanks to Henri Menke)
- Version 1.2.6 (2020-12-16)
The highlight of this release is the option to draw circles around transistors; moreover, a handful of new component and several bug fixes.
 - added option to have transistors with circles, suggested by user `@myzinsky`
 - added closed position for normally open button and the other way around (suggested by user `@septatrix`)
 - added a `tip` anchor for push buttons
 - added text anchor for legacy `linestub` component
 - added an option for a different style of european logic xnor port (suggested by user `@Schlepptop`)
 - added dynode tubes electrodes (suggested by user `@ferdymercury`)
 - fixed a bug in style-files (thanks to user `@Alex` on `tex.stackexchange.com`)
 - added a comment about relative coords (thanks to user `@septatrix`)
 - several fixes to the manual
- Version 1.2.5 (2020-10-14)
Mainly a bugfix release for `raised` voltage style.
 - added macro to access labels and annotations anchors and direction
 - fixed a bug in “raised” voltages’ positions with `invert` and/or `mirror`
- Version 1.2.4 (2020-10-04)
 - several documentation enhancement
 - added a couple of block elements: allpass filter, generic two-sides block (suggested by user `@myzinsky`)
 - added transmission gate (only IEEE style version) suggested by several users (`@SJulianS` on github, Philipp Birkel on `TeX.SX`)
 - added a resistive splitter block symbol by `@matthuszagh`
 - added depletion-type `nmosd` and `pmosd` MOSFET simplified symbols
 - added depletion-type `nfetd` and `pfetd` for plain full-symbol MOSFET
- Version 1.2.3 (2020-08-07)
Several fixes and small enhancement all over the map, changes in the documentation to better explain the reasons and effect of the path-building changes of 1.2.0 and 1.2.1.
 - added a Mach-Zehnder-Modulator block symbol as node `mzm` by user `@dl1chb`
 - add an `open poles fill` option to simplify circuits where the background is different from white
 - restyled the FAQ and added the explanation of “gaps with `nodes`” that happens again after 1.2.1
 - Fixed size of “not circle” in flip-flops to match european style `not circle` when used without the IEEE style
 - Block anchors: add border anchors for round elements and deprecate old 1, 2, 3, 4 anchors
 - Fixed some bipole border size to avoid overlapping labels; document it
- Version 1.2.2 (2020-07-15)
Bug-fix release: coordinate name leakage. The node and coordinate names are global; the internal coordinate names have been made stronger.

- Version 1.2.1 (2020-07-06)

Several changes, both internal and user-visible. These are quite risky, although they *should* be backward-compatible (if the circuit code is correct).

From the user point of view:

- there is now a new style of voltages (“raised American”)
- a powerful mechanism for customize voltages, current and flows has been added.

The internal changes are basically the re-implementation of the macros that draw the path elements (`to[...]`), which have been completely rewritten. Please be sure to read the possible incompatibilities in the manual (section 1.9).

- Added access to voltages, currents and flows anchors
- Added “raised american” voltage style
- Rewrite of the path generation macros
- Several small bugs fixed (no one ever used some “f^>” options...)

- Version 1.2.0 (2020-06-21)

In this release, the big change is the rewriting of the voltages output routine. Now all voltage options (american, european, and straight) take into account the shape (square border) of the component. The adjusting parameters are now (at least for passive elements) acting in similar way for all the options, too.

- Bumped version number to 1.2 (potentially incompatible changes!)
- Added 1.1.2 checkpoint
- New path-style not, buffer, and Schmitt logic ports
- New tutorial (using the “inline not” component)
- Voltage output routine rewrite; now it takes into account the shape of the component also for “american” and “straight” voltages
- Several fixes in the logic ports: fixed IEEE `invschmitt` name, added symmetry to the three-style shorthands for the ports, and so on
- Fixed a gross bug in square poles anchor borders
- Fixed size of not circles in flip-flops (based on logic ports style)
- Fixed the order of initial options, to avoid “european” overwriting single options

- Version 1.1.2 (2020-05-17)

- Blocks and component for three-phase networks (3-lines wire, AC/DC and DC/AC converters blocks and grid node block) added by user `@olffline` on GitHub
- added transformer sources with optional vector groups for three-phase networks by `@olffline` on Github
- added subsections to the examples
- fixed position of american voltages on open circuits (suggested by user `@rhandley` on GitHub)

- Version 1.1.1 (2020-04-24)

One-line bugfix release for the IEEE ports “not” circle thickness

- Version 1.1.0 (2020-04-19)

Version bumped to 1.1 because the new logic ports are quite a big addition: now there is a new style for logic ports, conforming to IEEE recommendations.

Several minor additions all over the map too.

- added IEEE standard logic ports suggested by user Jason-s on GitHub
- added configurability to european logic port “not” output symbol, suggested by j-hap on GitHub

- added **inverter** component by user Tadashi on GitHub
- added variable outer base height for IGBT, suggested by user RA-EE on GitHub
- added configurable “+” and “-” signs on american-style voltage generators
- text on amplifiers can be positioned to the left or centered
- Version 1.0.2 (2020-03-22)
 - added Schottky transistors (thanks to a suggestion by Jérôme Monclard on GitHub)
 - fixed formatting of **CHANGELOG.md**
- Version 1.0.1 (2020-02-22)

Minor fixes and addition to 1.0, in time to catch the freeze for TL2020.

 - add v1.0 version snapshots
 - added crossed generic impedance (suggested by Radványi Patrik Tamás)
 - added open barrier bipole (suggested by Radványi Patrik Tamás)
 - added two flags to flip the direction of light’s arrows on LED and photodiode (suggested by karlkappe on GitHub)
 - added a special key to help with precision loss in case of fractional scaling (thanks to AndreaDiPietro92 on GitHub for noticing and reporting, and to Schrödinger’s cat for finding a fix)
 - fixed a nasty bug for the flat file generation for ConTeXt
- Version 1.0 (2020-02-04)

And finally... version 1.0 (2020-02-04) of **circuitikz** is released.

The main updates since version 0.8.3, which was the last release before Romano started co-maintaining the project, are the following — part coded by Romano, part by several collaborators around the internet:

 - The manual has been reorganized and extended, with the addition of a tutorial part; tens of examples have been added all over the map.
 - Around 74 new shapes were added. Notably, now there are chips, mux-demuxes, multi-terminal transistors, several types of switches, flip-flops, vacuum tubes, 7-segment displays, more amplifiers, and so on.
 - Several existing shapes have been enhanced; for example, logic gates have a variable number of inputs, transistors are more configurable, resistors can be shaped more, and more.
 - You can style your circuit, changing relative sizes, default thickness and fill color, and more details of how you like your circuit to look; the same you can do with labels (voltages, currents, names of components and so on).
 - A lot of bugs have been squashed; especially the (very complex) voltage direction conundrum has been clarified and you can choose your preferred style here too.

A detailed list of changes can be seen below.

- Version 1.0.0-pre3 (not released)
 - Added a Reed switch
 - Put the copyright and license notices on all files and update them
 - Fixed the loading of style; we should not guard against reload
- Version 1.0.0-pre2 (2020-01-23)

Really last additions toward the 1.0.0 version. The most important change is the addition of multiplexer and de-multiplexers; also added the multi-wires (bus) markers.

 - Added mux-demux shapes
 - Added the possibility to suppress the input leads in logic gates

- Added multiple wires markers
- Added a style to switch off the automatic rotation of instruments
- Changed the shape of the or-type american logic ports (reversible with a flag)
- Version 1.0.0-pre1 (2019-12-22)

Last additions before the long promised 1.0! In this pre-release we feature a flip-flop library, a revamped configurability of amplifiers (and a new amplifier as a bonus) and some bug fix around the clock.

 - Added a flip-flop library
 - Added a single-input generic amplifier with the same dimension as “plain amp”
 - Added border anchors to amplifiers
 - Added the possibility (expert only!) to add transparency to poles (after a suggestion from user @matthuszagh on GitHub)
 - Make plus and minus symbol on amplifiers configurable
 - Adjusted the position of text in triangular amplifiers
 - Fixed “plain amp” not respecting “noinv input up”
 - Fixed minor incompatibility with ConTeXt and Plain TeX
- Version 0.9.7 (2019-12-01)

The important thing in this release is the new position of transistor’s labels; see the manual for details.

 - Fix the position of transistor’s text. There is an option to revert to the old behavior.
 - Added anchors for adding circuits (like snubbers) to the flyback diodes in transistors (after a suggestion from @EdAlvesSilva on GitHub).
- Version 0.9.6 (2019-11-09)

The highlights of this release are the new multiple terminals BJTs and several stylistic addition and fixes; if you like to pixel-peep, you will like the fixed transistors arrows. Additionally, the transformers are much more configurable now, the “pmos” and “nmos” elements have grown an optional bulk connection, and you can use the “flow” arrows outside of a path.

Several small and less small bugs have been fixed.

 - Added multi-collectors and multi-emitter bipolar transistors
 - Added the possibility to style each one of the two coils in a transformer independently
 - Added bulk connection to normal MOSFETs and the respective anchors
 - Added “text” anchor to the flow arrows, to use them alone in a consistent way
 - Fixed flow, voltage, and current arrow positioning when “auto” is active on the path
 - Fixed transistors arrows overshooting the connection point, added a couple of anchors
 - Fixed a spelling error on op-amp key “noinv input down”
 - Fixed a problem with “quadpoles style=inner” and “transformer core” having the core lines running too near
- Version 0.9.5 (2019-10-12)

This release basically add features to better control labels, voltages and similar text “ornaments” on bipoles, plus some other minor things.

On the bug fixes side, a big incompatibility with ConTeXt has been fixed, thanks to help from @TheTeXnician and @hmenke on github.com.

 - Added a “midtap” anchor for coils and exposed the inner coils shapes in the transformers
 - Added a “curved capacitor” with polarity coherent with “ecapacitor”
 - Added the possibility to apply style and access the nodes of bipole’s text ornaments (labels, annotations, voltages, currents and flows)

- Added the possibility to move the wiper in resistive potentiometers
- Added a command to load and set a style in one go
- Fixed internal font changing commands for compatibility with ConTeXt
- Fixed hardcoded black color in “elko” and “elmech”
- Version 0.9.4 (2019-08-30)

This release introduces two changes: a big one, which is the styling of the components (please look at the manual for details) and a change to how voltage labels and arrows are positioned. This one should be backward compatible *unless* you used **voltage shift** introduced in 0.9.0, which was broken when using the global **scale** parameter.

The styling additions are quite big, and, although in principle they are backward compatible, you can find corner cases where they are not, especially if you used to change parameters for **pgfcirc.defines.tex**; so a snapshot for the 0.9.3 version is available.

 - Fixed a bug with “inline” gyrators, now the circle will not overlap
 - Fixed a bug in input anchors of european not ports
 - Fixed “tlinestub” so that it has the same default size than “tline” (TL)
 - Fixed the “transistor arrows at end” feature, added to styling
 - Changed the behavior of “voltage shift” and voltage label positioning to be more robust
 - Added several new anchors for “elmech” element
 - Several minor fixes in some component drawings to allow fill and thickness styles
 - Add 0.9.3 version snapshots.
 - Added styling of relative size of components (at a global or local level)
 - Added styling for fill color and thickness
 - Added style files
- Version 0.9.3 (2019-07-13)
 - Added the option to have “dotless” P-MOS (to use with arrowmos option)
 - Fixed a (puzzling) problem with coupler2
 - Fixed a compatibility problem with newer PGF (>3.0.1a)
- Version 0.9.2 (2019-06-21)
 - (hopefully) fixed ConTeXt compatibility. Most new functionality is not tested; testers and developers for the ConTeXt side are needed.
 - Added old ConTeXt version for 0.8.3
 - Added tailless ground
- Version 0.9.1 (2019-06-16)
 - Added old LaTeX versions for 0.8.3, 0.7, 0.6 and 0.4
 - Added the option to have inline transformers and gyrators
 - Added rotary switches
 - Added more configurable bipole nodes (connectors) and more shapes
 - Added 7-segment displays
 - Added vacuum tubes by J. op den Brouw
 - Made the open shape of dcisources configurable
 - Made the arrows on vcc and vee configurable
 - Fixed anchors of diamondpole nodes
 - Fixed a bug (#205) about unstable anchors in the chip components
 - Fixed a regression in label placement for some values of scaling

- Fixed problems with cute switches anchors
- Version 0.9.0 (2019-05-10)
 - Added Romano Giannetti as contributor
 - Added a CONTRIBUTING file
 - Added options for solving the voltage direction problems.
 - Adjusted ground symbols to better match ISO standard, added new symbols
 - Added new sources (cute european versions, noise sources)
 - Added new types of amplifiers, and option to flip inputs and outputs
 - Added bidirectional diodes (diac) thanks to Andre Lucas Chinazzo
 - Added L,R,C sensors (with european, american and cute variants)
 - Added stacked labels (thanks to the original work by Claudio Fiandrino)
 - Make the position of voltage symbols adjustable
 - Make the position of arrows in FETs and BJTs adjustable
 - Added chips (DIP, QFP) with a generic number of pins
 - Added special anchors for transformers (and fixed the wrong center anchor)
 - Changed the logical port implementation to multiple inputs (thanks to John Kormylo) with border anchors.
 - Added several symbols: bulb, new switches, new antennas, loudspeaker, microphone, coaxial connector, viscoelastic element
 - Make most components fillable
 - Added the oscilloscope component and several new instruments
 - Added viscoelastic element
 - Added a manual section on how to define new components
 - Fixed american voltage symbols and allow to customize them
 - Fixed placement of straightlabels in several cases
 - Fixed a bug about straightlabels (thanks to @fotesan)
 - Fixed labels spacing so that they are independent on scale factor
 - Fixed the position of text labels in amplifiers
- Version 0.8.3 (2017-05-28)
 - Removed unwanted lines at to-paths if the starting point is a node without a explicit anchor.
 - Fixed scaling option, now all parts are scaled by bipoles/length
 - Surge arrester appears no more if a to path is used without []-options
 - Fixed current placement now possible with paths at an angle of around 280°
 - Fixed voltage placement now possible with paths at an angle of around 280°
 - Fixed label and annotation placement (at some angles position not changable)
 - Adjustable default distance for straight-voltages: ‘bipoles/voltage/straight label distance’
 - Added Symbol for bandstop filter
 - New annotation type to show flows using $f=\dots$ like currents, can be used for thermal, power or current flows
- Version 0.8.2 (2017-05-01)
 - Fixes pgfkeys error using alternatively specified mixed colors(see pgfplots manual section “4.7.5 Colors”)
 - Added new switches “ncs” and “nos”

- Reworked arrows at spst-switches
- Fixed direction of controlled american voltage source
- “v<=” and “i<=” do not rotate the sources anymore(see them as “counting direction indication”, this can be different then the shape orientation); Use the option “invert” to change the direction of the source/appearance of the shape.
- current label “i=” can now be used independent of the regular label “l=” at current sources
- rewrite of current arrow placement. Current arrows can now also be rotated on zero-length paths
- New DIN/EN compliant operational amplifier symbol “en amp”
- Version 0.8.1 (2017-03-25)
 - Fixed unwanted line through components if target coordinate is a name of a node
 - Fixed position of labels with subscript letters.
 - Absolute distance calculation in terms of ex at rotated labels
 - Fixed label for transistor paths (no label drawn)
- Version 0.8 (2017-03-08)
 - Allow use of voltage label at a [short]
 - Correct line joins between path components (to[...])
 - New Pole-shape .-. to fill perpendicular joins
 - Fixed direction of controlled american current source
 - Fixed incorrect scaling of magnetron
 - Fixed: Number of american inductor coils not adjustable
 - Fixed Battery Symbols and added new battery2 symbol
 - Added non-inverting Schmitttrigger
- Version 0.7 (2016-09-08)
 - Added second annotation label, showing, e.g., the value of an component
 - Added new symbol: magnetron
 - Fixed name conflict of diamond shape with tikz.shapes package
 - Fixed varcap symbol at small scalings
 - New packet-option “straightvoltages, to draw straight(no curved) voltage arrows
 - New option “invert” to revert the node direction at paths
 - Fixed american voltage label at special sources and battery
 - Fixed/rotated battery symbol(longer lines by default positive voltage)
 - New symbol Schmitttrigger
- Version 0.6 (2016-06-06)
 - Added Mechanical Symbols (damper,mass,spring)
 - Added new connection style diamond, use (d-d)
 - Added new sources voosource and ioosource (double zero-style)
 - All diode can now drawn in a stroked way, just use globel option “strokediode” or stroke instead of full/empty, or D-. Use this option for compliance with DIN standard EN-60617
 - Improved Shape of Diodes:tunnel diode, Zener diode, schottky diode (bit longer lines at cathode)
 - Reworked igbt: New anchors G,gate and new L-shaped form Lnight, Lpigbt

- Improved shape of all fet-transistors and mirrored p-chan fets as default, as pnp, pmos, pfet are already. This means a backward-incompatibility, but smaller code, because p-channels mosfet are by default in the correct direction(source at top). Just remove the ‘yscale=-1’ from your p-chan fets at old pictures.
- Version 0.5 (2016-04-24)
 - new option boxed and dashed for hf-symbols
 - new option solderdot to enable/disable solderdot at source port of some fets
 - new parts: photovoltaic source, piezo crystal, electrolytic capacitor, electromechanical device(motor, generator)
 - corrected voltage and current direction(option to use old behaviour)
 - option to show body diode at fet transistors
- Version 0.4
 - minor improvements to documentation
 - comply with TDS
 - merge high frequency symbols by Stefan Erhardt
 - added switch (not opening nor closing)
 - added solder dot in some transistors
 - improved ConTeXt compatibility
- Version 0.3.1
 - different management of color...
 - fixed typo in documentation
 - fixed an error in the angle computation in voltage and current routines
 - fixed problem with label size when scaling a tikz picture
 - added gas filled surge arrester
 - added compatibility option to work with Tikz’s own circuit library
 - fixed infinite in arctan computation
- Version 0.3.0
 - fixed gate node for a few transistors
 - added mixer
 - added fully differential op amp (by Kristofer M. Monisit)
 - now general settings for the drawing of voltage can be overridden for specific components
 - made arrows more homogeneous (either the current one, or latex’ bt pgf)
 - added the single battery cell
 - added fuse and asymmetric fuse
 - added toggle switch
 - added varistor, photoresistor, thermocouple, push button
 - added thermistor, thermistor ptc, thermistor ptc
 - fixed misalignment of voltage label in vertical bipoles with names
 - added isfet
 - added noiseless, protective, chassis, signal and reference grounds (Luigi «Liverpool»)
- Version 0.2.4
 - added square voltage source (contributed by Alistair Kwan)

- added buffer and plain amplifier (contributed by Danilo Piazzalunga)
- added squid and barrier (contributed by Cor Molenaar)
- added antenna and transmission line symbols contributed by Leonardo Azzinnari
- added the changeover switch spdt (suggestion of Fabio Maria Antoniali)
- rename of context.tex and context.pdf (thanks to Karl Berry)
- updated the email address
- in documentation, fixed wrong (non-standard) labelling of the axis in an example (thanks to prof. Claudio Beccaria)
- fixed scaling inconsistencies in quadrupoles
- fixed division by zero error on certain vertical paths
- introduced options straighlabels, rotatelabels, smartlabels
- Version 0.2.3
 - fixed compatibility problem with label option from tikz
 - Fixed resizing problem for shape ground
 - Variable capacitor
 - polarized capacitor
 - ConTeXt support (read the manual!)
 - nfet, nifete, nigfetd, pfet, pigfete, pigfetd (contribution of Clemens Helfmeier and Theodor Borsche)
 - njfet, pjfet (contribution of Danilo Piazzalunga)
 - pigbt, nigbt
 - *backward incompatibility* potentiometer is now the standard resistor-with-arrow-in-the-middle; the old potentiometer is now known as variable resistor (or vR), similarly to variable inductor and variable capacitor
 - triac, thyristor, memristor
 - new property “name” for bipoles
 - fixed voltage problem for batteries in american voltage mode
 - european logic gates
 - *backward incompatibility* new american standard inductor. Old american inductor now called “cute inductor”
 - *backward incompatibility* transformer now linked with the chosen type of inductor, and version with core, too. Similarly for variable inductor
 - *backward incompatibility* styles for selecting shape variants now end are in the plural to avoid conflict with paths
 - new placing option for some tripoles (mostly transistors)
 - mirror path style
- Version 0.2.2 - 20090520
 - Added the shape for lamps.
 - Added options `europeanresistor`, `europeaninductor`, `americanresistor` and `americaninductor`, with corresponding styles.
 - FIXED: error in transistor arrow positioning and direction under negative `xscale` and `yscale`.
- Version 0.2.1 - 20090503
 - Op-amps added
 - added options `arrowmos` and `noarrowmos`, to add arrows to pmos and nmos

- Version 0.2 - 20090417 First public release on CTAN
 - *Backward incompatibility*: labels ending with *:angle* are not parsed for positioning anymore.
 - Full use of TikZ keyval features.
 - White background is not filled anymore: now the network can be drawn on a background picture as well.
 - Several new components added (logical ports, transistors, double bipoles, ...).
 - Color support.
 - Integration with `{siunitx}`.
 - `Voltage, american style`.
 - `Better code, perhaps. General cleanup at the very least.`
- Version 0.1 - 2007-10-29 First public release

Index

Here you will find, in alphabetical order, all the components, keys, and styles defined by the package. Components have the page number in straight text, while keys and styles have *italic* page numbers.

- *-o, [262](#)
- o, [262](#)
- 7-segment display, [219](#)
- a (annotation, [221](#))
- a2 halign, [224](#)
- a2 valign, [224](#)
- a2=..., [224](#)
- ac symbol, [100](#)
- ac symbol/height, [101](#)
- ac symbol/width, [101](#)
- adc, [112](#)
- add async SR, [203](#)
- adder, [110](#)
- afuse, [97](#)
- ageneric, [58](#)
- aGTOb, [73](#), [74](#), *see also* agtobar
- aGTOb*, [73](#), *see also* full agtobar
- aGTOb-, [73](#), *see also* stroke agtobar
- agtobar, [73](#), [74](#)
- aGTObo, [73](#), *see also* empty agtobar
- all leads, [193](#)
- allornothing, [114](#)
- allpass, [111](#)
- ALU, [207](#)
- american, [13](#), [19](#)
- american and port, [187](#)
- american buffer port, [188](#)
- american controlled current source, [80](#)
- american controlled voltage source, [79](#)
- american current source, [78](#)
- american currents, [80](#), [233](#)
- american gas filled surge arrester, [97](#), [97](#)
- american inductive sensor, [66](#), *see also* sL
- american inductor, [65](#), *see also* L
- american inductors, [65](#)
- american light dependent resistor, [58](#), *see also* ldR
- american nand port, [187](#)
- american nor port, [188](#)
- american not port, [188](#)
- american or port, [187](#)
- american or shape, [192](#)
- american ports, [190](#)
- american potentiometer, [40](#), *see also* pR, [58](#), *see also* pR
- american resistive sensor, [58](#), *see also* sR
- american resistor, [40](#), *see also* resistor, [58](#), *see also* R
- american resistors, [58](#)
- american voltage source, [78](#)
- american voltages, [78](#), [80](#), [235](#), [236](#)
- american xnor port, [188](#)
- american xor port, [188](#)
- americancurrent, [233](#)
- americancurrents, [18](#), [80](#)
- americangfsurgearrester, [18](#), [97](#)
- americaninductors, [18](#), [65](#)
- americanports, [18](#), [190](#)
- americanresistors, [18](#), [58](#)
- americanvoltage, [235](#), [236](#)
- americanvoltages, [18](#), [78](#), [80](#)
- ammeter, [40](#), [89](#)
- amp, [112](#)
- amp minus, [173](#)
- amp plus, [173](#)
- amp symbol font, [173](#)
- amp, boxed, [112](#)
- amplifier border anchors, [172](#)
- amplifier pin length, [174](#)
- amplifier, generic shapes, [176](#)
- amplifiers defined with muxdemux, [176](#)
- amplifiers/minus, [173](#)
- amplifiers/plus, [173](#)
- amplifiers/scale, [172](#)
- annotation distance, [222](#)
- antenna, [159](#)
- arrowmos, [19](#), [134](#)
- asymmetric fuse, [97](#), *see also* afuse
- baertty, [78](#)
- bandpass, [111](#)
- bandpassp, [115](#)
- bandpassp, ideal filter, [115](#)
- bandstop, [111](#)
- bandstopp, [115](#)
- bandstopp, ideal filter, [115](#)
- bare jumper, [185](#)
- bare7seg, [219](#)
- bareantenna, [158](#)
- bareRXantenna, [158](#)
- bareTXantenna, [158](#)
- barrier, [14](#), [97](#)
- batteries, [77](#)
- batteries/scale, [84](#)
- battery, [77](#)
- battery1, [77](#)
- battery2, [78](#)
- bgenerator, [112](#)
- biast, [113](#)
- biD*, [70](#), *see also* full bidirectionaldiode
- biDo, [70](#), *see also* empty bidirectionaldiode
- bipole annotation style, [241](#)
- bipole current style, [241](#)

bipole flow style, [241](#)
 bipole label style, [241](#)
 bipole nodes, [253](#)
 bipole voltage style, [241](#)
 bipole/barrier/width, [97](#)
 bipole/is voltage, [230](#)
 bipole/openbarrier/gap, [97](#)
 bipole/openbarrier/width, [97](#)
 bipole/override source vif, [230](#)
 bipoles/border margin, [36](#)
 bipoles/crossing/size, [103](#)
 bipoles/currtap, [90](#)
 bipoles/cutechoke/ctick, [165](#)
 bipoles/cuteswitch/shape, [180](#)
 bipoles/cuteswitch/thickness, [179](#)
 bipoles/dcsource/angle, [84](#)
 bipoles/fix tunable direction, [267](#)
 bipoles/inductors/core distance, [68](#)
 bipoles/inductors/dot x distance, [68](#)
 bipoles/inductors/dot y distance, [68](#)
 bipoles/is current, [230](#)
 bipoles/jumpers/shape, [185](#)
 bipoles/length, [37](#), [38](#)
 bipoles/mic/bar thickness, [99](#)
 bipoles/mstline/height, [160](#)
 bipoles/mstline/width, [160](#)
 bipoles/oosourcetrans/upper circle offset, [88](#)
 bipoles/oosourcetrans/upper circle size default, [87](#)
 bipoles/oscope/height, [93](#)
 bipoles/oscope/waveform, [91](#)
 bipoles/oscope/width, [93](#)
 bipoles/qmeter/index position, [91](#)
 bipoles/smeter/index position, [91](#)
 bipoles/thickness, [39](#)
 bipoles/tline/width, [160](#)
 bipoles/vsourceam/inner plus, [85](#)
 bipoles/vsourceam/margin, [85](#)
 bipoles/vsourcesam/inner minus, [85](#)
 bipoles/wfuse/dots, [99](#)
 bipoles/wfuse/shape, [99](#)
 bjt multi height, [142](#)
 bjt pins width, [142](#)
 bjtnpn, collectors=1, emitters=2, [126](#)
 bjtnpn, collectors=2, emitters=2, bjt pins width=0, bjt multi height=0.8, [143](#)
 bjtnpn, collectors=3, emitters=2, [126](#)
 block lateral anchors pos, [117](#)
 block left anchors pos, [117](#)
 block right anchor pos, [117](#)
 blocks dc in segments, [123](#)
 blocks dc out segments, [123](#)
 blocks dc segments, [123](#)
 blocks/filter grid/, [124](#)
 blocks/filter plot/relative thickness, [124](#)
 blocks/filter plot/rounding, [124](#)
 blocks/gridnode/gridlines, [123](#)
 blocks/inner color, [123](#)
 blocks/inner thickness, [123](#)
 bmultiwire, [101](#), *see also* bmultiwire, [101](#)
 bnc, [107](#)
 bodydiode, [136](#)
 bodydiode anchors, [136](#)
 border text distance, [121](#)
 box, [122](#)
 box only, [122](#)
 boxed, [122](#)
 boxed only, [122](#)
 buffer, [170](#)
 bulb, [98](#)
 bulk, [139](#)
 buzzer, [98](#)
 C, [63](#), *see also* capacitor
 camera, [116](#)
 capacitive sensor, [64](#)
 capacitor, [63](#)
 capacitors/height, [65](#)
 capacitors/scale, [65](#)
 capacitors/width, [65](#)
 cC, [63](#), *see also* curved capacitor
 cceI, [79](#), *see also* cute european controlled current source
 cceV, [79](#), *see also* cute european controlled voltage source
 ccgsw, [178](#), *see also* cute closing switch
 ccsw, [178](#), *see also* cute closed switch
 ceI, [78](#), *see also* cute european current source
 centergap, [149](#)
 ceV, [78](#), *see also* cute european voltage source
 cgenerator, [112](#)
 cground, [55](#)
 chips/scale, [216](#)
 circ, [22](#), [106](#)
 circleinv, scale=2, [206](#)
 circulator, [110](#)
 cisource, [79](#), *see also* european controlled current source
 cisourceAM, [80](#), *see also* american controlled current source
 cisourceC, [79](#), *see also* cute european controlled current source
 cisourceEU, [79](#), *see also* european controlled current source
 cisourcesin, [80](#), *see also* controlled sinusoidal current source
 clockwedge, [202](#)
 closed double solder jumper, [187](#)
 closed jumper, [185](#)
 closed solder jumper, [187](#)
 closing normal closed switch, [177](#)
 closing normal open switch, [177](#)
 closing switch, [176](#)
 cncs, [177](#), *see also* closing normal closed switch
 cnos, [177](#), *see also* closing normal open switch
 cogsw, [178](#), *see also* cute opening switch
 color, [258](#)

compatibility, [19](#)
 component text, [121](#), [174](#)
 component text: up/down, [132](#)
 component text=left, [197](#)
 connectors/scale, [107](#)
 controlled isourcesin, [80](#), *see also* controlled
 sinusoidal current source
 controlled sinusoidal current source, [80](#)
 controlled sinusoidal voltage source, [80](#)
 controlled vsourcesin, [80](#), *see also* controlled
 sinusoidal voltage source
 core east, [66](#), [68](#)
 core west, [66](#), [68](#)
 cosw, [178](#), *see also* cute open switch
 coupler, [116](#)
 coupler2, [116](#)
 cpe, [64](#), *see also* cpe, [64](#)
 crossing, [102](#)
 crossing vertical, [103](#)
 csI, [80](#), *see also* controlled sinusoidal current
 source
 csources, [77](#)
 csources/scale, [84](#)
 cspst, [176](#), *see also* closing switch
 csV, [80](#), *see also* controlled sinusoidal voltage
 source
 ctikzactivatecurrentdirections, [250](#)
 ctikzactivateflowdirections, [250](#)
 ctikzactivatevoltage directions, [250](#)
 ctikzflip, [41](#)
 ctikzgetanchor, [246](#)
 ctikzgetdirection, [242](#), [246](#)
 ctikzloadstyle, [46](#)
 ctikzprocessvif, [249](#)
 ctikztextnot, [202](#)
 currarrow, [104](#), [133](#)
 current arrow scale, [104](#)
 current point is local, [37](#)
 current/distance, [229](#), [244](#)
 currtap, [90](#)
 curved capacitor, [63](#)
 cute choke, [66](#), [68](#)
 cute closed switch, [178](#)
 cute closing switch, [178](#)
 cute european controlled current source, [79](#)
 cute european controlled voltage source, [79](#)
 cute european current source, [78](#)
 cute european voltage source, [78](#)
 cute inductive sensor, [65](#), *see also* sL
 cute inductor, [65](#), *see also* L
 cute inductors, [65](#)
 cute open switch, [178](#)
 cute opening switch, [178](#)
 cute spdt down, [178](#)
 cute spdt down arrow, [42](#), [179](#)
 cute spdt mid, [178](#)
 cute spdt mid arrow, [178](#)
 cute spdt up, [178](#)
 cute spdt up arrow, [178](#)
 cuteinductors, [18](#), [65](#)
 cvsource, [79](#), *see also* european controlled
 voltage source
 cvsourceAM, [79](#), *see also* american controlled
 voltage source
 cvsourceC, [79](#), *see also* cute european controlled
 voltage source
 cvsourceEU, [79](#), *see also* european controlled
 voltage source
 cvsourcesin, [80](#), *see also* controlled sinusoidal
 voltage source
 D*, [70](#), [71](#), *see also* full diode
 D-, [70](#), [71](#), *see also* stroke diode
 dac, [112](#)
 damper, [96](#), [269](#)
 dashed blocks pattern, [122](#)
 dc symbol, [100](#)
 dc symbol segments, [101](#)
 dc symbol/height, [101](#)
 dc symbol/segments, [101](#)
 dc symbol/text offset, [101](#)
 dc symbol/width, [101](#)
 dcisource, [84](#)
 dcvsources, [84](#)
 delta, [82](#)
 demux, [207](#)
 detector, [113](#)
 diamondpole, [106](#)
 dinantenna, [158](#)
 diode sloped whiskers, [77](#)
 diode straight whiskers, [77](#)
 diodes/scale, [44](#), [75](#)
 diodetube, [151](#)
 diodetube,filament, [152](#)
 diodetube,filament,nocathode, [152](#)
 diodetube,fullcathode, [152](#)
 dipchip, [215](#)
 displays/scale, [220](#)
 distance from node, override, [228](#)
 Do, [69](#), *see also* empty diode
 dot on notQ, [202](#)
 double bipole, [163](#)
 double bipole generic, [167](#)
 double bipole L, [168](#)
 double bipole L invert, [168](#)
 double bipole R, [168](#)
 double bipole R invert, [168](#)
 doublegate, [128](#)
 dsp, [112](#)
 dynode, [156](#)
 dynode customization, [157](#)
 eC, [64](#), *see also* ecapacitor
 ecapacitor, [64](#)
 ecsources, [80](#), *see also* empty controlled source
 EFvoltages, [20](#), [225](#)
 eground, [55](#)

eground2, 55
 elko, 64, *see also* ecapacitor
 elmech, 161
 empty agtobar, 73
 empty bidirectionaldiode, 70
 empty controlled source, 80
 empty diode, 69
 empty diodes, 69
 empty gto, 72
 empty gtobar, 73
 empty igct, 73
 empty laser diode, 69
 empty led, 69
 empty photodiode, 69
 empty put, 72
 empty Schottky diode, 69
 empty Shockley diode, 69
 empty thyristor, 72
 empty triac, 71
 empty tunnel diode, 69
 empty TVS diode, 69
 empty varcap, 69
 empty Zener diode, 69
 empty ZZener diode, 69
 emptycircle, 134
 emptydiode, 19, 69
 emptymoscircle, 19
 en amp, 169
 en amp text A, 175
 en amp text=, 175
 ource, 81
 european, 13, 19
 european and port, 190
 european blank not port, 190
 european blank port, 190
 european buffer port, 190
 european controlled current source, 79
 european controlled voltage source, 79
 european current source, 78
 european currents, 78, 80
 european gas filled surge arrester, 97, 97
 european inductive sensor, 66, *see also* sL
 european inductor, 66, *see also* L
 european inductors, 66
 european light dependent resistor, 59, *see also* ldR
 european nand port, 190
 european nor port, 190
 european not port, 190
 european or port, 190
 european ports, 190
 european ports font, 200
 european potentiometer, 58, *see also* pR
 european resistive sensor, 58, *see also* sR
 european resistor, 58, *see also* R
 european resistors, 58
 european voltage source, 78
 european voltages, 80, 234
 european xnor port, 190
 european xnor style, 200
 european xor port, 190
 europeancurrents, 18, 78, 80
 europeanngfsurgearrester, 18, 97
 europeaninductors, 18, 66
 europeanports, 18, 190
 europeanresistors, 18, 58
 europeanvoltage, 234
 europeanvoltages, 18, 78, 80
 every double bipole L, 168
 every double bipole R, 168
 external pins width, 204, 208
 eyw, 82
 f (flow, current), 233
 f label, 248
 f symbols, 243
 fd inst amp, 169
 fd op amp, 169
 fdsoidot, 140
 feC, 64
 ferrocap, 64, *see also* feC
 ferroel, 138
 ferroel gate, 138
 fetbodydiode, 19
 fetsolderdot, 19, 127
 Ffrom, 244
 fft, 112
 fiber, 114
 fill (color), 259
 fill opacity, 261
 fix tunable direction, 15
 Flab, 245, 246
 flipflop, 201
 flipflop AB, 201
 flipflop D, 202
 flipflop def, 201
 flipflop JK, 202
 flipflop JK, add async SR, 203
 flipflop JK, add async SR, external pins width=0, 204
 flipflop JK, dot on notQ, 202
 flipflop myJK, 203
 flipflop SR, 202
 flipflop T, 202
 flipflops/scale, 204
 flow/distance, 229
 flowarrow, 104
 flyback diode, 136
 fourport, 116
 Fpos, 245
 fpu, 13
 Fto, 244
 full agtobar, 73
 full bidirectionaldiode, 70
 full diode, 70, 71
 full diodes, 69
 full gto, 72

full gtobar, [73](#)
 full igct, [73](#)
 full laser diode, [70](#)
 full led, [70](#)
 full photodiode, [70](#)
 full put, [72](#)
 full Schottky diode, [70](#)
 full Shockley diode, [70](#)
 full thyristor, [72](#)
 full triac, [72](#)
 full tunnel diode, [70](#)
 full TVS diode, [70](#)
 full varcap, [70](#)
 full Zener diode, [70](#)
 full ZZener diode, [70](#)
 fulldiode, [19](#), [69](#)
 fuse, [97](#)

GaN hemt, [127](#)
 gate, [75](#)
 gcenter, [148](#)
 generic, [57](#)
 genericsplitter, [111](#)
 gm amp, [169](#)
 gridnode, [114](#)
 ground, [55](#)
 grounds/scale, [56](#)
 GTO, [72](#), *see also* gto, [74](#), *see also* gto
 gto, [72](#), [74](#)
 gto gate arrows, [75](#)
 GTO*, [72](#), *see also* full gto
 GTO-, [73](#), *see also* stroke gto
 GTOb, [73](#), [74](#), *see also* gtobar
 GTOb*, [73](#), *see also* full gtobar
 GTOb-, [73](#), *see also* stroke gtobar
 gtobar, [73](#), [74](#)
 GTObo, [73](#), *see also* empty gtobar
 GTOo, [72](#), *see also* empty gto
 gyrator, [162](#)

harmonics, [158](#)
 hemt, [127](#)
 hemt, nobase, [127](#)
 hide numbers, [216](#)
 highpass, [111](#)
 highpass2, [111](#)
 highpassp, [115](#)
 highpassp, ideal filter, [115](#)

i, [22](#)
 i (current), [222](#), [231](#)
 i label, [248](#)
 i symbols, [243](#)
 iamp, [112](#)
 iamp, boxed, [112](#)
 ideal filter, [115](#)
 ideal filter number low, [124](#)
 ideal filter numbers, [124](#)
 iec connector, [107](#)
 iecconn, [107](#), *see also* iec connector
 iecconnshape, [108](#)
 iecplugL, [108](#)
 iecplugR, [108](#)
 iecsocketL, [108](#)
 iecsocketR, [108](#)
 ieee double tgate, [189](#)
 ieee ports, [190](#)
 ieee tgate, [189](#)
 ieeestd and port, [188](#)
 ieeestd buffer port, [189](#)
 ieeestd invschmitt port, [189](#)
 ieeestd nand port, [189](#)
 ieeestd nor port, [189](#)
 ieeestd not port, [189](#)
 ieeestd or port, [189](#)
 ieeestd ports customizazion, [198](#)
 ieeestd ports/height, [196](#)
 ieeestd schmitt port, [189](#)
 ieeestd xnor port, [189](#)
 ieeestd xor port, [189](#)
 Ifrom, [244](#)
 IGCT, [73](#), [74](#), *see also* igct
 igct, [73](#), [74](#)
 IGCT*, [73](#), *see also* full igct
 IGCT-, [73](#), *see also* stroke igct
 IGCTo, [73](#), *see also* empty igct
 Ilab, [245](#), [246](#)
 iloop, [90](#)
 iloop2, [90](#)
 in down, [172](#)
 in up, [172](#)
 inductors/coils, [66](#)
 inductors/scale, [66](#), [165](#)
 inductors/width, [66](#)
 inerter, [96](#)
 inline buffer, [191](#)
 inline double tgate, [191](#)
 inline invschmitt, [191](#)
 inline not, [191](#)
 inline proximeter, [180](#)
 inline schmitt, [191](#)
 inline tgate, [191](#)
 inner blocks dashed, [122](#)
 inner inputs, [196](#)
 input leads, [193](#), [208](#), [216](#)
 inputarrow, [104](#)
 inst amp, [169](#)
 inst amp ra, [170](#)
 instruments/scale, [90](#)
 invert, [39](#)
 invschmitt, [188](#)
 ioosource, [14](#), [82](#)
 Ipos, [245](#)
 isfet, [131](#)
 isource, [78](#), *see also* european current source
 isourceAM, [78](#), *see also* american current source

isourceC, [78](#), *see also* cute european current source
 isourceEU, [78](#), *see also* european current source
 isourceN, [80](#), *see also* noise current source
 isourcesin, [79](#), *see also* sinusoidal current source
 Ito, [244](#)

Jack Tap, [105](#)
 jump crossing, [102](#)

L, [65](#), [66](#)
 l (label), [221](#)
 l2 halign, [224](#)
 l2 valign, [224](#)
 l2=, [224](#)
 label, [60](#)
 label distance, [222](#)
 label/align, [223](#)
 lamp, [98](#)
 lasD, [69](#), *see also* empty laser diode
 lasD*, [70](#), *see also* full laser diode
 lasD-, [71](#), *see also* stroke laser diode
 latch, [202](#)
 latexslim, [104](#)
 lazymos, [19](#)
 ldR, [58](#), [59](#)
 led arrows from anode, [75](#)
 led arrows from cathode, [75](#)
 leD*, [70](#), *see also* full led
 leD-, [71](#), *see also* stroke led
 leDo, [69](#), *see also* empty led
 left double solder jumper, [187](#)
 left text distance, [174](#)
 leftedge, [172](#)
 legacy transistors text, [131](#)
 legacytransistorstext, [19](#), [131](#)
 Lnigbt, [125](#)
 logic gates, ieee, multiple inputs, [195](#)
 logic ports draw input leads, [193](#), [208](#)
 logic ports draw leads, [193](#)
 logic ports draw output leads, [193](#)
 logic ports origin, [194](#)
 logic ports/scale, [192](#)
 logic ports=ieee, [188](#)
 loudspeaker, [98](#)
 lowpass, [111](#)
 lowpass2, [111](#)
 lowpassp, [115](#)
 lowpassp, ideal filter, [115](#)
 Lpigbt, [126](#)
 Lpigbt, bodydiode, [126](#)
 lr dot, [68](#)

magnetron, [156](#)
 mass, [96](#)
 match, [159](#)
 matube, [155](#)
 matube (multi-anode tube) customizations, [156](#)
 matube, nixieanode, anodedot, nogrid, [155](#)
 mechanicals/scale, [96](#)
 memristor, [58](#)
 mic, [98](#)
 midtap, [67](#)
 mirror, [39](#)
 misc/scale, [99](#)
 misc/thickness, [99](#)
 mixer, [110](#)
 mixer, boxed, [110](#)
 modifier thickness, [61](#), [138](#)
 monopoles/msport/width, [160](#)
 monopoles/msrstub/radius, [160](#)
 monopoles/vcc/arrow, [56](#)
 monopoles/vee/arrow, [56](#)
 mov, [59](#)
 Mr, [58](#), *see also* memristor
 mslstub, [158](#)
 msport, [158](#)
 msrstub, [158](#)
 mstline, [158](#)
 mstlinelen, [160](#)
 multipoles/dipchip/width, [216](#)
 multipoles/draw only * pins, [208](#)
 multipoles/draw only pins, [217](#)
 multipoles/external pad fraction, [216](#)
 multipoles/external pins thickness, [208](#), [216](#)
 multipoles/external pins width, [208](#), [216](#)
 multipoles/flipflop/, [201](#)
 multipoles/flipflop/clock wedge size, [205](#)
 multipoles/flipflop/font, [204](#)
 multipoles/flipflop/fontud, [204](#)
 multipoles/flipflop/pin spacing, [204](#)
 multipoles/flipflop/width, [204](#)
 multipoles/font, [216](#)
 multipoles/muxdemux/..., [207](#)
 multipoles/muxdemux/base len, [208](#)
 multipoles/qfpchip/pin spacing, [216](#)
 multipoles/rotary/angle, [181](#)
 multipoles/rotary/arrow, [181](#)
 multipoles/rotary/channels, [181](#)
 multipoles/rotary/shape, [183](#)
 multipoles/rotary/thickness, [183](#)
 multipoles/rotary/wiper, [181](#)
 multipoles/thickness, [216](#)
 multiwire, [101](#), *see also* multiwire, [101](#)
 muxdemux, [206](#)
 muxdemux bgpicture, [213](#)
 muxdemux label, [210](#)
 muxdemux/border label font, [211](#)
 muxdemux/border label sep, [211](#)
 muxdemux/border label xsep, [211](#)
 muxdemux/border label ysep, [211](#)
 muxdemux/clock wedge size, [212](#)
 muxdemux/inner label font, [210](#)
 muxdemux/inner label sep, [210](#)
 muxdemux/inner label xsep, [210](#)
 muxdemux/inner label ysep, [210](#)
 muxdemux/outer label font, [211](#)

- muxdemux/outer label sep, [211](#)
- muxdemux/outer label xsep, [211](#)
- muxdemux/outer label ysep, [211](#)
- muxdemuxes/fill, [208](#)
- muxdemuxes/scale, [208](#)
- muxdemuxes/thickness, [208](#)
- mzm, [111](#)
- name, [150](#), [151](#)
- ncpb, [177](#), *see also* normally closed push button
- ncpbo, [177](#), *see also* normally closed push button open
- ncs, [177](#), *see also* normal closed switch
- neonlampac, [98](#)
- neonlampcc, [98](#)
- nfet, [127](#)
- nfet, fdsoi, [129](#)
- nfetd, [127](#)
- ngenerator, [112](#)
- ngfet, [131](#)
- nground, [55](#)
- nI, [80](#), *see also* noise current source
- nigbt, [125](#), [140](#)
- nigfetd, [128](#)
- nigfetd, doublegate, [129](#)
- nigfete, [127](#)
- nigfete, doublegate, [128](#)
- nigfete, fdsoi, [130](#)
- nigfete,solderdot, [128](#)
- nigfete,solderdot, doublegate, [128](#)
- nigfetebulk, [128](#)
- nigfetebulk, doublegate, [129](#)
- nigfetebulk, fdsoi, [129](#)
- njfet, [130](#)
- nmos, [126](#), [134](#)
- nmos, bulk, [139](#)
- nmos, bulk, fdsoi, [140](#)
- nmosd, [127](#), [134](#)
- nmosd, bulk, [139](#)
- nmosd, bulk, fdsoidot, [140](#)
- no f symbols, [243](#)
- no ferroel base, [138](#)
- no i symbols, [243](#)
- no input leads, [193](#), [198](#), [208](#), [216](#)
- no inputs pin, [200](#)
- no leads, [193](#)
- no output leads, [193](#)
- no schottky base, [137](#)
- no topmark, [216](#)
- no v symbols, [243](#)
- noarrowmos, [19](#), [134](#)
- nobase, [130](#), [139](#)
- nobulk, [139](#)
- nocircle, [134](#)
- nodes width, [107](#)
- nofetbodydiode, [19](#)
- nofetsolderdot, [19](#), [127](#)
- nogate, [139](#)
- noinv input down, [172](#)
- noinv input up, [172](#), [175](#)
- noinv output down, [172](#)
- noinv output up, [172](#)
- noise current source, [80](#)
- noise sources/fillcolor, [81](#)
- noise voltage source, [80](#)
- nolegacytransistorstext, [19](#)
- nooldvoltage, [20](#), [225](#)
- no pb, [177](#), *see also* push button
- no pb, [177](#), *see also* normally open push button closed
- norator, [83](#)
- normal closed switch, [177](#)
- normal open switch, [176](#)
- normally closed push button, [177](#)
- normally closed push button open, [177](#)
- normally open push button, [177](#), *see also* push button
- normally open push button closed, [177](#)
- nos, [176](#), *see also* normal open switch
- nosunitx, [19](#)
- not radius, [198](#)
- notchp, [115](#)
- notchp, ideal filter, [115](#)
- notcirc, [189](#)
- n pn, [42](#), [125](#)
- n pn, bodydiode, [125](#)
- n pn, schottky base, [125](#)
- n pn, tr circle, [143](#)
- n pn, tr circle, bodydiode, [143](#)
- n pn,photo, [125](#)
- nujt, [130](#)
- nujt, nobase, [131](#)
- nullator, [83](#)
- nullor, [168](#)
- num pins, [216](#)
- number inputs, [192](#)
- nV, [80](#), *see also* noise voltage source
- o-, [262](#)
- o-o, [262](#)
- ocirc, [106](#)
- odiamondpole, [106](#)
- ohmmeter, [89](#)
- oldvoltage, [19](#), [225](#)
- oncs, [177](#), *see also* opening normal closed switch
- one bit adder, [207](#)
- onos, [177](#), *see also* opening normal open switch
- oo upper winding, [87](#)
- oosource, [82](#)
- oosourceatrans, [82](#)
- oosourceatran, [14](#)
- oosourceatrans, [82](#)
- ootrans size, [84](#)
- op amp, [24](#), [169](#)
- open, [16](#), [57](#), [239](#)
- open double solder jumper, [187](#)
- open jumper, [185](#)
- open nodes fill, [106](#)

open poles fill, [263](#)
 open poles opacity, [255](#)
 open solder jumper, [186](#)
 open voltage position, [239](#)
 openbarrier, [14](#), [97](#)
 opening normal closed switch, [177](#)
 opening normal open switch, [177](#)
 opening switch, [176](#)
 openshape, [58](#)
 opto arrows, [61](#), [76](#)
 oscillator, [110](#)
 oscscope, [90](#)
 ospst, [176](#), *see also* opening switch
 osquarepole, [106](#)
 out down, [172](#)
 out up, [172](#)

 pd arrows to anode, [75](#)
 pd arrows to cathode, [75](#)
 pD*, [70](#), *see also* full photodiode
 pD-, [71](#), *see also* stroke photodiode
 pDo, [69](#), *see also* empty photodiode
 pentode, [151](#)
 pentode suppressor to cathode, [152](#)
 pfet, [16](#), [128](#)
 pfet, fdsoi, [130](#)
 pfetd, [128](#)
 pgfcirctriples.tex, [174](#)
 pgfet, [131](#)
 pground, [55](#)
 phaseshifter, [113](#)
 photoresistor, [59](#), *see also* phR
 phR, [59](#)
 piattenuator, [113](#)
 pic, [51](#), [52](#), [267](#)
 pic anchor, [52](#)
 piezoelectric, [64](#)
 pigbt, [125](#)
 pigbt, nobase, [140](#)
 pigfetd, [16](#), [128](#)
 pigfetd, doublegate, [129](#)
 pigfete, [16](#), [128](#)
 pigfete, doublegate, [129](#)
 pigfete, fdsoi, doublegate, solderdot, [130](#)
 pigfetebulk, [16](#), [128](#)
 pigfetebulk, doublegate, [129](#)
 pigfetebulk, fdsoi, [130](#)
 pjfet, [130](#)
 plain amp, [42](#), [170](#)
 plain crossing, [102](#)
 plain gm mono amp, [170](#)
 plain mono amp, [170](#)
 pmos, [127](#), [134](#)
 pmos, bulk, [139](#)
 pmos, bulk, fdsoidot, [140](#)
 pmos,emptycircle, [134](#)
 pmos,nocircle,arrowmos, [134](#)
 pmosd, [127](#), [134](#)
 pmosd, bulk, [140](#)

 pmosd, bulk, fdsoi, [140](#)
 pnp, [125](#)
 pnp, schottky base, [125](#)
 pnp,photo, [125](#)
 polar capacitor, [64](#)
 poles/open fill opacity, [255](#)
 power, [116](#)
 power supplies/scale, [56](#)
 pR, [40](#), *see also* pR, [40](#), [58](#)
 prim, [82](#)
 proximeter, [180](#)
 proximeter/hlines position, [180](#)
 proximeter/hlines thickness, [180](#)
 proximeter/width, [180](#)
 pujt, [130](#)
 push button, [177](#)
 PUT, [72](#), *see also* put, [74](#), *see also* put
 put, [72](#), [74](#)
 PUT*, [72](#), *see also* full put
 PUT-, [72](#), *see also* stroke put
 PUTo, [72](#), *see also* empty put
 pvmodule, [82](#)
 pvsource, [81](#)
 PZ, [64](#), *see also* piezoelectric

 qfpchip, [215](#)
 qgenerator, [112](#)
 qiprobe, [89](#)
 qpprobe, [89](#)
 quadpoles style, [166](#)
 quadpoles style inline, [166](#)
 quadpoles style inward, [166](#)
 quadpoles/*/heighth, [165](#)
 quadpoles/*/inner, [165](#)
 quadpoles/*/width, [165](#)
 quadpoles/transformer core/core width, [165](#)
 qvprobe, [89](#)

 R, [40](#), *see also* resistor, [58](#)
 raised voltages, [237](#)
 rbuzzer, [99](#)
 rectjoinfill, [106](#)
 reed, [177](#)
 register, [113](#)
 relais, [97](#)
 resistor, [40](#)
 resistors/scale, [38](#), [60](#)
 resistors/thickness, [61](#)
 resistors/width, [60](#)
 resistors/zigs, [60](#)
 resistors/zigzag join, [62](#)
 resistors/zigzag stub, [62](#)
 resonantp, [115](#)
 resonantp, ideal filter, [115](#)
 RF/scale, [160](#)
 rground, [55](#)
 right double solder jumper, [187](#)
 rmeter, [88](#)
 rmeterwa, [88](#)

rotary switch, [181](#)
 rotary switch -, [181](#)
 rotary switch ->, [181](#)
 rotary switch <-, [181](#)
 rotary switch <->, [181](#)
 rotary switches, [181](#)
 rotaryswitch, [43](#), [181](#)
 rotated instruments, [92](#)
 rotated numbers, [210](#), [218](#)
 rotatelabels, [19](#), [223](#)
 RPlvoltage, [20](#), [225](#)
 rxantenna, [159](#)

sacac, [114](#)
 sacdc, [114](#)
 saturation, [113](#)
 sC, [64](#), *see also* capacitive sensor
 scale, [38](#)
 schmitt, [188](#)
 schmitt symbol, [190](#)
 schottky base, [137](#)
 sD*, [70](#), *see also* full Schottky diode
 sD-, [71](#), *see also* stroke Schottky diode
 sdcac, [114](#)
 sdcdc, [114](#)
 sDo, [69](#), *see also* empty Schottky diode
 sec, [82](#)
 seven seg/bits, [219](#)
 seven seg/box, [219](#)
 seven seg/box sep, [219](#)
 seven seg/dot, [219](#)
 seven segment bits, [219](#)
 seven segment val, [219](#)
 seven-segment display, [219](#)
 sgeneric, [57](#)
 sground, [55](#)
 shD*, [70](#), *see also* full Shockley diode
 shDo, [69](#), *see also* empty Shockley diode
 short, [22](#), [57](#)
 shortshape, [58](#)
 show numbers, [216](#)
 sI, [79](#), *see also* sinusoidal current source
 sigmoid, [113](#)
 sinetable, [113](#)
 singeneric, [57](#)
 sinusoidal current source, [79](#)
 sinusoidal voltage source, [79](#)
 siunitx, [19](#), [221](#), [251](#)
 sL, [65](#), [66](#)
 smartlabels, [19](#), [223](#)
 smeter, [89](#)
 snubbers, [148](#)
 solar, [78](#)
 sources, [77](#)
 sources/scale, [84](#)
 sources/symbol/delta rot, [87](#)
 sources/symbol/delta scale, [86](#)
 sources/symbol/eyw scale, [86](#)
 sources/symbol/prim rot, [87](#)
 sources/symbol/rotate, [85](#)
 sources/symbol/sign rotation, [86](#)
 sources/symbol/thickness, [85](#)
 sources/symbol/wye scale, [86](#)
 sources/symbol/xdelta split/, [82](#)
 sources/symbol/zig scale, [86](#)
 sparkgap, [98](#)
 sparkgap end arrow, [100](#)
 sparkgap, sparkgap/circle, [98](#)
 sparkgap, sparkgap/dot, sparkgap/circle, [98](#)
 sparkgap/circle, [100](#)
 sparkgap/dot, [100](#)
 sparkgap/gap, [100](#)
 spdt, [177](#)
 splitter, [110](#)
 spring, [96](#)
 spst, [176](#), *see also* switch
 square voltage source, [81](#)
 squarepole, [106](#)
 squid, [97](#)
 sqV, [81](#), *see also* square voltage source
 sR, [58](#)
 straight instruments, [92](#)
 straight numbers, [210](#), [218](#)
 straight voltages, [236](#)
 straightlabels, [19](#), [223](#)
 straightvoltages, [18](#), [236](#)
 stroke agtobar, [73](#)
 stroke diode, [70](#), [71](#)
 stroke diodes, [69](#)
 stroke gto, [73](#)
 stroke gtobar, [73](#)
 stroke igct, [73](#)
 stroke laser diode, [71](#)
 stroke led, [71](#)
 stroke photodiode, [71](#)
 stroke put, [72](#)
 stroke Schottky diode, [71](#)
 stroke thyristor, [72](#)
 stroke tunnel diode, [71](#)
 stroke TVS diode, [71](#)
 stroke varcap, [71](#)
 stroke Zener diode, [71](#)
 stroke ZZener diode, [71](#)
 strokediode, [19](#), [69](#)
 sV, [79](#), *see also* sinusoidal voltage source
 switch, [176](#)
 switch arrows, [183](#)
 switch end arrow, [183](#)
 switchable start arrow, [183](#)
 switches/scale, [176](#)
 swr, [116](#)

tacac, [114](#)
 tacdc, [114](#)
 tattenuator, [113](#)
 tD*, [70](#), *see also* full tunnel diode
 tD-, [71](#), *see also* stroke tunnel diode
 tdcac, [114](#)

tDo, 69, *see also* empty tunnel diode
tert, 82
tetrode, 151
text anchors, 14
tgate scale, 199
tgeneric, 58
tground, 55
thermistor, 59, *see also* thR
thermistor ntc, 59, *see also* thRn
thermistor ptc, 59, *see also* thRp
thermocouple, 97
thR, 59
three-pins jumper, 186
thRn, 59
thRp, 59
thyristor, 72, 74
thyristor style, 74
tikzmark, 52
tjumper connections, 186
TL, 159
TL, bipoles/tline/bare=true, 159
tlground, 55
tline, 159, *see also* TL, bipoles/tline/bare=true, 159, *see also* TL
tlinestub, 159
tlmic, 98
tmultiwire, 101, *see also* tmultiwire, 101
toggle switch, 177
topmark, 216
Tr, 71, *see also* triac, 74, *see also* triac
tr circle, 135
tr gap fill, 142
Tr*, 72, *see also* full triac
transform shape, 44, 50
transformer, 162
transformer core, 163
transformer core customizations, 165
transformer L1, 166
transformer L2, 166
transistor bodydiode/, 145
transistor circle/, 143
transistor circle/default base in, 143
transistor circle/partial border, 144
transistor circle/partial border dash, 144
transistor circle/scale circle radius, 143
transistor solderdot scale, 141
transistor/arrow pos, 133
transistors/scale, 133
transistors/gate thickness, 133
transistors/jfet gate height, 142
transistors/rounded caps, 133
transistors/thickness, 133
transmission line, 159, *see also* TL,
 bipoles/tline/bare=true, 159, *see also*
 TL
trarrow, 104
triac, 71, 74
triode, 151
tripoles/bjt/multi height, 142
tripoles/bjt/pins width, 142
tripoles/en amp/font, 174
tripoles/en amp/font2, 174
tripoles/en amp/input height, 174
tripoles/european not symbol, 200
tripoles/hemt, 137
tripoles/igbt/outer base height, 138
tripoles/igbt/outer base thickness, 138
tripoles/mos style/arrows, 134
tripoles/nmosd/depletion color, 141
tripoles/op amp/port width, 174
tripoles/op amp/width, 174
tripoles/pmosd/depletion color, 142
tripoles/schottky base size, 137
tripoles/spdt/width, 183
Tro, 71, *see also* empty triac
trx, 116
tubes customization, 153
tubes/partial border, 154
tubes/scale, 153
tunable end arrow, 61, 105
tunable start arrow, 61, 105
tV, 81, *see also* vsourcetri
tvsD*, 70, *see also* full TVS diode
tvsD-, 71, *see also* stroke TVS diode
tvsDo, 69, *see also* empty TVS diode
tvset, 116
twoport, 111
twoport/width, 117
twoportsplit, 111, 121
twoportsplit/width, 117
txantenna, 159
Ty, 72, *see also* thyristor, 74, *see also* thyristor
Ty*, 72, *see also* full thyristor
Ty-, 72, *see also* stroke thyristor
Tyo, 72, *see also* empty thyristor

ur dot, 68
use auto vif, 249

v (voltage), 222
v label, 248
v symbols, 243
vallpass, 112
vamp, 113
vamp, boxed, 113
variable american inductor, 66, *see also* vL
variable american resistor, 58, *see also* vR
variable capacitor, 64
variable cute inductor, 65, *see also* vL
variable european inductor, 66, *see also* vL
variable european resistor, 58, *see also* vR
varistor, 59
vC, 64, *see also* variable capacitor
VC*, 70, *see also* full varcap
VC-, 71, *see also* stroke varcap
vcc, 56
vcenter, 148

VCo, 69, *see also* empty varcap
vco, 111
vco,box, 111
Vcont1, 244
Vcont2, 244
vee, 56
Vfrom, 244
vif queue add, 248
viscoe, 96, 272
vL, 65, 66
Vlab, 245, 246
voltage dir, 225
voltage shift, 22, 237
voltage shift sources adjust, 238
voltage, american, 236
voltage, european, 234
voltage, raised, 237
voltage, straight, 236
voltage/american font, 240
voltage/american label distance, 239
voltage/american minus, 240
voltage/american plus, 240
voltage/bump b, 228
voltage/distance from node, 228, 237
voltage/european label distance, 228
voltage/shift, 239
voltages shift, 238
voltmeter, 89
voosource, 14, 82
vphaseshifter, 113
vpiattenuator, 113
vR, 58
vsource, 78, *see also* european voltage source
vsourceAM, 78, *see also* american voltage source
vsourceC, 78, *see also* cute european voltage
source
vsourceEU, 78, *see also* european voltage source
vsourceN, 80, *see also* noise voltage source
vsourcesin, 79, *see also* sinusoidal voltage source
vsourcesquare, 81, *see also* square voltage source
vsourcetri, 81
vtattenuator, 113
Vto, 244

waves, 158
wbulb, 98
wedge inversion mark/width, 213
wedgeinv, scale=2, 206
wfuse, 97
wiggly fuse, 97, *see also* wfuse
wilkinson, 110
wiper, 75
wiper end arrow, 61
wiper pos, 59
wiper start arrow, 61, 183
wye, 82

xdelta, 82
xgeneric, 57
xing, 102, *see also* crossing
xscale, 41

yscale, 41

zD*, 70, *see also* full Zener diode
zD-, 71, *see also* stroke Zener diode
zDo, 69, *see also* empty Zener diode
zig, 82
zzD*, 70, *see also* full ZZener diode
zzD-, 71, *see also* stroke ZZener diode
zzDo, 69, *see also* empty ZZener diode